

Reference Manual

CEDARtools.TraceEncoder (CTTE)

Version 2.3 ~ 2026-08-28

Describes CEDARtools.TraceEncoder release 1.0.0 (2026-08-28)



ACCEMIC
TECHNOLOGIES

Table of Contents

Revision History	2
1. Introduction, Scope and Naming	5
1.1. What this manual describes	5
1.2. How this manual is organised	5
1.3. Relationship to the public documentation	6
1.4. Naming	6
1.5. Conventions	7
2. CTTE in Context	8
2.1. What the encoder is for	8
2.2. Accemic's trace layers	9
2.3. What the integrator must supply	11
2.3.1. Cross-trigger fabric	11
2.3.2. Debug unit	11
3. Standards Baseline	12
3.1. The four documents	12
3.2. How the four documents relate	12
3.3. What CTTE implements from each document	13
3.3.1. N-Trace 1.0 – the primary back end	13
3.3.2. IEEE-ISTO 5001-2012 – messages beyond the N-Trace profile	14
3.3.3. Efficient Trace for RISC-V 2.0 – ingress always, back end optionally	15
3.3.4. TCI 1.0 – the register model and the sinks	16
3.4. Where the documents overlap or contradict	17
3.5. Which document to measure against	18
4. Conformance and Differentiation	19
4.1. The conformance position in one paragraph	19
4.2. Documented deviations	19
4.3. Differentiation – what CTTE adds beyond the four documents	21
4.4. What supports these statements	24
5. Encoder Architecture	25
5.1. Overview	25
5.2. Elements	26
5.3. Two Back Ends, One Front End	26
5.4. Clock Domains	27
5.5. Top-Level Ports	28
5.6. Elaboration Guards	28
5.7. Instance Attributes	29
5.8. Pipeline Stages	29

5.9. Backpressure, Ordering and Flush	30
5.10. Global Determinism Guarantees	31
5.11. What This Chapter Does Not Cover	31
6. Trace Sources and Core Adapters	33
6.1. The Trace Ingress Port as the Contract	33
6.1.1. The <i>itype</i> Vocabulary	33
6.1.2. Four Rules Every Adapter Has to Get Right	34
6.1.3. What Each Core's Ingress Can Carry	34
6.2. MINRES TGC	36
6.3. Fraunhofer IPMS EMSA5	36
6.4. CV32E4	36
6.5. CVA6	36
6.5.1. Maturity	37
6.5.2. Measured ITI Beat Semantics	37
6.5.3. Adapter Behaviour	38
6.5.4. <i>itype</i> Refinement	39
6.5.5. Upstream Defects	40
6.6. MicroBlaze™ V	40
6.6.1. Verified Configuration	41
6.6.2. Confirmed Port List	42
6.6.3. Measured Semantics	42
6.6.4. <i>itype</i> Decoding	43
6.6.5. Sequentially Inferable Jumps	43
6.7. Rocket	44
6.7.1. Measured Semantics	44
6.7.2. Upstream Defects	45
6.7.3. Adapter Behaviour	45
6.8. Multi-Core and Mixed-Protocol Integration	45
6.9. Peripherals	46
6.9.1. Bus Trace	46
6.9.2. DDR4 Trace	46
7. Preprocessor Stage	47
7.1. Overview	47
7.2. Determinism and Path Alignment	47
7.3. Next Program Counter Evaluation	48
7.4. Branch History Accumulation	49
7.5. Synchronization Generator	49
7.5.1. Request Pacing	50
7.6. TIP Delay	50

7.7. Comparator and Filter Engine.....	51
7.8. Control-Flow Qualifier.....	52
7.9. Data-Flow Qualifier and Range Filter.....	52
7.10. Timestamp Unit.....	52
7.11. Performance Counters.....	53
7.12. Instrumentation: ACT-CAP, ACT-ST, ACT-Proc.....	54
7.12.1. ACT-CAP.....	54
7.12.2. ACT-ST.....	54
7.12.3. ACT-Proc.....	54
7.13. AXIS Composer.....	55
7.14. eTIP Composer.....	55
7.15. Interfaces Between the Blocks.....	57
8. Processing Stage.....	58
8.1. Overview.....	58
8.2. Generic Message Generator.....	58
8.2.1. I-CNT Accounting.....	58
8.2.2. Branch History.....	60
8.2.3. Synchronization Handling.....	60
8.2.4. Emission Ordering.....	61
8.2.5. Idle and Empty.....	61
8.2.6. Backpressure, Ordering and Determinism.....	61
8.2.7. Latency.....	62
8.3. Nexus Message Formatter.....	62
8.4. E-Trace te_inst Generator.....	62
9. Compression Suite.....	64
9.1. Principle.....	64
9.2. Feature Matrix.....	65
9.3. Interactions and Exclusions.....	66
9.4. Implicit Return.....	66
9.5. Repeated History.....	67
9.5.1. Partial Match.....	67
9.6. Wide Instruction Count.....	68
9.7. Repeat Branch.....	68
9.8. Jump-Target Cache.....	69
9.9. Branch Prediction.....	69
9.10. Indirect Branch History with Sync (IBHS).....	70
9.11. Repeat Instruction.....	71
9.12. How much the suite actually saves.....	71
9.13. Standards Positioning.....	72

9.14. The E-Trace Back End	73
9.15. Verification	73
10. Output Stage	74
10.1. Overview	74
10.2. MSEO/MDO Formatter.....	74
10.2.1. MSEO Sequencing.....	75
10.2.2. MDO Width	75
10.2.3. Assembly Throughput.....	75
10.2.4. Backpressure, Framing and Latency	75
10.3. Compact Packer	76
10.4. eTIP Serialization	76
10.5. eTIP FIFO Fill-Level Histogram	77
10.6. Split-Load Data Trace	78
10.7. Overflow and Overrun Handling	78
11. Trace Funnel.....	79
11.1. Purpose	79
11.2. Parameters.....	79
11.3. Control and Status	80
11.4. Packet Detection	81
11.5. Arbitration	81
11.6. Backpressure.....	82
11.7. Flush	82
11.8. Synchronization Request.....	83
11.9. Dual-Protocol Operation (E-Trace Reference-Raw Channels)	83
11.10. Determinism and Ordering.....	84
11.11. Verification	84
12. Parallel Trace Output (PIB).....	85
12.1. Scope	85
12.2. The PIB that ships with the examples.....	85
12.3. The reference PIB implementation	86
12.3.1. Block Overview	86
12.3.2. Output Modes	87
12.3.3. Clocking and Data Rate	87
12.3.4. Calibration Patterns.....	88
12.3.5. Egress Selection.....	88
12.3.6. PIB Sink Status and Overflow	88
12.3.7. PIB Funnel: Arbitration	88
12.3.8. PIB Funnel: Overload Packet-Shedding.....	89
12.3.9. Register Map Overview	90

12.3.10. KR260 PMOD #3 Pinout	90
12.3.11. Integration	91
12.3.12. Implementation and Verification Status	91
13. Trace Format	93
13.1. One Encoder, Two Wire Formats	93
13.2. N-Trace Wire Format.	93
13.2.1. Two Serializations, One Format	94
13.2.2. Framing: MSEO and MDO	94
13.2.3. Field Types and Leading-Zero Suppression.	95
13.2.4. Program Address Compression	95
13.2.5. Instruction Count.	96
13.2.6. Branch History	97
13.2.7. From Core Event to Message	97
13.2.8. Data-Flow Mapping.	98
13.2.9. Timestamps.	99
13.2.10. Bandwidth Optimizations Not Implemented	100
13.3. E-Trace Wire Format (te_inst)	100
14. N-Trace Message Catalog	101
14.1. Specification baseline used in this chapter	102
14.2. Complete TCODE catalog.	102
14.2.1. What the reset configuration produces.	106
14.3. On-wire field order	107
14.3.1. Program-trace messages	107
14.3.2. Data-trace and instrumentation messages	108
14.3.3. Context, identification and error messages	109
14.3.4. Watchpoint (TCODE 15) – the bridge from ACT-ST into the stream.	109
14.4. Enumerated field values	110
14.4.1. SYNC – synchronization reason	110
14.4.2. RCODE – Resource Full reason.	111
14.4.3. EVCODE – Correlation event	111
14.4.4. BTYPE – branch type	112
14.4.5. ETYPE and ECODE – error message	112
14.4.6. DSZ and ELSZ – data-trace access size	112
14.5. Field type vocabulary.	113
14.6. Notes and findings	113
15. Data Acquisition Payload (DQDATA)	114
15.1. What the standard defines and what CTTE fills in	114
15.2. IDTAG	115
15.3. Payload structure	115

15.3.1. Leading-zero suppression across the whole payload	116
15.4. Layout per command	116
15.4.1. The packed context element of command 6	117
15.4.2. Access type and size	117
15.5. What a decoder needs	118
15.6. The same event on the AXI-Stream path	118
16. Register Model	119
16.1. Source of truth and generation	119
16.2. What the generated tables do and do not tell you	120
16.3. Address spaces and components	121
16.4. The capability principle: CAPS and ENAB	122
16.5. Programming order and the Enable lock	123
16.6. The indirect watchpoint table	123
16.7. Register tables	124
16.8. Elaboration parameters	124
16.9. Component overview	125
16.10. Component <code>te</code> (base 0)	125
16.10.1. <code>te.trTeControl</code> – offset 0x000	126
16.10.2. <code>te.trTeImpl</code> – offset 0x004	131
16.10.3. <code>te.trTeInstFeatures</code> – offset 0x008	131
16.10.4. <code>te.trTeInstFilters</code> – offset 0x00C	132
16.10.5. <code>te.trTeDataControl</code> – offset 0x010	133
16.10.6. <code>te.trTeDataFilters</code> – offset 0x01C	135
16.10.7. <code>te.trTeProtocolSel</code> – offset 0x030	135
16.10.8. <code>te.trTsControl</code> – offset 0x040	136
16.10.9. <code>te.trTsCounterLow</code> – offset 0x048	137
16.10.10. <code>te.trTsCounterHigh</code> – offset 0x04C	137
16.10.11. <code>te.trTeTrigDbgControl</code> – offset 0x050	138
16.10.12. <code>te.trTeTrigExtInControl</code> – offset 0x054	138
16.10.13. <code>te.trTeTrigExtOutControl</code> – offset 0x058	139
16.10.14. <code>te.trTeFilter[] .Control</code> – offset 0x400	140
16.10.15. <code>te.trTeFilter[] .Match</code> – offset 0x404	141
16.10.16. <code>te.trTeFilter[] .MatchChoiceEcauseLow</code> – offset 0x408	142
16.10.17. <code>te.trTeFilter[] .MatchChoiceEcauseHigh</code> – offset 0x40C	142
16.10.18. <code>te.trTeFilter[] .MatchValueImpdef</code> – offset 0x410	143
16.10.19. <code>te.trTeFilter[] .MatchMaskImpdef</code> – offset 0x414	143
16.10.20. <code>te.trTeFilter[] .MatchChoiceData</code> – offset 0x418	143
16.10.21. <code>te.trTeComp[] .Control</code> – offset 0x600	144
16.10.22. <code>te.trTeComp[] .SMaskLow</code> – offset 0x608	145

16.10.23. te.trTeComp[] .SMaskHigh — offset 0x60C	146
16.10.24. te.trTeComp[] .PMatchLow — offset 0x610	146
16.10.25. te.trTeComp[] .PMatchHigh — offset 0x614	147
16.10.26. te.trTeComp[] .SMatchLow — offset 0x618	147
16.10.27. te.trTeComp[] .SMatchHigh — offset 0x61C	148
16.10.28. te.trTeCsrControl — offset 0xE00	148
16.10.29. te.trTeTipFifoStatus — offset 0xE04	149
16.10.30. te.trTeSyncStatus — offset 0xE08	150
16.10.31. te.trTeTipFifoHistCtrl — offset 0xE10	150
16.10.32. te.trTeTipFifoHistData — offset 0xE14	151
16.11. Component atb (base 0x1000)	151
16.11.1. atb.trAtbBridgeControl — offset 0x1000	152
16.11.2. atb.trAtbBridgeImpl — offset 0x1004	152
16.12. Component pc (base 0x3000)	153
16.12.1. pc.trPcControl — offset 0x3000	153
16.12.2. pc.trPcImpl — offset 0x3004	154
16.12.3. pc.trTeConstants — offset 0x3008	154
16.12.4. pc.trPerfCntControl — offset 0x3010	155
16.12.5. pc.trTePerfCntIFetchRange[] .Low — offset 0x3100	156
16.12.6. pc.trTePerfCntIFetchRange[] .High — offset 0x3104	156
16.12.7. pc.trTePerfCntDataRdThRange[] .Low — offset 0x3200	157
16.12.8. pc.trTePerfCntDataRdThRange[] .High — offset 0x3204	157
16.12.9. pc.trTePerfCntDataRdRange[] .Low — offset 0x3300	158
16.12.10. pc.trTePerfCntDataRdRange[] .High — offset 0x3304	158
16.12.11. pc.trTePerfCntDataWrRange[] .Low — offset 0x3400	158
16.12.12. pc.trTePerfCntDataWrRange[] .High — offset 0x3404	159
16.13. Component wp (base 0x4000)	159
16.13.1. trWpControl — offset 0x4000	160
16.13.2. trWpImpl — offset 0x4004	160
16.13.3. trWpMask — offset 0x4008	161
16.13.4. trWpIndex — offset 0x400C	162
16.13.5. trWpDataLow — offset 0x4010	163
16.13.6. trWpDataHigh — offset 0x4014	163
16.13.7. trWpReadLow — offset 0x4018	164
16.13.8. trWpReadHigh — offset 0x401C	165
16.13.9. trWpCap — offset 0x4020	165
16.14. Component df (base 0x5000)	166
16.14.1. trDfControl — offset 0x5000	166
16.14.2. trDfImpl — offset 0x5004	167

16.15. Component <code>mem1</code> (base 0x6000)	167
16.15.1. <code>mem1.Key0</code> – offset 0x6000	167
16.15.2. <code>mem1.Key1</code> – offset 0x6004	168
16.16. Declared but not addressable	169
16.16.1. <code>trActCapStCmd</code> – offset 0xB10 (not decoded).....	169
17. Interface Contract.....	170
17.1. Top-level module	170
17.1.1. Parameters	170
17.1.2. Ports	172
17.2. Clocks.....	172
17.3. Resets.....	173
17.3.1. Resetting one domain without the others.....	173
17.4. Clock-domain crossing	174
17.5. Instruction ingress	174
17.5.1. Signals	175
17.5.2. Beat semantics.....	175
17.5.3. Block ingress	176
17.5.4. Core adapters.....	177
17.6. Trace egress	177
17.7. Event egress	178
17.8. Control interface	178
17.9. Shutdown	178
17.10. Feature switches, runtime controls and capability bits	179
18. N-Trace Self-Identification and Configuration Messages	183
18.1. The Vendor Config message (TCODE 58)	183
18.1.1. Field list.....	183
18.1.2. Version and forward compatibility	184
18.1.3. The capability map	184
18.1.4. The enabled map.....	185
18.1.5. The parameter words	186
18.1.6. When it is emitted.....	186
18.2. The Device ID message (TCODE 1)	187
18.3. What a decoder does with it	188
18.4. Failure mode: a stream that never says what it is.....	188
19. Hardware-Supported Instrumentation (ACT-CAP / ACT-ST / ACT-Proc)	190
19.1. What the instrumentation path is for.....	191
19.2. ACT-CAP – commands observed on the TIP.....	191
19.3. ACT-ST – commands synthesized by the encoder.....	192
19.3.1. Trigger source and matcher.....	192

19.3.2. Programming the table	193
19.3.3. Address-width limit at XLEN = 64	193
19.4. ACT-Proc – merge, priority and side effects	194
19.4.1. Merge and priority.	194
19.4.2. Sink decode	194
19.4.3. TE control side effects	194
19.4.4. Ordering, latency and backpressure.	195
19.5. How an ACT-ST hit becomes visible in the trace stream	195
19.6. Synchronization requests from the traced hart.	196
19.7. Determinism and reset behaviour	196
19.8. Licensing of ACT-CAP and ACT-ST	197
19.9. Related mechanisms in other architectures.	197
19.10. Verification status	197
20. The trActCapStCmd Command CSR	199
20.1. Register definition	199
20.2. Cmd field: trActCapStCmd_e	199
20.2.1. Two behaviours that differ from the obvious reading	200
20.3. Sink field: trActCapStSink_e	201
20.4. DirectData field	201
20.5. Software use via ACT-CAP.	202
20.5.1. C helper types	202
20.5.2. Constructing and issuing a command.	202
20.6. Use via the Smart Trigger (ACT-ST)	204
20.7. How a command reaches the wire	204
20.8. DAQ bandwidth and backpressure	205
20.9. Summary	206
21. HSI Transfer	207
21.1. Control structure	207
21.2. Data sources a command can reach	207
21.3. Accessing TE control registers through HSI	208
21.4. Nexus message composition	208
21.4.1. Event triggers.	209
21.4.2. Data Acquisition message (TCODE 7)	209
21.4.3. Watchpoint message (TCODE 15)	209
21.4.4. Synchronization message.	210
21.5. Region control.	210
21.6. TracePoint programming.	210
21.7. Trace destination	210
21.8. HSI sequence correctness.	211

21.9. Concept elements not implemented	212
22. Raw Event Path via AXI-Stream	213
22.1. Where the port sits	213
22.2. Signal contract	213
22.3. Packetisation and payload	214
22.3.1. The timestamp element	215
22.4. Flow control and loss behaviour	215
22.4.1. The reference shim	215
22.4.2. Alternative: a passive capture buffer	216
22.5. Relationship to the ATB trace path	216
22.6. Verification support	217
22.7. Integration checklist	217
23. Performance Analysis and Counters	219
23.1. The performance-counter component	219
23.1.1. Enable-locking, and why it is asymmetric	220
23.2. Counter banks	220
23.2.1. Range matching	221
23.3. Threshold measurement	221
23.4. Reading the counters	221
23.5. Determinism and latency	222
23.6. Using the counters	222
23.7. Related measurements documented elsewhere	223
24. Footprint Profiles and Resource/Timing Matrix	224
24.1. What a profile is, and where it is defined	224
24.2. Resource and timing matrix	225
24.2.1. How to read these numbers	225
24.2.2. One placed-and-routed data point	226
24.3. The structural levers between the profiles	226
24.4. What the instrumentation path costs	227
24.5. Choosing a profile	228
25. Integration Guide	229
25.1. Clocks and resets	229
25.2. Instruction ingress (<code>tip_if</code>)	230
25.3. Control interface and programming contract	231
25.4. Trace egress and protocol selection	231
25.5. Pause, stop, flush — the shutdown contract and bring-up order	232
25.6. Build switches, runtime fields, and structural parameters	232
25.7. Versioning caveat for core-specific adapters	233
26. Configuration Recipes and Pitfalls	234

26.1. Baseline programming order	234
26.2. Pitfalls.....	234
26.2.1. Vendor Config message silently suppressed by a stale control word.....	235
26.2.2. <code>InstSyncMax</code> reset changed from 0 to 6 (2026-08-12).....	235
26.2.3. Data-address compression mode invisible on a data-only stream	236
26.2.4. Trace-quota window counter does not re-arm the way a naive model expects.	236
26.2.5. Setting both or neither protocol parameter aborts elaboration	237
26.2.6. Waveform debug of encoder internals crashes Vivado™ XSIM	237
26.3. Recipe: bounding the re-anchor distance for a live/ring consumer.....	237
27. Decoding the Trace Stream.....	239
27.1. Three decoding paths	239
27.1.1. HDL-level decoding (in simulation).....	239
27.1.2. Offline decoding – CTTD, the reference decoder.	239
27.1.3. Online / live decoding.	240
27.2. Auto-configuration from the stream.....	240
27.3. The CTXP export format.....	241
27.4. Program-structure input: PCINFO.....	241
28. Demonstrators and Reference Designs.....	243
28.1. Entry levels	244
28.2. Catalog overview	245
28.3. KV260 board-gated examples	246
28.3.1. MicroBlaze™ V (<code>mbv</code>)	246
28.3.2. Dual TGC5B watchpoint/DAQ testbed (<code>tgc5b2_axis_wp</code>).....	246
28.3.3. MicroBlaze™ V + TGC5B, funnelled (<code>duo</code>)	247
28.3.4. CVA6 Linux (<code>cva6_linux</code>).....	248
28.4. The shared TGC5B base (<code>kv260/common/tgc5b</code>)	249
28.5. More KV260 examples, board-gated 2026-08-21.....	249
28.5.1. CVA6 RV64 Linux (<code>cva6_linux64</code>).....	249
28.5.2. Rocket RV64, single and dual hart (<code>rocket_linux</code> , <code>rocket2</code>)	250
28.5.3. CVA6 AMP pair (<code>cva6_2</code>).....	251
28.5.4. Three cores with mixed protocols (<code>trio</code>).....	252
28.6. Dashboard demo mode.....	253
28.6.1. Protocol interlock	253
28.7. Operating the KV260 flows	254
28.8. Tool-version note for the KV260 flows	255
29. Verification Strategy.....	256
29.1. The oracle principle	256
29.2. What makes a check a gate	257
29.3. Components	257

29.4. The reference decoder is a gate input, not a given	258
29.5. Test scenarios and round-trip gates	259
29.6. Robustness gates (state across discontinuities)	260
29.7. Standing in-RTL invariants (SVA).....	260
29.8. Formal property gates (SymbiYosys).....	261
29.9. Hardware-in-the-loop and SoC integration.....	262
29.10. Timing and resource verification.....	263
29.11. Reference-stream families and supersession	263
29.12. CI stages	264
30. Test Catalog.....	265
30.1. Basic control-flow and exception semantics	265
30.2. Compression suite	266
30.3. Robustness, status and register features	266
30.4. Sync cadence, ICNT and wide-field edge cases	268
30.5. Multi-source separation	269
30.6. E-Trace and protocol-parameter legs	270
30.7. Data trace	271
30.8. Hardware-supported instrumentation	272
30.9. Overflow and robustness (cross-cutting)	272
30.10. Combined end-to-end.....	274
31. Coverage Evidence	275
31.1. Measured vs. effective, and what closes the gap between them.....	275
31.2. The waiver ledger.....	275
31.3. Gap-closure history	277
31.4. Residual carryover clusters from Round 3-4, dispositioned in Round 5.....	277
31.5. Measurement mechanics	278
31.6. RTL defects the coverage campaign found	279
31.7. Provenance	279
31.8. Currency	279
32. Implementation Status Matrix.....	281
32.1. Legend	281
32.2. Status Matrix.....	281
32.3. Features added in the 1.0.0 cycle	285
33. Known Limitations and Deviations	288
33.1. Limitations that constrain integration (by design).....	288
33.2. Spec-compatibility corners.....	288
33.3. Deliberately not implemented / follow-on packages.....	289
33.4. Caller-less helper code (retirement candidates)	290
33.5. Vendor extensions – no standard-conformance claim.....	290

34. Development Support	291
34.1. Repository layout and build flow	291
34.1.1. Starting from a fresh clone or worktree	291
34.1.2. Simulation backend	292
34.1.3. Toolchain discovery	292
34.2. Register (RDL) flow	292
34.3. Testbench architecture	293
34.3.1. Per-module: directed queue-based scoreboarding	293
34.3.2. System level: scripted CPU model with an oracle event stream	294
34.3.3. <code>tt</code> assertions (<code>rtl/external/testtools/tt.sv</code>)	294
34.4. Simulation helper modules	295
34.5. Long-running simulations: watchdog policy	295
34.6. Recommended workflow	296
35. Demonstrator Register Maps	297
35.1. Scope: which SystemRDL source appears where	297
35.2. Reading the generated tables	298
35.3. The unified KV260 control window	299
35.3.1. Component overview	299
35.3.2. Component <code>common</code> (base 0)	300
<code>common.control</code> – offset 0x000	300
<code>common.status</code> – offset 0x004	301
<code>common.trace_beats</code> – offset 0x008	301
<code>common.trace_bytes</code> – offset 0x00C	302
<code>common.axis_beats</code> – offset 0x010	302
<code>common.trace_bufsz</code> – offset 0x014	303
35.3.3. Component <code>sinks</code> (base 0x18)	304
<code>sinks.sink_ctrl</code> – offset 0x018	304
<code>sinks.ddd_base</code> – offset 0x01C	305
<code>sinks.ddd_size</code> – offset 0x020	306
<code>sinks.ddd_wptr</code> – offset 0x024	306
<code>sinks.sink_stat</code> – offset 0x028	307
<code>sinks.ddd_drops</code> – offset 0x02C	308
<code>sinks.pib_drops</code> – offset 0x030	308
<code>sinks.ddd_beats</code> – offset 0x038	309
35.4. The TGC5B example SoC	309
35.4.1. KV260 application view (physical addresses)	310
35.4.2. PS control port (aperture-relative)	310
35.4.3. Registers of <code>ctrl</code> (base 0)	311
<code>ctrl.control</code> – offset 0x000	311

ctrl.status — offset 0x004	311
ctrl.trace_beats — offset 0x008	312
ctrl.trace_bytes — offset 0x00C	312
ctrl.axis_beats — offset 0x010	313
35.4.4. CPU data bus (dBus)	313
35.4.5. Generated peripheral block	314
35.5. The delivered two-core device (tgc5b2_axis_wp)	314
35.5.1. KV260 device view (physical addresses)	315
35.5.2. Registers of wpctrl (base 0xA0400000)	315
wpctrl.magic — offset 0xA0400000	315
wpctrl.shim0_drop — offset 0xA0400004	316
wpctrl.shim0_fill — offset 0xA0400008	316
wpctrl.shim0_ovf — offset 0xA040000C	316
wpctrl.shim1_drop — offset 0xA0400010	317
wpctrl.shim1_fill — offset 0xA0400014	317
wpctrl.shim1_ovf — offset 0xA0400018	318
wpctrl.ftime_lo — offset 0xA040001C	318
wpctrl.ftime_hi — offset 0xA0400020	319
wpctrl.rx_fifo_words — offset 0xA0400024	319
wpctrl.shim_records — offset 0xA0400028	320
35.5.3. PS control port (aperture-relative)	320
35.5.4. Registers of ctrl (base 0)	321
ctrl.control — offset 0x000	321
ctrl.status — offset 0x004	322
ctrl.trace_beats — offset 0x008	322
ctrl.trace_bytes — offset 0x00C	323
ctrl.trace_bufsz — offset 0x010	323
ctrl.sink_ctrl — offset 0x018	323
ctrl.ddd_base — offset 0x01C	324
ctrl.ddd_size — offset 0x020	325
ctrl.ddd_wptr — offset 0x024	325
ctrl.sink_stat — offset 0x028	326
ctrl.ddd_drops — offset 0x02C	326
ctrl.pib_drops — offset 0x030	327
ctrl.ddd_beats — offset 0x038	327
35.6. The runtime-verification testbed (tgc5b2_rvcfi)	327
35.6.1. KV260 device view (physical addresses)	328
35.6.2. Registers of wpctrl (base 0xA0400000)	329
wpctrl.shim0_drops — offset 0xA0400004	329

wpctrl.shim0_fill — offset 0xA0400008.....	329
wpctrl.shim0_overflow — offset 0xA040000C.....	330
wpctrl.shim1_drops — offset 0xA0400010	330
wpctrl.shim1_fill — offset 0xA0400014.....	330
wpctrl.shim1_overflow — offset 0xA0400018.....	331
35.6.3. PS control port (aperture-relative)	331
35.6.4. Registers of ctrl (base 0).....	332
ctrl.control — offset 0x000.....	332
ctrl.status — offset 0x004	333
ctrl.trace_beats — offset 0x008	334
ctrl.trace_bytes — offset 0x00C.....	334
ctrl.trace_bufsz — offset 0x010	334
35.6.5. Registers of obs (base 0x40).....	335
obs.db0_hits — offset 0x040.....	335
obs.db0_last — offset 0x044.....	335
obs.db1_hits — offset 0x048.....	336
obs.db1_last — offset 0x04C.....	336
obs.actcap0 — offset 0x050	337
obs.actcap1 — offset 0x054	337
obs.shared_sz — offset 0x058.....	338
obs.magic — offset 0x05C	338
35.6.6. Registers of console (base 0x60).....	339
console.con0_stat — offset 0x060.....	339
console.con0_pop — offset 0x064	340
console.con0_push_drops — offset 0x068	340
console.con1_stat — offset 0x070.....	340
console.con1_pop — offset 0x074	341
console.con1_push_drops — offset 0x078	341
35.6.7. Registers of ring0 (base 0x80)	342
ring0.ring_ctrl — offset 0x080	342
ring0.ring_base — offset 0x084	343
ring0.ring_size — offset 0x088	343
ring0.ring_wptr — offset 0x08C.....	344
ring0.ring_stat — offset 0x090	344
ring0.ring_drops — offset 0x094.....	345
ring0.ring_beats — offset 0x098.....	345
35.6.8. Core-side map (not reachable from the PS)	345
35.7. The interrupt peripherals of the example SoC	346
36. References	347

36.1. Standards and specifications	347
36.2. Core and trace-source adapters.	347
36.3. Tools.	348
36.4. Related Accemic documents	349
36.5. Public repositories	349
36.6. TRISTAN acknowledgment.	350
36.7. Related ecosystem references.	350
37. Copyright, License Model and Trademarks	351
37.1. This document's license	351
37.1.1. References used, and attribution for excerpts	351
37.1.2. Notice required by the Nexus 5001 Forum™ standard	352
37.2. IP license model	352
37.2.1. Hardware IP – dual license.	353
37.2.2. Third-party IP – the MINRES TGC5B core	353
37.2.3. Software – ISC	354
37.2.4. Documentation – CC BY 4.0	354
37.2.5. The reference decoder is fetched, not vendored.	354
37.2.6. Commercial licensing	355
37.2.7. Evaluation	355
37.3. Contributions and the Contributor License Agreement (CLA)	355
37.4. Mechanisms described by earlier revisions	355
37.5. ACT-CAP / ACT-ST attribution	356
37.6. Trademarks and naming	356
37.6.1. Attribution	356
37.6.2. Use of the RISC-V mark	357
37.7. Summary of obligations.	357

Public ~ Build 9f968d5e

Revision History

The following table shows the revision history for this document.

Version	Date	Changes
1.0 – 1.3	2025-06-20 – 2026-08-03	See <code>rm-ctrace.adoc</code> (superseded by this document)
2.0	2026-08-18	Complete rewrite as the CEDARtools.TraceEncoder reference manual. Restores the interface and register chapters removed in 1.2 (now generated from the SystemRDL and RTL sources instead of hand-written); adds a full message catalog, the core-adapter chapters (MicroBlaze™ V, Rocket, CVA6 ITI), the demonstrator chapter, a single resource/timing table with tool version and artefact paths, the configuration-pitfall section, and the standards-baseline and conformance chapters.
2.1	2026-08-24	Adds the demonstrator register maps as an appendix, generated from the example designs' own SystemRDL under the same drift gate as the encoder tables. The encoder tables gain the one register the SystemRDL declares and hides (<code>trActCapStCmd</code> , not addressable on the control bus) and the descriptions of the range-filter key words. States that a build profile changes the answer a configuration field gives, never its address.

Version	Date	Changes
2.2	2026-08-25	Brought up to CEDARtools.TraceEncoder release 1.0.0, which the title page now names. Corrects every source path the encoder repository moved (the shared TGC5B base, the dissolved <code>ci/</code> tree, the renamed core adapters, the <code>featparity</code> build profile) and drops the retired repository prefix from repo-relative paths. Recounts the test suite (62 system-level testbenches) and the SystemRDL sources (ten), and prints the address map of the delivered two-core KV260 device as a further generated appendix. States the ingress each shipped core can carry, and therefore on which cores the instrumentation commands produce a payload at all. Records the second cause of the instruction-count cap defect, the board conditions of the 2026-08-21 runs, and the reproduction limits of the coverage figures. Removes two figures that showed a different subject than their captions claimed, and the blanket "documentation is being updated" banner: where a chapter is incomplete, the gap is named in Chapter 33 instead.

Version	Date	Changes
2.3	2026-08-27	Structures the format documentation by protocol: the trace-format chapter opens with the N-Trace/E-Trace split, groups the message machinery under an explicit N-Trace section and gains an E-Trace wire-format section; the message catalog and the self-identification chapter carry the N-Trace name in their titles. Replaces the two remaining predecessor-era architecture figures with CTTE-own drawings verified against the RTL – the timestamp unit is drawn at last, the overflow injection is named and placed where it lives (inside the eTIP composer), dead blocks are gone – and replaces a third figure whose content did not match its caption. Prints the runtime-verification testbed's register maps as a further generated appendix, now that the example carries its own SystemRDL source, and records that testbed's first board verdicts (2026-08-26), which also end the "never exercised on silicon" statement for the doorbell ACT-CAP path. Corrects the Trace-FS reference (the demonstrator as published integrates an earlier C-Trace release, not CTTE 1.0.0), the wall-clock role (periodic-sync cadence, not the timestamp source), and every claim that cited a gitignored or untracked file as if it were part of the published tree; the publication guard now checks citations and counts against git's tracked-file set instead of the working tree, which is how those claims had survived it.

Chapter 1. Introduction, Scope and Naming

1.1. What this manual describes

CEDARtools.TraceEncoder (CTTE) is a RISC-V trace encoder in SystemVerilog. It takes a core's instruction-trace port, compresses the execution stream into Nexus-style trace messages, and emits them on a 32-bit ATB output; an additional, uncompressed AXI-Stream path carries the same events to on-chip consumers. Beyond branch trace it provides hardware-assisted instrumentation, data trace, performance counters and a self-identification message that lets a decoder configure itself from the stream.

This manual is the **reference** for that IP: what it does, how it is parameterised, what contract it presents to the surrounding SoC, and what has been verified. It is written for three readers, and each will use a different part of it:

Reader	Where to start
Integrator – dropping the encoder into an SoC	Chapter 17 (the signal and clocking contract), then Chapter 25 and Chapter 26 . The register model is Chapter 16 .
Software and tools author – programming the encoder, decoding its output	Chapter 13 and Chapter 14 for the wire format, Chapter 16 for the control surface, Chapter 19 for instrumentation, and Chapter 27 for the decode path.
Assessor – judging maturity and evidence	Chapter 29 , Chapter 30 , Chapter 31 and Chapter 32 , then Chapter 33 .

1.2. How this manual is organised

Part	Contents
A – Orientation	This chapter, the position of the encoder in the surrounding trace system, the standards it implements, and where it goes beyond them.
B – Architecture	The encoder's stages from trace ingress to formatted output: sources and core adapters, preprocessor, message generation, the compression suite, the output stage, the funnel and the parallel pin output.
C – Formats and interfaces	The wire format, the complete message catalog, the Data Acquisition payload, the register model, the interface contract, and self-identification.
D – Features	Hardware-supported instrumentation (ACT-CAP, ACT-ST, ACT-Proc), the command CSR, HSI transfer, the raw event path, performance analysis, and the footprint profiles with their measured resource and timing figures.
E – Integration and operation	The integration guide, configuration recipes and their pitfalls, the decode paths, and the demonstrator designs.
F – Verification and status	The verification strategy, the test catalog, coverage evidence, the implementation-status matrix, and known limitations.
G – Appendices	Development support, references, and the licence and trademark terms.

1.3. Relationship to the public documentation

The encoder is open source. Part of its documentation is published with the source and is deliberately more detailed than this manual — in particular the integration guide, the verification description and the trace-format description in the repository's `doc/` tree.

The division is by purpose, not by secrecy:

- **This manual** states the contract and the decisions: what the encoder guarantees, what a user must configure, what has been verified, and what the current limitations are. It is meant to be read end to end once, and consulted thereafter.
- **The public documentation** carries the derivations, the long-form argumentation and the per-case detail behind those statements.

Where both cover the same subject, this manual states it once and points outward rather than repeating it. Two chapters go the other way: the register model ([Chapter 16](#)) and the message catalog ([Chapter 14](#)) are **generated** from the SystemRDL and RTL sources during the build, so that they cannot drift from the implementation they describe.

1.4. Naming

The product is **CEDARtools.TraceEncoder**, abbreviated **CTTE**. The name places the encoder in the CEDARtools family from Accemic Technologies — tools for the live processing of trace data — because it grew out of the experience of building that family.

The earlier name **C-Trace** remains admissible as informal shorthand. It survives in the source identifier prefix `ct_*`, and — outside the source — in names that are nailed down beyond the repository: the nine KV260 application names `_ctrace_kv260`, the device-tree overlay `ctrace_resmem.dtso`, the reserved-memory node `ctrace-pl-ddr` and the block design `mbv_ctrace_soc`. It is no longer a general prefix in the RTL: the core adapters were renamed to `*_to_ctte_tip` on 2026-08-21, and the encoder's prose no longer uses the old name at all. Its C stands for *CEDARtools, the same house mark the current name carries in full.

NOTE

The `ACT-` prefix of the instrumentation protocols ACT-CAP, ACT-ST and ACT-Proc reads *A*ccemic *C*EDAR*T*ools. It derives from the house mark, not from the encoder's own name, and is therefore unaffected by the rename.

Several component and repository names changed during 2026. Where a reader may still encounter the earlier name, both are given:

Earlier name	Current name	Note
C-Trace	CEDARtools.TraceEncoder (CTTE)	The product described by this manual.

Earlier name	Current name	Note
NexRv for C-Trace	CTTD – CEDARtools.Trace Decoder	The reference decoder; see Chapter 27 . Descended from the NexRv reference code of the RISC-V Nexus trace project.
C-Trace eXPort format	CTXP	The architecture-agnostic trace export format; see Chapter 27 .

1.5. Conventions

Authority. Where this manual and the implementation disagree, the implementation is right and the manual has a defect. The order of authority is: the RTL and the SystemRDL register model first, then the measurement and test artefacts, then the published documentation, then this manual. Chapters state which of these a statement rests on.

Numbers. Every measured figure in this manual names its source – a file in the source tree, a commit, a test, or a report path – so that a reader can reproduce it. Where a figure could not be traced to such a source, it has been left out rather than estimated; the affected chapter says so.

Register and signal names are written as they appear in the sources (`trTeControl.SendConfig`, `iretire`), and are generated into the register chapter rather than transcribed.

Version. This revision describes **CEDARtools.TraceEncoder release 1.0.0** (2026-08-28), as the title page states. The release notes of that version list what changed and what is known not to work; a reader who needs to know whether a given statement applies to a different netlist has a finer instrument in the register chapter, where every generated table carries the SHA-256 of the SystemRDL source it was produced from ([Chapter 16](#)). Document revision and product release are separate numbers and are not kept in step: this manual is revised when its text is wrong, the encoder when its behaviour changes.

Paths. Every path written in backticks without a repository prefix – `rtl/ct_encoder.sv`, `tests/instruction/`, `doc/integration.adoc` – is relative to the root of the CTTE repository. The earlier prefix `c-trace/` has been dropped throughout: the repository is named `accemic/TraceEncoder`, and a repo-relative path survives the next rename where a prefixed one does not.

Standards. Statements about what a standard does or does not define carry the clause of the standard they rest on. Where the position is genuinely unclear, this manual says so instead of resolving the doubt in its own favour; see [Chapter 4](#).

Chapter 2. CTTE in Context

2.1. What the encoder is for

CTTE is built for **continuous** observation of a running system, not for post-mortem capture of a short window. That premise shapes every design decision in this manual: the compression suite exists so that a full instruction trace fits through a narrow link for hours rather than milliseconds; the instrumentation path exists so that software can contribute observations without stopping; the self-identification message exists so that a decoder attaching to a long-running stream can configure itself from what it receives.

The use cases it was designed against:

- measurement of test completeness (code coverage),
- test-case prioritisation,
- hybrid WCET (worst-case execution time) analysis,
- performance optimisation,
- online control-flow-integrity checking.

The design goals that follow from them:

- strict adherence to standard interfaces — the RISC-V Trace Ingress Port (TIP), AXI-Stream, ATB and Nexus messages ([Chapter 17](#)),
- comprehensive support for hardware-assisted instrumentation ([Chapter 19](#)),
- multi-core support,
- controllable forwarding of TIP data elements to a core acting as watchdog, CFI checker or runtime-verification unit ([Section 17.7](#)),
- no architectural side effects on the observed system: the encoder observes the trace port, it does not participate in execution.

The encoder is organised into clearly separated stages — ingress from the trace port, filtering and synchronisation control, message generation, and formatting — described in [Chapter 5](#).

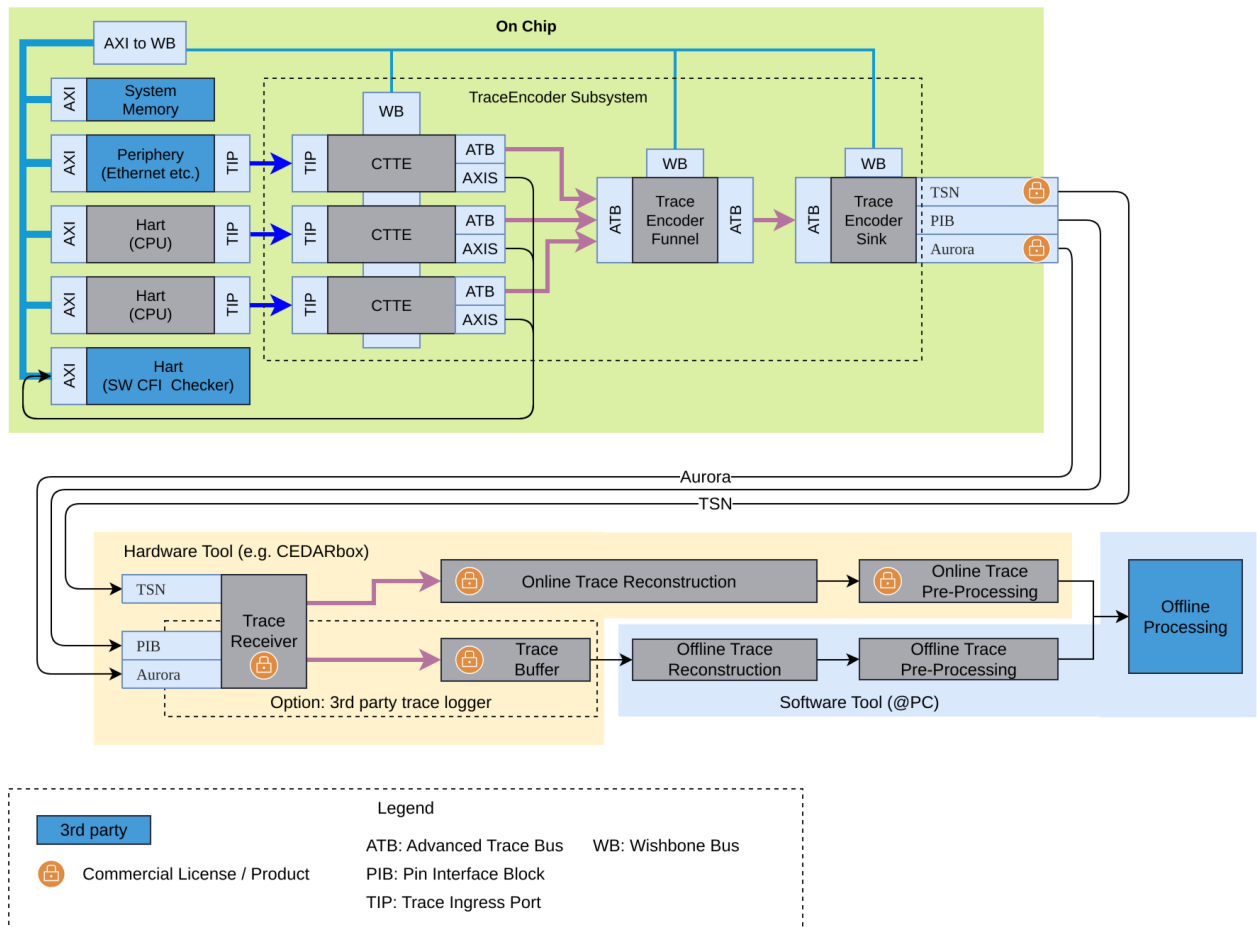


Figure 1. CTTE Overview

2.2. Accemic's trace layers

The CEDARtools trace system follows a layered architecture (L0-L7) with defined interfaces (IF0-IF6), covering physical transport, trace collection and encoding, context handling, application reconstruction and metrics, compilation and instrumentation mapping, and objective evaluation across the toolchain and the system under test.

The encoder described by this manual implements layers L1-L3 – Transport, Trace and Context. The remaining layers belong to the surrounding CEDARtools ecosystem and to host-side tooling, and are outside the scope of this document.

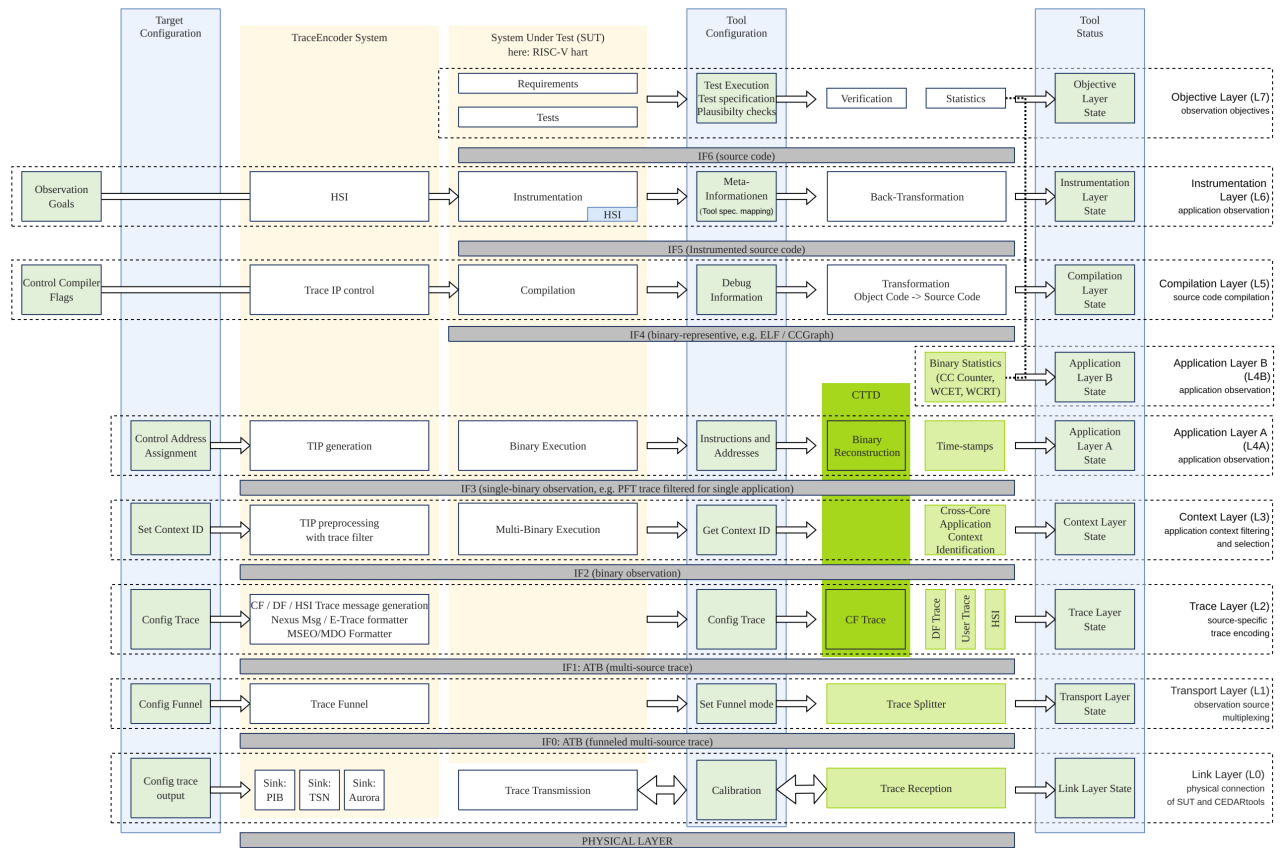


Figure 2. CTTE layer model

Table 1. Layer definitions

Layer	Name	Description
L0	Link Layer	Physical I/O for trace, including sinks such as Aurora, transmission, calibration and reception; exposes a Link Layer State for health and readiness.
L1	Transport Layer	Multiplexes sources through a trace funnel and splitter, controls funnel mode, and reports a Transport Layer State for source-aggregation status.
L2	Trace Layer	Generates CF/DF/HSI trace messages and formats them (Nexus, MSEO/MDO), with a Trace Layer State covering source-specific encoding.
L3	Context Layer	Filters and preprocesses TIP/trace, assigns or retrieves context IDs, and performs cross-core application-context identification.
L4	Application Layer	Reconstructs execution from instructions and addresses, adds timestamps, and derives binary statistics such as counters and WCET/WCRT.
L5	Compilation Layer	Correlates binaries and source through debug information and code/object transformations, tracking source-object mapping fidelity.
L6	Instrumentation Layer	Applies HSI instrumentation, attaches meta-information (tool-spec mapping) and supports back-transformation.
L7	Objective Layer	Drives test execution and plausibility checks and consolidates verification and statistics against observation objectives.

Table 2. Interface definitions

Interface	Description
IF0	Funnelled multi-source ATB trace on the physical link between transport concentration and the receiver path.
IF1	Multi-source ATB trace prior to funnelling/splitting, for source-aggregation control.
IF2	Binary observation stream after trace decoding into CF/DF/HSI events, for downstream context selection.
IF3	Single-binary observation (for example filtered to one application), enabling per-application reconstruction and timing.
IF4	Binary representation boundary (ELF, call-context graph) connecting reconstruction and statistics with compilation artefacts.
IF5	Instrumented source code handed to compilation and mapping tools.
IF6	Source-code boundary for requirements, tests and instrumentation planning.

2.3. What the integrator must supply

Two functions that a trace system normally needs are deliberately **not** part of the CTTE deliverable. Both are named here so that an integrator plans for them rather than discovering the gap during bring-up.

2.3.1. Cross-trigger fabric

Cross-triggering lets components broadcast events to each other. CTTE does not implement a cross-trigger fabric; an integrator who wants it pairs the encoder with an external implementation such as ARM CoreSight.

References:

- Arm CoreSight SoC-400 Technical Reference Manual (DDI0480G): Cross Trigger (p. 332)
- Arm CoreSight SoC-400 Technical Reference Manual (DDI0480G): Cross Trigger Registers (p. 192)

2.3.2. Debug unit

CTTE relies on an external debug IP for system-bus access and for configuring its control and status registers. That debug IP is outside the CTTE deliverable and must be supplied by the integrator. The register interface CTTE presents to it is specified in [Section 17.8](#).

Reference: [RISC-V Debug Specification](#)

Chapter 3. Standards Baseline

The CEDARtools.TraceEncoder is built against four public specifications. This chapter is the map: which document, in which revision, defines what; which parts of each CTTE implements; which parts it deliberately does not; and where the documents overlap or contradict each other. After reading it, an assessor knows which document to hold the encoder against for any given observation — a byte on the trace port, a register access, an ingress signal.

Two conventions from [Chapter 1](#) apply with full force here. Every statement about what a standard does or does not define carries the clause it rests on, and each such statement was checked against the primary document, not against a summary of it. Where the position is genuinely unclear, this chapter says so and takes the more cautious reading. What CTTE does **beyond** these documents is deliberately not this chapter's subject — that is [Chapter 4](#).

3.1. The four documents

Specification	Revision	Role for CTTE
IEEE-ISTO 5001™-2012, <i>The Nexus 5001 Forum™ Standard for a Global Embedded Processor Debug Interface</i>	v3.0.1 (2012)	The base standard. Assigns every transfer code (Table 4-5, p. 35), defines the message formats (sections 4.3.1 — 4.3.29), the field vocabulary and transmission order (Tables 4-2/4-3), and the vendor-defined code spaces CTTE's extensions live in.
<i>RISC-V N-Trace (Nexus-based Trace) Specification</i> , RISC-V International	Version 1.0, ratified 2024-11-21	The primary wire protocol. A profile of the Nexus standard for RISC-V: message subset (Table 9), fixed MSEO/MDO framing (chapter 3), control semantics (chapter 4, Table 8), synchronization (section 8.4, Table 25) and the optional bandwidth optimizations (chapter 9). The stream of an N-Trace-built CTTE instance is measured against this document first.
<i>Efficient Trace for RISC-V</i> , RISC-V International	Version 2.0, ratified 2022-05-05	Two distinct roles. Its chapter 4 defines the trace ingress interface that CTTE's TIP follows (Chapter 6) — this role is active in every build. Its chapter 7 defines the <code>te_inst</code> packet format that CTTE's alternative E-Trace back end emits — this role is active only in an <code>EN_ETRACE</code> build.
<i>RISC-V Trace Control Interface Specification (TCI)</i> , RISC-V International	Version 1.0, ratified 2024-11-21	The register model. Component conventions and address map (chapter 4), the Trace Encoder register set (chapter 6), the trace funnel component (chapter 8), the PIB sink and its pin protocol (chapter 9), and the ATB bridge (chapter 10).

The reference list with source locations and mirrors is in [Chapter 36](#).

3.2. How the four documents relate

N-Trace is a profile of Nexus, not a fork. Its own words: it describes "an appropriate selection of N-Trace messages compatible with the original IEEE-5001 Nexus Standard" (N-Trace chapter 1), and its transmission protocol is "a strict subset of IEEE-5001 Nexus Standard trace" (N-Trace Table 5).

Where Nexus leaves the port geometry to the vendor — one or two MSEO bits, at least two MDO bits, vendor-defined bit order — N-Trace fixes one choice: 6-bit MDO plus 2-bit MSEO packed into a byte, MSEO in the least significant bits, least significant bit of each field first (N-Trace Table 5, section 3.3). A TCODE can therefore be a fully defined Nexus Public Message **and** be "Reserved for future extensions" from an N-Trace decoder's point of view (N-Trace Table 9); the complete per-TCODE resolution of that double bookkeeping is [Chapter 14](#).

N-Trace declares exactly one incompatibility with Nexus. Its ECODE field is variable-length where Nexus defines a fixed-length error code, so that the optional TSTAMP field stays byte-aligned (N-Trace chapter 12). Everything else N-Trace adds is marked as a compatible, default-off extension in its text. CTTE follows the N-Trace form.

The ingress port comes from E-Trace. N-Trace does not define its own core interface; it references the "ratified Efficient Trace for RISC-V Specification ... chapter 4 Instruction Trace Interface" (N-Trace section 1.1). CTTE's TIP carries the full 4-bit instruction-type vocabulary of E-Trace Table 4.4, which is why the same eTIP stream can feed either back end ([Chapter 6](#), [Chapter 5](#)).

The register model comes from TCI and is shared. "Both N-Trace and E-Trace encoders are controlled by the same set of bits/fields in the same trTe??? registers" (TCI section 4.3); N-Trace chapter 4 (Table 8) then states, field by field, which of those TCI controls apply to N-Trace, which are optional, and which "must be hard coded as 0". Because almost every TCI field is individually optional and WARL, an implementation reports an absent option by legalizing the write — which is exactly how CTTE's register model behaves ([Chapter 16](#)).

Nexus compliance classes do not transfer. The Nexus standard defines compliance Classes 1 — 4, but from Class 2 upward they mandate the Nexus-defined interfaces, and from Class 3 the Auxiliary Access read/write service (IEEE-ISTO 5001-2012 section 4.1, Table 4-1). N-Trace says of these levels that they "are not directly applicable to N-Trace as Nexus levels always include debug levels" (N-Trace chapter 5, note). CTTE takes the same position: it implements Nexus **message formats** under the N-Trace profile, it does not implement the Nexus AUX port, the TAP path or the read/write-access service, and it claims no Nexus compliance class — and no certification of any kind; see [Chapter 4](#).

3.3. What CTTE implements from each document

3.3.1. N-Trace 1.0 — the primary back end

CTTE implements the complete N-Trace 1.0 program-trace message set — every message of Table 9 — in both instruction trace modes: history (HTM, `InstMode` = 6) and branch (BTM, `InstMode` = 3). Table 8 of the specification requires "at least a value of 3 or 6" to be settable; CTTE makes both settable when `CT_EN_BT` is built. The per-message generation conditions are in [Chapter 14](#); the reset configuration emits N-Trace Table 9 messages exclusively, with the single exception of the self-identification message ([Section 14.2.1](#)).

Beyond the mandatory core, CTTE implements the specification's optional parts as follows:

N-Trace 1.0 feature	Clause	CTTE
Repeated history (ResourceFull <code>RCODE</code> = 2 with repeat count)	Table 20, section 9.3	Implemented (<code>InstEnRepeatedHistory</code> , the control bit the specification itself names). Section 9.3 leaves the pattern detection method open ("the exact way to detect these patterns is not specified"); CTTE's partial-window trim lives inside that latitude. See Chapter 9 .
Implicit return	Section 9.2, TCI Table 11 bits [3] and [7:6]	Implemented as the specification's strongest variant — a stack of full expected return addresses (the <code>trTeInstImplicitReturnMode</code> = 3 behaviour), 16 entries deep, within the specification's 32-level bound. The TCI mode field itself is not implemented as a register; the structure is reported in the configuration message instead (Chapter 18).
Sequential jump inference	Section 9.1, TCI Table 11 bit [2]	Implemented as a build-time ingress convention (<code>CT_SIJUMP</code>) of the attached core rather than a runtime register bit; advertised on the wire as <code>CAPS</code> bit 6. See Chapter 4 for the register-side consequence.
Virtual-address MSB elision	Section 9.4 (<code>trTeInstExtendAddrMSB</code>)	Not implemented — the one chapter-9 optimization CTTE omits. The control field is a commented stub in the register source. At 32-bit addresses the measured benefit is zero, because leading-zero suppression is already byte-minimal there; the public trace-format documentation records this as planned for a 64-bit follow-on.
Timestamps	Section 8.5	Implemented exactly as specified: absolute value on every synchronizing message, delta to the previous timestamp otherwise, variable-length with leading zeros suppressed (Chapter 14).
Synchronization reasons	Table 25	All standard codes the ingress can observe, including the three Required codes 2, 3 and 7; one vendor code (14) from the range Table 25 designates for vendors. Codes 10–13, which Table 25 reserves for future standard use, are not emitted.
Address compression	Sections 8.1, 9.4	The base scheme: XOR against the previous transmitted address in the same address thread, least significant always-zero bit dropped, leading zeros suppressed (see Chapter 13).

3.3.2. IEEE-ISTO 5001-2012 — messages beyond the N-Trace profile

The N-Trace profile is deliberately narrower than the standard it profiles. CTTE optionally implements six Nexus Public Message groups that N-Trace 1.0 Table 9 does not adopt — each defined by the base standard, each off by default, each leaving the strict N-Trace 1.0 envelope when enabled (details and switches in [Chapter 14](#)):

- **Data trace** — Data Write/Read (TCODE 5/6, section 4.3.17) and their with-sync forms (13/14, section 4.3.18). N-Trace 1.0 excludes data trace ("must be 0 as IEEE-5001 Nexus Standard data trace messages are not part of version 1.0", Table 8) while naming Nexus-compatible data trace a likely future enhancement (Table 27).

- **Data Acquisition** (TCODE 7, section 4.3.21) – a Nexus **Public Message**, not a vendor extension; the payload contract is [Chapter 15](#).
- **Device ID** (TCODE 1, section 4.3.2, Table 4-7) and **Watchpoint** (TCODE 15, section 4.3.23).
- **Repeat Instruction** (TCODE 31/32, sections 4.3.14/4.3.15) – defined by Nexus, Reserved from N-Trace 1.0's point of view.
- **Vendor-defined messages** (TCODE 56/57/58) – inside the range 56 – 62 that both Table 4-5 of the standard and Table 9 of N-Trace designate for exactly this purpose.

The remaining Nexus message space CTTE does not implement, with the reason stated rather than implied:

Nexus message	Clause	Why not
Debug Status (0)	Section 4.3.1	No core contract for the 16 status bits: all three ingress adapters tie the debug-related TIP inputs to 0, and the two facts CTTE could state (debug entry, stop) are already carried more precisely by Correlation EVCODE 0/1 and SYNC 3/9. See Chapter 17 .
Program Trace Correction (10)	Section 4.3.12	The message retracts an instruction count already sent for a branch later cancelled. CTTE does not emit speculative trace, so there is nothing to correct. N-Trace 1.0 likewise leaves TCODE 10 Reserved (Table 9).
Port Replacement (20/21), Auxiliary Access (22 – 26)	Sections 4.3.24 – 4.3.29	CTTE is a trace source. It does not implement the Nexus auxiliary port or its read/write-access service; register access is by the TCI system-bus path instead (Section 17.8).
In-Circuit Trace (34/35)	Sections 4.3.19/4.3.20	Not implemented; no generation logic exists.

CTTE also does not implement the Nexus physical AUX port. The MSEO/MDO **framing** is used as N-Trace fixes it; the physical serialization onto pins is the TCI PIB (chapter 9 of the TCI specification), not the Nexus AUX (see [Chapter 12](#)).

3.3.3. Efficient Trace for RISC-V 2.0 – ingress always, back end optionally

The ingress role is unconditional: the TIP's instruction-type vocabulary is E-Trace Table 4.4, its privilege and trap sidebands follow E-Trace chapter 4, and the adapters for cores with narrower native vocabularies normalize into it ([Chapter 6](#)).

The back-end role is per instance: exactly one of `EN_NTRACE` / `EN_ETRACE` is set per encoder ([Chapter 5](#)). An E-Trace instance implements the **instruction trace** packet format of chapter 7 – the Format 3 synchronization, trap and support packets (subformats 0, 1 and 3, including `qual_status = trace_lost` loss reporting), Format 2 and Format 1 address and branch-map reporting, and both optional Format 0 efficiency extensions (0.0 correctly-predicted-branch count, 0.1 jump-target cache index, section 7.8) – following the reference algorithm of chapter 9 (baseline, with the optional modes of section 3.2 that CTTE's compression suite maps onto; see [Chapter 9](#)).

Not implemented on the E-Trace side:

- **Data trace** – neither the data-trace interface (E-Trace sections 4.3/4.4) nor the data-trace packets (chapter 8). Data trace, where built, is Nexus data trace on the N-Trace back end.
- **The Format 3 subformat 2 context packet** (E-Trace section 7.4) – not emitted by the `te_inst` generator (its format inventory is the module header of `rtl/ct_l2_te_inst_gen.sv`); process context on the N-Trace back end travels as the Ownership message instead.
- **64-bit addresses** – an `EN_ETRACE` build with `CT_XLEN = 64` is rejected at elaboration rather than truncating silently (Chapter 33).
- **A non-zero `irdepth` field width** – with the field at width zero, one implicit-return corner cannot be expressed on the wire; the consequence is measured and stated in Chapter 33.

3.3.4. TCI 1.0 – the register model and the sinks

CTTE implements the TCI component convention throughout: each component carries its `tr??Control` register with the `Active` bit at offset `0x000` and its `tr??Impl` identification register at `0x004` (TCI Table 3), and optional features are WARL fields whose write-read behaviour reports their presence (TCI chapter 6). Of the component set in TCI Table 1, CTTE provides:

TCI component	Type code	CTTE
Trace Encoder	0x1	The <code>te</code> block: control, instruction and data trace configuration, timestamp unit (TCI section 6.1), trigger registers (section 6.2, Tables 19 – 21), and the filter/comparator window <code>0x400 – 0x7FC</code> (section 6.3, Tables 22 – 24). See Chapter 16.
Trace ATB Bridge	0xE	The <code>atb</code> block (TCI chapter 10).
Trace PIB Sink	0xA	The examples ship a sink implementing the chapter-9 4-bit DDR mode (Section 12.2); the full component, including the Table 55 calibration patterns and a TCI chapter-8 funnel with its own <code>trFunnel??</code> register block, is a separate reference implementation outside this repository. See Chapter 12.
Trace Funnel	0x8	Not implemented as a TCI register component on the encoder side: CTTE's own funnel is configured by elaboration ports and has no registers (Chapter 11). The only <code>trFunnel??</code> registers in the system belong to the PIB's funnel.
Trace RAM Sink	0x9	Not implemented (TCI chapter 7). Off-chip capture uses the ATB/PIB path; on-chip consumers use the AXI-Stream export (Chapter 17).

For the trigger registers CTTE follows the TCI prescription for absent hardware: external trigger input #0 is implemented with the Table 20 actions 0/2/3/4, inputs #1 – 7, the trigger outputs (Table 21) and the vendor debug-trigger register (Table 19) read as fixed 0 – "if external trigger input #0 does not exist, then its action is fixed at 0" is the specification's own wording for this answer. The hart-stall options (`trTeInstStallEna`, `trTeDataStallEna`; TCI Table 9 bit 13, Table 13 bit 4) are not implemented: CTTE's ingress port is unidirectional by construction and drives no signal toward the hart; the drop policy (`trTeDataDropEna`, a TCI-defined control) is the implemented alternative (Chapter 4).

Three register-map components – performance counters (`pc`, type `0x3`), watchpoints (`wp`, `0x4`) and the data-flow range filter (`df`, `0x5`) – are CEDARtools extensions. Their type codes are not assigned by TCI Table 1, and the TCI does not designate a vendor range for component type codes; the registers carry `[CTTE ...]` markers in the generated tables, and the conformance consequences are stated in [Chapter 4](#).

3.4. Where the documents overlap or contradict

These are the known friction points between the four documents themselves – places where an assessor with two of the PDFs open side by side will see a difference. Each row names CTTE's position.

Friction point	The two readings	CTTE's position
Watchpoint mask polarity (within the Nexus standard itself)	Section 4.3.23 prose: watchpoints "can be masked ... by writing a '1'" – reads as 1 = suppressed. Table B-26 (register B.8.4): <code>WEM</code> "1 = enabled for WTM".	Follows Table B-26: a 1 enables reporting. A consumer written against the prose reading inverts the mask (Section 14.3.4).
ECODE field length	Nexus: fixed-length error code (Table 4-34 context). N-Trace: made variable-length – the profile's single declared incompatibility (N-Trace chapter 12).	Follows N-Trace: variable-length ECODE, TSTAMP stays byte-aligned.
ECODE bit meanings	Nexus Table 4-36 defines the full bitmap (watchpoint, data trace, program trace, ownership, status, data acquisition, in-circuit trace, vendor). N-Trace Table 16 adopts only program trace, ownership and vendor, and marks the rest Reserved – calling its encoding "a subset of IEEE-5001 Nexus Standard encoding".	Emits the full Nexus Table 4-36 granularity. The positions N-Trace leaves Reserved carry their Nexus meanings, which is the reading the N-Trace subset wording points back to (Chapter 14).
Synchronization cadence bound	Nexus: a with-sync message at least every 256 trace messages (Table 4-2, "Periodic Message Counter"). N-Trace/TCI: a configurable mechanism (<code>trTeInstSyncMode / InstSyncMax</code> , interval $2^{(InstSyncMax+4)}$ in a selectable unit) that does not restate the 256 bound.	Implements the N-Trace/TCI mechanism. A legal N-Trace configuration can therefore exceed the older Nexus bound; the CTTE reset cadence is stated, with its measured rationale, in the register model (Chapter 16).
Sequential-instruction-counter sync (<code>SYNC = 4</code>)	N-Trace section 8.3: "may be only used in BTM mode".	CTTE optionally uses it in HTM as a full synchronization – a documented deviation, off by default (Chapter 4).
Branch prediction and jump-target cache	E-Trace section 7.8 defines both as optional efficiency extensions. N-Trace Table 8 requires their TCI enable bits to be "hard coded as 0".	Standard-defined on the E-Trace back end; a documented vendor extension (TCODE 56/57, writable enables) on the N-Trace back end (Chapter 4).
Device ID field width and occasion	Nexus Table 4-7: ID is a fixed 32-bit field, emitted after a debug-logic reset of an AUX-only implementation.	CTTE emits the ID variable-length and per trace-on edge – both documented deviations, stated with their reasons in Chapter 4 .

Friction point	The two readings	CTTE's position
Terminology: "branch"	Nexus calls every flow change a branch. N-Trace deliberately uses RISC-V ISA terms – direct conditional branch , unconditional jump (N-Trace chapter 6, note).	This manual follows the N-Trace/RISC-V usage except when quoting Nexus message names.

3.5. Which document to measure against

Observation	Yardstick
Byte stream of an N-Trace instance	N-Trace 1.0 Table 9 and chapter 3 framing; field formats per IEEE-ISTO 5001-2012 sections 4.3.x for the messages the profile does not detail; the per-TCODE resolution is Chapter 14 .
Byte stream of an E-Trace instance	E-Trace 2.0 chapter 7, reference algorithm chapter 9.
Register accesses	TCI 1.0 chapters 4/6 with the N-Trace clarifications of N-Trace chapter 4 (Table 8); authoritative field-by-field tables, generated from the same source as the hardware, in Chapter 16 .
Ingress signals of the attached core	E-Trace 2.0 chapter 4, with the per-core adapter contracts of Chapter 6 .
Instrumentation payloads (TCODE 7)	IEEE-ISTO 5001-2012 section 4.3.21 for the message; Chapter 15 for the CTTE payload contract inside its DQDATA field.
Everything CTTE claims beyond these documents	Chapter 4 , one claim per row, each with its checked clause.

Chapter 4. Conformance and Differentiation

This chapter answers two questions an assessor asks in this order: **where does CTTE deviate from the specifications it implements**, and **what does it do that those specifications do not define**. The deviations come first, because a differentiation claim is only worth reading from a document that is honest about its departures.

Three ground rules govern every statement here:

- **The comparison is against four published documents** – IEEE-ISTO 5001-2012, RISC-V N-Trace 1.0, Efficient Trace for RISC-V 2.0 and the RISC-V Trace Control Interface 1.0, in the revisions named in [Chapter 3](#) – **and against nothing else**. "Not defined there" is checkable against those texts. It is not a claim about any vendor's product, and this chapter makes no such claim.
- **Every clause citation was checked against the primary document**, not against a summary. The line-by-line check, including the claims that did **not** survive it, is recorded in an internal audit protocol.
- **CTTE implements; it is not certified**. No Nexus compliance class is claimed ([Chapter 3](#)), and no conformance mark, compliance certification or endorsement by RISC-V International or by the IEEE-ISTO is claimed or implied.

4.1. The conformance position in one paragraph

A freshly reset CTTE instance with the committed full build profile emits a stream that stays within the N-Trace 1.0 message set, with one exception: the self-identification message, a Vendor-Defined message in the TCODE range 56 – 62 that both IEEE-ISTO 5001-2012 Table 4-5 and N-Trace 1.0 Table 9 designate for vendor use, and which any N-Trace decoder frames and skips by the self-delimiting MSEO framing even if it does not understand it. Setting `trTeControl.SendConfig = CFG_NONE` removes that exception ([Section 14.2.1](#)). Every departure from that envelope must be switched on twice – a compile switch and a runtime control, both default-off – and each such departure is listed either in the deviation table below or in the message catalog ([Chapter 14](#)).

4.2. Documented deviations

Each row is a place where CTTE's behaviour differs from a clause of one of the four documents. "Documented deviation" means: deliberate, default-off where a default exists, stated with its reason, and – where the wire is affected – announced to the decoder in-band.

Deviation	Standard position (clause)	CTTE behaviour and reason
SYNC = 4 used in history mode	N-Trace 1.0 section 8.3: the sequential-instruction-counter synchronization "may be only used in BTM mode"; Table 25 marks the code "Not required".	Optionally emitted in HTM as a full synchronization instead of the leaner <code>RCODE = 0</code> drain (<code>CT_EN_SEQ_SYNC</code> and <code>trTeControl.InstSeqSyncEnable</code> , reset 0). A full re-anchor at a known instruction-count cadence is what a mid-stream consumer can attach to; the deviation is off by default and byte-invisible until enabled.
Branch-prediction and jump-target-cache enables are writable on the N-Trace back end	N-Trace 1.0 Table 8: <code>trTeInstEnBranchPrediction</code> and <code>trTeInstEnJumpTargetCache</code> "must be hard coded as 0".	When built (<code>CT_EN_BP / CT_EN_JTC</code>), the two TCI Table 11 bits [4]/[5] are writable and the resulting messages are vendor TCODEs 56/57 in the designated vendor range. The stream deliberately leaves the N-Trace 1.0 message set; a decoder without the bit-identical model cannot follow it, which is why both structure sizes and enables are advertised in the configuration message (Chapter 18). Reset 0 on both bits keeps the default stream conformant.
Data trace built despite the N-Trace 1.0 profile	N-Trace 1.0 Table 8: <code>trTeDataImplemented</code> and "other <code>trTeData...</code> " "must be 0", because Nexus data-trace messages "are not part of version 1.0"; Table 27 names Nexus-compatible data trace a likely future enhancement.	With <code>CT_EN_DATA_TRACE</code> , CTTE implements the IEEE-ISTO 5001-2012 data-trace messages (sections 4.3.17/4.3.18) and the TCI <code>trTeDataControl</code> register. This is an opt-in Nexus extension outside strict N-Trace 1.0, reset-off (<code>trTeDataControl.DataTracing = 0</code>).
Device ID field width and emission occasion	IEEE-ISTO 5001-2012 Table 4-7: the ID is a fixed 32-bit field; section 4.3.2 ties emission to a tool-initiated debug-logic reset of an AUX-only implementation.	CTTE emits the ID as a variable-length field (leading zeros stripped) and once per trace-on edge. Reasons: a fixed field would need a special case in both egress paths for a value a decoder reads as an integer anyway, and CTTE has no Nexus AUX or tool-reset concept – the trace-on edge is the moment a decoder needs the identity. A consumer expecting 32 fixed bits must be told; the pinned reference decoder reads the variable form (Section 29.4).
Sequential-jump inference without its TCI register bit	TCI 1.0 Table 11 bit [2] (<code>trTeInstEnSequentialJump</code>) is the runtime enable for the N-Trace section 9.1 optimization.	CTTE fixes the choice at elaboration (<code>CT_SIJUMP</code> , an attribute of the attached core's ingress convention); the TCI bit is not implemented and reads 0 even when the inference is active. A tool that reads only the register would misjudge the stream; the authoritative advertisement is <code>CAPS</code> bit 6 in the configuration message (Chapter 18). The same holds for the implicit-return mode field (TCI Table 11 bits [7:6], not implemented): the model structure is reported in-band, not in the register map.

Deviation	Standard position (clause)	CTTE behaviour and reason
CTTE register extensions occupy TCI-reserved bit positions	TCI 1.0 Table 9 reserves <code>trTeControl</code> bits [8:7], [10], [19:18] and [31:27]; Table 11 reserves <code>trTeInstFeatures</code> bits [15:11]. TCI's designated vendor space is the register range 0xE00 – 0xFFF (Table 3); no vendor designation exists for reserved bits ("software should always write 0 to reserved bits", TCI chapter 5).	CTTE places <code>SendConfig</code> [8:7], <code>InstSeqSyncEnable</code> [10], the two upper <code>InstSyncMode</code> bits [19:18], <code>SendDeviceId</code> [29:28] and the four extension enables <code>InstEnRepeatBranch</code> [11], <code>InstEnWideIcnt</code> [12], <code>InstEnIbhs</code> [13], <code>InstEnRepeatInstr</code> [14] in reserved positions. All reset to the value a TCI-only tool would write (0), except <code>SendConfig</code> (reset <code>CFG_ONCE</code> , documented in Chapter 18); every one is marked [CTTE ...] in the generated register tables (Chapter 16). Risk, stated plainly: a future TCI minor revision may assign these bits differently; the register source would then be revised, and the <code>tr??Impl</code> version fields are the discovery path for that case.
Extension components use unassigned TCI component type codes	TCI 1.0 Table 1 assigns component types 0x1, 0x8, 0x9, 0xA, 0xE; it does not assign 0x3, 0x4, 0x5 and does not designate a vendor range for type codes.	The performance-counter, watchpoint and range-filter components report types 0x3/0x4/0x5. Whether unassigned codes are free for vendor use is not stated by the TCI – the honest reading is "not assigned by TCI Table 1", and a future revision could assign them. The components follow the TCI register conventions (<code>Control/Impl</code> pair, <code>Active</code> bit) so that TCI-aware tooling can at least identify and skip them (Chapter 16).
Repeat Instruction messages on an N-Trace stream	IEEE-ISTO 5001-2012 sections 4.3.14/4.3.15 define <code>TCODE 31/32</code> ; N-Trace 1.0 Table 9 leaves them Reserved.	Opt-in (<code>CT_EN_REPEAT_INSTR</code> , <code>InstEnRepeatInstr</code> , reset 0): an enabled stream is an Accemic/ISTO extension rather than a strict N-Trace 1.0 stream. A strict N-Trace decoder skips the messages by framing but loses the loop compression (Chapter 9).

Two earlier deviations no longer exist and are listed so that a reader of older captures can date them: CTTE once used the standard-reserved sync codes 12/13 for vendor sync requests (now the single vendor-designated code 14, N-Trace Table 25) and once emitted Correlation messages with `CDF = 0` in history mode (now always `CDF = 1` with the `HIST` field present, per N-Trace Table 24). Both corrections are pinned by the byte-identity test legs ([Chapter 30](#)).

4.3. Differentiation – what CTTE adds beyond the four documents

The discipline of this table is the reason it is short. For every candidate row the question was asked verbatim: **does one of the four documents already define this?** If yes, the row does not appear as differentiation – however well CTTE implements it. What survives is listed with the clause that marks the boundary of the standard's territory and the in-repository evidence for CTTE's part. Announced loss, source filtering, repeated-history compression, implicit return and delta timestamps are examples of capabilities that did **not** survive – the standards define them, and CTTE simply implements them; the audit protocol records each such verdict.

Capability	What the standards define (clause)	What CTTE adds	Evidence
Software-triggered instrumentation	IEEE-ISTO 5001-2012 section 4.3.21 defines the Data Acquisition message as a Public Message and expressly allows "vendor-defined mechanisms for instrumenting software" as its trigger. N-Trace 1.0 Table 9 does not adopt TCODE 7.	The vendor-defined mechanism itself: the ACT-CAP/ACT-ST command protocol—a single retiring <code>csrw</code> to CPU CSR <code>0x0B10</code> composes a complete DQM with up to three 64-bit elements (192-bit payload) and routes it to the trace port, the AXI-Stream port, or both. The message on the wire is standard; the composition path is the CEDARtools contribution.	Chapter 19 , Chapter 15
Event-triggered instrumentation with identity	IEEE-ISTO 5001-2012 section 4.3.23 defines the Watchpoint message and B.8.4/Table B-26 its 16-bit enable mask. N-Trace 1.0 does not adopt TCODE 15.	A 1023-slot watchpoint table driven through the ACT-ST command path, reported through the standard 16-bit <code>WPHIT/WEM</code> mechanism with a software-owned bit-to-slot convention—and a documented resolution of the standard's own mask-polarity conflict (Chapter 3).	Chapter 19 , Section 14.3.4
Announced loss, with measured recovery behaviour	Loss announcement itself is standard three times over: IEEE-ISTO 5001-2012 section 4.3.22 (Error before the next sync), N-Trace 1.0 section 7.4 with <code>SYNC = 7</code> Required (Table 25), and E-Trace 2.0 section 7.5 (<code>qual_status = trace_lost</code>). CTTE claims no novelty for announcing loss.	What the documents do not provide is evidence of how an implementation behaves across a loss: CTTE carries a 15-scenario overflow/robustness suite—injected and natural overflow, compression-model state across recovery, pause edges, trap-window anchors, flush ordering—each gated in CI, plus the full IEEE-ISTO Table 4-36 <code>ECODE</code> class granularity beyond the subset N-Trace 1.0 Table 16 adopts.	Chapter 30 , Chapter 33
Data-trace shedding with a non-desynchronizing announcement	TCI 1.0 Table 13 defines the policy bit (<code>trTeDataDropEna</code> : suppress data trace "at some watermark level" to protect the instruction trace)—the policy is standard.	The defined contract on top of the bit: shedding starts at a named watermark (3/4 of the eTIP queue), each drop episode is announced by exactly one Error message with <code>ECODE = 0x02</code> and deliberately without the <code>SYNC = 7</code> re-anchor—the instruction trace was not lost and is not forced to re-anchor. The consequence of getting this wrong is measured: a decoder treating the marker as a full desynchronization recovered 269 of 760 PCs on the gate test; the scoped reference decoder recovers 760 of 760 (<code>tests/data/06_df_drop</code>).	Chapter 30 , Chapter 32
Filter gaps announced in standard vocabulary	Source filtering is standard: TCI 1.0 section 6.3 defines the filter and comparator register machinery, E-Trace 2.0 section 2.3 the filtering model. CTTE claims no novelty for filtering at the source.	The gap contract: leaving a filtered region emits Correlation <code>EVCODE = 4</code> , re-entering re-anchors with <code>SYNC = 5</code> —the existing pause vocabulary, so a stock decoder needs no change to consume a filtered stream; plus per-process filtering by routing the TIP context bus into a standard comparator input. Verified end to end in simulation with real core RTL; the board-level kernel-written-ASID leg is an open follow-on, stated as such.	Chapter 16 , Chapter 33

Capability	What the standards define (clause)	What CTTE adds	Evidence
In-band self-description of build and configuration	The vendor space is standard-designated (Table 4-5; N-Trace 1.0 Table 9: TCODEs 56 – 62 "for use by Vendor Defined messages"). E-Trace 2.0 reports its run-time option bits in-band (section 7.5 <i>ioptions</i>) and its structural parameters out-of-band (chapter 10 discovery). Neither Nexus nor N-Trace defines a configuration message.	A vendor message (TCODE 58) that separates compiled-in (CAPS) from currently enabled (ENAB) and carries the structural sizes of the compression models — return stack, jump-target cache, branch predictor — and the address width, so a decoder attaching to a stream configures both of its model sets from the stream alone, with no out-of-band step. Forward compatibility is append-only and versioned (CFGVER).	Chapter 18
Compression beyond the defined schemes	N-Trace 1.0 chapter 9 defines sequential jump, implicit return and repeated history; IEEE-ISTO 5001-2012 defines Repeat Branch and Repeat Instruction; E-Trace 2.0 section 7.8 defines branch-prediction and jump-target-cache packet formats. The schemes as such are standard territory.	Three additions. (1) Branch prediction (512-entry) and jump-target cache (64-entry) on the N-Trace back end , as vendor TCODEs 56/57 with the model structure advertised in-band — N-Trace itself hard codes these features off. (2) A replayability condition on Repeat Branch collapse that the base standard's match rule (section 4.3.13: same I-CNT and target) does not require but that decode correctness does — found by seeded soak, pinned by a regression gate. (3) A per-feature proof obligation: every compression feature has a byte-identity leg showing the same decoded PC sequence at a strictly smaller byte count, and an off-state byte-identical to a build without the feature.	Chapter 9, Chapter 30
Per-instance protocol choice from one netlist	TCI 1.0 Table 9 (<i>trTeFormat</i>) anticipates that a trace encoder speaks E-Trace or N-Trace; none of the documents addresses building both from one source with per-instance selection.	One netlist source elaborates to an N-Trace or an E-Trace encoder per instance (EN_NTRACE/EN_ETRACE, exactly one), with read-only discovery (<i>trTeProtocolSel</i> , <i>trTeImpl.ProtocolMajor</i>) and a hardware framing advertisement (<i>atb_te_raw</i>) so downstream logic parses what the netlist actually emits. The funnel's CTMX container carries an N-Trace stream next to E-Trace reference-raw channels and re-splits them byte-identically.	Chapter 17, Section 11.9
Uncompressed on-chip event output	Within the four documents, trace leaves an encoder through the trace path: TCI 1.0 Table 2 enumerates the allowed component connections (encoder to sink, funnel or ATB bridge); no second, uncompressed consumer-facing output is defined. The statement is scoped to these documents, not to the market.	A 96-bit AXI-Stream export that carries instrumentation events — including program-counter and timestamp elements — to in-fabric consumers (watchdogs, CFI checkers, runtime verification) in parallel with, and independent of, the compressed trace stream.	Section 17.7, Chapter 19

Capability	What the standards define (clause)	What CTTE adds	Evidence
Per-region performance counters at the encoder	None of the four documents defines a performance-counter component (mechanically checked: the term does not occur in any of the four texts); the TCI component list (Table 1) has no entry for one. Scoped to these documents.	A counter block with address-range regions and stall thresholds, implemented as a CEDARtools register-map component (type 0x3 – see the deviation table above for the type-code caveat).	Chapter 16

4.4. What supports these statements

Conformance claims are only as good as the evidence path behind them. Three properties of the CTTE verification setup carry the weight:

- **An independent decoder is the oracle.** Every stream check round-trips through CTTE, a separately maintained decoder pinned by checksum, and is compared against the CPU model's expected output – the encoder is never its own referee ([Chapter 29, Section 29.4](#)).
- **The wire format is pinned byte-exactly.** Byte-identity legs turn any unintended stream change into a red gate, which is what makes "the reset stream stays inside N-Trace 1.0" a regression-checked property rather than a design intention ([Chapter 30](#)).
- **Coverage and status are published with their provenance.** The line-coverage figures, their waivers and their currency caveats are in [Chapter 31](#); the per-feature implementation status, with its evidence column, is [Chapter 32](#); the known limitations, including the spec-compatibility corners, are [Chapter 33](#).

On real hardware, the deepest single conformance datapoint to date is the CVA6/Linux demonstrator: an OpenSBI and Linux boot captured through the trace-encoder port and decoded to 4,619,090 instructions with zero errors ([Chapter 28](#)).

Chapter 5. Encoder Architecture

5.1. Overview

The CEDARtools.TraceEncoder (CTTE) converts a RISC-V hart's Trace Ingress Port (TIP) into a compressed, standards-based trace byte stream on an ATB output, plus an uncompressed AXI-Stream event path for on-chip processing. It is the central element of the trace subsystem: everything upstream of it is core-specific adaptation, everything downstream of it is transport and decoding.

The top-level module is `ct_encoder (rtl/ct_encoder.sv)`. Its responsibilities, in the order the data flows through them, are:

- accept TIP retirement beats in the `tip_clk` domain and preprocess them into an internal **extended TIP (eTIP)** event stream (`ct_L23_preproc`),
- turn eTIP events into protocol messages in the `proc_clk` domain (`ct_L2_msg_gen` for N-Trace, `ct_L2_te_inst_gen` for E-Trace),
- serialize those messages into the on-wire byte format and cross into the `atb_atclk` output domain (`ct_L2_nexus_formatter` + `ct_L2_mseo_mdo_formatter`, or `ct_L2_compact_packer`, or `ct_L2_te_packetizer`),
- and export selected instrumentation payloads on AXI-Stream without passing through the trace format at all.

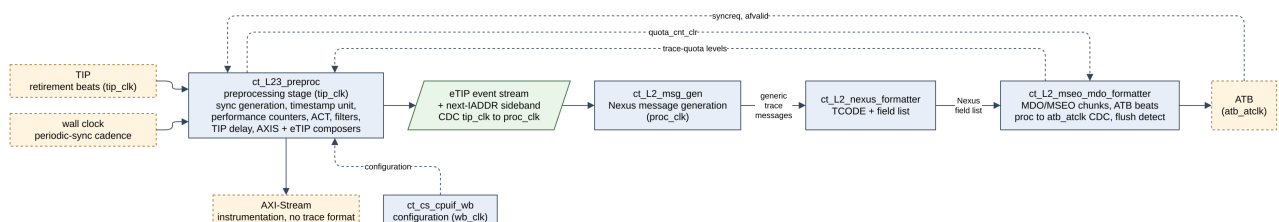


Figure 3. CTTE trace encoder overview – the N-Trace egress path

The overview above and the element diagram further below draw the **N-Trace** egress chain only – message generation, the Nexus formatter and the MSEO/MDO formatter. The figure below shows all three egress paths: the same preprocessor output, eTIP, reaches the ATB output on any of them.

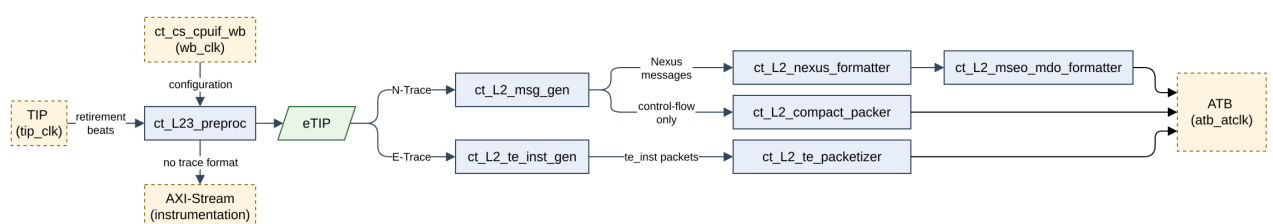


Figure 4. One event stream, three egress paths

The three are alternatives, not stages. The N-Trace chain is `ct_L2_msg_gen` → `ct_L2_nexus_formatter` → `ct_L2_mseo_mdo_formatter`; the compact path replaces the last two with `ct_L2_compact_packer` for control-flow-only builds; the E-Trace path is `ct_L2_te_inst_gen` → `ct_L2_te_packetizer`. Which of them a given build contains is fixed at synthesis, not selected at runtime – see [Section 5.3](#). The AXI-Stream output leaves the preprocessor without passing through any trace format at all, and the Wishbone CSR shim in the `wb_clk` domain configures every stage.

5.2. Elements

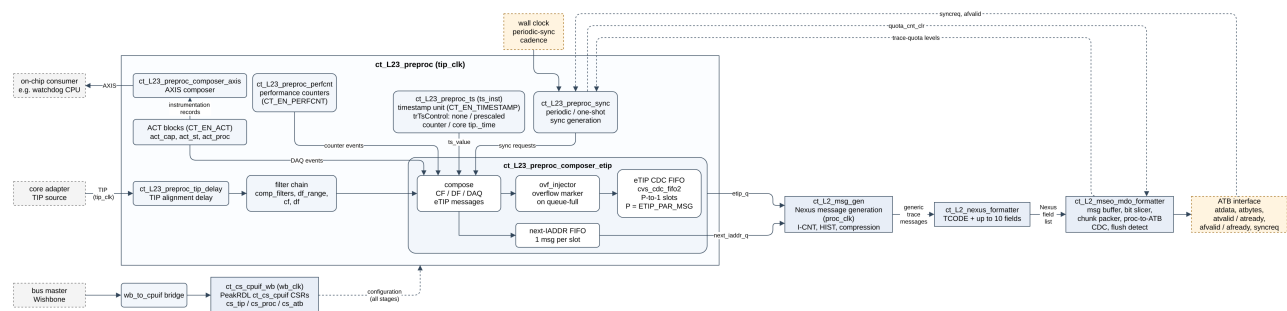


Figure 5. CTTE trace encoder elements – the N-Trace egress path

Data flows left to right. The TIP enters in the `tip_clk` domain; the preprocessor crosses it into the `proc_clk` pipeline through a dual-clock FIFO; the output stage crosses again into the `atb_atclk` domain. The Wishbone CSR shim (`wb_clk`) configures every stage. The free-running `wall_clk` paces the periodic-synchronization generator (`ct_L23_preproc_sync`); it is **not** the timestamp source – timestamps come from the preprocessor’s own timestamp unit (`ct_L23_preproc_ts`, built with `CT_EN_TIMESTAMP`), whose source is selected by `trTsControl`: `none`, a prescaled internal counter in the `tip_clk` domain, or the core’s own `tip._time`.

The module naming encodes the layer:

Prefix	Meaning
<code>ct_L23_*</code>	Layer 2/3 preprocessing, <code>tip_clk</code> domain (<code>rtl/ct_L23_preproc.sv</code> and <code>rtl/preproc/*</code>).
<code>ct_L2_*</code>	Layer 2 message generation, formatting and egress, <code>proc_clk/atb_atclk</code> domains.
<code>ct_L1_*</code>	Layer 1 transport-level aggregation – the trace funnel (<code>rtl/ct_L1_funnel.sv</code>), see Chapter 11 .

The design is deliberately built from small, self-contained stages with explicit interfaces, so individual blocks can be modified, replaced or added without disturbing the rest of the encoder.

5.3. Two Back Ends, One Front End

CTTE implements two output protocols. Which one is built is a **synthesis** choice, not a runtime one:

Back end	Generator	Egress
N-Trace	<code>ct_L2_msg_gen</code> (<code>rtl/ct_L2_msg_gen.sv</code>) – Nexus messages per the RISC-V N-Trace specification.	<code>ct_L2_nexus_formatter</code> + <code>ct_L2_mseo_mdo_formatter</code> , or <code>ct_L2_compact_packer</code> in CF-only profiles.
E-Trace	<code>ct_L2_te_inst_gen</code> (<code>rtl/ct_L2_te_inst_gen.sv</code>) – <code>te_inst</code> packets per the RISC-V Efficient Trace specification.	<code>ct_L2_te_packetizer</code> (<code>rtl/ct_L2_te_packetizer.sv</code>), reference-raw framed <code>te_inst</code> bytes.

Everything upstream of the back end is protocol-agnostic: `ct_L23_preproc` turns TIP beats into the eTIP event stream that **both** generators consume.

The selection is per **instance**, through the parameters `EN_NTRACE` / `EN_ETRACE` (defaulting to `ct_pkg::CT_EN_NTRACE` / `ct_pkg::CT_EN_ETRACE`). Exactly one of the two must be set; an elaboration guard in `rtl/ct_encoder.sv` rejects both-zero and both-one builds. Because the choice is per instance, a multi-encoder SoC can build one encoder as N-Trace and the next as E-Trace out of the same netlist sources. The output port `atb_te_raw` advertises the framing this instance produces (0 = Nexus MSEO/MDO chunks, 1 = E-Trace reference-raw `te_inst` bytes) so a downstream funnel or sink parses what the netlist actually emits rather than a second, software-programmed copy of that fact. The read-only discovery mirrors `trTeProtocolSel.Protocol` and `trTeImpl.ProtocolMajor` report the same truth to software.

NOTE

The historical dual build with a runtime protocol select has been retired; the protocol is a synthesis parameter (`rtl/ct_encoder.sv`, `genDualBackend` guard).

5.4. Clock Domains

`ct_encoder` spans five clock domains, crossed internally by explicit CDC logic (`rtl/ct_encoder.sv` module header).

Clock	Reset	Role
<code>tip_clk</code>	<code>tip_rst</code>	TIP ingress – the core's retirement rate. The whole preprocessor runs here.
<code>proc_clk</code>	<code>proc_rst</code>	Internal processing pipeline: message generation and formatting.
<code>atb_atclk</code>	<code>atb_atresetn</code>	ATB trace output.
<code>wb_clk</code>	<code>wb_rst</code>	Wishbone CSR access.
<code>wall_clk</code>	<code>wall_clk_rst</code>	Free-running cadence reference for periodic synchronization (<code>ct_L23_preproc_sync</code>). Not the timestamp source – see the note under the element diagram.

An additional input `ct_cs_rst` provides a soft reset of the CSR shim independent of the functional resets. A single-clock build is possible (`ct_pkg::CT_SINGLE_CLOCK`), which shrinks the CDC FIFO depths accordingly (`ETIP_CDC_FIFO_DEPTH` in `rtl/pkg/ct_pkg.sv`).

The clocking, reset and CDC contract that an integrator has to satisfy is stated in [the interface](#)

[chapter](#); this chapter only names the domains.

5.5. Top-Level Ports

The module connects through SystemVerilog interfaces rather than flat port lists:

Interface	Direction	Carries
tip (tip_if.slave)	in	Instruction and data retirement from the core: itype, ecause, iaddr, priv, _context, iretire, ilastsize, and for data trace dretire, dtype, daddr, dsize, data (or, in split-load mode, sdata / lresp / ldata). Field set defined in rtl/pkg/tip_if.sv and rtl/pkg/tip_pkg.sv.
wb (wb_if.slave)	in	Wishbone control/status register access. The register layout is defined in rdl/ct_cs_cpuif.rdl.
atb (atb_if.master)	out	32-bit ATB trace stream (atdata, atbytes, atvalid/atready, flush handshake afvalid/syncreq).
axis (axis_if.master)	out	AXI-Stream instrumentation / DAQ export. The payload width is ct_pkg::ACT_CAP_AXIS_TDATA_WIDTH = 3 x ACT_CAP_INT_ELEMENT_WIDTH (32 bit), i.e. three 32-bit elements per beat.
atb_te_raw	out	Constant framing advertisement of this instance (see above).

The detailed signal-level contract – timing, valid/ready rules, CDC requirements – belongs to [the interface chapter](#) and is not repeated here.

5.6. Elaboration Guards

ct_encoder refuses to build configurations whose failure mode would be **silent**, i.e. would produce a well-formed trace stream that is nevertheless wrong. This is a deliberate design decision: each of the guards below covers a case in which no capture, no assertion and no decoder could recover the lost information after the fact.

Guard (generate label)	Rejected configuration
genNoBackend / genDualBackend	Neither or both of EN_NTRACE / EN_ETRACE set.
genEtraceNoSideband	EN_ETRACE = 1 in a netlist built with ct_pkg::CT_EN_ETRACE = 0. The eTIP sideband widths (privilege, ecause, tval, last-instruction size) are sized by the package, so such a build would emit te_inst format 3.x messages assembled from 1-bit stubs.
genBadXlen	ct_pkg::CT_XLEN other than 32 or 64. The configuration message advertises the address width as a single boolean capability bit (ADDR64), so no other value can be described on the wire.
genCoreXlenUndeclared / genCoreXlenMismatch	The integration parameter CORE_XLEN is 0 (undeclared) or disagrees with ct_pkg::CT_XLEN. A 64-bit hart on a 32-bit ingress loses its upper address bits inside the adapter, before the encoder sees anything.
genBadCtxWidth	ct_pkg::CT_CONTEXT_WIDTH outside 1 .. nexus_vendor::NEXUS_MSG_PROCESS_WIDTH (44). A wider context bus would be truncated into the Ownership PROCESS field without a warning.

Guard (generate label)	Rejected configuration
genBadIretireWidth	ct_pkg::CT_IRETIRE_WIDTH outside 2 .. nexus_vendor::NEXUS_MSG_I_CNT_WIDTH (8) while CT_EN_BLOCK_TIP = 1. A single beat that alone reaches the I-CNT pre-drain threshold would wedge the indirect-branch history hold.
genEtrace64	EN_ETRACE = 1 together with ct_pkg::CT_ADDR64. The te_inst address fields are sized by CT_ETRACE_ADDR_BITS = 31 and would truncate silently. A 64-bit E-Trace build is not implemented.

The `CORE_XLEN` and `EN_ETRACE` guards are each present **twice** in the RTL — once as a bare elaboration `$fatal` and once as an initial `$fatal`. The duplication is intentional: the repository's simulation back end runs Verilator through `abc-flow` with `-Wno-fatal`, under which an elaboration `$fatal` is demoted to a warning and the run continues. The initial twin stops the simulation at time 0, before the first TIP beat. A guard that only prints is exactly the state these guards exist to end.

5.7. Instance Attributes

Three parameters are **integration declarations** rather than netlist knobs. They describe the environment the instance is wired into and are advertised on the wire, so that a decoder does not have to be told out of band.

Parameter	Meaning
CT_SIJUMP	The attached core's ingress uses the sequentially-implicit-jump convention. Advertised as configuration-message capability bit CAPS.6. The MicroBlaze™ V adapter sets it to 1; see Chapter 6 .
CT_DEVICE_ID	Device identity of this encoder instance, width <code>nexus_vendor::NEXUS_MSG_DEVID_WIDTH</code> = 32 bit, layout per IEEE-ISTO 5001-2012 Table B-5 (RN PN MID). Default 0 = no device ID assigned. Nothing reaches the wire until software sets <code>trTeControl.SendDeviceId</code> ; the value is sampled only at the emission site.
CORE_XLEN	The architectural address width (RISC-V XLEN) of the hart this instance's TIP is wired to. Not a knob of the netlist — the netlist's own width is <code>ct_pkg::CT_XLEN</code> — but the integrator's declaration of what the adapter feeds in. See the elaboration guards above.
SPLIT_DATA_ACCESS	Selects the data-trace path. 0 = dretire-combined mode (data value on <code>tip.data</code> at retirement). 1 = split-load mode: store data on <code>tip.sdata</code> , load data and response on <code>tip.ldata</code> / <code>tip.lresp</code> . Propagated to the data-flow qualifier and the eTIP composer.

Feature toggles proper (instruction versus data tracing, sync modes, filter and comparator counts, timestamp source, and the entire compression suite) are **not** top-level parameters: they are compile-time switches in `rtl/pkg/ct_pkg.sv` paired — for the compression suite — with runtime enables in the register map. See [Chapter 9](#) and [the register-model chapter](#).

5.8. Pipeline Stages

Stage (module)	Responsibility
ct_L23_preproc rtl/ct_L23_preproc.sv	TIP delay and path alignment, CDC <code>tip_clk</code> → <code>proc_clk</code> , control-/data-flow filtering and range watchpoints, periodic / trace-enable / quota synchronization generation, performance counters, timestamp selection, ACT-CAP / ACT-ST / ACT-Proc instrumentation processing, composition of the eTIP streams (<code>etip_q</code> , <code>next_iaddr_q</code>) and of the AXI-Stream instrumentation output. See Chapter 7 .
ct_L2_msg_gen rtl/ct_L2_msg_gen.sv	N-Trace: generates generic trace messages from the eTIP, accumulates I-CNT and branch history, emits synchronizing, resource-full and correlation messages, and hosts the compression suite. Drives <code>trace_msg</code> . See Chapter 8 and Chapter 9 .
ct_L2_te_inst_gen rtl/ct_L2_te_inst_gen.sv	E-Trace: generates <code>te_inst</code> packets from the same eTIP stream. Drives the packet payload/length/type handshake toward the packetizer.
ct_L2_nexus_formatter rtl/ct_L2_nexus_formatter.sv	Maps a generic trace message to a concrete Nexus message: a TCODE plus an ordered field list of up to <code>nexus_vendor::NEXUS_MAX_FIELDS = 10</code> fields. Drives <code>nexus_msg</code> .
ct_L2_mseo_mdo_formatter rtl/ct_L2_mseo_mdo_formatter.sv	CDC <code>proc_clk</code> → <code>atb_atclk</code> ; serializes the field list into MDO/MSEO chunks with leading-zero suppression and packs them into 32-bit ATB beats. See Chapter 10 .
ct_L2_compact_packer rtl/ct_L2_compact_packer.sv	Alternative single-module egress for CF-only profiles (<code>ct_pkg::CT_COMPACT_PACKER</code>): produces the MDO/MSEO chunks directly from a per-TCODE layout table, bypassing the ten-field array. The wire stream is byte-identical to the historical chain.
ct_L2_te_packetizer rtl/ct_L2_te_packetizer.sv	E-Trace egress: frames <code>te_inst</code> packets onto ATB and crosses into <code>atb_atclk</code> .
ct_L1_funnel (optional) rtl/ct_L1_funnel.sv	Packet-aware merge of several ATB streams into one, without switching mid-packet. See Chapter 11 .

5.9. Backpressure, Ordering and Flush

Backpressure flows strictly upstream through per-stage `ready` handshakes. In the N-Trace chain the nets are `mseo_mdo_formatter_ready` → `nexus_formatter_ready` → `ct_L2_msg_gen`; in the compact-packer configuration the packer drives `nexus_formatter_ready` directly. No stage buffers speculatively: a stage that cannot hand its output on holds it stable and stalls its own input. Message order is therefore preserved by construction, and no message can be dropped by backpressure alone.

Two signals run **against** the data flow and are declared at the top of `ct_encoder`:

- `synq_req_trace_byte_count` and `synq_req_trace_msg_count` — held trace-quota overflow levels reported by the egress module back to the preprocessor's synchronization generator, so that quota-based resynchronization is driven by what actually left the encoder.
- `quota_cnt_clr` — the crossed `SyncCntClr` rearm from the preprocessor back to the egress quota counters.

The `trTeControl.Empty` status is likewise a chain rather than a single flag: the upstream term covers the preprocessor's CDC FIFOs plus whatever the generator still holds back

(`!etip_q.valid && !next_iaddr_q.valid && msg_gen_idle`, respectively `te_gen_idle`), and the ATB tail module appends its own stages before producing the ATB-domain verdict on `cs_atb.trTeEmpty`. An integrator polling `Empty` therefore observes the whole pipeline, not one stage of it.

Flush is a side-channel, not a message class of its own: on `atb_afvalid` the preprocessor stops feeding the TIP FIFO and raises the TIP side-channel `do_flush`, which propagates through the downstream stages so the ATB receiver can align to a packet boundary. The idle-core case – a flush request with no further `tip.iretire` – is covered by the eTIP composer's per-cycle flush path (`rtl/ct_L23_preproc.sv` module header).

5.10. Global Determinism Guarantees

All CTTE modules operate deterministically with respect to the TIP input stream and the configuration register state. No arbitration mechanism depends on metastable, timing-dependent or random conditions. Given identical RTL configuration and identical TIP stimulus, the generated trace stream is cycle-accurately reproducible. This property is what makes trace-based safety validation and post-mortem analysis possible at all, and it is enforced by four concrete design rules:

1. **Retire-qualified emission.** An output that represents an architectural event is emitted only on a cycle that retires an instruction or a retired data access, so message boundaries line up with the core's architectural stream.
2. **Fixed base latency.** Every preprocessing block has a fixed intrinsic latency, and reports it, so that the encoder-wide alignment can be computed rather than tuned.
3. **Selectable alignment latency.** Parallel preprocessing paths are aligned through a common `extra_delay` shift-pipeline pattern rather than by re-timing internal logic. `ct_L23_preproc` computes the maximum path delay across all blocks and derives each block's `extra_delay` from it; the bound is `ct_pkg : EXTRA_DELAY_MAX (= PREPROC_DELAY_MAX, 44 when CT_EN_ACT = 1)`. See [Chapter 7](#).
4. **Order-preserving backpressure.** Stalls delay emission but never reorder or drop events (see above).

Determinism is a property of the encoder given its inputs. It does not extend across a change of configuration mid-stream, and it does not claim anything about the **core's** determinism: if the hart retires a different instruction sequence, the trace differs accordingly.

5.11. What This Chapter Does Not Cover

- The signal-level contract of TIP, ATB, AXI-Stream, Wishbone and the CDC requirements – see [the interface chapter](#).
- The register map, capability (CAPS) and enable (ENAB) bits and their reset values – see [the register-model chapter](#), generated from `rdl/ct_cs_cpuiif.rdl`.
- The on-wire message format and the TCODE catalog – see [the trace-format chapter](#) and [the](#)

[message catalog](#).

Chapter 6. Trace Sources and Core Adapters

6.1. The Trace Ingress Port as the Contract

CTTE supports any RISC-V hart that presents a compliant Trace Ingress Port (TIP). Integration is independent of the microarchitecture as long as the hart supplies the required instruction-trace signals – and, where data tracing is wanted, the data-trace signals – on the TIP. The signal-level contract is stated in [the interface chapter](#) of this manual and defined in `rtl/pkg/tip_if.sv` and `rtl/pkg/tip_pkg.sv`; this chapter is about what it takes to **drive** that port from a given core.

Most cores do not present a TIP directly. Between the core's own trace or retirement interface and the TIP sits an **ingress adapter**, and the adapter is where the interesting work is: the core's interface is usually a truthful description of that core's pipeline rather than of the abstract retirement stream a trace encoder needs.

The adapters shipped with CTTE live under `rtl/adapters/`, and each one has a **specification-level** document under `doc/adapters/`. Those documents are deliberately truth tables – empirically measured signal semantics of a specific core and tool version – rather than architecture prose. That distinction matters for an integrator: a rule marked as measured holds for the named version and needs re-measuring when the version changes.

6.1.1. The `itype` Vocabulary

The TIP carries a 4-bit instruction type per retirement beat (`tip_itype_e`, `rtl/pkg/tip_pkg.sv`), which is the full E-Trace vocabulary:

Code	Meaning	Code	Meaning
0	OTHER	8	UNINFERABLE_CALL
1	EXCEPTION_TRAP	9	INFERRABLE_CALL
2	INTERRUPT	10	UNINFERABLE_TAIL_CALL
3	EXCEPTION_IR (trap return)	11	INFERRABLE_TAIL_CALL
4	NOT_TAKEN_BRANCH	12	CO_ROUTINE_SWAP
5	TAKEN_BRANCH	13	RETURN
6	UNINFERABLE_JUMP (only populated at a 3-bit <code>itype</code> width)	14	OTHER_UNINFERABLE_JUMP
7	reserved	15	OTHER_INFERRABLE_JUMP

A core that exports only the 3-bit subset (codes 0..6) is usable, but the encoder then cannot distinguish calls from returns, and the implicit-return compression ([Chapter 9](#)) has nothing to work with. Two of the adapters therefore **reconstruct** the missing codes from the instruction word; see the CVA6 and MicroBlaze™ V sections below.

The exception cause enum `tip_ecause_e` follows the RISC-V `mcause` exception codes. Its "no

exception" sentinel deliberately sits at code 14 — a code the architecture reserves — rather than at 15, which is a real cause (store/AMO page fault). On a bare-metal RV32 build without paging the collision was invisible; under Sv39 Linux it is one of the most frequent traps there is, and a waveform, a filter bitmap or a dump reader would have read the most common store trap as "no exception".

6.1.2. Four Rules Every Adapter Has to Get Right

The three shipped adapters were each characterized by measurement, and the same four issues came up every time. They are stated here once, because they are what an integrator writing a **new** adapter will meet.

1. A trap beat must not retire. Several cores mark the faulting instruction as retired on their trace interface. The encoder counts instruction halfwords on `iretire`, so a trap beat with `iretire = 1` makes the faulting instruction count towards the instruction count and shifts the decoder. Every adapter therefore **forces** `iretire = 0` on exception and interrupt beats.

2. `itype` must only be driven on a real event. The encoder evaluates `itype` for control flow **without** gating it on `iretire` — `iretire` only steers the halfword count. A core whose trace bus holds the last instruction's classification through idle cycles therefore produces **ghost** control-flow events. This is not cosmetic: it produces extra indirect-branch messages, the message stream derails, and the decoder aborts. The adapter rule is: on a trap indication, drive the trap `itype`; otherwise, on a valid retirement, drive the decoded `itype`; otherwise drive `OTHER`.

3. `ilastsize` is the halfword count of the last instruction, and it is what the encoder's instruction count advances by. Where a core's trace bus carries the raw instruction word, the value follows directly from the low two bits of that word (compressed versus 32-bit encoding) and needs no extra source.

4. The adapter is an observer and never applies backpressure. No shipped adapter can stall its core. If the encoder cannot keep up, the loss is made visible through the encoder's own error and resynchronization path ([Chapter 10](#)), not by holding up the core.

A fifth item is a **declaration** rather than a rule: the address width of the attached hart must be declared at the encoder instantiation (`CORE_XLEN`, [Chapter 5](#)). A 64-bit hart wired into a 32-bit ingress loses its upper address bits inside the adapter, before the encoder sees anything, and the resulting stream is well-formed and wrong. The declaration is the only place where both widths are known at once.

6.1.3. What Each Core's Ingress Can Carry

The four cores documented in this chapter do not offer the encoder the same ingress, and the difference stays invisible until a feature that needs the data channel silently produces nothing. The table states it per core, read from the adapters rather than inferred ([examples/kv260/README.md](#)):

Core	Adapter	Data channel (dretire/dtype/ daddr/sdata)	CSR writes visible	ACT-ST	ACT-CAP
MINRES TGC5B	examples/kv260/common/tgc5b/rtl/ct_tip_adapter.sv	yes – from the H2E port	no	yes	the only core on which it is reachable at all
CVA6	rtl/adapters/cva6/cva6_i ti_to_ctte_tip.sv	no – dretire tied to 0	no	yes	no
Rocket	rtl/adapters/rocket/rocket_tci_to_ctte_tip.sv	no – dretire tied to 0	no	yes	no
AMD MicroBlaze™ V	rtl/adapters/amd_microblaze_v/mbv_to_ctte_tip.sv	no – dretire tied to 0	no	yes	no

Three consequences follow, and they are worth keeping apart.

ACT-ST works everywhere. It triggers on a retired instruction address and needs nothing beyond `iaddr`, which every one of the four adapters delivers.

Everything that needs the data channel works on the TGC5B only. That covers data trace, the data-flow range filter and the `DAQ_DATA`, `DAQ_DADDR` and `DAQ_DATA_DADDR` commands. On the other three cores those commands are not less precise – they carry no useful payload at all, because the encoder never receives a data beat.

No shipped core reports CSR writes, which is what ACT-CAP decodes (`dretire && dtype == CSR_READ_WRITE && daddr == 0x0B10`, `rtl/preproc/ct_L23_preproc_act_cap.sv`). The reason differs between the TGC5B and the rest: the TGC5B does decode the vendor CSR window `0xB10–0xB9F` and raises no exception for it, but its H2E port derives `dtype` from the data bus alone (0 = load, 1 = store), so the write never reaches the trace port; on CVA6, Rocket and MicroBlaze™ V there is no store beat at all. For CVA6 in particular, RVFI is not an escape route: the wrapper ties its RVFI outputs off, and the CVA6 RVFI CSR structure is a fixed list of named architectural CSRs with no field for an arbitrary address/value pair.

Because the TGC5B adapter does see stores, an integration can close the gap locally: a store to a chosen doorbell address is converted into the CSR beat the encoder expects. That is an adapter-level change and needs no core change;

`examples/kv260/tgc5b2_rvcfi/rtl/ct_tip_adapter_actcap.sv` builds exactly this, together with the doorbell in `examples/kv260/tgc5b2_rvcfi/rtl/ct_soc_doorbell.sv`.

IMPORTANT

On the documented cores' **native** ingress ACT-CAP has consequently no silicon history; there its coverage comes from `tests/lib/cpu_model.sv`, which synthesizes the beat directly (scenario `sim-hsi-csr-cap`). The doorbell route above has been exercised on silicon: on 2026-08-26 the `tgc5b2_rvcfi` testbed ran doorbell-converted ACT-CAP commands on the KV260 with board verdicts ([Chapter 28](#), and the details in [Chapter 20](#)).

6.2. MINRES TGC

The MINRES TGC family (TGC5B in the shipped examples) drives the TIP natively – it was developed with a trace ingress port and needs no adapter.

The shared TGC5B building blocks live in `examples/kv260/common/tgc5b/`. That directory is not an example but the library eight KV260 examples build on: the vendored core, the AXI-Lite-to-Wishbone bridge, SoC RAM, CLINT and interrupt controller with their SystemRDL sources, and the `hello_trace` program. It carries its own SoC bench, which is the entry point that needs no board (`make sim-tgc5b-soc`). A two-core variant with the watchpoint and DAQ testbed is `examples/kv260/tgc5b2_axis_wp/`, board-gated on 2026-08-17 with 851 of 851 watchpoint records per core on the KV260 and a monotone cross-core merge. TGC5B is also one of the two cores in the `duo` funnel demonstrator.

NOTE

Some integration collateral for the TGC core is exchanged through partner and internal channels and is therefore not publicly linked from this documentation.

6.3. Fraunhofer IPMS EMSA5

The EMSA5 core provides the instruction-trace ingress signals directly, plus the optional data-trace signals when data tracing is enabled in the system. The signal set, timing requirements and configuration rules are those of the TIP contract and apply unchanged; see [the interface chapter](#).

NOTE

Some EMSA5 integration collateral and the corresponding internal repository links are not publicly accessible.

6.4. CV32E4

The CV32E4 core family is **not** supported at present. The reason is not a property of the encoder: no TIP-compatible instruction-trace interface instance is currently available for it that meets the ingress requirements. Should such an instance become available, the integration follows the same pattern as the others – either a native TIP or a thin adapter.

6.5. CVA6

CVA6 is supported through the ingress adapter `rtl/adapters/cva6/`, and it is the most thoroughly characterized of the three: the CVA6 Instruction Trace Interface (ITI) truth table in `doc/adapters/cva6 iti semantics.adoc` covers RV32, RV32 with Sv32 and S-mode, and RV64.

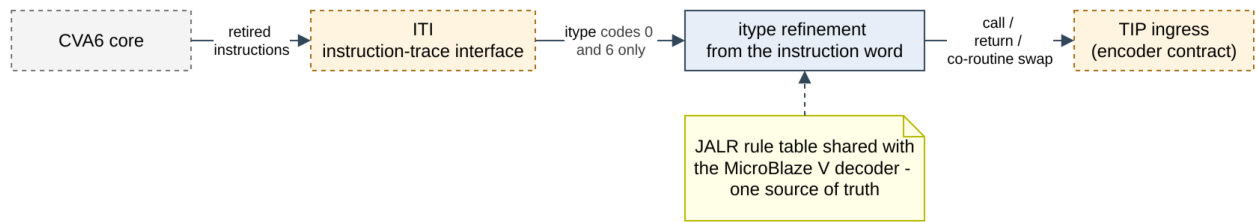


Figure 6. CVA6 adapter (ITI)

The core exports only `itype` codes 0 and 6; the adapter reconstructs calls and returns from the instruction word.

The JALR rule table is shared with the MicroBlaze™ V decoder.

6.5.1. Maturity

CVA6 is not a prospect. It runs.

Evidence	Result
Board run, KV260, 2026-08-18 (<code>c-trace</code> commit 2664bb5c, demonstrator pin <code>cva6_linux_ctrace_kv260</code> , bitstream 4dec7b54)	OpenSBI and Linux boot with tracing active; the captured stream decodes as 4 619 090 instructions with 0 error messages over the trace-encoder port. Raw artefact: <code>decode.log</code> of the run directory <code>examples/kv260/cva6_linux/board/board_run_cva6_linux/</code> (local to the measurement host – run directories are excluded from the published tree) – 890 412 bytes, 117 543 messages, 7.58 bytes per message, 1.542 bits per instruction.
Characterization RV32 (<code>cv32a60x</code> , single retirement, 2026-07-24)	68 ITI beats against 68 RVFI retirements, 1:1.
Characterization RV32 + Sv32 + S-mode (<code>cv32a6_ima_sv32_fpga</code> , 2026-07-26)	104 retirements against 104 ITI beats, 1:1, including page faults.
Characterization RV64 (<code>cv64a6_imac_sv39_ctrace</code> , 2026-08-03)	106 retirements against 106 ITI beats, 1:1.

Further CVA6 material in the example tree: `examples/kv260/cva6_linux64/` (RV64 Linux) and `examples/kv260/cva6_2/` (an asymmetric double-CVA6 pair; bitstream flow migrated, board leg pending).

6.5.2. Measured ITI Beat Semantics

One ITI beat per event; `iretire` is constant 1 and is an **instruction flag**, not a halfword count – the halfword count lives in `ilastsize`, which is exactly the CTTE contract (the instruction count advances by `1 << ilastsize` per beat with `iretire = 1`). Idle cycles carry no beat.

Event	ITI itype	Note
ALU, load, store, <code>jal</code> , CSR write, ...	0 STANDARD	<code>jal</code> is not classified as a call – the ITI's detector only marks <code>jalr</code> as an uninferable discontinuity.

Event	ITI itype	Note
mret	3 ERET	—
Branch not taken	4 NON_TAKEN_BR	—
Branch taken	5 TAKEN_BR	—
jalr (call or return)	6 UNINF_JMP	Calls, returns and co-routine swaps are indistinguishable at the ITI.
ecall	1 EXC	Beat carries <code>iretire</code> = 1 — clamped by the adapter.
Illegal instruction	1 EXC	Beat carries <code>iretire</code> = 1; <code>tval</code> is 0 on RV32 and the instruction word on RV64.
Timer interrupt	2 INT	Beat carries <code>iretire</code> = 1; <code>iaddr</code> is the interrupted instruction.

Under Sv32 with S-mode, three additional cases were measured and matter for a kernel trace:

- an **instruction** page fault beat carries the **target** virtual address, not the address of the jump that triggered it. That is the right information for a trace — execution would have continued there — but a consumer that reads `iaddr` as "PC of the retiring instruction" must know the case. The adapter forces `iretire` = 0, so the never-executed instruction never enters the instruction count.
- `tval` carries the faulting virtual address on load, store and instruction page faults. It never reaches the N-Trace wire; in the E-Trace format it is visible, and it is the most valuable addition for a kernel trace.
- the privilege change from M-mode to S-mode is visible in the beat and is passed through. The context or address-space identifier is a separate, optional path (see below).

On RV64 the beat semantics are identical. Addresses and `tval` arrive as full 64-bit values. Not exercised: program counters with bit 31 or 63 set — they are not representable in the measurement window, so the ITI's sign-extension behaviour above 2^{31} remains an open point.

6.5.3. Adapter Behaviour

`cva6_iti_to_ctte_tip` is a thin shim, because the ITI vocabulary is the E-Trace chapter-4 vocabulary that the TIP also speaks: the 3-bit codes 0..6 are value-identical to the lower TIP codes and zero extension is the correct mapping. What the shim adds are the convention adjustments:

1. Trap beats drive `iretire` = 0 (rule 1 above).
2. Idle cycles drive `itype` = OTHER and `iretire` = 0 (rule 2 above).
3. The exception cause carries the exception cause on an exception beat and the interrupt cause on an interrupt beat, both without the `mcause` most significant bit; otherwise the "no exception" sentinel.
4. Every unused TIP signal is explicitly tied off, so no undefined value can propagate into the encoder.

Width. The address ports follow the encoder netlist (`ct_pkg : CT_XLEN`); the `ITI_XLEN` parameter names the width of the **core** side and is checked against it at elaboration, for the reason given in rule 5 above.

Context (optional). `ITI_CONTEXT_WIDTH` defaults to 0, which is exactly the previous behaviour: the context is fixed at 0 and the encoder emits no ownership message. When a width is named, the context input carries the core's context identifier – for RISC-V, `satp.ASID` – and the shim produces what the ownership contract needs.

6.5.4. itype Refinement

The ITI's coarse classification is a limit of the **interface**, not of the adapter. Since 2026-08-09 the module `cva6_riscv_itype_refine` reconstructs the missing 4-bit codes from the instruction word, which the trace wrapper delivers cycle-exact with the ITI beat (both are registered from the same pipeline entry in the same clocked block). No modification of the vendored CVA6 tree is required for this.

Aspect	Detail
Switch	<code>ITI_ITYPE_REFINE</code> , default 0 = the previous behaviour, bit-identical.
Port	The instruction word input defaults to zero, and a runtime assertion fires if it is left unconnected while refinement is on – an all-zero word is not a legal RISC-V encoding, and a forgotten connection would otherwise be the most expensive kind of silent failure: folding would simply keep doing nothing.
Rules	Only ITI codes 0 (where <code>jal</code> hides) and 6 (the catch-all for every <code>jalr</code>) are refined, following the E-Trace JALR table: <code>jal</code> with a link destination becomes an inferable call; <code>jalr</code> with a link destination and a non-link source becomes an uninferable call; both link with equal registers, an uninferable call; both link with different registers, a co-routine swap; destination <code>x0</code> with a link source, a return. The rule table is shared with the MicroBlaze™ V decoder – deliberately one source of truth, not two.
Link set	<code>ITI_LINK_REG_MASK</code> , default <code>{x1, x5}</code> . Clamped to <code>{x1}</code> where decoder conformance is required: the reference converter knows only <code>x1</code> as a return base, and eight <code>jr t0</code> divergences were measured across a complete OpenSBI-plus-Linux image.

Two operating pitfalls belong with this feature, both measured:

- A disassembly listing generated with ABI register names makes the reference converter read **every** return as a call (its number parser accepts only `xNN` form) – measured as 17 calls and 0 returns on a call-dense test. Listings for implicit-return decoding must be generated with numeric register names and no aliases.
- The reset default of the periodic synchronization maximum forces a full synchronization every 16 instructions and chokes call-dense runs. Implicit-return measurements run at a larger value.

The end-to-end effect on a deliberately call-dense program (24 413 instructions, 3 200 calls and 3 200 returns) is worth quoting **with** its caveat:

Configuration	ATB bytes	Messages	Bits per instruction
Refinement off, implicit return on (baseline)	27 560	3 433	9.031
Refinement on, implicit return on	2 092	248	0.686
Refinement on, implicit return off	27 560	3 433	9.031

The decoded PC sequence is identical in all three legs. Refinement **alone** changes nothing – the first and third legs are byte-identical – and only refinement together with implicit return produces the reduction. On this program that is 92.4 % fewer bytes, but the program is one call or return every 7.6 instructions and is **not** a general ratio: on a real board window of 883 577 instructions the same measurement gives 5.25 %.

6.5.5. Upstream Defects

Two defects in the pinned CVA6 tree had to be patched for the ITI to be usable at all, and both are candidates for upstream. Neither file ships with this repository: the reference core is pinned by commit and fetched (`examples/kv260/third_party/fetch.sh`, pin in `CVA6_PIN.md`), and the patches are applied to the fetched tree. The files are named here rather than the patch series' delta identifiers, because those identifiers are not unique – two of them denote different changes in the list and in the patch text, and the repository's own pin document asks readers to cite the file.

File in the pinned tree	Defect
<code>core/cva6_rvfi.sv</code>	The ITI valid signal is tied to the commit acknowledge, so exceptions and interrupts never reach the ITI at all – zero trap beats.
<code>corev_apu/instr_tracing/ITI/cva6_iti/itype_detector.sv</code>	The exception condition is checked before the interrupt condition while the exception-valid signal is also asserted on interrupts, so every interrupt is classified as an exception.

Two further configuration defects surfaced during the Sv32 gate, both previously invisible because the smaller configuration generates the MMU away: a direction-reversed subrange in the MMU at a 32-bit width, and an FPGA configuration whose address regions pointed at a different board so that the test memory was not executable.

6.6. MicroBlaze™ V

The AMD MicroBlaze™ V soft core is supported through the ingress adapter `rtl/adapters/amd_microblaze_v/`, which consumes the core's TRACE bus. It is the core of the `examples/kv260/mbv/` demonstrator and one of the two cores in the duo funnel demonstrator.

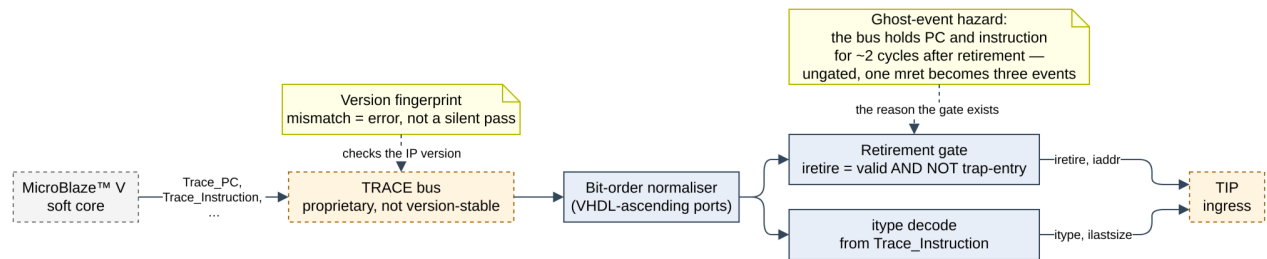


Figure 7. MicroBlaze™ V adapter (TRACE bus)

The TRACE bus is proprietary and not version-stable; every statement holds for the named IP and tool version only.

Ghost events: the bus holds PC and instruction for about two cycles after retirement. Without the gate, one `mret` is reported as three events and the decoder aborts.

Port names are the vendor's; the semantics beside them are Accemic measurements.

IMPORTANT

The TRACE bus is a proprietary, non-standardized and **not version-stable** interface. Every statement below holds for the named IP and tool version only, and must be re-measured on a version change. The adapter carries a version fingerprint; a mismatch at elaboration or run time is an error, not a silent pass.

NOTE

Where the information in this section comes from. The port names and widths are those of the vendor's own interface; the authoritative description is AMD's MicroBlaze™ V product guide, which this manual references rather than reproduces. Everything under *Measured Semantics* below is Accemic's own measurement on the delivered IP — behaviour the vendor documentation does not state — and is what the adapter had to be built against.

6.6.1. Verified Configuration

Item	Value
IP	xilinx.com:ip:microblaze_riscv:1.0, Vivado™ 2026.1, CONFIG.C_TRACE = 1
Part	Kria™ KR260, xck26-sfvc784-2LV-c
Core	RV32, C extension off, M extension on, machine mode, MMU off, trace bus on, the core's own native trace encoder off
Adapter	32-bit addresses and data, no compressed instructions, no data trace
Encoder	One-instruction-per-beat ingress, 32-bit addresses, history mode, combined data path, ATB output, reference decoder

Board status: gated on the KV260 on 2026-07-21/22 with 7 of 7 programs green on silicon — trace, branch, sequentially-inferable jump, trap, illegal instruction and misaligned access each with an exact reference prefix of 26.6 to 26.8 thousand program counters, plus an interrupt test with a hardware interrupt pulse generator. Re-gated from the consolidated example tree on 2026-08-17 with 26 772

of 26 772 program counters prefix-identical.

6.6.2. Confirmed Port List

The port list below was confirmed by block-design introspection on 2026-07-15 and **corrects** an earlier integration report that had been written from documentation.

TRACE port	Width	TIP field	Note
Trace_PC	32	iaddr	All multi-bit TRACE ports are VHDL-ascending, so the adapter's normalizer establishes the bit mapping explicitly.
Trace_Instruction	32	decoded into itype	Also the source of ilastsize for compressed instructions.
Trace_Valid_Instr	1	iretire, plus the trap rules	—
Trace_Jump_Taken	1	conditional-branch outcome	—
Trace_Exception_Taken	1	trap-entry flag	—
Trace_Exception_Kind	6	ecause	Bit 5 distinguishes an interrupt (1) from a synchronous exception (0); bits 4..0 carry the RISC-V cause directly. Measured: ecall = 0x0b, ebreak = 0x03, machine external interrupt = 0x2b.
Trace_OF_PipeRun, Trace_EX_PipeRun, Trace_MEM_PipeRun	1 each	—	Pipeline-stage run indicators. Diagnostic only.
Trace_Halted	1	—	Core halted for debug.

Two ports that the earlier report assumed do **not** exist on this IP version: a privilege-mode output and a process-identifier output. Consequently the privilege field is a constant machine mode and the context is 0 in this configuration. Should privilege or context tracing be needed, another source has to be found — instruction-decode of CSR accesses, or an IP configuration option.

The data-flow ports (Trace_Data_Access, Trace_Data_Address, Trace_Data_Read /_Write, Trace_Data_Write_Value, Trace_Data_Byte_Enable, Trace_Reg_Write, Trace_Reg_Addr, Trace_New_Reg_Value) exist but are not part of the supported configuration: the store value is directly derivable, the load value is not — the register-write port is ambiguous, so a full load trace would need a bus monitor.

6.6.3. Measured Semantics

The retirement and trap truth table was established in eight measurement rounds. Four findings shape the adapter:

Retirement and traps. iretire is the valid-instruction flag minus trap entries. A trap entry maps to an interrupt or an exception according to bit 5 of the cause port and carries iretire = 0; mret and sret map to a trap return and do retire. Both branches are confirmed across rounds covering six causes and 69 interrupt events.

Pipeline-run signals are not retirement signals. In a full-cycle measurement the operand-fetch run indicator was low for 26.8 % of cycles (295 single-cycle fetch stalls), while the execute and memory indicators stayed constant. Decisively, a valid instruction was signalled 68 times **while** the fetch indicator was low. These signals are usable as a debug sideband and must never enter the retirement path.

Trap entry never collides with a taken branch. In 69 interrupt events, the trap flag and the jump-taken flag never occurred together – a preempted branch does not resolve. The adapter still implements the priority defensively, but it is uncontested in hardware.

The trace bus holds its value after a retirement – the ghost-event finding. Measured at an `mret`: the program counter and the instruction word stayed stable for about two further cycles after the retirement while the valid flag was already low. Those cycles describe nothing. Because the encoder evaluates `itype` without an `iretire` gate, an ungated adapter reports one `mret` as **three** events, the message stream derails and the decoder aborts. This is rule 2 above, and it is where the rule came from.

NOTE

The adapter's own unit testbench had let this defect through. Its idle-cycle vector used a `nop`, which decodes to `OTHER` anyway – green without checking anything. An idle-cycle vector needs a **control-flow** word sitting on the bus, which is the constellation that actually occurs.

6.6.4. `itype` Decoding

The classification priority is: reset or idle, then trap entry, then trap return (`mret/sret` by instruction decode), then conditional branch (taken or not, from the jump-taken flag), then `jal`, then `jalr`, then everything else. The `jal` and `jalr` rules follow the standard link-register conventions – a `jal` with a link destination is an inferable call, a `jalr` resolves to an uninferable call, a return or a co-routine swap depending on the destination and source register relationship.

The non-trap rules are captured as a reference vector set:

`doc/adapters/itype_decoder_vectors.csv` is the source data for the decoder testbench's vector file. It covers all six conditional branches in both outcomes, `jal` with destinations `x0`, `x1`, `x5` and `x8`, the full `jalr` matrix including the co-routine-swap case, `mret` and `sret`, and two non-control instructions – 32 vectors in all, each naming the encoding, the expected `itype` and the expected `ilastsize`.

`ilastsize` is constant in the supported configuration (no compressed instructions). For a build with compressed instructions the rule was measured and is available directly from the instruction word's low two bits – the trace bus carries the **original** compressed encoding, which was the open risk of that gate and is now resolved.

6.6.5. Sequentially Inferable Jumps

The adapter implements the sequentially-inferable-jump idea as an optional promotion: a `jalr` whose target is statically computable from an immediately preceding `auipc` or `lui` is folded, and

the pair is detected in the retirement stream. The switch is `sijump_en`, default 0 = the previous behaviour, byte-identical. It is a **paired** convention: the static side of the decoder must be told to pair the same way, and the encoder advertises the convention in its configuration message through the `CT_SIJUMP` instance parameter (Chapter 5). The committed demonstrator configuration runs with the promotion off.

6.7. Rocket

The Rocket core is supported through the ingress adapter `rtl/adapters/rocket/`, which consumes the `rocket-chip TraceCoreInterface`. Example designs are `examples/kv260/rocket_linux/` and `examples/kv260/rocket2/` (RV64, two-hart SMP Linux); both have their bitstream flows migrated with the board leg still pending.

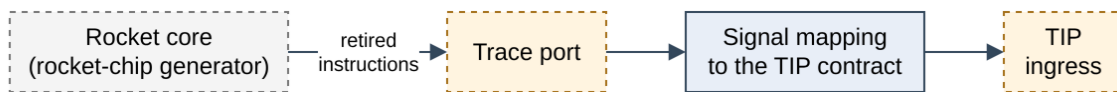


Figure 8. Rocket adapter (TCI semantics)

The simplest of the three adapters: signal mapping only, no reconstruction.

The truth table in `doc/adapters/rocket_tci_semantics.adoc` was measured against the generated netlist standalone – without the encoder and without the adapter – so that the interface’s behaviour and the adapter’s compensation for it stay separable.

6.7.1. Measured Semantics

The interface is a combinational tap of the writeback stage with one retirement per cycle. Three properties are decisive:

Trap beats retire. `iretire` is tied to the writeback valid signal, which includes the exception case, so all trap beats carry `iretire = 1`. The adapter’s clamp to 0 is therefore mandatory.

Trap returns are classified as ordinary returns. Every `mret` beat carries the return code rather than the trap-return code, and that code appears **only** at `mret` program counters – `jalr` returns carry a different code. The adapter’s remap is therefore unambiguous and lossless, and it is enabled by default.

Idle cycles are not neutral. The classification holds the decode of the last instruction while `iretire` is low. In a 432-cycle characterization run this produced 161 hanging classification lines – 37 taken branches, 71 uninferable and 53 inferable jumps. A consumer **must** gate the classification with `iretire`; Rocket’s own encoder chain does exactly that internally. Not a single hanging beat carried a trap classification, so the clamp swallows no traps.

Further measured properties: the privilege field is constant machine mode in the characterized configuration, with the debug flag not exercised; the time field is the free-running CSR cycle counter (measured ratio exactly 1.0 against the cycle count), **not** the platform timer, which ticks a hundred

times slower; and there is no context port at all on this netlist – the address-space-identifier path that an earlier report assumed does not exist at this pin’s interface export.

`ilastsize` distinguishes compressed from 32-bit encodings and was measured against the disassembly’s encoding length over 10 compressed and 94 full-width retirements with zero errors.

6.7.2. Upstream Defects

#	Defect	Consequence
U1	Trap beats carry <code>iretire = 1</code> .	The adapter’s clamp is mandatory; without it the encoder counts the faulting instruction.
U2	A trap return is classified as an ordinary return rather than as a trap return.	The adapter’s remap is unambiguous and lossless, and covers <code>sret</code> and <code>dret</code> as well, because the underlying condition is the CSR decode of any trap-return instruction.
U3	The trace cause is taken from a memory-stage multiplexer whenever the writeback exception flag is low. For traps the CSR raises internally – <code>ecall</code> and <code>ebreak</code> – the beat therefore carries a data-path leftover value instead of the architectural cause. Illegal instructions (correct cause and the instruction word in <code>tval</code>) and interrupts are unaffected.	The cause field is not trustworthy for <code>ecall</code> and <code>ebreak</code> , and the adapter cannot fix it: the interface carries no discriminating information. The N-Trace wire is unaffected, since the cause never leaves the encoder; a consumer of the E-Trace trap format must know the case.

The defect was confirmed structurally rather than only observationally: the CSR raises those two traps internally, and the value list of the trace-cause multiplexer does not contain the architectural codes at all – the defect sits exclusively in the trace path.

6.7.3. Adapter Behaviour

The Rocket adapter maps the interface’s 4-bit instruction type to the TIP type **one to one**; the TIP enum already carries the full E-Trace vocabulary that the interface follows. Only two clamps are active: trap beats forced to non-retiring, and the trap-return remap. The 3-bit down-mapping that an early design note proposed does not apply – it targeted the 3-bit ITI vocabulary, not the 4-bit interface.

The characterization run had to redirect the platform’s boot ROM, because the reset vector jumps into an SD-card bootloader that is a dead path in simulation. This is done by compiling a work-local netlist copy with one patched ROM word, which is simulator-independent; a conditional force on the ROM data bus did not take effect under the simulator used. The vendored file itself stays untouched, and the same patched image is the oracle basis.

6.8. Multi-Core and Mixed-Protocol Integration

Several harts share one trace port through the funnel ([Chapter 11](#)). The shipped demonstrators are:

Example	Cores	Status
examples/kv260/duo/	MicroBlaze™ V + TGC5B	Board-gated 2026-08-18: both streams decoded on the KV260, WNS +2.188 ns.
examples/kv260/trio/	MicroBlaze™ V + TGC5B + CVA6	Adds the per-instance protocol mix — a Nexus stream next to an E-Trace reference-raw stream in one merged container, with the demultiplexing tool vendored. Board leg pending.
examples/kv260/cva6_2/	Two CVA6 harts (RV32 and RV64 scenarios)	Bitstream flow migrated, board leg pending.
examples/kv260/rocket2/	Two Rocket harts, SMP Linux	Bitstream flow migrated, board leg pending.

The `trio` design is also where the funnel's per-channel arbitration priority becomes software-visible, through a SoC-level register ([Chapter 11](#)).

6.9. Peripherals

6.9.1. Bus Trace

Not implemented. References for the intended scope:

- AMBA™ AHB Trace Macrocell (HTM) Technical Reference Manual, revision r0p4 (Arm DDI0328E)
- Gaisler GRLIB IP Core User's Manual (GRIP): AHB trace buffer (p. 94)

6.9.2. DDR4 Trace

Not implemented. The intended functional scope is: transaction address; transaction attributes (direction, speculative read, read-modify-write, row state); forced page closings due to bank and sub-bank collisions, counted in total and excluding refresh-caused closings; and refresh events.

Chapter 7. Preprocessor Stage

7.1. Overview

The layer 2/3 preprocessor (`ct_L23_preproc`, `rtl/ct_L23_preproc.sv`) is the front end of the encoder. It runs in the `tip_clk` domain, observes the core's retirement stream, and turns it into the internal **extended TIP** (eTIP) event stream plus the AXI-Stream instrumentation output. Both trace back ends – N-Trace and E-Trace – consume the same eTIP stream ([Chapter 5](#)), so everything in this chapter is protocol-agnostic.

Its tasks are:

- observe the TIP retire events (`tip.iretire`, `tip.dretire`) and the associated context: program counter, data address, data, privilege and trap state;
- derive the **next** program counter for control-flow discontinuities;
- qualify retire cycles into control-flow (CF) and data-flow (DF) hits through a comparator/filter engine and address range searches;
- generate periodic, event-driven and requested **synchronization** reasons;
- pre-aggregate performance statistics;
- decode and synthesize instrumentation commands (ACT-CAP, ACT-ST, ACT-Proc);
- select and provide the timestamp;
- compose the time-aligned CF/DF/DAQ sub-messages into the eTIP stream and cross it into `proc_clk`.

All blocks are fully synchronous to `tip_clk`. State transitions occur exclusively on its rising edge; no internally derived clocks are used. Where another clock is involved – wall-clock counting, the crossing into `proc_clk`, or the write side of a configuration memory – explicit CDC modules and dual-clock FIFOs are instantiated.

7.2. Determinism and Path Alignment

Determinism is guaranteed for a given TIP input sequence and configuration register state: identical inputs produce identical outputs, cycle-accurately. There is no hidden state, no randomization and no timing-dependent arbitration. Three design rules carry this property:

Retire-qualified emission. Anything that represents an architectural event is emitted only on a cycle that retires an instruction or a retired data access.

Reported fixed latency. Every block reports its intrinsic latency on an `internal_delay` output rather than documenting it in prose. Most blocks report one cycle (the register stage that captures the event); the blocks that contain a binary search report `1 + vbs_delay`, and ACT-ST reports 2, including its match checker.

Computed alignment. `ct_L23_preproc` collects all `internal_delay` values, computes the maximum path delay, and derives each block's `extra_delay` from it. Each block implements the same pattern – a shift pipeline of depth `EXTRA_DELAY_MAX` whose output is tapped at `extra_delay` – so an integrator changes alignment without re-timing any internal logic and without re-synthesis. The bound is `ct_pkg::EXTRA_DELAY_MAX = ct_pkg::PREPROC_DELAY_MAX`, which is 44 with the instrumentation feature compiled in (`CT_EN_ACT = 1`) and smaller otherwise.

There is a consequence of this alignment that is easy to get wrong, and the RTL calls it out explicitly: **a control level must be sampled with the same delay as the beats it gates.** The trace-enable levels are therefore staged through their own shift pipelines (`ItaPipe` for `trTeEnable` & `trTeInstTracing`, `DtaPipe` for `trTeDataControl.DataTracing`) and tapped at the delay of the TIP copy that the consuming stage sees, instead of being read live from the CSR interface. Qualifying delayed beats with the live level breaks in both directions, and both were measured on 2026-08-04 in `tests/data/03_addr_compress`:

- at the **off** edge, an access that retired up to `max_delay - 1` cycles **before** the edge is silently dropped – a traced access lost with no error indication;
- at the **on** edge, an access that retired **before** the edge is emitted although data tracing was off at its retirement – a data-filter violation, and worse, it becomes the re-anchor point that the decoder subsequently computes differential addresses against.

Both directions are gated by that test. Where a build profile compiles data tracing out, the staging registers are hard-gated to a constant rather than left for synthesis to trim.

7.3. Next Program Counter Evaluation

Many control-flow messages need an explicit target address. The TIP input carries the address of the **retired** instruction (`iaddr`) and does not always carry the target of a taken control-flow change, so the preprocessor derives the next PC:

Case	Next PC
Sequential retirement	<code>iaddr + ilastsize</code> , where <code>ilastsize</code> counts halfwords: 1 for a 16-bit instruction, 2 for a 32-bit instruction.
Direct PC-relative jump or call	Computable from the instruction encoding (offset immediate) and <code>iaddr</code> .
Not-taken conditional branch	<code>iaddr + ilastsize</code> .
Taken conditional branch, indirect jump, return	Not known at retirement. The next retired <code>iaddr</code> is the effective target; the preprocessor captures it as <code>next_iaddr</code> and forwards it on the companion stream <code>next_iaddr_q</code> .

Because `next_iaddr` becomes available one TIP cycle later, the encoder may have to buffer the discontinuity information briefly while independent trace elements – data retire events, for instance – continue to pass. The classification into inferable and uninferable PC discontinuities follows the `itype` and `updiscon` concepts of the RISC-V trace specifications; the per-core mapping of `itype` is described in [Chapter 6](#).

7.4. Branch History Accumulation

In the history modes, conditional-branch outcomes are accumulated into the HIST field until either the buffer is full or a synchronization boundary forces emission. Overflow is signalled with a ResourceFull message rather than silently truncated, which bounds the decoder's state error (see [Chapter 8](#)). The compression schemes that operate on the accumulated history – repeated history, repeat-branch, branch prediction – are described in [Chapter 9](#).

7.5. Synchronization Generator

Module: `ct_L23_preproc_sync` (`rtl/preproc/ct_L23_preproc_sync.sv`).

The generator produces a synchronization reason (`nexus_sync_reason_e`) that the eTIP composer later injects into the stream and that the formatter maps into the SYNC field of a synchronizing message. Sync generation is retire-qualified: a reason is only produced on a cycle with `tip.iretire`, so synchronization aligns to an architectural boundary.

Table 3. Sources of a synchronization request

Source	Description
Periodic counters	Programmable counters over cycles, retired instructions, retired halfwords, wall-clock ticks, and the trace-output quota (emitted bytes or messages).
One-shot on reset exit	The first retired instruction after reset deassertion emits a non-periodic "exit from system reset" reason, then arms the periodic generator.
External request	The ATB synchronization request input, and the register field <code>trTeControl.InstSyncReq</code> . Each has its own lock/acknowledge handshake, and both are emitted as the explicit-request code.

All counters share one overflow threshold derived from a CSR field as $1 \ll (\text{trTeInstSyncMax} + 4)$ and run in saturation mode. The cycle counter increments every `tip_clk`, the instruction counter on `tip.iretire`, the halfword counter by `tip.ilastsize` at retirement, and the wall-clock counter in its own clock domain with CDC for its overflow and clear. The counter width is `ct_pkg::SYNCCNT_WIDTH = 21`, chosen to exceed the maximum `trTeInstSyncMax` plus four.

The **trace-output quota** deserves a note: the byte and message counters do not live here but in the L2 egress modules, because only there is it known what actually left the encoder. Their held overflow levels are crossed back into the preprocessor as `synq_req_trace_byte_count` / `synq_req_trace_msg_count`, and the rearm (`SyncCntClr`) is crossed forward as `quota_cnt_clr` ([Chapter 5](#)).

When a condition matches, `SyncCntClr` clears all counters and rearms them for the next period. After a sync, the design waits for all involved overflow flags to deassert before accepting new triggers, so a lingering overflow cannot immediately retrigger. If several conditions coincide, exactly one synchronization message is emitted per cycle, with a priority fixed in the RTL.

7.5.1. Request Pacing

The register-based sync request (`trTeControl.InstSyncReq`) is a self-clearing field, i.e. a one-cycle pulse – and a pulse is the wrong thing to hand to a clock-domain crossing: two pulses closer together than one destination clock period cancel in a toggle synchronizer and **both** requests are lost. `ct_sync_req_pacer` (`rtl/pkg/ct_sync_req_pacer.sv`) solves this by pacing:

- a request is raised only while no other one is outstanding;
- the consumer acknowledges when it has **served** the request;
- two requests are therefore always a full round trip apart, which is what makes the crossing lossless.

The depth is one plus one. A write that arrives while a request is outstanding is not dropped but remembered in a queued slot and raised the moment the handshake closes, so it becomes its own request and gets its own synchronization message. Only a write arriving while a request is **already** queued collapses into that queued one – it asks for exactly what is about to be asked for anyway.

What crosses the domain boundary is a **level**, not a strobe. An earlier version paced strobes across two toggle synchronizers, which is not survivable here: the two halves of one request live in two reset domains – the launch side in `wb_rst`, the pending latch in the consumer in `tip_rst` – and `ct_encoder` carries both as independent inputs. A consumer reset would then clear the pending latch without an acknowledgement while the strobe that carried the request is long gone; the request is silently lost, and depending on how the two toggle pairs come out of reset the launch side can be left waiting for an acknowledgement that never arrives.

7.6. TIP Delay

Module: `ct_L23_preproc_tip_delay` (`rtl/preproc/ct_L23_preproc_tip_delay.sv`).

This stage provides a configurable pipeline delay for the **entire** TIP context, so that downstream stages can be time-aligned without re-timing their internal logic. Every TIP field is delayed uniformly – timestamp, instruction and data retire qualifiers, addresses, sizes, data payload and context – so that a consumer always observes a self-consistent snapshot of one original cycle. The module does not alter semantics and generates no events; it re-times the stream and reports its intrinsic latency of one cycle.

The instance provides **two** independently tapped delayed outputs from one shared shift register: `ct_L23_preproc` uses `tip_delayed_composer` for the AXIS and eTIP composers and `tip_delayed_cfd` for the control-flow and data-flow stages, since those two groups sit at different points of the alignment tree. On reset every pipeline stage is cleared, so no spurious event can appear after start-up.

7.7. Comparator and Filter Engine

Module: `ct_L23_preproc_comp_filters`
(`rtl/preproc/ct_L23_preproc_comp_filters.sv`).

One shared engine evaluates up to `NUM_TRACE_FILTER` filter entries per cycle and produces the consolidated hits for both the instruction and the data path (`cf_filter`, `df_filter`). It follows the filter and comparator model of the RISC-V trace control interface.

Comparators. Each comparator selects a primary and a secondary 32-bit operand from the TIP fields (`iaddr`, `context`, `tval`, `daddr`) and evaluates a configurable comparison function: equal, not equal, greater or equal, less or equal, greater, less, always true, always false. Four match modes combine the two results:

Mode	Meaning
0	Primary result true.
1	Primary AND secondary true.
2	NOT (primary AND secondary).
3	Latched: set on the primary result, cleared on the secondary result after retirement.

Mode 3 latches are synchronized to retire boundaries (`tip.iretire` or `tip.dretire`) so a latch cannot open or close on a pipeline bubble.

Per-filter predicates. Beyond the comparators, each filter is gated by its enable bit, a privilege bitmap, data-type and data-size selectors, and the comparator selection mask. If the filter selection mask is zero, tracing is unconditional – the "trace all" policy of the specification.

Trap windows. Two window predicates implement the trap-scoped filtering of the Trace Control Interface specification, section 6.3, Table 24: "start matching from the exception or interrupt, stop matching upon return from the first matching trap". The obvious counter implementation cannot do this correctly, because the return-from-exception indication carries no information about **which** trap is returning: with a nested non-matching trap inside a matching one, a plain counter decrements on the wrong return, closes the window early, and then underflows on the real return.

The implementation therefore does not use a plain counter but a **stack with a match marker**: the depth is tracked in a shift register whose marker bits record which entries matched the filter, so a return can be attributed to the correct trap. Overflow and underflow are detected (hot start, attaching to a running system) and checked by simulation assertions. Saturating counters track the window depth for the CSR view.

NOTE

The predecessor manual carried a design note describing the counter-based limitation as an accepted approximation of the specification. That note is superseded: the stack-based implementation resolves the nesting case it described. The remaining bound is the stack depth, `ct_pkg::EXCEPTION_STACK_DEPTH`.

7.8. Control-Flow Qualifier

Module: `ct_L23_preproc_cf` (`rtl/preproc/ct_L23_preproc_cf.sv`).

For each retire cycle, the stage consumes the filter engine's instruction-side hit, performs a binary search over the configured **instruction address** ranges, and produces:

- `cf_hit` – the qualified control-flow hit that the eTIP composer uses to emit control-flow messages,
- `cf_region_entered / cf_region_exited` – region transitions, and the region index.

A region entry is asserted when the winning range index changes relative to the last valid one; an exit is asserted on a no-hit or on entry into a different region. The last valid index is updated only on hit cycles, which is what prevents spurious toggling on bubbles. The stage also supplies the conditional-branch outcomes that are accumulated into the HIST field in the history modes. Outputs are registered and pass through the shared `extra_delay` tap.

7.9. Data-Flow Qualifier and Range Filter

Modules: `ct_L23_preproc_df` and `ct_L23_preproc_df_range` (`rtl/preproc/`).

`ct_L23_preproc_df_range` performs the range lookup on the retired **data address**. On every cycle with `tip.dretire` it looks `tip.daddr` up in a programmable range set using `vector_binary_search_2clk` in RANGE mode; the range memory is populated through a separate write interface (`df_range_wext`) in the `wb_clk` domain. The tree depth parameter DIM (default 4) bounds the number of ranges at $2^{\text{DIM}} - 1$, i.e. 15 ranges at the default. Throughput is one lookup per cycle after pipeline fill; the reported latency is $1 + \text{vbs_delay}$, plus the configured `extra_delay`.

`ct_L23_preproc_df` consolidates the results. A data-flow event is qualified by `tip.dretire`, the **aligned** data-tracing level (see above), and a data type of LOAD or STORE; the filter hit and the range hit are then combined into the single `df_hit` output. Its intrinsic latency is one cycle before the delay tap.

NOTE

The header comment of `ct_L23_preproc_df_range.sv` refers to instruction addresses in a few places for historical reasons. The implementation operates on `tip.daddr / tip.dretire`; the description here follows the implementation.

7.10. Timestamp Unit

Module: `ct_L23_preproc_ts` (`rtl/preproc/ct_L23_preproc_ts.sv`).

The timestamp unit provides the value that messages carry in their TSTAMP field, selected by the `trTsControl` CSR:

Selection	Source
TR_TS_NONE	Zero.
TR_TS_SYSTEM	A free-running internal counter in the <code>tip_clk</code> domain, prescaled.
TR_TS_CORE	Pass-through of <code>tip._time</code> from the core.

`trTsActive` is the primary enable of the counter, `trTsCount` runs and stops it, `trTsReset` resets it synchronously, and `trTsPrescale` divides by $2^{(2n)}$, i.e. 1, 4, 16 or 64. The counter width is `ct_pkg::CT_TS_WIDTH = 64`. Where a build profile compiles the timestamp unit out (`CT_EN_TIMESTAMP = 0`), the TSTAMP fields disappear from the wire and the `trTs*` registers read as zero rather than reporting a unit that is not there.

7.11. Performance Counters

Module: `ct_L23_preproc_perfcnt (rtl/preproc/ct_L23_preproc_perfcnt.sv)`, with `perfcnt_range_unit` as the reusable range classifier.

The performance counter block pre-aggregates statistics in hardware so that they do not have to be reconstructed from the trace stream. It observes three access classes independently – data reads, data writes and instruction fetches – and maintains separate counter banks for plain events and for threshold-qualified events.

Range classification. For each class the retired address is compared against a vector of ranges, producing a one-hot hit plus a no-hit flag. The counter of the matching range increments on a hit; a dedicated no-hit counter increments when the access falls outside every configured range, so the total is conserved and a mis-configured range set is visible rather than silent. All counters saturate; they never wrap. Clearing is per bank and per source, so software can snapshot and reset one class without disturbing the others.

Threshold measurement. In addition to event counts, the block measures per-event latency in cycles. Small state machines accumulate the cycles between key retire events and raise an overflow when an 8-bit threshold (`ct_pkg::PERFCNT_TH_WIDTH = 8`) rolls over, which qualifies a thresholded count update on the next matching retire. Separate paths exist for instruction-fetch and data-read thresholding, each gated by its own threshold value and aligned to the corresponding retire signal so that one event window cannot produce several updates.

Uniformity. The range classification is not re-implemented per class: `perfcnt_range_unit` wraps `vector_range_checker` and is instantiated per access class and per thresholded variant, which gives identical behaviour and identical latency on all measurement paths and keeps timing closure to a single `EXTRA_DELAY` parameter.

Counter width is `ct_pkg::PERFCNT_WIDTH = 32`. Updates are strictly retire-qualified and registered; the block reports a fixed internal latency of one cycle.

7.12. Instrumentation: ACT-CAP, ACT-ST, ACT-Proc

The instrumentation path is described in full in the features part of this manual; this section covers only what the preprocessor contributes.

7.12.1. ACT-CAP

Module: `ct_L23_preproc_act_cap` (`rtl/preproc/ct_L23_preproc_act_cap.sv`).

ACT-CAP – the Accemic CEDARtools CSR Access Protocol – lets the encoder observe CPU writes to designated **virtual** CSRs through the TIP, without any modification to the core: such CSR writes are functional no-ops with no system-bus side effects, and only the trace encoder reacts to them. To address many virtual control registers, ACT-CAP uses a two-stage access: a target register index is written to a selector CSR, then the selected register is written through a second CSR access.

The module filters retired CSR read-write transactions for the ACT-CAP command address, captures the command fields, and asserts a one-cycle valid strobe that advances through the shared `extra_delay` shift pipeline. Relative command ordering is preserved. Writes to other CSR addresses are ignored; on reset the whole valid pipeline is cleared so no spurious command can escape. Intrinsic latency is one cycle.

7.12.2. ACT-ST

Module: `ct_L23_preproc_act_st` (`rtl/preproc/ct_L23_preproc_act_st.sv`).

ACT-ST – the Accemic CEDARtools Smart Trigger – extends ACT-CAP by generating the same **virtual** operations from hardware when a preconfigured runtime event occurs, with no CPU involvement at all. The supported trigger conditions are a match of a retired instruction address and a match of a configured interrupt identifier.

The retired instruction address is compared against an array of key addresses with `vector_binary_search`; on a match the associated command word is fetched from the control/status memory and injected into the alignment pipeline, formatted exactly like an ACT-CAP CSR write so that everything downstream is unaware of the difference. Several trigger addresses are monitored in parallel; multiple matches are serialized by the pipeline with their order preserved. The key/command memory is written through `act_st_wext` in the `wb_clk` domain. Intrinsic latency is fixed at 2 cycles, including the match checker.

7.12.3. ACT-Proc

Module: `ct_L23_preproc_act_proc` (`rtl/preproc/ct_L23_preproc_act_proc.sv`).

ACT-Proc merges the two command sources into one ordered stream and executes the side effects. Priority is fixed: **ACT-ST overrides ACT-CAP** when both are valid in the same retired instruction, which neutralizes the software-inserted instrumentation in favour of the hardware trigger.

Depending on the command encoding, the processor can:

- emit a configurable DAQ message,
- request an extra synchronization message,
- perform writes to selected trace-encoder CSRs through the `tip_clk`-domain control interface – including setting and clearing the instruction and data trace enables, which is how tracing can be activated dynamically without CPU involvement.

Temporal ordering as observed at the TIP is preserved; no reordering of trace-relevant events takes place. The added latency is fixed and bounded, does not depend on runtime data values or message types, and is documented in the module header of the RTL file. Downstream backpressure may delay emission but never changes ordering.

7.13. AXIS Composer

Module: `ct_L23_preproc_composer_axis`
(`rtl/preproc/ct_L23_preproc_composer_axis.sv`).

The AXIS composer converts selected ACT-CAP/ACT-ST DAQ commands into wide AXI-Stream payloads – an internal trace and instrumentation sink that bypasses the trace format entirely, for example toward a watchdog CPU. It samples the most recent retired instruction and data context (addresses, data, type and size) and, when a valid ACT command targets the AXIS sink, asserts `tvalid` for one cycle and fills the data elements according to the command opcode: current PC; PC plus the last PC before an exception; immediate data from the command; last retired data with size and type; last data address; or combinations of these.

For commands that request performance statistics, the composer reads the addressed counter through the `perfcnt` interface, places it into the first element and **atomically** asserts the matching clear bit, giving read-and-clear semantics without a software race.

Framing: one AXI beat of `TDATA_WIDTH = ct_pkg::ACT_CAP_AXIS_TDATA_WIDTH` bits, carrying `MAX_DAQ_DATA_ELEMENTS = 3` elements of `ACT_CAP_INT_ELEMENT_WIDTH = 32` bits; `tstrb` marks which element slots the issued command actually filled, and `tld` carries a command-derived identifier. The path from command acceptance to AXI emission is single-cycle and applies **no backpressure** – the producer must throttle the ACT DAQ rate if the sink cannot drain the stream in time.

7.14. eTIP Composer

Module: `ct_L23_preproc_composer_etip`
(`rtl/preproc/ct_L23_preproc_composer_etip.sv`).

The eTIP composer is the last stage of the preprocessor and the widest one. It collects the time-aligned CF, DF and DAQ sub-messages, stamps them with a common context, and serializes them through a FIFO into the `proc_clk` domain.

Sub-message capture. On each cycle the composer samples the CF hit together with an instruction-count accumulation that behaves like the Nexus I-CNT field (retired instruction halfwords), the DF hit with its data-side payload and size/type, and any DAQ command output, into one parallel composite record with per-slot validity bits and a shared timestamp and I-CNT snapshot. If a synchronization reason is pending, an I-Sync sub-message is injected and the instruction counter is reset, so subsequent distances are measured from the new synchronization point.

I-CNT pre-drain. The accumulator is drained before it can exceed the field width: the pre-drain fires when the accumulated halfword count plus the current beat's halfwords reaches $2^{\text{NEXUS_MSG_I_CNT_WIDTH}}$ and then restarts the accumulator at this beat's halfwords. This is only sound while a **single** beat stays below the threshold, which is exactly why the block-ingress width is bounded at elaboration ([Chapter 5](#), `genBadIretireWidth`): a beat that alone reached the threshold would leave the accumulator above it and wedge the downstream history hold.

Implicit-return stack. The composer maintains the return-address stack used by the implicit-return compression scheme: a call pushes the address after the call site, a return pops it, and a return whose actual target equals the predicted one needs no target on the wire. The depth is `ct_pkg::CT_RET_STACK_DEPTH = 16`. The stack is cleared at a trace pause edge and is not modified by a gap re-anchor — both properties are checked by simulation assertions, because a stale stack entry would make a later return "implicit" against a frame the decoder does not have, and the decoder would then pop the wrong address. The scheme itself is described in [Chapter 9](#).

Serialization and CDC. The composite vector is written into a compact-vector FIFO and then crossed into `proc_clk`. The read side emits at most one sub-message per beat, prioritizing CF, then DF, then DAQ, so the parallel per-cycle record is flattened into a serial stream with per-cycle ordering preserved. `ct_pkg::ETIP_SLOT_SITES = 13` slot sites feed the record; the FIFO depth is `ETIP_CVS_FIFO_DEPTH` (128 when data trace, ACT or DAQ is built, else 32) and the CDC stage depth is `ETIP_CDC_FIFO_DEPTH`.

Two structural options affect this stage and are the main footprint levers of the preprocessor:

- `CT_ETIP_SERIALIZE` presents eTIP slots one per cycle through a small skid queue instead of in parallel, so the FIFO stores one entry per slot rather than one per slot **and** parallel site. Message order, per-message timestamps and the resulting byte stream are unchanged.
- `CT_EN_FIFO_HIST` adds a fill-level histogram over the composer FIFO — see [Chapter 10](#), where the readout is described.

Flush. An ATB flush request, or a trace-off, raises `do_flush`. This is emitted as a standalone flush eTIP element even when the core is idle and no further `tip.iretire` arrives, so a flush can always terminate message composition cleanly ([Chapter 5](#)).

Trace-off correlation. When tracing is paused, a correlation message has to be pushed out before the pipeline goes quiet, and the implicit-return stack is cleared at the same edge; an assertion checks that the stack is indeed empty at the pause edge.

7.15. Interfaces Between the Blocks

The preprocessor uses SystemVerilog interfaces rather than flat signal bundles, which is what keeps the alignment scheme uniform:

Interface	Use
<code>tip_if</code>	The TIP context, also used for the two delayed copies <code>tip_delayed_composer</code> and <code>tip_delayed_cfdf</code> .
<code>ct_hit_if</code>	Hit/valid pairs from the filter engine and the qualifiers (<code>cf_filter</code> , <code>df_filter</code> , <code>df_range</code> , <code>cf_qualifier</code> , <code>df_qualifier</code>).
<code>ct_sync_if</code>	The synchronization reason.
<code>ct_act_cap_if</code>	ACT command streams (<code>act_cap</code> , <code>act_st</code> , and the merged <code>act_cap_st</code>).
<code>ct_perfcnt_if</code>	Counter values and their clear controls.
<code>ct_cs_tipclk_if</code>	The <code>tip_clk</code> -domain view of the control/status registers.
<code>ocram_write_if</code>	Write ports for the ACT-ST key/command memory and the DF range memory, in the <code>wb_clk</code> domain.
<code>source_if</code>	The outgoing eTIP streams <code>etip_q</code> and <code>next_iaddr_q</code> , in the <code>proc_clk</code> domain.

Chapter 8. Processing Stage

8.1. Overview

The processing stage runs in the `proc_clk` domain and turns the protocol-agnostic eTIP event stream into protocol messages. Which generator is built is a synthesis choice ([Chapter 5](#)):

- `ct_L2_msg_gen` produces **generic trace messages** for the N-Trace back end, which `ct_L2_nexus_formatter` then expands into a Nexus field list;
- `ct_L2_te_inst_gen` produces **E-Trace `te_inst` packets** directly.

Both consume the same two streams from the preprocessor: `etip_q`, carrying the eTIP elements with their sub-type (CF, DF, DAQ, other) and payload, and `next_iaddr_q`, supplying the next instruction address whenever the control-flow type requires an explicit target ([Chapter 7](#)).

This chapter describes message **construction**. The compression schemes that decide which message is emitted – and which is suppressed – are the subject of [Chapter 9](#); the complete list of TCODEs and their fields belongs to [the message catalog](#).

8.2. Generic Message Generator

Module: `ct_L2_msg_gen (rtl/ct_L2_msg_gen.sv)`.

For each non-empty eTIP element the generator produces one or more generic trace messages (`nexus::nexus_msg_struct_t`) by

- selecting the TCODE,
- mapping the eTIP fields into the generic CF, DF or DAQ sub-structures,
- maintaining the accumulator state (I-CNT, branch history) that the messages consume, and
- optionally emitting a non-Nexus **flush** message when the TIP side channel signals an ATB flush.

One eTIP element can produce several messages, emitted consecutively. The initial element after enable, for example, forces a synchronizing message followed by an ownership message; an element that overflows an accumulator forces a ResourceFull message ahead of the message it belongs to.

The generator covers the program-trace subset of the Nexus standard and the CTTE vendor extensions, using the TCODE, SRC, I-CNT, F-ADDR / U-ADDR, HIST, SYNC, RCODE/RDATA and TSTAMP fields as defined there.

8.2.1. I-CNT Accounting

The instruction count is accumulated in `CurrICnt`, whose width is `NEXUS_MSG_I_CNT_WIDTH_WIDE + 2 = 18 bits` – bounded by construction rather than by hope,

since the value is drained long before that.

The **cap** at which a drain fires is CSR-selectable. With the wide-I-CNT option compiled in and enabled (`trTeInstFeatures.InstEnWideIcnt`) the cap is $2^{16} - 1$, otherwise $2^8 - 1$ (`NEXUS_MSG_I_CNT_WIDTH = 8`). Because the on-wire I-CNT field is variable-length, widening the cap does not change the field encoding; it only changes **when** a drain occurs. With the default narrow cap the byte stream is identical to the original fixed-8-bit behaviour.

A drain is a `ResourceFull` message with `RCODE = 0` (`NEXUS_RCODE_ICNT_OVERFLOW`). Two values are involved and the split between them matters:

Value	Content
Drain value	<code>CurrICnt</code> – what accumulated before the current event.
Drain residue	The current event's own halfword count, carried forward into the fresh accumulator.

Both are capped, and the cap is checked rather than argued. `CurrICnt` is set to zero, to the event's count, or to a sum the guard has found small enough – but that guard sits behind other drains and can be skipped, so the inductive reading of it is incomplete (see below). The event's own count is bounded by the composer's tip-side pre-drain ([Chapter 7](#)). Assertions check the emitted field value, which is what caught both breaches described next.

The cap has been broken twice, by two independent mechanisms, and the second one is the more instructive.

First cause, corrected 2026-08-12. The eight inline drain arms emitted the **sum** of accumulator and current event and zeroed the accumulator. By the very condition that triggers such an arm, that sum always exceeds the cap, so every one of those drains carried an over-cap field value. It decoded – the field is variable-length and the reference decoder reads it – which is why it stayed invisible; it was found as `RCODE-0 RDATA` values of 256, 258 and 260 on the committed artefacts of four testbenches, including the one written for this very path (`tests/instruction/20_icnt_overflow`, which produced the stream but judged no field value). The correction is a re-split of one number across two messages, not a change of accounting: the decoder's total is unchanged, because the `RCODE-0` payload feeds the same adjustment accumulator that the next message's I-CNT adds to.

Second cause, corrected 2026-08-20. The same field values kept appearing after that. The accumulation `CurrICnt ← CurrICnt + etip_cf.icnt` runs unconditionally, while the cap guard sits in an `else if` behind the branch-history drain. When both conditions fire in the same beat, history wins and the I-CNT drain is skipped – correctly, since only one message can leave per beat – but the accumulation above it has already happened, so the excess is carried on silently and the next drain puts it on the wire. Measured: $252 + 6 = 258$. The fix splits rather than clamps, for the same reason as the first: discarding the excess would lose halfwords and desynchronize the decoder.

CAUTION

The inductive argument in the source comment at that line asserted the opposite – that `CurrICnt` is only ever set to zero, to the event's count, or to a sum the guard has already found small enough. The third case does not hold

precisely when the guard is skipped. An induction with one overlooked branch reads like a proof, and this one is the reason the second cause went unexamined for two weeks after the first was closed.

`tests/instruction/20_icnt_overflow` now carries the red counter-proof: four legs that restore the pre-fix form in a worktree and go red with exactly the values of the real defect (260 / 260 / 256 / 258).

8.2.2. Branch History

The HIST registers are sized to the **wire** field — `NEXUS_MSG_RDATA_WIDTH` = 30 bits including the stop bit — not to the generic maximum field width, because the window logic caps the number of data bits by construction. The valid-bit counter is 6 bits wide, bounded by the window-full drain at 29 data bits.

When the window fills, the generator emits `ResourceFull` with `RCODE` = 1 (`NEXUS_RCODE_HIST_OVERFLOW`). The repeated-history compression replaces that message with `RCODE` = 2 (`NEXUS_RCODE_HIST_OVERFLOW_REPEATED`) where it applies; see [Chapter 9](#).

Bounding the accumulators this way — and signalling the bound explicitly — is what keeps a decoder's state error bounded: the alternative, silently truncating, would leave the decoder to discover the loss later, if at all.

8.2.3. Synchronization Handling

A synchronizing message re-anchors the decoder at a full address, which makes the treatment of the **pending** accumulator state the delicate part.

Ordinary sync in history mode. A periodic or externally requested sync would clear the accumulated history bits without emitting them, leaving the decoder unable to resolve the direct branches between the previous flush and the sync point. The generator therefore holds the sync element back for one cycle and emits a `ResourceFull HIST flush` first.

IBHS. With `trTeInstFeatures.InstEnIbhs` set, the synchronizing message **carries** the pending history itself (`IndirectBranchHistorySync`, `TCODE` 29) instead of pre-flushing it as a separate `ResourceFull`. The hold above is then bypassed. The decoder walks the I-CNT resolving branches from the HIST field and then hard re-anchors at the full address — exactly the two-message semantics in one message.

FIFO overrun. A sync whose reason is a FIFO overrun is treated differently from every other sync, and the reasoning is worth stating because it is easy to get backwards. Upstream of the generator, the composer's overflow injector has already **dropped** control-flow events, so the generator's accumulators represent only the subset that reached it. Pre-flushing the residual history bits would walk the decoder through branches without the matching in-between halfwords and land it at a PC that disagrees with the sync's own full address. The generator therefore overrides the I-CNT to zero on a FIFO-overrun sync, and the sync reset zeroes the history in the same cycle, turning the recovery sync into a **pure re-anchor** at the full address — the only ground truth available at that

point. A decoder's I-CNT adjustment heuristic can absorb a small over-count, but the post-overflow under-count is unbounded and would underflow as soon as the gap exceeds the in-message field; it has been observed on a hardware capture as a FIFO-overflow sync carrying a non-zero I-CNT.

The exit-from-reset sync never has pending history and is excluded from the hold for that reason.

8.2.4. Emission Ordering

Several accumulators may hold content that the decoder must see **before** the message that moves or repositions its PC. The generator enforces the order explicitly:

- Buffered repeated-history runs are drained before any message that walks or repositions the decoder PC – an indirect-branch-history message, any sync except FIFO overflow, the trace-off correlation, and an ATB flush request. The buffered patterns are the oldest pending content, and the decoder replays them from its current PC.
- ResourceFull with `RCODE = 0` deliberately does **not** trigger that drain: its payload only accumulates into the same adjustment counter that history walks subtract from, so it commutes with the other two RCODEs, and draining would cost compression for nothing.
- A FIFO-overflow sync **discards** the buffer instead of draining it, for the same reason the residual history is discarded.
- An open repeat-instruction run and a pending repeat-branch count are closed before any state-moving consumption, so a later history bit cannot be replayed ahead of the run it belongs to.

8.2.5. Idle and Empty

`msg_gen_idle` feeds the `trTeControl.Empty` chain ([Chapter 5](#)). It means: no message output is presented **and** no compression substance is being held back. The I-CNT and history accumulators deliberately do **not** count as generated trace – they are accumulator state, not a message that was produced but not yet emitted; the flush and correlation paths drain them before `Empty` becomes relevant.

8.2.6. Backpressure, Ordering and Determinism

The downstream formatter chain can deassert `ready_in`. In that case the generator holds its output stable and does not acknowledge new input from `etip_q` or `next_iaddr_q`. Concretely, the RTL

- consumes input only while `ready_in` is asserted,
- holds the output message stable while it is deasserted, and
- acknowledges `next_iaddr_q` only when a control-flow message actually requires the target and the entry is valid.

This preserves message order and makes message loss by backpressure impossible.

8.2.7. Latency

The generator is primarily a single-cycle transform in `proc_clk`. Observable latency is dominated by the upstream eTIP CDC FIFO and by downstream backpressure, not by the transform itself.

8.3. Nexus Message Formatter

Module: `ct_L2_nexus_formatter` (`rtl/ct_L2_nexus_formatter.sv`).

The formatter converts a generic trace message into an explicit Nexus **field list** (`nexus_message_t`) – still without MDO/MSEO packing. A Nexus message is a sequence of fields and always begins with a TCODE field; the array holds up to `nexus_vendor::NEXUS_MAX_FIELDS = 10` fields, each up to `NEXUS_MAX_FIELD_DATA_WIDTH` bits wide.

For each input message the formatter:

- emits the TCODE field,
- optionally emits a SRC field, depending on configuration (`trTeControl.InhibitSrc`),
- appends the message-specific payload fields – SYNC, ICNT, PC_FADDR, UADDR, HIST, RCODE / RDATA, DQDATA, DSZ/ELSZ and so on,
- optionally appends a TSTAMP field when timestamping is enabled, and
- clears or initializes unused field slots so that simulation cannot read stale content.

Timestamps follow the standard's two-mode scheme: a **synchronizing** message carries an absolute timestamp, a non-synchronizing message a relative one that a decoder adds to the last absolute or relative value.

The formatter also maintains the protocol state that spans messages, in particular the **reference address** used for address compression – the value against which the next differential address is computed. That state is mirrored bit for bit in the compression logic of the generator ([Chapter 9](#)), because both sides must agree on what the decoder's reference will be.

No buffering. The module accepts a new message only while its own `ready_in` is asserted. When downstream stalls, it holds `nexus_msg` stable and deasserts `ready_out` to stall upstream. This is what makes the ordering guarantee of the chain hold end to end.

In the compact-packer profiles this module is not built at all: `ct_L2_compact_packer` produces the MDO/MSEO chunks directly from a per-TCODE layout table, and the wire stream is byte-identical ([Chapter 10](#)).

8.4. E-Trace te_inst Generator

Module: `ct_L2_te_inst_gen` (`rtl/ct_L2_te_inst_gen.sv`).

The E-Trace generator consumes the same eTIP stream and produces `te_inst` packets of the RISC-V Efficient Trace specification, baseline algorithm, instruction trace only. Its event mapping mirrors the reference encoder model shipped with the specification:

eTIP event	<code>te_inst</code> output
Plain taken / not-taken branch	One <code>branch_map</code> bit (0 = taken, least significant bit oldest). The 31st bit forces a Format 1 packet without an address.
Uninferable discontinuity	Format 1 with the pending branches, or Format 2 with the differential address of the jump target.
Trap (exception or interrupt)	A Format 1/2 flush at the trap point, then a deferred Format 3.1 carrying the handler address. The deferral by one event is what allows the synchronization "branch" bit to be resolved when the anchor instruction is itself a branch.
Enable sync (first beat)	Format 3.3 SUPPORT with <code>ienable = 1</code> , followed by a Format 3.0 full-address anchor.
FIFO-overflow sync	Format 3.3 SUPPORT with the trace-lost qualifier, followed by Format 3.0.
Trace-off / correlation	Format 1/2 flush, then Format 3.3 SUPPORT with <code>ienable = 0</code> and the ended-reporting qualifier.
Composer I-CNT drain	Consumed silently – E-Trace has no I-CNT field.

Mid-trace synchronizations from the composer – periodic, explicit request, external trigger, sequential – re-anchor through a deferred Format 3.0. The sideband values `ecause`, `tval`, `priv` and `ilastsize` ride the eTIP sideband, which is why an E-Trace instance requires those fields to be built at full width ([Chapter 5](#), `genETraceNoSideband`).

This implementation is a control-flow-only profile; the restriction is enforced by an elaboration guard in the module.

The packets are framed onto ATB by `ct_L2_te_packetizer` (`rtl/ct_L2_te_packetizer.sv`), which emits the reference-raw byte framing – one byte per beat, a header byte `{0, msg_type[1:0], payload_len[4:0]}` followed by the payload – and crosses into the ATB clock domain. That framing is what the funnel's dual-protocol mode consumes ([Chapter 11](#)).

Chapter 9. Compression Suite

9.1. Principle

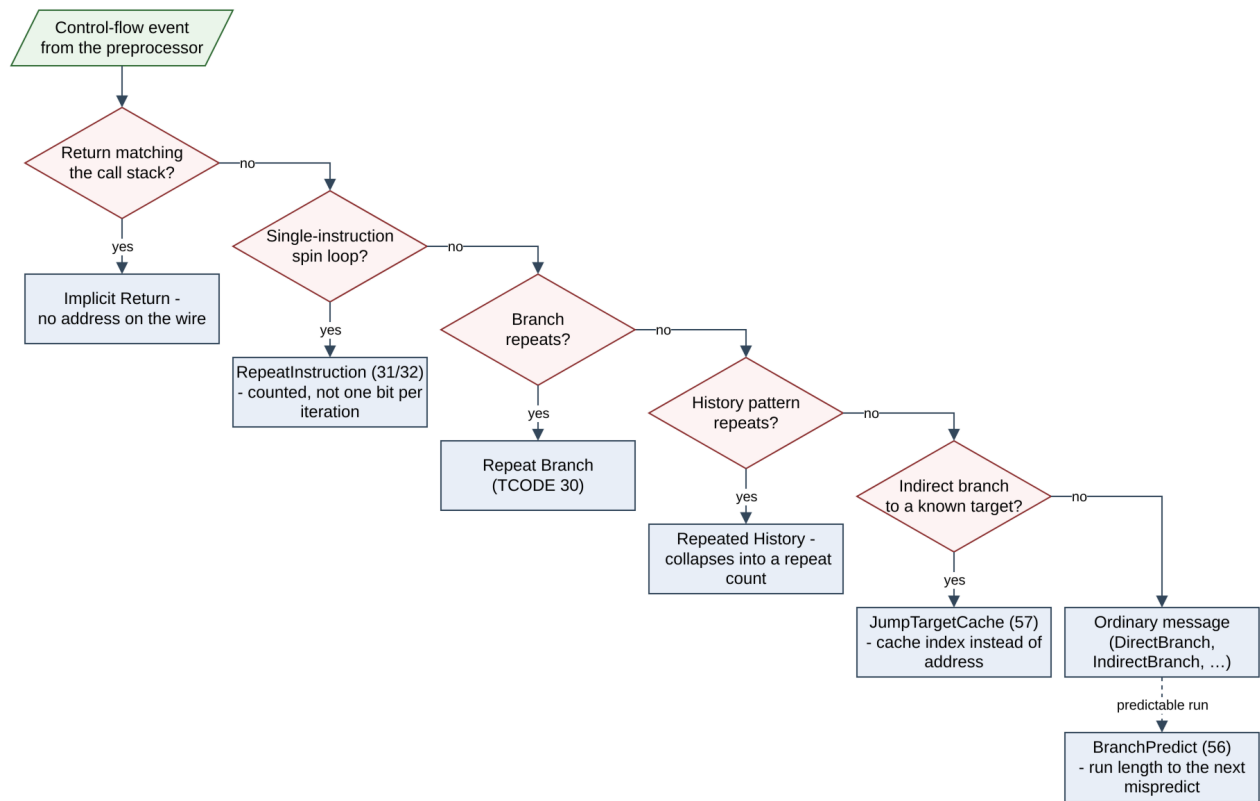


Figure 9. Which compression feature applies, and in which order

Every feature is individually switchable, and the full suite is not always the best choice: on a pointer-heavy workload it measures worse than the same build without branch prediction.

The order shown is the evaluation order, not a priority ranking of value.

CTTE bundles a set of individually switchable program-trace compression features. They reuse the standard message machinery – vendor TCODEs, reserved RCODEs, variable-length fields – so a Nexus-aware decoder can still **frame** every stream, whether or not it understands a particular scheme.

Three properties hold for the whole suite and are worth stating before the individual schemes:

Every feature has two switches. A compile-time switch in `rtl/pkg/ct_pkg.sv` (`CT_EN_*`) decides whether the logic exists at all, and a runtime enable field in `trTeInstFeatures` decides whether it is used. This pairing is a property of **this suite**, not of every CTTE feature: the event groups, for instance, are compile switches without a dedicated runtime enable.

Every feature resets to off. The runtime enables all reset to 0, so a freshly reset encoder produces the uncompressed baseline stream. This is what makes "byte-identical to the previous build" a

checkable property for every feature addition.

Every feature is a two-sided model. The compression schemes do not remove redundancy that the decoder can find by itself – they remove information the decoder can **reconstruct** because it runs the same model. The return stack, the jump-target cache and the branch predictor exist twice, once in the encoder and once in the decoder, and they must agree **bit for bit**. A one-bit disagreement desynchronizes the two models silently. That is why the folding functions are spelled out as loops in the RTL rather than hand-unrolled, why the model state is cleared at exactly the same events on both sides, and why the capability and structure constants are advertised in the configuration message rather than configured out of band.

Which features are compiled in (CAPS) and which are currently enabled (ENAB) are carried in the vendor configuration message together with the structural sizes of the return stack, the jump-target cache and the branch predictor, so a decoder can configure its models from the stream itself. See [the self-identification chapter](#).

9.2. Feature Matrix

Feature	Runtime enable (<code>trTeInstFeatures, te offset 0x008</code>)	CAPS	Reset	Full profile	Measured cost
Implicit return	<code>InstEnImplicitReturn [3]</code>	0	0	1	+71 LUTs / +553 FFs—the highest FF-to-LUT ratio in the encoder: <code>CT_RET_STACK_DEPTH (16) x 32-bit address = 512</code> register bits, i.e. 93 % of the flip-flops, with trivial push/pop logic.
Repeated history	<code>InstEnRepeatedHistory [8]</code>	1	0	1	+86 LUTs / +91 FFs
Wide instruction count	<code>InstEnWideIcnt [12]</code>	2	0	1	+66 LUTs / +29 FFs
Repeat branch	<code>InstEnRepeatBranch [11]</code>	3	0	1	+31 LUTs / +162 FFs
Jump-target cache	<code>InstEnJumpTargetCache [5]</code>	4	0	1	+262 LUTs / +71 FFs
Branch prediction	<code>InstEnBranchPrediction [4]</code>	5	0	1	+618 LUTs / +56 FFs—the most expensive of the switches measured this way.
Indirect branch history with sync (IBHS)	<code>InstEnIbhs [13]</code>	8	0	1	full-profile delta of -446 LUTs, not causally isolable.
Repeat instruction	<code>InstEnRepeatInstr [14]</code>	9	0	1	full-profile delta of -381 LUTs, not causally isolable.

NOTE

The cost column is quoted from the feature-flag table in the public integration guide (`doc/integration.adoc`). The six isolable figures are differential out-of-context syntheses on `xck26-sfvc784-2LV-c` measured 2026-08-11/12 with Vivado™ 2026.1 at a 5.0 ns target against base **A** (`featparity`, compact packer on: 3 552

LUTs / 3 598 FFs / 6.5 BRAM, WNS +1.196 ns). The two IBHS and repeat-instruction figures are full-profile deltas; per the interpretation rule of that table, full-profile deltas below roughly +/-300 LUTs are restructuring-affected, so a negative delta does **not** mean the feature is free. The consolidated profile-level resource and timing table is in the footprint-profile chapter, not here.

9.3. Interactions and Exclusions

Branch prediction **replaces** the history accumulation that the history-based loop compressions buffer and replay. Mixing the two models would produce a stream neither side can walk, so the exclusion is hard-gated in the RTL rather than left to the programmer: with branch prediction enabled, repeated history, repeat branch and repeat instruction are forced off, whatever the CSR says. A misprogrammed register therefore cannot mix the models, and with branch prediction off the behaviour is bit-identical to a build without it.

The other interaction to know about concerns **ordering**, and it is described in [Chapter 8](#): buffered compression state must reach the wire before any message that walks or repositions the decoder's PC.

9.4. Implicit Return

Compile switch `CT_EN_IMPLICIT_RETURN`, enable `InstEnImplicitReturn`.

A return whose target the decoder can predict does not need to carry that target. The encoder maintains a return-address stack in the eTIP composer

(`rtl/preproc/ct_L23_preproc_composer_etip.sv`): a call pushes the address after the call site, a return pops it. Depth is `ct_pkg: :CT_RET_STACK_DEPTH = 16` slots, advertised in the configuration message.

A return folds – is treated like an inferable branch and emits no indirect-branch message – when **all** of the following hold:

- the composer's stack produced a prediction,
- that prediction equals the actual next address,
- the element carries no synchronization reason, and
- the runtime enable is set.

A sentinel value means "the stack had no prediction" and never folds, because the decoder's stack would be empty as well.

The comparison itself was moved into the composer during a resource pass; only the one-bit result rides the next-address sideband into the message generator.

Stack hygiene is the part that is easy to get wrong, and it is guarded by assertions:

- At a **trace pause** the stack is cleared. A frame pushed before the pause would make a return after the resume "implicit" against a frame the decoder does not have, and the decoder would pop a wrong or empty frame. Assertion I3 checks that the stack is empty when the trace-off correlation is emitted.
- A **gap re-anchor** must not modify the stack. Assertion I5 checks this.

Known limitation on the E-Trace back end: with the `irdepth` field width at zero, an implicit-return-folded loop that crosses a positional binding point — a trap or trace-off flush, a window or resynchronization anchor — cannot express the visit count. Measured as 6 of 33 red among the implicit-return-armed runs of a free-axis calibration (`efix_cal1`, seed 20260806). The implicit-return bit is therefore masked from the E-Trace soak draw until an `irdepth` field exists; every other compression bit is drawn.

9.5. Repeated History

Compile switch `CT_EN_REPEATED_HISTORY`, enable `InstEnRepeatedHistory`.

A full history window that would be emitted as `ResourceFull RCODE = 1` is instead **buffered**. An identical follow-up pattern only increments a counter. The buffered run is emitted as a single `ResourceFull` with `RCODE = 2` (`HIST_OVERFLOW_REPEATED`, first payload = the pattern, second = the count) — or as a plain `RCODE = 1` when the pattern occurred exactly once — as soon as the pattern changes or a PC-walking message has to go out.

The scheme matches the behaviour of the reference software encoder, including its partial-pattern trim.

9.5.1. Partial Match

The exact-window match alone is weaker than it looks. The window carries `NEXUS_MSG_RDATA_WIDTH - 1 = 29` data bits, and 29 is prime — so **every** non-uniform periodic branch pattern phase-drifts against the window boundary and never matches exactly. Loop bodies whose period does not divide 29 would therefore never compress, which is most of them.

The partial-match conversion fixes this by trimming the window. For a trim by s bits, the branch stream is periodic with a period dividing $W - s$ if and only if

1. the previous pattern closes across the trimmed-window boundary (its lowest s bits equal its highest s bits), and
2. the current window continues the previous one, shifted by the s -bit drift.

Two shifts are implemented: $s = 1$ gives a 28-bit window, which covers periods 2, 4, 7, 14 and 28 — the `if/else-in-a-loop` shape — and $s = 2$ gives a 27-bit window, covering periods 3, 9 and 27, the three-way unrolls. The check runs only on the **second** window of a run, matching the reference encoder. A run with shift 0 is an exact full-window run; a run with shift s carries a pattern of $29 - s$ data bits, and the s least significant bits of each window are carried into the next

accumulation.

9.6. Wide Instruction Count

Compile switch `CT_EN_WIDE_ICNT`, enable `InstEnWideIcnt`.

This raises the **internal** cap on the accumulated instruction count from 8 to 16 bits, which reduces how often a `ResourceFull RCODE = 0` drain has to be emitted. The on-wire I-CNT field is variable-length either way, so an enabled wide cap changes **when** drains occur, not how a field is encoded. With the narrow cap the byte stream is identical to the original fixed-8-bit behaviour. The details of the drain accounting are in [Chapter 8](#).

9.7. Repeat Branch

Compile switch `CT_EN_REPEAT_BRANCH`, enable `InstEnRepeatBranch`, wire message `TCODE 30`.

An indirect-branch-history message identical to the **last emitted** one — same history, same instruction count, same target — is suppressed and counted. The run is closed by a repeat-branch message carrying the suppression count, and the decoder replays its saved previous message that many times.

Two hard constraints follow from how the decoder implements the replay, and both shape the encoder:

The count must be drained before every other emission. The reference decoder saves only the first two message fields across the parse of a repeat-branch message, so **any** intervening message clobbers the remembered fields. This drain is stricter than the repeated-history one: it applies even to `RCODE = 0` drains, and the "last message was an indirect-branch-history" flag is cleared whenever any other message goes out.

Only byte-identical messages may collapse. The decoder replays the saved message verbatim, so a replayed indirect-branch-history message re-applies its **differential** address against the moved address reference. That is correct only if the differential address was zero — then the reference already sits at the target and stays there through every replay. A jump-target-cache message replays soundly in every case, because its cache index resolves the absolute target and no cache install can happen between the message and its replays (any other emission clears the flag).

NOTE

The earlier match criterion — same history, count and target — collapsed two **different** calls that happened to share a target into one repeat whose head message carried a non-zero differential address. The decoder then walked a stale loop iteration and derailed. Found as soak finding S-7 (seed 1317011359) and fixed by adding the replayability condition. The encoder mirrors the packer's address-compression reference (the last transmitted program-trace address) for exactly this test, updated centrally at the output handshake because the differential address of

the message being emitted is computed one stage downstream.

9.8. Jump-Target Cache

Compile switch `CT_EN_JTC`, enable `InstEnJumpTargetCache`, wire message vendor TCODE 57.

An indirect branch whose target is already installed in a direct-mapped cache is emitted with a 6-bit **cache index** instead of the differential address. The cache holds `ct_pkg::CT_JTC_ENTRIES = 64` entries, advertised in the configuration message.

Install rule. The cache installs on every **emitted** plain indirect-branch-history message, i.e. on a cache miss. The decoder mirrors exactly that rule — learn on a plain indirect-branch-history message, read-only on a cache-index message — so both models stay bit-identical without any explicit synchronization.

Index function. The index is an XOR fold over 6-bit slices of the target address, not a simple bit slice. The reason is empirical: the targets where the cache wins most are far apart and typically have zero low bytes, so a plain `[7:2]` index would make them all collide.

The highest address bit the fold consumes is a **knob**, not simply "all bits":

Address width	Folded bits
32-bit	Up to bit 25 — the four slices <code>[7:2]</code> , <code>[13:8]</code> , <code>[19:14]</code> , <code>[25:20]</code> . The fold has to stay bit-identical here, which is why the bound is explicit rather than derived from the address width.
64-bit	The remaining eleven slices join in. Leaving them out would make the index blind above bit 25, and under Sv39 the kernel/user split lives entirely above it — every kernel target would hash as if its high half were zero and collide with the user-space target sharing its low bits. The last slice is short (bits 63:62) and enters zero-extended.

The decoder's fold must switch the same way, driven by the address-width capability bit it reads from the configuration message.

Clearing. The cache is cleared at trace-off (the correlation message), so a decoder starting at the next session can never see an index it could not have learned.

9.9. Branch Prediction

Compile switch `CT_EN_BP`, enable `InstEnBranchPrediction`, wire message vendor TCODE 56.

With branch prediction enabled, direct branches do **not** accumulate history bits at all. Both sides run a bit-identical predictor:

Property	Value
Structure	Direct-mapped table of <code>ct_pkg::CT_BP_ENTRIES = 512</code> two-bit saturating counters, advertised in the configuration message.
Index	<code>iaddr[10:2]</code> (9 bits).

Property	Value
Init	Weakly not-taken.
Update	With the actual outcome of every processed direct branch.

A correctly predicted branch is **silent** – the encoder only counts it. The decoder resolves it from its own predictor while walking the instruction count of whatever message eventually covers it. Only a **mispredict** emits a message: TCODE 56 carrying the number of correctly predicted branches since the last PC-walking emission. The decoder then eagerly walks that many branches predictor-resolved, plus one more inverted. The message carries no instruction-count field; it follows the same walking convention as the history-overflow RCODEs.

The prediction counter therefore resets on every PC-walking emission – an indirect-branch-history message, a cache-index message, a sync, a correlation, or a mispredict message – and **not** on ResourceFull drains, because an $\text{RCODE} = 0$ drain does not walk and $\text{RCODE} = 1/2$ walk only carried-over seed bits. Branches carried by a synchronizing message keep the existing one-bit history seed path: the decoder consumes pending history bits before consulting its predictor, and updates the predictor either way.

Storage. A clear loop over 512 entries would force the table into flip-flops. Instead each entry carries a 16-bit **epoch tag**: clearing the model – at trace-off, on FIFO-overflow recovery, at reset – bumps the epoch, and an entry with a stale tag simply **reads** as the initial value, bit-identically to a real clear. A background scrubber rewrites one stale entry per idle cycle, so a tag value can never alias even after a full wrap of the epoch counter. The table maps into LUTRAM rather than registers.

Model synchrony under loss. The table is cleared at trace-off and on FIFO-overflow recovery, and the decoder mirrors both. Branches dropped by an overrun are never seen by the message generator and never walked by the decoder, so the two models stay in lockstep across a gap.

9.10. Indirect Branch History with Sync (IBHS)

Compile switch `CT_EN_IBHS`, enable `InstEnIbhs`, wire message TCODE 29.

Without this feature, a synchronizing message in history mode has to be preceded by a ResourceFull history flush, because the sync would otherwise clear the accumulated history bits without emitting them and leave the decoder unable to resolve the direct branches before the sync point. With IBHS enabled, the synchronizing message **carries** the pending history itself, and the pre-flush is skipped: one message where two went before.

The decoder semantics are unchanged – it walks the instruction count resolving branches from the history field, then hard re-anchors at the full address. That is exactly the two-message semantics in one message.

Two synchronization reasons stay outside the feature: a FIFO-overflow sync, whose history is **discarded** rather than carried (see [Chapter 8](#)), and the exit-from-reset sync, which never has pending history.

9.11. Repeat Instruction

Compile switch `CT_EN_REPEAT_INSTR`, enable `InstEnRepeatInstr`, wire messages TCODE 31 and 32.

A single-instruction spin loop – a taken branch whose target is its own address – is compressed into a count instead of one history bit per iteration. The first execution is accounted normally (a history bit and its halfwords); every following identical iteration is silently counted. An iteration counts only when the address **and** the per-iteration halfword count match the armed run, which is a defensive match rather than a necessary one.

The run closes in one of two ways:

Message	Decoder contract
TCODE 31 (repeat instruction)	Emitted before any other message. The decoder walks the instruction count with the history – which ends back at the loop PC after the first execution's taken bit – and then re-emits the loop PC as many times as the repeat count says. The replays touch no instruction-count accounting; the count restarts at zero after this message.
TCODE 32 (repeat instruction with sync)	Emitted when a synchronization lands on the armed loop branch mid-run. The decoder walks the instruction count with the history, then performs repeat-count-plus-one replays – the counted iterations plus the sync-carrying iteration itself, whose halfwords the encoder deliberately drops so that neither the count nor the replays carry them – and then hard re-anchors at the full address, which is the loop PC.

The counter is 18 bits wide and saturates; an iteration at the cap does not extend the run, it starts a new one. A FIFO-overflow sync discards the count, like the other buffered compressions, because it belongs to the pre-gap stream.

The feature is excluded under branch prediction, whose predictor already absorbs spin loops.

9.12. How much the suite actually saves

The honest answer is: **it depends on the code, far more than on the encoder**. A single figure per feature would be misleading, and the predecessor manual's per-feature percentages could not be traced to any workload, configuration or artefact and are therefore not carried forward.

What can be shown is a measured spectrum. One core, one encoder netlist, one configuration, synchronization every 1024 instructions, PC-range filtered, two windows per point – the only difference between the rows is the code being executed:

Table 4. Bits per retired instruction by compression feature set (measurement series BM1, 2026-08-09)

Workload	none	no BP	full suite	best	dominant structural property
loop	0.661	0.406	0.206	–69 %	97.5 % predictable branches, no indirect jumps
coremark	0.860	0.643	0.726	–25 %	92.2 % predictable, 0.6 % indirect
sqlite	2.084	1.710	1.784	–18 %	87.3 % predictable, 2.9 % indirect

Workload	none	no BP	full suite	best	dominant structural property
ptr	4.627	4.129	4.900	-11 %	73.3 % predictable, 8.1 % indirect, 99.4 % new history patterns

Three things in that table matter more than any single number:

- **The range is a factor of 23.8** (0.206 to 4.900 bits per instruction) on an unchanged encoder. Bandwidth sizing therefore has to start from the workload, not from an encoder datasheet figure – [Chapter 26](#) gives the sizing procedure.
- **The full suite is not always the best choice.** On `ptr` it is measurably worse than the same build with branch prediction disabled (4.900 against 4.129), because a workload whose history patterns are 99.4 % new defeats a predictor and pays its message overhead without the benefit. The features are individually switchable for exactly this reason.
- **The benefit correlates with predictability**, not with the number of features enabled.

NOTE

The series was produced for the CTTE publication; the raw data live outside this repository. The figures are reproduced here for orientation and carry the same reproducibility caveat as the coverage figure in [Chapter 31](#) – see the note at the end of this chapter.

9.13. Standards Positioning

CTTE's **reset** configuration produces a stream that is within N-Trace 1.0, with the single exception of the configuration message. Switching on a compression feature can leave that envelope, and the public trace-format documentation states per feature why:

Feature	Position
Branch prediction (56), jump-target cache (57)	Vendor TCODEs in the Nexus vendor range 56-62. Both sides must run the bit-identical model; a decoder without it cannot reconstruct the flow.
Repeat instruction (31/32)	IEEE-ISTO 5001-2012 sections 4.3.14 and 4.3.15 messages; N-Trace 1.0 Table 9 marks TCODEs 31/32 as reserved for future extensions. Enabling the feature therefore makes the stream an Accemic/ISTO extension rather than a strict N-Trace 1.0 stream.
Wide instruction count	Internal accounting only. The on-wire field stays variable-length per Nexus, so an enabled wide cap is not a wire-format extension – it only changes how often ResourceFull (I-CNT) messages appear.
Implicit return, repeated history (RCODE 2), repeat branch (30), IBHS (29)	Standard message types and RCODEs; the compression lies in the encoder/decoder model agreement, not in a new field.

Sequential-jump inference is not a runtime switch at all but an ingress convention of the attached core, fixed at build time by the `CT_SIJUMP` parameter and advertised in the configuration message ([Chapter 6](#)).

9.14. The E-Trace Back End

The E-Trace generator implements the equivalent schemes in the forms the Efficient Trace specification provides for them: branch prediction as Format 0.0 and the jump-target cache as Format 0.1, plus the optional implicit-return and sequential-jump modes. The models and their clearing rules are the same; only the packet form differs. The E-Trace-specific tests are `tests/instruction/25_etrace_jtc`, `26_etrace_bp`, `21_etrace_ir` and `34_etrace_resync_ir`.

9.15. Verification

Each compression feature has its own directed test:

Test	Feature
<code>tests/instruction/06_implicit_return</code>	Implicit return
<code>tests/instruction/07_repeated_history</code>	Repeated history, including the partial-match trims
<code>tests/instruction/08_repeat_branch</code>	Repeat branch
<code>tests/instruction/09_jump_target</code>	Jump-target cache
<code>tests/instruction/10_branch_predict</code>	Branch prediction
<code>tests/instruction/13_ibhs</code>	IBHS
<code>tests/instruction/14_repeat_instr</code>	Repeat instruction
<code>tests/instruction/19_feature_matrix</code>	Feature combinations
<code>tests/instruction/20_instruction_count_overflow</code>	Instruction-count drains, including the wide cap

Beyond the per-feature tests, the suite carries **byte-identity legs** (`cli_bp_test.sh`, `cli_ir_test.sh`, `cli_jtc_test.sh`, `cli_rh_test.sh`, `cli_rb_test.sh`): the same stimulus is run with the feature off and on. The off run's prefix must equal the reference stream, and the on run must decode to **the same PC sequence at a strictly smaller byte count**. These legs pin the wire format: any by-product byte change in an unrelated fix turns them red.

That is also the exact claim the suite supports, and it is worth being precise about it: each feature is proven **lossless** — the decoded PC sequence is identical with the feature off and on — and proven to **reduce** the byte count on its dedicated stimulus. How much it reduces depends entirely on the workload, and a figure without a named workload, profile and comparison baseline would not be reproducible.

Chapter 10. Output Stage

10.1. Overview

The output stage turns a message into bytes on the ATB and crosses from `proc_clk` into `atb_atclk`. Three paths exist, and exactly one is built:

Build	Path
N-Trace, historical (default)	<code>ct_L2_nexus_formatter</code> → <code>ct_L2_mseo_mdo_formatter</code> (message buffer, bit slicer, MSEO controller, ATB chunk packer, CDC FIFO, flush detect).
N-Trace, compact (<code>ct_pkg::CT_COMPACT_PACKER = 1</code>)	<code>ct_L2_compact_packer</code> replaces formatter, message buffer, bit slicer and MSEO controller with one module; the ATB tail is reused unchanged.
E-Trace (<code>EN_ETRACE</code>)	<code>ct_L2_te_packetizer</code> , reference-raw <code>te_inst</code> framing (Chapter 8).

An essential property of all N-Trace variants: **the wire bytes are invariant against assembly timing**. Only the number of cycles spent assembling a message differs between the variants; the byte stream does not.

10.2. MSEO/MDO Formatter

Module: `ct_L2_mseo_mdo_formatter` (`rtl/ct_L2_mseo_mdo_formatter.sv`) with the submodules in `rtl/mseo_mdo/`.

The formatter accepts one `nexus_message_t` whenever `ready_out` is high, slices and encodes it into MDO/MSEO chunks, packs the chunks into ATB beats, and crosses into the ATB clock domain. The message formatting follows the Nexus standard.

Submodule	Responsibility
<code>ct_L2_mseo_mdo_formatter_msg_buffer</code>	Single-entry message buffer with valid/ready handshake.
<code>ct_L2_mseo_mdo_formatter_bit_slicer</code>	Slices the field list into MDO-width data slices. It is a dispatch wrapper: the supported MDO widths are bound as hard-wired modules through <code>generate(_mdo6, _mdo14)</code> , each pinning its width at the module boundary and sharing one tuned core so the slice algorithm cannot drift between widths. An unsupported width fails elaboration.
<code>ct_L2_mseo_mdo_formatter_mseo_controller</code>	Generates the MSEO patterns and stalls data slices during control-only cycles.
<code>ct_L2_mseo_mdo_formatter_atb_chunk_packer</code>	Packs MDO/MSEO chunks into ATB-width payloads, inserts alignment padding at end of message, and emits flush payload beats on request.
<code>ct_L2_mseo_mdo_formatter_atb_flush_detect</code>	ATB-domain flush detector: watches the transmitted chunks for the flush marker and holds <code>afready</code> high until the external flush request is released.

10.2.1. MSEO Sequencing

The MSEO controller implements the single-pin and dual-pin sequences of the Nexus standard, Section 5:

Event	Single-pin	Dual-pin
Idle	1	11
Start of message	1, 1, 0	11 → 00
End of variable-length packet	0, 1, 0	00 → 01
End of message	0, 1, 1, 1, ...	00 or 01 → 11

The chunk packer's end-of-message **alignment padding** is what allows a downstream funnel to detect packet boundaries from the chunk-local MSEO bits alone: after an end-of-message chunk the rest of the beat is padding, so a beat never carries the end of one message together with the start of the next ([Chapter 11](#)).

10.2.2. MDO Width

`nexus_vendor::NEXUS_MDO_WIDTH` selects the chunk width; the default is 6, giving the classic 8-bit chunk `{MDO[5:0], MSEO[1:0]}` — four chunks per 32-bit ATB beat. Width 14 gives two chunks per beat. The historical generic width 30 has been removed from the slicer; it survives only as a parsing configuration on the funnel side.

10.2.3. Assembly Throughput

`ct_pkg::CT_SLICER_STEPS` trades slicer area against sustained assembly rate. At 2 an output slice may cross one field boundary per cycle, which sustains roughly one slice per cycle even across many short fields. At 1 the field-multiplexer and shift datapath is halved, and a boundary-crossing slice then needs two cycles — a lower sustained wire rate on short-field messages, which is acceptable where the profile is bandwidth-uncritical. **The wire bytes are identical either way**; only assembly timing changes.

10.2.4. Backpressure, Framing and Latency

The formatter is part of the `ready_in / ready_out` chain and must neither drop nor reorder packets under stall conditions; messages are buffered and managed through an internal stall signal. When a flush request is propagated from the TIP side channel, the chain emits an explicit flush marker so that the ATB receiver can align to a packet boundary.

Packing and emission are pipelined in `proc_clk`. The base latency is fixed for a given configuration; variable latency comes only from ATB-sink backpressure.

Three figures are worth keeping in mind when sizing the ATB tail: `ct_pkg::ATB_MAX_CHUNKS = 4` (chunks per beat at the 8-bit chunk width), `ATB_CVS_FIFO_DEPTH = 8` and

ATB_CDC_FIFO_DEPTH = 8.

10.3. Compact Packer

Module: `ct_l2_compact_packer` (`rtl/ct_l2_compact_packer.sv`), selected by `ct_pkg::CT_COMPACT_PACKER`.

The compact packer produces the MDO/MSEO chunk stream **directly** from a per-TCODE layout table, replacing the generic ten-field array of the Nexus formatter, the message buffer and the barrel bit slicer with a single module. The downstream ATB tail – chunk packer, CDC FIFO, flush detect – is reused unchanged.

Wire contract (byte-identical to the historical path):

- Fields in per-TCODE order, transmitted least significant bit first.
- All **fixed** fields of a message form one contiguous bit string – TCODE, optional SRC, and the per-TCODE subfields. Dual-pin MSEO marks no fixed-field boundaries, so pre-merging them changes no byte.
- Every **variable** field is emitted with leading-zero suppression, at least one bit wide, and ends its MDO slice: the last slice holding its bits is zero-padded to the slice boundary.
- MSEO per chunk: end-of-message on the message's last slice, end-of-variable-field on a slice that ends a variable field mid-message, normal otherwise.
- A flush message emits no data chunks; it pulses the chunk packer's flush start, exactly as the historical message-buffer flush detect does.
- The address-compression reference state and the timestamp delta/absolute handling replicate the Nexus formatter exactly – including the rule that a repeat-branch message carries no timestamp, and the historical truncation quirk when the SRC width is configured as zero.

Scope. The layout table covers the program-trace path, ERROR and FLUSH – everything the message generator can emit in a control-flow-only profile. Data-trace and data-acquisition formats stay on the historical path, and elaboration **fails** for a profile that could emit them. The switch therefore requires the DAQ, data-trace and instrumentation features to be compiled out.

Cost. Replacing the formatter, slicer and MSEO controller is the cheaper of the two output paths in the control-flow-only profile; the sourced differential measurement is in [Chapter 24](#). The compact pair tests report 12 of 12 verdicts and 14 of 14 message-identical streams.

10.4. eTIP Serialization

Switch: `ct_pkg::CT_ETIP_SERIALIZE`.

Strictly speaking this lever sits in the preprocessor's eTIP composer ([Chapter 7](#)), but it belongs here because it is one of the two structural levers between the footprint profiles and because its effect is measured on the same artefacts as the packer.

With serialization on, the eTIP slots are presented **one per cycle** through a small skid queue instead of in parallel, so the composer FIFO stores one entry per slot rather than one entry per slot and parallel site. Message order and per-message timestamps are unchanged, and the byte stream is identical. A multi-slot beat that exceeds the skid drops **whole** into the designed overflow-to-resynchronization path — structurally unreachable for control flow plus data flow at at most one event per instruction; DAQ-dense workloads are gated by a budget analysis instead.

Cost. Serialization is the full-profile default, with the full test suite passing and a manifest of 15 of 15 byte-identical reference streams. On the slim and compact profiles the skid costs more than it saves, so those profiles set the switch to 0 explicitly (`ct_pkg.sv`). The resource figures per profile are in [Chapter 24](#).

Two related switches diet the same interface: `CT_ETIP_CDC_SLIM` shrinks the decouple stage behind the composer FIFO in single-clock builds from depth 4 to a minimal register pair — removing the stage entirely was measured and **halved** the peak delivery rate, so it keeps the prefetch the consumer needs. `CT_EN_ETIP_WATERMARK` controls whether the `trTeTipFifoMaxFill` watermark chain exists in hardware; with it off the CSR reads zero and the tracking logic trims away, while the end-of-simulation watermark report remains as a simulation-only shadow, so depth-tuning evidence is unchanged.

10.5. eTIP FIFO Fill-Level Histogram

Switch: `ct_pkg::CT_EN_FIFO_HIST`, bin count `ct_pkg::CT_FIFO_HIST_BINS` (default 16).

The maximum-fill watermark answers "how close did we get". The histogram answers the question an integrator actually has when sizing a FIFO: **how is the fill distributed**. It is a set of saturating 16-bit counters, one per fill threshold, where counter b increments on the **upward** crossing of the level $(b+1) * ETIP_CVS_FIFO_DEPTH / BINS$.

Constraints on the bin count, checked at elaboration: even (two 16-bit bins are read per access), between 2 and 64, and a divisor of the FIFO depth, so that the thresholds are exact integers.

Serial readout. The read-out is deliberately serial rather than random access: one data register `trTeTipFifoHistData` (te offset `0xE14`) returns the bin pair $[2 * RdIdx, 2 * RdIdx + 1]$ and auto-increments the pointer, wrapping after $BINS / 2$ reads. The CSR map is therefore **independent** of the bin count — changing `CT_FIFO_HIST_BINS` needs no RDL regeneration — and the expensive per-register readback is avoided. The control register `trTeTipFifoHistCtrl` (`0xE10`) carries `HistClear` (level: clear counters and pointer, hold at 0; write 0 to re-arm), the self-clearing `RdRewind` (rewind the pointer without clearing, to re-read the histogram) and the read-only `RdIdx`.

Read contract. There is deliberately **no CDC**: the counters live in `tip_clk` and are wired straight to the register read path. They are therefore valid only while the trace is quiescent (`trTeControl.Enable = 0`); a read during tracing may tear. This is a pure diagnostic feature — the on-wire stream is identical with and without it.

Cost. The counter arithmetic carries a synthesis attribute that maps it into otherwise unused DSP slices rather than fabric LUTs. Compiled out, the registers are **omitted** from the register block

entirely – no address decode, no readback, no counters – so a slim profile is exactly neutral. The feature is covered by test 18 including a smoke run at 8 bins (`ct_implementation_status.adoc`).

10.6. Split-Load Data Trace

Elaboration parameter: `SPLIT_DATA_ACCESS` on `ct_encoder`; discovery bit `trTeDataControl.DataSplitLoad`.

A core that reports a load's **address** at retirement but its **data** later, when the memory response arrives, cannot be traced by the combined `dretire` path without either stalling or guessing. Split-load mode handles this case:

Access	Behaviour
Store	Data taken from <code>tip.sdata</code> at <code>dretire</code> .
Load	Address and size captured at <code>dretire</code> ; the data-flow message is emitted later, at the load response (<code>tip.lresp</code>) using <code>tip.ldata</code> together with the saved address and size.

The pending-load state lives in the data-flow qualifier (`PendingLoadHit`) and in the eTIP composer, which holds the captured address and size until the response arrives, and which handles the case of a second load overwriting a still-pending one.

`trTeDataControl.DataSplitLoad` is a **read-only discovery bit**, not a control: it reads 1 exactly when the netlist was built with `SPLIT_DATA_ACCESS = 1`. Software cannot switch the path at run time, because the two paths need different signals from the core.

The mode is implemented but not the default. It is covered by `tests/data/02_split_load`, which elaborates with the parameter set and exercises the store, load, response and pending-overwrite paths.

10.7. Overflow and Overrun Handling

When the eTIP path cannot accept an event, the loss is made explicit rather than silent. An overflow injector on both the serialized and the parallel path turns the drop into an in-band ERROR message and a subsequent resynchronization, so the decoder learns of the gap and re-anchors at a full address instead of drifting (the resynchronization semantics are described in [Chapter 8](#)).

Three defects on this path were found and fixed on 2026-07-20: a counter truncation in the parallel configuration, a wrong resynchronization anchor (full address plus instruction count instead of the address alone), and level-triggered pending clears. The path is covered by `tests/overflow/01_overrun_recovery`, whose gates check three ERROR messages under the overflow stimulus, the correlation messages under the disable stimulus, decoding to the end of the stream and the data verdict; the reference stream is byte-neutral (`ct_implementation_status.adoc`).

Chapter 11. Trace Funnel

11.1. Purpose

The CTTE funnel (`ct_L1_funnel`, `rtl/ct_L1_funnel.sv`) merges several ATB trace streams into one downstream ATB stream. It is the component that lets multiple encoders – in a multi-hart SoC, one encoder per hart – share a single trace sink such as an on-chip ring buffer, an ATB bridge, an Aurora link or the parallel trace output described in [Chapter 12](#).

The funnel is a **transport and arbitration** component. It does not interpret the trace payload beyond what packet-boundary detection requires, and it never rewrites a message. Its guarantees are:

- whole messages only – a channel is never switched in the middle of a packet, so messages from different sources are never interleaved,
- order preservation within each input stream,
- transparent forwarding of `ATDATA` and `ATID`,
- deterministic arbitration (strict priority across levels, round-robin within a level),
- propagation of ATB backpressure to exactly the selected channel,
- per-channel and global flush coordination.

Block diagram

```

ATB IN[0] --+
ATB IN[1] --+
ATB IN[2] ---> ct_L1_funnel --> ATB OUT
ATB IN[N-1]--+

```

11.2. Parameters

The parameters are those of the RTL module (`rtl/ct_L1_funnel.sv`); there are no others.

Parameter	Default	Meaning
<code>N_STREAMS</code>	4	Number of ATB input channels instantiated.
<code>MAX_PRIO</code>	3	Highest priority level. The per-channel priority input is $\lceil \log_2(\text{MAX_PRIO}+1) \rceil$ bits wide; priority 0 disables a channel.
<code>MSEO_WIDTH</code>	2	Nexus MSEO field width. Fixed: an elaboration check rejects any other value.
<code>MDO_WIDTH</code>	30	Nexus MDO field width of the incoming chunks. 30 = one 32-bit chunk per beat (the historical fixed-width reference); 6 and 14 are the byte and halfword chunk configurations of the MSEO/MDO formatter, giving four respectively two chunks per beat. The chunk width is <code>MDO_WIDTH + MSEO_WIDTH</code> and must divide the ATB data width exactly.

Parameter	Default	Meaning
EN_TE_RAW	0	Elaboration switch for E-Trace reference-raw framing support (see Section 11.9). At 0 every <code>te_raw</code> input tie is constant-folded away and the netlist is identical to the Nexus-only funnel.

The ATB widths are **not** parameters of the funnel: `ATB_DATA_WIDTH`, `ATB_BYTES_WIDTH` and `ATB_ID_WIDTH` are taken from `atb_pkg`. The fixed-width reference implementation requires `ATDATA_WIDTH == 32` and enforces it at elaboration.

Table 5. Elaboration checks

Condition	Rejected because
<code>MSEO_WIDTH != 2</code>	The packet-boundary parser decodes Nexus MSEO, which is two bits.
<code>ATDATA_WIDTH != 32</code>	The fixed-width reference funnel emits whole 4-byte beats.
<code>(1 << ATB_BYTES_WIDTH) < 4</code>	<code>ATBYTES</code> must be able to encode a full beat.
<code>ATDATA_WIDTH % (MDO_WIDTH + MSEO_WIDTH) != 0</code>	A beat must carry an integer number of chunks.
<code>EN_TE_RAW && N_STREAMS > 128</code>	The source tag byte encodes at most 128 channels.

11.3. Control and Status

The funnel is configured through **ports**, not through a register block of its own:

Port	Direction	Meaning
<code>chan_prio[N_STREAMS]</code>	in	Per-channel arbitration priority. 0 disables the channel.
<code>chan_flush_participate[]</code>	in	Whether the channel takes part in a global flush.
<code>chan_flush_req[]</code>	in	Per-channel flush request.
<code>chan_flush_done[]</code>	out	Per-channel flush completion (request still asserted and the source has acknowledged).
<code>global_flush_req</code>	in	Global flush request from the system side.
<code>global_flush_done</code>	out	All participating channels have acknowledged.
<code>chan_te_raw[]</code>	in	Per-channel framing select (quasi-static mirror of that source's <code>trTeProtocolSel</code>).
<code>te_tag_always</code> , <code>te_tag_resync</code>	in	Source-tag policy of the merged container (see Section 11.9).
<code>atclk</code> , <code>atresetn</code>	in	Single ATB clock domain and active-low reset.

All input channels and the output channel are synchronous to the same `atclk`; the funnel contains no clock crossing.

Where software-visible control is wanted, the **integration** provides it. In the KV260 Trio demonstrator the SoC control region carries a `FUNNEL_CTRL` register at offset `0x34` whose per-channel fields drive `chan_prio` and are live-writable, taking effect at the next message boundary (examples/dashboard/gen_regmap.py, entry `soc.FUNNEL_CTRL`;

examples/kv260/trio/rtl/trio_soc_top.sv).

NOTE

The reference-PIB funnel CSRs (`trFunnelArbMode`, `trFunnelShed*`) described in [Chapter 12](#) belong to the PIB implementation and are **not** part of this build – `examples/dashboard/gen_regmap.py` states this explicitly. Consequently the funnel has no `trFunnelActive` bit and does not follow the Trace-Control-Interface component reset/discovery pattern; it is reset by `atresetn` and disabled by setting every channel priority to 0.

11.4. Packet Detection

For a Nexus channel, packet boundaries are derived from the chunk-local MSEO bits only.

MSEO value	Meaning for the funnel
START_TRANSMISSION	Start of a packet; the channel enters the in-packet state.
(variable / continuation)	Payload continuation; the beat carries data only while the channel is in a packet.
END_IDLE	End of packet, but only if the channel was in a packet. Outside a packet an <code>END_IDLE</code> chunk is idle, not data.

The parser folds the chunk states across all chunks of one beat, chunk 0 being the lowest bits and therefore the first chunk on the wire. It relies on the chunk packer's end-of-message alignment padding ([Chapter 10](#)): after an `END_IDLE` chunk the rest of the beat is padding, so a beat never carries the end of one message together with the start of the next. For `MDO_WIDTH = 30` this degenerates to the original single-chunk classification.

Idle beats are consumed and dropped. A beat whose chunks carry no in-packet data – flush padding, repeated `END_IDLE` outside a packet – is acknowledged on **every** active channel, not only on the selected one, and is not forwarded. Without this, an unselected source's ATB queue would wedge behind padding it can never hand on.

11.5. Arbitration

Arbitration is evaluated only **between** packets. While the selected channel is inside a packet, or has just presented a beat that starts one, the selection is locked (`current_locked` in the RTL), regardless of what higher-priority channels do in the meantime.

The rules are:

- A channel is **eligible** when its priority is non-zero, `ATVALID` is asserted, and the previewed beat carries data.
- Priority levels are scanned from `MAX_PRIO` down to 1; the first level with an eligible channel wins outright.
- Within a level, selection is round-robin. Each level keeps its own round-robin head pointer, which advances past the channel that just completed a packet.

- Priority 0 disables the channel entirely – it is neither eligible nor served, though idle-beat dropping still applies.

This yields deterministic scheduling and prevents starvation between channels of equal priority. A simulation assertion in the RTL checks the invariant that a selected channel never has priority 0 outside an ongoing packet continuation.

In the **default** configuration all channels carry the same non-zero priority; the funnel then behaves as a plain round-robin, message-atomic funnel and nothing about it is visible to trace tools.

11.6. Backpressure

Only the selected channel observes the downstream `ATREADY`. All other channels see `ATREADY` deasserted – except for the idle-beat drop described above, which is the one case in which an unselected channel is acknowledged. Because a channel is only ever switched on a packet boundary, backpressure can delay a packet but can neither fragment it nor reorder it against another packet of the same source.

The output always presents whole beats: `ATBYTES` is driven to $4 - 1$ for a Nexus channel and to 0 (one valid byte) for a reference-raw channel. `ATDATA` and `ATID` are forwarded unmodified from the selected input; the funnel inserts no channel markers into a Nexus stream.

11.7. Flush

Flush uses the ATB flush handshake. Because the funnel is an ATB **slave** toward its sources and an ATB **master** toward the sink, the signal roles differ on the two sides:

Signal	Side	Role
<code>afvalid</code>	input channel	Driven by the funnel toward the source: flush request to source .
<code>afready</code>	input channel	Driven by the source toward the funnel: flush acknowledge / completion .
<code>afvalid</code>	output	Observed from downstream: external global flush request .
<code>afready</code>	output	Driven by the funnel: global flush completion .

Semantics:

- A per-channel flush request (`chan_flush_req[i]`) raises `afvalid` toward that source until the source acknowledges with `afready`; `chan_flush_done[i]` then reports completion while the request is still asserted. Clearing the request re-arms the channel.
- A global flush is raised either by `global_flush_req` or by the downstream side asserting `afvalid`. It is propagated to every channel that is both active (priority non-zero) and marked as a participant.
- Non-participating channels are treated as immediately done and therefore never block completion. A disabled channel cannot block a global flush.

- `global_flush_done` – and hence the output-side `afready` – asserts only once **all** participating channels have acknowledged. A simulation assertion guards this.
- Flush never interrupts a packet: switching still happens on packet boundaries only.

11.8. Synchronization Request

`syncreq` is broadcast from the output side to **all** input channels: the funnel drives each channel's `syncreq` with the value it observes on `atb_out.syncreq`. A sink that asks for resynchronization therefore reaches every attached encoder, and each encoder decides for itself how to answer – synchronization remains an encoder-level function (see [Chapter 7](#), synchronization generator). The funnel neither combines nor generates synchronization requests of its own.

11.9. Dual-Protocol Operation (E-Trace Reference-Raw Channels)

With `EN_TE_RAW = 1` a channel may be marked `chan_te_raw[i] = 1`, which tells the funnel that this source emits the E-Trace reference-raw framing produced by `ct_l2_te_packetizer`: one byte per beat (`ATBYTES = 0`), a packet being a header byte `{1'b0, msg_type[1:0], payload_len[4:0]}` followed by `payload_len` payload bytes. Packet boundaries come from that length field rather than from MSEO, and the framing has no idle beats. A zero-length packet is a complete packet consisting of the header byte alone.

The reference-raw framing carries no source field, and an on-chip ring that stores `ATDATA` only loses `ATID` as well. The funnel therefore makes the merged byte stream self-describing by inserting a **source tag** byte `8'h80 | channel index` ahead of a packet. The container is called **CTMX**. Splitting it yields per-source byte streams that are byte-identical to a single-core capture and are therefore consumable by an unmodified E-Trace reference decoder; the demultiplexer is `examples/kv260/trio/tools/etrace_trio_demux.py`.

Tag policy:

- By default a tag is emitted on every **source change**.
- `te_tag_always` emits a tag before every packet – more overhead, but resynchronizable at any packet boundary.
- `te_tag_resync` is a level input that forgets which source was tagged last, so the next packet carries a tag again. It should be wired to whatever re-arms the capture, otherwise a capture started mid-stream begins without a source tag.

Payload bytes may themselves have bit 7 set, so the tag is unambiguous only to a state machine that starts at a packet boundary – which is exactly what the demultiplexer does. A tag occupies one output beat during which the input beat is **not** consumed; the selection stays locked because the pending header beat still asserts `start_seen`, and the tag is not duplicated if the sink applies backpressure.

Mixed operation is legal: a Nexus MSEO channel and a reference-raw channel may be merged in the same session — the tag disambiguates the reference-raw part, and `ATBYTES` distinguishes the two framings beat by beat.

11.10. Determinism and Ordering

Packets from one input stream keep their order. Packets from different channels are interleaved according to the arbitration policy, but a packet is never fragmented, never reordered against another packet of the same source, and never altered. Backpressure delays transmission without changing content or order. The merged stream is therefore reconstructible by a decoder that knows the arbitration policy only in the weak sense of "some order" — it does not need to know it at all, because every packet is self-contained and attributed by `ATID` (Nexus) or by the source tag (reference-raw).

11.11. Verification

Three directed testbenches cover the funnel (`rtl/test/`):

Testbench	Covers
<code>ct_L1_funnel_tb.sv</code>	Fixed-width reference (<code>MDO_WIDTH = 30</code> , <code>N_STREAMS = 4</code>): packet switching, strict priority, round-robin, <code>ATID/ATBYTES</code> propagation, backpressure, repeated- <code>END_IDLE</code> ignore behaviour, flush handling.
<code>ct_L1_funnel_mdo6_tb.sv</code>	Byte-chunk configuration (<code>MDO_WIDTH = 6</code> , four chunks per beat, <code>N_STREAMS = 2</code>): multi-chunk beat parsing including mid-beat end-of-message with alignment padding, message-atomic round-robin switching, idle-beat drop, switch locking under backpressure.
<code>ct_L1_funnel_teraw_tb.sv</code>	Dual framing (<code>EN_TE_RAW = 1</code> , <code>N_STREAMS = 3</code>): length-prefixed packet parsing, source-tag insertion on source change and in tag-always mode, zero-length packets, packet atomicity across sources, tag beats under backpressure, mixed MSEO/reference-raw operation.

In addition, the RTL carries non-synthesis assertions for the selected-channel priority invariant, `ATREADY` exposure on unselected channels, `ATDATA/ATBYTES/ATID` agreement with the selected input, the source-tag bit-7 property, no channel switch during a packet continuation, the one-byte-per-beat contract of a reference-raw source, and the global-flush completion condition.

Chapter 12. Parallel Trace Output (PIB)

12.1. Scope

A **Pin Interface Block (PIB)** serializes an ATB trace byte stream onto chip pins. It is the **Trace PIB Sink** component of the RISC-V Trace Control Interface specification (reference 148, Chapter 9), and in either form it is an **additive** output path: it sits downstream of the encoder's 32-bit ATB output and changes neither the encoder, nor the core netlist, nor an existing Ethernet/TSN or Aurora datapath.

Two implementations are in play, and a reader building an example needs to know which one is in front of them.

- **The PIB that ships with the examples** is `examples/kv260/common/ct_soc_pib.sv`, a small sink used by eight of the KV260 designs. It is described first, in [Section 12.2](#), because it is what a reader building from this repository gets.
- **The reference PIB implementation** is a fuller component with its own trace funnel, packet-shedding stage and register model. It is **not part of this repository** and lives in a separate internal engineering project; the rest of this chapter describes it as the design the shipped sink deliberately matches on the wire, so that its hardware adapter fits both.

IMPORTANT

The reference implementation brings its **own** trace funnel (a Trace Control Interface Chapter 8 component with a `trFunnel*` register block). It is a different implementation from the encoder-side funnel described in [Chapter 11](#), which is port-configured and has no register block. The two should not be confused: `trFunnelArbMode` and the `trFunnelShed*` registers described below belong to the reference PIB funnel only, and to no build of this repository.

12.2. The PIB that ships with the examples

`examples/kv260/common/ct_soc_pib.sv` is the parallel off-chip trace port of the KV260 examples. Eight designs instantiate it—`mbv`, `duo`, `trio`, `rocket_linux`, `rocket2`, `cva6_linux`, `cva6_linux64` and `cva6_2`—each with its own pin constraints (`examples/kv260/<name>/fpga/*_pib_pmod.xdc`).

Four properties define it, and each is stated in its own header:

- **Additive observer.** Like the DDR sink, it never back-pressures the trace path. When its small internal FIFO overflows it drops **whole beats**, and it counts them. A dropped beat is therefore visible as a number, not as a corrupted stream.
- **Wire format: five pins, a subset of the reference.** It implements exactly the reference mode `trPibMode = PIB_PAR_4` ("4-bit DDR"), so the existing KR260 PMOD adapter of the

reference implementation can be used unchanged. `pib_data` carries `TRC_DATA[3:0]` LSB first: the low nibble of each byte is stable around the rising port-clock edge, the high nibble around the falling edge. The forwarded source-synchronous `pib_clk` has a period of $2^{(\text{div_i}+1)}$ core clocks—at 75 MHz with `div_i = 1` that is 18.75 MHz. `div_i = 0` is clamped to 1, so the port clock is at most a quarter of the core clock.

- **No frame pin.** Receivers align on the all-ones idle between messages, using the Nexus MSEO structure—exactly as the reference does. While `enable_i` is high the port clock keeps running so a receiver can stay locked, and the data lines hold `0xF`.
- **Byte order is capture order.** Bytes leave in ATB beat order, byte 0 (bits [7:0]) first—the same order the reference decoder consumes from the on-chip ring. A captured PIB stream is therefore byte-identical to the ring content of the same run, which is what makes the two comparable at all.

It also implements the three calibration patterns of the reference (`STANDARD = AA 55 00 FF` per specification table 55 for 4 bits, `MOVING_ONE`, `MOVING_ZERO`); while calibration is active the trace input is not consumed. What it does **not** have is the reference's funnel, its shedding stage or its register block: the control it needs—enable, clear, divider, calibration—arrives as ports from the example's own control window ([Section 35.3](#)).

12.3. The reference PIB implementation

The remainder of this chapter describes the reference implementation. It is the fuller component—funnel, packet-shedding under overload, a complete register model, further output modes—and it is the design whose wire format the shipped sink matches in one mode.

CAUTION

None of what follows is part of this repository or of any build produced from it. It is documented here because it is the contract the shipped sink was built against, and because an integrator evaluating the parallel port needs to know what the full component offers beyond the subset. Where a register, a mode or a behaviour below has no counterpart in [Section 12.2](#), it is not available in the examples.

12.3.1. Block Overview

```
N ATB sources --> Trace Funnel --> Packet-Shedding --> PIB Sink --> { PMOD pins | Aurora }
                    (round-robin /      (overload QoS)      (unpack,
                    static /                                     serialize,
                    dynamic priority)                             calibrate,
                                                                DDR/SDR out)
```

The PIB receives trace via ATB (32-bit `ATDATA`, `ATBYTES`, `ATID`, `ATVALID/ATREADY` backpressure, `AFVALID/AFREADY` flush, `SYNCREQ`), transmits least-significant bit and byte first (`TRC_DATA[0] =` LSB of each beat) and holds the line in the all-ones idle state between messages.

Module decomposition of the reference implementation: `pib_pkg`, the generated APB3 register

`block pib_regs, pib_funnel_input / pib_funnel_arb / pib_funnel, pib_atb_unpack, pib_byte_serializer, pib_calib_gen, pib_timebase, pib_ddr_out, pib_sink` and the glue level `pib_top`.

12.3.2. Output Modes

The WARL register field `trPibMode` selects the pin configuration.

<code>trPibMode</code>	Pins	Description
<code>PIB_OFF (0)</code>	—	Outputs held idle.
<code>PIB_PAR_1 (8)</code>	<code>TRC_CLK + 1</code> <code>TRC_DATA</code>	1-bit parallel.
<code>PIB_PAR_2 (9)</code>	<code>TRC_CLK + 2</code> <code>TRC_DATA</code>	2-bit parallel.
<code>PIB_PAR_4 (10)</code>	<code>TRC_CLK + 4</code> <code>TRC_DATA</code>	4-bit DDR , MIPI / Cortex-M parallel-trace style: low nibble on the rising edge of <code>TRC_CLK</code> , high nibble on the falling edge.
<code>PIB_PAR_8 (11)</code>	<code>TRC_CLK + 8</code> <code>TRC_DATA</code>	Nexus 8-bit direct : <code>TRC_DATA[1:0] = MSEO</code> , <code>TRC_DATA[7:2] = MDO[5:0]</code> , one byte per beat. No funnel required on a single-source path.
<code>PIB_PAR_16 (12)</code>	—	Encoding reserved; not pinned on the KR260 and rejected by WARL.

The serial SWT protocols of the specification (Manchester, UART) reserve their mode encodings but are not implemented in this iteration.

Two further fields shape the timing: `trPibDivider` divides the serialization timebase (input clock / (`trPibDivider + 1`)), and `trPibClkCenter` places a `TRC_CLK` edge at the centre of each `TRC_DATA` bit period, which halves the effective data rate.

The programming sequence is: set `trPibActive`, then `trPibMode` / `trPibDivider` / `trPibClkCenter` / `trPibCalibrate` — at which point `TRC_CLK` starts while the data stays quiescent — and finally `trPibEnable`.

12.3.3. Clocking and Data Rate

Clock	Role
<code>atb_clk</code> (100 MHz)	ATB ingress, funnel, ingress FIFO and byte-stream formatting. Equal to the encoder's ATB clock.
<code>pib_clk</code> (200 MHz)	DDR generation clock; clocks the <code>ODDRE1</code> output stage. One <code>pib_clk</code> period emits two bits per lane.
<code>pib_clk_4x</code> (400 MHz)	Optional, reserved for the fine-grained serializer / phase state machine (sub-byte modes, <code>ClkCenter</code> phase steps). The default build derives all phases from <code>pib_clk</code> .

Table 6. Data rate on PMOD #3, LVCMOS33 (source: the reference implementation's requirements document, "Data-rate summary" — an internal engineering document outside this delivery)

Mode	<code>TRC_CLK</code>	Lane rate	Gross rate
<code>PIB_PAR_4</code> (4-bit DDR)	100 MHz	200 Mb/s	800 Mb/s
<code>PIB_PAR_8</code> (8-bit SDR)	100 MHz	100 Mb/s	800 Mb/s

These figures are the design targets of the KR260 integration and stay inside the 250 Mb/s per pin limit of the HDIO bank stated in the Kria™ K26 SOM data sheet. They are simulation and constraint figures, not measured link rates (see [Section 12.3.12](#)).

12.3.4. Calibration Patterns

With `trPibCalibrate = 1` the PIB stops consuming ingress data and emits a repeating pattern selected by `trPibCalibPattern`:

Pattern	Content
STANDARD (0)	The pattern the Trace Control Interface specification defines for the active mode (Table 55): AA 55 00 FF for the 1/2/4-bit modes, 66 66 CC 33 for the 8-bit mode.
MOVING_ONE (1)	A walking "1": one set lane stepping across the active TRC_DATA width, all other lanes 0, advancing one lane per beat and wrapping. Recorded as a vendor pattern (FR-CAL-03).
MOVING_ZERO (2)	A walking "0": one cleared lane stepping across the active width, all other lanes 1. Vendor pattern (FR-CAL-04).

The two moving patterns exist to bring up and de-skew a parallel link on a scope or logic analyzer: a walking bit makes lane order, lane skew and a stuck or swapped lane visible without a decoder.

12.3.5. Egress Selection

`trPibEgress` routes the trace stream at run time to either the parallel PIB pins (`PINS`) or an existing Aurora serial link (`AURORA`). In Aurora egress the parallel pins are held idle. Egress selection is an additive output multiplexer at the PIB stage and does not modify a legacy build-time datapath selection.

12.3.6. PIB Sink Status and Overflow

The sink applies backpressure by deasserting `ATREADY` when its ingress FIFO cannot accept a full beat; it never accepts a partial beat. If a beat nevertheless arrives while the FIFO is full and not draining, the beat is dropped and the saturating counter `trPibNumOverflows` increments. `trPibFifoMaxFill` tracks the maximum fill level in beats. Both counters have a write-1-clear / write-0-rearm bit (`trPibFifoMaxFillClear`, `trPibNumOverflowsClear`). `trPibEmpty` reads 1 when the internal buffers are empty.

12.3.7. PIB Funnel: Arbitration

The PIB funnel aggregates up to `NUM_FUNNEL_INPUTS` (default 4) ATB sources into one stream. It switches inputs only on a message boundary, so packets from different sources are never interleaved, and a continuous stream on one input does not starve the others. `trFunnelArbMode` selects between three modes:

- **Round-robin (0)** – one whole message per ready input in turn.
- **Static priority (1)** – the input indexed by `trFunnelArbPrioInput` is served first; the others are

served round-robin only while it has nothing pending.

- **Dynamic priority (2)** – at each message boundary the ready input with the highest ingress-FIFO fill wins; ties are resolved round-robin.

`trFunnelDisInput[n]` causes input *n*'s data to be read and discarded and excludes it from arbitration. Clearing `trFunnelEnable` flushes queued trace to the output; `trFunnelEmpty` reads 1 once all internal buffers are empty. `trFunnelGrant` reports the currently granted input, and `trFunnelFillLevel` the fill of the input selected by `trFunnelFillSel`.

12.3.8. PIB Funnel: Overload Packet-Shedding

Packet-shedding is a graceful-degradation QoS layer: under overload it drops **whole low-value messages** instead of arbitrary ATB beats, so the stream stays decodable. It works with a single trace source as well as with several – it is a stream filter, not a funnel feature.

Classification. Each message is classified by its Nexus TCODE, taken from the six MDO bits of the message's first chunk, into the coarse classes SYNC, CF, DF, DAQ and OTHER.

`trFunnelShedClassMask` enables shedding per class (bit 0 = SYNC ... bit 4 = OTHER); the recommended overload setting sheds CF and DF while keeping SYNC and DAQ. ERROR messages (TCODE 8) are never shed, and ERROR messages from different sources pass independently, distinguished by their SRC field.

Modes and hysteresis. `trFunnelShedMode` is OFF (nothing is shed), FORCED (always shed per class mask, regardless of fill) or OVERLOAD (shed only while the downstream ingress fill is high).

OVERLOAD is a Schmitt trigger on the fill level: shedding starts at

`trFunnelShedEnterWatermark`, stops at `trFunnelShedExitWatermark` (which must be less than or equal to the enter level) and holds its state in between.

Gap signalling. Every gap is announced **in band** with a Nexus ERROR message built in exactly the format the encoder uses for its own queue-overflow message – TCODE 8, SRC, ETYPE = QUEUE_OVERRUN, ECODE = the OR of the lost class bits – so a decoder that already handles the encoder's overflow message handles this one too. `trFunnelShedMarkerSrc` attributes the gap to the funnel. A gap-open marker is emitted once at the start of each contiguous gap; an optional gap-close marker (`trFunnelShedGapCloseEnable`) carries ECODE = 0.

CAUTION

The presence of the SRC field must match the encoder's `trTeControl.InhibitSrc` setting – SRC present versus omitted changes the byte layout of the marker. The shedding block builds the marker for the configured layout; a marker built for the wrong layout is not decodable.

Resume semantics. The classes resume differently, because they differ in how self-contained they are:

- **Control flow** is broken by any dropped CF message, so CF non-sync messages stay dropped until a **forwarded I-Sync** re-establishes the PC. The flag is kept **per source** (`cf_resync_pending[SRC]`), so each hart resumes on its own I-Sync while another may still

be waiting.

- **DF, DAQ and OTHER** messages are self-contained and resume the instant shedding clears, optionally with a gap-close marker.

Counters. `trFunnelShedDropped` (messages, saturating), `trFunnelShedGaps` (episodes, saturating) and the sticky `trFunnelShedOccurred`, with a common write-1-clear bit.

12.3.9. Register Map Overview

The PIB block has its own base address in the integration (0xA0A00000 in the KR260 design, APB decode `paddr[23:20] == 0xA`) and is split into two Trace-Control-Interface components:

Offset	Component	CompType
0x0000	Trace Funnel	0x8
0x1000	Trace PIB Sink	0xA

Both follow the specification's two-register discovery header (`tr??Control` at 0x000 with the `Active` bit, `tr??Impl` at 0x004 with `VerMajor/VerMinor/CompType`); the CTTE-specific registers live in the vendor range from 0xE00 upwards. The reference register model is generated from its own SystemRDL by PeakRDL and is never hand-edited; that RDL is authoritative for the reference implementation and the tables in this chapter are a human-readable mirror of it. It is not a source of this repository, and none of these registers exists in a build produced from it.

12.3.10. KR260 PMOD #3 Pinout

For the KR260, the 4-bit DDR output uses PMOD #3 (J19, HDIO bank 46). The Digilent I2C-convention pins (physical pins 3 and 4) are deliberately left reserved so an I2C peripheral can run in parallel.

PMOD pin	Function	SOM signal	FPGA pin	Use
1	GPIO/INT	som240_2_d44	AE12	TRC_CLK, forwarded clock (ODDRE1)
2	GPIO/RST	som240_2_d49	AF11	TRC_DATA[0]
3	SCL	som240_2_d45	AF12	RESERVED – I2C SCL
4	SDA	som240_2_d50	AG11	RESERVED – I2C SDA
7	GPIO	som240_2_d46	AG10	TRC_DATA[1]
8	GPIO	som240_2_c46	AH12	TRC_DATA[2]
9	GPIO	som240_2_d48	AH10	TRC_DATA[3]
10	GPIO	som240_2_c47	AH11	Reserve

I/O standard LVCMOS33, `SLEW_FAST`, `DRIVE_8`; the reference implementation carries its own PMOD constraint file; the shipped sink's constraints are the per-example `*_pib_pmod.xdc` files named in [Section 12.2](#). DDR data and the forwarded clock both leave through ODDRE1 output registers – the trace clock is never routed to the pin directly from the clock tree. For a TRC_CLK at

or above 100 MHz in DDR mode, source-series termination of about 33 Ohm and a cable or adapter shorter than 10 cm are recommended.

The 8-bit Nexus output needs eight data lanes and therefore spans a second PMOD or a Mictor/Samtec adapter; the 4-bit DDR mode fits on PMOD #3 alone with I2C still reserved.

12.3.11. Integration

The block is added as an additional APB slave and taps the encoder's ATB stream. Funnel input 0 is wired to the encoder's ATB master; inputs 1..3 are available for further trace encoders and are tied idle on a single-core design. `trPibEgress = AURORA` routes the PIB's Aurora ATB master into the existing datapath, `= PINS` drives the PMOD outputs. The integration is additive: no change to the encoder, the core netlist or the existing datapath, and no change to the carrier card.

12.3.12. Implementation and Verification Status

The reference RTL is verified by self-checking SystemVerilog testbenches under `xsim`: funnel round-robin fairness, static and dynamic priority, starvation, input disable, flush/empty; serializer bit and byte order for all four widths; idle behaviour; the standard and both moving calibration patterns; 4-bit DDR nibble/edge order and the ClkCenter phase; the 8-bit lane mapping; divider scaling; backpressure, overflow counting and max-fill tracking; the programming sequence; a top-level APB-program-to-pins scoreboard with a DDR sampler model; and corner cases such as partial `ATBYTES`, back-to-back messages without idle, single-byte messages, odd divider values, a mode change while active, and a flush mid-stream.

Item	Status	Evidence
SystemRDL register model	done	the reference implementation's own RDL and its generated register block
Trace Funnel RTL	done	<code>pib_funnel*</code> , <code>tb_pib_funnel</code> green
PIB Sink RTL	done	<code>pib_sink + unpack / serializer / calib / timebase</code> , <code>tb_pib_sink</code> , <code>tb_pib_serializer</code> green
DDR output stage	done	<code>pib_ddr_out</code> , <code>tb_pib_ddr_out</code> green; ODDRE1 for synthesis
Overload packet-shedding	done	<code>pib_shed</code> , <code>tb_pib_shed</code> green
Top integration	done	<code>pib_top</code> , <code>tb_pib_top</code> green (APB + datapath + pins)
PMOD #3 constraints	done	the reference implementation's own constraint file
<code>kr260_top</code> integration hook	documented	additive APB slave + ATB tap
Bitstream / hardware bring-up	out of scope of that iteration	—

IMPORTANT

Simulation is functional evidence only. DDR signal integrity and the 3.3 V LVCMOS timing on the PMOD connector are modelled and constrained, not measured; no on-hardware claim is made for this output path until a bring-up has been logged. A behavioural DDR fallback exists for portable simulation

and is not the synthesis path.

Chapter 13. Trace Format

This chapter describes how execution becomes bytes. Because the encoder can be built with either of two wire formats, the chapter states the protocol split first, then describes each format: for N-Trace the framing that delimits messages on the trace port, the compression applied to program addresses, the meaning of the instruction count and the branch history, and the mapping from a core event to a message; for E-Trace the packet framing and the properties in which it deviates from everything the N-Trace section says. The catalogue of N-Trace message types and their field orders is the next chapter, [Chapter 14](#); this one explains the machinery those messages are built from.

13.1. One Encoder, Two Wire Formats

CTTE emits one of two on-wire trace formats. Which one a given instance emits is fixed at synthesis, not selected at run time ([Section 5.3](#)); the output port `atb_te_raw` advertises the choice to the downstream transport.

- **N-Trace** — Nexus messages per the RISC-V N-Trace specification, framed by the MSEO/MDO scheme. This is the full-featured format: program trace, data trace, instrumentation (Data Acquisition), ownership, error and correlation messages, the vendor configuration message, and the entire compression suite.
- **E-Trace** — `te_inst` packets per the RISC-V Efficient Trace specification, framed by that specification's reference-raw byte framing. CTTE's E-Trace profile is control-flow-only.

Everything **upstream** of the back end is shared: filtering, watchpoints, synchronization requests, instrumentation capture and the AXI-Stream export behave identically in both builds. The split concerns only how events become bytes — and therefore which parts of this manual's format documentation apply:

Topic	N-Trace build	E-Trace build
Wire-format machinery	Section 13.2 (this chapter)	Section 13.3 (this chapter)
Message / TCODE catalog	Chapter 14	not applicable — packet formats are defined by the E-Trace specification, not by a TCODE catalog
Data Acquisition payload	Chapter 15	not emitted (control-flow-only profile)
In-stream self-identification	Chapter 18 (Vendor Config and Device ID messages)	none — the decoder is configured out of band
Message construction from eTIP events	Chapter 8 , generic message generator	Chapter 8 , <code>te_inst</code> generator

13.2. N-Trace Wire Format

Every statement in this section describes the N-Trace back end. One property has to be stated before the format itself, because every statement below is meant to hold for both implementations of it: the N-Trace back end has two serialization paths, and they must produce the same bytes.

13.2.1. Two Serializations, One Format

- The **historical formatter path** (`rtl/ct_L2_nexus_formatter.sv` into `rtl/ct_L2_mseo_mdo_formatter.sv`) handles the full message set, including data trace and instrumentation.
- The **compact packer** (`rtl/ct_L2_compact_packer.sv`) is a lean alternative for control-flow-only builds. Elaboration rejects it in any profile that has data trace or the instrumentation blocks enabled: the lean egress path and the data-flow message formats are mutually exclusive by construction (`rtl/ct_L2_compact_packer.sv:85`).

The two paths are not two formats. They are two implementations of one format, and their byte-for-byte equivalence in the profiles where both elaborate is a verification obligation, not an assumption.

13.2.2. Framing: MSEO and MDO

A Nexus trace is a stream of messages, each a sequence of fields. Framing is carried out of band from the payload, in two dedicated bits per chunk:

- **MDO** — message data output, the payload bits.
- **MSEO** — message start/end/other, the framing bits.

A **chunk** is `MDO_WIDTH` payload bits plus 2 framing bits. The committed configuration uses a 6-bit MDO, so a chunk is one byte (`nexus_vendor_riscv_pkg.sv:136`). Multiple chunks are packed into one egress beat; leftover bits of a beat are padded with all ones.

MSEO	Name	Meaning
00	start of transmission	First byte of a message. Also every byte occupied by a fixed-length field, and every interior byte of a longer variable-length field.
01	end of variable-length field	The last byte of a variable-length field, with the message continuing.
10	reserved	Reserved for future extensions. Not produced.
11	end of message / idle	The last byte of a message. The same code with all payload bits set — a full 0xFF byte — is an idle byte between messages or a padding byte.

Source: `nexus_vendor_riscv_pkg.sv:450`; the generator is `rtl/mseo_mdo/ct_L2_mseo_mdo_formatter_mseo_controller.sv:63`.

NOTE

On the final chunk of a message the encoder emits the end-of-message code 11 rather than the end-of-field code 01, even when that chunk also ends a variable-length field. This is deliberate compatibility with the decoders in circulation, which terminate the current packet as soon as they see the end-of-message code after a variable field (`ct_L2_mseo_mdo_formatter_mseo_controller.sv:73`).

Two MDO widths are implemented: 6 and 14. Any other width aborts

elaboration in both egress paths

```
(rtl/mseo_mdo/ct_L2_mseo_mdo_formatter_bit_slicer.sv:104,
rtl/ct_L2_compact_packer.sv:88).
```

13.2.3. Field Types and Leading-Zero Suppression

Every field is one of four kinds, and the distinction is what a decoder needs to know how to read the next bits. The vocabulary is the Nexus standard's; IEEE-ISTO 5001-2012 Table 4-2 defines **variable**, **vendor-fixed** and **vendor-variable**. The encoder's own enumeration is `nexus_field_type_e` (`nexus_vendor_riscv_pkg.sv:396`); the per-message field lists are in [Chapter 14](#).

Leading-zero suppression applies to every variable-length field. The encoder transmits a field from its least significant bit up to and including its highest set bit, with a minimum length of one bit — an all-zero value therefore occupies one bit, not zero (`LengthWoLeadingZeros`, `nexus_vendor_riscv_pkg.sv:471`). A variable-length field ends on a chunk boundary; if the payload does not fill the last chunk, the remaining slice bits are zero-padded and the next field starts in the following chunk.

Two consequences for a consumer:

1. **A field's transmitted length carries no information about the width of the underlying value.** A 12-bit value and a 32-bit value whose upper 20 bits are zero occupy the same bytes. Where that distinction matters — the architectural address width — the fact is transmitted explicitly rather than inferred; see [Chapter 18](#).
2. **The value must be zero-extended back to its declared width** before being interpreted. This matters most for the instrumentation payload, where a short field is evidence of leading zeros, never of fewer payload elements; see [Chapter 15](#).

13.2.4. Program Address Compression

Program addresses on the wire are compressed in two steps, and both are lossless for RISC-V code.

Shift. Instructions are at least half-word aligned, so bit 0 of an instruction address is always zero. Every program-trace address is right-shifted by one before encoding

(`NEXUS_MSG_PC_ADDR_SHIFT = 1`, `nexus_vendor_riscv_pkg.sv:36`). A decoder shifts back.

Exclusive-or against a rolling reference. A non-synchronizing message carries `U-ADDR`, the exclusive-or of the shifted target address with the shifted address of the previous address-bearing program-trace message. Combined with leading-zero suppression, a branch that stays near the previous one costs very few bits (`GetUaddr`, `nexus_vendor_riscv_pkg.sv:483`). A synchronizing message carries `F-ADDR`, the full shifted address, which re-seats the reference on both sides.

The reference register is kept in shifted coordinates so that the exclusive-or and the emitted values live in the same space (`ct_L2_nexus_formatter.sv:196`). It is re-anchored by every address-bearing message:

TCODE	Message	New reference
9	ProgramTraceSync	current instruction address
11	DirectBranchSync	branch target
4	IndirectBranch	branch target
12	IndirectBranchSync	branch target
29	IndirectBranchHistorySync	anchor address
32	RepeatInstructionSync	loop address
57	Vendor JumpTargetCache	branch target
28	IndirectBranchHistory	branch target

Source: the per-message arms of `ct_L2_nexus_formatter.sv:391-568`.

Two messages deliberately do **not** move the reference, and a decoder must match that: TCODE 3 carries no address at all, and TCODE 56 carries only a count – the decoder resolves the branch targets of a predicted run by walking the code (`ct_L2_nexus_formatter.sv:517`).

TIP

This is one rolling reference for the whole program-trace stream, not one per message type. Data-trace addresses use a **separate** reference, with its own re-anchor contract – see below.

13.2.5. Instruction Count

`I-CNT` is the field that lets a decoder walk from one message to the next. It counts **retired 16-bit instruction units** since the last transmitted count: a compressed instruction counts as one, a standard 32-bit instruction as two, and a longer instruction as its number of half-units. The retiring control-transfer instruction is included in the count of the message that reports it. An exception or interrupt is counted only if it occurred after the retirement of the instruction it is attached to.

NOTE

IEEE-ISTO 5001-2012 Table 4-3 leaves the unit to the architecture: "the actual number of instructions" for fixed-size architectures, or "the number of instruction units" where instructions vary in size. The RISC-V profile fixes that choice to half-words, which is what the encoder implements and what a decoder must assume.

The internal accumulator has a cap, and the cap is selectable. The default is 8 bits; the wide-count feature raises it to 16 (`ct_L2_msg_gen.sv:332`). **This is an internal cap, not a wire format.** The field on the wire is variable-length either way, so raising the cap changes only how often the accumulator has to be drained – long linear code with the narrow cap produces frequent Resource Full messages with the instruction-count reason. That is legal and decodes correctly; it costs bandwidth.

A drain emits Resource Full (TCODE 27) with reason code 0 and the accumulated count, and it emits **the accumulated count, not the count plus the pending event**. The difference matters: the sum that triggers the drain is by construction larger than the cap, so emitting it would put an over-cap value into a field the specification bounds.

13.2.6. Branch History

In history mode the encoder does not emit a message per conditional branch. It accumulates their outcomes into `HIST` and transmits the accumulated field with the next message that carries an address.

The accumulation follows IEEE-ISTO 5001-2012 Table 4-3: the register is pre-loaded with a single 1 that acts as a **stop bit**, a taken branch shifts in a 1, a not-taken branch shifts in a 0 (`ct_L2_msg_gen.sv:1371`). The stop bit is not part of the history; it tells the decoder where the recorded bits end. An empty history is therefore transmitted as the value `0x1`, never as zero.

The field is 30 bits wide including the stop bit (`NEXUS_MSG_HIST_WIDTH`, `nexus_vendor_riscv_pkg.sv:90`). When it fills, it is drained by a Resource Full message with reason code 1. The instruction count is deliberately **not** reset by that drain: it keeps accumulating across the flush, and the next history-bearing message reports the whole span. The decoder subtracts what it walks while resolving the flushed history, so the span is counted exactly once (`ct_L2_msg_gen.sv:1383`).

Reason code 2 is the repeated-history compression: when the same full history window recurs, the run is closed as one Resource Full carrying the pattern and a repeat count in a second data field, instead of one message per repetition.

One special case is worth naming because it looks like an inconsistency in a capture: when the history holds nothing but the stop bit, a reason-1 drain would transmit the payload `0x1`, which the reference decoder discards. The encoder substitutes a reason-0 instruction-count drain instead (`ct_L2_msg_gen.sv:857`).

13.2.7. From Core Event to Message

The mapping depends on the instruction-trace mode. History mode is the reset value and the bandwidth-lean choice; branch mode exists for decoders and workflows that require a message per branch. The mode is selected at run time through `trTeControl.InstMode`, and the compression suite is history-mode only — it is inert in branch mode by construction.

Core event	History mode (<code>InstMode = 6</code>)	Branch mode (<code>InstMode = 3</code>)
Taken conditional branch	one history bit, no message	DirectBranch (3)
Not-taken conditional branch	one history bit, no message	silent, count accumulates
Inferable jump or call	silent, count accumulates	silent, count accumulates
Return, predicted by the return stack	silent, count accumulates	silent, count accumulates

Core event	History mode (<code>InstMode = 6</code>)	Branch mode (<code>InstMode = 3</code>)
Return, not predicted	IndirectBranchHistory (28), or JumpTargetCache (57) on a cache hit	IndirectBranch (4)
Uninferable jump or call	IndirectBranchHistory (28) / JumpTargetCache (57)	IndirectBranch (4)
Interrupt	as above, branch type "interrupt"	IndirectBranch (4), branch type "interrupt"
Trap	as above, branch type "exception"	IndirectBranch (4), branch type "exception"
Return from trap	as above, branch type "indirect"	IndirectBranch (4), branch type "indirect"
Synchronization	ProgramTraceSync (9), DirectBranchSync (11), IndirectBranchSync (12), or IndirectBranchHistorySync (29)	9 / 11 / 12, all history-free
Trace off	Correlation (33) with the trace-disabled event	same
Trace on	next message upgraded to a synchronization with the trace-enable reason	same

Source: the mode arms of `ct_L2_msg_gen.sv:1236` (branch mode) and `:1332` (history mode).

Two substitutions in history mode replace a message rather than add one, and both require the decoder to run a model in lockstep with the encoder:

- **Jump-target cache.** An indirect branch whose target is already in the cache is emitted as TCODE 57 with a cache index in place of the differential address. The reference address still advances to the real target, so later differential addresses stay correct.
- **Branch prediction.** With prediction enabled, direct branches do not accumulate history at all. Both sides run a bit-identical predictor; only a misprediction emits a message, carrying the number of correctly predicted branches before it.

Return from a trap is mapped to the **indirect** branch type rather than to the interrupt type. That is deliberate: the interrupt type would attach the handler body's half-words to a branch type that a consumer reads as "an interrupt happened here" (`ct_L2_msg_gen.sv:1616`).

13.2.8. Data-Flow Mapping

A retired data access produces a Data Trace message: a read produces TCODE 6, a write TCODE 5 (`ct_L2_msg_gen.sv:1843`). Each carries the access size in two fields—the total access size and the element size within it—followed by the address and the value.

CAUTION

The data-trace size fields use the Nexus **byte-count code** (1, 2, 4, 8 as distinct code points). They are not the same encoding as the size field inside the instrumentation payload, which is a base-two logarithm. The two are easy to confuse and a decoder that swaps them produces plausible-looking nonsense. See [Chapter 15](#).

Data addresses have their own compression mode and their own reference register, independent of the program-trace reference:

- **Full** (the reset value): the address is transmitted unmodified. This is byte-identical to the historical stream.
- **Exclusive-or**: the address is exclusive-ored against the previous data-trace message's address (`GetDaddrXor, nexus_vendor_riscv_pkg.sv:493`). No shift is applied — data accesses carry no alignment guarantee.

In the compressed mode the reference has to be re-seated whenever the decoder may have lost track. The first data-trace message after such an event is emitted as the synchronizing form, TCODE 13 or 14, carrying the full address. The re-anchor events are: any synchronizing control-flow message, the rising edge of data tracing, and any Error message (`ct_L2_nexus_formatter.sv:278`). The last one is decoder symmetry: the reference decoder invalidates its data-address reference on an Error, so the encoder must re-seat rather than assume.

The upgrade is a substitution at the emission site: the message generator always selects TCODE 5 or 6, and the formatter overwrites the transfer code when the re-anchor condition holds.

13.2.9. Timestamps

When timestamping is enabled, every message carries a timestamp as its last field. Synchronizing messages carry the **absolute** value; every other message carries the **delta** since the previously emitted timestamp. The baseline is updated on every emission, so a decoder reconstructs the absolute time by accumulating (`ct_L2_nexus_formatter.sv:228`).

The set of messages treated as synchronizing for the timestamp is TCODE 9, 11, 12, 29 and 32; the data-trace synchronizing forms 13 and 14 are added by their own condition, because the message generator never selects them directly.

Two messages carry no timestamp at all:

- The internal flush token, which never reaches the wire anyway.
- **TCODE 30**, the repeat-branch message. This is a decoder constraint rather than an encoder limitation: the reference decoder preserves only the first two fields of the preceding message across a TCODE-30 parse, so a third field would overwrite the saved instruction count of the repeated message and the replay would walk zero instructions (`ct_L2_nexus_formatter.sv:692`).

The contrast with TCODE 56 is instructive: it also carries a single count field, but it **may** carry a timestamp, because to the decoder it is a regular message rather than a replay instruction.

Timestamp width is a build parameter; the field is variable-length and leading-zero-suppressed like every other, so the width bounds the value, not the bytes.

13.2.10. Bandwidth Optimizations Not Implemented

One optimization of the applicable standard chapter is deliberately not implemented: **virtual-address most-significant-bit elision**. The control field is stubbed and commented out in the SystemRDL. The reason is measurement rather than effort: at 32-bit addresses leading-zero suppression is already byte-minimal, so the elision has no benefit to capture. It becomes relevant with 64-bit addresses.

13.3. E-Trace Wire Format (te_inst)

The E-Trace back end emits `te_inst` packets of the RISC-V Efficient Trace specification – baseline algorithm, instruction trace only. The packet payload formats themselves (format 1 branch map, format 2 differential address, format 3 synchronization and support) are defined by that specification and are not repeated here; [Chapter 8](#) describes how CTTE maps eTIP events onto them. What a consumer of the byte stream needs to know about **this implementation** is:

- **Framing.** `ct_L2_te_packetizer (rtl/ct_L2_te_packetizer.sv)` applies the specification's reference-flow **raw** framing: one header byte `{0, msg_type[1:0], payload_len[4:0]}` followed by `payload_len` payload bytes, emitted one byte per ATB beat. A capture of the ATB stream is therefore directly consumable as a raw `te_inst` file by the reference decoder flow.
- **Addresses.** Address fields are sized by `ct_pkg::CT_ETRACE_ADDR_BITS = 31` – a 32-bit instruction address after the specification's one-bit shift. A 64-bit E-Trace build is rejected at elaboration rather than truncated silently ([Chapter 5](#), `genEtrace64`).
- **No instruction count, no MSEO/MDO.** E-Trace counts conditional branches in the branch map and re-anchors with full addresses; nothing from the N-Trace section above – I-CNT, HIST, TCODEs, MSEO/MDO framing, the rolling address reference – applies to an E-Trace stream. A composer instruction-count drain arriving on the eTIP is consumed silently ([Chapter 8](#)).
- **No timestamps.** The specification's optional time field is not implemented; `te_inst` packets carry no time information in this release.
- **No in-stream self-identification.** There is no E-Trace counterpart to the N-Trace Vendor Config message ([Chapter 18](#)). A format 3.3 SUPPORT packet reports the qualification status – trace enable, the trace-lost qualifier on a FIFO-overflow recovery, the ended-reporting qualifier at trace-off – but not the build capabilities; an E-Trace decoder is configured out of band.
- **Sideband requirement.** An E-Trace instance requires the eTIP sideband fields (`ecause`, `tval`, `priv`, `ilastsize`) at full width; a netlist built without them is rejected at elaboration ([Chapter 5](#), `genEtraceNoSideband`).

The funnel's dual-protocol mode transports both framings side by side and keeps them apart by the per-source `atb_te_raw` advertisement ([Chapter 11](#)).

Chapter 14. N-Trace Message Catalog

This chapter is the complete reference for every transfer code (TCODE) the CEDARtools.TraceEncoder knows. TCODEs exist only on the N-Trace wire; an E-Trace stream carries `te_inst` packets instead and has no message catalog of its own (Section 13.3). The chapter answers two different questions that the previous revisions of this manual answered only partly, and never in one place:

1. **Decoding.** Given a TCODE observed in a captured stream, what is it, which specification defines it, and which fields follow it?
2. **Expectation.** Given a build and a runtime configuration, which TCODEs can appear at all? A TCODE that never occurs is as much a property of the configuration as one that does.

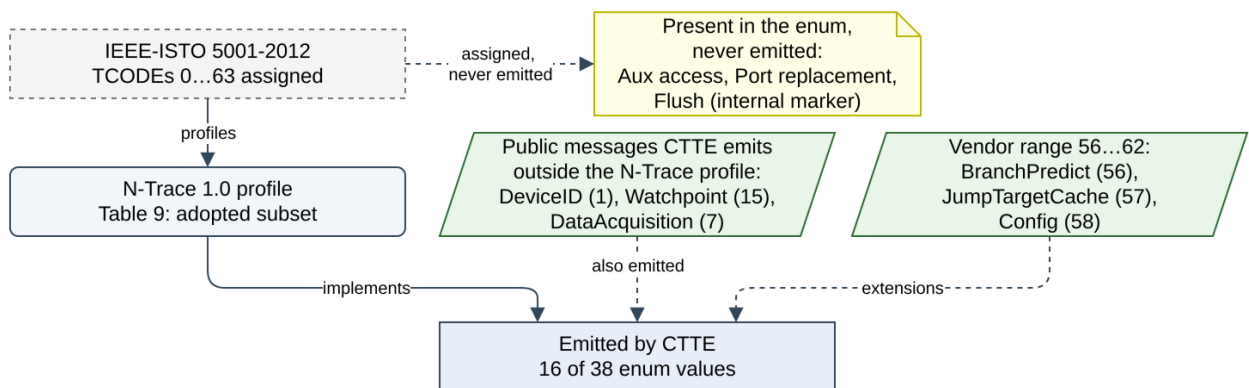


Figure 10. Message space: Nexus, N-Trace profile and vendor range

A TCODE can be a fully defined Nexus public message and still be Reserved from an N-Trace decoder's point of view. Values present in the enumeration are not evidence of emission: the Auxiliary Access and Port Replacement range and the internal flush marker are never sent.

Both answers are derived from the register-transfer sources, not from prose. The authoritative places are:

- `rtl/pkg/nexus_vendor_riscv_pkg.sv:168` — the `nexus_tcode_e` enumeration: the complete set of TCODE constants the design knows.
- `rtl/ct_L2_nexus_formatter.sv:390` — the per-TCODE field composition of the historical egress path; this is where the on-wire field order is decided.
- `rtl/ct_L2_compact_packer.sv:255` — the same layouts in the lean control-flow-only egress path. Both paths must produce identical bytes; the equivalence is a verification obligation, not an assumption.
- `rtl/ct_L2_msg_gen.sv` — the message **selection**: which core event becomes which TCODE.

NOTE

The existence of a constant in `nexus_tcode_e` is **not** evidence that the encoder emits it. The enumeration also carries values that exist for completeness (the

Auxiliary Access and Port Replacement range), values used as internal markers only (NEXUS_MSG_FLUSH), and the two range delimiters of the Nexus vendor space. The **Emitted by CTTE** column below is derived from the message-generation and formatter arms, not from the enumeration.

14.1. Specification baseline used in this chapter

Three primary documents are cited by section and table number:

Short form	Document	Use in this chapter
IEEE-ISTO 5001-2012	IEEE-ISTO 5001™-2012 v3.0.1, <i>The Nexus 5001 Forum™ Standard for a Global Embedded Processor Debug Interface</i>	Table 4-5 (p. 35) assigns every TCODE value; sections 4.3.1 – 4.3.29 define the message formats; Table 4-3 (p. 31) defines the shared fields (TCODE, SRC, I-CNT, HIST, F-ADDR, U-ADDR, TSTAMP).
N-Trace 1.0	<i>RISC-V N-Trace (Nexus-based Trace) Specification v1.0</i> , ratified 2024-11-21	Table 9 (p. 18) lists the message subset the N-Trace profile adopts; every TCODE not listed there is "Reserved for future extensions of N-Trace specification". Table 24 (p. 33) defines the Correlation message, Table 25 (p. 41) the SYNC field values.
TCI 1.0	<i>RISC-V Trace Control Interface v1.0</i>	The control-register semantics referenced from the Emitted by CTTE column; see Chapter 16 .

The N-Trace profile is deliberately narrower than the Nexus standard it profiles. A TCODE can therefore be a fully defined Nexus Public Message **and** be Reserved from an N-Trace decoder's point of view – Device ID (1) and Watchpoint (15) are the two cases where CTTE implements exactly such a message. N-Trace 1.0 Table 9 (p. 18) instructs decoders to ignore Reserved and Vendor Defined messages for program-flow reconstruction and to offer an option to display them or raise a warning; because the MSEO framing is self-delimiting, such a message never breaks the parse.

14.2. Complete TCODE catalog

The table lists all 38 constants of `nexus_tcode_e` (`rtl/pkg/nexus_vendor_riscv_pkg.sv:168–219`) in numerical order.

Reading the last column. `yes` = emitted in every build, no runtime knob. `conditional` = the named compile switch (`rtl/pkg/ct_pkg.sv`) **and**, where one exists, the named runtime control must both be set; the runtime reset value is given because it decides what an unconfigured encoder does. `no` = the design contains no generation arm for this TCODE.

TCODE	Name (<code>nexus_tcode_e</code>)	Defined in	Emitted by CTTE
0	NEXUS_MSG_DEBUG_STATUS	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.1. N-Trace 1.0 Table 9: not listed.	no. No generation arm exists. The message carries a 16-bit status register describing the core's debug state, and the ingress interface has no source for those bits — all three core adapters tie the debug-related inputs to 0. See Chapter 17 and <code>doc/trace-format.adoc</code> (section "Not yet implemented").
1	NEXUS_MSG_DEVICE_ID	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.2. N-Trace 1.0 Table 9: not listed.	conditional — <code>CT_EN_DEVICE_ID</code> (<code>ct_pkg.sv:187</code>) and <code>trTeControl.SendDeviceId = DID_ONCE</code> (reset <code>DID_NONE</code> , Enable-locked WARL). Emitted on every trace-on edge, ahead of TCODE 58 (<code>ct_L23_preproc_composer_etip.sv:582</code> , ordering assertion <code>a_p4_devid_before_cfg at :2172</code>).
2	NEXUS_MSG_OWNERSHIP_TRACE	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.3. N-Trace 1.0 Table 9: adopted ("Ownership").	conditional — <code>CT_EN_OWNERSHIP</code> (<code>ct_pkg.sv:119</code>) and <code>trTeControl.Context = 1</code> (reset 0).
3	NEXUS_MSG_PROGRAM_TRACE_DIRECT_BRANCH	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.4. N-Trace 1.0 Table 9: adopted ("DirectBranch").	conditional — branch-trace mode only: <code>CT_EN_BTM</code> (<code>ct_pkg.sv:225</code>) and <code>trTeControl.InstMode = 3</code> (reset 6 = history mode). <code>ct_L2_msg_gen.sv:1247</code> .
4	NEXUS_MSG_PROGRAM_TRACE_INDIRECT_BRANCH	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.5. N-Trace 1.0 Table 9: adopted ("IndirectBranch").	conditional — branch-trace mode only, same gate as TCODE 3. <code>ct_L2_msg_gen.sv:1299</code> and <code>:1309</code> .
5	NEXUS_MSG_DATA_TRACE_WRITE	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.17. N-Trace 1.0 Table 9: not listed (data trace is outside the N-Trace program-trace profile).	conditional — <code>CT_EN_DATA_TRACE</code> (<code>ct_pkg.sv:48</code>) and <code>trTeDataControl.DataTracing = 1</code> (reset 0). <code>ct_L2_msg_gen.sv:1846</code> .
6	NEXUS_MSG_DATA_TRACE_READ	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.17. N-Trace 1.0 Table 9: not listed.	conditional — same gate as TCODE 5. <code>ct_L2_msg_gen.sv:1843</code> .
7	NEXUS_MSG_DATA_ACQUISITION	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.21 — a Public Message , not a vendor extension.	conditional — <code>CT_EN_DAQ</code> (<code>ct_pkg.sv:47</code>). No enable bit: emission is command-driven through the ACT-CAP / ACT-ST command CSR at core CSR address <code>0x0B10</code> . <code>ct_L2_msg_gen.sv:1865</code> ; payload see Chapter 15 .
8	NEXUS_MSG_ERROR	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.22. N-Trace 1.0 Table 9: adopted ("Error", ETYPE [4] + ECODE [Var]).	yes — no compile switch, no runtime enable. Used for queue overrun and for the data-trace drop announcement. <code>ct_L23_preproc_composer_etip.sv:1012</code> and <code>:1514</code> .

TCODE	Name (<code>nexus_tcode_e</code>)	Defined in	Emitted by CTTE
9	NEXUS_MSG_PROGRAM_TRACE_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.11. N-Trace 1.0 Table 9: adopted ("ProgTraceSync"), and required.	yes. <code>ct_L2_msg_gen.sv:1187 / :1214</code> . The cadence is configurable (<code>trTeControl.InstSyncMode / InstSyncMax</code>); the message itself is unconditional.
10	NEXUS_MSG_PROGRAM_TRACE_CORRECTION	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.12. N-Trace 1.0 Table 9: not listed.	no. No generation arm. The encoder does not emit speculative trace, so there is nothing to correct.
11	NEXUS_MSG_PROGRAM_TRACE_DIRECT_BRANCH_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.6. N-Trace 1.0 Table 9: adopted ("DirectBranchSync").	yes — selected when a synchronization coincides with a direct control-flow event. <code>ct_L2_msg_gen.sv:1192</code> .
12	NEXUS_MSG_PROGRAM_TRACE_INDIRECT_BRANCH_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.7. N-Trace 1.0 Table 9: adopted ("IndirectBranchSync").	yes — selected when a synchronization coincides with an indirect control-flow event. <code>ct_L2_msg_gen.sv:1195</code> .
13	NEXUS_MSG_DATA_TRACE_WRITE_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.18. N-Trace 1.0 Table 9: not listed.	conditional — <code>CT_EN_DF_ADDR_COMPRESS</code> (<code>ct_pkg.sv:164</code>), <code>trTeDataControl.DataAddrCompress = 1</code> (XOR; reset 0), plus the TCODE-5 gate. Produced by TCODE substitution in the formatter, not by the message generator: <code>ct_L2_nexus_formatter.sv:222</code> and <code>:569</code> .
14	NEXUS_MSG_DATA_TRACE_READ_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.18. N-Trace 1.0 Table 9: not listed.	conditional — same gate as TCODE 13, on the read side.
15	NEXUS_MSG_WATCHPOINT	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.23. N-Trace 1.0 Table 9: not listed.	conditional — <code>CT_EN_WATCHPOINT_MSG</code> (<code>ct_pkg.sv:205</code> ; requires <code>CT_EN_ACT</code>), a non-zero <code>trWpMask.WEM</code> (reset 0, Enable-locked) and an ACT-ST command <code>ACT_CAP_ST_WATCHPOINT</code> (command code 14) whose masked result is non-zero. <code>ct_L23_preproc_composer_etip.sv:1040</code> , <code>ct_L2_nexus_formatter.sv:638</code> . See Section 14.3.4 .
20	NEXUS_MSG_PORT_REPLACEMENT_OUT	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.24. N-Trace 1.0 Table 9: not listed.	no. Enumeration constant only; no generation arm anywhere in the design.
21	NEXUS_MSG_PORT_REPLACEMENT_IN	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.25.	no. Enumeration constant only.
22	NEXUS_MSG_AUX_READ	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.26.	no. Enumeration constant only. CTTE is a trace source; it does not implement the Nexus read/write-access service.

TCODE	Name (<code>nexus_tcode_e</code>)	Defined in	Emitted by CTTE
23	NEXUS_MSG_AUX_WRITE	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.27.	no. Enumeration constant only.
24	NEXUS_MSG_AUX_READ_NEXT	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.28.	no. Enumeration constant only.
25	NEXUS_MSG_AUX_WRITE_NEXT	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.28.	no. Enumeration constant only.
26	NEXUS_MSG_AUX_RESPONSE	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.29.	no. Enumeration constant only.
27	NEXUS_MSG_PROGRAM_TRACE_RESOURCE_FULL	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.8. N-Trace 1.0 Table 9: adopted ("ResourceFull", RCODE [4] + RDATA [Var]).	yes — no switch. Drains a full instruction counter (RCODE 0) or a full history buffer (RCODE 1, and RCODE 2 with a repeat count). Twelve generation sites in <code>ct_L2_msg_gen.sv</code> , the most of any message.
28	NEXUS_MSG_PROGRAM_TRACE_INDIRECT_BRANCH_HISTORY	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.9. N-Trace 1.0 Table 9: adopted ("IndirectBranchHist").	conditional — history mode: <code>trTeControl.InstMode = 6</code> (the reset value). The workhorse message of the default configuration. <code>ct_L2_msg_gen.sv:1529 / :1580</code> .
29	NEXUS_MSG_PROGRAM_TRACE_INDIRECT_BRANCH_HISTORY_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.10. N-Trace 1.0 Table 9: adopted ("IndirectBranchHistSync").	conditional — <code>CT_EN_IBHS</code> (<code>ct_pkg.sv:393</code>) and <code>trTeInstFeatures.InstEnIbhs</code> (reset 0). Without it a synchronization in history mode is carried by TCODE 9 / 11 / 12 and the pending history drains separately. <code>ct_L2_msg_gen.sv:1172</code> .
30	NEXUS_MSG_PROGRAM_TRACE_REPEAT_BRANCH	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.13. N-Trace 1.0 Table 9: adopted ("RepeatBranch", B-CNT [Var]).	conditional — <code>CT_EN_REPEAT_BRANCH</code> (<code>ct_pkg.sv:385</code>) and <code>trTeInstFeatures.InstEnRepeatBranch</code> (reset 0). <code>ct_L2_msg_gen.sv:1759</code> .
31	NEXUS_MSG_PROGRAM_TRACE_REPEAT_INSTRUCTION	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.14. N-Trace 1.0 Table 9: not listed — Reserved for future extensions.	conditional — <code>CT_EN_REPEAT_INSTR</code> (<code>ct_pkg.sv:403</code>) and <code>trTeInstFeatures.InstEnRepeatInstr</code> (reset 0). Enabling it takes the stream outside the N-Trace 1.0 message set; see Section 14.2.1 . <code>ct_L2_msg_gen.sv:1786</code> .
32	NEXUS_MSG_PROGRAM_TRACE_REPEAT_INSTRUCTION_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.15. N-Trace 1.0 Table 9: not listed — Reserved for future extensions.	conditional — same gate as TCODE 31. <code>ct_L2_msg_gen.sv:1145</code> .
33	NEXUS_MSG_PROGRAM_TRACE_CORRELATION	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.16. N-Trace 1.0 Table 9: adopted ("ProgTraceCorrelation", EVCODE [4] CDF [2]); field detail in N-Trace 1.0 Table 24.	yes for the trace-off event (EVCODE = 4); conditional for the two other events — EVCODE = 0 (debug entry) needs <code>CT_EN_DEBUG_EVENTS</code> (<code>ct_pkg.sv:99</code>), EVCODE = 1 (low-power entry) needs <code>CT_EN_POWER_EVENTS</code> (<code>ct_pkg.sv:100</code>). <code>ct_L2_msg_gen.sv:993</code> , <code>ct_L23_preproc_composer_etip.sv:638</code> .

TCODE	Name (<code>nexus_tcode_e</code>)	Defined in	Emitted by CTTE
34	NEXUS_MSG_INCIRCUIT_TRACE	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.19.	no. Enumeration constant only.
35	NEXUS_MSG_INCIRCUIT_TRACE_SYNC	IEEE-ISTO 5001-2012 Table 4-5, section 4.3.20.	no. Enumeration constant only.
36	NEXUS_MSG_FLUSH	IEEE-ISTO 5001-2012 Table 4-5: TCODE 36 falls in the Reserved range 36 – 55.	no – internal marker, never on the wire. The message generator produces it (<code>ct_L2_msg_gen.sv:1874</code>) as a pipeline token; both egress paths recognise it and drop it before serialization (<code>ct_L2_mseo_mdo_formatter.sv:174</code> and <code>:213</code> , <code>ct_L2_compact_packer.sv:199</code>). It triggers the ATB flush handshake; no byte of it reaches the trace port.
56	NEXUS_MSG_VENDOR_BRANCH_PREDICT	IEEE-ISTO 5001-2012 Table 4-5: Vendor-Defined Message range 56 – 62. N-Trace 1.0 Table 9: "56..62 Designated for use by Vendor Defined messages".	conditional – CT_EN_BP (<code>ct_pkg.sv:387</code>) and <code>trTeInstFeatures.InstEnBranchPrediction</code> (reset 0). Requires a decoder running the bit-identical predictor model. <code>ct_L2_msg_gen.sv:1820</code> .
57	NEXUS_MSG_VENDOR_JUMP_TARGET_CACHE	Vendor-Defined Message range 56 – 62 (as above).	conditional – CT_EN_JTC (<code>ct_pkg.sv:386</code>) and <code>trTeInstFeatures.InstEnJumpTargetCache</code> (reset 0). Requires a decoder mirroring the cache. <code>ct_L2_msg_gen.sv:1525 / :1576</code> .
58	NEXUS_MSG_VENDOR_CONFIG	Vendor-Defined Message range 56 – 62 (as above).	conditional – CT_EN_CONFIG_MSG (<code>ct_pkg.sv:120</code>) and <code>trTeControl.SendConfig</code> other than <code>CFG_NONE</code> . This is the one message whose full-profile reset is not "off": <code>SendConfig</code> resets to <code>CFG_ONCE</code> . See Chapter 18 .
62	NEXUS_MSG_VENDOR_1_END	Upper bound of the Vendor-Defined Message range (IEEE-ISTO 5001-2012 Table 4-5).	no. Range delimiter, not a message.
63	NEXUS_MSG_VENDOR_EXT	IEEE-ISTO 5001-2012 Table 4-5: Vendor-Defined Extension Message, TCODE 63 (0x3F).	no. Enumeration constant only.

14.2.1. What the reset configuration produces

Read down the **Emitted by CTTE** column with every runtime control at its reset value and the committed full build profile. The message set that can occur is then TCODE 8, 9, 11, 12, 27, 28, 33 and 58 – and nothing else. Every one of those except 58 is adopted by N-Trace 1.0 Table 9; TCODE 58 is a Vendor-Defined message and therefore conformant use of the vendor range, but a decoder that ignores it loses the self-description. Set `SendConfig = CFG_NONE` for a stream that contains adopted TCODEs only.

That is the point of the reset values: an encoder nobody configured produces a stream a stock N-Trace decoder can read. Every message type that leaves that envelope has to be switched on deliberately, twice – once at build time and once at run time.

The messages that leave the N-Trace 1.0 message set when enabled are TCODE 5/6/13/14 (data trace), 7 (Data Acquisition), 15 (Watchpoint), 1 (Device ID), 31/32 (Repeat Instruction) and the vendor TCODEs 56/57/58. For 5/6/7/13/14/15/1/31/32 the relevant statement is narrow and checkable: they are defined Nexus Public Messages (IEEE-ISTO 5001-2012 Table 4-5) that N-Trace 1.0 Table 9 does not list. Only 56/57/58 are Accemic-defined message formats, and they sit in the range the Nexus standard reserves for exactly that purpose.

14.3. On-wire field order

The tables below give the field sequence as the formatter emits it, first field first. All messages are transmitted least significant bit first (IEEE-ISTO 5001-2012 Table 4-2, "Message Transmission Order").

Two fields are governed globally and are therefore not repeated in every row:

SRC. Inserted as the second field of every message unless

`trTeControl.InhibitSrc` is set, and never for the internal flush token

(`ct_L2_nexus_formatter.sv:380`). Its value is

`trTeInstFeatures.SrcID`, its width `trTeInstFeatures.SrcBits`.

TSTAMP. Appended as the last field of every message when `trTsControl.Enable` is set, with two exceptions: the internal flush token, and TCODE 30 (`ct_L2_nexus_formatter.sv:698`). Synchronizing messages carry the absolute timestamp, every other message the delta to the previously emitted TSTAMP; the baseline is updated on every emission. The set of TCODEs treated as synchronizing for this purpose is 9, 11, 12, 29 and 32 (`ct_L2_nexus_formatter.sv:253`); TCODE 13/14 are additionally synchronizing for the data-address reference (`nexus_vendor_riscv_pkg.sv:544`).

The reason TCODE 30 carries no timestamp is a decoder constraint, not an encoder limitation: the reference decoder preserves only the first two fields of the preceding message across a TCODE-30 parse, so a third field would overwrite the saved instruction count of the repeated message (`ct_L2_nexus_formatter.sv:692`).

14.3.1. Program-trace messages

TCODE	Message	Field sequence after TCODE and SRC
3	DirectBranch	ICNT
4	IndirectBranch	BTYPE, ICNT, UADDR
9	ProgramTraceSync	SYNC, ICNT, F-ADDR (current address)
11	DirectBranchSync	SYNC, ICNT, F-ADDR (next address)

TCODE	Message	Field sequence after TCODE and SRC
12	IndirectBranchSync	SYNC, BTYPE, ICNT, F-ADDR (next address)
27	ResourceFull	RCODE, RDATA0; plus RDATA1 if and only if RCODE = 2
28	IndirectBranchHist	BTYPE, ICNT, UADDR, HIST
29	IndirectBranchHistSync	SYNC, BTYPE, ICNT, F-ADDR, HIST
30	RepeatBranch	B-CNT (no TSTAMP— see above)
31	RepeatInstruction	R-CNT, ICNT, HIST
32	RepeatInstructionSync	SYNC, R-CNT, ICNT, F-ADDR, HIST
33	ProgramTraceCorrelation	EVCODE (4 bit), CDF (2 bit, always 1), ICNT, HIST
56	Vendor BranchPredict	B-CNT (count of correctly predicted direct branches)
57	Vendor JumpTargetCache	BTYPE, ICNT, cache index, HIST

Source: `ct_L2_nexus_formatter.sv:391–568`. The HIST and B-CNT payloads travel in the internal `rdata0` / `rdata1` slots of the control-flow message structure; the debug field names of `nexus_field_name_e` therefore read `RDATA0` / `RDATA1` where the specification says HIST / B-CNT / R-CNT.

For TCODE 33 the CDF field is emitted as the constant 2'b01 and the HIST field is always present, encoded as 0x1 when the history is empty.

In history mode (HTM) this matches the specification. N-Trace 1.0 Table 24 requires CDF = 1 in history mode "even if HIST field is empty, encoded as 0x1", so a decoder need not read CDF to know the field exists.

CAUTION

In branch mode (BTM) this is a deviation. Table 24 specifies CDF = 0 there, and the encoder emits 2'b01 regardless of mode (verified in the RTL). A third-party decoder that dispatches on CDF will misread the correlation message in BTM. Because the HIST field is present in either case, a decoder that reads the field unconditionally — as the reference decoder does — is unaffected. The deviation is listed in [Chapter 33](#).

For TCODE 57 the cache index replaces the differential UADDR of TCODE 28. The address reference still advances to the real branch target, so subsequent UADDR differences remain correct; a decoder must mirror the cache to follow (`ct_L2_nexus_formatter.sv:526`).

14.3.2. Data-trace and instrumentation messages

TCODE	Message	Field sequence after TCODE and SRC
5 / 6	DataTraceWrite / Read	DSZ, ELSZ, address slot, DQDATA
13 / 14	DataTraceWrite / ReadSync	DSZ, ELSZ, ADDR (full address), DQDATA
7	DataAcquisition	IDTAG (12 bit, fixed), DQDATA

Source: `ct_L2_nexus_formatter.sv:569–607`. The address slot of TCODE 5/6 carries the

unmodified address while `trTeDataControl.DataAddrCompress` is `FULL` (the reset value), and the exclusive-OR against the previous data-trace address in mode `XOR`. The first data-trace message after a re-anchor event is upgraded to TCODE 13/14 carrying the full address, which re-seeds the reference on both sides. The DAQ payload layout is described in [Chapter 15](#).

14.3.3. Context, identification and error messages

TCODE	Message	Field sequence after TCODE and SRC
1	DeviceID	DEVID
2	OwnershipTrace	PROCESS
8	Error	ETYPE (4 bit), ECODE (8 bit)
15	Watchpoint	WPHIT
58	Vendor Config	CFGVER (4 bit, fixed), CAPS, ENAB, PARAM0, PARAM1, PARAM2, PARAM3

Source: `ct_L2_nexus_formatter.sv:609–682`. The `PROCESS` payload is the packed structure `{CONTEXT, V, PRV, FORMAT}`, least significant field first, per `nexus_process_t` (`nexus_vendor_riscv_pkg.sv:297`). The TCODE-58 payload is documented field by field in [Chapter 18](#).

14.3.4. Watchpoint (TCODE 15) – the bridge from ACT-ST into the stream

TCODE 15 is the message that makes a hardware watchpoint hit visible in the trace stream by **identity** rather than by mere occurrence. The distinction matters for a consumer:

- The trigger path (`trTeControl.InstTrigEnable`, and the Trace Control Interface action 4) marks **that** something fired, by upgrading the next synchronization to `SYNC = 6` (Trace Event).
- TCODE 15 names **which** watchpoint fired, in a bitmap.

The `WPHIT` payload is the bitwise AND of the command's `DirectData[15:0]` and the mask `trWpMask.WEM`, computed in the composer so that the formatter receives the already reportable set (`ct_L2_nexus_formatter.sv:638`). Two consequences follow:

1. The bit-to-slot assignment is a **software convention**. Software gives each watchpoint table entry the `DirectData` bit it wants to see and enables that bit in `WEM`. The hardware imposes no mapping.
2. A command whose masked result is zero emits nothing. With the reset value `WEM = 0` the feature is byte-neutral until software opts in.

The internal payload width is 16 bits

(`NEXUS_MSG_WPHIT_IMPL_WIDTH`, `nexus_vendor_riscv_pkg.sv:76`), matching the 16 mask bits of `trWpMask.WEM`. The on-wire field is variable-length with leading zeros stripped, so the width bounds the

internal payload, not the wire.

NOTE

IEEE-ISTO 5001-2012 contains a conflict in its own description of this message: the prose of section 4.3.23 describes the `WPHIT` field as "masked by 1", which reads as an inverted mask, while Table B-26 defines `WEM` as "1 = enabled". CTTE follows Table B-26 — a 1 enables reporting. A consumer written against the prose reading would invert the mask.

14.4. Enumerated field values

14.4.1. SYNC — synchronization reason

`nexus_sync_reason_e`, `rtl/pkg/nexus_vendor_riscv_pkg.sv:246`. Reference: N-Trace 1.0 Table 25 (p. 41).

Value	Name	Emitted	Meaning / gate
0	External Trace Trigger	conditional	<code>CT_EN_EVTI</code> ; a pulse on the external-trigger input while tracing.
1	Exit from Reset	yes	First synchronization after a reset that did not stop the core.
2	Periodic Synchronization	yes	The cadence configured by <code>trTeControl.InstSyncMode / InstSyncMax</code> . Required by N-Trace 1.0 Table 25.
3	Exit from Debug Mode	conditional	<code>CT_EN_DEBUG_EVENTS</code> . Required by N-Trace 1.0 Table 25 where debug mode is observable.
4	Sequential Instruction Counter	conditional	<code>CT_EN_SEQ_SYNC</code> and <code>trTeControl.InstSeqSyncEnable</code> (reset 0). N-Trace 1.0 Table 25 marks this code "Not required"; the binding to branch-trace mode is stated in the instruction-count section, not in the table itself. using it in history mode as a full synchronization, instead of the leaner instruction-count drain, is a deliberate deviation — see Chapter 4 .
5	Trace Enable	yes	Re-anchor after a trace gap — software pause, filter region entry.
6	Trace Event	conditional	<code>CT_EN_TRIG_SYNC</code> and <code>trTeControl.InstTrigEnable</code> .
7	Restart from FIFO overrun	yes	First synchronization after an announced loss. Required by N-Trace 1.0 Table 25.
8	Reserved	no	Reserved for future standard use (N-Trace 1.0 Table 25).
9	Exit from Powerdown	conditional	<code>CT_EN_POWER_EVENTS</code> .
10, 11	Reserved / Message Contention	no	11 is defined in the enumeration but has no generation site; 10 is reserved for future standard use.

Value	Name	Emitted	Meaning / gate
12, 13	Reserved	no	Reserved for future standard use (N-Trace 1.0 Table 25). Earlier CTTE revisions used these two codes for vendor sync requests; that was corrected – see the note below.
14	Explicit sync request	conditional	CT_EN_INST_SYNC_REQ. Vendor-defined code (N-Trace 1.0 Table 25 assigns 14..15 to vendor codes). Covers all request sources – control register, ACT-CAP command, trace-port request; the source is read back from <code>trTeSyncStatus.SyncReqSource</code> .
15	(no sync)	no	Internal marker, never on the wire.

NOTE

The collapse of the three former CTTE request codes into the single vendor code 14 was a deliberate correction: N-Trace 1.0 Table 25 reserves 10..13 for future **standard** use and assigns only 14..15 to vendors, and the request source is irrelevant to a decoder because every synchronization is a full re-anchor. The source remains readable for diagnosis in `trTeSyncStatus.SyncReqSource` (`nexus_vendor_riscv_pkg.sv:238`).

14.4.2. RCODE – Resource Full reason

`nexus_rcode_e, rtl/pkg/nexus_vendor_riscv_pkg.sv:262.`

Value	Name	On the wire
0	I-CNT overflow	yes – <code>RDATA0</code> carries the drained instruction count.
1	HIST overflow	yes – <code>RDATA0</code> carries the drained history.
2	HIST overflow, repeated	yes – <code>RDATA0</code> carries the history, <code>RDATA1</code> the repeat count. This is the only RCODE that adds a second data field.
3, 4, 5	internal correlation markers	no . Composer-to-generator markers that request a Correlation message (TCODE 33) with a specific <code>EVCODE</code> ; they are consumed by the message generator and never leave it.
15	none	no . Debug value.

Only RCODE 0, 1 and 2 occur on the wire
(`nexus_vendor_riscv_pkg.sv:252`).

14.4.3. EVCODE – Correlation event

`rtl/pkg/nexus_vendor_riscv_pkg.sv:268`. Reference: N-Trace 1.0 Table 24 (p. 33).

Value	Name	Emitted
0	Entry into Debug Mode	conditional – <code>CT_EN_DEBUG_EVENTS</code> . N-Trace 1.0 Table 24 marks this "Required (do not send 4 instead!)".
1	Entry into Low-power Mode	conditional – <code>CT_EN_POWER_EVENTS</code> . Optional per Table 24.
4	Program Trace Disabled	yes – the trace-off event. Optional per Table 24.

14.4.4. BTYPE – branch type

nexus_btype_e, rtl/pkg/nexus_vendor_riscv_pkg.sv:277:

0 = indirect branch, 1 = exception or interrupt, 2 = exception,
3 = interrupt. The message generator maps interrupt-class retirements to
3 and trap-class retirements to 2
(ct_L2_msg_gen.sv:1177).

For a non-control-flow synchronization the encoder emits BTYPE = 0. N-Trace 1.0 section 8.4 sanctions this explicitly ("B-TYPE=0 ... will not mean indirect flow change").

14.4.5. ETYPE and ECODE – error message

nexus_etype_e and the NEXUS_ECODE_* constants,
rtl/pkg/nexus_vendor_riscv_pkg.sv:319 and :321.

ETYPE is 4 bits: 0 = queue overrun caused one or more messages to be lost, 1 = contention with higher-priority messages caused loss.

ECODE is an 8-bit bitmap naming the class of message that was lost:

Bit value	Lost message class
0x01	Watchpoint message
0x02	Data-trace message
0x04	Control-flow message
0x08	Ownership message
0x10	Status message
0x20	Data-acquisition message
0x40	In-circuit-trace message
0x80	Vendor message

IMPORTANT

ECODE = 0x02 is used twice with different consequences. As part of the generic overflow path it announces lost data-trace messages **and** is followed by a SYNC = 7 re-anchor. As the data-drop policy announcement it is emitted **without** a re-anchor, because the instruction trace was not lost and must not be forced to re-anchor for a shed data-trace message. A consumer that treats every Error message as a full desynchronization discards exactly the instruction trace this marker promises is intact. Scope the reaction to the data-trace reference.

14.4.6. DSZ and ELSZ – data-trace access size

nexus_dsz_e and nexus_elsz_e, rtl/pkg/nexus_vendor_riscv_pkg.sv:306 and :314.

Both are derived from the ingress access-size code by `GetDsz()` / `GetElsz()` (:499 and :516), which map the log2 size 0..3 to 1, 2, 4 and 8 bytes. Access sizes above 8 bytes are not supported by the regular Nexus data-trace messages and are rejected in simulation.

14.5. Field type vocabulary

`nexus_field_type_e` (`rtl/pkg/nexus_vendor_riscv_pkg.sv:396`) distinguishes four field kinds that a decoder must treat differently. The vocabulary is the Nexus standard's; IEEE-ISTO 5001-2012 Table 4-2 defines **variable**, **vendor-fixed** and **vendor-variable**.

Kind	Decoder consequence
FIXED	Fixed width defined by the standard; always present.
VENDOR_FIXED	Fixed width chosen by the vendor; may be zero-length, in which case it is absent. A tool must learn the width out of band or from the device identification. CTTE transmits the widths it chose in the Config message (Chapter 18), which removes the out-of-band dependency.
VARIABLE	At least one bit; ends on a packet boundary signalled by the framing. Leading zeros are suppressed.
VENDOR_VARIABLE	Variable-length field whose existence is vendor-determined. Normally reserved for the end of a message.

Leading-zero suppression applies to every variable-length field. A decoder therefore cannot infer a value's width from the field: a 12-bit value and a 32-bit value whose upper 20 bits are zero occupy the same bytes. Where that distinction matters – the address width – the information is transmitted explicitly, as bit 23 of the `CAPS` map.

14.6. Notes and findings

Chapter 15. Data Acquisition Payload (DQDATA)

The Data Acquisition message (TCODE 7) is an N-Trace message: CTTE's E-Trace profile is control-flow-only and has no counterpart to it ([Section 13.3](#)). It is how hardware-supported instrumentation reaches the trace stream. A single command from the instrumented program produces one message whose payload carries what the command asked for – a program counter, a data value, a data address, a performance counter, a caller-supplied tag – packed into the DQDATA field.

This chapter defines that packing. It is the one message whose payload a decoder cannot parse from the message alone: the number and meaning of the payload elements follow from the command, and the command is identified by the IDTAG field. Everything a decoder needs to build that table is here.

NOTE

Which commands carry a payload at all is a property of the **core**, not of this format. The three data-context commands – DAQ_DATA, DAQ_DADDR and DAQ_DATA_DADDR – need a data beat on the trace ingress port, which only the MINRES TGC5B among the four documented cores delivers; on CVA6, Rocket and MicroBlaze™ V they produce no useful payload, because the encoder never receives one ([Section 6.1.3](#)). The commands that report a program counter, a performance counter or a caller-supplied tag are unaffected: their payload comes from the encoder itself.

15.1. What the standard defines and what CTTE fills in

The Data Acquisition message is a **Nexus Public Message**, not a vendor extension. IEEE-ISTO 5001-2012 Table 4-5 assigns it TCODE 7; section 4.3.21 and its Table 4-33 define the format:

Field	Type per Table 4-33	Standard's description
TCODE	fixed, 6 bits	Value 7.
SRC	vendor-fixed	Client that is the source of the message; may be omitted on single-client targets.
IDTAG	vendor-fixed	"Data ID tag. This specifies which group of data is included in the Data Acquisition Message."
DQDATA	variable	"One or more packets of data values."
TSTAMP	vendor-variable	Timestamp; may be omitted.

The standard deliberately leaves the payload semantics open: section 4.3.21 states that "usage of the IDTAG is left to the discretion of the instrumentation software and external development tool", and that messages "may also be generated through vendor-defined mechanisms for instrumenting software".

CTTE fills that discretion in as follows, and this is the whole of the CEDARtools-specific part of this message:

- IDTAG carries the **command code** of the instrumentation command that produced the message.
- DQDATA carries **up to three 64-bit elements**, whose count and meaning are determined by that command code.
- The mechanism that generates the message is the ACT-CAP / ACT-ST command register, reachable from the instrumented program through a core control-and-status register.

The message is otherwise an ordinary Nexus message: it is framed, timestamped and source-tagged like every other, and a decoder that does not know the command table still parses past it correctly.

NOTE

N-Trace 1.0 Table 9 does not list TCODE 7, so a strict N-Trace decoder treats it as reserved and skips it by framing. That is a property of the profile, not a defect of the message – data acquisition sits outside the N-Trace program-trace profile in the same way data trace does.

15.2. IDTAG

IDTAG is a **fixed 12-bit field** (NEXUS_IDTAG_WIDTH, rtl/pkg/nexus_vendor_riscv_pkg.sv:107), emitted as vendor-fixed (ct_L2_nexus_formatter.sv:599). Being fixed, it is **not** subject to leading-zero suppression: it always occupies its full 12 bits, whatever the value.

Its value is the command code, taken directly from the command register's command field (ct_L23_preproc_composer_etip.sv:1062). Software does not set IDTAG separately; it selects a command, and the command code becomes the tag. A caller-supplied identifier does exist, but it rides **inside** the payload as the direct-data element – see below.

15.3. Payload structure

The payload is a fixed-size vector of three elements, packed least significant element first:

Element	Bit range	Note
e10	[63:0]	Present in every command that produces a message.
e11	[127:64]	Present in commands with two or three elements.
e12	[191:128]	Present in three-element commands.

The maximum payload width is 192 bits: three elements of 64 bits

(MAX_DAQ_DATA_ELEMENTS = 3, rtl/pkg/ct_pkg.sv:906;
NEXUS_DQDATA_WIDTH = NEXUS_MSG_DATA_WIDTH = 192 when the
instrumentation or acquisition blocks are built in,
rtl/pkg/nexus_vendor_riscv_pkg.sv:46 and :106).

Elements the command does not use are zero: the composer clears the whole vector before filling it (ct_L23_preproc_composer_etip.sv:1063).

15.3.1. Leading-zero suppression across the whole payload

DQDATA is a variable-length field, and the encoder applies leading-zero suppression to the **entire concatenated 192-bit vector**, not per element (`ct_L2_nexus_formatter.sv:600`). This has a consequence a decoder must get right:

IMPORTANT

A short received DQDATA field is evidence of leading zeros, never of fewer elements. A decoder determines the element count from the IDTAG command code, zero-extends the received bits to `count × 64`, and only then splits the vector into elements. Treating the received length as the payload length produces silently wrong values for any command whose upper elements happen to be small.

The reverse also holds: a command that uses only `e10` still produces a field whose length is decided by the highest set bit of `e10`, so a zero-valued single-element payload occupies one bit.

15.4. Layout per command

The command codes are the enumeration `trActCapStCmd_e` in `rd1/ct_cs_cpuif.rdl`. The table gives, for every command that produces a Data Acquisition message, the element assignment as the composer builds it (`ct_L23_preproc_composer_etip.sv:1064–1110`).

"Direct data" throughout is the 24-bit caller-supplied field of the command word. Its role is the caller's tag; the value 0 means "no tag" and costs nothing on the wire, because leading-zero suppression removes it.

Code	Command	Elements	Payload
1	DAQ_PC_CURR	2	<code>e10</code> = current program counter. <code>e11</code> = direct data.
2	DAQ_PC_CURR_LAST	3	<code>e10</code> = current program counter. <code>e11</code> = the last program counter before the most recent exception or interrupt. <code>e12</code> = direct data.
3	DAQ_DIRECT_DATA	1	<code>e10</code> = direct data, zero-extended. The minimal command: it records that a point in the program was reached, with a caller-chosen value.
4	DAQ_DATA	3	<code>e10</code> = the value of the next data access. <code>e11</code> = its access context (type and size, see below). <code>e12</code> = direct data. The address is not reported.
5	DAQ_DADDR	3	<code>e10</code> = the address of the next data access. <code>e11</code> = its access context. <code>e12</code> = direct data. The value is not reported.
6	DAQ_DATA_DADDR	3	<code>e10</code> = value. <code>e11</code> = address. <code>e12</code> = context and direct data packed together – the only command whose element carries two things; see the packing below.
8	DAQ_IFETCH_TH	1	<code>e10</code> = the instruction-fetch threshold counter of the region selected by the low 8 bits of direct data. The counter is cleared afterwards.
9	DAQ_DATA_RD_TH	1	<code>e10</code> = the read-threshold counter of the selected region, then cleared.

Code	Command	Elements	Payload
10	DAQ_DATA_WR	1	e10 = the write counter of the selected region, then cleared.
11	DAQ_DATA_RD	1	e10 = the read counter of the selected region, then cleared.

Three command codes of the same enumeration produce **no** Data Acquisition message and are listed here only so the table is not read as exhaustive: code 12 requests a synchronization message, code 14 produces a Watchpoint message (see [Section 14.3.4](#)), and code 13 sets an encoder control field. All three are excluded from the acquisition arm explicitly (ct_L23_preproc_composer_etip.sv:1057).

15.4.1. The packed context element of command 6

Command 6 is the only one that needs two payloads in one element, because it already spends e10 and e11 on the value and the address:

Bits	Content	Note
[5:0]	access size	Base-two logarithm of the byte count.
[9:6]	access type	See the type encoding below.
[33:10]	direct data	The caller's 24-bit tag.
[63:34]	reserved	Zero. A decoder ignores these bits but must tolerate them being set by a future revision.

Source: pack_daq_context_direct,
rtl/preproc/ct_L23_preproc_composer_etip.sv:187.

For commands 4 and 5 the same access context occupies its own element, zero-extended: bits [9:0] as above, bits [63:10] reserved.

15.4.2. Access type and size

The access-size field is a **base-two logarithm of the byte count**: 0 is one byte, 1 is two, 2 is four, 3 is eight (rtl/pkg/tip_pkg.sv:243).

CAUTION

This is **not** the encoding of the DSZ and ELSZ fields of the data-trace messages, which are Nexus byte-count codes where the value is the byte count itself. The two fields describe the same physical property with different encodings, they appear in neighbouring messages of the same stream, and confusing them yields values that look plausible. See [Chapter 13](#).

The access-type field is four bits, from the ingress type enumeration (rtl/pkg/tip_pkg.sv:200): 0 = load, 1 = store, 4 – 6 = the three control-register access forms, 8 – 14 = the atomic operations, 15 = failed conditional store. Values 2, 3 and 7 are unused.

For a consumer that only needs the direction, the rule is: **type 0 is a read, every other value is a write-class access.**

15.5. What a decoder needs

The message is not self-describing. A decoder needs:

1. The **command table** above, keyed by `IDTAG`, to know the element count and the element meanings. There is no element-count field on the wire; the message consists of exactly `TCODE`, `SRC`, `IDTAG`, `DQDATA` and `TSTAMP` (verified against the formatter unit test, which asserts a five-field message, `rtl/test/ct_L2_nexus_formatter_tb.sv:574`).
2. **Zero-extension before splitting** — see the suppression rule above.
3. The **access-size distinction** from the data-trace size fields.
4. Tolerance for reserved bits: a set reserved bit is a newer encoder, not a corrupt message.
5. The convention that **direct data 0 means "no tag"**, so a tag-less event is not reported as a tagged event with tag zero.

15.6. The same event on the AXI-Stream path

A command can be routed to the trace port, to the on-chip AXI-Stream egress, or to both. The same event looks different on the two paths, and the difference is not cosmetic:

	Trace port (TCODE 7)	AXI-Stream
Payload width	up to 3 x 64 bit	3 x 32 bit (96-bit beat)
Compression	leading-zero-suppressed, variable-length	none; fixed-format beat
Framing	Nexus chunks, interleaved with the trace stream	one beat, no framing
Command identification	<code>IDTAG</code> field	<code>tid</code> signal
Element validity	implied by the command	<code>tstrb</code> : 0xF, 0xFF or 0xFFF for one, two or three elements
Timestamp	<code>TSTAMP</code> field, per the timestamp rules	optional third element on the current-program-counter command, when the corresponding build switch is set

The AXI-Stream path exists for consumers that process instrumentation inside the fabric, where the bandwidth saved by compression is not worth the decode. See [Chapter 17](#).

Chapter 16. Register Model

This chapter is the control- and status-register reference of the CEDARtools.TraceEncoder. It returned to this manual in revision 2.0: it had been removed in revision 1.2 on the grounds that the register map was maintained in the public repository, which left three chapters of this manual pointing at an interface description that the manual no longer contained.

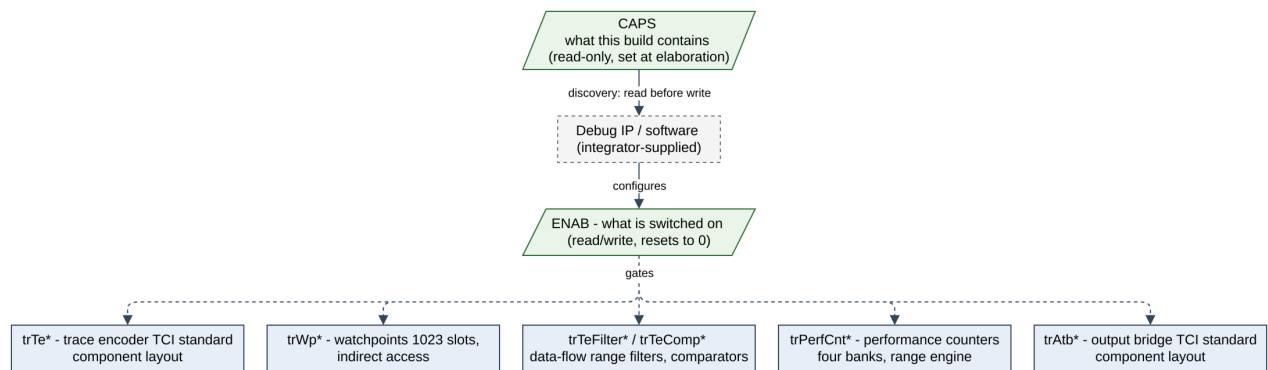


Figure 11. Register space: components and capability gating

Notation: an asterisk denotes the whole register group with that prefix – `trTe*` means every register whose name begins with `trTe`, not a single register called `trTe`.

CAPS is read-only and fixed at elaboration; ENAB is writable and resets to 0. Software reads CAPS before it writes ENAB.

A feature that was not built has no registers at all – reading zero is not how software detects it.

It returns in a different form. The tables are **not written by hand**. They are generated from the SystemRDL source that the hardware itself is generated from, so a register that changes in the hardware changes in this manual in the same step or the build fails. The prose around the tables – address spaces, the capability principle, WARL behaviour, the programming order – is written; the numbers are not.

16.1. Source of truth and generation

`rdl/ct_cs_cpuif.rdl` in the CEDARtools.TraceEncoder repository is the single source of truth for the register layout. From it, `make rdl` generates the SystemVerilog register block (`rtl/pkg/ct_cs_cpuif.sv`, `ct_cs_cpuif_pkg.sv`, `ct_cs_cpuif_types_pkg.sv`, `ct_cs_cpuif_wb_pkg.sv`) and the simulation helper (`tests/lib/ct_cs_cpuif_wb_helper.sv`); the generated SystemVerilog is committed next to the RDL so the repository builds without the RDL toolchain (`rdl/README.md`).

The tables in [Section 16.7](#) come from the same file, through `scripts/gen_ctte_regs.py` in this documentation repository:

```
1 make check-generated # the gate the documentation build runs
```

```
2 python3 scripts/gen_ctte_regs.py # regenerate the fragment
```

On Windows the interpreter is called `py`; `make check-generated` selects whichever of the two the build host provides.

The generator elaborates the SystemRDL with the `systemrdl-compiler`, walks the elaborated map in address order and writes

`src/product_internal/ctte/_generated/registers.adoc`. Two runs over the same input produce byte-identical output: nothing time-, host- or path-dependent enters the file. The only provenance stamp is the SHA-256 of the RDL source text, which the generated fragment carries in its header comment – so a reader can tell which revision of the register map a given printed manual describes.

WARNING

The `--check` mode is wired into the documentation build as the `check-generated` prerequisite of `pre-build`: a stale fragment breaks the build. The check needs both the RDL source and the `systemrdl-compiler` package. Where neither is in reach – on a build agent without a working copy of the encoder repository, for instance – the generator exits with code 3 and the build records the check as skipped, with the reason in the log; a missing prerequisite is never reported as drift. The gate therefore holds on the workstations where the fragment is regenerated. Without it, the fragment can fall behind the RDL silently, which is exactly the failure this chapter exists to prevent.

NOTE

The first thing this mechanism actually found was not drift. Regenerating the KV260 example tables on 2026-08-24 produced a field with an empty value list, and the cause was a defect in the SystemRDL itself: `sink_ctrl.pib_pattern` **declared** the enumeration `pib_pattern_e` and never assigned it, and in SystemRDL it is the field's own `encode` property that binds a type to a field. The defect is invisible everywhere one would look for it – the file compiles, the field reads and writes unchanged, every simulation stays green and the RTL is not affected at all. It shows up only as a missing value list in generated documentation, which is why a documentation generator found it (4feabc2b77c; the same run, with only that one line changed, turns "none of six enumeration strings rendered" into "all six"). The encoder repository now guards the class with `make check-rdl-encode` (`scripts/check_rdl_encode.py`). This is also the argument for generating these tables rather than transcribing them, made with a finding instead of a principle: a transcription would have carried the same missing list and nothing would have contradicted it.

16.2. What the generated tables do and do not tell you

The generated tables state, per field: offset, register, field name, bit range, software access with its qualifiers, reset value, and the description carried in the RDL itself. Four things they deliberately do not state, because the RDL is not where those live:

WARL legalization. Several fields are WARL — write any value, read a legal one. The legalization is **not** in the generated register block; it lives in the Wishbone adapter `rtl/pkg/ct_cs_cpuif_wb.sv`, which is hand-written. The RDL descriptions name the rule for each affected field (`trTeControl.InstMode`, `trTeDataControl.DataAddrCompress`, `trTeControl.SendDeviceId`, `trTeTrigExtInControl.ExtInAction0`), and the rule usually depends on a compile switch: a value is legal **if and only if** the corresponding feature was built in, otherwise the write is legalized to the safe default. That is the register map's honest way of reporting an absent option — writing it and reading back tells you whether the netlist has it.

Build-profile omission. The tables show the committed **full** profile. A slim build regenerates the register block with the profile defines of `rdl/ct_profile.inc.rdl` (produced from the `CT_EN_*` switches by `scripts/gen_rdl_profile.py`). A `CT_PROFILE_NO_<X>` define turns that feature group's configuration fields into `sw=r` constants: storage and write decode vanish, the hardware interface keeps its member with the constant reset value, and reads return the reset value. Two profiles go further and remove registers from the map entirely — `CT_PROFILE_NO_SYNC_STATUS` omits `te.trTeSyncStatus`, and `CT_PROFILE_NO_FIFO_HIST` omits `te.trTeTipFifoHistCtrl` and `te.trTeTipFifoHistData`. What a profile does **not** change is where anything sits: offsets are fixed by the address map, not by the profile, so a slim build has the same addresses as the full one and software compiled against these tables addresses a slim netlist correctly. The difference it sees is the answer, not the address — a compiled-out configuration field reads its reset value and ignores writes.

The Enable lock as a mechanism. The tables mark an Enable-locked field as such, but the lock is a single RDL construct applied in one place: a block of `→swwel = te.trTeControl.Enable` assignments at the end of `rdl/ct_cs_cpuif.rdl`. That block is the authoritative list of what may not be written while the encoder is armed. It is census-checked: `scripts/check_swwel_census.py` counts the write gates in the generated register block against the list, and `make lint` fails if a field leaves the list without either a lock or a documented reason.

Which register a feature belongs to. The mapping from compile switch to runtime control to capability bit is a separate table, and it lives in [Chapter 17](#) — one row per `CT_EN_*` switch.

16.3. Address spaces and components

The map is 32-bit-register-wide throughout (default `regwidth = 32`) and its default access is read-only, so every writable field states its access explicitly. The control interface decodes 15 address bits, i.e. a 32 KiB window (`rtl/pkg/ct_cs_cpuif_wb.sv`).

It is divided into components, each following the RISC-V Trace Control Interface convention that a component carries a `trXxControl` register with an `Active` bit at offset `0x000` of its base and a `trXxImpl` register with version and component type at `0x004`:

Range	Component	Type	Content
0x0000 – 0x0FFF	te – Trace Encoder	0x1 (N-Trace)	Session control, instruction and data trace configuration, timestamp unit, trigger registers, instruction and data filters, comparators, and the diagnostic status registers.
0x1000 – 0x1FFF	atb – Trace ATB Bridge	0xE (N-Trace)	The egress bridge's control and implementation pair.
0x3000 – 0x3FFF	pc – Performance Counters	0x3 (vendor)	The performance-counter block and its region registers.
0x4000 – 0x4FFF	wp – Watchpoints (ACT-ST)	0x4 (vendor)	The watchpoint-message mask and the indirect watchpoint-table access window.
0x5000 – 0x7FFF	df – Data-flow range filter	0x5 (vendor)	The range-filter control pair and the external range memory.

Source: the address-map description in `rdl/ct_cs_cpuif.rdl`. Components `pc`, `wp` and `df` are CEDARtools extensions and say so in their component type; `te` and `atb` follow the Trace Control Interface component numbering.

NOTE

`wp` and `df` appear in the generated tables as registers of the address map rather than of a named group. That is a SystemRDL restriction, not a layout choice: a `mem` instantiation may not live inside a `regfile`, and both components own an external memory, so their registers sit as siblings of that memory in the same window. The generator groups them back together by name prefix.

The convention for reading a component description: a register, field or enumeration value whose description ends in a `[CTTE ...]` marker – `[CTTE specific]`, or a qualified form such as `[CTTE container]` – is a CEDARtools extension. Everything else follows the RISC-V N-Trace control interface.

16.4. The capability principle: CAPS and ENAB

A register map alone cannot tell a **decoder** what the encoder can do, because the decoder sits at the far end of a one-way trace port and never sees the registers. The encoder therefore transmits the same information on the wire, in the Vendor Config message: `CAPS` is the compiled-in capability map, `ENAB` the currently enabled subset, one bit per feature at identical bit positions.

The relationship to this chapter is exact and worth stating once:

- A `CAPS` bit reports a **compile switch** – whether the netlist contains the feature at all. Its register-map counterpart is whether the field is writable or a read-only constant.
- An `ENAB` bit reports a **runtime control** – the value of the field in this chapter's tables. It is computed as the runtime term ANDed with the `CAPS` bit, so a feature that is enabled in a register but not built into the netlist can never appear as enabled on the wire.
- Six positions have no separate runtime enable and therefore mirror `CAPS`. Four more use **mode selection as the enable**: the bit is set when the corresponding field holds a non-default mode rather than by a dedicated enable bit.

The bit-by-bit definition, the mapping to the register fields of this chapter, and the parameter words P0--P3 are in [Chapter 18](#).

16.5. Programming order and the Enable lock

The register model has a bootstrap order, and it is not a style preference: most configuration fields are locked while the encoder is armed, so a program that arms first and configures afterwards silently loses its configuration writes.

1. **trTeControl.Active = 1.** *Active* gates the encoder's clock enables. It must stay set for the entire session, **including the shutdown polling** described in [Chapter 17](#).
2. **Configure.** Write everything that is Enable-locked: instruction mode, source identification, synchronization cadence, timestamp type and prescale, the compression feature bits of `te.trTeInstFeatures`, the filters and comparators, the watchpoint mask, the trigger action. All of these are writable only while `trTeControl.Enable = 0`. Writes after arming are void, and they are void **silently** – there is no error response.
3. **Arm.** Set `trTeControl.Enable`, then `trTeControl.InstTracing` (and `trTeDataControl.DataTracing` for data trace).

What remains writable while the trace runs is a deliberately short list: the session handles (*Active*, *Enable*, *InstTracing*, *DataTracing*, *InstSyncReq*), the status-clear bits (the RW1C overflow flags, the FIFO-histogram controls), the message-class enables (*SendConfig*, *Context*, *InstSeqSyncEnable*, *InstTrigEnable*), the timestamp run control `te.trTsControl`, the data-drop policy `trTeDataControl.DataDropEna`, and the two performance-counter stall thresholds. The performance-counter address **ranges** next to those thresholds are locked, because a range spans two registers and a mid-trace update would be observable as a nonsense interval.

CAUTION

Until 2026-08-16 the eight `te.trTeInstFeatures.InstEn*` bits were **not** Enable-locked – an omission in the `swvel` list found by an external register-access audit. They are locked now. Software written against the earlier behaviour, which configured compression features after arming, stops taking effect against a current netlist. This is the one place where the register contract changed in a way a caller can notice.

`trTeControl.Enable` also gates tracing implicitly: effective instruction tracing is `Enable AND InstTracing`. Clearing `Enable` therefore disables instruction tracing without rewriting `InstTracing`, and the resulting trace-off edge produces the Correlation message described in [Chapter 14](#). Raising it again is a trace-on edge, which re-triggers the one-shot Device ID and Config messages.

16.6. The indirect watchpoint table

The `wp` component is the one place where the register model is not a flat map. The ACT-ST watchpoint table has 1023 slots in the committed elaboration – far too many to expose as a register

window – so it is reached indirectly through a staging protocol:

1. Write the slot number to `trWpIndex.Idx`.
2. Stage the entry through `trWpDataLow (address)` and `trWpDataHigh (command)`; the write to `trWpDataHigh` commits the pair.
3. Read the entry back through `trWpReadLow` and `trWpReadHigh`.

`trWpCap` reports the implemented capacity, so software discovers the table size rather than assuming it. The staging registers are Enable-locked like the rest of the configuration.

The separate register `trWpMask.WEM` decides which slots may report a hit as a Watchpoint message; it is a 16-bit mask, and its reset value 0 means no hit is reported until software opts in. The bit-to-slot assignment is a software convention – see [Section 14.3.4](#).

NOTE

This indirect window replaced a direct 64-bit memory window at `0x4100` and is a breaking change against pre-2026-08 software. The direct window did not scale past the historical 15-slot table.

16.7. Register tables

The tables below are generated. Do not edit them in this document; edit the SystemRDL and regenerate.

The last generated section, "Declared but not addressable", is not part of the address map. It lists the one register the SystemRDL declares and then hides with `ispresent = false`: the ACT-CAP command word `trActCapStCmd`, which exists as a type anchor for the RTL and for the watchpoint table's command word, and which the encoder does not decode on its control bus at all. It is listed rather than dropped because a reader searching this chapter for that name has to find out why it is missing; its full contract, including the CPU-side CSR address it is reachable at, is in [Chapter 20](#).

The register maps of the example designs are not in this chapter. They are separate SystemRDL sources, generated the same way and under the same drift gate, in [Chapter 35](#).

Register, field and enumeration descriptions in the tables below are reproduced verbatim from the SystemRDL source. For the components that the RISC-V N-Trace specification and the RISC-V Trace Control Interface specification (TCI) define, that description follows the specification's own text; both specifications are CC BY 4.0 and are credited in the legal appendix. CEDARtools extensions carry a `[CTTE ...]` marker.

16.8. Elaboration parameters

The register map is parameterised. The values below are the ones the committed SystemRDL elaborates with; a build that overrides a parameter changes the corresponding array length.

Parameter	Value
NUM_TRACE_FILTER	16
NUM_TRACE_COMPARATORS	8
NUM_PERFCNT_IFETCH_TH_RANGES	3
NUM_PERFCNT_DATA_RD_TH_RANGES	3
NUM_PERFCNT_DATA_RD_RANGES	7
NUM_PERFCNT_DATA_WR_RANGES	7
NUM_ACT_ST	1023
NUM_DF_RANGE	15
MAX_I_FILTER	8
MAX_J_FILTER	8
CSC_COUNTER_DATA_RANGE_COUNT	2

16.9. Component overview

Base	Component	Registers
0	te	32
0x1000	atb	2
0x3000	pc	12
0x4000	wp	9
0x5000	df	2
0x6000	mem1	2

16.10. Component te (base 0)

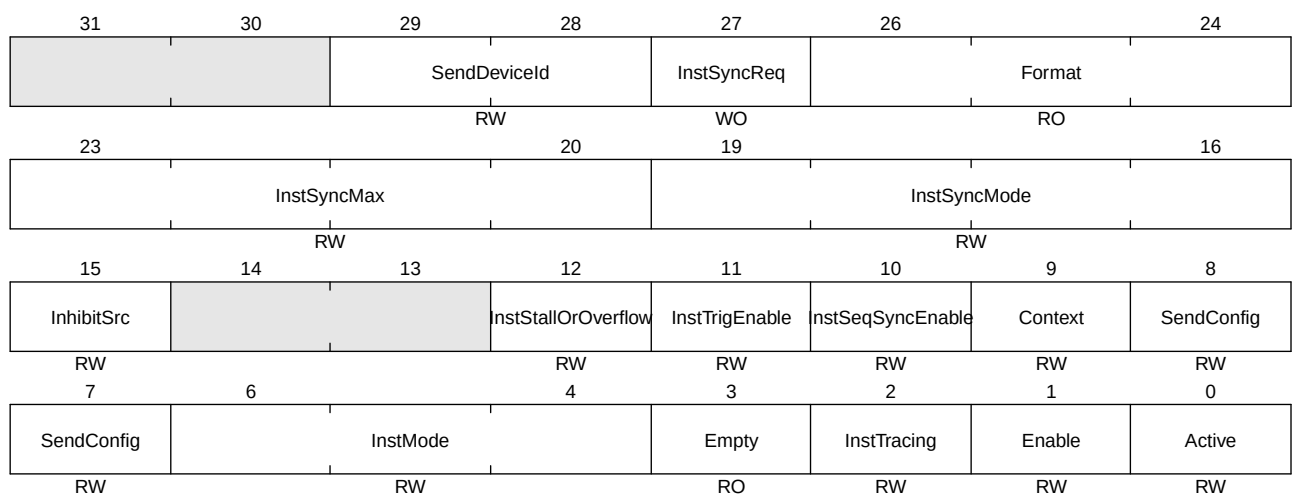
Trace Encoder sub-component (N-Trace compliant, CompType=0x1). See RISC-V N-Trace Control Interface spec Table 4.

Offset	Register	Fields
0x000	te.trTeControl	16
0x004	te.trTeImpl	5
0x008	te.trTeInstFeatures	10
0x00C	te.trTeInstFilters	1
0x010	te.trTeDataControl	7
0x01C	te.trTeDataFilters	1
0x030	te.trTeProtocolSel	1
0x040	te.trTsControl	7
0x048	te.trTsCounterLow	1
0x04C	te.trTsCounterHigh	1

Offset	Register	Fields
0x050	<code>te.trTeTrigDbgControl</code>	1
0x054	<code>te.trTeTrigExtInControl</code>	2
0x058	<code>te.trTeTrigExtOutControl</code>	2
0x400	<code>te.trTeFilter[] .Control</code>	13
0x404	<code>te.trTeFilter[] .Match</code>	2
0x408	<code>te.trTeFilter[] .MatchChoiceEcauseLow</code>	1
0x40C	<code>te.trTeFilter[] .MatchChoiceEcauseHigh</code>	1
0x410	<code>te.trTeFilter[] .MatchValueImpdef</code>	1
0x414	<code>te.trTeFilter[] .MatchMaskImpdef</code>	1
0x418	<code>te.trTeFilter[] .MatchChoiceData</code>	2
0x600	<code>te.trTeComp[] .Control</code>	7
0x608	<code>te.trTeComp[] .SMaskLow</code>	1
0x60C	<code>te.trTeComp[] .SMaskHigh</code>	1
0x610	<code>te.trTeComp[] .PMatchLow</code>	1
0x614	<code>te.trTeComp[] .PMatchHigh</code>	1
0x618	<code>te.trTeComp[] .SMatchLow</code>	1
0x61C	<code>te.trTeComp[] .SMatchHigh</code>	1
0xE00	<code>te.trTeCsrControl</code>	4
0xE04	<code>te.trTeTipFifoStatus</code>	4
0xE08	<code>te.trTeSyncStatus</code>	1
0xE10	<code>te.trTeTipFifoHistCtrl</code>	3
0xE14	<code>te.trTeTipFifoHistData</code>	2

16.10.1. `te.trTeControl` – offset 0x000

Primary control register for the CTTE Encoder



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
29:28	SendDeviceId	RW (Enable-locked)	0	Send a Device ID message (Nexus TCODE 1, IEEE-ISTO 5001 4.3.2) carrying the encoder instance's static identifier (elaboration parameter CT_DEVICE_ID of ct_encoder; layout per ISTO Table B-5 RN PN MID), so a stock Nexus decoder learns which device produced the stream without out-of-band setup. Resets to DID_NONE – deliberately UNLIKE SendConfig: N-Trace 1.0 does not adopt TCODE 1 (Table 9), so the default stream stays strictly N-Trace 1.0 and byte-identical to the pre-P4 reference. Requires the CT_EN_DEVICE_ID build switch; profiles without the feature read DID_NONE. Bit position: the CTTE-vendor area above InstSyncReq – bits [13:12] are reserved for the standard TE status/stall fields (trTelnstStallOrOverflow / trTelnstStallEna, see the commented stubs above). [CTTE specific] Values: 0 = DID_NONE; 1 = DID_ONCE.
27	InstSyncReq	WO (self-clearing)	0	Explicit instruction-trace synchronization request (TCI trTelnstSyncReq). Write-1 semantics: the write raises ONE request and the field clears itself in the next cycle, so it always reads back 0. The request is served by the sync generator on the first qualifying retired instruction and appears on the wire as the explicit-request code SYNC = 14 – the same code the ACT-CAP CF_SYNC command and the ATB sync-request input produce, because the source is irrelevant to a decoder (every synchronization message is a full re-anchor). For diagnosis the source is recorded in trTeSyncStatus.SyncReqSource (value 4, SYNC_REQ_TE). Deliberately NOT gated on trTelnstSyncMode: unlike the ATB request (mode 7) this field is itself the trigger and works in every cadence mode. A request written while instruction tracing is paused is DEFERRED, not dropped: it is served after the resume anchor. Request depth is one in flight plus one remembered: a write arriving while a request is still outstanding is NOT swallowed – it is launched as its own request as soon as the outstanding one has been served and produces its OWN synchronization message. Only a write that arrives when a request is already waiting to be launched collapses into it (it asks for what is about to be asked anyway). Measured on the system test (scripts/cli_tesyncreq_test.sh, legs req/req2/req3): one write → 1 SYNC = 14, two writes back to back → 2, three writes back to back → 2. Requires the CT_EN_INST_SYNC_REQ build switch; without it the write is still accepted and still auto-clears, but nothing consumes it (the pre-2026-08 behaviour). [CTTE specific]
26:24	Format	RO	1	Trace recording format. Only accessible on trTeControl. Enable=0 Values: 1 = TR_FORMAT_N.

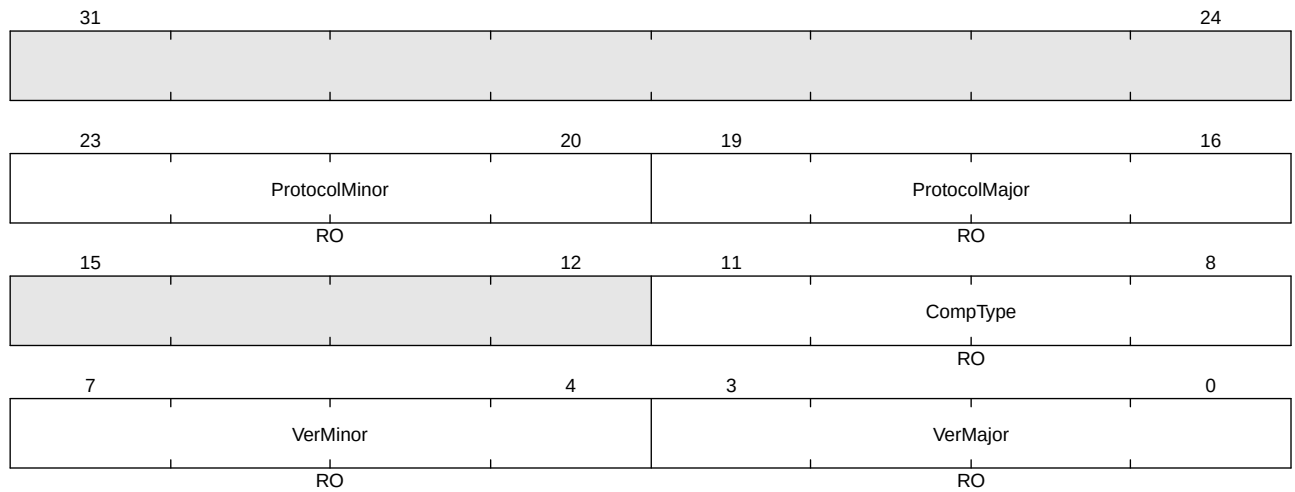
Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
23:20	InstSyncMax	RW (Enable-locked)	6	Maximum interval between sync messages ($2^{(trTeInstSyncMax+4)}$). The unit follows InstSyncMode: on-wire trace messages (mode 1), hart clock cycles (2), instruction half-words (3), emitted ATB trace bytes (4), wallclock cycles (5) or instructions (6). Resets to 6, i.e. a period of $2^{(6+4)} = 1024$ units in whichever unit InstSyncMode selects. The reset value matters because the natural programming path is ONE write to trTeControl ('periodic sync, counted in X'), which takes the mode from software and the period from here. 1024 is the knee of the measured cost curve: the periodic sync then costs 2.5 % of the trace stream, while every wider period can save at most those 2.5 % and doubles the blind window of a consumer that attaches mid-stream. The minimum period (0 = every 16 units) costs a factor 2.4 on the same workload and loses trace against a finite sink – it remains available as an explicit stress configuration, but it is not a product default (P0-02; gate tests/instruction/37_sync_cadence – the raw measurement behind those numbers is not part of this repository). Only accessible on trTeControl. Enable=0
19:16	InstSyncMode	RW (Enable-locked)	0	Periodic instruction trace synchronization mode. This field combines bits 17 and 16 (standard trTeInstSyncMode) with the two reserved bits 19 and 18. Only accessible on trTeControl. Enable=0 Values: 0 = ITR_SYNC_OFF; 1 = ITR_SYNC_TRACE_MSG; 2 = ITR_SYNC_CLK_CYCLES; 3 = ITR_SYNC_HALFWORDS; 4 = ITR_SYNC_TRACE_BYTES; 5 = ITR_SYNC_WALL_CLK; 6 = ITR_SYNC_INSTRUCTIONS; 7 = ITR_SYNC_ATB.
15	InhibitSrc	RW (Enable-locked)	1	Add source field to trace message. Only accessible on trTeControl. Enable=0

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
12	InstStallOrOverflow	RW (WOCLR, hwset, hwclr)	0	Sticky status: set when the encoder generated an OVERFLOW message, i.e. when the eTIP drop path discarded messages and injected the Error(QueueOverrun) marker (N-Trace Required status bit). Write 1 to clear (RW1C); additionally cleared by hardware while trTeControl.Enable = 0, so every trace session starts from a known-clear bit. Pure status – setting or clearing it has NO effect on the wire stream. STALL HALF NOT IMPLEMENTED: the second half of the standard semantics ('or TE requests hart stall') needs a back channel to the hart. The CTTE ingress port (rtl/pkg/tip_if.sv) is UNIDIRECTIONAL – there is no stall/ready signal towards the core – so no CTTE adapter can stall a hart and the companion control bit trTeControl.InstStallEna [13] stays unimplemented (see the commented stub below). The bit therefore reports overflow only. [N-Trace Required, overflow half]
11	InstTrigEnable	RW	0	Enable instruction trace trigger: while set, a pulse on the generic tip.trigger event input upgrades the next retire to a synchronization message with SYNC=6 (Trace Event / watchpoint marker, N-Trace Table 25). Integrators wire tip.trigger to a core watchpoint unit or external trigger fabric; the MBV adapter ties it 0 (integration requirement, see doc/supported_configurations.md).
10	InstSeqSyncEnable	RW	0	Emit a full synchronization message with SYNC=4 (Sequential Instruction Counter) when the internal instruction-count accumulator reaches its cap, instead of the bandwidth-lean ResourceFull(RCODE=0) drain. CAUTION: N-Trace 1.0 allows SYNC=4 only in BTM mode; enabling it in the HTM-only CTTE is a documented Accemic extension (any decoder still treats it as a full re-anchor). Reset 0 = RCODE=0 drains (strictly N-Trace). Applies to the composer-side pre-drain (the dominant path for long linear runs); msg_gen-internal drains stay RCODE=0. [CTTE specific]
9	Context	RW	0	Send Ownership messages (TCODE 2) to indicate processor context: emitted on a context-report ingress event (ctype != 0), on a privilege-level change between retires, and after every synchronizing message (N-Trace 7.1). PROCESS payload = {CONTEXT, V, PRV, FORMAT} per Table 13. [CTTE granularity; TCODE 2 itself is N-Trace standard]

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
8:7	SendConfig	RW	1	Send NEXUS_MSG_VENDOR_CONFIG (TCODE 58) messages: the encoder describes its compiled-in capabilities and runtime configuration on-wire so a decoder can configure itself from the stream (self-describing trace; payload layout in rtl/pkg/ct_pkg.sv). Resets to CFG_ONCE in the full profile (AW E6, seq 24); profiles without the feature read CFG_NONE. [CTTE specific] Values: 0 = CFG_NONE; 1 = CFG_ONCE; 2 = CFG_ON_SYNC.
6:4	InstMode	RW (Enable-locked)	6	Instruction trace generation mode. Only accessible on trTeControl. Enable=0. WARL: legal values are 6 (ITR_BRANCH_HIST, HTM) always, and 3 (ITR_BRANCH, BTM) iff the CT_EN_BT_M build switch is set. With both modes settable, N-Trace Table 8 ('3 OR 6 settable') is fully met. A write of any illegal value is legalized to 6 before it reaches the register; when CT_EN_BT_M=0 a write of 3 is likewise legalized to 6 (honest 'BTM unsupported' read-back). Legalization lives in ct_cs_cpuif_wb.sv (the Wishbone adapter), not in this generated block. [CTTE specific] Values: 3 = ITR_BRANCH; 6 = ITR_BRANCH_HIST.
3	Empty	RO (hw-written)	1	Reads as 1 when all generated trace has been emitted
2	InstTracing	RW (hwset, hwclr)	0	Instruction trace generation. Effective instruction tracing is (trTeControl.Enable AND this bit). The 1→0 edge of effective instruction tracing (this bit cleared, or Enable cleared) makes the encoder emit a Program Trace Correlation Message (TCODE 33, EVCODE=Program Trace Disabled) carrying the residual instruction count and branch history, so a decoder can resolve the final instructions up to the trace-off point; the 0→1 edge emits a TRACE_ENABLE synchronization message to re-anchor the decoder. [CTTE specific]
1	Enable	RW	0	1: Trace Encoder is enabled. Allows trTelnstTracing and trTeDataTracing to turn tracing on and off. Setting trTeEnable to 0 flushes any queued trace data to the sink or funnel attached to this encoder. This bit can be set to 1 only by direct writing to it. This write of 1 should be done after all other settings are done. Enable gates instruction and data tracing: effective instruction tracing is (Enable AND trTelnstTracing). Clearing Enable therefore implicitly disables instruction (and data) tracing without rewriting those bits; the resulting instruction-tracing-disabled edge generates the trace-off Program Trace Correlation Message (see trTelnstTracing). [CTTE: the implicit-disable gating is RTL, not an RDL auto-clear.]
0	Active	RW	0	Primary enable/reset for the Trace Encoder

16.10.2. te.trTeImpl – offset 0x004

Contains version and protocol information for the CTTE Encoder



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
23:20	ProtocolMinor	RO	0	Trace Protocol Minor Version
19:16	ProtocolMajor	RO (hw-written)	1	Trace Protocol Major Version (1 = N-Trace, 2 = E-Trace; driven by the compiled-in back end)
11:8	CompType	RO	1	Trace Encoder Component Type
7:4	VerMinor	RO	0	Trace Encoder Component Minor Version
3:0	VerMajor	RO	1	Trace Encoder Component Major Version

16.10.3. te.trTeInstFeatures – offset 0x008

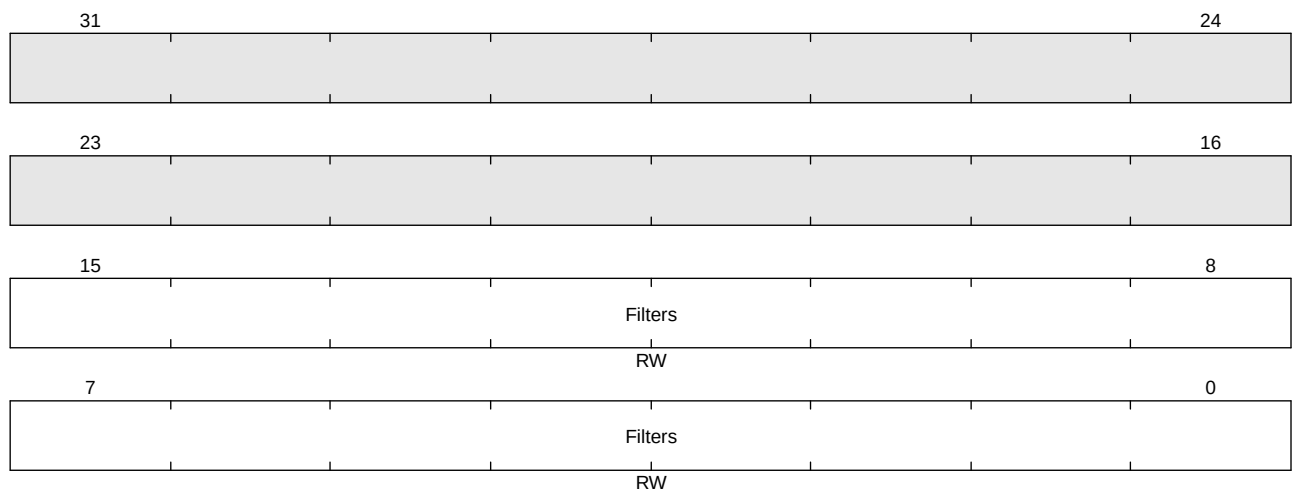
Defines the trace instruction features configuration for the CTTE Encoder. Only accessible on trTeControl. Enable=0



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:28	<code>SrcBits</code>	RW (Enable-locked)	4	Number of bits in the trace source field Writable only while <code>trTeControl.Enable=0</code> .
27:16	<code>SrcID</code>	RW (Enable-locked)	0xA	TE source ID Writable only while <code>trTeControl.Enable=0</code> .
14	<code>InstEnRepeatInstr</code>	RW (Enable-locked)	0	Emit RepeatInstruction messages (TCODE 31/32) for single-instruction spin loops (a taken branch targeting its own address); iterations after the first are counted and the run closes as ONE message with R-CNT instead of one HIST bit per iteration. CAUTION: TCODE 31/32 are IEEE-ISTO-5001-2012 (4.3.14/15); N-Trace 1.0 keeps them reserved – enabling this makes the stream an Accemic/ISTO extension. Reset 0 keeps the stream strictly N-Trace 1.0. Writable only while <code>trTeControl.Enable=0</code> . [CTTE specific]
13	<code>InstEnIbhs</code>	RW (Enable-locked)	0	Emit IndirectBranchHistorySync messages (TCODE 29): synchronizing messages carry the pending branch history instead of pre-flushing it as a separate ResourceFull(RCODE=1) – one message where two went before (bandwidth win). 0 = historical two-message form. Writable only while <code>trTeControl.Enable=0</code> . [CTTE granularity; TCODE 29 itself is N-Trace standard]
12	<code>InstEnWideIcnt</code>	RW (Enable-locked)	0	Widen the internal ICNT cap from 8 to 16 bit (fewer ResourceFull ICNT drains; wire ICNT stays variable-length per Nexus). Writable only while <code>trTeControl.Enable=0</code> .
11	<code>InstEnRepeatBranch</code>	RW (Enable-locked)	0	Enable RepeatBranch messages (TCODE 30) for identical repeated IndirectBranchHistory messages. Writable only while <code>trTeControl.Enable=0</code> .
8	<code>InstEnRepeatedHistory</code>	RW (Enable-locked)	0	Enable repeated branch history. Writable only while <code>trTeControl.Enable=0</code> .
5	<code>InstEnJumpTargetCache</code>	RW (Enable-locked)	0	Enable jump target cache (vendor TCODE 57: target as cache index instead of UADDR). Writable only while <code>trTeControl.Enable=0</code> .
4	<code>InstEnBranchPrediction</code>	RW (Enable-locked)	0	Enable branch prediction (vendor TCODE 56: mispredict count instead of HIST bits; excludes <code>InstEnRepeatedHistory/InstEnRepeatBranch</code>). Writable only while <code>trTeControl.Enable=0</code> .
3	<code>InstEnImplicitReturn</code>	RW (Enable-locked)	0	Treat returns as inferable PC discontinuities when returning from a recent call on a stack. Writable only while <code>trTeControl.Enable=0</code> .

16.10.4. `te.trTeInstFilters` – offset 0x00C

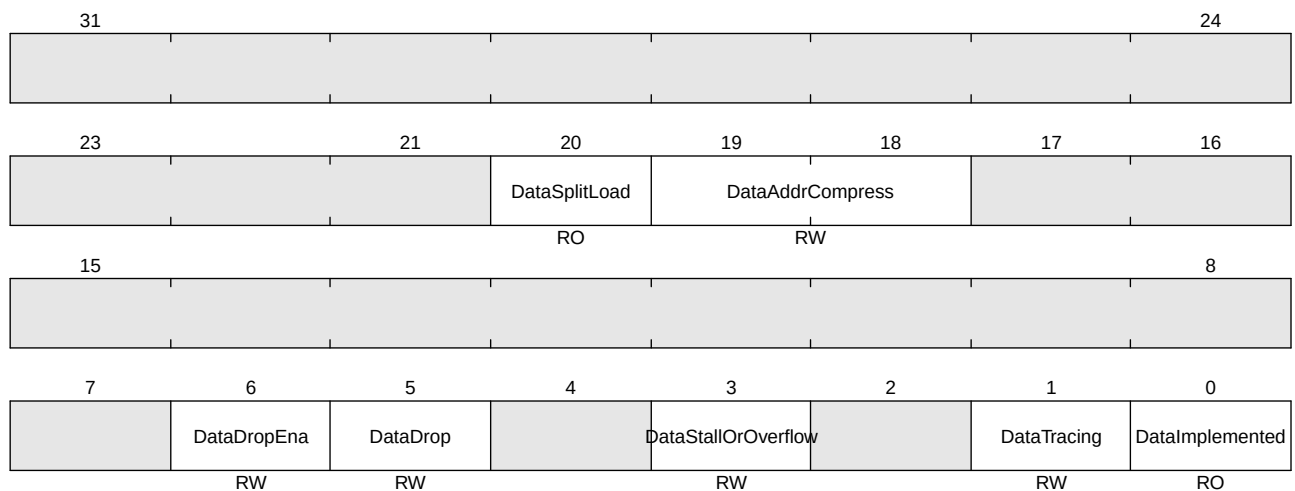
Defines filters for qualifying control flow CTTE generation. Only accessible on `trTeControl`. `Enable=0`



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15:0	<code>Filters</code>	RW (Enable-locked)	0	Determine which filters qualify an instruction trace. Writable only while <code>trTeControl.Enable=0</code> .

16.10.5. `te.trTeDataControl1` – offset 0x010

Controls data trace behavior



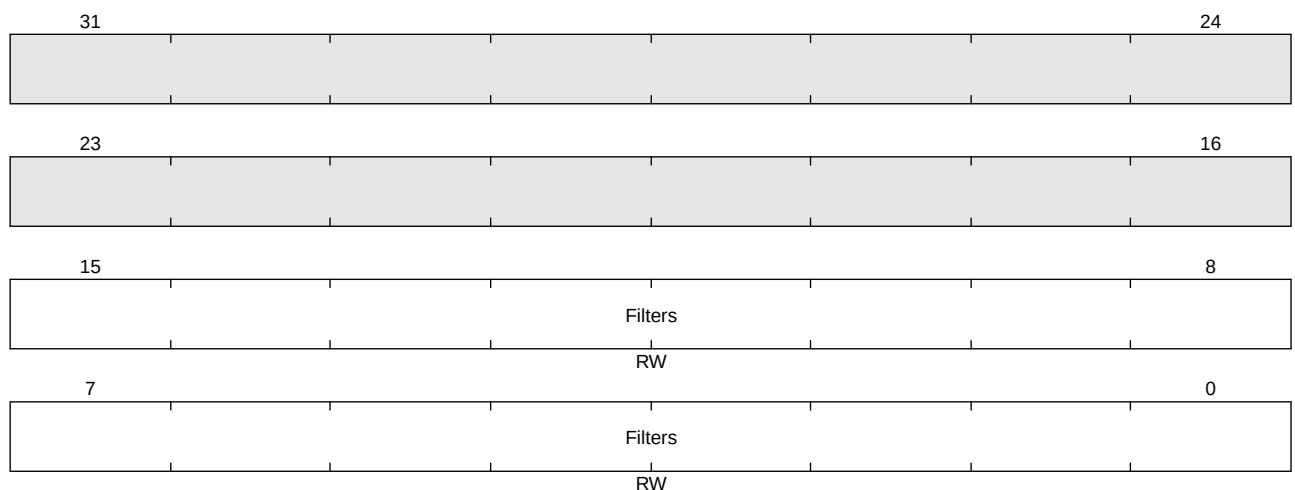
Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
20	<code>DataSplitLoad</code>	RO (hw-written)	0	Read as 1 if split-load tracing is implemented (sdata/lresp/ldata path). Force to 1 in generated RTL when <code>SPLIT_DATA_ACCESS</code> parameter=1 (see <code>ct_cs_cpuif.sv</code>). [CTTE specific]

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
19:18	DataAddrCompress	RW (Enable-locked)	0	Data trace address compression mode. 0 = full addresses (reset; byte-identical legacy stream). 1 = XOR against the previous data-trace message's address; the first DF after a re-anchor event (any synchronizing message, a DataTracing rising edge, any ERROR message, reset) is emitted as the synchronizing TCODE 13/14 form carrying the full address. Modes 2/3 are not implemented (sign/encodability, see the reference manual) – WARL: writes of 2/3 (and of 1 when the CT_EN_DF_ADDR_COMPRESS build switch is 0) legalize to 0. Writable only while trTeControl.Enable=0. Values: 0 = DTR_ADDR_FULL; 1 = DTR_ADDR_XOR.
6	DataDropEna	RW	0	Allow dropping data trace to avoid instruction trace overflows. While set, the encoder suppresses data-trace messages as soon as the eTIP queue fill reaches the drop watermark (ct_pkg::CT_DF_DROP_WATERMARK = 3/4 of the queue depth), so the remaining capacity stays available for the instruction trace. Each drop episode is announced ON THE WIRE by ONE Error message (TCODE 8) with ETYPE = QueueOverrun and ECODE = 0x02 (DataMessageLost) – deliberately WITHOUT the ProgramTraceSync (SYNC=7) re-anchor of the generic overflow path: forcing an instruction re-anchor for every dropped data message would defeat the purpose of the policy. RUNTIME policy, therefore NOT gated by trTeControl.Enable (unlike the configuration fields): software may arm or disarm the drop while the trace is running. Reset 0 = never drop (byte-identical legacy behaviour); reads 0 and is not writable in a build without CT_EN_DF_DROP. Advertised in the vendor config message as CAPS/ENAB bit 22 (DF_DROP).
5	DataDrop	RW (WOCLR, hwset, hwclr)	0	Sticky status: set when the DataDropEna policy actually dropped data-trace messages (one set event per drop episode). Write 1 to clear (RW1C); additionally cleared by hardware while trTeControl.Enable = 0. Reads 0 in a build without the CT_EN_DF_DROP switch (nothing can set it).

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
3	<code>DataStallOrOverflow</code>	RW (WOCLR, hwset, hwclr)	0	Sticky status: set when data-trace messages were LOST – either because the generic eTIP drop path discarded a beat (the same event that sets <code>trTeControl.InstStallOrOverflow</code>) or because the <code>DataDropEna</code> policy suppressed data-trace messages at the watermark. Write 1 to clear (RW1C); additionally cleared by hardware while <code>trTeControl.Enable = 0</code> . STALL HALF NOT IMPLEMENTED – same reason as <code>trTeControl.InstStallOrOverflow</code> : the ingress port is unidirectional, so the encoder cannot stall a hart and <code>DataStallEna</code> [4] stays unimplemented. [CTTE specific]
1	<code>DataTracing</code>	RW (hwset, hwclr)	0	Data trace is being generated
0	<code>DataImplemented</code>	RO	1	Read as 1 if data trace is implemented

16.10.6. `te.trTeDataFilters` – offset 0x01C

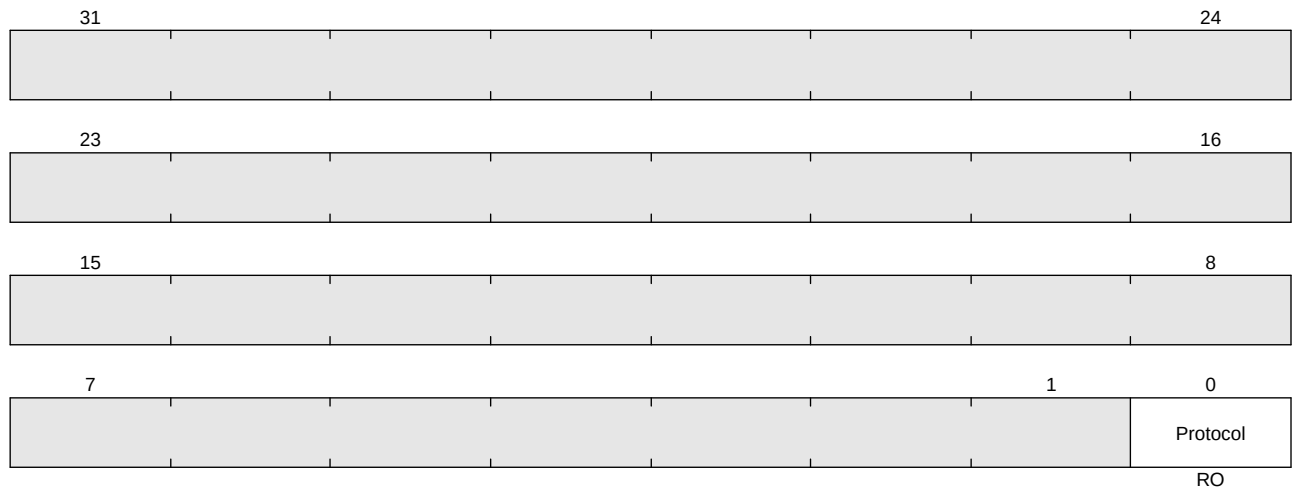
Defines filters for qualifying data trace generation. Only accessible on `trTeControl`. `Enable=0`



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15:0	<code>Filters</code>	RW (Enable-locked)	0	Determine which filters qualify a data trace Writable only while <code>trTeControl.Enable=0</code> .

16.10.7. `te.trTeProtocolSel` – offset 0x030

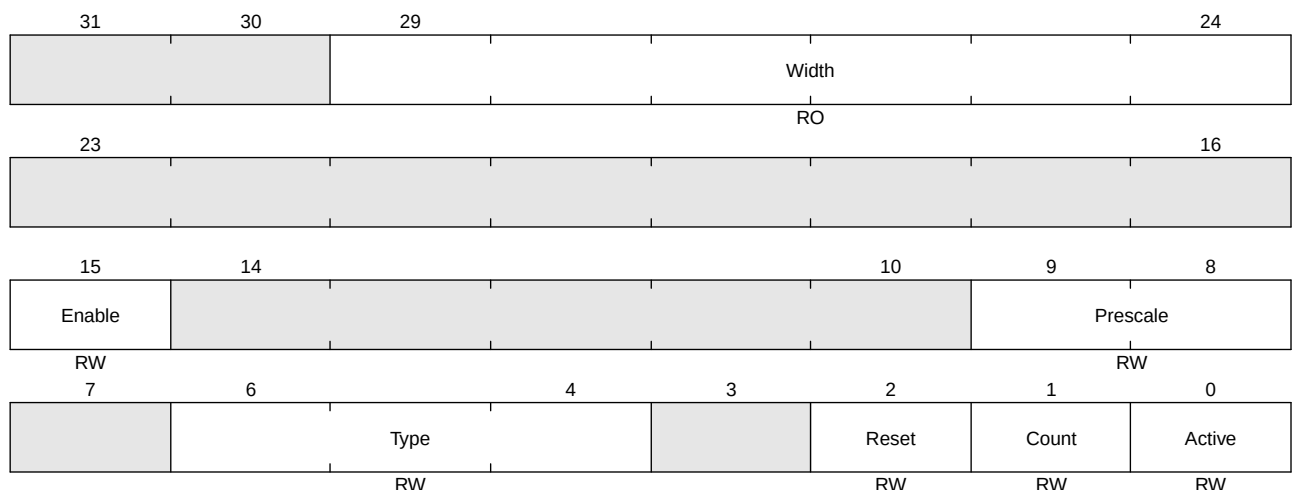
Active trace protocol: 0 = N-Trace 1.0 (Nexus), 1 = E-Trace 2.0 (`te_inst`). Read-only discovery of the compiled-in back end (synthesis parameter, per encoder instance).



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
0	Protocol	RO (hw-written)	0	Compiled-in trace protocol of this encoder instance

16.10.8. te.trTsControl1 – offset 0x040

Controls the behavior of the timestamp unit

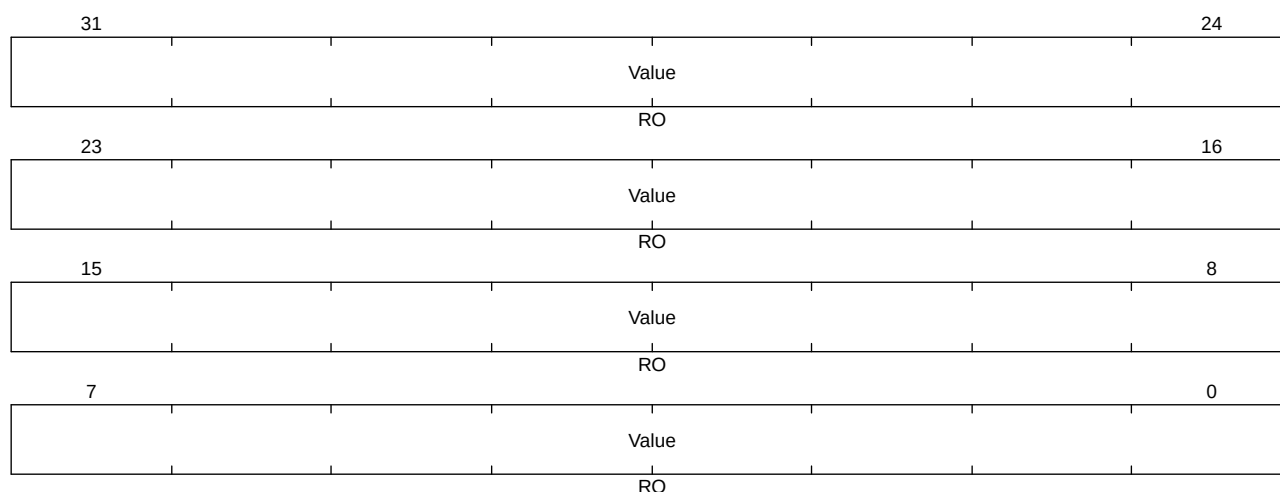


Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
29:24	Width	RO	0x3F	Width of timestamp in bits (0..63). Only accessible on <code>trTeControl</code> . Enable=0
15	Enable	RW	1	Global enable for timestamp field in trace messages/packets
9:8	Prescale	RW (Enable-locked)	0	Internal timestamp only. Prescale timestamp clock by 2^{2n} (1, 4, 16, 64). Only accessible on <code>trTeControl</code> . Enable=0
6:4	Type	RW (Enable-locked)	0	Type of Timestamp unit. Only accessible on <code>trTeControl</code> . Enable=0 Values: 0 = <code>TR_TS_NONE</code> ; 2 = <code>TR_TS_SYSTEM</code> ; 3 = <code>TR_TS_CORE</code> .
2	Reset	RW	0	Internal timestamp only. Write 1 to reset the timestamp counter

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
1	Count	RW	0	Internal timestamp only. 1: counter runs, 0: counter stopped
0	Active	RW	0	Primary reset/enable for timestamp unit

16.10.9. `te.trTsCounterLow` – offset 0x048

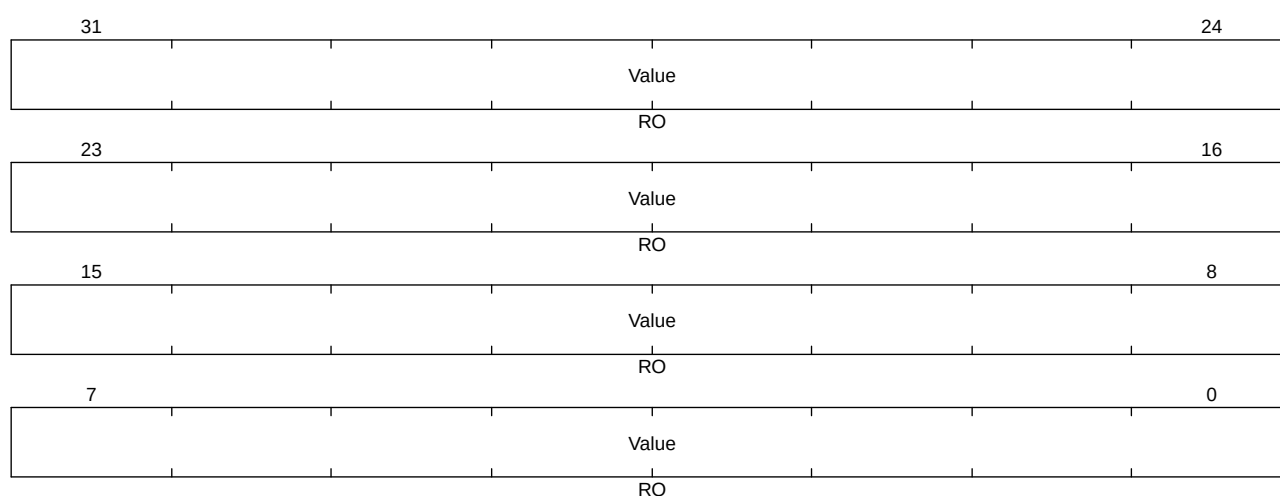
Stores the lower 32 bits (bits [31:0]) of the current 64-bit timestamp counter.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RO (hw-written)	0	Lower 32 bits (bits [31:0]) of the timestamp counter

16.10.10. `te.trTsCounterHigh` – offset 0x04C

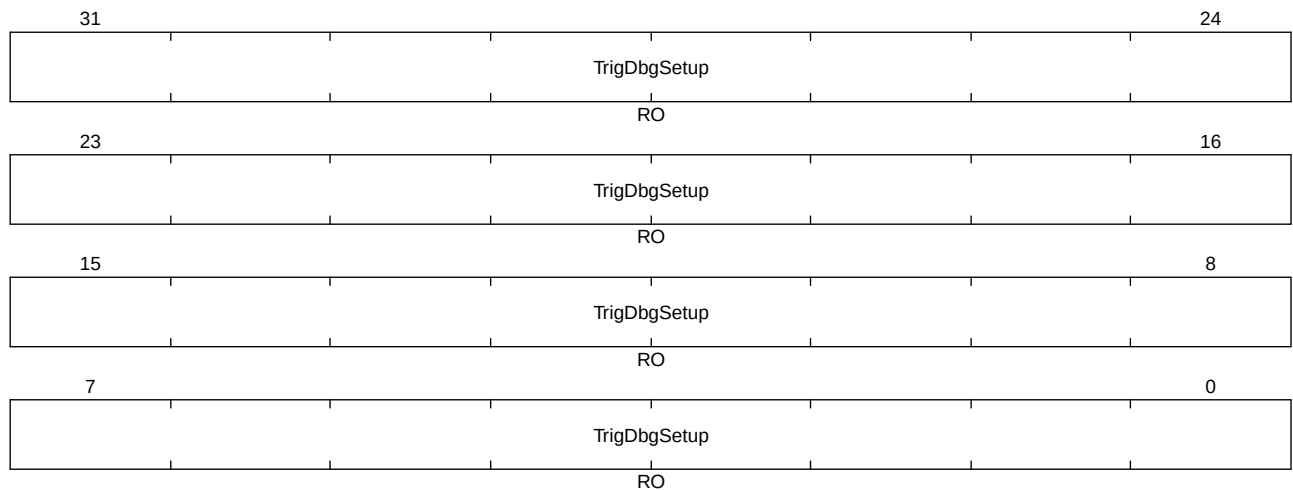
Stores the upper 32 bits (bits [63:32]) of the current 64-bit timestamp counter.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RO (hw-written)	0	Upper 32 bits (bits [63:32]) of the timestamp counter

16.10.11. `te.trTeTrigDbgControl` – offset 0x050

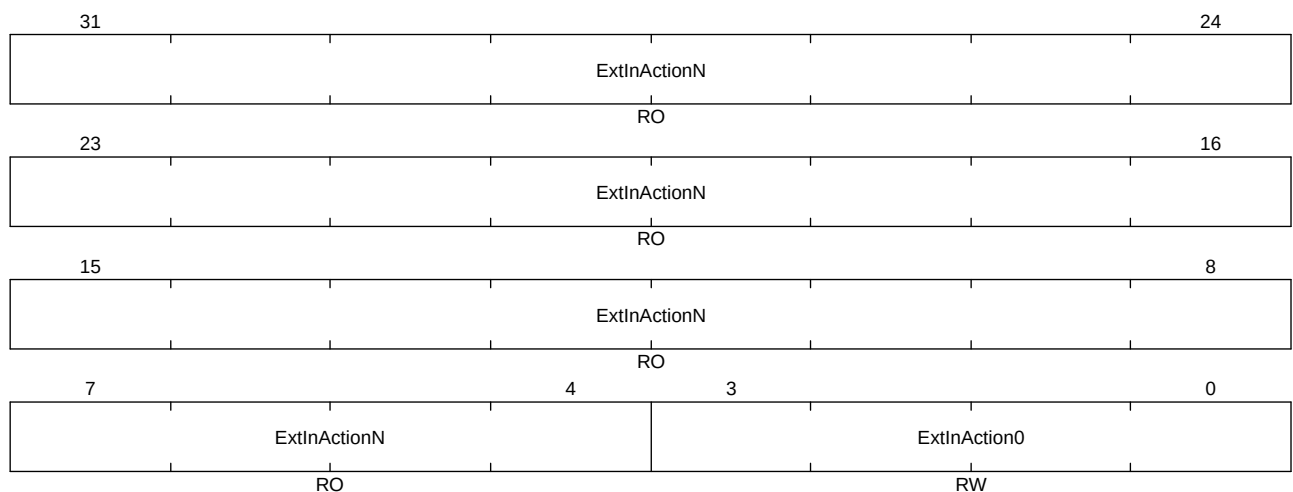
Vendor-specific setup for RISC-V Debug-Module triggers whose action is a trace action (TCI Table 19). CTTE has no debug-trigger interface: the ingress port carries no mcontrol trigger-hit sideband, so there is nothing to configure. Read-only 0 in every profile; writes are ignored. The register is kept in the map (rather than left unmapped) so a TCI tool gets a defined 'no vendor trigger setup' answer instead of a bus error. [TCI Table 19]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	TrigDbgSetup	RO	0	Vendor-specific trigger setup. Fixed at 0 (no debug-trigger interface).

16.10.12. `te.trTeTrigExtInControl` – offset 0x054

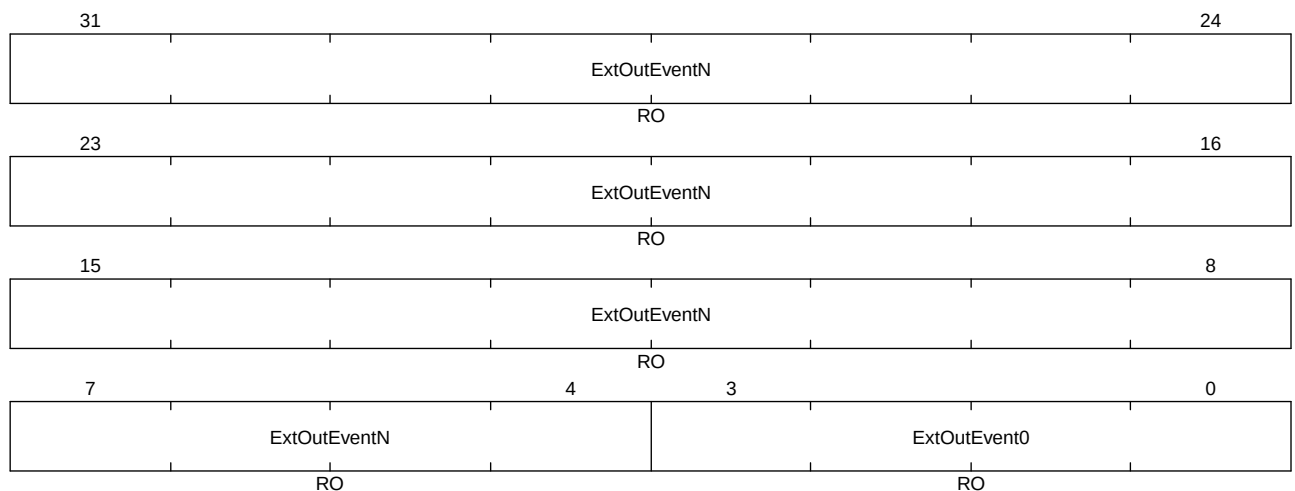
Selects the action performed when an external trigger input fires (TCI Table 20). CTTE implements external trigger input `\#0` = the generic tip.trigger event port; inputs `\#1..7` do not exist and their action fields are fixed at 0 as the spec prescribes.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:4	ExtInActionN	RO	0	Actions for external trigger inputs #1..7 (4 bits each, same encoding as ExtInAction0). These inputs do not exist in CTTE, so per TCI Table 20 their action fields are fixed at 0.
3:0	ExtInAction0	RW (Enabled-locked)	0	Select action to perform when external trigger input \#0 fires. External trigger input \#0 is the generic tip.trigger event port of the ingress interface – the same pulse that the historical <code>trTeControl.InstTrigEnable</code> path consumes. WARL: legal values are 0 (no action), 2 (trace-on), 3 (trace-off) and – only with the <code>CT_EN_TRIG_SYNC</code> build switch – 4 (trace-notify); every other value, and any value at all in a build without <code>CT_EN_TRIG_REGS</code> , is legalized to 0 before it reaches the register (legalization lives in <code>ct_cs_cpuif_wb.sv</code> , not in this generated block). RELATION TO <code>InstTrigEnable</code> (documented deviation): CTTE used <code>InstTrigEnable</code> as the enable of the <code>SYNC=6</code> notify marker long before this register existed, and that behaviour is UNCHANGED – action 4 is an ADDITIONAL source of the same one-shot, so a configuration with <code>InstTrigEnable = 1 AND Action0 = 4</code> still produces exactly ONE marker per trigger pulse. Writable only while <code>trTeControl.Enable = 0</code> . Values: 0 = <code>TRIG_ACT_NONE</code> ; 2 = <code>TRIG_ACT_ON</code> ; 3 = <code>TRIG_ACT_OFF</code> ; 4 = <code>TRIG_ACT_NOTIFY</code> .

16.10.13. `te.trTeTrigExtOutControl` – offset 0x058

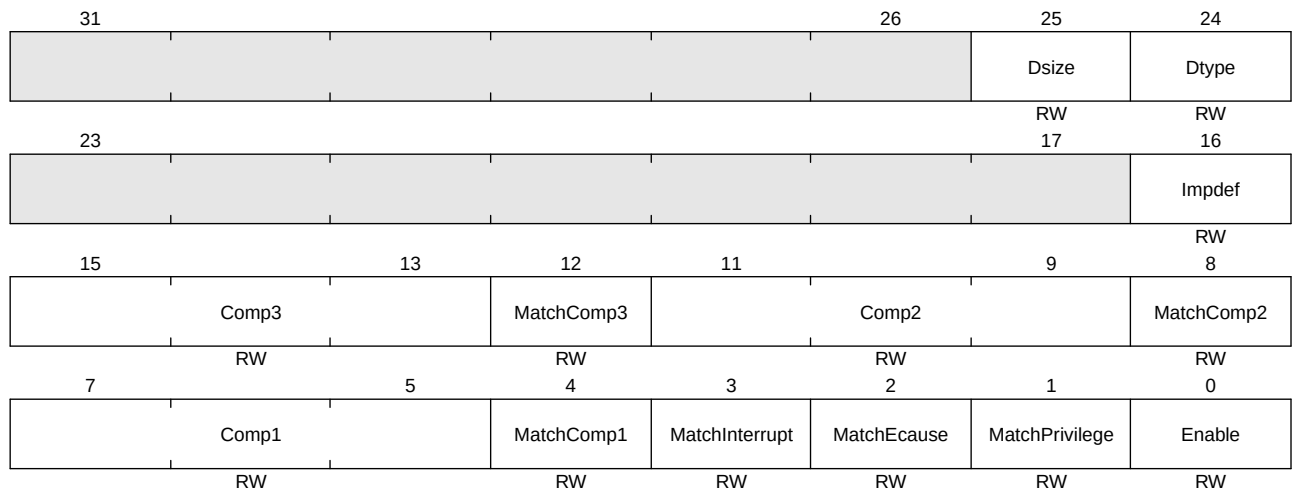
Bitmap selecting which events fire an external trigger OUTPUT (TCI Table 21). CTTE has no trigger output port – adding one would change the encoder's top-level signal contract – so per TCI Table 21 all event bits are fixed at 0 ('If external trigger output #0 does not exist, then all bits are fixed at 0'). Writes are ignored.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:4	ExtOutEventN	RO	0	Event bitmaps for external trigger outputs #1..7. Fixed at 0 (outputs do not exist).
3:0	ExtOutEvent0	RO	0	Event bitmap for external trigger output #0. Fixed at 0 (output does not exist).

16.10.14. te.trTeFilter[] .Control – offset 0x400

Enable, input select, function, match mode, inversion and latching

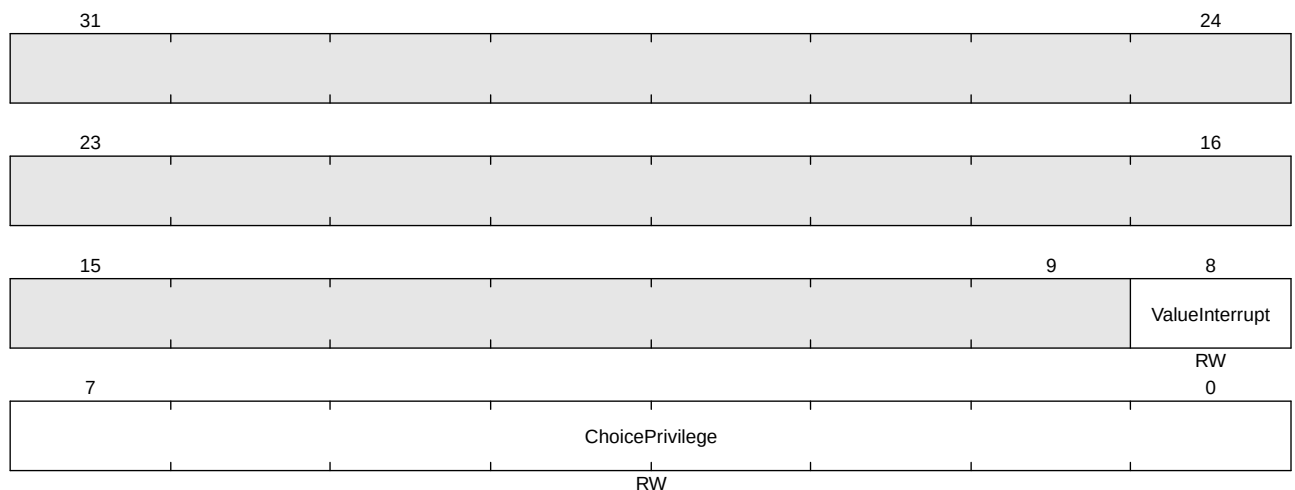


Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
25	Dsize	RW (Enable-locked)	0	When set, match dsize values as specified by trTeFilterMatchChoiceDsize field for filter #i) Writable only while trTeControl.Enable=0.
24	Dtype	RW (Enable-locked)	0	When set, match dtype values as specified by trTeFilterMatchChoiceDtype field for filter #i. Writable only while trTeControl.Enable=0.
16	Impdef	RW (Enable-locked)	0	When set, match impdef values as specified by trTeFilterMatchValueImpdef and trTeFilterMatchMaskImpdef fields for filter #i. Writable only while trTeControl.Enable=0.
15:13	Comp3	RW (Enable-locked)	0	Specifies the comparator unit to use for the 3rd comparison Writable only while trTeControl.Enable=0.
12	MatchComp3	RW (Enable-locked)	0	When set, the output of the comparator selected by trTeFilterComp3 must be true for the filter to match Writable only while trTeControl.Enable=0.
11:9	Comp2	RW (Enable-locked)	0	Specifies the comparator unit to use for the 2nd comparison. Writable only while trTeControl.Enable=0.
8	MatchComp2	RW (Enable-locked)	0	When set, the output of the comparator selected by trTeFilterComp2 must be true for the filter to match Writable only while trTeControl.Enable=0.

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
7:5	Comp1	RW (Enable-locked)	0	Specifies the comparator unit to use for the 1st comparison. Writable only while trTeControl.Enable=0.
4	MatchComp1	RW (Enable-locked)	0	When set, the output of the comparator selected by trTeFilterComp1 must be true for the filter to match Writable only while trTeControl.Enable=0.
3	MatchInterrupt	RW (Enable-locked)	0	When set, start matching from either an interrupt or exception as specified by trTeFilterMatchValueInterrupt field for filter #i, and stop matching upon return from the 1st matching trap. Writable only while trTeControl.Enable=0.
2	MatchEcause	RW (Enable-locked)	0	When set, start matching from exception cause codes specified by trTeFilterMatchChoiceEcause field for filter #i, and stop matching upon return from the 1st matching exception. Writable only while trTeControl.Enable=0.
1	MatchPrivilege	RW (Enable-locked)	0	When set, match privilege levels specified by trTeFilterMatchChoicePrivilege field for filter #i Writable only while trTeControl.Enable=0.
0	Enable	RW (Enable-locked)	0	Overall filter enable for filter #i Writable only while trTeControl.Enable=0.

16.10.15. te.trTeFilter[] .Match – offset 0x404

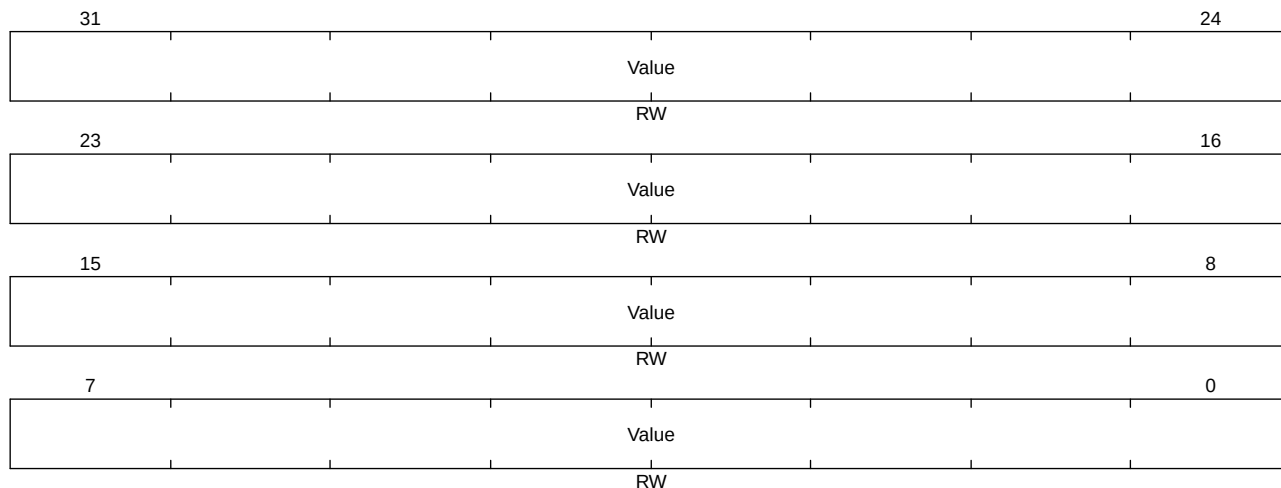
Privilege bitmap and interrupt/exception selector. Only accessible on trTeControl. Enable=0



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
8	ValueInterrupt	RW (Enable-locked)	0	Interrupt/exception selector for start condition. 0: EXCEPTION_TRAP, 1: INTERRUPT Writable only while trTeControl.Enable=0. Values: 0 = TR_FILTER_MATCHVALUE_EXCEPTION_TRAP; 1 = TR_FILTER_MATCHVALUE_INTERRUPT.
7:0	ChoicePrivilege	RW (Enable-locked)	0	Privilege bitmap; bit N=1 matches priv=N Writable only while trTeControl.Enable=0.

16.10.16. te.trTeFilter[] .MatchChoiceEcauseLow – offset 0x408

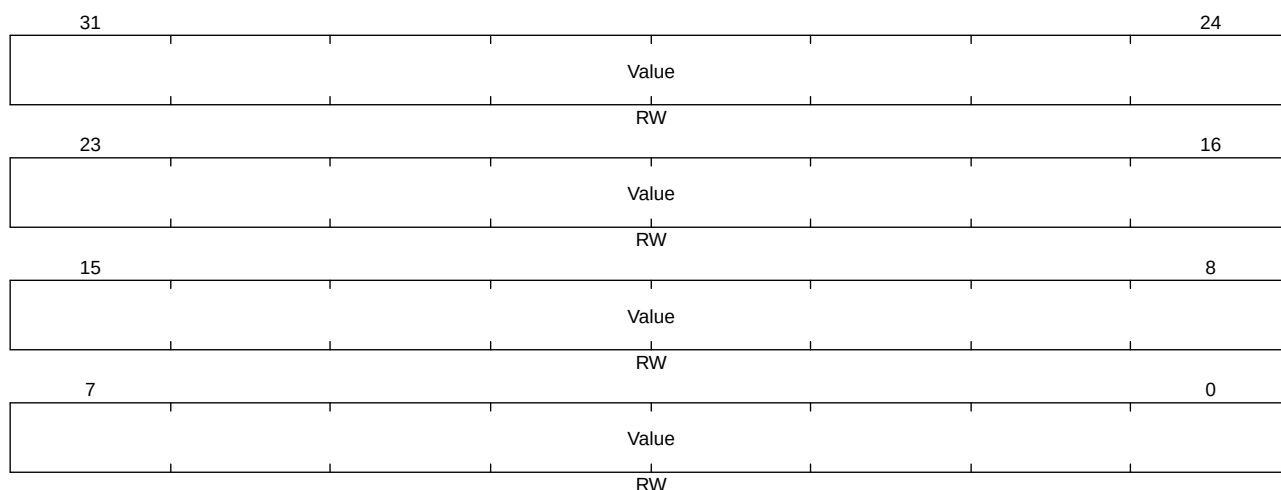
Bits 31:0 of Ecause bitmap. Only accessible on trTeControl. Enable=0. Default configuration: Only Bits 0...15 are used.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Bitmap for ecause codes 0..31; bit N=1 matches ecause=N Writable only while trTeControl.Enable=0.

16.10.17. te.trTeFilter[] .MatchChoiceEcauseHigh – offset 0x40C

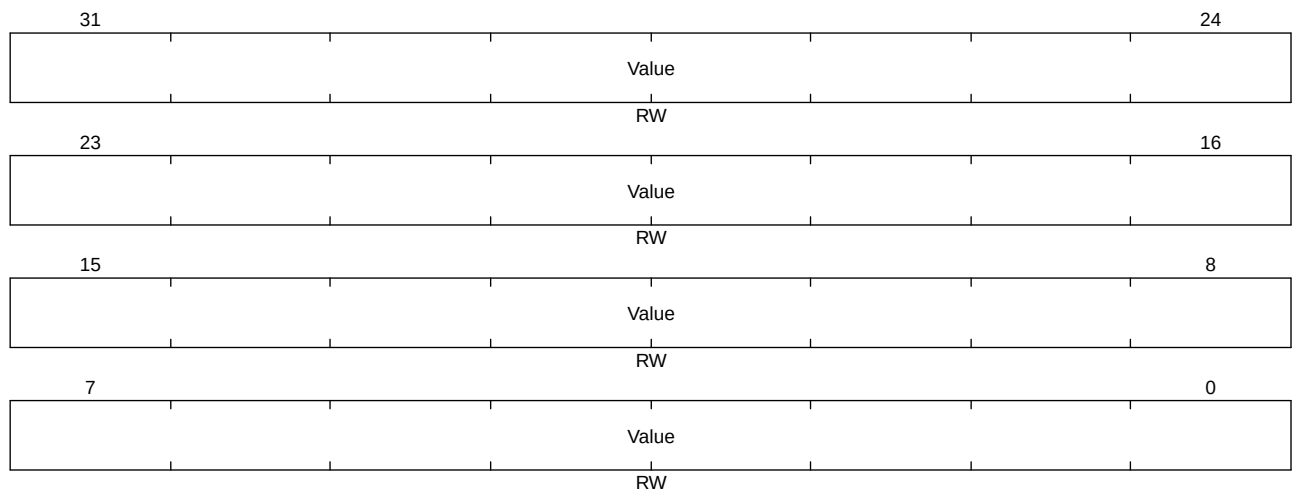
Bits 63:32 of Ecause bitmap. Only accessible on trTeControl. Enable=0. Not used in default configuration



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Bitmap for ecause codes 32..63; bit N=1 matches ecause=N+32 Writable only while trTeControl.Enable=0.

16.10.18. `te.trTeFilter[] .MatchValueImpdef` – offset 0x410

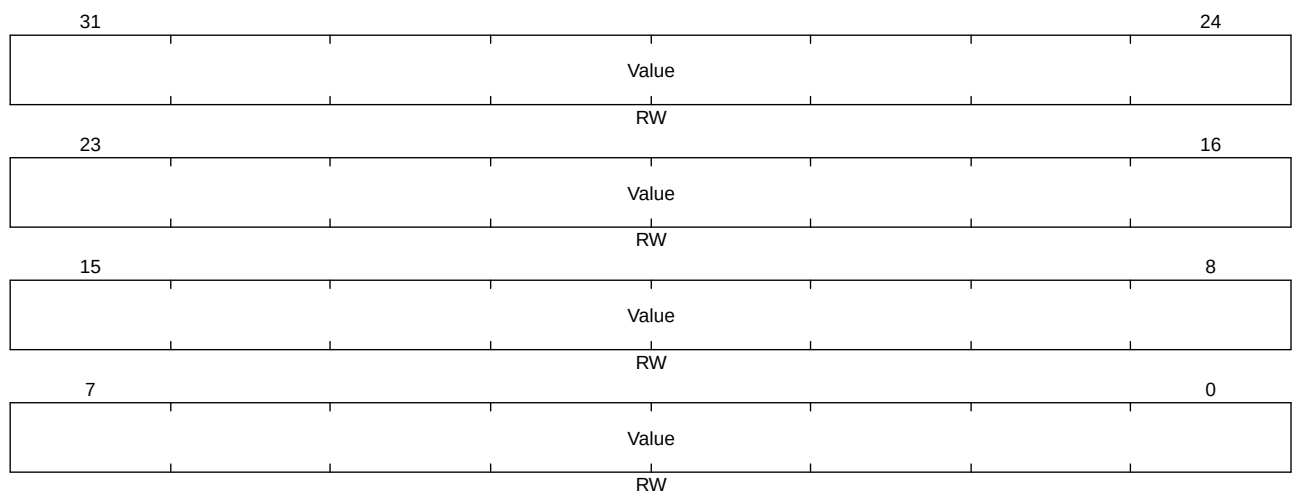
Implementation-defined value (used with mask). Only accessible on `trTeControl`. Enable=0



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Value compared with (impdef & mask) Writable only while <code>trTeControl.Enable=0</code> .

16.10.19. `te.trTeFilter[] .MatchMaskImpdef` – offset 0x414

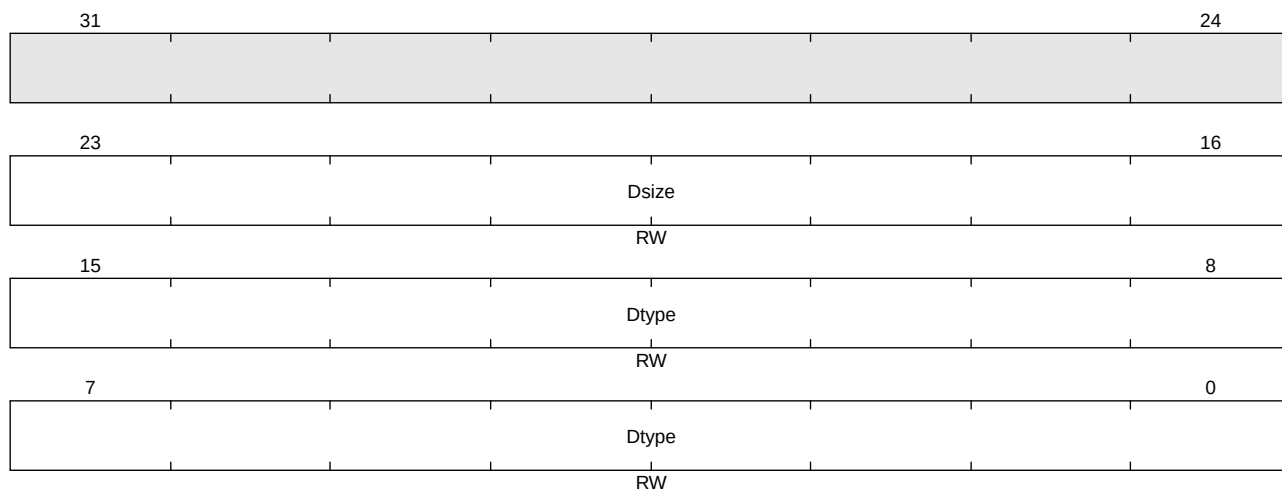
Implementation-defined mask. Only accessible on `trTeControl`. Enable=0



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Mask for implementation-defined compare Writable only while <code>trTeControl.Enable=0</code> .

16.10.20. `te.trTeFilter[] .MatchChoiceData` – offset 0x418

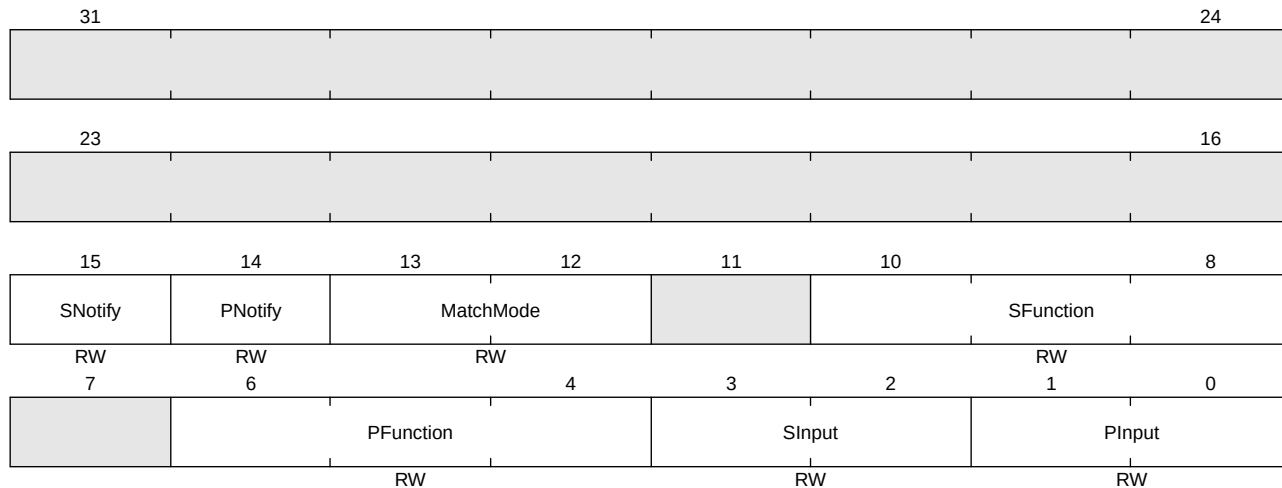
`dtype/dsize` selection bitmaps. Only accessible on `trTeControl`. Enable=0



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
23:16	<code>Dsize</code>	RW (Enable-locked)	0	Bitmap for <code>dsz</code> ; bit N=1 matches <code>dsz=N</code> Writable only while <code>trTeControl.Enable=0</code> .
15:0	<code>Dtype</code>	RW (Enable-locked)	0	Bitmap for <code>dtype</code> ; bit N=1 matches <code>dtype=N</code> Writable only while <code>trTeControl.Enable=0</code> .

16.10.21. `te.trTeComp[] .Control` – offset 0x600

Input select, compare functions, mode, notify

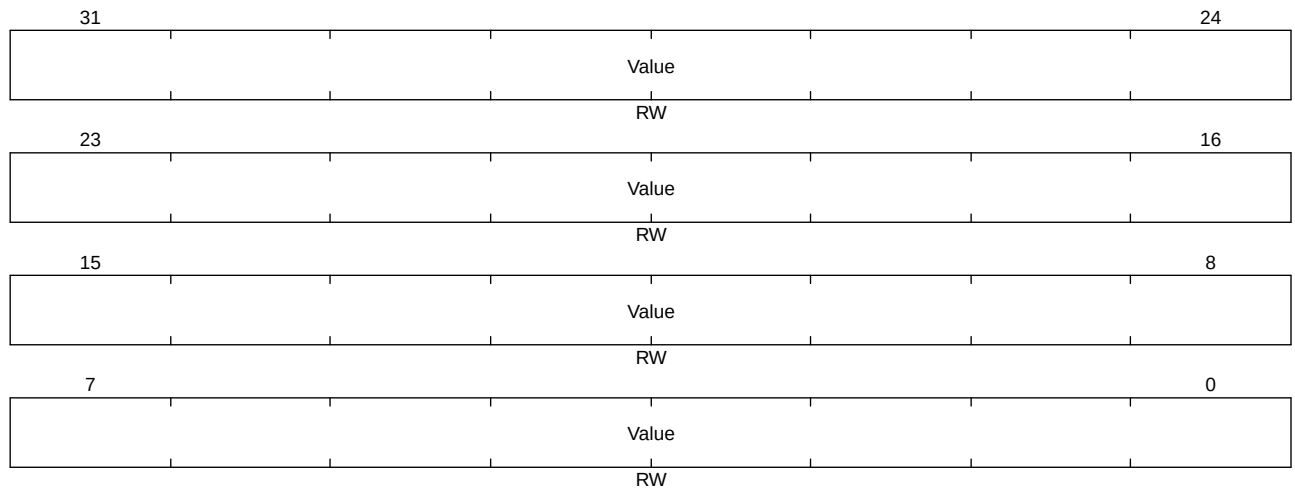


Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15	<code>SNotify</code>	RW (Enable-locked)	0	Generate a trace packet explicitly reporting the address of the final instruction in a block that causes a SECONDARY match. This is also known as a watchpoint. Writable only while <code>trTeControl.Enable=0</code> .
14	<code>PNotify</code>	RW (Enable-locked)	0	Generate a trace packet explicitly reporting the address of the final instruction in a block that causes a PRIMARY match. This is also known as a watchpoint. Writable only while <code>trTeControl.Enable=0</code> .

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
13:12	MatchMode	RW (Enable-locked)	0	Match mode (0..3, see desc) Writable only while <code>trTeControl.Enable=0</code> . Values: 0 = <code>TR_COMP_MATCH_MODE0</code> ; 1 = <code>TR_COMP_INPUT_MODE1</code> ; 2 = <code>TR_COMP_INPUT_MODE2</code> ; 3 = <code>TR_COMP_INPUT_MODE3</code> .
10:8	SFunction	RW (Enable-locked)	0	Secondary function Writable only while <code>trTeControl.Enable=0</code> . Values: 0 = <code>TR_COMP_SFUNC_EQUAL</code> ; 1 = <code>TR_COMP_SFUNC_NOT_EQUAL</code> ; 2 = <code>TR_COMP_SFUNC_LESS</code> ; 3 = <code>TR_COMP_SFUNC_LESS_EQUAL</code> ; 4 = <code>TR_COMP_SFUNC_GREATER</code> ; 5 = <code>TR_COMP_SFUNC_GREATER_EQUAL</code> ; 6 = <code>TR_COMP_SFUNC_PMASK</code> ; 7 = <code>TR_COMP_SFUNC_ALWAYS_MATCH</code> .
6:4	PFunction	RW (Enable-locked)	0	Primary function Writable only while <code>trTeControl.Enable=0</code> . Values: 0 = <code>TR_COMP_PFUNC_EQUAL</code> ; 1 = <code>TR_COMP_PFUNC_NOT_EQUAL</code> ; 2 = <code>TR_COMP_PFUNC_LESS</code> ; 3 = <code>TR_COMP_PFUNC_LESS_EQUAL</code> ; 4 = <code>TR_COMP_PFUNC_GREATER</code> ; 5 = <code>TR_COMP_PFUNC_GREATER_EQUAL</code> ; 6 = <code>TR_COMP_PFUNC_RESERVED_MATCH</code> ; 7 = <code>TR_COMP_PFUNC_ALWAYS_MATCH</code> .
3:2	SInput	RW (Enable-locked)	0	Secondary input select Writable only while <code>trTeControl.Enable=0</code> . Values: 0 = <code>TR_COMP_INPUT_IADDR</code> ; 1 = <code>TR_COMP_INPUT_CONTEXT</code> ; 2 = <code>TR_COMP_INPUT_TVAL</code> ; 3 = <code>TR_COMP_INPUT_DADDR</code> .
1:0	PInput	RW (Enable-locked)	0	Primary input select Writable only while <code>trTeControl.Enable=0</code> . Values: 0 = <code>TR_COMP_INPUT_IADDR</code> ; 1 = <code>TR_COMP_INPUT_CONTEXT</code> ; 2 = <code>TR_COMP_INPUT_TVAL</code> ; 3 = <code>TR_COMP_INPUT_DADDR</code> .

16.10.22. `te.trTeComp[]`. `SMaskLow` – offset 0x608

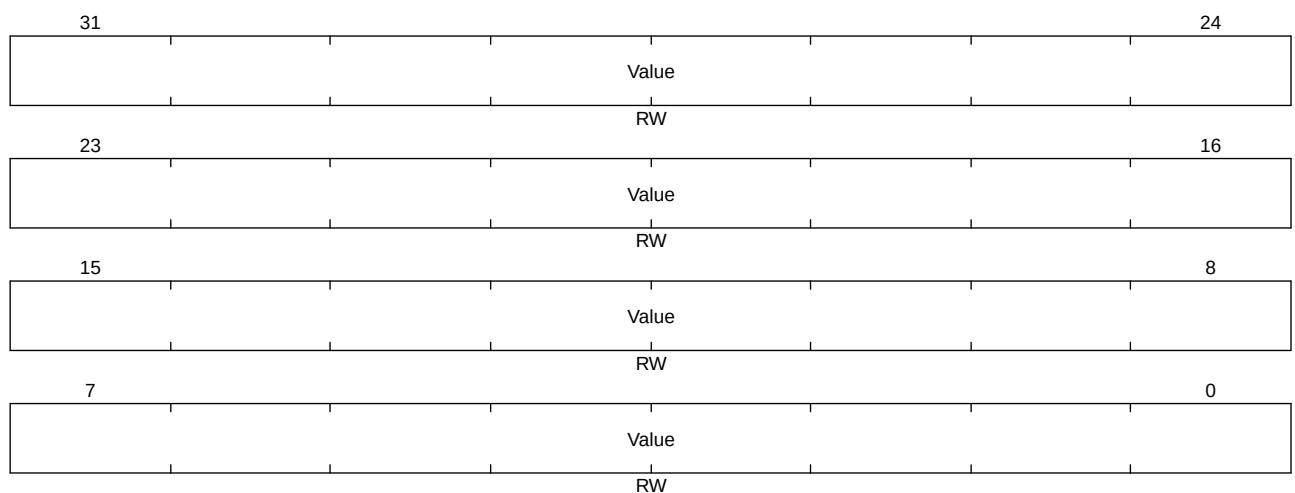
Bits 31:0 of the PMASK mask. Only accessible on `trTeControl`. `Enable=0`. [CTTE specific] Used ONLY by `SFunction = TR_COMP_SFUNC_PMASK`, which matches `(value AND mask) == SMatch`. Until X2a this mask was read out of `SMatchHigh`, which made the high half of the secondary bound unusable – with 64-bit addresses `SMatch{High,Low}` IS the bound, so the mask needs its own register.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Secondary mask value, bits [31:0]. Writable only while <code>trTeControl.Enable=0</code> .

16.10.23. `te.trTeComp[] .SMaskHigh` – offset 0x60C

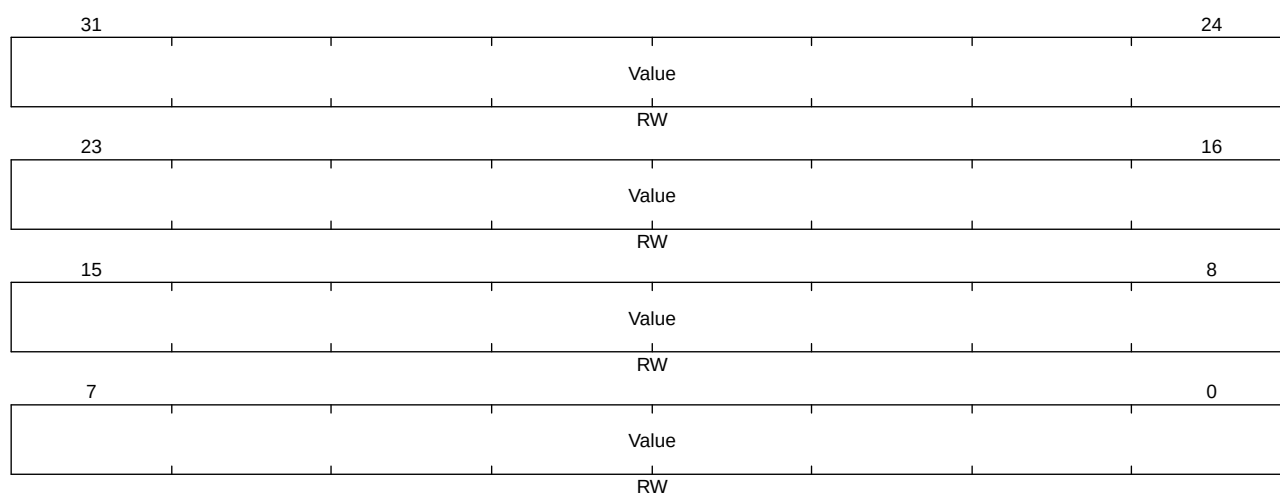
Bits 63:32 of the PMASK mask. Only accessible on `trTeControl`. `Enable=0`. Not used in a 32-bit address configuration.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Secondary mask value, bits [63:32]. Writable only while <code>trTeControl.Enable=0</code> .

16.10.24. `te.trTeComp[] .PMatchLow` – offset 0x610

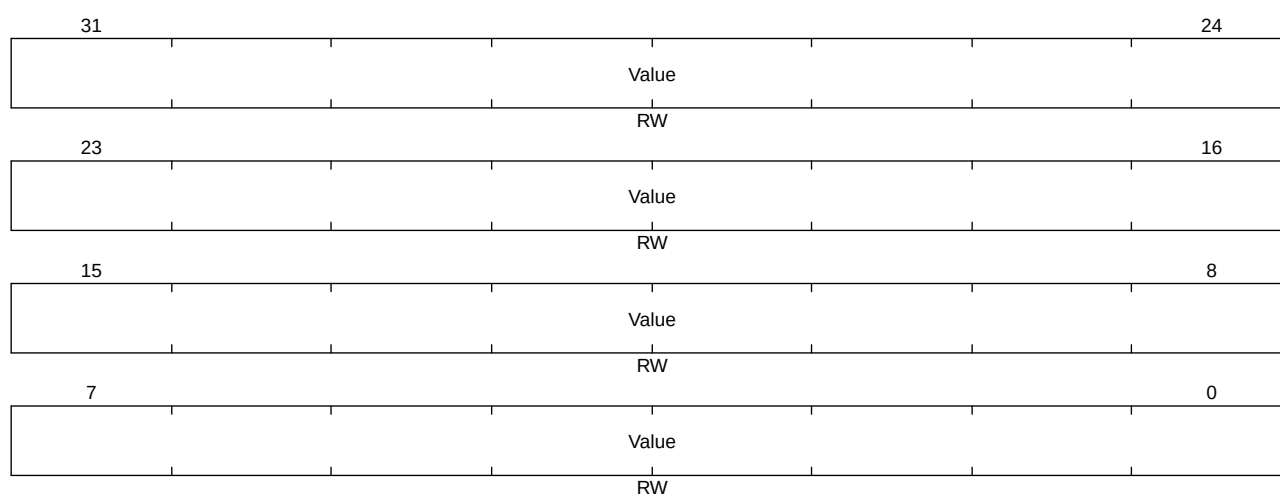
Bits 31:0. Only accessible on `trTeControl`. `Enable=0`



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Primary match value, bits [31:0]. Writable only while <code>trTeControl.Enable=0</code> .

16.10.25. `te.trTeComp[] . PMatchHigh` – offset 0x614

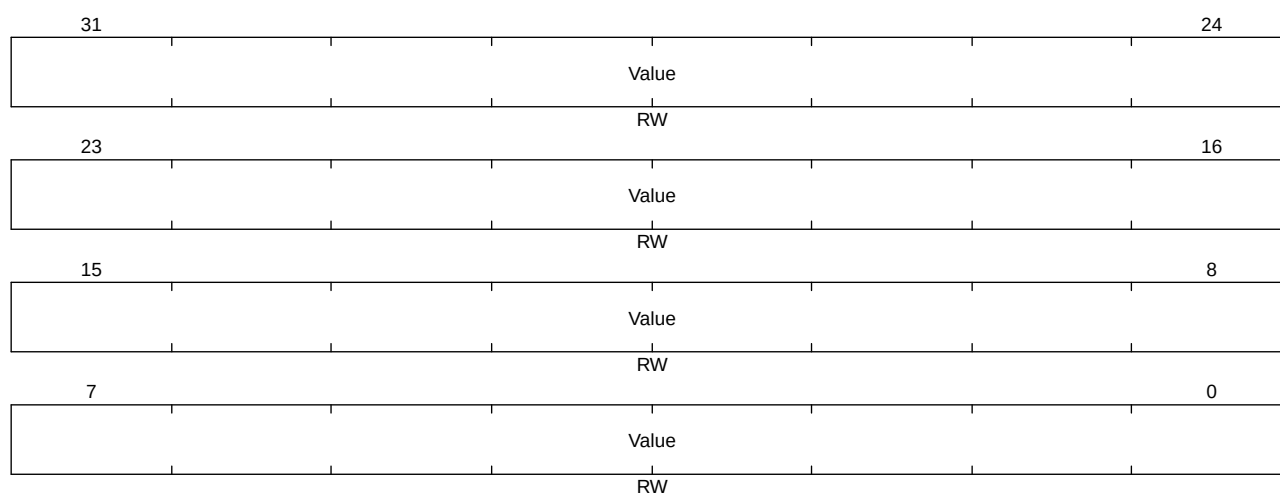
Bits 63:32. Only accessible on `trTeControl.Enable=0`



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Primary match value, bits [63:32]. Writable only while <code>trTeControl.Enable=0</code> .

16.10.26. `te.trTeComp[] . SMatchLow` – offset 0x618

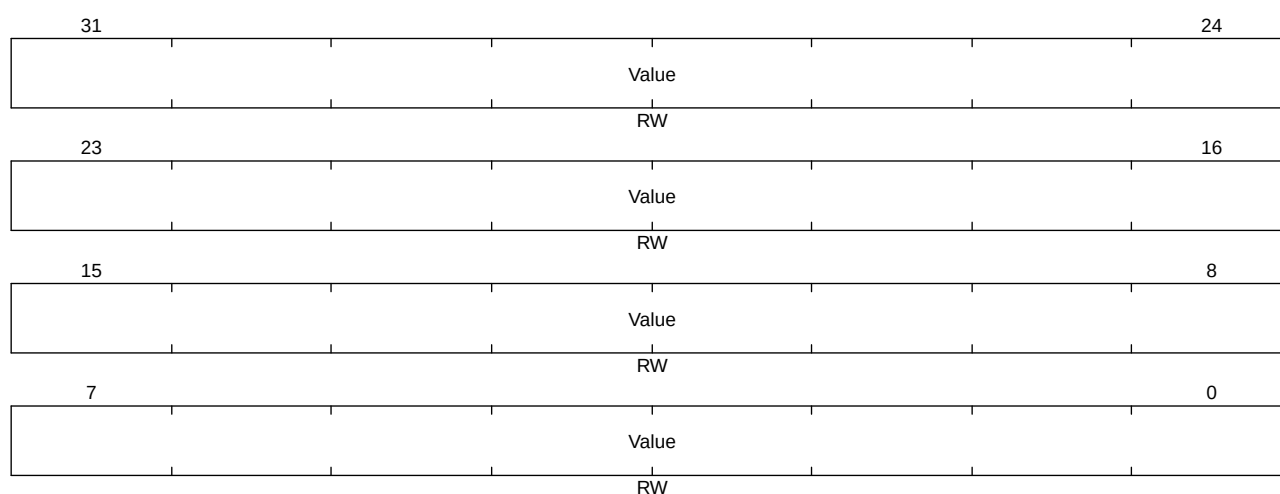
Bits 31:0. Only accessible on `trTeControl.Enable=0`



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Secondary match value, bits [31:0]. Writable only while <code>trTeControl.Enable=0</code> .

16.10.27. `te.trTeComp[]`. `SMatchHigh` – offset 0x61C

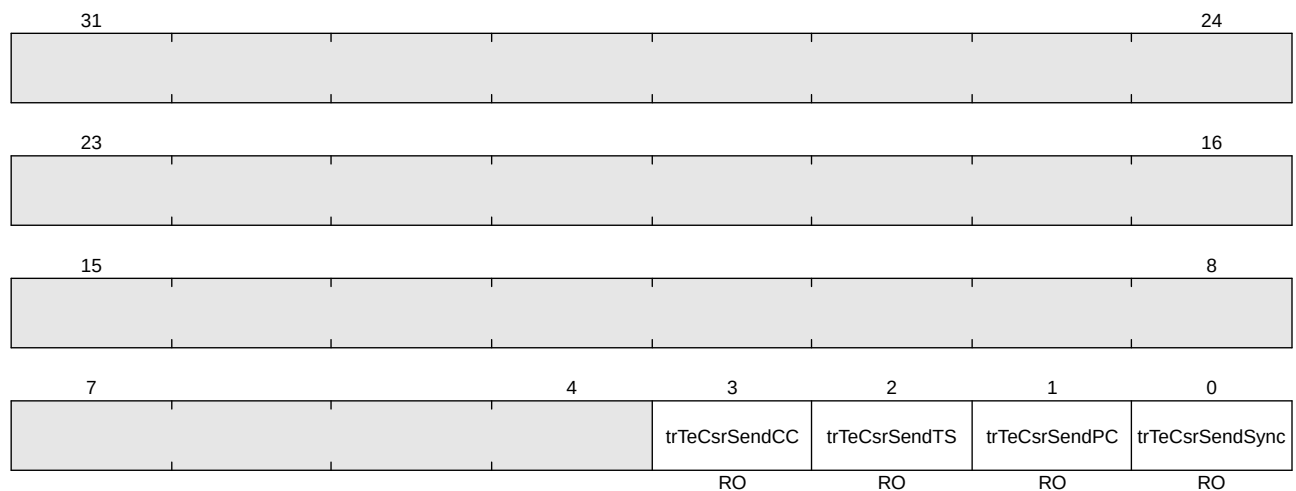
Bits 63:32. Only accessible on `trTeControl`. `Enable=0`



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Secondary match value, bits [63:32]. Writable only while <code>trTeControl.Enable=0</code> .

16.10.28. `te.trTeCsrControl` – offset 0xE00

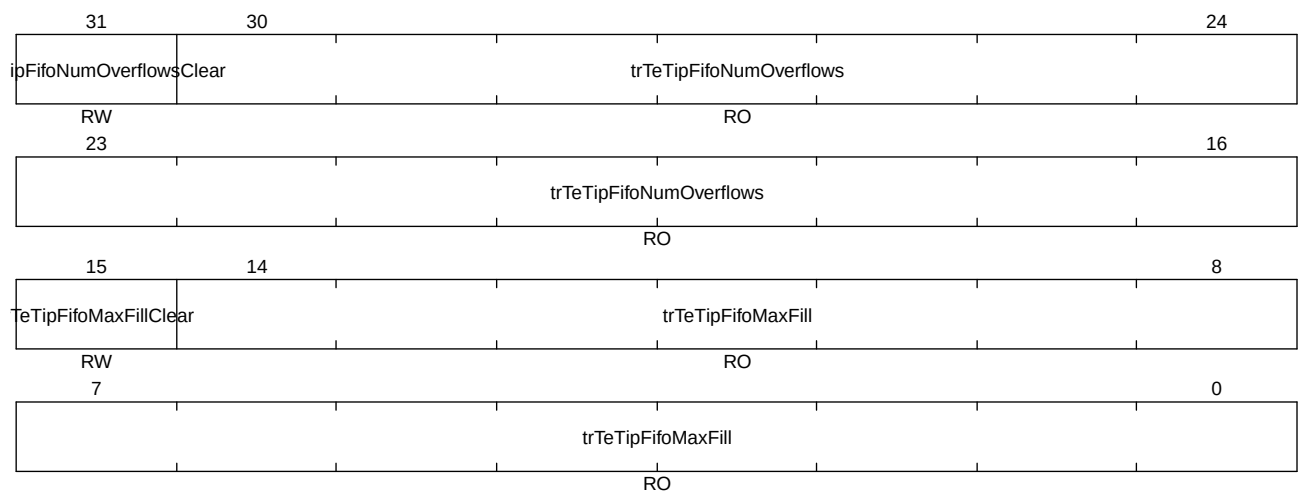
TE control via CSR. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
3	trTeCsrSendCC	RO (hw-written)	0	Do send Clock Cycle
2	trTeCsrSendTS	RO (hw-written)	0	Do send TS
1	trTeCsrSendPC	RO (hw-written)	0	Do send PC
0	trTeCsrSendSync	RO (hw-written)	0	Do send Sync

16.10.29. te.trTeTipFifoStatus – offset 0xE04

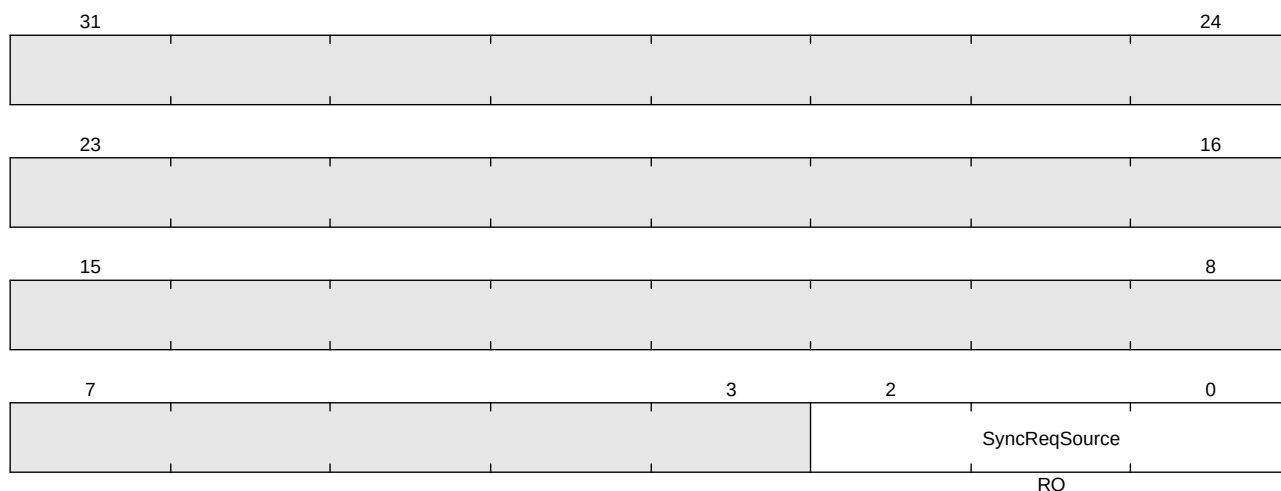
Provides TIP FIFO statistics. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31	trTeTipFifoNumOverflowsClear	RW	0	Write 1 to clear trTeTipFifoNumOverflows and hold it at 0, write 0 to re-arm.
30:16	trTeTipFifoNumOverflows	RO (hw-written)	0	Count of discard events (dropped message beats) since the last clear. Saturates at 0x7FFF.
15	trTeTipFifoMaxFillClear	RW	0	Write 1 to clear trTeTipFifoMaxFill and hold it at 0, write 0 to re-arm.
14:0	trTeTipFifoMaxFill	RO (hw-written)	0	Max fill level of internal buffering fifo since the last clear. Saturates at 0x7FFF.

16.10.30. `te.trTeSyncStatus` – offset 0xE08

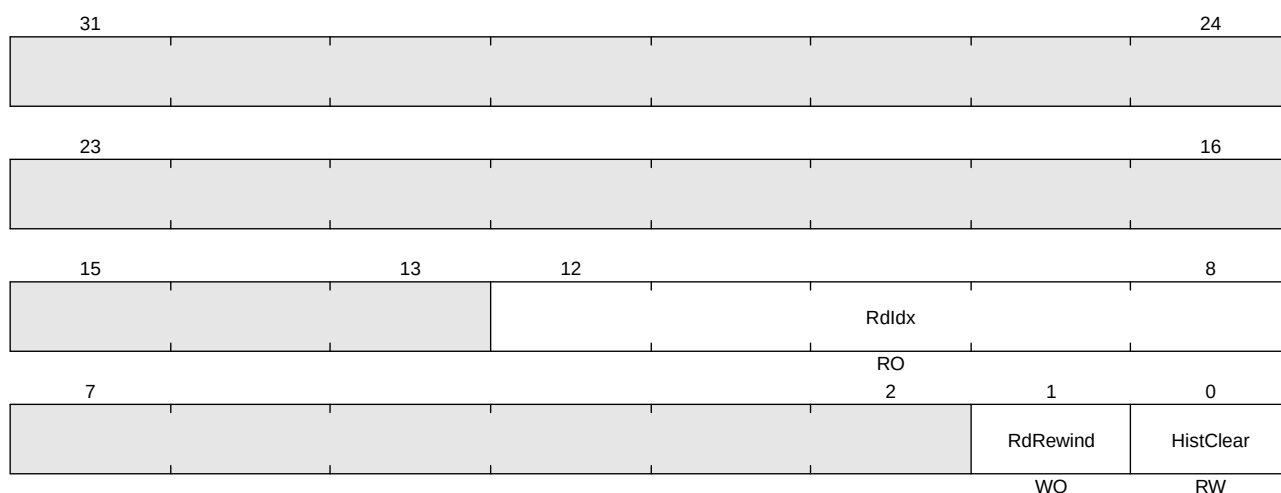
Diagnostic status for explicit synchronization requests. On the wire every explicit sync request is emitted with the single vendor SYNC code 14 (explicit sync request) – N-Trace 1.0 Table 25 reserves codes 10-13 for future standard use, so the former per-source codes 12 (CSR) and 13 (ATB) are no longer used. The request source does not matter to a decoder (every synchronization message is a full re-anchor); it is kept readable here for diagnosis. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
2:0	<code>SyncReqSource</code>	RO (hw-written)	0	Source of the most recent explicit synchronization request. [CTTE specific] Values: 0 = <code>SYNC_REQ_NONE</code> ; 1 = <code>SYNC_REQ_CSR</code> ; 2 = <code>SYNC_REQ_ATB</code> ; 3 = <code>SYNC_REQ_QUOTA</code> ; 4 = <code>SYNC_REQ_TE</code> .

16.10.31. `te.trTeTipFifoHistCtrl` – offset 0xE10

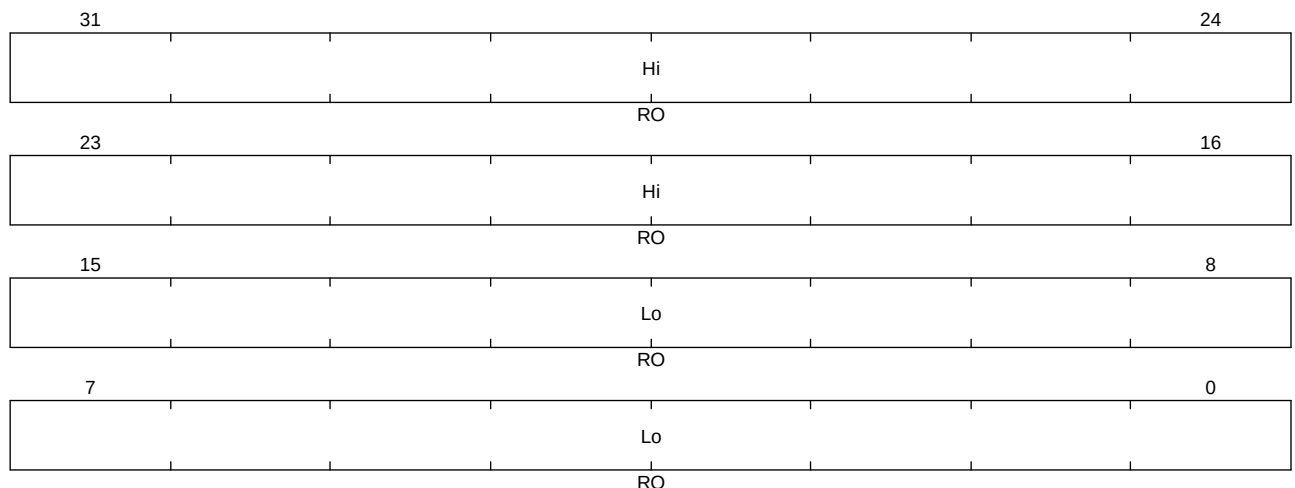
Control for the SERIAL fill-level histogram read-out via `trTeTipFifoHistData`. The number of bins (`CT_FIFO_HIST_BINS`, default 16) is an RTL parameter – the CSR map is independent of it (one data register, serial pointer). [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
12:8	RdIdx	RO (hw-written)	0	Current serial read pointer (bin-pair index the next trTeTipFifoHistData read will return). [CTTE specific]
1	RdRewind	WO (self-clearing)	0	Write 1 to rewind the serial read pointer to bin pair 0 WITHOUT clearing the counters (re-read the histogram). Self-clearing. [CTTE specific]
0	HistClear	RW	0	Write 1 to clear all histogram counters AND the read pointer and hold them at 0, write 0 to re-arm. Level, no CDC – change only while the trace is quiescent (trTeControl.Enable=0). [CTTE specific]

16.10.32. te.trTeTipFifoHistData – offset 0xE14

SERIAL window into the CT_FIFO_HIST_BINS saturating 16-bit upward-crossing counters (AW: not random access – one register, DSP-backed counters, minimal readback). Each read returns bin pair $[2 * RdIdx \text{ (Lo)}, 2 * RdIdx + 1 \text{ (Hi)}]$ and auto-increments RdIdx (wraps after BINS/2). Bin b counts crossings of fill level $(b+1) * DEPTH / BINS$, $DEPTH = ETIP_CVS_FIFO_DEPTH$. Read protocol: HistClear or RdRewind, then BINS/2 consecutive reads. No CDC – read only while the trace is quiescent (trTeControl.Enable=0). [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:16	Hi	RO (hw-written)	0	Bin $2 * RdIdx + 1$ – crossings of level $(2 * RdIdx + 2) * DEPTH / BINS$. [CTTE specific]
15:0	Lo	RO (hw-written)	0	Bin $2 * RdIdx$ – crossings of level $(2 * RdIdx + 1) * DEPTH / BINS$. [CTTE specific]

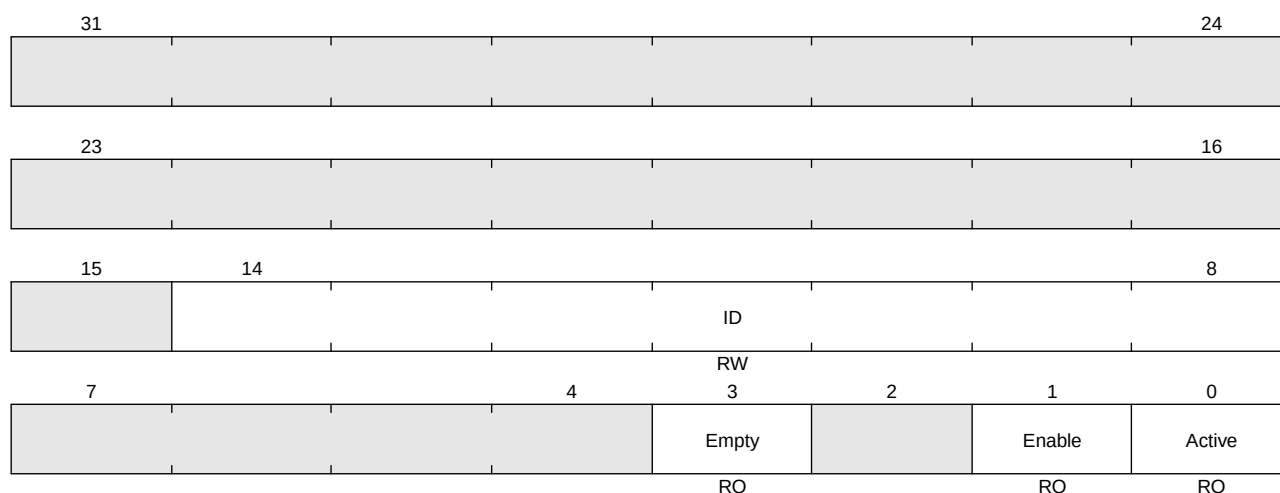
16.11. Component atb (base 0x1000)

Trace ATB Bridge sub-component (N-Trace compliant, CompType=0xE). See RISC-V N-Trace Control Interface spec Chapter 10 (Tables 56/57).

Offset	Register	Fields
0x1000	<code>atb.trAtbBridgeControl</code>	4
0x1004	<code>atb.trAtbBridgeImpl</code>	4

16.11.1. `atb.trAtbBridgeControl` – offset 0x1000

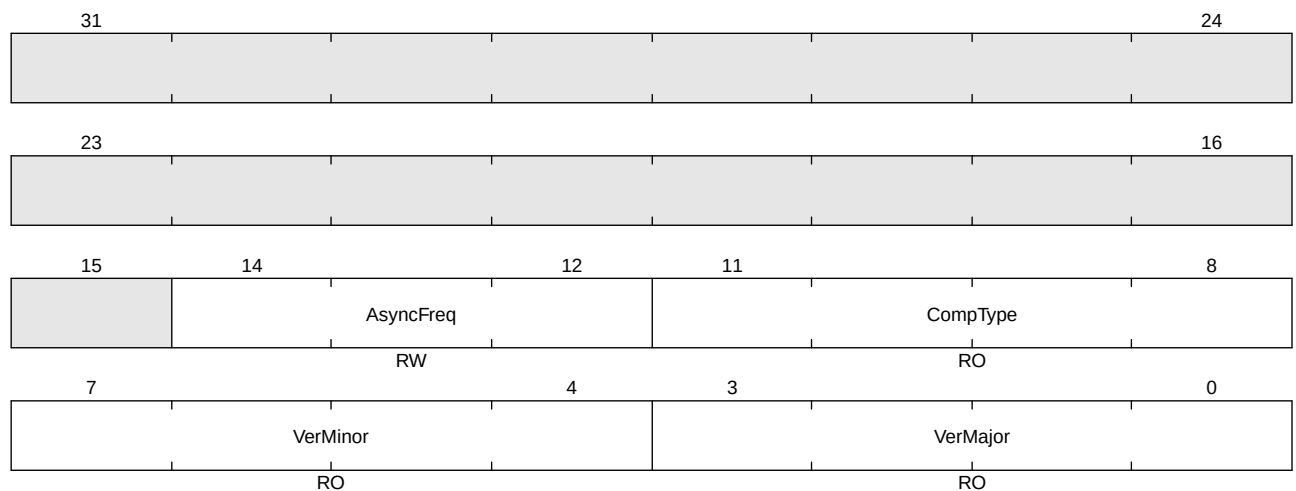
Primary control register for the ATB Bridge.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
14:8	ID	RW (Enable-locked)	8	ID of this node on ATB. Values 0x00 and 0x70-0x7F are reserved by the ATB specification and should not be used. Writable only while <code>trTeControl.Enable=0</code> .
3	Empty	RO (hw-written)	1	Reads as 1 when ATB Bridge internal buffers are empty.
1	Enable	RO	1	1: ATB Bridge enabled. Setting Enable to 0 flushes any queued trace data to ATB.
0	Active	RO	1	Primary activate/reset for the ATB Bridge. When 0, the ATB Bridge may have clocks gated off or be powered down, and other register locations may be inaccessible. [Not implemented: read-only constant 1; there is no clock-gate/power-down control behind it.]

16.11.2. `atb.trAtbBridgeImpl` – offset 0x1004

Contains version and protocol information for the ATB Bridge.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
14:12	AsyncFreq	RW (Enable-locked)	0	Async packet frequency. 0: Async disabled; 1..7: different levels of alignment synchronization. Writable only while trTeControl.Enable=0.
11:8	CompType	RO	0xE	ATB Bridge Component Type (0xE)
7:4	VerMinor	RO	0	ATB Bridge Component Minor Version
3:0	VerMajor	RO	1	ATB Bridge Component Major Version

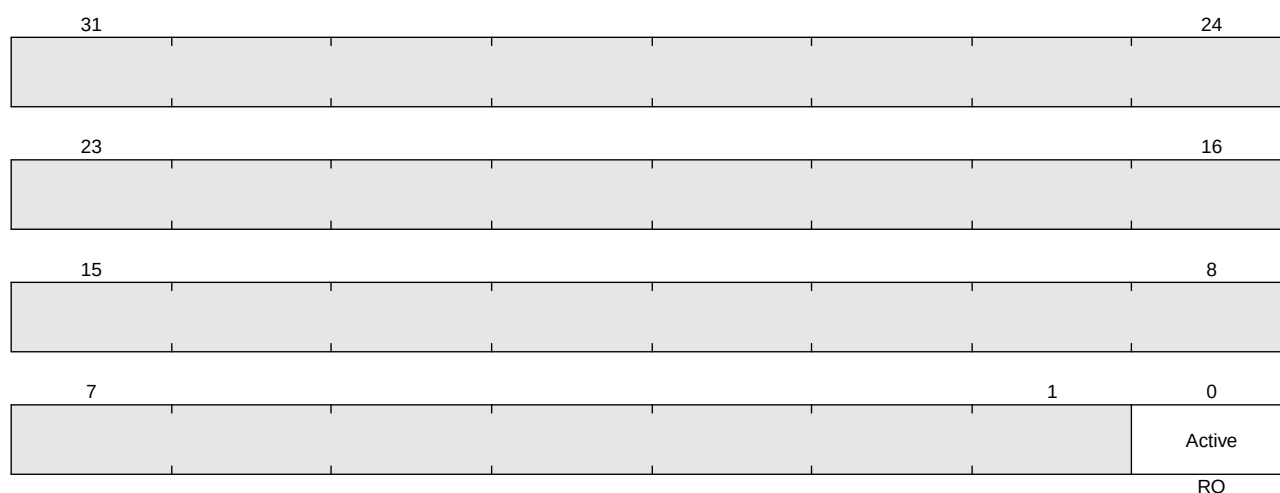
16.12. Component **pc** (base 0x3000)

Performance Counter sub-component (CTTE, vendor CompType=0x3). [CTTE specific]

Offset	Register	Fields
0x3000	pc.trPcControl	1
0x3004	pc.trPcImpl	3
0x3008	pc.trTeConstants	6
0x3010	pc.trPerfCntControl	2
0x3100	pc.trTePerfCntIFetchRange[] .Low	1
0x3104	pc.trTePerfCntIFetchRange[] .High	1
0x3200	pc.trTePerfCntDataRdThRange[] .Low	1
0x3204	pc.trTePerfCntDataRdThRange[] .High	1
0x3300	pc.trTePerfCntDataRdRange[] .Low	1
0x3304	pc.trTePerfCntDataRdRange[] .High	1
0x3400	pc.trTePerfCntDataWrRange[] .Low	1
0x3404	pc.trTePerfCntDataWrRange[] .High	1

16.12.1. **pc.trPcControl** – offset 0x3000

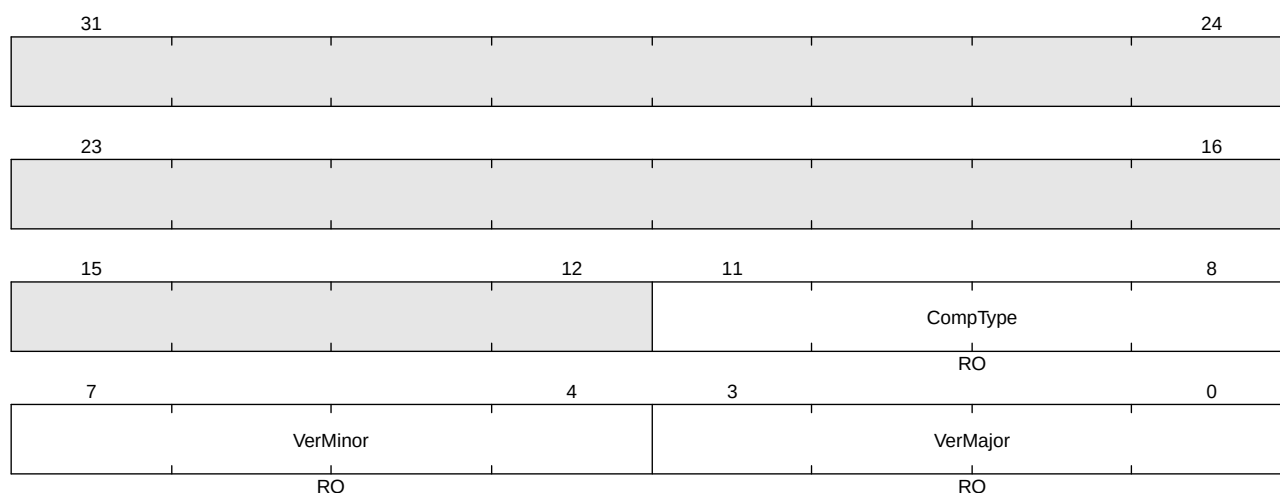
Primary activate/reset for the Performance Counter component. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
0	Active	RO	1	Primary activate/reset bit. [Not implemented: read-only constant 1; there is no clock-gate/power-down control behind it.]

16.12.2. pc.trPcImpl – offset 0x3004

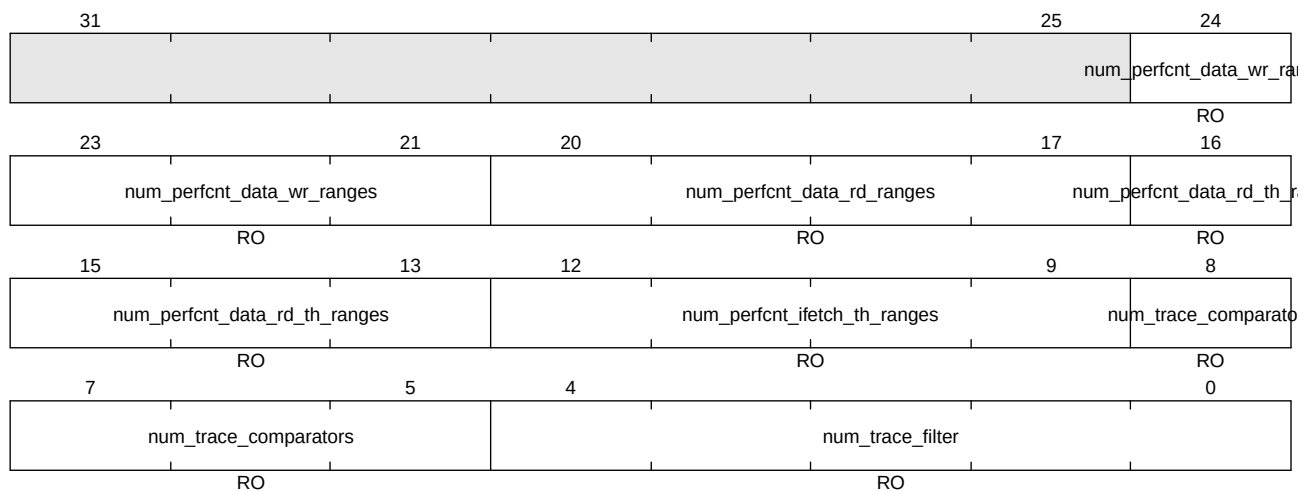
Version and component type information for the Performance Counter component. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
11:8	CompType	RO	3	Component Type (vendor: 0x3)
7:4	VerMinor	RO	0	Component Minor Version
3:0	VerMajor	RO	1	Component Major Version

16.12.3. pc.trTeConstants – offset 0x3008

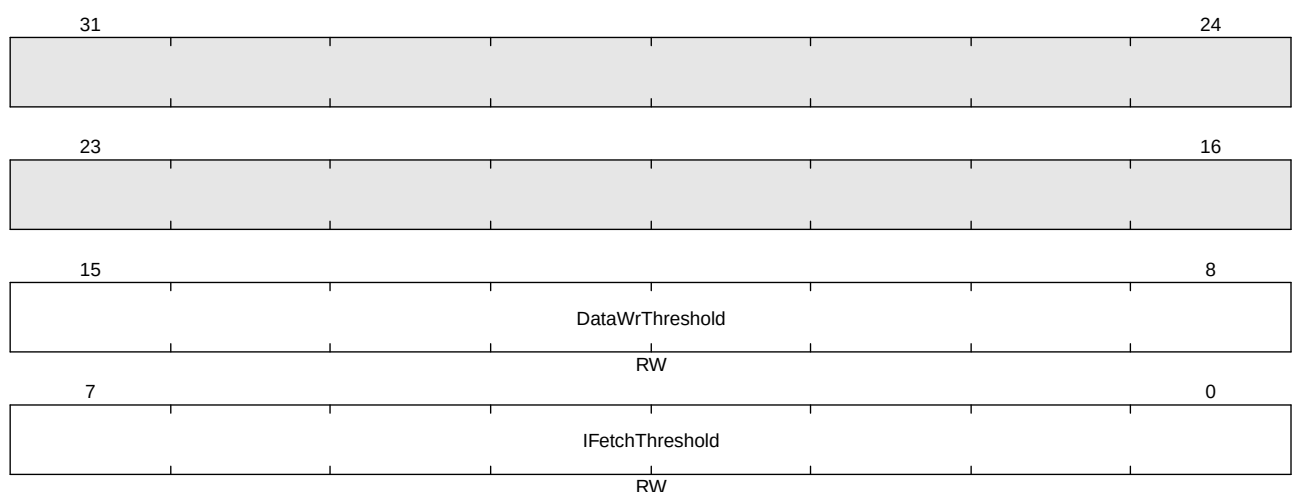
Implementatopn parameters for synchronization with RDL implementation. (RDL has no direct access PeakRDL parameters.) [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
24:21	num_perfcnt_data_wr_ranges	RO	7	NUM_PERFCNT_DATA_WR_RANGES
20:17	num_perfcnt_data_rd_ranges	RO	7	NUM_PERFCNT_DATA_RD_RANGES
16:13	num_perfcnt_data_rd_th_ranges	RO	3	NUM_PERFCNT_DATA_RD_TH_RANGES
12:9	num_perfcnt_ifetch_th_ranges	RO	3	NUM_PERFCNT_IFETCH_TH_RANGES
8:5	num_trace_comparators	RO	8	NUM_TRACE_COMPARATORS
4:0	num_trace_filter	RO	0x10	NUM_TRACE_FILTER

16.12.4. pc.trPerfCntControl – offset 0x3010

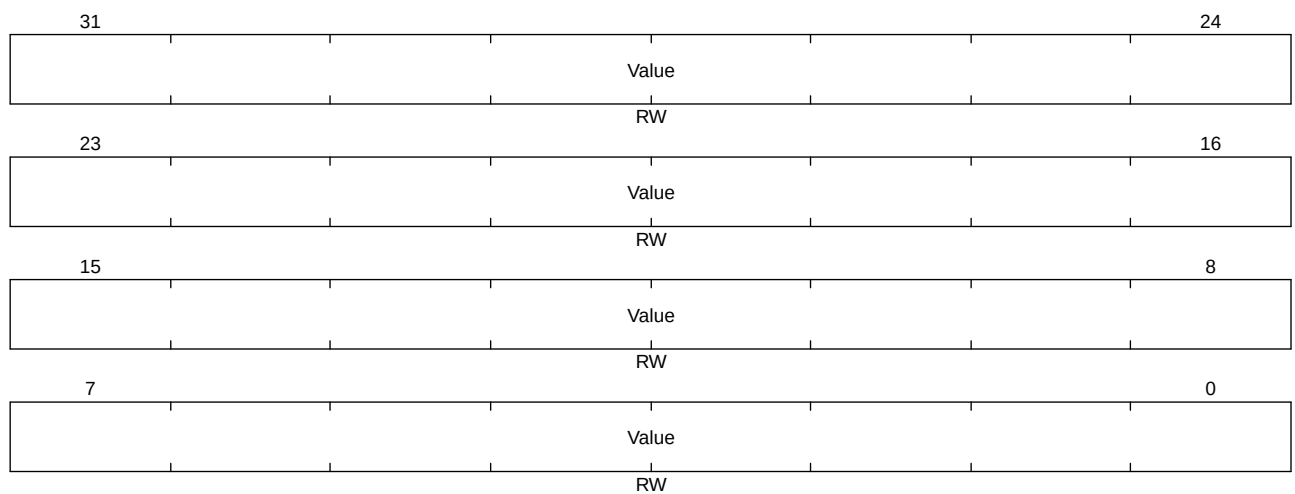
Control register for the encoder's performance counter. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15:8	DataWrThreshold	RW	4	CPU cycles threshold. When required CPU clock cycles exceed this threshold, the respective counter increments.
7:0	IFetchThreshold	RW	4	CPU cycles threshold for instruction fetch. When required CPU clock cycles exceed this threshold, the respective counter increments.

16.12.5. pc.trTePerfCntIFetchRange[] . Low – offset 0x3100

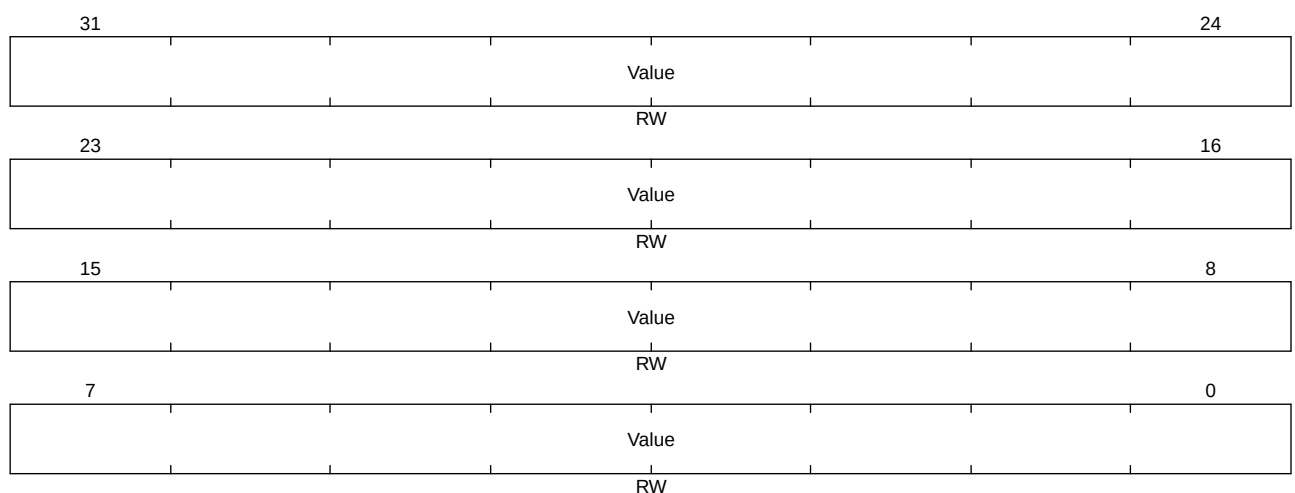
Inclusive low address of instruction fetch range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Start address of the range (inclusive). Writable only while <code>trTeControl.Enable=0</code> .

16.12.6. pc.trTePerfCntIFetchRange[] . High – offset 0x3104

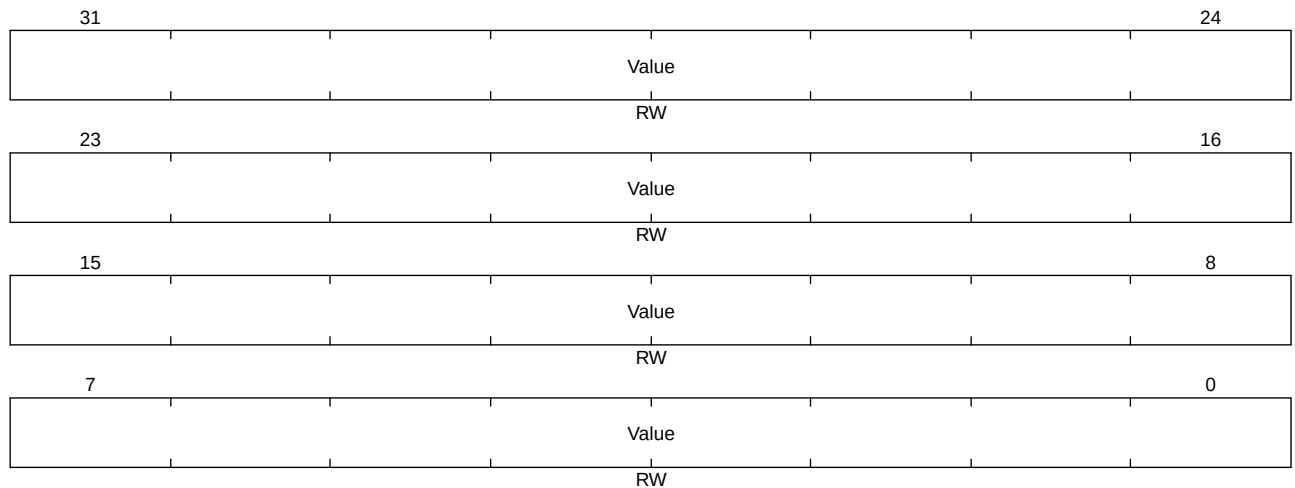
Inclusive high address of instruction fetch range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	End address of the range (inclusive). Writable only while trTeControl.Enable=0.

16.12.7. pc.trTePerfCntDataRdThRange [] .Low – offset 0x3200

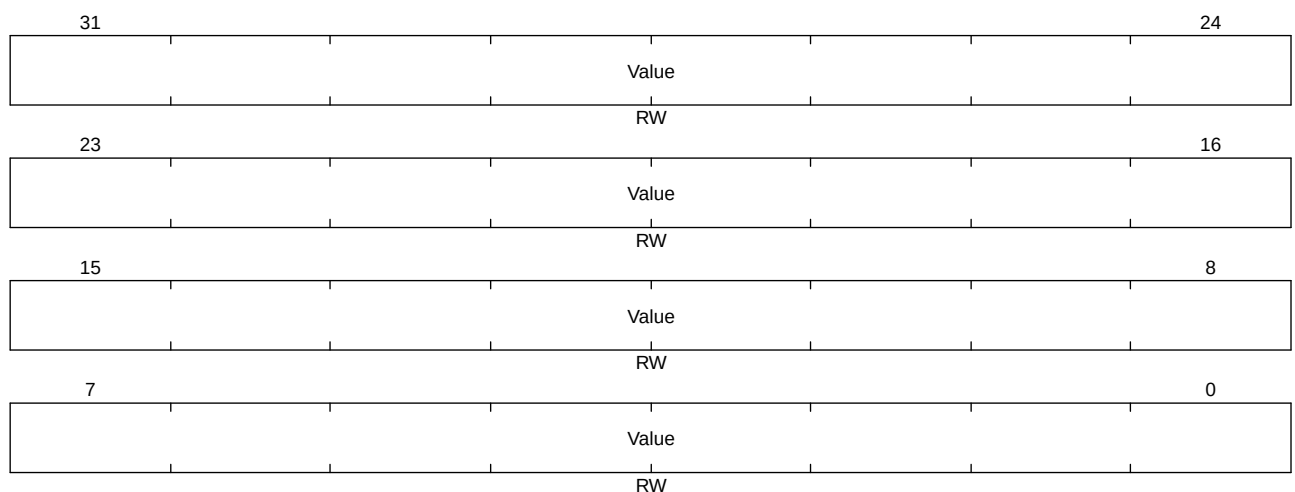
Inclusive low address of data read range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Start address of the range (inclusive). Writable only while trTeControl.Enable=0.

16.12.8. pc.trTePerfCntDataRdThRange [] .High – offset 0x3204

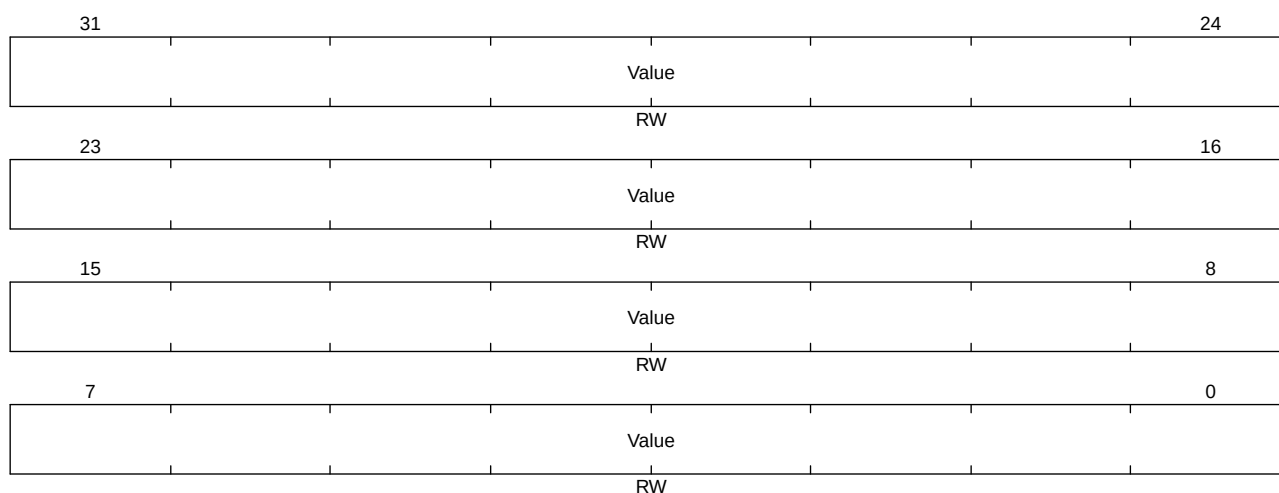
High address of data write threshold range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	End address of the range (inclusive). Writable only while trTeControl.Enable=0.

16.12.9. pc.trTePerfCntDataRdRange[] . Low – offset 0x3300

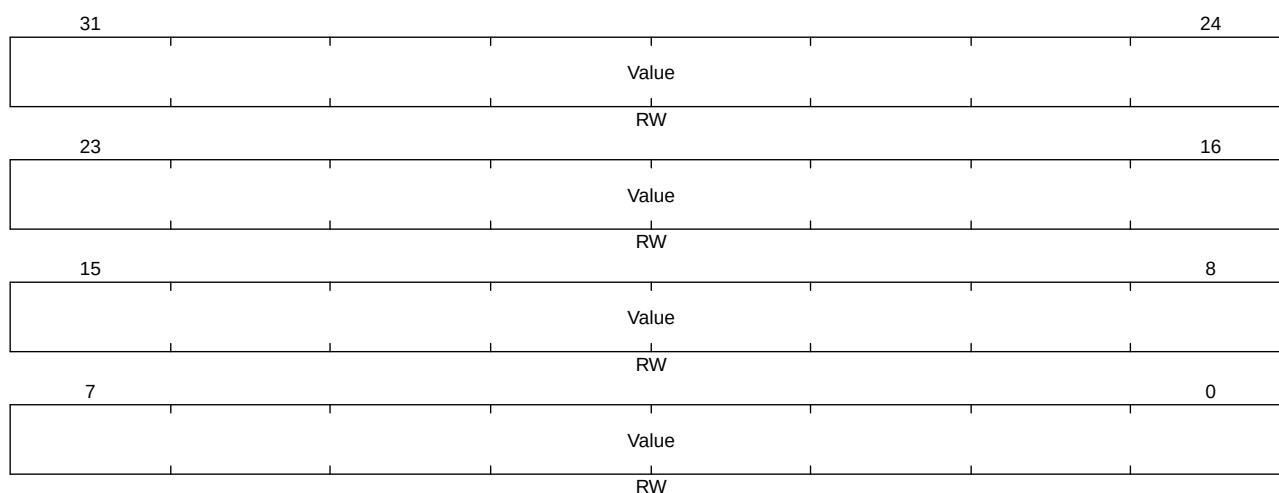
Low address of data read range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Start address of the range (inclusive). Writable only while trTeControl.Enable=0.

16.12.10. pc.trTePerfCntDataRdRange[] . High – offset 0x3304

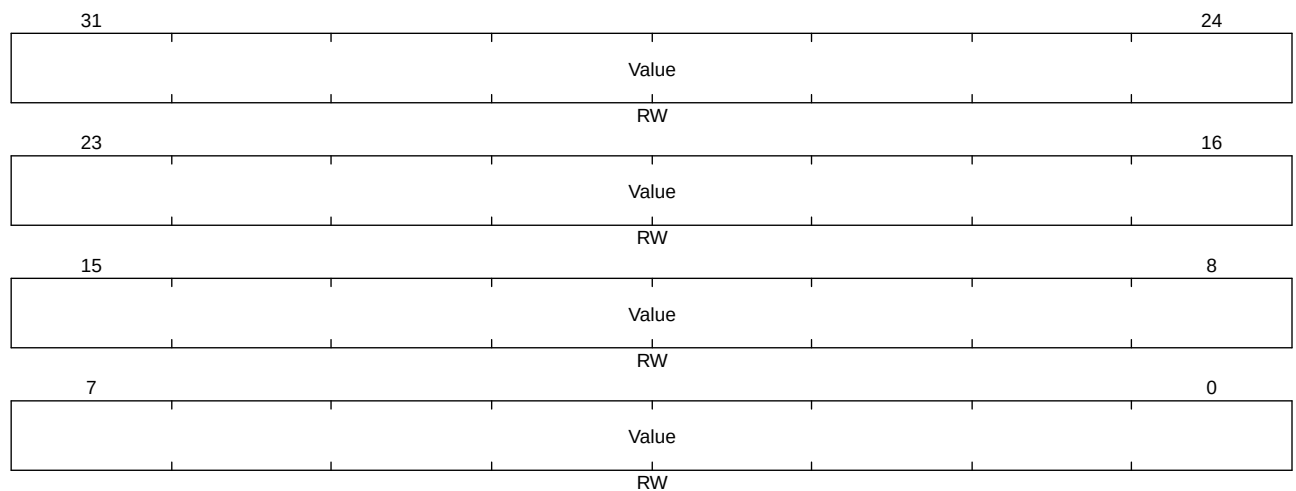
High address of data write range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	End address of the range (inclusive). Writable only while trTeControl.Enable=0.

16.12.11. pc.trTePerfCntDataWrRange[] . Low – offset 0x3400

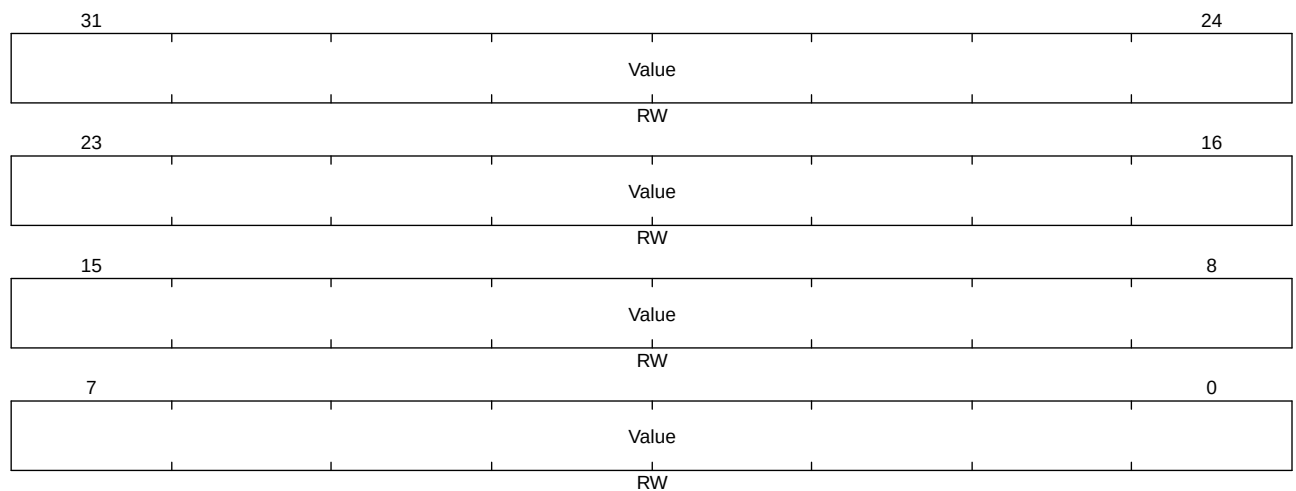
Low address of data write range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Start address of the range (inclusive). Writable only while <code>trTeControl.Enable=0</code> .

16.12.12. `pc.trTePerfCntDataWrRange[] .High` — offset 0x3404

High address of data write range.



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	End address of the range (inclusive). Writable only while <code>trTeControl.Enable=0</code> .

16.13. Component **wp** (base 0x4000)

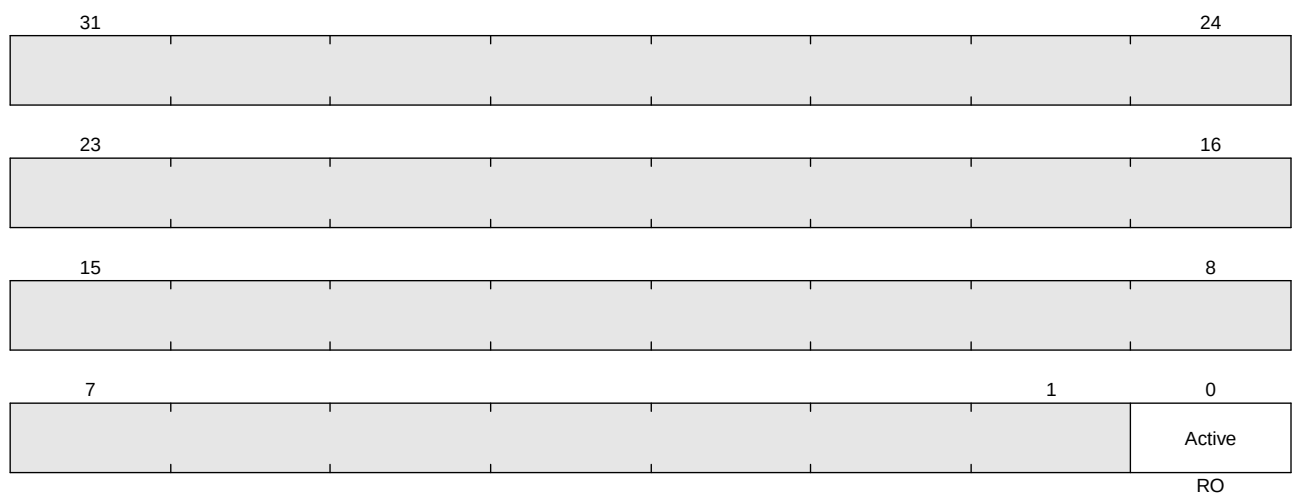
Registers the SystemRDL places directly into the address map: a component that owns an external memory cannot be a regfile.

Offset	Register	Fields
0x4000	<code>trWpControl</code>	1

Offset	Register	Fields
0x4004	trWpImpl	3
0x4008	trWpMask	1
0x400C	trWpIndex	1
0x4010	trWpDataLow	1
0x4014	trWpDataHigh	3
0x4018	trWpReadLow	1
0x401C	trWpReadHigh	3
0x4020	trWpCap	1

16.13.1. trWpControl – offset 0x4000

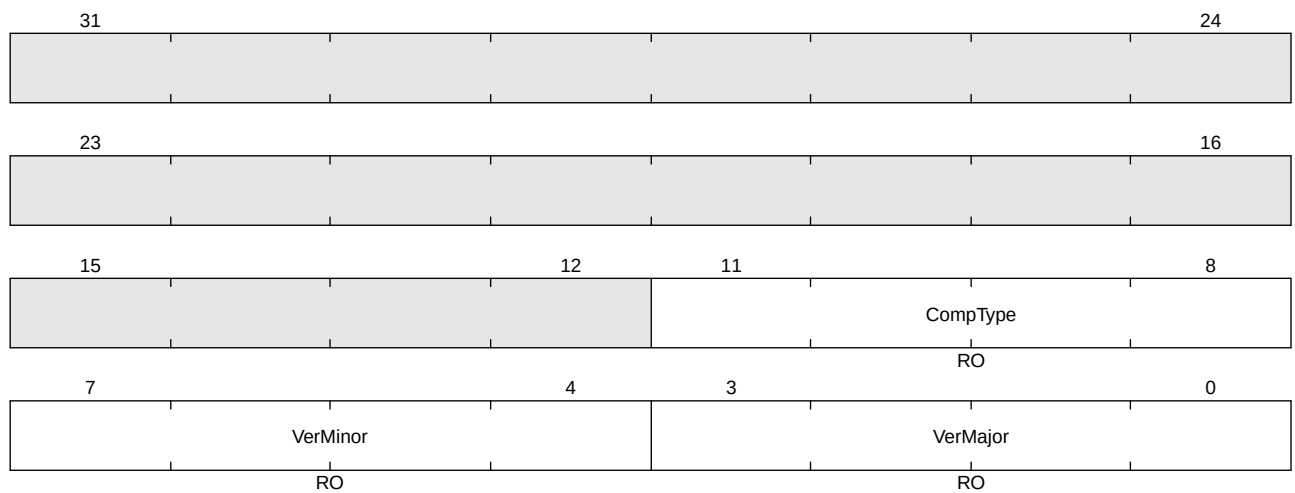
Primary activate/reset for the Watchpoints component. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
0	Active	RO	1	Primary activate/reset bit. [Not implemented: read-only constant 1; there is no clock-gate/power-down control behind it.]

16.13.2. trWpImpl – offset 0x4004

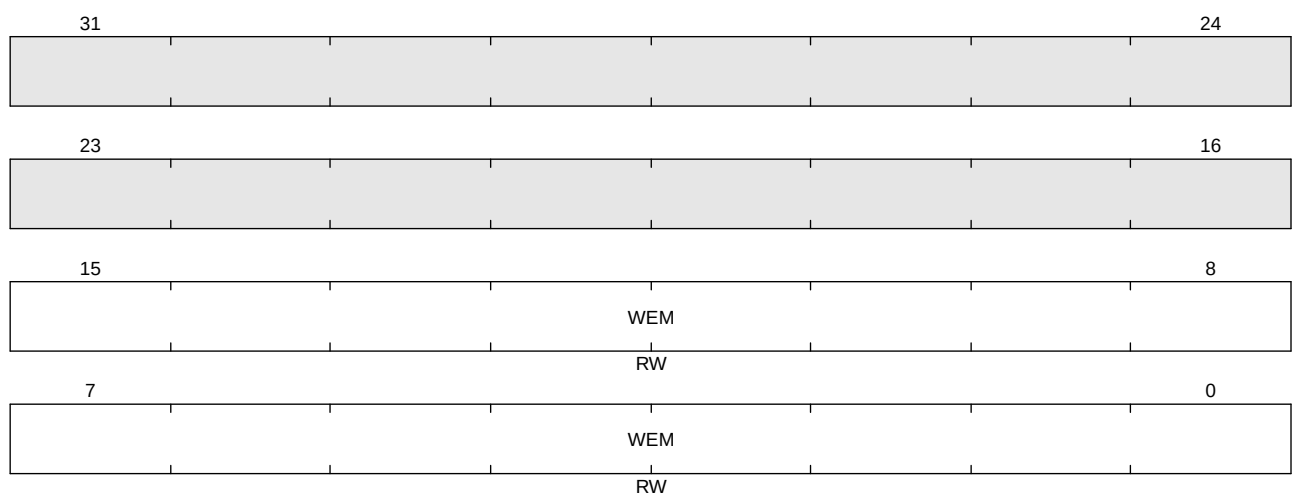
Version and component type information for the Watchpoints component. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
11:8	CompType	RO	4	Component Type (vendor: 0x4)
7:4	VerMinor	RO	0	Component Minor Version
3:0	VerMajor	RO	1	Component Major Version

16.13.3. **trWpMask** – offset 0x4008

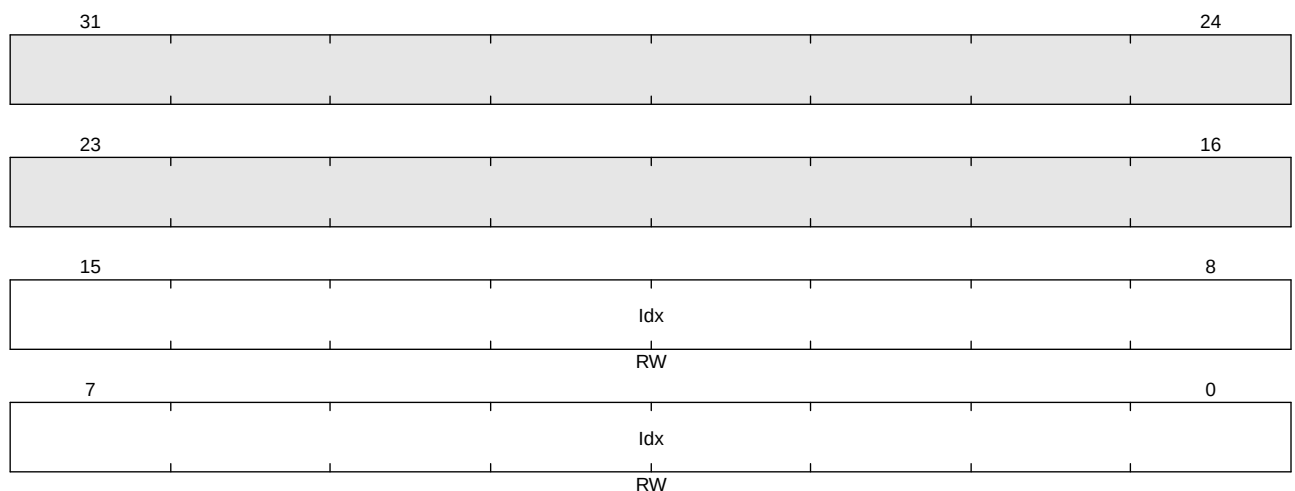
Enable mask for Watchpoint messages (Nexus TCODE 15). [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15:0	WEM	RW (Enable-locked)	0	Watchpoint Event Mask: one bit per reportable watchpoint slot. A 1 ENABLES reporting of that bit (IEEE-ISTO 5001 Table B-26, 'WEM ... 1 = enabled'). NOTE: the prose of ISTO 4.3.23 says the WPHIT field is 'masked by 1', which reads like the inverse; CTTE follows Table B-26. The emitted WPHIT of an ACT_CAP_ST_WATCHPOINT command is Cmd.DirectData[15:0] AND WEM, so which slot owns which bit is a SOFTWARE convention: program the watchpoints[] entry with the DirectData bit you want to see and enable that bit here. A command whose masked result is 0 emits no message. Reset 0 masks every hit, so adding the feature to an existing build changes no wire byte until software opts in. Writable only while trTeControl.Enable=0. Requires the CT_EN_WATCHPOINT_MSG build switch (which itself requires CT_EN_ACT); profiles without the feature read 0.

16.13.4. trWpIndex – offset 0x400C

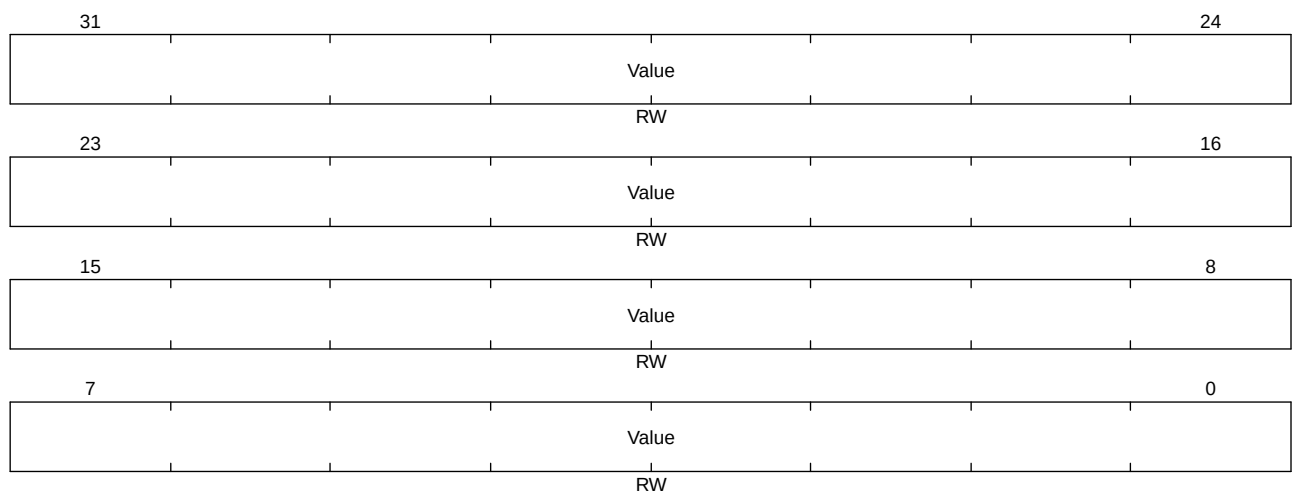
Slot index for the indirect watchpoint load/readback path. A trWpDataHigh write commits the staged entry to slot ldx and then increments ldx; a trWpReadHigh read returns the shadow entry at ldx and also increments it. Both wrap from NUM_ACT_ST-1 back to 0. Software normally writes the start slot once and streams NUM_ACT_ST Low/High pairs. Reads return the CURRENT index, so after a burst of n commits starting at i the register reads $(i + n) \bmod \text{NUM_ACT_ST}$ – cheap load verification. Values $\geq \text{NUM_ACT_ST}$ are out of range: hardware uses the low address bits and the first increment normalizes to 0. The one in-window out-of-range value, ldx = NUM_ACT_ST (= 1023), addresses the physical padding slot behind the last tree node: a commit or readback there is inert for matching (the search never visits that slot) and the following increment wraps to 0. Writable only while trTeControl.Enable=0. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15:0	Idx	RW (Enable-locked, hw-written)	0	Watchpoint slot index the next commit/readback targets. [CTTE specific]

16.13.5. **trWpDataLow** – offset 0x4010

Address (PC) that arms the watchpoint entry being staged. When the retired CPU PC matches Addr, the Cmd committed with it is dispatched through the CTTE unit. Write Low first, then **trWpDataHigh** – the High write commits the pair to slot **trWpIndex.Idx**; a Low write alone has no effect. Writable only while **trTeControl.Enable=0**. [CTTE specific]

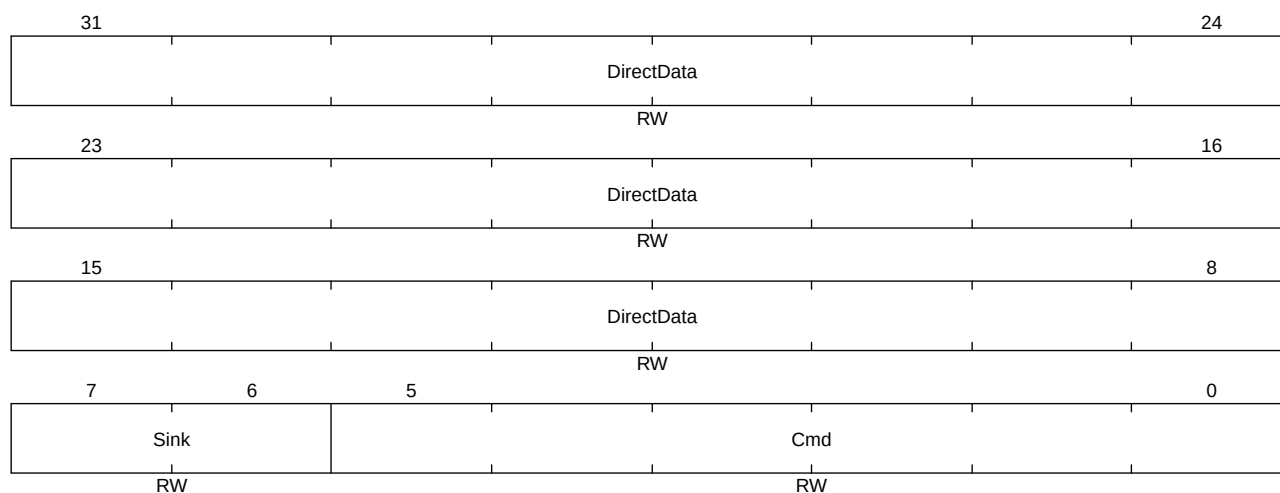


Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW (Enable-locked)	0	Addr (PC) to stage. [CTTE specific]

16.13.6. **trWpDataHigh** – offset 0x4014

Command of the watchpoint entry being staged. Writing this register COMMITTS {**trWpDataLow**, this word} to slot **trWpIndex.Idx** and increments **Idx** – identical commit semantics to the historical +0x4 write of the removed 0x4100 window. Bit layout is shared with RISC-V CSR 0x0B10 (ACT-CAP protocol), which the CPU can write directly to dispatch the same command immediately without going through the table. Writable only while **trTeControl.Enable=0** (the lock also suppresses the commit). Rules for programming the table: 1. Fill ALL **NUM_ACT_ST** slots (**trWpCap.Entries**). Unused slots are not optional – leaving a slot at its reset value can mask real watchpoints. 2. Addr values across slot 0 .. slot **NUM_ACT_ST-1** must be strictly ascending and unique (**Addr[i] < Addr[i+1]**). 3. Each slot is a pair: write **trWpDataLow** first, then this register. The High write is what commits the entry. 4. Update the entire table with tracing disabled (**trTeControl.Enable = 0**). The index/staging registers are hardware-locked while enabled; a table changed mid-trace would produce undefined matches. 5. Watchpoint messages (Cmd = **ACT_CAP_ST_WATCHPOINT**, Nexus TCODE 15): which BIT of the on-wire WPHIT field stands for which slot is a SOFTWARE convention, not hardware. Put the bit you want to see into that slot's **DirectData[15:0]** (one-hot is

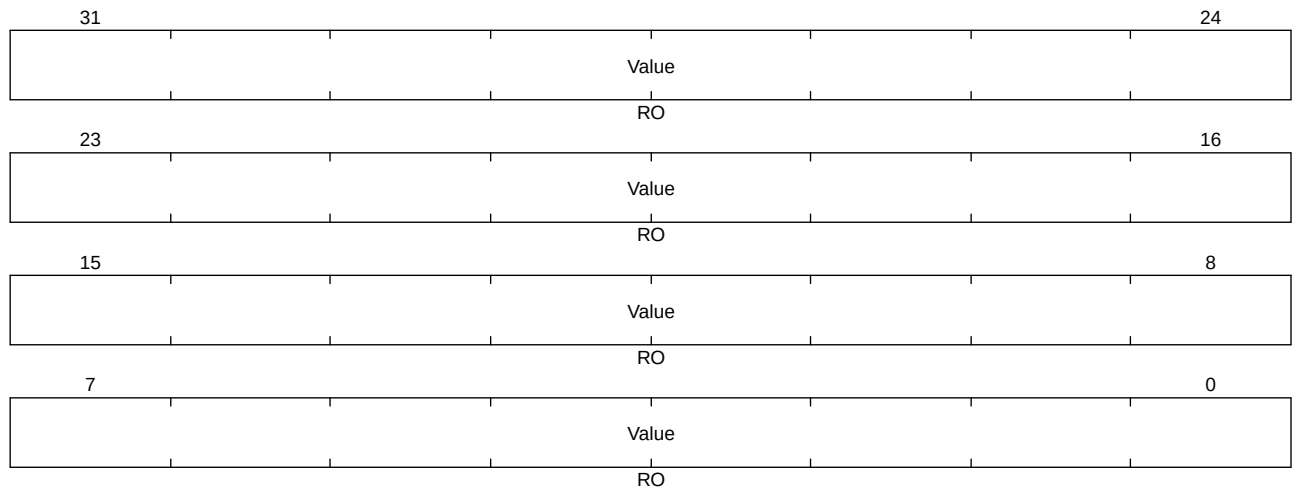
the obvious choice) and enable the same bit in trWpMask.WEM; the encoder emits WPHIT = DirectData[15:0] AND WEM and stays silent when that is 0. For slots you do not need as real watchpoints, pick an Addr the CPU will never execute (odd instruction addresses are ideal – the core forces PC[0] = 0, so odd Addr values never match) and set Cmd = ACT_CAP_ST_NONE. The Addr values of such padding slots still have to satisfy rule 2. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:8	DirectData	RW (Enable-locked)	0	Direct data (24 bits). Always included in the DAQ message payload. A value of 0 is cost-free: Nexus MDO/MSEO variable-length encoding strips leading zeros, so an all-zero field adds no MDO cycles.
7:6	Sink	RW (Enable-locked)	0	Sink (trActCapStSink_e)
5:0	Cmd	RW (Enable-locked)	0	HSI command (trActCapStCmd_e). A write commits the staged pair. [CTTE specific]

16.13.7. trWpReadLow – offset 0x4018

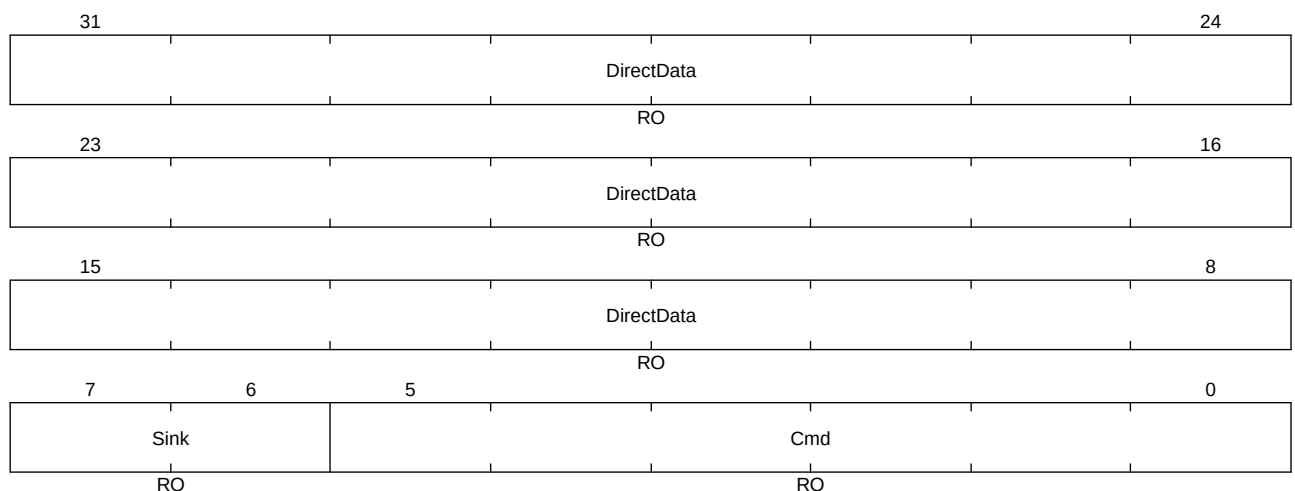
Addr word of the shadow entry at trWpIndex.Idx. The shadow is zero-initialized at configuration load and NOT cleared by a soft reset – it reflects everything committed since configuration, the same persistence the search-tree memory itself has. Reading this register does not move Idx. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RO (hw-written)	0	Shadow Addr at Idx. [CTTE specific]

16.13.8. **trWpReadHigh** – offset 0x401C

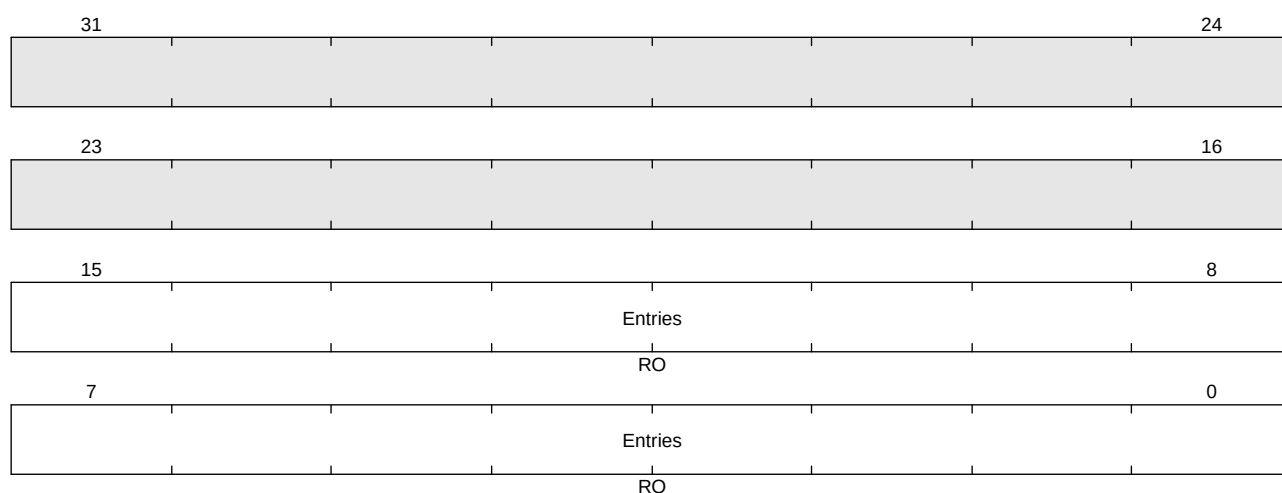
Cmd word of the shadow entry at **trWpIndex.Idx**. Reading this register increments **Idx** (wrap at **NUM_ACT_ST-1**) – serial readback: set **Idx** once, then alternate **trWpReadLow** / **trWpReadHigh** per slot. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:8	DirectData	RO (hw-written)	0	Shadow DirectData at Idx. [CTTE specific]
7:6	Sink	RO (hw-written)	0	Shadow Sink at Idx. [CTTE specific]
5:0	Cmd	RO (hw-written)	0	Shadow Cmd at Idx – reading increments Idx. [CTTE specific]

16.13.9. **trWpCap** – offset 0x4020

Number of implemented watchpoint slots (**NUM_ACT_ST**) – software discovery for the indirect load path. Profiles without the ACT blocks read 0. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
15:0	Entries	RO	0x3FF	Implemented watchpoint slots. [CTTE specific]

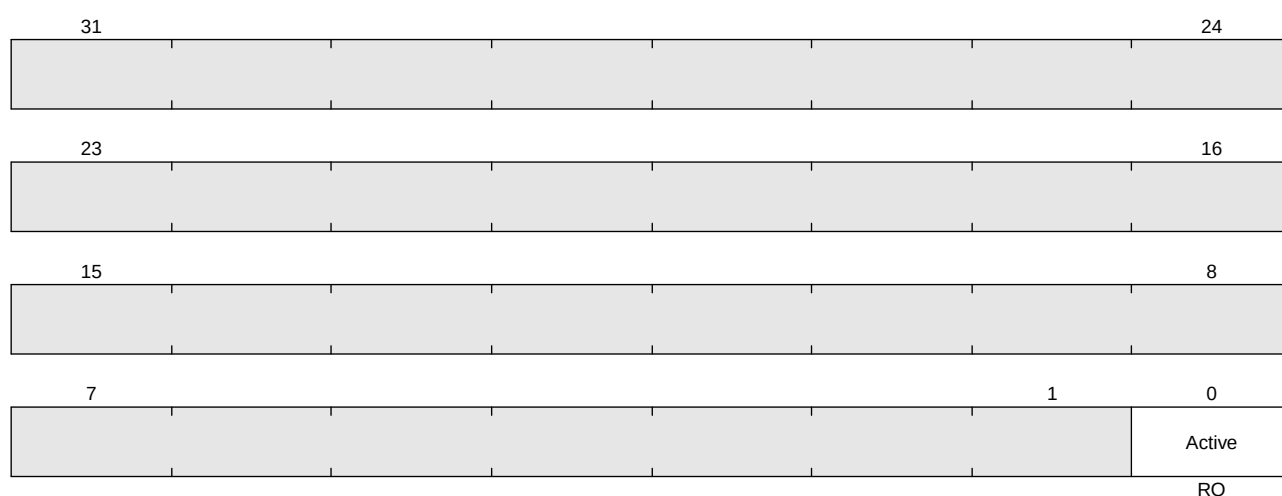
16.14. Component df (base 0x5000)

Registers the SystemRDL places directly into the address map: a component that owns an external memory cannot be a regfile.

Offset	Register	Fields
0x5000	trDfControl	1
0x5004	trDfImpl	3

16.14.1. trDfControl – offset 0x5000

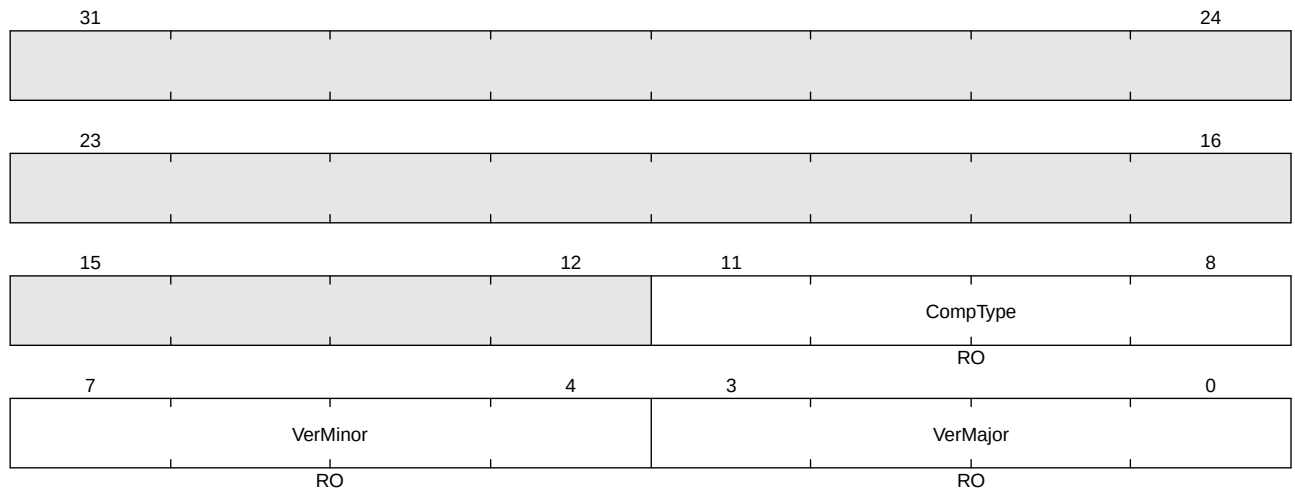
Primary activate/reset for the DF RangeFilter component. [CTTE specific]



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
0	Active	RO	1	Primary activate/reset bit. [Not implemented: read-only constant 1; there is no clock-gate/power-down control behind it.]

16.14.2. `trDfImp1` – offset 0x5004

Version and component type information for the DF RangeFilter component. [CTTE specific]



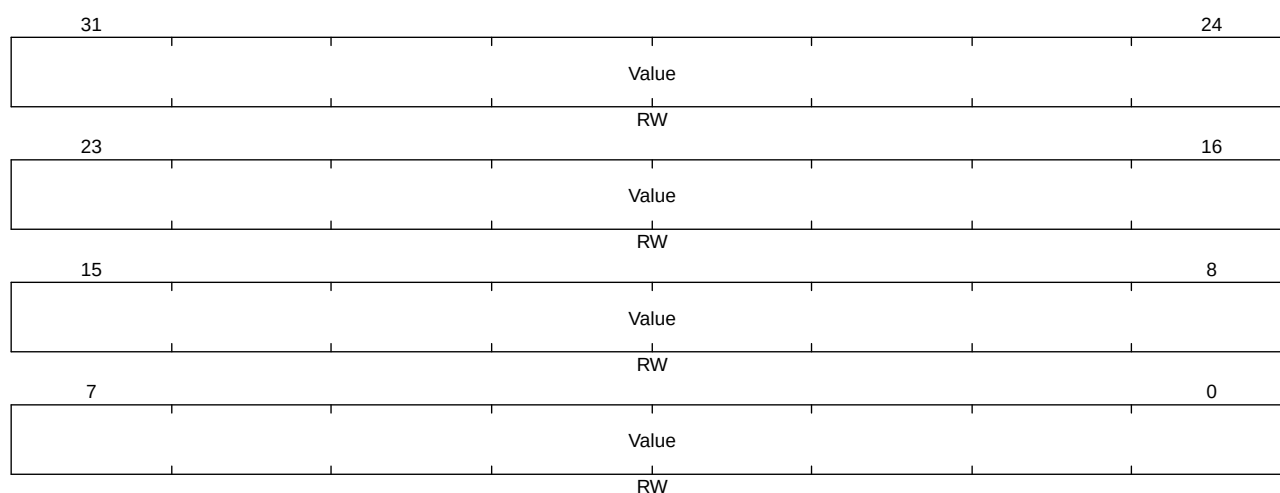
Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
11:8	CompType	RO	5	Component Type (vendor: 0x5)
7:4	VerMinor	RO	0	Component Minor Version
3:0	VerMajor	RO	1	Component Major Version

16.15. Component `mem1` (base 0x6000)

Read/write DF range filter memory. Each 64-bit entry is a {Key1, Key0} pair with Key0 $\Leftarrow$ daddr $\Leftarrow$ Key1. Software writes/reads the lower 32-bit Key0 word first, then the upper 32-bit Key1 word. Memory has NUM_DF_RANGE entries (sized to match the hardware binary-search tree depth M1_DIM; 15 entries for the current M1_DIM=4 configuration). [CTTE specific]

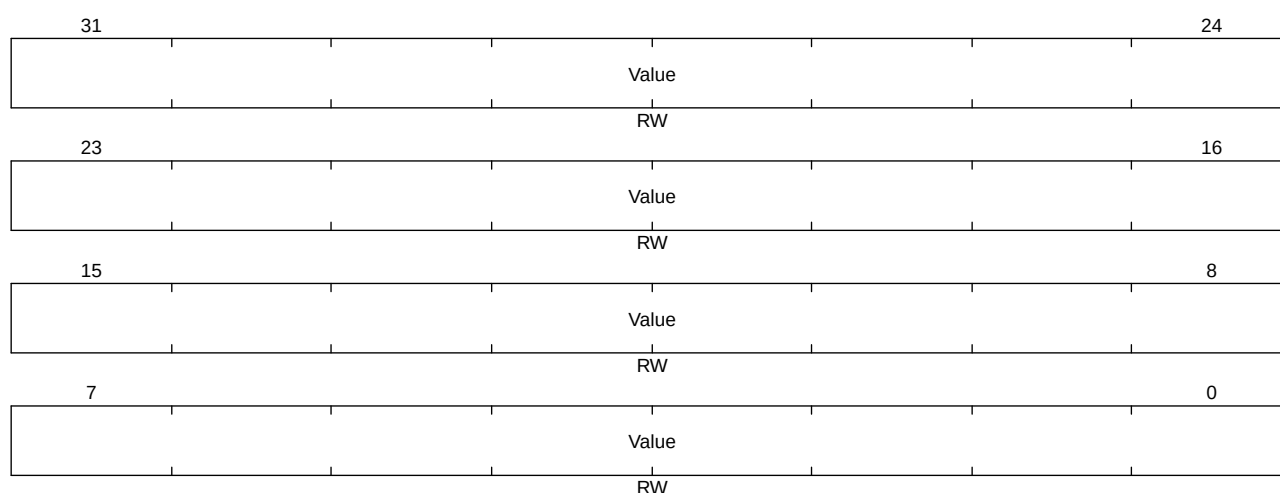
Offset	Register	Fields
0x6000	<code>mem1.Key0</code>	1
0x6004	<code>mem1.Key1</code>	1

16.15.1. `mem1.Key0` – offset 0x6000



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW	0	Lower 32-bit word of a 64-bit DF range-filter entry. Holds Key0, the inclusive low bound of the data-address range ($\text{Key0} \leq \text{daddr} \leq \text{Key1}$). Writable only while $\text{trTeControl.Enable}=0$ (wrapper-enforced; keys must stay sorted and non-overlapping). [CTTE specific]

16.15.2. mem1 . Key1 – offset 0x6004



Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:0	Value	RW	0	Upper 32-bit word of a 64-bit DF range-filter entry. Holds Key1, the inclusive high bound of the data-address range ($\text{Key0} \leq \text{daddr} \leq \text{Key1}$). Writable only while $\text{trTeControl.Enable}=0$ (wrapper-enforced; the write of this word is what commits the pair). [CTTE specific]

16.16. Declared but not addressable

The SystemRDL declares the register below and then hides it with `ispresent = false`: the hardware does not decode that address on the control bus, and no exporter – register block, C header, HTML reference – offers it. It is listed because a map that silently drops a declared register is not a complete map. The chapter named in the description carries the contract.

16.16.1. trActCapStCmd – offset 0xB10 (not decoded)

Bits	Field	Access	Reset	Description (SystemRDL, N-Trace/TCI)
31:8	DirectData	RW	0	Direct data (24 bits).
7:6	Sink	RW	0	Sink Values: 0 = ACT_CAP_ST_SINK_NEXUS; 1 = ACT_CAP_ST_SINK_AXIS; 2 = ACT_CAP_ST_SINK_AXIS_NEXUS; 3 = ACT_CAP_ST_SINK_TE.
5:0	Cmd	RW	0	HSI command Values: 0 = ACT_CAP_ST_NONE; 1 = ACT_CAP_ST_DAQ_PC_CURR; 2 = ACT_CAP_ST_DAQ_PC_CURR_LAST; 3 = ACT_CAP_ST_DAQ_DIRECT_DATA; 4 = ACT_CAP_ST_DAQ_DATA; 5 = ACT_CAP_ST_DAQ_DADDR; 6 = ACT_CAP_ST_DAQ_DATA_DADDR; 8 = ACT_CAP_ST_DAQ_IFETCH_TH; 9 = ACT_CAP_ST_DAQ_DATA_RD_TH; 10 = ACT_CAP_ST_DAQ_DATA_WR; 11 = ACT_CAP_ST_DAQ_DATA_RD; 12 = ACT_CAP_ST_CF_SYNC; 14 = ACT_CAP_ST_WATCHPOINT; 13 = ACT_CAP_ST_TE.

Chapter 17. Interface Contract

This chapter is the hardware contract of the CEDARtools.TraceEncoder: the parameters and ports of the top-level module, the clock and reset domains, the clock-domain-crossing obligations, the three data interfaces (instruction ingress, trace egress, event egress), the control interface, and the shutdown sequence.

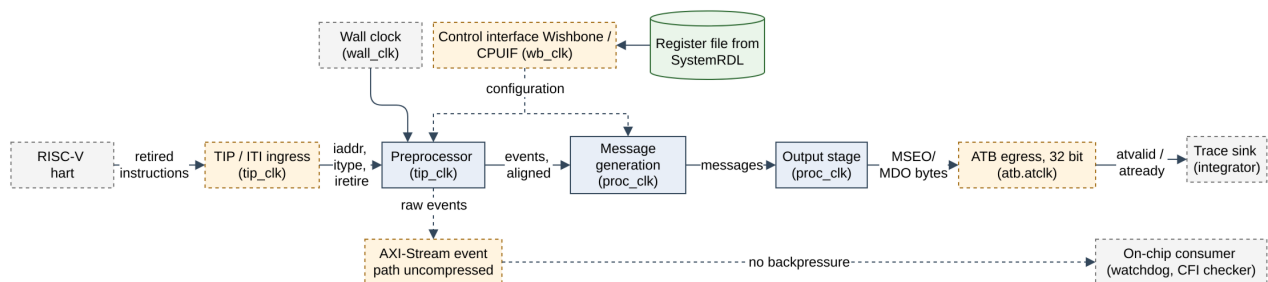


Figure 12. Interface contract: clock domains and data path

Clock domains: `tip_clk`, `proc_clk` and `atb.atclk`; the control interface runs on `wb_clk`. CDC bridges sit at the domain boundaries.

The AXI-Stream path does not sample `tready`: a back-pressuring consumer loses beats.

It, too, returned in revision 2.0. Revision 1.2 removed it, and three places in the manual went on referring to a "CTTE interface chapter" that no longer existed – which made the manual unusable for the one task it is most often opened for: integrating a core.

The condensed form here is deliberate. The public integration guide of the CEDARtools.TraceEncoder repository (`doc/integration.adoc`) carries the same substance at roughly five times the length, with the full measurement apparatus behind every figure. This chapter states the **contract**; where a detail has a longer treatment, the anchor of the public document is named so it can be followed.

17.1. Top-level module

The integration unit is the module `ct_encoder (rtl/ct_encoder.sv:50)`. Everything below is its interface.

17.1.1. Parameters

Parameter	Type	Default	Meaning
<code>SPLIT_DATA_ACCESS</code>	<code>bit</code>	<code>0</code>	Shape of the data-trace ingress. <code>0</code> = the legacy mode in which the data value is sampled from <code>tip.data</code> at retirement. <code>1</code> = split-load mode: <code>tip.sdata</code> carries store data at retirement, <code>tip.lresp</code> and <code>tip.ldata</code> carry the load response when it arrives.

Parameter	Type	Default	Meaning
CT_SIJUMP	bit	0	Integration attribute, not a feature switch: declares that the attached core's ingress uses the sequentially-implicit-jump convention. It has no effect inside the encoder; it is advertised to the decoder as capability bit 6 so that the decoder knows to apply the matching convention. The MicroBlaze™ V adapter sets it.
CT_DEVICE_ID	logic [31:0]	0	The static identity of this encoder instance, laid out per IEEE-ISTO 5001-2012 Table B-5. 0 means "no device ID assigned". Nothing reaches the wire until software sets <code>trTeControl.SendDeviceId</code> ; see Chapter 18 .
EN_NTRACE / EN_ETRACE	bit	build profile	Per-instance protocol back end. Exactly one must be set – elaboration aborts if both or neither are (<code>rtl/ct_encoder.sv:220</code>). A multi-encoder system-on-chip can therefore mix protocols out of one set of netlist sources. The historical dual build with a runtime protocol select is retired.
CORE_XLEN	int unsigned	0 (invalid)	The integrator's declaration of the architectural address width of the hart this instance's ingress is wired to. It must equal the netlist's own <code>ct_pkg::CT_XLEN</code> or elaboration aborts (<code>rtl/ct_encoder.sv:294</code>). See the note below – this parameter has no useful default and the invalid default is the point.

IMPORTANT

CORE_XLEN exists because nothing else can see a width mismatch. The ingress interface is parameterless; its address fields are sized by the netlist's CT_XLEN. An adapter that drives a 64-bit program counter into a 32-bit netlist has its upper address bits truncated by SystemVerilog's silent width adaptation **before** the encoder sees anything. Inside the encoder those bits never existed, so no assertion, no capture and no decoder downstream can miss them. The declaration is the only place where both widths are known at once – which is why its default is an error rather than a match: an integrator who says nothing would otherwise keep exactly the silent truncation the parameter exists to stop.

The same guard is written twice, as a bare elaboration `$fatal` and as an `initial $fatal`, because one simulation backend in the tool flow runs with `-Wno-fatal`, which demotes the elaboration form to a warning and lets the run continue. Measured on 2026-08-12 before the fix: 15 ATB transfers were traced out of a declared 64-bit hart on a 32-bit ingress (`rtl/ct_encoder.sv:294-301`).

Four further elaboration guards enforce contracts that are otherwise silent: `ct_pkg::CT_XLEN` must be 32 or 64 (`:256`); `EN_ETRACE` requires the corresponding build switch, or the ingress sideband fields are one-bit stubs (`:246`); `ct_pkg::CT_CONTEXT_WIDTH` must fit the message field (`:310`); and with block ingress enabled, `CT_IRETIRE_WIDTH` must lie between 2 and the instruction-count field width (`:332`). The combination of the E-Trace back end with 64-bit addresses is rejected (`:346`).

17.1.2. Ports

Port	Kind	Domain	Role
tip_clk, tip_rst	input	—	Ingress clock and its synchronous, active-high reset.
tip	tip_if.slave	tip_clk	Instruction and data ingress — see Section 17.5 .
wb_clk, wb_rst	input	—	Control-interface clock and its synchronous, active-high reset.
wb	wb_if.slave	wb_clk	Register access — see Section 17.8 .
ct_cs_rst	input	wb_clk	Additional register-block reset, logically OR-ed with wb_rst at every register in that domain.
axis	axis_if.master	tip_clk	Uncompressed instrumentation egress — see Section 17.7 .
atb_atclk, atb_atresetn	input	—	Egress clock and its active-low reset.
atb	atb_if.master	atb_atclk	Trace egress — see Section 17.6 .
atb_te_raw	output	atb_atclk	Framing advertisement: 0 = Nexus chunks, 1 = E-Trace raw bytes. Constant per instance, tied to EN_ETRACE.
proc_clk, proc_rst	input	—	Internal pipeline clock and its synchronous, active-high reset.
wall_clk, wall_clk_rst	input	—	Free-running wall-clock timestamp reference and its synchronous, active-high reset.

NOTE

The AXI-Stream interface carries its own `aclk` / `aresetn` in the interface definition rather than as ports of `ct_encoder`. The encoder drives that interface from the ingress domain, so the integrator binds them to the ingress clock and reset. It is not a sixth clock domain.

17.2. Clocks

Five clock domains, each with its own reset. A fully single-clock integration — all five tied together — is supported and is the configuration the hardware robustness campaign runs on; a structural build knob removes the internal crossing logic in that case.

Clock	Drives	Constraint
tip_clk	Instruction ingress and the preprocessor front end	Must be the core clock. The ingress bus is synchronous to retirement; there is no handshake to make it otherwise.
proc_clk	Message generation and formatting	May equal the ingress clock. One-to-one is the proven board profile; a faster processing clock buys headroom.

Clock	Drives	Constraint
atb_atclk	Trace egress and the Empty status view	Sink-side clock. One-to-one with the ingress clock is proven on hardware. A slower drain does not lose data silently – it raises the rate of announced-loss episodes.
wb_clk	Register access	Any frequency.
wall_clk	Wall-clock timestamp source	Optional feature clock.

17.3. Resets

Reset	Polarity	Notes
tip_rst	active high	Synchronous to tip_clk.
proc_rst	active high	Synchronous to proc_clk.
wb_rst	active high	Synchronous to wb_clk.
ct_cs_rst	active high	Synchronous to wb_clk; always OR-ed with wb_rst.
atb_atresetn	active low	Synchronous to atb_atclk; inverted internally.
wall_clk_rst	active high	Synchronous to wall_clk.

The encoder takes its reset inputs as already synchronous to their domains; it instantiates no reset synchronizer of its own at the top level. No minimum reset-hold length is specified.

17.3.1. Resetting one domain without the others

This is an explicit contract, not an accident of the implementation (`doc/integration.adoc`, anchor `independent-resets`). An integrator may hold the ingress domain – and with it the whole core – in reset while the control domain keeps running. That is what makes the encoder programmable across a core reset.

One property is deliberate and worth knowing, because a naive design would not have it: **an explicit synchronization request survives a reset of the consuming domain**. The request crosses as a four-phase level handshake, so an ingress-domain reset clears the generator's pending state without acknowledging the request; the still-standing request is simply seen again when the domain returns. The same holds for the egress-side sync request. Proven formally for a single clock (`formal/preproc_sync`, task `tereqrst`) and measured at two unrelated clocks (`scripts/cli_syncreqrst_test.sh`) in the CEDARtools.TraceEncoder repository; the same measurement shows that a pulse-based request would be dropped and the requesting side would wait forever.

Two limits belong with that guarantee:

- The direction matters. A reset of the **control** domain resets the launching side, where a request has nowhere to survive: a request written but not yet served is gone, with no message, no error and no status bit. Survival is claimed for the ingress reset only.
- It may cost one extra synchronization message, if the reset lands after the generator served the

request but before the launching side saw the acknowledgement. A synchronization message is always safe to emit, so this is a byte, not a hazard. Exactly-once delivery across a one-sided reset is not claimed.

Everything else resets per domain in the ordinary way: state in the reset domain is gone, and the stream re-anchors on the first qualifying retirement afterwards.

17.4. Clock-domain crossing

All crossings are internal. The integrator connects clocks and resets and applies the constraint fragments; there is no crossing the integrator has to build.

Primitive	Used for	Constraint fragment
signal_cdc	A slowly changing single bit; two-flop synchronizer with the synthesis attributes that keep it from being re-timed or absorbed into a shift register.	signal_cdc.xdc
strobe_cdc	A single-cycle strobe, carried as a toggle and recovered by an edge detector.	(via signal_cdc)
vector_cdc2	A multi-bit value, carried by a four-phase request/acknowledge handshake so the receiver never samples a partially updated vector.	—
reset_cdc	Asynchronous-assert, synchronous-deassert reset synchronization.	reset_cdc.xdc
Gray-pointer CDC FIFO	The trace payload itself between the processing and egress domains.	fifo2clk_fwft.xdc

Two obligations fall on the integrator:

Include the constraint fragments. They ship next to the RTL and carry the false paths and datapath-only maximum delays the synchronizers need. The register-distribution bundles need one in particular: the control/status values that reach the ingress, processing and egress domains are pseudo-static and combinational by design, and `rtl/pkg/ct_cs_cpuiif_wb.xdc` false-paths them as a group. Without it, a timing tool reports a large set of impossible cross-domain paths.

Group the clocks as asynchronous at the top level. The encoder's own out-of-context synthesis figures assume exactly that grouping; cross-domain paths are outside that analysis by construction, which makes grouping them a system-level responsibility rather than an optional tightening.

17.5. Instruction ingress

The ingress is the interface `tip_if` (`rtl/pkg/tip_if.sv:21`), with its types in `rtl/pkg/tip_pkg.sv`. It is a fixed-latency retirement bus with **no handshake**: there is no ready signal and no backpressure toward the core. That is a deliberate property, not an omission — see the note at the end of this section.

17.5.1. Signals

Signal	Width	Meaning
Instruction flow		
<code>iretire</code>	1 or CT_IRETIRE_WIDTH	Retirement indication. One bit in the default mode; a half-word count in block mode.
<code>iaddr</code>	CT_XLEN	Address of the retired instruction (block mode: of the first instruction of the block).
<code>ilastsize</code>	2	Size of the retired instruction (block mode: of the last instruction of the block).
<code>itype</code>	4	Type of the retirement (block mode: how the block terminated).
<code>ecause</code>	4	Exception cause; meaningful only when <code>itype</code> is the trap type.
<code>tval</code>	CT_XLEN	Trap value.
<code>priv</code>	3	Privilege level.
<code>_context</code>	CT_CONTEXT_WIDTH	Context identifier.
<code>ctype</code>	2	Context-report type; a non-zero value is a context event.
<code>impdef</code>	8	Implementation-defined sideband.
<code>_time</code>	64	Time reference.
Data flow		
<code>dretire</code>	1	Data access retirement.
<code>dtype</code>	4	Access type; the load encoding is 0, every other value is a write-class access.
<code>daddr</code>	CT_XLEN	Data address.
<code>dsize</code>	6	Access size as a base-two logarithm of the byte count.
<code>data</code>	64	Data value in the legacy (non-split) mode.
<code>sdata</code>	32	Store data, valid at <code>dretire</code> , in split-load mode.
<code>lresp</code>	2	Load response, valid when its upper bit is set; 2 = ok, 3 = error.
<code>ldata</code>	32	Load data, valid with the load response.
Event sideband		
<code>debug_mode</code>	1	Level. Debug mode is active.
<code>power_down</code>	1	Level. The hart is powered down.
<code>evti</code>	1	Pulse , at least one clock wide. External trace trigger input.
<code>trigger</code>	1	Pulse. Watchpoint or trigger-fabric event.

17.5.2. Beat semantics

- One beat per retirement opportunity. `iretire` marks a retired instruction; `iaddr` and `ilastsize` must be valid on that beat.
- **A synchronously faulting instruction does not retire.** It is signalled with `iretire` = 0 and the trap type, and its program counter never appears in the decoded stream. Interrupts likewise use

`iretire = 0`; the pre-empted instruction is re-executed after the return from the trap and appears then. This one-to-one retirement rule is what lets a consumer treat the decoded stream as an execution record rather than an attempt record.

- `itype` must be valid whenever it is not the "other" value, **even on a beat with `iretire = 0`** – trap markers ride non-retiring beats.
- The two level signals must rise after the last retirement before the window and fall before the first retirement after it. Debug-mode entry produces a Correlation message, its exit a re-anchor with the exit-from-debug reason; power-down behaves the same way with its own reason code. While either level is high, no instruction trace is generated or counted.
- An adapter that has no source for a sideband signal ties it to zero. All three current core adapters do this for the debug-related inputs, which is why the Debug Status message is not implemented – see [Chapter 14](#).

17.5.3. Block ingress

Setting the block-ingress build switch redefines the bus so that one beat can report a whole basic block, and widens the retirement field to `CT_IRETIRE_WIDTH`:

Signal	Default meaning	Block meaning
<code>iretire</code>	one instruction retired	half-words retired by the block
<code>iaddr</code>	that instruction's address	address of the first instruction
<code>ilastsize</code>	that instruction's size	size of the last instruction
<code>itype</code>	that instruction's type	how the block terminated

The rules that follow from it:

1. Only the last instruction of a block may be a control-flow event. The encoder derives the control-flow source address as `iaddr + 2 * (iretire - 2^ilastsize)` and the successor as `iaddr + 2 * iretire`.
2. `iretire` must fit `CT_IRETIRE_WIDTH`, which elaboration constrains to 2..8. A core whose block generator has no natural bound must have its adapter split long linear runs into several blocks that terminate with the "other" type. That is free on the wire: such a block raises no message, it only accumulates the count.
3. Trap beats are unchanged – still `iretire = 0`.
4. Address comparators, the watchpoint search and the performance counters compare one address per beat, which in block mode is the block start. **Instruction-granular address filtering is not expressible on a block ingress.** That is a property of the port, not a shortfall of the filters.

The switch adds no message type and no wire field, and carries no capability bit: in the reset configuration a block beat produces exactly the messages a serialized beat sequence would, with the same instruction count. That equivalence is measured, not asserted (`scripts/cli_blocktip_test.sh`).

What is **not** invariant: the periodic-synchronization cadence mode that counts retirement beats counts beats, not instructions, so periodic synchronization lands after a different amount of program.

17.5.4. Core adapters

The adapters that translate a specific core's trace port into this interface live outside the encoder – for the AMD MicroBlaze™ V trace bus, for the CVA6 instruction trace interface fed from its verification interface probes, and for the Rocket trace core interface. Their port mappings and instruction-type semantics are in [Chapter 6](#).

NOTE

The absence of backpressure is a contract in both directions. The core is expected to always be able to drive the ingress; the encoder is expected never to stall the core. When the encoder's internal queues cannot keep up, the result is an **announced** loss on the egress side plus a counted overflow in the status registers – never a silent gap and never a stalled hart. The one standard feature this forecloses is the data-trace hart-stall policy: stalling a hart needs a back channel, the ingress port is unidirectional by construction, and no adapter could route one. The implemented alternative is the data-drop policy described in [Chapter 14](#).

17.6. Trace egress

The trace stream leaves through the interface `atb_if` (`rtl/external/amba/atb_if.sv:14`).

Signal	Width	Direction	Meaning
<code>atdata</code>	32	encoder → sink	Trace bytes.
<code>atbytes</code>	2	encoder → sink	Number of valid bytes on <code>atdata</code> .
<code>atid</code>	7	encoder → sink	Stream identifier.
<code>atvalid</code>	1	encoder → sink	Beat valid.
<code>atready</code>	1	sink → encoder	Beat accepted.
<code>afvalid</code>	1	sink → encoder	Flush request.
<code>afready</code>	1	encoder → sink	Flush acknowledgement.
<code>syncreq</code>	1	sink → encoder	Synchronization request from the sink side.

Backpressure through `atready` is legal at any time and for any duration. Sustained backpressure raises the rate of overflow episodes, and an overflow episode is **announced**: an Error message plus a recovery re-anchor tell the decoder exactly what it lost and where to resume. Nothing is dropped silently.

The flush handshake is: assert `afvalid`, wait for `afready` to fall, wait for it to rise, release `afvalid` (`rtl/external/amba/atb_pkg.sv:18` carries the reference sequence as a task).

The framing on this port is advertised by `atb_te_raw` and is constant per instance. A funnel or sink

downstream therefore parses the framing the netlist actually produces, rather than a second, software-programmed copy of that fact.

17.7. Event egress

Independently of the trace port, an AXI-Stream master (`rtl/external/amba/axis_if.sv:12`) carries instrumentation events to consumers inside the fabric. It exists for the case where the instrumentation data is **processed on chip** rather than exported.

The stream carries only the command-driven instrumentation path, and it carries it **uncompressed**: one fixed-format beat per command, with `tid` set to the command code so the beat is self-describing, and `tstrb` marking which of the fixed-width element slots are valid. `tkeep`, `tlast`, `tdest` and `tuser` are present in the interface but not driven.

The difference to the same event on the trace port is worth stating, because the two paths are easy to confuse: the trace port carries the event as a Data Acquisition message, variable-length and leading-zero-suppressed, framed into the Nexus byte stream (see [Chapter 15](#)); the AXI-Stream path carries it as one raw beat with no framing and no compression.

17.8. Control interface

Registers are reached over a Wishbone slave (`wb_if`, `rtl/external/wishbone/wb_package.sv:75`), 32 bits wide, with a 32-bit address of which 15 bits are decoded – a 32 KiB window. Internally the Wishbone transaction is adapted to the register block's own request interface by `rtl/external/wishbone/wb_to_cpuif.sv`, instantiated in its combinational form; an AXI4-Lite bridge is available a level up for systems whose control fabric is AXI.

The control interface is its own clock domain. Every value that has to reach the ingress, processing or egress domain crosses through one of the primitives listed above, inside `rtl/pkg/ct_cs_cpuif_wb.sv`.

The register map itself, the programming order and the Enable lock are in [Chapter 16](#).

17.9. Shutdown

Reaching a quiescent encoder is a sequence, and the order is not interchangeable:

1. **Flush** – assert the flush request on the trace port, or use the sink's flush control.
2. **Stop tracing while the core is still retiring** – clear `trTeControl.InstTracing`.
3. **Stop the core.**
4. **Poll `trTeControl.Empty`** until it reads 1, with `trTeControl.Active` still set.

Step 2 is the one that is easy to get wrong, and the failure is confusing rather than obvious. With

instruction tracing still enabled and the core already stopped, exactly one control-flow event is permanently in flight — its successor address can never be observed, because no further instruction will retire. `Empty` then reads 0 forever, and that report is **correct**: there really is unresolvable trace in the pipeline. Closing the stream while the core still retires resolves that last event.

`trTeControl.Empty` is a conservative, crossing-safe status: it rises only after the entire chain — accumulators, formatter, packer, crossing queues, egress queue — has been observably quiet, and it falls immediately on any visible substance.

This sequence is contract-tested rather than merely documented; the verification chapter names the gate.

17.10. Feature switches, runtime controls and capability bits

Each optional feature of the encoder exists at three levels, and an integrator needs all three: a compile switch decides whether the netlist contains it, a register field decides whether it is active in this session, and a capability bit tells the decoder both facts over the wire. The table below is the mapping, condensed from the reference table in `doc/integration.adoc` (anchor `feature-flags`) of the CEDARtools.TraceEncoder repository.

Two columns need a word of reading instruction. **Reset** is the reset value of the runtime control, which is what an encoder does if nobody configures it — and it is 0 almost everywhere, deliberately: the unconfigured stream stays inside the standard message set. **CAPS** is the bit position in the capability map of the Vendor Config message ([Chapter 18](#)); a dash means the feature is not advertised, which for most of them is because it changes no wire format.

Build switch	Runtime control	Reset	CAPS	Slim reference profiles
CT_EN_NTRACE / CT_EN_ETRACE	<code>trTeProtocolSel.Protocol</code> (read-only discovery, driven by the instance parameters)	—	—	as built
CT_EN_BLOCK_TIP	— (build-time shape of the ingress)	—	— ^ (no message type, no wire field) ^	off
CT_EN_BTМ	<code>trTeControl.InstMode = 3</code> (WARL; legal only if compiled in)	6	—	on
CT_EN_CONFIG_MSG	<code>trTeControl.SendConfig</code>	1 (CFG_ONCE)	— ^ (by design: it is the feature that transmits the map) ^	on
CT_EN_OWNERSHIP	<code>trTeControl.Context</code>	0	7	on
CT_EN_DATA_TRACE	<code>trTeDataControl.DataTracing</code>	0	16	off

Build switch	Runtime control	Reset	CAPS	Slim reference profiles
CT_EN_DF_ADDR_COMPRESS	trTeDataControl.DataAddrCompress (WARL; requires data trace)	0	21	off
CT_EN_DF_DROP	trTeDataControl.DataDropEna (runtime policy, not Enable-locked)	0	22	off
CT_EN_DAQ	— (command-driven through the instrumentation command register)	—	17 ^(shared)	off
CT_EN_ACT	indirect watchpoint table, performance-counter registers (no single enable bit)	—	17 ^(shared)	off
CT_EN_FILTERS	trTeInstFilters, trTeDataFilters, trTeFilter[], trTeComp[]	0	—	off
CT_EN_FILTER_SYNC	none of its own — armed by a non-zero instruction filter	0	— [^] (a filter gap uses the existing pause vocabulary) [^]	off
CT_EN_TIMESTAMP	trTsControl.Enable (plus type, prescale, width)	1	15	on / off
CT_EN_AXIS_TS	— (build-time shape of the event beat; requires timestamps and the instrumentation blocks)	—	— [^] (not a Nexus wire format) [^]	off
CT_EN_DEBUG_EVENTS	— (ingress event port)	—	10	on
CT_EN_POWER_EVENTS	— (ingress event port)	—	11	on
CT_EN_EVTI	— (ingress event port)	—	12	on
CT_EN_TRIG_SYNC	trTeControl.InstTrigEnable	0	13	on
CT_EN_SEQ_SYNC	trTeControl.InstSeqSyncEnable	0	14	on
CT_EN_TRIG_REGS	trTeTrigExtInControl.ExtInAction 0 (WARL)	0 (no action)	— [^] (the on-wire effect is capability bit 13) [^]	on
CT_EN_QUOTA_SYNC	trTeControl.InstSyncMode = 1 or 4, with InstSyncMax	0	18	on
CT_EN_DEVICE_ID	trTeControl.SendDeviceId (WARL)	0 (DID_NONE)	19	on
CT_EN_WATCHPOINT_MSG	trWpMask.WEM plus the watchpoint command (requires the instrumentation blocks)	0 (mask empty)	20	off
CT_EN_IMPLICIT_RETURN	trTeInstFeatures.InstEnImplicitReturn	0	0	off
CT_EN_REPEATED_HISTORY	trTeInstFeatures.InstEnRepeatedHistory	0	1	off

Build switch	Runtime control	Reset	CAPS	Slim reference profiles
CT_EN_WIDE_ICNT	trTeInstFeatures.InstEnWideIcnt	0	2	off
CT_EN_REPEAT_BRANCH	trTeInstFeatures.InstEnRepeatBranch	0	3	off
CT_EN_JTC	trTeInstFeatures.InstEnJumpTargetCache	0	4	off
CT_EN_BP	trTeInstFeatures.InstEnBranchPrediction	0	5	off
CT_EN_IBHS	trTeInstFeatures.InstEnIbhs	0	8	on
CT_EN_REPEAT_INSTR	trTeInstFeatures.InstEnRepeatInstr	0	9	on
CT_EN_INST_SYNC_REQ	trTeControl.InstSyncReq (write-1, self-clearing, not Enable-locked)	0	— [^] (the vendor sync reason exists either way) [^]	on
CT_EN_SYNC_STATUS	trTeSyncStatus.SyncReqSource (read-only; the register is omitted from the map when off)	0	—	on
CT_EN_FIFO_HIST	trTeTipFifoHistCtrl, trTeTipFifoHistData (registers omitted when off)	0	—	on
CT_EN_ETIP_WATERMARK	trTeTipFifoStatus.trTeTipFifoMaxFill (reads 0 when off)	0	—	on / off
—	CT_SIJUMP (elaboration attribute, no register)	—	6	as built
CT_XLEN = 64	— (a synthesis parameter, not a runtime mode)	—	23	as built

Two entries in the **CAPS** column are not features and are listed for completeness: bit 6 reports the ingress convention declared by CT_SIJUMP, and bit 23 reports the address width. Bit 23 has to be on the wire because a decoder cannot derive it from the stream — the address fields are variable-length with leading zeros stripped, so a 32-bit address and a 64-bit address whose upper bits happen to be zero occupy the same bytes.

The consistency of this mapping with the compile switches, the SystemRDL and the capability map is machine-checked in the source repository (`scripts/check_feature_flags.py`); the elaboration dependencies between switches — data-address compression and the drop policy following data trace, the watchpoint message following the instrumentation blocks — are checked separately (`scripts/check_profile_deps.py`).

NOTE

The measured resource cost of each switch is deliberately not in this table. Those figures are per profile and per tool version, and mixing them into a contract table invites comparing numbers that were never measured against each other. They belong with the footprint profiles; the underlying per-switch measurements and

their artefact paths are in `doc/integration.adoc` (anchor `feature-flags`) of the source repository.

Chapter 18. N-Trace Self-Identification and Configuration Messages

A trace decoder has to know how the encoder was built and how it was configured. Every compression feature changes what the byte stream means, not just how much of it there is: a stream with the jump-target cache enabled contains cache indices where an otherwise identical stream contains addresses, and a decoder that does not know cannot tell the difference from the bytes.

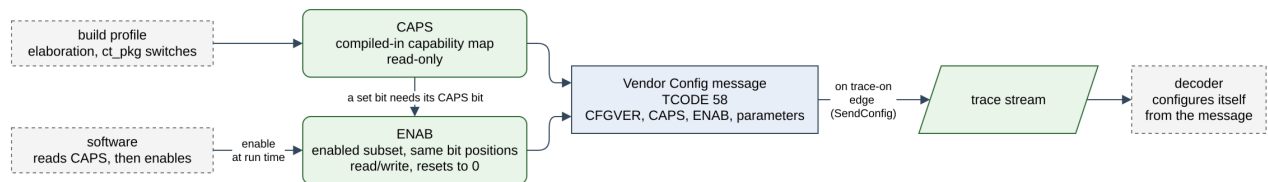


Figure 13. Capability and enable registers

There are two ways to give the decoder that knowledge. Either it is configured out of band – command-line flags, a configuration file, a convention – or the encoder puts it into the stream. CTTE's N-Trace back end does the second: the Vendor Config message (TCODE 58) carries the compiled-in capabilities and the currently enabled subset, and the Device ID message (TCODE 1) carries the instance identity. Both are N-Trace messages; the E-Trace back end has no in-stream counterpart, so an E-Trace decoder stays with the first way (Section 13.3).

This chapter defines both messages, the controls that emit them, and what a decoder can and cannot do without them. The last section documents a failure mode that stays invisible until a second, unrelated setting changes.

18.1. The Vendor Config message (TCODE 58)

TCODE 58 sits in the vendor-defined range that IEEE-ISTO 5001-2012 Table 4-5 reserves for exactly this purpose (56 – 62); N-Trace 1.0 Table 9 confirms the same range as "designated for use by Vendor Defined messages". A decoder that does not know the message still frames it correctly through the message framing and can skip it.

18.1.1. Field list

After the transfer code and the source field, the message carries one fixed field and six variable-length ones
(`ct_L2_nexus_formatter.sv:661`):

Field	Type	Max bits	Content
CFGVER	vendor-fixed	4	Payload format version.
CAPS	vendor-variable	24	Compiled-in capability map.

Field	Type	Max bits	Content
ENAB	vendor-variable	24	Currently enabled subset, same bit positions.
PARAM0	vendor-variable	16	Source identification.
PARAM1	vendor-variable	12	Source inhibition, synchronization cadence, instruction mode.
PARAM2	vendor-variable	12	Timestamp unit configuration.
PARAM3	vendor-variable	13	Structural sizes of the compression models.

The maximum widths are upper bounds, not on-wire lengths: the six variable fields are leading-zero-suppressed like every other variable-length field, so a lightly configured encoder emits a short message.

18.1.2. Version and forward compatibility

CFGVER is currently **1** (`ct_pkg.sv:377`). The forward-compatibility rule is **append-only**: a future version may add fields after `PARAM3`, never reinterpret an existing one. A decoder written against version 1 therefore reads the first seven fields of any future message and ignores what follows – which is what the reference decoder does, with an informational note.

That rule is the reason CFGVER did not move when the address-width bit was added: the information is boolean and fitted into the existing capability map, and adding a `PARAM4` would have been a structural change in the lean egress path, whose variable segments are all taken.

18.1.3. The capability map

CAPS reports what the netlist contains. One bit per feature, built directly from the compile switches (`ct_cfgmsg_caps`, `ct_pkg.sv:723`):

Bit	Name	Reports
0	IR	Implicit-return inference
1	RH	Repeated-history compression
2	WIDE_ICNT	Widened internal instruction-count cap
3	RB	Repeat-branch messages
4	JTC	Jump-target cache
5	BP	Branch prediction
6	SIJUMP	The attached core's ingress uses the sequentially-implicit-jump convention – an integration attribute of the encoder instance, not a build switch
7	OWNERSHIP	Ownership messages
8	IBHS	Indirect-branch-history synchronization messages
9	REPEAT_INSTR	Repeat-instruction messages
10	DEBUG_EVENTS	Debug-mode entry and exit events
11	POWER_EVENTS	Power-down entry and exit events
12	EVTI	External trace-trigger input

Bit	Name	Reports
13	TRIG_SYNC	Trigger-event synchronization marker
14	SEQ_SYNC	Sequential-instruction-counter synchronization
15	TIMESTAMP	Timestamp unit
16	DATA_TRACE	Data trace
17	DAQ	Data acquisition and the instrumentation blocks – one shared bit, because the two are not separable
18	QUOTA_SYNC	Trace-output quota synchronization cadence
19	DEVICE_ID	Device ID message
20	WATCHPOINT_MSG	Watchpoint message
21	DF_ADDR_COMPRESS	Data-address compression
22	DF_DROP	Data-trace drop policy
23	ADDR64	Not a feature: the architectural address width of this netlist

Bit 23 is the one entry that does not report a capability, and it has to be on the wire because a decoder cannot derive it from the stream. Address fields are variable-length with leading zeros stripped, so a 40-bit address and a 64-bit address whose upper bits happen to be zero are byte-for-byte identical. A decoder reads the bit and sizes its own program-counter state – and, critically, its jump-target-cache index fold – accordingly. A one-bit disagreement in that fold desynchronizes the two cache models **silently**.

Two features have no capability bit by design: the configuration message itself, because it is the feature that transmits the map, and the trigger registers, whose on-wire effect is already reported as bit 13.

18.1.4. The enabled map

ENAB uses identical bit positions and reports what is enabled **right now**. It is computed as the runtime term **ANDed with CAPS** (`ct_L2_nexus_formatter.sv:151`), so a bit set in a register of a netlist that does not contain the feature can never appear as enabled.

Most bits mirror a single register field:

Bit	Runtime source
0–5, 8, 9	The corresponding bit of <code>te.trTeInstFeatures</code>
7	<code>trTeControl.Context</code>
13	<code>trTeControl.InstTrigEnable</code>
14	<code>trTeControl.InstSeqSyncEnable</code>
15	<code>trTsControl.Enable</code>
16	<code>trTeDataControl.DataTracing</code>
22	<code>trTeDataControl.DataDropEna</code>

Four bits use **mode selection as the enable**, because the feature has no separate enable

bit – selecting a non-default mode is switching it on:

Bit	Condition
18	<code>trTeControl.InstSyncMode</code> selects one of the two trace-output quota cadence modes
19	<code>trTeControl.SendDeviceId</code> is not the "none" value
20	<code>trWpMask.WEM</code> is non-zero
21	<code>trTeDataControl.DataAddrCompress</code> is not the full-address mode

Six bits have no runtime enable at all and mirror CAPS: 6 (the ingress convention), 10, 11, 12 (the three event inputs), 17 (the instrumentation group) and 23 (the address width).

18.1.5. The parameter words

Word	Bits	Field	Source
PARAM0	[15:4]	source identifier	<code>trTeInstFeatures.SrcID</code>
	[3:0]	source-field width	<code>trTeInstFeatures.SrcBits</code>
PARAM1	[11]	source field inhibited	<code>trTeControl.InhibitSrc</code>
	[10:7]	synchronization interval exponent	<code>trTeControl.InstSyncMax</code>
	[6:3]	synchronization cadence mode	<code>trTeControl.InstSyncMode</code>
	[2:0]	instruction-trace mode	<code>trTeControl.InstMode</code>
PARAM2	[11:6]	timestamp width	<code>trTsControl.Width</code>
	[5:4]	timestamp prescale	<code>trTsControl.Prescale</code>
	[3:1]	timestamp type	<code>trTsControl.Type</code>
	[0]	timestamp enable	<code>trTsControl.Enable</code>
PARAM3	[12:8]	return-stack depth	<code>ct_pkg::CT_RET_STACK_DEPTH</code>
	[7:4]	branch-predictor size, base-two logarithm	<code>ct_pkg::CT_BP_ENTRIES</code>
	[3:0]	jump-target-cache index width	<code>ct_pkg::CT_JTC_ENTRIES</code>

Source: `ct_L2_nexus_formatter.sv:125` (the third word, a build-time constant) and `:177` (the three runtime words).

PARAM3 is worth a sentence of its own. It is built from the same package constants the implementing modules consume, so the advertised model sizes cannot drift from the built hardware – a decoder that sizes its return stack, its predictor table and its cache from these three fields is by construction in step with the encoder.

18.1.6. When it is emitted

Emission is controlled by `trTeControl.SendConfig`:

Value	Name	Behaviour
0	CFG_NONE	Never emitted.
1	CFG_ONCE	Once per trace-on edge – the rising edge of effective instruction tracing.
2	CFG_ON_SYNC	As above, plus one message after every periodic synchronization.

The reset value is **CFG_ONCE**. This is the one feature in the full build profile whose reset is not "off", and it is a deliberate trade: an unconfigured encoder produces a self-describing stream. Set **CFG_NONE** for a stream that contains only transfer codes the N-Trace profile adopts.

"Once per trace-on edge" is not "once per session". After an instruction-tracing pause the message is sent again – which is exactly what a decoder attaching during the pause needs (`ct_L23_preproc_composer_etip.sv:610`).

The payload is sampled at the emission site, not latched when `SendConfig` was written. That is safe because the fields it reports are quasi-static: they are Enable-locked and can only change while the encoder is disarmed, and every such change implies a new trace-on edge and therefore a new message. `SendConfig` itself is **not** Enable-locked and may be changed while the trace runs.

18.2. The Device ID message (TCODE 1)

TCODE 1 carries the static identity of the encoder instance: the value of the `CT_DEVICE_ID` elaboration parameter, laid out per IEEE-ISTO 5001-2012 Table B-5 (revision number, part number, manufacturer identifier). The default 0 means the integrator assigned no identity.

Emission is controlled by `trTeControl.SendDeviceId`, whose reset value is "none" – deliberately unlike `SendConfig`. The reason is conformance arithmetic rather than preference: N-Trace 1.0 Table 9 does not adopt TCODE 1, so leaving it off keeps the default stream inside the adopted message set. Setting it to the "once" value emits one message per trace-on edge, with the same trigger as **CFG_ONCE**.

The on-wire order is **Device ID, then Config, then the first synchronizing message**. This is a contract, not an ordering that happens to fall out: the Device ID slot is filled before the Config slot in the same beat, and an assertion checks the relation (`ct_L23_preproc_composer_etip.sv:576` and the ordering assertion at `:2172`). A decoder therefore learns **which device** produced the stream before it learns **how that device is configured**, and both before it has to decode anything.

One documented deviation applies: IEEE-ISTO 5001-2012 Table 4-7 specifies the identifier field as "Fixed 32", and CTTE emits it as a variable-length field with leading zeros stripped. The reason is that a fixed field would need a special case in both egress paths and in the decoder's table mechanism, for a value a decoder reads as an integer anyway. A decoder that expects 32 fixed bits must be told; the reference decoder reads the variable form.

18.3. What a decoder does with it

The reference decoder uses the configuration message to configure itself, and its rules are worth stating because they define what the message is **for**:

- **Enable-only.** An enabled bit switches a decoder feature on; a cleared bit never switches one off that a command-line flag or an earlier message enabled. A configuration message therefore cannot break a correctly configured decoder.
- **The command line wins.** Where an explicit flag and the message disagree, the flag is used and a warning is printed.
- **Some values are checked, not applied.** The return-stack depth is compared against the decoder's configured mirror and a mismatch is warned about rather than silently resized — re-initializing the mirror mid-stream would discard live entries.
- **The address width is applied on first sight and never narrowed.** A configuration message that arrives after decoding has already started produces a warning, because the lines already printed are narrower than those that follow.

Without the message, a decoder falls back to its command-line configuration. That works when the operator knows the encoder configuration and passes it correctly. The failure mode is that nothing checks the assumption — which is the subject of the next section.

18.4. Failure mode: a stream that never says what it is

This is the most expensive configuration mistake the encoder allows, and it is worth a section of its own because every property that makes it expensive is a property of **not** noticing.

The account below is taken from commit `7d91c2f8` of the CEDARtools.TraceEncoder repository, which fixed it; the two control words and the symptom are quoted from that commit's message and its diff of `examples/kv260/cva6_linux/board/cva6_linux_boot_trace.sh`.

What happened. A board bring-up script programmed the control register with a constant that had `SendConfig = CFG_NONE`. No configuration message was emitted, so the decoder ran entirely on its command-line defaults.

Why nothing was visible. The same constant had the synchronization interval at its minimum, so periodic synchronization re-anchored the decoder every 16 instruction units. The branch-history buffer never filled, so the repeated-history compression never emitted a Resource Full message with the repeated-history reason — and that message class was precisely where the missing configuration would have mattered. The setup worked, repeatedly, for as long as that unrelated setting stayed at its non-default value.

When it broke. Raising the synchronization interval to the register's reset value — 1024 units instead of 16 — let the history buffer overflow for the first time. At the first repeated-history message the decoder's assumptions parted company with the encoder's behaviour and it walked 20 million program counters into a startup hang symbol — the figure the fixing commit records. Not a decode

error, not a warning: a plausible-looking, entirely wrong reconstruction.

The two control words, decoded field by field against
Chapter 16:

Field	Broken 0x01060067	Corrected 0x016600E7	Effect
Active, Enable, InstTracing	1, 1, 1	1, 1, 1	unchanged
InstMode	6 (history)	6 (history)	unchanged
SendConfig	0 – CFG_NONE	1 – CFG_ONCE	no configuration message at all
InhibitSrc	0	0	unchanged
InstSyncMode	6 (instructions)	6 (instructions)	unchanged
InstSyncMax	0 – period 16	6 – period 1024	the setting that made the first defect visible
Format	1	1	unchanged
SendDeviceId	0 (none)	0 (none)	unchanged

The two words differ in exactly three bits: bit 7, the low bit of `SendConfig`, and bits 21 and 22 of `InstSyncMax`. Everything else is identical.

What to take from it. Three things, in order of usefulness:

1. **A control-register constant is a configuration, and it deserves the same review as one.** A hexadecimal literal in a script hides which fields it sets and, more importantly, which it leaves at zero. Prefer composing the word from named fields.
2. **`SendConfig = CFG_NONE` should be a deliberate choice, never a leftover.** There is one good reason to select it – a stream that must contain only transfer codes the N-Trace profile adopts – and it is worth writing down when it applies.
3. **A defect that only manifests under a second, unrelated setting is not a rare defect; it is a latent one.** The absence of a configuration message is invisible for as long as no message class needs it. That is an argument for emitting it by default, which is why the reset value is `CFG_ONCE`.

The configuration recipes that follow from this – how to compose a control word, and the other pitfalls of the same family – are in Chapter 26.

NOTE

One case cannot be fixed by `SendConfig` alone. A stream with instruction tracing disabled carries **no** configuration message, because the emission trigger is the instruction-trace-on edge. A decoder joining a data-only stream mid-stream therefore cannot auto-detect the data-address compression mode. Either keep a program-trace stream alive alongside it, or provide the mode to the decoder out of band.

Chapter 19. Hardware-Supported Instrumentation (ACT-CAP / ACT-ST / ACT-Proc)

CEDARtools.TraceEncoder (CTTE) carries a hardware-supported instrumentation (HSI) path that lets a traced hart – or the encoder itself – inject instrumentation events into the trace stream without a bus transaction and without a change to the CPU.

IMPORTANT

What of this path is reachable depends on the core, and the answer is not the same for the three blocks. **ACT-ST works everywhere:** it triggers on a retired instruction address and needs nothing but `iaddr`, which all four cores documented in [Chapter 6](#) deliver. **ACT-CAP is reachable on no shipped core as delivered**, because none of the four reports CSR writes on the trace ingress port – the per-core table in [Section 6.1.3](#) states this together with the reason and with the adapter-level way out. That way out has since carried ACT-CAP onto silicon: the `tgc5b2_rvcfi` testbed drives doorbell-converted ACT-CAP commands on the KV260 with board verdicts of 2026-08-26 ([Chapter 28](#)). **Everything that needs the data channel** – data trace, the data-flow range filter and the `DAQ_DATA / DAQ_DADDR / DAQ_DATA_DADDR` commands – **works on the MINRES TGC5B only**; on CVA6, Rocket and MicroBlaze™ V those commands carry no payload at all, because the encoder never receives a data beat. This chapter describes the mechanism; the table says where it can be exercised.

Three preprocessor blocks implement the path:

Block	Long name	RTL module
ACT-CAP	Accemic CEDARtools CSR Access Protocol – software-issued commands, observed as CSR writes on the Trace Ingress Port (TIP)	<code>rtl/preproc/ct_L23_preproc_act_cap.sv</code>
ACT-ST	Accemic CEDARtools Smart Trigger – hardware-synthesized commands, fired by a retired-PC match against the watchpoint table	<code>rtl/preproc/ct_L23_preproc_act_st.sv</code>
ACT-Proc	Accemic CEDARtools Instrumentation Processor – merge, arbitration and side-effect dispatch for both sources	<code>rtl/preproc/ct_L23_preproc_act_proc.sv</code>

NOTE

The shared `ACT-` prefix reads `*A*ccemic *C*EDAR*T*ools`; it names the house mark rather than any single product.

All three blocks are instantiated behind the `CT_EN_ACT` build switch (`rtl/ct_L23_preproc.sv`, generate block `genAct`). With `CT_EN_ACT = 0` the stub branch `genActStub` ties the downstream command interface to "no command" and drives the four tracing-override signals to 0, so a control-

flow-only profile contains no ACT logic at all and the composer's DAQ, watchpoint and sync-request arms are provably unreachable.

19.1. What the instrumentation path is for

The motivating cases, in the order they drove the design:

- **Design-time data trace** — emit a chosen value, address or program counter at a chosen source line, without a data-trace configuration that would export everything around it.
- **Run-time reconfiguration of the trace subsystem from the traced hart** — turn instruction or data tracing on and off, or request a synchronization message, without any access to the encoder's control bus.
- **Branch coverage and MC/DC evidence** — a marker per decision outcome, carried in the same stream as the control-flow trace and therefore ordered against it.
- **Timestamp control at meaningful program points** — a timestamp taken where the analysis needs one, instead of a fixed cadence that is either too coarse to be useful or dense enough to push the stream into overflow.

The last point is the reason the mechanism is a command rather than a counter setting: a synchronization or timestamp emitted every n branches ignores the structure of the control flow, and raising the rate until it does not costs bandwidth in every part of the program that did not need it.

19.2. ACT-CAP — commands observed on the TIP

ACT-CAP rests on one property of the Trace Ingress Port: every retired data access of the traced hart, including CSR accesses, is visible to the encoder. When the hart writes the command CSR, the encoder sees the write together with its target CSR number and decodes it — no CPU modification, no dedicated port, and no system-bus transaction. Functionally the write behaves like a NOP; the only requirement on the integration is that the write must be permitted at the executing privilege level, so that it does not raise an exception.

The decode is a single qualified comparison in

`ct_L23_preproc_act_cap.sv`:

```

1 if (tip.dretire && (tip.dtype == CSR_READ_WRITE)) begin
2     if (tip.daddr == ACT_CAP_CMD) begin
3         ValidPipe[0] <= 1;
4         CmdPipe [0] <= tip_data_to_cmd(tip.data);
5     end
6 end

```

ACT_CAP_CMD is CSR_ACT_CAP_BASE + 0 = 0x0B10 (rtl/pkg/nexus_vendor_riscv_pkg.sv:144); the surrounding 0x0B10--0x0B9F core-specific CSR window is reserved for the protocol but no second address is decoded in the current

implementation. The write data is unpacked by `tip_pkg::tip_data_to_cmd` (`rtl/pkg/tip_pkg.sv:457`) into the `{Cmd, Sink, DirectData}` command word described in the chapter *The trActCapStCmd Command CSR*; the inverse function `cmd_to_tip_data` is what the verification environment uses to drive a command, so both directions come from one definition.

`0x0B10` is a **CPU-side** CSR. The encoder does not decode it on its control bus; the register appears in the SystemRDL source only as a type anchor and is marked `ispresent = false` there, so no register exporter offers it as a memory-mapped register (`rdl/ct_cs_cpuif.rdl:112-120`).

The captured command advances through a shift pipeline whose tap is selected by `extra_delay`, so ACT-CAP events stay aligned with the control-flow and data-flow preprocessing paths. The block's own latency is one cycle (`internal_delay = 1`); reset clears the whole valid pipeline, so no command can survive a reset.

19.3. ACT-ST – commands synthesized by the encoder

ACT-ST removes the software step. The block watches the retired instruction stream and compares the retired instruction address against the watchpoint table; on a match it fetches the 32-bit command word stored with that entry and injects it into the same pipeline an ACT-CAP write would have entered. The software under test needs no instrumentation at all, and the reaction latency is fixed and known.

19.3.1. Trigger source and matcher

The comparison is a RAM-backed perfect binary search tree (`vector_binary_search_2clk`, one on-chip RAM per level, one lookup per cycle) instantiated with the key type `tip_iaddr_t` and qualified by `TipBeatRetires` (`tip.iretire`) – that is, by a retiring beat, not by a raw strobe, so the block behaves identically for serialized and block TIP ingress.

Property	Value	Source
Tree dimension <code>M0_DIM</code>	10	<code>rtl/pkg/ct_pkg.sv:988</code>
Table capacity <code>M0_N = 2^{M0_DIM} - 1</code>	1023 slots	<code>rtl/pkg/ct_pkg.sv:989</code> ; software discovery via <code>trWpCap.Entries</code> (0x4020)
Matcher chain latency	$4 \times M0_DIM = 40$ cycles	elaboration guard in <code>ct_L23_preproc_act_st.sv:92</code> ; runtime guard <code>idelay_act_st == 4*M0_DIM</code> in <code>ct_L23_preproc.sv:160</code>
Preprocessor alignment budget with ACT	<code>PREPROC_DELAY_MAX = 44</code>	<code>rtl/pkg/ct_pkg.sv:925</code> (ACT branch: <code>act_st 41 + act_proc 1</code>)

The alignment budget is charged to `CT_EN_ACT` alone: a profile with data trace but without ACT keeps the historical budget of 20, and a control-flow-only profile 4 or 2. That asymmetry is deliberate – every `EXTRA_DELAY_MAX`-deep alignment pipe in the preprocessor is sized by this constant, so an ACT profile pays for the ten-level tree in every one of them.

19.3.2. Programming the table

The table is loaded through an indirect register pair; the direct memory window at 0x4100 that earlier revisions used no longer exists, because 1023 entries of 8 bytes would overlap the DF range-filter component at 0x5000. The registers are defined in `rdl/ct_cs_cpuif.rdl:1420–1541`:

Register	Offset	Function
<code>trWpIndex.Idx</code>	0x400C	Slot index the next commit or readback targets. Wraps at <code>NUM_ACT_ST - 1</code> . Reads return the current pointer, so after <i>n</i> commits from slot <i>i</i> it reads $(i + n) \bmod \text{NUM_ACT_ST}$.
<code>trWpDataLow.Value</code>	0x4010	Staged <code>Addr</code> (the PC that arms the entry). A <code>Low</code> write alone has no effect.
<code>trWpDataHigh.{Cmd, Sink, DirectData}</code>	0x4014	Staged command word. Writing this register commits the staged pair to slot <code>Idx</code> and increments <code>Idx</code> . Same bit layout as CSR 0x0B10.
<code>trWpReadLow / trWpReadHigh</code>	0x4018 / 0x401C	Readback of the shadow copy at <code>Idx</code> . The <code>High</code> read advances <code>Idx</code> (serial readback); the <code>Low</code> read does not.
<code>trWpCap.Entries</code>	0x4020	Implemented slot count. Reads 0 in profiles without the ACT blocks.

Four programming rules follow from the tree layout and are stated normatively in the register description:

1. **Fill all `trWpCap.Entries` slots.** The flat slot index is the position in the key-sorted array, which the hardware maps in order onto the tree (even indices are the leaves, an odd index takes the inner level given by its count of trailing one bits, the median index is the root). Leaving slots at their reset value can mask real watchpoints, because a lookup that reaches an empty inner node stops there.
2. **Keys must be strictly ascending and unique** across slot 0 to slot `Entries - 1`.
3. **Each slot is a pair:** write `trWpDataLow`, then `trWpDataHigh`.
4. **Load the table only while `trTeControl.Enable = 0`.** This is enforced in hardware: the three load registers are write-locked while enabled, and the commit pulse is suppressed as well. Readback stays live, by design, so a debugger can walk the table of a running session.

For slots that are not real watchpoints, choose an address the hart can never execute – an odd instruction address is the natural choice, since the core forces `PC[0] = 0` – and set `Cmd = ACT_CAP_ST_NONE`. Padding keys still have to satisfy rule 2.

The shadow copy behind `trWpReadLow/trWpReadHigh` is zero-initialized at configuration load and is **not** cleared by a soft reset; it reflects everything committed since configuration, the same persistence the search tree itself has.

19.3.3. Address-width limit at `XLEN = 64`

The matcher key follows the architectural address width (`M0_K = logic [CT_XLEN-1:0]`), but the software-visible table entry keeps its 32-bit `Addr` word. In a 64-bit configuration the upper key half is therefore written as zero: a watchpoint can be placed anywhere in the low 4 GiB and never

matches a higher address. This is a configuration limit of the load path, not a truncation in the comparison – the full-width address enters the comparison unchanged
(rtl/pkg/ct_pkg.sv:992–1000).

19.4. ACT-Proc – merge, priority and side effects

ACT-Proc takes the two command interfaces and produces one ordered command stream for the composers, plus the direct control side effects.

19.4.1. Merge and priority

The arbitration is fixed and combinational: if both sources are valid in the same cycle, **ACT-ST wins** and the software-issued ACT-CAP command of that instruction is dropped. The intent is that a smart trigger placed on an instrumented instruction neutralizes the instrumentation it replaces, rather than producing two events for one program point.

19.4.2. Sink decode

The `Sink` field of the command word selects the dispatch path. ACT-Proc implements exactly two branches:

Sink	Effect in ACT-Proc
ACT_CAP_ST_SINK_NEXUS, ACT_CAP_ST_SINK_AXIS, ACT_CAP_ST_SINK_AXIS_NEXUS	The command is admitted into the output pipeline if its <code>Cmd</code> is one of the ten <code>DAQ_*</code> commands, <code>ACT_CAP_ST_CF_SYNC</code> or <code>ACT_CAP_ST_WATCHPOINT</code> . Any other <code>Cmd</code> value is silently discarded – an unsupported command produces no message, which is what the register description promises.
ACT_CAP_ST_SINK_TE	No message of any kind. <code>DirectData</code> is reinterpreted as a TE control request and applied directly; the command is not forwarded to the composers.

IMPORTANT

For a TE control request the `Cmd` field is not examined at all – `Sink = ACT_CAP_ST_SINK_TE` is what selects the behaviour (ct_L23_preproc_act_proc.sv:138–150). Conversely, `Cmd = ACT_CAP_ST_TE (13)` sent to a message sink matches none of the admitted command values and therefore does nothing.

19.4.3. TE control side effects

With `Sink = ACT_CAP_ST_SINK_TE` the 24-bit `DirectData` field is cast to the packed structure `ct_act_cap_te_t` (rtl/pkg/ct_pkg.sv:971–979):

Sub-field	Bits of <code>DirectData</code>	Meaning
ctrl	[23:16]	Selects the TE control target: 0 = <code>ACT_CAP_TE_INSTR_TRACING</code> (<code>trTeControl.InstTracing</code>), 1 = <code>ACT_CAP_TE_DATA_TRACING</code> (<code>trTeDataControl.DataTracing</code>). Any other value is ignored.

Sub-field	Bits of DirectData	Meaning
data	[15:0]	Carries the new state. For <code>ctrl = 0</code> the encoder evaluates <code>data[0]</code> ; for <code>ctrl = 1</code> it evaluates <code>data[1]</code> . A 1 asserts the corresponding ... <code>TracingSet</code> strobe, a 0 the ... <code>TracingClr</code> strobe.

Both strobes are single-cycle pulses into the CSR shim, which is how a traced hart switches instruction or data tracing on and off without any control-bus access.

19.4.4. Ordering, latency and backpressure

ACT-Proc preserves the temporal ordering of events as observed at the TIP; no reordering of trace-relevant events takes place. Its own latency is one cycle, on top of which the selectable `extra_delay` tap aligns the ACT branch with the peer preprocessing paths. The latency is a function of the build configuration only – it does not depend on runtime data values or on the message type. Backpressure from downstream logic can delay emission but cannot change the order in which events were observed.

19.5. How an ACT-ST hit becomes visible in the trace stream

Two mechanisms make a trigger visible; they are separate and can be used independently.

The message the command asks for. The `Cmd` field decides what is emitted: a Data Acquisition message (TCODE 7) for the ten `DAQ_*` commands, an instruction synchronization message for `ACT_CAP_ST_CF_SYNC`, or a Watchpoint message for `ACT_CAP_ST_WATCHPOINT`.

The Watchpoint message, TCODE 15. This is the direct answer to the question "which watchpoint fired". The command `ACT_CAP_ST_WATCHPOINT` (value 14) makes the encoder raise a Watchpoint message whose `WPHIT` field is a bitmap of the watchpoints reported:

```
WPHIT = Cmd.DirectData[15:0] AND trWpMask.WEM
```

The slot-to-bit assignment is a **software** convention, not hardware: give each `watchpoints[]` entry the `DirectData` bit that should identify it – one-hot is the obvious choice – and enable the same bit in `trWpMask.WEM` (0x4008). A command whose masked result is zero emits nothing. The mask resets to 0, so adding the feature to an existing build changes no wire byte until software opts in.

The message is emitted with the field order of the Nexus standard's Watchpoint message (TCODE, SRC, WPHIT, TSTAMP; `rtl/pkg/nexus_vendor_riscv_pkg.sv:660–670`, referencing IEEE-ISTO 5001-2012 §4.3.23 and Table 4-37). The internal payload width is 16 bits, one per reportable slot; on the wire `WPHIT` remains a variable-length MDO field with leading zeros stripped, so the internal width bounds the eTIP payload, not the wire.

Three properties are worth stating explicitly because they are easy to get wrong:

- A watchpoint command produces **no** Data Acquisition message. The DAQ arm of the eTIP composer excludes both `ACT_CAP_ST_WATCHPOINT` and `ACT_CAP_ST_CF_SYNC` (`ct_L23_preproc_composer_etip.sv:1049-1060`), and an assertion checks that the two never share one beat (`a_p4_wp_daq_exclusive`, same file, line 2161).
- The feature is compile-gated by `CT_EN_WATCHPOINT_MSG`, which itself requires `CT_EN_ACT`. In a build without it, `trWpMask.WEM` reads as a constant 0 and the composer raises no slot, so a forwarded watchpoint command is inert rather than an error.
- On-wire capability is advertised in the vendor Config message (TCODE 58) as `CAPS/ENAB` bit 20 (`rtl/pkg/ct_pkg.sv:728`), so a decoder can discover whether the stream can contain TCODE 15 at all.

Where the watchpoint mechanism is **not** used, a trigger still reaches the stream as the anonymous synchronization reason `NEXUS_SYNC_WATCHPOINT` (sync reason 6, "trace event"). The difference is exactly the identity of the trigger: reason 6 says that something fired, TCODE 15 says which.

For the full field layout of both messages see the message catalog chapter; for the DQDATA payload of the DAQ commands see the Data Acquisition payload chapter.

19.6. Synchronization requests from the traced hart

`ACT_CAP_ST_CF_SYNC` (12) requests an instruction synchronization message. The command rides on the retiring `csrw 0x0B10` — an instruction of type `OTHER` — and is turned into a synchronization exactly as a periodic sync landing on a non-control-flow instruction would be. If a real synchronization reason is already present on that beat, the real reason takes precedence; a synchronization is emitted either way.

On the wire the reason is the single vendor code `NEXUS_SYNC_REQ = 14` ("explicit sync request"). N-Trace 1.0 Table 25 reserves codes 10 to 13 for future standard use, which is why the earlier per-source codes 12 (CSR) and 13 (ATB) are no longer emitted. The source of the request stays readable for diagnosis in the read-only field `trTeSyncStatus.SyncReqSource` (0xE08), which reports `SYNC_REQ_CSR = 1` for an `ACT-CAP CF_SYNC` and distinguishes it from `SYNC_REQ_ATB`, `SYNC_REQ_QUOTA` and `SYNC_REQ_TE`.

A decoder does not need the distinction: every synchronization message is a full re-anchor.

19.7. Determinism and reset behaviour

The instrumentation path is deterministic in the sense a trace-based verification argument needs:

- **Fixed, configuration-dependent latency.** Each block reports its own latency to the preprocessor (`internal_delay`): 1 cycle for ACT-CAP, 1 + matcher for ACT-ST (40 cycles at `M0_DIM = 10`), 1 cycle for ACT-Proc. None of these depends on the command, the payload or the message type.
- **Elaboration-time budget checks.** ACT-ST checks its own chain length against `EXTRA_DELAY_MAX` at elaboration and fails the build if the budget is too small;

`ct_L23_preproc` repeats the check at simulation time against the reported delay. A silent misalignment between the ACT branch and the control-flow branch is therefore not a possible failure mode.

- **Order preservation.** Multiple matches are serialized by the pipeline with per-event order preserved; ACT-Proc adds no reordering.
- **Clean reset.** Every valid pipeline is cleared on reset, in all three blocks, so no command emitted before the reset can appear after it.

The one state that deliberately survives is the watchpoint table and its readback shadow: both are configuration, not trace state, and are not cleared by a soft reset.

19.8. Licensing of ACT-CAP and ACT-ST

ACT-CAP and ACT-ST are implemented in the hardware IP under `rtl/` and are covered by the same licence as the rest of it. The terms, and the question of whether a third-party implementation of these mechanisms carries any obligation beyond that licence, are stated once in [Section 37.5](#) – not here.

19.9. Related mechanisms in other architectures

The idea of a software-issued marker that the trace hardware picks up without a bus transaction exists elsewhere; the table is given so an integrator can place CTTE's mechanism against what they already know. It is not a feature comparison.

Mechanism	Character
Nexus Data Acquisition message, TCODE 7 (IEEE-ISTO 5001-2012 §4.3.21)	The message CTTE uses to carry the DAQ payload. It is a public message of the standard, not an Accemic extension; what is CTTE-specific is the command set that produces it and the payload layout inside <code>DQDATA</code> .
Arm A64 <code>TRCIT</code> (an alias of <code>SYS</code>)	A dedicated instruction that writes a value into the instruction-trace stream.
Intel <code>PTWRITE</code>	A dedicated instruction that writes a value into a Processor Trace packet.
Arm System Trace Macrocell (STM)	A separate trace source with memory-mapped stimulus ports and its own global timestamp, used for system-activity and trigger indication.

CTTE's variant differs in what it costs the integration rather than in what it achieves: the command is an ordinary CSR write on an existing port, so it needs no new instruction, no new bus master and no change to the core.

19.10. Verification status

Aspect	Evidence
Full ACT-CAP command set, both sinks	tests/hsi/01_csr_cap/csr_cap_tb.sv – fires every command that produces output to ACT_CAP_ST_SINK_AXIS_NEXUS, self-checks the AXIS beat via ct_axis_decoder, and checks the ATB capture for the synchronization messages (≥ 2 : startup plus the CF_SYNC).
ACT-ST table: capacity, boundary slots, index autoincrement, readback, soft-reset persistence	tests/instruction/19_feature_matrix/feature_matrix_tb.sv, phases Ph.10 / Ph.10b / Ph.10c / Ph.11 – full 1023-slot load in ascending key order; hits proven on slot 0, slot 511 (tree root) and slot 1022 (last leaf); an unarmed PC proven silent; trWpCap.Entries checked against M0_N.
Watchpoint message (TCODE 15) and the WEM mask	tests/instruction/31_status_msgs/status_msgs_tb.sv – scenarios wp (WEM = 0xFFFF, expected WPHIT 0x1 / 0x2 / 0x8001), wpmask (WEM = 0x0002, negative control: exactly one message survives), both (device ID, config and watchpoint messages in one stream) and wpaxis (the same command on the AXIS sink). The test also asserts that a watchpoint command produces no Data Acquisition message.
Block-level behaviour of the three ACT modules	rtl/preproc/test/ct_L23_preproc_act_cap_tb.sv, ct_L23_preproc_act_st_tb.sv, ct_L23_preproc_act_proc_tb.sv

Chapter 20. The trActCapStCmd Command CSR

CTTE carries every ACT-CAP and ACT-ST command in one 32-bit word. Software delivers it by writing the RISC-V CSR `trActCapStCmd` at CSR address `0x0B10`; the ACT-ST hardware delivers the identical word from a watchpoint table entry. Downstream of the instrumentation processor the two are indistinguishable – everything is driven by the command word.

This chapter is the normative description of that word. The authoritative source for the layout and the enumerations is `rdl/ct_cs_cpuif.rdl:56-120`; the SystemVerilog typedefs the RTL uses are regenerated from it into `rtl/pkg/ct_cs_cpuif_types_pkg.sv` by `scripts/generate_wb_pkg.py`.

20.1. Register definition

Field	Bits	Description
Cmd	[5:0]	Instrumentation command. Encoded as <code>trActCapStCmd_e</code> ; see Section 20.2 . Reset 0.
Sink	[7:6]	Destination of the command's effect. Encoded as <code>trActCapStSink_e</code> ; see Section 20.3 . Reset 0.
DirectData	[31:8]	24-bit command-specific payload. Reset 0.

IMPORTANT

`0x0B10` is a **CPU-side** CSR of the traced hart, not a memory-mapped register of the encoder. The encoder never decodes `0x0B10` on its control bus; it observes the write on the Trace Ingress Port. In the SystemRDL source the register carries `ispresent = false` and exists only as a type anchor, so no register exporter – HTML, C header or regblock – offers it as an addressable register. The same bit layout is reachable through the control bus as the watchpoint table's command word, `trWpDataHigh` at `0x4014`.

The word is placed on the TIP data channel by `tip_pkg::cmd_to_tip_data` and recovered by `tip_pkg::tip_data_to_cmd` (`rtl/pkg/tip_pkg.sv:446-467`), both of which use exactly the bit positions above.

20.2. Cmd field: trActCapStCmd_e

Enumerator	Value	Meaning
ACT_CAP_ST_NONE	0	Do nothing. The command word is ignored by the ACT processing pipeline. Use it as the "no operation" value for unused watchpoint slots.
ACT_CAP_ST_DAQ_PC_CURR	1	Transmit a DAQ message containing the current program counter plus <code>DirectData[23:0]</code> .
ACT_CAP_ST_DAQ_PC_CURR_IAS T	2	Transmit a DAQ message with the current PC, the previous PC before an exception or interrupt, plus <code>DirectData[23:0]</code> . Useful to correlate a trap with the instruction that caused it.

Enumerator	Value	Meaning
ACT_CAP_ST_DAQ_DIRECT_DATA	3	Transmit a DAQ message carrying only the 24-bit <code>DirectData</code> value. The cheapest marker: an application-defined event ID with no automatic address or data capture.
ACT_CAP_ST_DAQ_DATA	4	Transmit a DAQ message with the data value of the last retired data access and its context (data type and size), plus <code>DirectData[23:0]</code> .
ACT_CAP_ST_DAQ_DADDR	5	Transmit a DAQ message with the data address of the last retired data access and its context, plus <code>DirectData[23:0]</code> . Traces which memory location was touched without exporting the value.
ACT_CAP_ST_DAQ_DATA_DADDR	6	Both of the above — value and address of the last retired data access, with context, plus <code>DirectData[23:0]</code> . The most information-rich DAQ command and correspondingly the most expensive in bandwidth.
(reserved)	7	Reserved. The RDL source carries a commented-out <code>ACT_CAP_ST_DAQ_STALL_COUNTER</code> at this value; it is not implemented and the value must not be used.
ACT_CAP_ST_DAQ_IFETCH_TH	8	Transmit a DAQ message with the count of instruction fetches that exceeded the cycle threshold in region <code>DirectData[7:0]</code> , plus <code>DirectData[23:8]</code> , and clear that counter.
ACT_CAP_ST_DAQ_DATA_RD_TH	9	As above for read accesses that exceeded the cycle threshold in region <code>DirectData[7:0]</code> ; clears the counter.
ACT_CAP_ST_DAQ_DATA_WR	10	Transmit a DAQ message with the number of write accesses counted for region <code>DirectData[7:0]</code> , plus <code>DirectData[23:8]</code> , and clear that counter.
ACT_CAP_ST_DAQ_DATA_RD	11	Transmit a DAQ message with the number of read accesses counted for region <code>DirectData[7:0]</code> , plus <code>DirectData[23:8]</code> , and clear that counter.
ACT_CAP_ST_CF_SYNC	12	Request an instruction synchronization message. Produces a Nexus synchronization message with the vendor reason code 14 and no DAQ message.
ACT_CAP_ST_TE	13	Nominally "set a TE control field". See the note below: the effect is selected by <code>Sink</code> , not by this value.
ACT_CAP_ST_WATCHPOINT	14	Transmit a Watchpoint message (Nexus TCODE 15) reporting which watchpoint fired: <code>WPHIT = DirectData[15:0] AND trWpMask.WEM</code> . A command whose masked result is 0 emits nothing. Requires the <code>CT_EN_WATCHPOINT_MSG</code> build switch (which itself requires <code>CT_EN_ACT</code>). Unlike the DAQ commands this one produces no Data Acquisition message.

Any `Cmd` value not listed above is discarded by the instrumentation processor and produces no message.

20.2.1. Two behaviours that differ from the obvious reading

The data-context commands report the **LAST** retired data access, not the next one. Commands 4, 5 and 6 are served from the registers `PrevData`, `PrevDAddr` and `PrevDtypeDsize`, which the composers latch on every `tip.dretire` beat (`ct_L23_preproc_composer_etip.sv:1385–1389`, `ct_L23_preproc_composer_axis.sv:103–107`). The command therefore reports the access that has already been observed when the command retires — the usual software pattern is

load or store, then command, not the reverse. There is no pending-request state and no arming step.

ACT_CAP_ST_TE is not what selects a TE write. The instrumentation processor branches on `Sink` first; for `Sink = ACT_CAP_ST_SINK_TE` it applies the TE control request and never looks at `Cmd`, and for the message sinks the value 13 matches none of the admitted commands and is discarded (`ct_L23_preproc_act_proc.sv:99-155`). A TE control request is therefore a command with `Sink = 3`; the `Cmd` value is free. Setting `Cmd = 13` with a message sink does nothing.

20.3. Sink field: trActCapStSink_e

Enumerator	Value	Meaning
ACT_CAP_ST_SINK_NEXUS	0	Emit the result as a Nexus message on the encoder's trace output only. The classic trace-only path.
ACT_CAP_ST_SINK_AXIS	1	Emit the result as an AXI-Stream beat to an on-chip consumer only. No Nexus message is generated.
ACT_CAP_ST_SINK_AXIS_NEXUS	2	Emit both: local processing and off-chip trace visibility.
ACT_CAP_ST_SINK_TE	3	Emit nothing. <code>DirectData</code> is applied as a TE control request instead (instruction tracing on/off, data tracing on/off).

The two composers gate independently on the sink value: the eTIP composer accepts `NEXUS` and `AXIS_NEXUS`, the `AXIS` composer accepts `AXIS` and `AXIS_NEXUS`. `CF_SYNC` and `WATCHPOINT` are Nexus-side commands – the `AXIS` composer has no arm for them and its default branch drops the beat.

20.4. DirectData field

`DirectData` is a 24-bit payload with three roles, selected by the command:

- **User tag.** For the PC-based and data-based DAQ commands the value is carried into the DAQ payload as an application-defined identifier, so tools can correlate a trace packet with a logical event.
- **Region selector.** For the four counter commands (8, 9, 10, 11) `DirectData[7:0]` selects the region; `DirectData[23:8]` remains available as a tag.
- **Watchpoint bitmap.** For `ACT_CAP_ST_WATCHPOINT`, `DirectData[15:0]` is the `WPHIT` bitmap before masking.
- **TE control request.** With `Sink = ACT_CAP_ST_SINK_TE`, `DirectData[23:16]` selects the control target and `DirectData[15:0]` carries the new state.

NOTE

There is no "non-negative" guard on `DirectData` and no sentinel that suppresses it. `DirectData` is always part of the DAQ payload; a value of 0 is free, because the Nexus MDO/MSEO variable-length field encoding strips leading zeros and an all-zero field occupies no additional MDO cycles. Callers that need no auxiliary data simply leave `DirectData = 0` (`rdl/ct_cs_cpuif.rdl:50-55`).

20.5. Software use via ACT-CAP

20.5.1. C helper types

The command word has no generated C header: `trActCapStCmd` is hidden from every PeakRDL exporter by `ispresent = false`, so the C-header exporter does not see it either. The definitions below are the **recommended** software-side declarations; they mirror `rdl/ct_cs_cpuif.rdl` field for field, and that file remains the normative source.

```

1 typedef union {
2     struct {
3         uint32_t Cmd          : 6; /* [5:0] instrumentation command */
4         uint32_t Sink         : 2; /* [7:6] sink selection */
5         uint32_t DirectData :24; /* [31:8] command-specific payload */
6     } b;
7     uint32_t w;
8 } CT_CSR_trActCapStCmd_Type;
9
10 typedef enum {
11     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_NONE          = 0,
12     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_PC_CURR   = 1,
13     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_PC_CURR_LAST = 2,
14     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DIRECT_DATA = 3,
15     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DATA      = 4,
16     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DADDR     = 5,
17     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DATA_DADDR = 6,
18     /* 7 reserved -- do not use */
19     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_IFETCH_TH = 8,
20     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DATA_RD_TH = 9,
21     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DATA_WR   = 10,
22     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DATA_RD   = 11,
23     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_CF_SYNC      = 12,
24     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_TE           = 13,
25     CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_WATCHPOINT   = 14
26 } ct_csr_trActCapStCmd_Cmd_e;
27
28 typedef enum {
29     CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_NEXUS    = 0,
30     CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_AXIS    = 1,
31     CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_AXIS_NEXUS = 2,
32     CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_TE      = 3
33 } ct_csr_trActCapStCmd_Sink_e;
```

A bitfield union is convenient but not portable across compilers with different bitfield packing rules. Where portability matters, build the word with shifts:

```

1 #define CT_ACT_CMD(cmd, sink, direct) \
2     ( ((uint32_t)(cmd) & 0x3Fu) \
3       | (((uint32_t)(sink) & 0x03u) << 6) \
4       | (((uint32_t)(direct) & 0xFFFFFu) << 8) )
```

20.5.2. Constructing and issuing a command

```

1 /* ACT-CAP command CSR of the traced hart. */
2 static inline void ct_act_cap_write(uint32_t value)
3 {
```

```

4   __asm__ volatile ("csrw 0xB10, %0" :: "r"(value));
5 }
6
7 /* Emit a DAQ marker carrying the current PC and an 8-bit user tag,
8    visible in the off-chip trace stream only. */
9 void ct_mark_pc(uint8_t tag)
10 {
11     ct_act_cap_write(CT_ACT_CMD(
12         CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_PC_CURR,
13         CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_NEXUS,
14         tag));
15 }
16
17 /* Sample the value of the load or store immediately preceding this call,
18    to both sinks. */
19 void ct_sample_last_access(uint32_t *p, uint8_t tag)
20 {
21     volatile uint32_t v = *p;          /* the access that will be reported */
22     (void)v;
23     ct_act_cap_write(CT_ACT_CMD(
24         CT_CSR_TRACTCAPSTCMD_CMD_ACT_CAP_ST_DAQ_DATA,
25         CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_AXIS_NEXUS,
26         tag));
27 }
28
29 /* Turn instruction tracing off from the traced hart itself:
30    Sink = TE, ctrl = 0 (InstTracing) in DirectData[23:16], state in bit 0. */
31 void ct_inst_tracing(int on)
32 {
33     ct_act_cap_write(CT_ACT_CMD(
34         0, CT_CSR_TRACTCAPSTCMD_SINK_ACT_CAP_ST_SINK_TE,
35         (0u << 16) | (on ? 1u : 0u)));
36 }

```

CAUTION

The CSR number is architecturally a **core-specific** CSR. Whether a given hart implements it, and at which privilege level a write is permitted, is an integration property. On a core that does not implement it, `csrw 0xB10` raises an illegal-instruction exception – which is exactly how the MicroBlaze™ V demonstrator uses it as a deliberate trap stimulus (examples/kv260/mbv/sw/rob_stress.S:35–38, 125). Confirm with the SoC documentation before instrumenting production code.

CAUTION

ACT-CAP works only on cores that **report CSR write accesses on the TIP as dretire with dtype = CSR_READ_WRITE**. A core that retires the write without reporting it produces no command, silently. This is the first thing to check when instrumentation appears to do nothing – and on the four cores this manual documents, it is not a caveat but the present state: **none of them reports CSR writes**, for two different reasons (Section 6.1.3). The MINRES TGC5B decodes the vendor CSR window and raises no exception for it, but its H2E port derives `dtype` from the data bus alone, so the write never reaches the trace port; CVA6, Rocket and MicroBlaze™ V tie `dretire` to 0 and produce no data beat at all.

An integration can close this locally on a core whose adapter **does** see stores: a store to a chosen doorbell address is converted into the CSR beat the encoder expects. That is an adapter-level change and needs no core change. The runtime-verification testbed builds exactly this

pair—`examples/kv260/tgc5b2_rvcfi/rtl/ct_tip_adapter_actcap.sv` with the doorbell in `examples/kv260/tgc5b2_rvcfi/rtl/ct_soc_doorbell.sv`. On CVA6, Rocket and MicroBlaze™ V no such local fix exists, because there is no store beat to convert; for CVA6 in particular, RVFI is not an escape route, since the wrapper ties its RVFI outputs off and the CVA6 RVFI CSR structure is a fixed list of named architectural CSRs with no field for an arbitrary address/value pair.

IMPORTANT

On the four documented cores' **native** ingress, ACT-CAP therefore still has no silicon history; there its coverage comes from `tests/lib/cpu_model.sv`, which synthesizes the beat directly (scenario `sim-hsi-csr-cap`). The doorbell pair above is the first silicon exercise of the mechanism: on 2026-08-26 the runtime-verification testbed drove doorbell-converted ACT-CAP commands on the KV260 with board verdicts over both of its record transports (twin table `BOARD_VERDICTS_OK`, ring drops 0/0—the closing status chapter of `examples/kv260/tgc5b2_rvcfi/TUTORIAL_runtime_verification.md` carries the tables). Those records leave through the AXI-Stream record path; a Data Acquisition **message** emitted from silicon over the ATB trace port is still outstanding.

20.6. Use via the Smart Trigger (ACT-ST)

With ACT-ST the hart writes nothing. The command word is stored in the watchpoint table and the encoder issues it when a retired PC matches the entry's `Addr`:

1. Decide which `Cmd` / `Sink` / `DirectData` combination expresses the intended action—for example "on function entry, emit `DAQ_PC_CURR` with tag 42".
2. Build the 32-bit word with the same helper used for ACT-CAP.
3. Write the arming PC to `trWpDataLow` (0x4010), then the command word to `trWpDataHigh` (0x4014). The `High` write commits the pair to slot `trWpIndex.Idx` and advances the index.
4. Observe the four programming rules of the table (fill all slots, strictly ascending unique keys, pair-wise writes, load only while `trTeControl.Enable = 0`)—see the instrumentation chapter.

When the entry fires, the encoder emits exactly what a `csrw 0xB10, value` with the same word would have produced.

20.7. How a command reaches the wire

For a message sink the instrumentation processor forwards the command to the composers, and the eTIP composer decides the message class:

Command class	Message
the ten DAQ_* commands	Data Acquisition, TCODE 7. The IDTAG field carries the command value itself (ct_L23_preproc_composer_etip.sv:1062), so a decoder can tell from the tag how to read the payload; the payload rides in DQDATA as up to three elements (MAX_DQDATA_ELEMENTS = 3).
ACT_CAP_ST_CF_SYNC	An instruction synchronization message, on-wire reason 14. No DAQ message.
ACT_CAP_ST_WATCHPOINT	Watchpoint, TCODE 15, WPHIT = DirectData[15:0] AND trWpMask.WEM. No DAQ message; an assertion in the composer checks that a watchpoint and a DAQ message never share a beat.

The element-by-element payload layout of the DAQ message is in the Data Acquisition payload chapter; the AXIS beat layout is in the raw event path chapter.

20.8. DAQ bandwidth and backpressure

An instrumentation stream is trace bandwidth like any other. Too many DAQ messages saturate the trace interface, cause backpressure on the encoder and can lead to loss of trace elements – which the encoder reports rather than hides:

- On sustained backpressure the composer raises a queue-overflow condition and the encoder emits an Error message (TCODE 8, ETYPE = QUEUE_OVERRUN) whose ECODE bitfield records **which message classes** were lost. The DAQ bit is set only in builds that can produce a DAQ message at all – (CT_EN_DQ || CT_EN_ACT) in ct_L23_preproc_composer_etip.sv:1502 – so a profile without instrumentation never advertises a DAQ loss that could not have happened.
- After the overrun the encoder re-anchors with a FIFO_OVERRUN synchronization, so a decoder regains position.

Mitigations, in the order of effect:

1. **Reduce the number of arming points.** A static analysis of the program before the run identifies the hot spots; fewer watchpoints in a hot loop is the cheapest fix by a wide margin.
2. **Reduce the payload.** DAQ_DIRECT_DATA (one element) costs a fraction of DAQ_DATA_DADDR (three elements plus context). A DirectData of 0 costs nothing on the wire.
3. **Use the address/range and privilege filtering of the trace configuration** so that the regular data trace does not compete with the instrumentation for the same interface.
4. **Route very high-rate instrumentation to the AXIS sink instead of the Nexus stream** (Sink = ACT_CAP_ST_SINK_AXIS). The on-chip path does not consume trace-port bandwidth at all. Note its own loss behaviour, described in the raw event path chapter.
5. **Watch the FIFO fill-level histogram** (trTeTipFifoHistCtrl / trTeTipFifoHistData, CT_EN_FIFO_HIST) during a pilot run rather than guessing: it reports the distribution of the internal fill level, not just the peak watermark.

20.9. Summary

- `trActCapStCmd` is the single command word for both ACT-CAP and ACT-ST.
- `Cmd` selects what is emitted – PC, data context, counter value, an extra synchronization, or a watchpoint report.
- `Sink` selects where the effect goes: trace stream, on-chip AXI-Stream, both, or the TE control registers.
- `DirectData` carries 24 bits of tag, region index, watchpoint bitmap or control value, and costs nothing on the wire when it is 0.
- ACT-CAP issues the word with a CSR write; ACT-ST issues an identical word from the watchpoint table. Downstream they are the same event.

Chapter 21. HSI Transfer

"HSI transfer" is the path an instrumentation command takes from the moment it becomes visible to the encoder until the resulting message leaves the encoder. The command word itself is described in the command CSR chapter; this chapter describes the transfer – the control structure, the data sources a command can reach, the control-register access it can perform, how the resulting Nexus message is composed, and what the implementation does and does not guarantee about the ordering of a command relative to the data it reports.

21.1. Control structure

The control structure is the 32-bit command word `{Cmd, Sink, DirectData}`, delivered either by a single CSR write to `0x0B10` (ACT-CAP) or from a watchpoint table entry (ACT-ST).

That it is a **single** 32-bit write is a property worth stating explicitly, because it removes an entire class of failure. A protocol that first writes a data register and then a command register has an interruptible window between the two: an interrupt whose handler also uses those registers corrupts the transfer. CTTE has no such window for the command itself – there is one write, it is atomic by construction, and it is therefore safe to issue from any context, including an interrupt handler, without a critical section.

The window that does remain is a **semantic** one and concerns only the three data-context commands; it is described in [Section 21.8](#).

NOTE

A command's **route** into the encoder and its **payload** have different prerequisites, and both depend on the core. The CSR route (ACT-CAP) needs the core to report a CSR write on the ingress port, which no shipped core does as delivered; the watchpoint route (ACT-ST) needs only a retired instruction address and works on all four. Independently of the route, the three data-context commands need a data beat, which only the MINRES TGC5B supplies. The per-core table is [Section 6.1.3](#).

21.2. Data sources a command can reach

The `Cmd` field selects which of the encoder's observation points feeds the message. All of them are values the encoder already holds from the Trace Ingress Port; no additional bus access takes place.

Source	Content	Reached by
Current program counter	<code>tip.iaddr</code> of the beat carrying the command	<code>DAQ_PC_CURR</code> , <code>DAQ_PC_CURR_LAST</code>
Program counter before the last trap	<code>LastIAddrBeforeException</code> , updated on an <code>EXCEPTION_TRAP</code> or <code>INTERRUPT</code> itype	<code>DAQ_PC_CURR_LAST</code>
Value of the last retired data access	<code>PrevData</code> , latched on every <code>tip.dretire</code>	<code>DAQ_DATA</code> , <code>DAQ_DATA_DADDR</code>
Address of the last retired data access	<code>PrevDAddr</code> , latched on every <code>tip.dretire</code>	<code>DAQ_DADDR</code> , <code>DAQ_DATA_DADDR</code>

Source	Content	Reached by
Context of the last retired data access	<code>PrevDtypeDsize = {tip.dtype, tip.dsize}</code> — access type (load, store, CSR, atomic) and size	DAQ_DATA, DAQ_DADDR, DAQ_DATA_DADDR
Direct data	the 24-bit <code>DirectData</code> field of the command word itself	every DAQ command
Per-region access and threshold counters	the four counter banks of the performance-counter block, indexed by <code>DirectData[7:0]</code>	DAQ_DATA_RD, DAQ_DATA_WR, DAQ_IFETCH_TH, DAQ_DATA_RD_TH
Timestamp	the value selected by the timestamp unit (<code>trTsControl.Type</code> , <code>Active</code> , <code>Count</code> , <code>Prescale</code>)	the <code>TSTAMP</code> field of the message, and — with <code>CT_EN_AXIS_TS</code> — element 2 of a DAQ_PC_CURR_AXIS beat

The four counter commands **clear** the counter they report. On the Nexus path the clear pulse comes from the eTIP composer (`ct_L23_preproc_composer_etip.sv:1413–1433`), on the AXIS path from the AXIS composer; each drives its own clear input of the counter, so the two sinks cannot clear each other's reading.

21.3. Accessing TE control registers through HSI

A command with `Sink = ACT_CAP_ST_SINK_TE` performs no transfer to a sink at all; instead its `DirectData` field is applied to the encoder's control registers. This is the mechanism by which a traced hart reconfigures the trace subsystem without any access to the control bus.

Two control targets are implemented

(`ct_pkg.sv:971–979`, `ct_L23_preproc_act_proc.sv:138–150`):

<code>DirectData[23:16]</code>	Target	State taken from
0	<code>trTeControl.InstTracing</code> — instruction tracing on/off	<code>DirectData[0]</code>
1	<code>trTeDataControl.DataTracing</code> — data tracing on/off	<code>DirectData[1]</code>

The block asserts a single-cycle `...TracingSet` or `...TracingClr` strobe into the CSR shim, which is the same hardware path the external control bus uses. Any other value in `DirectData[23:16]` is ignored.

The same registers remain reachable over the control bus. The format of the control structure is identical either way, which is the point: a debugger and a traced hart can drive the same field, and the encoder cannot tell them apart.

21.4. Nexus message composition

21.4.1. Event triggers

Exactly two events start an HSI transfer:

- a CSR write to 0x0B10 observed as `dretire` with `dtype = CSR_READ_WRITE` (ACT-CAP), or
- a retired instruction address matching a watchpoint table entry (ACT-ST).

Both produce the same command word, and the instrumentation processor gives the ACT-ST command priority when both occur on the same instruction.

21.4.2. Data Acquisition message (TCODE 7)

The ten `DAQ_*` commands produce one Data Acquisition message. Its field order and types come from the message-format table in `rtl/pkg/nexus_vendor_riscv_pkg.sv:672-681`:

Field	Type / width	Content
TCODE	fixed, 6 bit	7
SRC	fixed, NEXUS_MSG_SOURCE_WIDTH	source identifier of the encoder instance; present only when the source field is configured
IDTAG	vendor fixed, 12 bit	the command value itself (<code>ct_L23_preproc_composer_etip.sv:1062</code>). A decoder reads IDTAG to know how to interpret the payload.
DQDATA	vendor variable, up to NEXUS_DQDATA_WIDTH = NEXUS_MSG_DATA_WIDTH = 192 bit in a build with DAQ messages (<code>rtl/pkg/nexus_vendor_riscv_pkg.sv:46,106</code>)	the payload elements, up to <code>MAX_DAQ_DATA_ELEMENTS = 3</code> per message (<code>rtl/pkg/ct_pkg.sv:906</code>); encoded with leading zeros stripped, so unused elements and zero values cost nothing on the wire
TSTAMP	vendor variable, up to NEXUS_MSG_TSTAMP_WIDTH	present when the timestamp unit is enabled for wire fields (<code>trTsControl.Enable</code>)

Two consequences follow from IDTAG carrying the command:

- One DAQ message carries **one** command's complete payload. There is no grouping of several messages into a logical unit and no sequence bit that would tie them together—the three elements of, say, `DAQ_DATA_DADDR` ride in one `DQDATA` field of one message.
- A decoder needs no side channel to parse `DQDATA`; the element layout is a function of IDTAG. That layout is tabulated in the Data Acquisition payload chapter.

The `DQDATA` width is the reason DAQ is used at all for this purpose: a standard data-trace message carries fewer bits than three address-or-data elements plus context require.

21.4.3. Watchpoint message (TCODE 15)

`ACT_CAP_ST_WATCHPOINT` produces a Watchpoint message instead of a DAQ message: TCODE, SRC, WPHIT, TSTAMP, with `WPHIT = DirectData[15:0] AND trWpMask.WEM`. The internal

payload is 16 bit wide; on the wire `WPHIT` is a variable-length field. See the instrumentation chapter for the slot-to-bit convention and the message catalog for the field table.

21.4.4. Synchronization message

`ACT_CAP_ST_CF_SYNC` produces an instruction synchronization message with the vendor reason code 14, and no DAQ message. The requesting actor is recorded in `trTeSyncStatus.SyncReqSource` for diagnosis only.

21.5. Region control

The "regions" the counter commands index are the address ranges of the performance-counter block at `0x3000`, not a dedicated register pair. Four independent range banks exist – data read, data write, instruction fetch above threshold, data read above threshold – each with its own set of `Low`/`High` register pairs and an extra counter for accesses that fall outside every configured range. The range registers are write-locked while `trTeControl.Enable = 1`, deliberately: a range is two registers, and a mid-trace update would be visible for one bus cycle as a nonsense interval built from the new `Low` and the old `High`.

The bank sizes are elaboration parameters and are readable at runtime in `trTeConstants (0x3008)`. The performance chapter carries the full description.

21.6. TracePoint programming

What earlier design notes called the "TracePoint register" is implemented as the ACT-ST watchpoint table: each slot is an `{Addr, Cmd}` pair, where `Addr` is the arming instruction address and `Cmd` the command word dispatched on a match. The table is loaded through the indirect register path (`trWpIndex`, `trWpDataLow`, `trWpDataHigh`) and read back through `trWpReadLow`/`trWpReadHigh`; the programming rules are in the instrumentation chapter.

The matching is performed by `ct_L23_preproc_act_st.sv` against `tip.iaddr`. Separate control words for an interrupt entry, an interrupt return or an interrupt-enable change – which the original concept sketch foresaw – are not implemented; see [Section 21.9](#).

21.7. Trace destination

The destination of a transfer is the `Sink` field of the command word: `ACT_CAP_ST_SINK_NEXUS` for the external trace port, `ACT_CAP_ST_SINK_AXIS` for the on-chip AXI-Stream consumer, `ACT_CAP_ST_SINK_AXIS_NEXUS` for both, `ACT_CAP_ST_SINK_TE` for a control-register write with no message at all.

The encoder instantiates **one** AXI-Stream master (`axis_if.master` on `ct_encoder`), so the choice is between the two ports, not among several numbered stream destinations. The `tid` field of the beat carries the command value; `tdest` is not driven by the encoder. Fan-out to several on-chip consumers is an integration task – the shared TGC5B base (`examples/kv260/common/tgc5b/`)

and the `tgc5b2_axis_wp` example show one way of doing it.

21.8. HSI sequence correctness

Three commands depend on a data access that happened **before** them: `DAQ_DATA`, `DAQ_DADDR` and `DAQ_DATA_DADDR`. They report the most recent retired data access the composer has latched. The intended pattern is therefore:

<code>load or store</code>	<code><-</code> the access to be reported
<code>csrw 0xB10, <command></code>	<code><-</code> the command that reports it

Two properties follow, and both matter for a correctness argument:

What the implementation guarantees. The latch is updated on every `tip.dretire`, in the same clock domain and in the same beat order the TIP delivers. The value reported is therefore always a real, retired access of the traced hart — never a partially updated or speculative one. Ordering between the instrumentation branch and the control-flow branch is preserved by the alignment pipelines.

What it does not guarantee. Nothing marks the pairing. If an interrupt is taken between the access and the command, and the handler performs a data access of its own before returning, the command reports **the handler's** access, because that is the most recent one. The encoder cannot detect this: there is no pending-request state, no tag linking a command to a particular access, and no sequence check.

Mitigations available today, in order of cost:

1. **Place the pair where it cannot be split.** Inside a critical section, or in code that is not interruptible in the first place.
2. **Mask interrupts around the pair** if the measurement must be exact. On RISC-V this is the usual save/clear/restore of `mstatus.MIE`:

1	<code>csrr t0, mstatus</code>	<code>/* 1) save</code>	<code>*/</code>
2	<code>li t1, 1 << 3</code>	<code>/* MIE</code>	<code>*/</code>
3	<code>csrrc x0, mstatus, t1</code>	<code>/* 2) disable machine interrupts</code>	<code>*/</code>
4			
5	<code>lw t2, 0(a0)</code>	<code>/* 3a) the access to be reported</code>	<code>*/</code>
6	<code>csrw 0xB10, t3</code>	<code>/* 3b) the reporting command</code>	<code>*/</code>
7			
8	<code>csrw mstatus, t0</code>	<code>/* 4) restore</code>	<code>*/</code>

Note that this protects the **pairing**, not the command: the command write itself needs no protection.

3. **Use `DAQ_DIRECT_DATA` instead** where the value is known to the program anyway. A tag needs no pairing at all and is the cheapest message in the set.
4. **Use `ACT-ST` rather than `ACT-CAP`** where the arming point is a program address. The command

is then issued by the encoder on the matching instruction, so no software sequence exists that could be interrupted.

21.9. Concept elements not implemented

The HSI concept as originally sketched is wider than what is built. The following elements appear in earlier design notes and in the predecessor reference manual; none of them is present in the current RTL or register map. They are listed so that a reader of the older documents can tell concept from implementation, and so that a decoder author does not look for fields that never appear.

Concept element	Status
Two-register control structure (CSR_DATA_ADDR + CSR_CTRL_ADDR) and the "not interrupt safe" transfers built on it (CSR_WRITE1/2/4, LAST_WRITTEN_DATA, LAST_WRITTEN_ADDR, LAST_READ_ADDR)	Not implemented. One CSR address is decoded, 0x0B10, and it carries the complete command in one 32-bit write.
Special counters: wall-clock, stall, instruction-cache miss, data-cache miss, and the per-region write/read counters as a clearable special counter set with an SCNT difference-to-previous-reading field	Not implemented as special counters. A wall-clock reference exists as the timestamp unit's source; per-region read/write and threshold counters exist in the performance-counter block and are read (and cleared) with the four counter commands. There are no stall or cache-miss counters – the TIP does not carry those events.
Counter Emit Control register (CCE, WCE, SCE, ICMCE, DCMCE, WR0CE, ...) selecting which counters ride along with every DAQ message	Not implemented. Each counter command reports exactly one counter, chosen by Cmd and DirectData[7:0].
A DAQ message group with a one-bit sequence identifier tying several messages together	Not implemented. One command produces one message; IDTAG identifies it.
Dedicated Region Control register with two fixed regions (R0L/R0H/R1L/R1H)	Superseded. The performance-counter block carries four independent range banks with several ranges each plus an outside-of-all-ranges counter.
TracePoint control words for interrupt entry (WPIC), interrupt return (WPIRC) and interrupt enable/disable	Not implemented. A watchpoint slot carries one command, dispatched on an instruction-address match.
Numbered internal AXI-Stream destinations [0..n]	Not implemented. One AXI-Stream master exists; tdest is not driven.
Encoder-side stack that assigns a pending data access to a later command by interrupt level and ownership ID	Not implemented. See Section 21.8 .
Power-consumption information in the trace	Not implemented; listed in the original notes as a possible improvement.

Chapter 22. Raw Event Path via AXI-Stream

Besides the encoded trace stream that leaves the encoder through the ATB port, CTTE offers a second, independent export path: an AXI4-Stream master that carries instrumentation events **uncompressed** to a consumer inside the same chip. It exists so that an on-chip agent can act on program events at low latency without owning a Nexus decoder and without competing for the external trace port.

Typical consumers:

- a local CPU acting as a complex watchdog – for example a control-flow integrity checker,
- a runtime-verification monitor evaluating properties over program events,
- a just-in-time compiler that uses observed execution as optimisation input.

The path is fed exclusively by the ACT-CAP/ACT-ST command path. Whether an event goes to this port, to the ATB port or to both is decided per command by the `Sink` field of the command word; the two paths are otherwise independent and can be used simultaneously.

22.1. Where the port sits

The AXI-Stream master is a top-level port of `ct_encoder(axis_if.master axis)`, driven by `ct_L23_preproc_composer_axis` inside the preprocessor. The composer reads the same command interface as the eTIP composer and the same TIP observation registers, so an event that goes to both sinks reports the same values on both.

Two elaboration guards restrict the configuration

(`ct_L23_preproc_composer_axis.sv:51-63`):

- `CT_EN_AXIS_TS` requires `CT_EN_TIMESTAMP` – without the timestamp unit the timestamp element would be a constant zero marked valid.
- `CT_EN_AXIS_TS` requires `CT_EN_ACT` – the sink is fed by the ACT-CAP command path and has no other producer.

Both are structural: a violating configuration fails elaboration on every back end rather than producing a quietly wrong netlist.

22.2. Signal contract

The port is an ordinary AXI4-Stream interface (`rtl/external/amba/axis_if.sv`) in the preprocessor clock domain. The widths every reference design uses:

Signal	Width	Driven by the encoder
<code>tvalid</code>	1	Yes – one cycle per emitted event.

Signal	Width	Driven by the encoder
tdata	96 bit (ACT_CAP_AXIS_TDATA_WIDTH = 3 × 32)	Yes — three 32-bit elements, element 0 in bits [31:0], packed LSB-first.
tstrb	12 bit (tdata width / 8)	Yes — four strobe bits per 32-bit element, so 0xF / 0xFF / 0xFFF mark one, two or three valid elements.
tid	8 bit (ACT_CAP_AXIS_TID_WIDTH)	Yes — the Cmd value of the command that produced the beat.
tready	1	Not sampled. See Section 22.4 .
tkeep, tlast, tdest, tuser	—	Not driven. The beat is single-cycle, so there is no packet boundary to mark, and there is one destination.

The composer derives its element count from the interface ($\text{NUM_ELEMENTS} = \text{TDATA_WIDTH} / \text{ACT_CAP_DATA_WIDTH}$), so the width is nominally an integration choice; the payload layout below assumes the 96-bit configuration that every reference design, the verification environment and the dump/decode helpers use. Two of those helpers hard-code it (`tests/lib/ct_axis_dump.sv` writes a fixed 25-byte record and states the assumption in its header), so a different width needs them adjusted.

22.3. Packetisation and payload

One beat per command. There are no multi-beat packets and no framing. Each beat is self-describing: `tid` says which command produced it and therefore how to read the payload; `tstrb` says how many elements carry data.

Command (<code>tid</code>)	Element 0	Element 1	Element 2
DAQ_PC_CURR	current PC	direct-data tag	timestamp[31:0] (only with CT_EN_AXIS_TS)
DAQ_PC_CURR_LAST	current PC	last PC before exception or interrupt	direct-data tag
DAQ_DIRECT_DATA	direct-data tag	—	—
DAQ_DATA	value of the last retired data access	{dtype, dsize} of that access	—
DAQ_DADDR	address of the last retired data access	{dtype, dsize} of that access	—
DAQ_DATA_DADDR	value of the last retired data access	address of that access	{dtype, dsize} of that access
DAQ_DATA_RD, DAQ_DATA_WR, DAQ_IFETCH_TH, DAQ_DATA_RD_TH	counter value of the region selected by DirectData[7:0]	—	—

Source: `rtl/preproc/ct_L23_preproc_composer_axis.sv:109–202`.

Three commands have **no** AXIS beat: `ACT_CAP_ST_CF_SYNC` and `ACT_CAP_ST_WATCHPOINT` are Nexus-side commands and fall into the composer's default arm, which clears `tvalid`; a command with `Sink = ACT_CAP_ST_SINK_TE` produces no message on either sink. A counter command whose region index exceeds the bank size likewise emits nothing.

22.3.1. The timestamp element

With `CT_EN_AXIS_TS` — the full-profile default — a `DAQ_PC_CURR` beat carries the value the timestamp unit produces (per `trTsControl.Type`, `Active`, `Count` and `Prescale`) as element 2, and the strobe is `0xFFF` instead of the historical `0xFF`. This lets an in-fabric consumer order and correlate PC samples without any Nexus decoding.

Note the deliberate split: `trTsControl.Enable` gates only the `TSTAMP` fields of the Nexus and E-Trace messages, **not** this element. The AXIS sink is not a Nexus wire format, so its shape is a build-time property and the ATB byte stream is identical either way. A build without `CT_EN_AXIS_TS` keeps the two-element shape.

The verification environment checks both directions: with the switch on, element 2 must be valid and strictly monotonic across at least two beats; compiled out, it must be invalid and the strobe must be `0xFF` (`tests/hsi/01_csr_cap/csr_cap_tb.sv`).

22.4. Flow control and loss behaviour

This is the property an integrator must design around, and it differs from what a plain reading of "AXI4-Stream" would suggest:

IMPORTANT

The encoder's AXIS master **never samples `tready`**. `tvalid` is driven directly from the composer's registered valid flag (`ct_L23_preproc_composer_axis.sv:207`), with no dependence on the sink's handshake. Every beat must therefore be consumed in the cycle it is presented, or it is lost — silently, as far as the encoder is concerned.

The design reason is the same one that makes the whole instrumentation path deterministic: backpressure from an on-chip consumer must not be able to stall the trace pipeline, and a consumer that is too slow is a property of the consumer, not an encoder fault. The consequence is that **the shim in front of the consumer owns the loss accounting**.

22.4.1. The reference shim

`examples/kv260/common/ct_axis_wp_shim.sv` is the pattern to copy. It accepts every encoder beat unconditionally and converts the 96-bit single-beat stream into a fully handshaken 32-bit stream:

- **Admission.** Each incoming beat is written into an internal FIFO of `FIFO_DEPTH` records of 116 bit (`tdata + tstrb + tid`). If the FIFO is full the beat is dropped.

- **Accounting.** `drop_count` increments on every dropped beat and saturates at `32'hFFFFFF_FFFF`; `overflow_sticky` latches and is cleared only by reset; `fill_level` reports the records currently held. The top level maps all three into a control window, so a host can tell a complete capture from a lossy one instead of guessing.
- **Serialisation.** Each record leaves as four 32-bit words on a stall-safe master with a full handshake — no loss after admission, back-to-back capable:

Word	Content
0	<code>tdata[31:0]</code> — element 0
1	<code>tdata[63:32]</code> — element 1
2	<code>tdata[95:64]</code> — element 2
3	<code>{8'h00, CORE_ID[3:0], tstrb[11:0], tid[7:0]}</code> ; <code>tlast</code> is asserted on this word, <code>tkeep</code> is constant <code>4'hF</code>

- **Clocking.** Single clock domain, synchronous active-high reset, no CDC — any clock crossing belongs to the downstream FIFO IP, not to the shim.

The `tgc5b2_axis_wp` example on the KV260 uses one shim per core and drains the result over an `axi_fifo_mm_s` that a Linux host reads through `/dev/mem`, in parallel with an ordinary Nexus capture of the same run — which is what makes the two views cross-checkable.

22.4.2. Alternative: a passive capture buffer

Where the events only need to be inspected after the fact, `examples/kv260/common/tgc5b/rtl/ct_soc_axis_buf.sv` shows the simpler pattern: a passive observer that stores one 128-bit record per accepted beat into a block RAM readable from Linux with `devmem`. Its header states the same two cautions that apply to any consumer: decode a beat from its command in `tid`, not from `tkeep`, and treat a full buffer as evidence that beats were dropped.

22.5. Relationship to the ATB trace path

Property	AXI-Stream path	ATB (Nexus) path
Content	Instrumentation events only	Program trace, data trace, instrumentation and status messages
Encoding	Uncompressed 32-bit elements	Nexus MSEO/MDO messages, variable-length fields, leading zeros stripped
Consumer	On-chip logic or a local CPU; no decoder needed	Trace probe or on-chip sink plus a Nexus decoder
Selection	<code>Sink = AXIS</code> or <code>AXIS_NEXUS</code>	<code>Sink = NEXUS</code> or <code>AXIS_NEXUS</code>
Flow control	None towards the encoder (Section 22.4)	Backpressure-aware; overrun produces an Error message and a re-anchoring synchronization

Property	AXI-Stream path	ATB (Nexus) path
Loss visibility	In the consumer's own accounting (shim <code>drop_count / overflow_sticky</code>)	In the stream itself: TCODE 8 with the lost message classes in ECODE
Timestamp	Element 2 of a DAQ_PC_CURR beat, gated by <code>CT_EN_AXIS_TS</code>	TSTAMP field, gated by <code>trTsControl.Enable</code>
Ordering against program flow	By the order of the beats only	By message position in the stream

With `Sink = ACT_CAP_ST_SINK_AXIS_NEXUS` both paths carry the same event. This is the configuration to use when an on-chip monitor must react quickly **and** the event must remain in the off-chip record for later analysis. The two composers sample the same source registers in the same beat, and the timestamp element of the AXIS beat is taken from the same selected value the eTIP composer puts into TSTAMP, so the two views agree.

The counter-clearing commands are the one place where the two sinks touch shared state, and they are separated on purpose: each composer drives its own clear input of the performance counters, so a read on one sink does not consume the other sink's measurement.

22.6. Verification support

Helper	Purpose
<code>tests/lib/ct_axis_decoder.sv</code>	Simulation sink that converts one beat into a structured record – command from <code>tid</code> , three elements from <code>tdata</code> , per-element validity from <code>tstrb</code> – so a testbench asserts on fields instead of slicing vectors. It also flags partial element strobes as an error. Drives <code>tready</code> constantly high.
<code>tests/lib/ct_axis_dump.sv</code>	Simulation-only dump of every accepted beat to a text and a raw file. The raw record is 25 bytes: <code>tid</code> , <code>tdata[95:0]</code> as 12 little-endian bytes, and the 12 strobe bits expanded to one byte each. Instantiating the module is the gate – there is no run-time switch.
<code>rtl/external/amba/test/axis_dump.sv</code>	The generic AXI-Stream dump used at block level.
<code>rtl/preproc/test/ct_L23_preproc_axis_tb.sv</code>	Block-level testbench for the AXIS composer, driving ACT-CAP commands through the TIP.
<code>tests/hsi/01_csr_cap/csr_cap_tb.sv</code>	System-level check: fires every command that produces output to <code>ACT_CAP_ST_SINK_AXIS_NEXUS</code> , counts the beats, matches the <code>DAQ_DIRECT_DATA</code> beat against its expected command and payload, and checks the timestamp element in both <code>CT_EN_AXIS_TS</code> configurations. The beat is identical whether the sink is <code>AXIS</code> or <code>AXIS_NEXUS</code> .
<code>tests/instruction/31_status_msgs/status_msgs_tb.sv</code> , <code>scenario wpaxis</code>	Confirms that a watchpoint command issued on the <code>AXIS</code> sink produces the identical Nexus watchpoint message – the <code>AXIS</code> routing does not change what the trace stream sees.

22.7. Integration checklist

1. **Decide the consumer's admission policy first.** The encoder will not wait. Either guarantee single-cycle consumption, or put a shim with a FIFO and drop accounting in front (`ct_axis_wp_shim`

is the reference).

2. **Expose the drop accounting.** A capture without a drop counter cannot be used as evidence, because a missing event and an event that never happened look the same.
3. **Decode from `t.id`, never from `t.keep` or `t.last`.** Those lines are not driven by the encoder and read back as zero through most integration wrappers.
4. **Match the widths.** 96-bit `t.data`, 12-bit `t.strb`, 8-bit `t.id`. The dump and shim helpers check this at elaboration and stop the build on a mismatch.
5. **Check the profile.** The port exists only with `CT_EN_ACT`; the timestamp element only with `CT_EN_AXIS_TS`, which additionally requires `CT_EN_TIMESTAMP`. In a profile without ACT the port is present but permanently idle.
6. **Budget the rate.** One beat per command means the beat rate equals the command rate. The AXIS path costs no trace-port bandwidth, which makes it the right sink for high-rate instrumentation – but the consumer then carries the rate instead.

Chapter 23. Performance Analysis and Counters

Performance analysis needs detailed information about what a core actually does — which regions it fetches from, which regions it reads and writes, and how long individual accesses take. The Trace Ingress Port already carries that information, so CTTE can aggregate it in hardware without any change to the core and without an additional bus master.

The aggregation happens in one preprocessor block,

`rtl/preproc/ct_L23_preproc_perfcnt.sv`, and is read out through the instrumentation commands described in the command CSR chapter. The counters are part of the ACT group and exist only in builds with `CT_EN_ACT`.

23.1. The performance-counter component

The component occupies `0x3000--0x3FFF` of the control-register map and reports the vendor component type `0x3` in `trPcImpl`. Its registers:

Register	Offset	Function
<code>trPcControl.Active</code>	<code>0x3000</code>	Required component stub. Read-only constant 1 — there is no clock gate or power-down control behind it.
<code>trPcImpl</code>	<code>0x3004</code>	Component version and type; <code>CompType = 0x3</code> , the vendor-assigned type of the performance-counter component.
<code>trTeConstants</code>	<code>0x3008</code>	Read-only implementation parameters, so software can discover the sizes instead of assuming them: <code>num_trace_filter</code> , <code>num_trace_comparators</code> , <code>num_perfcnt_ifetch_th_ranges</code> , <code>num_perfcnt_data_rd_th_ranges</code> , <code>num_perfcnt_data_rd_ranges</code> , <code>num_perfcnt_data_wr_ranges</code> .
<code>trPerfCntControl</code>	<code>0x3010</code>	The two cycle thresholds. <code>IFetchThreshold[7:0]</code> , reset 4, and <code>DataWrThreshold[15:8]</code> , reset 4.
<code>trTePerfCntIFetchRange[i]</code>	<code>0x3100 + 8 × i</code>	Instruction-address ranges for the fetch-threshold counters. Each entry is an inclusive Low/High pair.
<code>trTePerfCntDataRdThRange[i]</code>	<code>0x3200 + 8 × i</code>	Data-address ranges for the read-threshold counters.
<code>trTePerfCntDataRdRange[i]</code>	<code>0x3300 + 8 × i</code>	Data-address ranges for the plain read counters.
<code>trTePerfCntDataWrRange[i]</code>	<code>0x3400 + 8 × i</code>	Data-address ranges for the plain write counters.

The committed configuration provides three instruction-fetch threshold ranges, three data-read threshold ranges, seven data-read ranges and seven data-write ranges (`rtl/pkg/ct_cs_cpuif_pkg.sv:17–20`). These are elaboration parameters of the register map; read `trTeConstants` rather than hard-coding them.

There is no enable bit for the counters and no register that reads a counter value. The counters run whenever the block is instantiated, and the only way to obtain a value is an instrumentation command — which is also what clears it. That is a deliberate design point: a counter read is an event

in the trace stream, ordered against the control flow around it, rather than an asynchronous bus read whose position in the program is unknown.

23.1.1. Enable-locking, and why it is asymmetric

The **range** registers are write-locked while `trTeControl.Enable = 1`. The two **thresholds** in `trPerfCntControl` are not.

The asymmetry is deliberate and is argued in the register source (`rdl/ct_cs_cpuif.rdl:1846–1858`): a range is two registers, `Low` and `High`, so a mid-trace update is visible for one bus cycle as a nonsense interval built from the new `Low` and the old `High`. A single-field scalar cannot be torn that way. The thresholds are compared combinationally against a counter that restarts at every retire, so a new value simply applies from the next measurement window on – no cross-window state can go stale, and the counters they feed leave the encoder as instrumentation values, never as instruction-trace grammar.

23.2. Counter banks

Four independent banks exist, one per access class:

Bank	Counted event	Qualifier in the RTL
Data read	a retired load whose address falls in one of the configured ranges	<code>tip.dretire && (tip.dtype == LOAD)</code>
Data write	a retired store whose address falls in one of the configured ranges	<code>tip.dretire && (tip.dtype == STORE)</code>
Instruction fetch above threshold	a retiring instruction whose elapsed-cycle count has reached the instruction threshold, with the fetch address in one of the ranges	<code>tip.iretire && (IThCnt == trPerfCntControl.IFetchThreshold)</code>
Data read above threshold	a retired load whose elapsed-cycle count has reached the data threshold, with the address in one of the ranges	<code>tip.dretire && (tip.dtype == LOAD) && (DThCnt == trPerfCntControl.DataWrThreshold)</code>

Each bank holds $N + 1$ counters for N configured ranges: one per range, plus one that counts every qualifying access that falls **outside** all configured ranges. The extra counter is what makes a measurement interpretable – without it, a low count in a range cannot be told apart from a low total.

Counter properties:

- **Width 32 bit** (`PERFCNT_WIDTH`, `rtl/pkg/ct_pkg.sv:894`).
- **Saturating.** The counters are instantiated with `MODE_SATURATION` and hold at all-ones instead of wrapping, so a stale reading is visibly saturated rather than silently small.
- **Increment only.** There is no down-count mode and no decrement path.
- **Two independent clear inputs**, one per sink (`...clr_etip` and `...clr_axis`). The eTIP composer and the AXIS composer each drive their own, so a read on one sink does not disturb a measurement being taken on the other.

23.2.1. Range matching

Each bank instantiates one `perfcnt_range_unit`, a thin wrapper around `vector_range_checker`. It compares the address against all Low/High pairs of the bank and returns a hit flag, a no-hit flag and the index of the matching range, in one cycle plus an optional configurable extra delay. The same unit is used for all four banks, so the classification behaviour and latency are identical across them.

Ranges are inclusive at both ends. Overlapping ranges are not rejected by hardware, but the unit returns exactly one index, so an address in two ranges is attributed to one of them; configure disjoint ranges if the attribution must be unambiguous.

23.3. Threshold measurement

The two threshold banks do not count accesses — they count accesses that were **slow**. The measurement is a small state machine (`ct_L23_preproc_perfcnt.sv:70–128`) that maintains two 8-bit cycle counters, `IThCnt` and `DThCnt`, and exists to make sure one event window produces at most one counter update:

State	Behaviour
TH_IDLE	On a retiring instruction both counters restart. On a retiring data access both restart and the machine moves to TH_CNT_IRETIRE. On a cycle with neither, both counters increment; when <code>IThCnt</code> reaches all-ones the machine moves to TH_ITH_OVERFLOW.
TH_CNT_IRETIRE	Counts the cycles between the data access and the next instruction retire. Returns to TH_IDLE on that retire, or moves to TH_ITH_OVERFLOW if the counter saturates first.
TH_ITH_OVERFLOW	Waits for the next data retire and returns to TH_IDLE. The window is discarded.

The qualifier that admits a counter update is an **equality** comparison against the threshold, evaluated on the retire beat — `IThCnt == trPerfCntControl.IFetchThreshold` for the fetch bank, `DThCnt == trPerfCntControl.DataWrThreshold` for the read bank. The counters are therefore a measure of how often an event took exactly the configured number of idle cycles, which for a threshold at the low end of the distribution is in practice "took at least that long, and was the first beat to reach it".

23.4. Reading the counters

A counter is read with one of four instrumentation commands. All four take the region index in `DirectData[7:0]`, may carry a tag in `DirectData[23:8]`, and **clear the counter they report**:

Command	Value	Reads
ACT_CAP_ST_DAQ_IFETCH_TH	8	the instruction-fetch threshold counter of the selected range
ACT_CAP_ST_DAQ_DATA_RD_TH	9	the data-read threshold counter of the selected range
ACT_CAP_ST_DAQ_DATA_WR	10	the data-write counter of the selected range
ACT_CAP_ST_DAQ_DATA_RD	11	the data-read counter of the selected range

Each produces a Data Acquisition message (TCODE 7) whose `IDTAG` is the command value and whose `DQDATA` carries the counter value; on the AXIS sink the same value appears as element 0 of a single beat with `tid = command`.

The index is range-checked before the read. On the AXIS path an index above the bank size takes the `else` branch and no beat is emitted; on the Nexus path the arm is likewise guarded. Both out-of-range branches are exercised by the feature-matrix test rather than left to inspection (`tests/instruction/19_feature_matrix/feature_matrix_tb.sv`, phases Ph.10 and Ph.10b, which drive an in-range command, an out-of-range command and an unknown command through both the ACT-ST table and the direct ACT-CAP CSR path).

Because the read clears, a series of reads yields **deltas**: the count since the previous read of the same counter. A measurement over a whole run is therefore the sum of the reported values, and a lost message – which the encoder reports as an Error message with the DAQ bit set – loses one interval rather than corrupting the total silently.

23.5. Determinism and latency

The counter block reports a fixed internal latency of one cycle (`internal_delay = 1`) for alignment with the adjacent preprocessing stages. Updates are qualified strictly by retire signals, and each bank's clear handling is explicit and independent, so software can snapshot and reset one class without cross-coupling into the read, write and fetch domains of the others. The latency does not depend on the values counted or on the number of configured ranges.

23.6. Using the counters

A workable measurement procedure, given that the counters have no enable bit and no bus read path:

1. **Configure the ranges while tracing is disabled** (`trTeControl.Enable = 0`). Set the thresholds too; they may also be changed later.
2. **Decide how the read happens.** Either instrument the program with ACT-CAP commands at the points where a reading is wanted, or arm ACT-ST watchpoints on those addresses and leave the program untouched. The second is preferable for production binaries and for tight loops.
3. **Choose a sink.** `ACT_CAP_ST_SINK_NEXUS` puts the readings into the trace stream, ordered against the control flow – which is what makes a reading attributable to a program phase. `ACT_CAP_ST_SINK_AXIS` keeps them on-chip and costs no trace-port bandwidth.
4. **Read the first sample early** to clear the counters, so that the first interval starts at a known point rather than at reset.
5. **Watch the interval length.** A counter read per iteration of a hot loop is a message per iteration; the bandwidth guidance in the command CSR chapter applies unchanged.

23.7. Related measurements documented elsewhere

- **Timestamps.** The timestamp unit (`trTsControl`, 0x040, with the 64-bit counter at 0x048 / 0x04C) is described with the trace format; its value rides as the `TSTAMP` field of synchronization, data and DAQ messages, and – with `CT_EN_AXIS_TS` – as element 2 of a `DAQ_PC_CURR_AXIS` beat.
- **Internal FIFO fill level.** The fill-level histogram (`trTeTipFifoHistCtrl` 0xE10 / `trTeTipFifoHistData` 0xE14, `CT_EN_FIFO_HIST`) and the peak watermark (`trTeTipFifoStatus.trTeTipFifoMaxFill` 0xE04) measure the **encoder**, not the core. They are the right instrument for sizing a trace configuration; see the development-support appendix. The histogram registers carry a read contract: valid only while the trace is quiescent.
- **Trace-stream reduction by the compression suite.** Reduction figures for the individual compression features belong with the compression chapter, together with the workload they were measured on.

Chapter 24. Footprint Profiles and Resource/Timing Matrix

CTTE is not one netlist. Which logic is built is decided at elaboration by the `CT_EN_*` and structural switches in `rtl/pkg/ct_pkg.sv`, and a **profile** is one named combination of them. This chapter defines the profiles the project measures, states the measured resource and timing figures for each, and names the tool version, commit and report file behind every number.

24.1. What a profile is, and where it is defined

A profile is a set of assignments to the switches in `ct_pkg.sv`. Every measurement flow applies them from **one** definition, `scripts/ct_profiles.sh`, rather than hand-editing the package — so the resource matrix below and the per-switch cost figures in the integration guide describe the same builds.

`ct_profile_names` currently yields five names: `voll_gold`, `featparity_gold`, `slimfull_gold`, `only`, `eonly`. They are built from two bases plus a fixed set of nine feature switches:

Definition	Content
<code>ct_profile_voll_base_in</code>	The maximum configuration: data trace, DAQ/ACT, filters and the complete compression suite on; multi-clock; 64-bit timestamp; two slicer steps; Nexus formatter egress; eTIP serialization on.
<code>ct_profile_featparity_base_in</code>	The compact control-flow-only configuration: data trace, DAQ/ACT, filters and the compression suite compiled out; single clock; 32-bit timestamp; one slicer step; compact-packer egress; serialization off. <code>CT_EN_WATCHPOINT_MSG</code> and <code>CT_EN_AXIS_TS</code> follow <code>CT_EN_ACT</code> , and <code>CT_EN_DF_DROP</code> / <code>CT_EN_DF_ADDR_COMPRESS</code> follow <code>CT_EN_DATA_TRACE</code> — those are elaboration dependencies, not preferences (<code>scripts/check_profile_deps.py</code>).
<code>CT_GOLD_FEATS</code> (the <code>_gold</code> suffix)	Nine switches turned on over the base: <code>CT_EN_DEBUG_EVENTS</code> , <code>CT_EN_POWER_EVENTS</code> , <code>CT_EN_EVTI</code> , <code>CT_EN_TRIG_SYNC</code> , <code>CT_EN_SEQ_SYNC</code> , <code>CT_EN_OWNERSHIP</code> , <code>CT_EN_CONFIG_MSG</code> , <code>CT_EN_IBHS</code> , <code>CT_EN_REPEAT_INSTR</code> .

Profile	Definition
<code>voll_gold</code>	<code>voll_base</code> + the nine gold switches, N-Trace back end. This is the committed full profile.
<code>featparity_gold</code>	<code>featparity_base</code> + the nine gold switches, N-Trace back end.
<code>slimfull_gold</code>	<code>featparity_gold</code> plus <code>CT_ETIP_CDC_SLIM = 1</code> , <code>CT_EN_ETIP_WATERMARK = 0</code> , <code>CT_EN_TIMESTAMP = 0</code> (and therefore <code>CT_EN_AXIS_TS = 0</code>). The smallest profile the project measures.
<code>only</code>	<code>voll_gold</code> with the back-end selector explicitly set to N-Trace. The two definitions produce a byte-identical <code>ct_pkg.sv</code> , so this is the same build as <code>voll_gold</code> and its run serves as a control.
<code>eonly</code>	<code>voll_gold</code> with the E-Trace back end selected instead.

The `_gold` profiles deliberately leave `CT_EN_BTM`, `CT_EN_SYNC_STATUS`, `CT_EN_FIFO_HIST`,

CT_EN_QUOTA_SYNC, CT_EN_DEVICE_ID, CT_EN_TRIG_REGS and CT_EN_INST_SYNC_REQ at their committed full-profile value of 1. A build that wants a smaller footprint than `slimfull_gold` has to switch those off as well; that build is not measured here.

24.2. Resource and timing matrix

Table 7. Out-of-context synthesis of `ct_encoder`, Vivado™ 2022.1.2, commit 7a0b7b2, device xck26-sfvc784-2LV-c, 2026-08-06 – reports under `verification/evidence/P10/<profile>/`

Profile	CLB LUTs	Register as FF	Block RAM tiles	DSP48E2	WNS @ 13.333 ns (75 MHz)	WNS @ 5.0 ns (200 MHz)	Derived $F_{\max}$
<code>voll_gold</code> —committed full profile, N-Trace back end	25 784	20 168	9.5	16	+4.175 ns	−4.158 ns	109.2 MHz
<code>nonly</code> —same build as <code>voll_gold</code> , run as a control	25 784	20 168	9.5	16	+4.175 ns	−4.158 ns	109.2 MHz
<code>eonly</code> —full profile, E-Trace back end	26 372	16 953	9.5	16	+1.753 ns	−6.580 ns	86.4 MHz
<code>featparity_gold</code> —control-flow only, compact packer, single clock	4 938	4 824	10	16	+8.672 ns	+0.339 ns	214.5 MHz
<code>slimfull_gold</code> — <code>featparity_gold</code> + slim eTIP CDC, no watermark, no timestamp unit	4 290	3 984	10	16	+8.695 ns	+0.362 ns	215.6 MHz

Provenance of every row in this table. Out-of-context synthesis of the top module `ct_encoder` on the device `xck26-sfvc784-2LV-c`, run on 2026-08-06 with Vivado™ 2022.1.2 (build 3605665, the version `.abc.config pins`) at commit **7a0b7b2**. Reproduce with

```
bash scripts/p10_timing_matrix.sh
```

LUTs, flip-flops, block RAM and DSP are read from

`verification/evidence/P10/<profile>/timing_75mhz_util_flat.rpt`; the two WNS columns from `timing_75mhz_summary.rpt` and `timing_200mhz_summary.rpt` in the same directory; the verdict from `verification/evidence/P10/timing.log`.

How $F_{\max}$ is derived, since the derivation is otherwise invisible: critical path = budget − WNS, $F_{\max} = 1000 / \text{critical path in ns}$. For `voll_gold`: $13.333 - 4.175 = 9.158 \text{ ns} \rightarrow 109.2 \text{ MHz}$. The figure is a post-synthesis derivation, not a characterized maximum frequency.

24.2.1. How to read these numbers

- **At the 75 MHz board budget every profile meets timing.** The smallest margin is `eonly` with +1.753 ns; Vivado's own line in all five reports is "All user specified timing constraints are met", with zero failing endpoints.
- **At 200 MHz only the two slim profiles meet timing** (+0.339 ns and +0.362 ns); the three full-

profile builds miss it by 4.158 ns and 6.580 ns. 200 MHz is not a requirement of this project – the column is headroom information and is reported as measured.

- **This is post-synthesis, not placed-and-routed.** Routing takes some of the margin back. The synthesis is out of context with one clock per clock port at the same budget and `set_clock_groups -asynchronous` between the five clock domains; the encoder's CDC structures are exactly what a real SoC constraint file decouples the same way, so cross-domain paths are outside this analysis.
- **A budget without its slack figure says nothing, and neither does a slack figure without its budget.** That is why two budgets are reported and why the constrained clock of the reference design is named.
- **Two controls came out of the runs themselves.** `only` and `voll_gold` are the same netlist, and the two independent runs agree on every figure including total negative slack and failing-endpoint count. And the two budgets of each profile agree on the critical path to the picosecond, which is what makes the second run of a profile a control rather than a repetition.

24.2.2. One placed-and-routed data point

The single place-and-route figure the project holds is the `mbv_kv260` reference design of 2026-08-01: **23 428 LUTs / 19 065 FFs**, WNS +2.679 ns at 75 MHz single-clock. It predates the P2 to P9 work and is therefore **not** comparable with the matrix above; it is quoted only as an order-of-magnitude check that an in-design build lands in the same region as the out-of-context one. (The value was corrected on 2026-08-14: the previously quoted +2.186 ns was Vivado's router estimate, not the routed timing summary.)

24.3. The structural levers between the profiles

The distance between the full profile and the slim ones is not one switch. Four structural knobs carry most of it, and they are the ones to reach for first when a footprint target is missed.

Knob	Effect
CT_COMPACT_PACKER	Replaces the formatter, slicer and MSEO stages of the egress path with a lean control-flow-only packer. Measured: the historical formatter path costs +1 018 LUTs / +623 FFs over the compact packer in the same profile (<code>doc/integration.adoc</code> , differential bases A = <code>featparity</code> with the packer on, 3 552 LUTs / 3 598 FFs / 6.5 BRAM, and B = <code>featparity_noc</code> with it off, 4 570 LUTs / 4 221 FFs / 6.5 BRAM; OOC, Vivado™ 2026.1, 5.0 ns target). The packer and the data-flow/DAQ message formats are mutually exclusive by construction – <code>ct_L2_compact_packer</code> stops elaboration with <code>"CT_COMPACT_PACKER` requires a CF-only profile"</code> .
CT_ETIP_SERIALIZE	Serializes the internal eTIP message path instead of carrying the parallel slots. Full-profile default since the E4 round; the resulting stream is byte-identical, proven on 15 of 15 reference streams. See the finding below on the cost figure.
CT_ETIP_CDC_SLIM and CT_EN_ETIP_WATERMARK	The two levers between <code>featparity_gold</code> and <code>slimfull_gold</code> , together with <code>CT_EN_TIMESTAMP</code> . Measured as the difference between those two rows of the matrix: –648 LUTs / –840 FFs. Of that, switching the timestamp unit off is worth about –300 LUTs / –490 FFs and the watermark about –24 LUTs / –32 FFs in the <code>featparity</code> context, so the slim eTIP CDC carries the remainder.

Knob	Effect
CT_SINGLE_CLOCK, CT_TS_WIDTH, CT_SLICER_STEPS, CT_MICRO_CSR, CT_ETIP_FIFO_STYLE, CT_FIFO_HIST_BINS	The remaining structural sizing knobs; each is documented at its definition in <code>rtl/pkg/ct_pkg.sv</code> . The base profiles set the first three; the others stay at their committed values in every measured profile.

24.4. What the instrumentation path costs

The material of Part D – the ACT blocks, the DAQ message path and the watchpoint message – is compiled out together in the slim profiles, so its cost does not appear as a separate row above. Two measurements bound it.

The group figure. `CT_EN_DAQ` + `CT_EN_ACT` + `CT_EN_WATCHPOINT_MSG` together measure **+8 965 LUTs / +10 514 FFs / +12.5 BRAM** against differential base **B** (OOC, Vivado™ 2026.1, 5.0 ns target; `doc/integration.adoc`, feature-flag table). The three switches are quoted as a group because they cannot be separated: `CT_EN_DAQ` alone is functionally dead – its only producer sits behind `CT_EN_ACT` – and `CT_EN_WATCHPOINT_MSG` fails elaboration without `CT_EN_ACT`. The group cannot be measured against base **A** at all, because the compact packer and the DAQ message format are mutually exclusive.

Five mechanisms are code-verified as the drivers, and they compound inside the same message queue: the DAQ payload (three 32-bit elements plus a 32-bit address = 128 bits) becomes the widest arm of the packed message union and is therefore paid by **every** queued message; the queue depth quadruples from 32 to 128; the parallel slot count grows from three to four; the watchpoint read-back shadow is an explicit block-RAM copy; and the ACT-ST engine is a pipelined binary search with 32-bit comparators per level. No bit-level accounting is claimed.

The watchpoint capacity step. The group figure above was measured at the historical table shape (`M0_DIM` = 4, a four-level tree with 15 slots). The current implementation has ten levels and 1023 slots, and the preprocessor alignment budget `PREPROC_DELAY_MAX` grew from 20 to 44 for `CT_EN_ACT` profiles with it. The 68-fold capacity step was measured as an OOC pair on the full profile (Vivado™ 2022.1.2, `xck26-sfvc784-2LV-c`; verdict `verification/evidence/C0b/ooc_pair.log`):

Build	CLB LUTs	Register as FF	Block RAM tiles
pre—commit 64ae043979, <code>M0_DIM</code> = 4 (<code>verification/evidence/C0b/ooc_pre_util_flat.rpt</code>)	26 307	20 743	9.5
post—commit b4cc804692, <code>M0_DIM</code> = 10 (<code>verification/evidence/C0b/ooc_post_util_flat.rpt</code>)	30 432	31 204	14.5
delta	+4 125	+10 461	+5

The flip-flop side is the alignment pipes – every `EXTRA_DELAY_MAX`-deep pipe in the preprocessor follows `PREPROC_DELAY_MAX`; the block-RAM side is the ten-level tree plus the 1024 × 64-bit

read-back shadow.

NOTE

This pair is a **different** commit and a different build state from the matrix in the previous section. Its rows may be compared with each other – that is what a differential pair is for – but not with the matrix.

24.5. Choosing a profile

If the requirement is	Start from
Full instrumentation: DAQ, ACT-CAP/ACT-ST, watchpoint messages, data trace, filters and the complete compression suite	<code>voll_gold</code> . This is also the profile the register map is committed for – a slim build regenerates the map through <code>scripts/gen_rdl_profile.py</code> .
The E-Trace back end instead of N-Trace	<code>eonly</code> . The protocol is a synthesis parameter, one back end per encoder instance.
Smallest control-flow-only encoder with the gold feature set	<code>slimfull_gold</code> , then <code>featparity_gold</code> if the timestamp unit is needed.
Smaller than <code>slimfull_gold</code>	<code>slimfull_gold</code> with <code>CT_EN_BT</code> , <code>CT_EN_SYNC_STATUS</code> , <code>CT_EN_FIFO_HIST</code> , <code>CT_EN_QUOTA_SYNC</code> , <code>CT_EN_DEVICE_ID</code> , <code>CT_EN_TRIG_REGS</code> and <code>CT_EN_INST_SYNC_REQ</code> switched off as well. That combination is not measured in this manual; measure it before quoting a figure for it.

Two rules apply to any profile change:

- 1. Regenerate the register map.** The committed `ct_cs_cpuif.sv` is the full profile. After editing the switches, run `python3 scripts/gen_rdl_profile.py`; it rewrites `rdl/ct_profile.inc.rdl` and the generated register block together, so a compiled-out feature's CSR bits read as 0 instead of holding a value nothing consumes.
- 2. Let the dependency checks run.** `scripts/check_profile_deps.py` and make `check-flags` verify that the switch combination is legal and that the feature-flag documentation still matches `ct_pkg.sv`, the SystemRDL source and the capability map. Several dependencies are enforced by elaboration guards rather than by convention, and an illegal combination fails the build rather than producing a quiet mis-configuration.

Chapter 25. Integration Guide

This chapter is the contract an integrator drops `ct_encoder` against: clocks and resets, the instruction ingress, the control interface's locking rules, trace egress and protocol selection, and the shutdown semantics. It condenses `doc/integration.adoc` (71 KB) to the decisions an integrator has to make and the guarantees they can rely on; the derivations, measured cost tables and every per-switch footnote stay in that public document, cited by section below. The pin-accurate signal-level contract (port lists, timing diagrams) is the interface contract chapter; the register-by-register reference is the register model chapter. Configuration recipes and the pitfalls found in practice are their own chapter, next.

25.1. Clocks and resets

`ct_encoder` spans five clock domains, each with its own synchronous reset. Every CDC crossing is internal (gray-pointer FIFOs plus 2-FF synchronizers) – the integrator only connects clocks (`doc/integration.adoc`, "Clocks and resets"):

Clock	Drives	Constraint
<code>tip_clk</code>	Instruction ingress (<code>tip_if</code>), preprocessor front	Must be the core clock – the <code>TRACE/TIP</code> bus is synchronous to retirement.
<code>proc_clk</code>	Message generation, formatting	May equal <code>tip_clk</code> (the proven board profile, 1:1) or run faster for headroom.
<code>atb_atclk</code>	ATB egress, the <code>trTeControl.Empty</code> view	Sink-side clock; 1:1 with <code>tip_clk</code> is proven on hardware. A slower drain only raises the FIFO-overflow rate (announced loss, see below).
<code>wb_clk</code>	Wishbone CSR access	Any frequency.
<code>wall_clk</code>	Wall-clock timestamp source	Optional feature clock.

A fully single-clock integration (all five tied to one clock, as on the KV260 reference designs in the demonstrators chapter) is supported and is the configuration the hardware robustness campaign runs on.

Resetting one domain without the others is allowed, and it has a defined cost. An integrator may hold the core (and `tip_rst`) in reset while the CSR domain keeps running – that is what makes the encoder programmable across a core reset. Two consequences are contract, not accident:

- An explicit sync request (`trTeControl.InstSyncReq`, and the ATB sync request the same way) survives a reset of the **consumer's** domain: the request crosses as a four-phase LEVEL handshake (`ct_sync_req_pacer`) specifically so that a `tip_rst` clearing the generator's pending state does not lose a request still standing on the launch side. The direction matters: a `wb_rst` resets the **launch** side, and there a pending-but-unserved request is simply gone (no message, no error, no status bit) – the correct behaviour, because a reset CSR domain has no pending software intent left to honour. Proven for one clock (`formal/preproc_sync`, task `tereqrst`) and measured across two unrelated clocks (`scripts/cli_syncreqrst_test.sh`).

- It may cost one extra synchronization message if the reset lands between the generator serving a request and the launch side observing the acknowledgement — always safe to emit, and a reset re-anchors the decoder anyway.

Every other domain resets in the ordinary way: state in the reset domain is gone, and the trace stream re-anchors on the first qualifying retirement afterward.

25.2. Instruction ingress (`tip_if`)

One beat per retirement opportunity, single-retirement by default (`CT_EN_BLOCK_TIP = 0`). The rules integrations get wrong most often (`doc/integration.adoc`, "Instruction ingress"):

- `iretire=1` marks a retired instruction; `iaddr/ilastsize` must be valid on that beat.
- A synchronously faulting instruction (`ecall/ebreak/illegal`) does **not** retire (`iretire=0`, `itype=EXCEPTION_TRAP`) and its PC never appears in the decoded stream — the 1:1 retirement rule. Interrupts use `itype=INTERRUPT` with `iretire=0`; the pre-empted instruction is re-executed after `mret` and appears then.
- `itype` must be valid whenever it is non-OTHER, even with `iretire=0` (trap markers ride non-retiring beats).
- Sideband levels (`debug_mode`, `power_down`) must rise after the last pre-window retire and fall before the first post-window retire.

Block ingress (`CT_EN_BLOCK_TIP = 1`) is the escape hatch for a superscalar hart that cannot serialize its commit stream: `iretire` becomes the halfword count retired by the beat (width `CT_IRETIRE_WIDTH`, elaboration-checked to 2..8 — the lower bound because a 1-bit block bus cannot report even one 32-bit instruction, the upper because a beat that alone reaches the `ICNT` pre-drain threshold wedges the indirect-branch history hold), `iaddr` becomes the block's **first** instruction, `ilastsize` the **last** instruction's size, and `itype` how the block **terminated**. Only the last instruction of a block may be a control-flow event. A core whose block generator has no bound of its own (CVA6's ITI accumulates until the next control-flow instruction) must have its adapter split longer linear runs into several `itype=OTHER` blocks — free on the wire, since an `OTHER` block raises no message and only accumulates the count. The switch adds no message type and no wire field (no CAPS bit): a block beat produces exactly the messages a serialised beat sequence would, measured byte-identical in the reset configuration (`scripts/cli_blocktip_test.sh`). Two consequences an integrator still has to reason about even though the stream stays well-formed either way: `InstSyncMode = 6` counts **retirement beats**, not instructions, so periodic sync lands after a different amount of program at a block ingress; and an armed instruction filter sees block starts only, so per-instruction address filtering is not expressible on a block ingress — a property of the port, not a filter shortfall. Use the single-retirement ingress where per-instruction filtering matters.

`tip_if` is deliberately the **only** ingress contract: everything core-specific lives in an adapter outside the encoder. Three adapter families exist today (AMD MicroBlaze™ V TRACE bus, the CVA6 instruction-trace interface fed from RVFI probes, and the Rocket TraceCoreInterface) — their port mapping and `itype` semantics are the trace-sources-and-adapters chapter's subject, not this one. Writing a fourth adapter means answering the same three questions for the new core — which

beat is a retirement, how a trap is signalled, whether the address is the instruction's or the next one's — against a measured truth table before it reaches RTL.

25.3. Control interface and programming contract

CSR access is Wishbone (`wb_if`); an AXI-Lite bridge (`ct_axil_to_wb`) is available at integration level. Feature configuration is **Enable-locked**: `trTeInstFeatures` and the sync-mode fields are writable only while `trTeControl.Enable=0` — writes issued after arming are silently void. The required order is **Active → configuration → Enable + InstTracing**. A short, deliberately non-locked exception list (session handles, status-clear bits, the message-class enables a decoder recognises by TCODE, the timestamp unit's own run control, and the two ACT stall thresholds) stays writable while the trace runs; the authoritative list is the `swwel` block header in `rdl/ct_cs_cpuif.rdl`, machine-checked by `make lint`. The configuration chapter documents the one historical gap in this lock (closed 2026-08-16) and the concrete failure mode of getting the ordering wrong — read it before writing a board driver from scratch.

`trTeControl.Active` gates the encoder's clock enables; keep it set for the whole session, including shutdown polling (see below). The decoder invoked against a capture must be told which features were programmed (e.g. `-bp` iff `InstEnBranchPrediction` was set) — or, since `trTeControl.SendConfig`, it can read that from the stream itself; see the decoding chapter.

25.4. Trace egress and protocol selection

Trace bytes leave on ATB (`atb_if`) in the framing of the back end the encoder was built with: Nexus MDO/MSEO chunks for N-Trace, reference-raw `te_inst` packets for E-Trace. **Which protocol an encoder speaks is a synthesis-time decision, not a runtime one.** `ct_encoder` takes two parameters, `EN_NTRACE` and `EN_ETRACE` (default from the build profile, `CT_EN_NTRACE` / `CT_EN_ETRACE` in `rtl/pkg/ct_pkg.sv`); exactly one must be set per instance, both-0 and both-1 abort elaboration. Only the selected back end is instantiated, so an unused protocol costs nothing, and a multi-encoder SoC can mix protocols across instances out of the same netlist sources — see the configuration chapter for the pitfall this creates (every instance needs an **explicit** parameter once more than one protocol is in the netlist). Software discovers the built-in protocol read-only: `trTeProtocolSel.Protocol` (0 = N-Trace, 1 = E-Trace) and `trTeImpl.ProtocolMajor` (1 = N-Trace 1.x, 2 = E-Trace 2.x); there is no write path — an earlier runtime-selectable dual build was removed in 2026-08 because it cost a full second back end in area and was never used in a product.

`atb_te_raw` (0 = Nexus, 1 = `te_inst`) advertises the framing to a downstream funnel or sink. The funnel shipped in this repository (`ct_l1_funnel`, see the funnel chapter) merges **N-Trace channels only** — it derives packet boundaries from the Nexus MSEO bits, which an E-Trace channel does not carry. Feeding an E-Trace instance into a mixed SoC therefore needs its own ATB sink until the raw-framing funnel extension lands.

Backpressure via `atready` is legal at any time; sustained backpressure raises FIFO-overflow episodes, which are **announced loss** — an ERROR message plus a recovery sync re-anchor the decoder, never a silent drop (verdict-tested across backpressure duty/random/burst profiles). A live

consumer that wants a **bounded** re-anchor distance inside a ring-buffer window programs the trace-quota cadence; the recipe with measured board numbers is in the configuration chapter.

25.5. Pause, stop, flush — the shutdown contract and bring-up order

These semantics are contract-tested (gate `11_pause_edge`, campaign verdicts V7/V8):

- **Pause** (`InstTracing 0→...→1` with `Enable` held): the encoder closes the stream with a trace-off correlation (TCODE 33), suppresses everything positionless, and re-anchors with a `TRACE_ENABLE` sync. Pauses may be issued asynchronously at any time.
- **Flush** (`afvalid` or the sink's flush control) drains all queued trace. To reach `Empty=1` afterward, **close the stream while the core still retires**: with `InstTracing=1` and a stopped core, exactly one control-flow event is permanently in flight (its successor address can never be observed), and `Empty=0` is the **correct** report in that state, not a bug.
- `trTeControl.Empty` is a conservative, CDC-safe status: it rises only after the entire chain (accumulators, formatter, packer, CDC FIFOs, ATB queue) has been observably quiet, and falls immediately on any visible substance.

The proven order, both for bring-up and for shutdown, is:

1. Hold the domains in reset as needed; bring up clocks per the table above.
2. `trTeControl.Active = 1`, kept set for the whole session.
3. Program configuration (`trTeInstFeatures`, sync mode/max, `SendConfig`, filters, ...) while `Enable = 0` — see the programming contract above and the configuration chapter's pitfalls before deviating from RDL reset values.
4. `Enable = 1`, then `InstTracing = 1` (and `DataTracing = 1` if used).
5. Run.
6. To shut down cleanly: `flush → InstTracing = 0` **while the core still retires** → stop the core → poll `trTeControl.Empty`.

25.6. Build switches, runtime fields, and structural parameters

The full switch-by-switch reference — every `CT_EN_*` compile switch against its runtime CSR field, reset value, CAPS bit, per-profile default and measured LUT cost — is generated from `rtl/pkg/ct_pkg.sv`, `rdl/ct_cs_cpuif.rdl` and the CAPS map (consistency machine-checked by `scripts/check_feature_flags.py`, `make check-flags`) into the register model chapter; it is not reproduced here; the reset footprint/timing figures per profile live in the footprint-profiles chapter. Two structural parameters that are decisions an integrator makes once, at elaboration, and cannot change afterward:

- **CT_XLEN** — the attached hart's architectural address width, 32 (default) or 64. Not a runtime

mode: it sizes the TIP address ports, the Nexus address fields, the address comparators, the jump-target-cache index fold and the eTIP payload in one place, and is advertised on the wire (CAPS bit 23, ADDR64) because a decoder cannot infer it from variable-length address fields with leading zeros stripped. Constraint: CT_EN_ETRACE with CT_XLEN = 64 is rejected at elaboration (the `te_inst` address fields are sized for 31 bits; 64-bit E-Trace is a separate step).

- **CT_CONTEXT_WIDTH** – width of the TIP context bus (`tip._context`), 2 by default or as wide as the attached hart's context identifier (16 for RISC-V `satp.ASID` under Sv39/Sv48, 9 under Sv32). Unlike CT_XLEN this is **not** advertised on the wire and needs no CAPS bit: the Ownership message carries it in a 44-bit variable-length field that is self-delimiting on the wire. Legal range 1..44, elaboration-checked – a wider bus would otherwise be truncated silently by the composer's cast and the stream would name the wrong process.

25.7. Versioning caveat for core-specific adapters

Adapters from non-standardized core trace buses (for example the AMD MicroBlaze™ V TRACE bus) are **pinned to a specific core/tool version** by construction – the bus itself is documented as not backward-compatible. Keep the adapter outside the encoder core and re-characterize it per core version; that characterization gate lives with the SoC integration project, not with the encoder (`doc/integration.adoc`, "Versioning caveat for core-specific adapters").

Chapter 26. Configuration Recipes and Pitfalls

This chapter collects the configuration mistakes that are known to have actually happened, with the exact register values involved, and the recipes that turn a documented contract (see the integration guide and the register model chapters) into a working configuration. Every entry below is sourced to a commit, a test, or an RDL field — none is a hypothetical "integrators might get this wrong".

26.1. Baseline programming order

Feature configuration is **Enable-locked**: all `trTeInstFeatures` fields and the sync-mode fields are writable only while `trTeControl.Enable=0` (the protocol back end is not among them — it is a synthesis parameter, not a runtime write, see the integration guide). Writes issued after arming are silently void. The required order is:

Active -> configuration -> Enable + InstTracing

Not everything is locked, and the exceptions are deliberate: the session handles (`Active`, `Enable`, `InstTracing`, `DataTracing`, `InstSyncReq`), the status-clear bits (including the `RW1C` overflow flags and the FIFO-histogram controls), the message-class enables a decoder recognises by `TCODE` (`SendConfig`, `Context`, `InstSeqSyncEnable`, `InstTrigEnable`), the timestamp unit's own run control (`trTsControl`), and the two `ACT` stall thresholds `trPerfCntControl.IFetchThreshold / DataWrThreshold` stay writable while the trace runs. The authoritative list, with a reason per group, is the header of the `swwel` block in `rdl/ct_cs_cpuif.rdl`; `make lint (check-swwel-census)` fails if a field leaves that list without either a lock or an entry (`doc/integration.adoc`, "Control interface and programming contract").

NOTE

This lock had a gap until 2026-08-16. An external register-access audit (finding `U10 F-1`) found that the eight `trTeInstFeatures.InstEn*` compression-suite bits were **not** on the `swwel` list, so they could be flipped mid-session even though the rest of the register already behaved as Enable-locked. They are locked now. Software written against an older build that relied on programming them after `Enable` was relying on a write that is documented-void today and must move those writes ahead of the arming step (`doc/integration.adoc`, same section).

26.2. Pitfalls

26.2.1. Vendor Config message silently suppressed by a stale control word

This is the pitfall to check first if a decoder hangs or loses sync partway through an otherwise-working capture.

`trTeControl.SendConfig[8:7]` controls whether the encoder emits the Vendor Config message (TCODE 58) that lets a decoder auto-configure from the stream (self-describing trace — see the decoding chapter and the self-identification chapter). The field resets to 1 (CFG_ONCE, `rdl/ct_cs_cpuif.rdl:218`) — but a hand-built control word that predates the field, or one copied from an older working example, can carry 0 (CFG_NONE) without anyone noticing, because the symptom is conditional on a second, unrelated setting.

Concretely (`c-trace commit 7d91c2f8e0c8458d96af4d6d1d7a787164008374`, `examples/kv260/cva6_linux/board/cva6_linux_boot_trace.sh`): a board driver used the constant `trTeControl = 0x01060067`, which has `SendConfig[8:7] = 0`. With `InstSyncMax = 0` (a sync every 16 instructions) this went unnoticed for weeks — a sync that frequent never let the branch-history buffer overflow, so the decoder never needed the CAPS map the Config message would have carried. Once `InstSyncMax` moved to its RDL reset value 6 (period 1024, see the next pitfall), the first Repeated-History overflow message (RCODE = 2) reached a decoder that had never seen a Config message and therefore did not know the stream's compression format — it walked the listing into `_start_hang` for 20 million PCs (observed on the KV260 reference board, 2026-08-17 23:38). The fix was `trTeControl = 0x016600E7` (`SendConfig = 1 / CFG_ONCE`, `InstSyncMax = 6`, both at their RDL reset values).

Control word	SendConfig[8:7]	Symptom
0x01060067	0 (CFG_NONE)	Symptomless with a tight sync cadence; decoder hang (20 M PCs into <code>_start_hang</code>) once the branch-history buffer actually overflows between syncs.
0x016600E7	1 (CFG_ONCE, RDL reset)	Config message sent on every trace-on edge; decoder auto-configures correctly.

Recipe: never hand-assemble `trTeControl` from an old constant. Start from the RDL reset value and only clear the fields a specific test deliberately needs off (`--sendconfig 0` in the driver above reproduces the old, broken behaviour on purpose, for regression testing only). If a capture must be decoded without a Config message (e.g. `SendConfig = CFG_NONE` for a stream that is meant to stay strict N-Trace 1.0, see the standards-baseline chapter), pass the equivalent flags to the decoder explicitly instead of relying on auto-configuration.

26.2.2. InstSyncMax reset changed from 0 to 6 (2026-08-12)

`trTeControl.InstSyncMax[23:20]` sets the periodic-sync period to $2^{(\text{InstSyncMax}+4)}$ units (unit selected by `InstSyncMode`). The RDL reset value changed from 0 (period 16) to 6 (period 1024) on 2026-08-12. `InstSyncMode` itself still resets to 0 (no periodic cadence at all), so a build that never enables the cadence is bit-for-bit unaffected — what changed is the period a design gets the moment it **does** enable a cadence without also writing `InstSyncMax`.

Measured over 100 010 retires with only the period varied (`doc/release-notes.adoc`, "Changed reset value", `gate tests/instruction/37_sync_cadence`): the old reset (period 16) cost 97 844 trace bytes, the new reset (period 1024) costs 41 072, against 40 060 with the cadence effectively off — a factor 2.44 versus a 2.5 % overhead. Against a finite sink (40 MB/s, same workload) the old reset announced 790 overflows and lost 5 064 events; the new reset is byte-identical to the same run against an infinite sink.

Recipe: if a configuration relies on the periodic-sync **period** matching what the previous reset value gave (16 units), write `InstSyncMax` explicitly — do not rely on the reset. The minimum period remains available as an explicit stress configuration, it is simply no longer the default. Anyone who already programs `InstSyncMax` explicitly sees no change from this reset move at all.

26.2.3. Data-address compression mode invisible on a data-only stream

`trTeDataControl.DataAddrCompress = 1` XOR-compresses data-trace addresses against the previous data-trace message's address (see the trace-format chapter). A decoder learns which mode is active from the Vendor Config message — but the Config message rides on **instruction** tracing (`SendConfig` fires on the instruction trace-on edge). A stream with `InstTracing = 0` (data trace only, no program trace) therefore carries no Config message at all, and a decoder attaching to such a stream mid-stream cannot auto-detect the compression mode from the stream itself (`doc/trace-format.adoc`, "Extensions beyond strict N-Trace 1.0", "Data-only caveat").

Recipe: for a data-only capture, either keep a synchronizing **program**-trace stream running alongside it so the Config message is still emitted, or provide the compression mode to the decoder out-of-band (a command-line flag, a sideband channel) instead of relying on auto-configuration.

26.2.4. Trace-quota window counter does not re-arm the way a naive model expects

With the trace-quota cadence (`CT_EN_QUOTA_SYNC`, `trTeControl.InstSyncMode = 1` or `4`), two behaviours are easy to get wrong when reasoning about worst-case re-anchor distance (`doc/release-notes.adoc`, "Known limitations", "Trace-quota window semantics"):

- The quota counter rearms when the sync is **emitted**, not when it is **requested** — the sync message's own bytes count into the new window, and no second request is raised while one is already pending (no sync avalanche at small quotas).
- After a FIFO-overflow recovery the counter is **not** re-based: the first post-recovery quota sync can arrive **earlier** than one full quota (a harmless extra re-anchor), but never later than the programmed bound.

With `CT_EN_QUOTA_SYNC = 0` compiled out, modes 1 and 4 stay writable on `InstSyncMode` but are inert — no WARL clamp, exactly the pre-feature behaviour, so a build that compiles the switch out does not need its integration software to avoid programming the mode.

26.2.5. Setting both or neither protocol parameter aborts elaboration

`ct_encoder` takes two per-instance parameters, `EN_NTRACE` and `EN_ETRACE`, defaulting to the build profile (`CT_EN_NTRACE / CT_EN_ETRACE` in `rtl/pkg/ct_pkg.sv`). Exactly one must be set per instance — both-0 and both-1 abort elaboration with a `$fatal` (`doc/integration.adoc`, "Protocol choice — a synthesis parameter, per encoder instance"). This only becomes a live pitfall in a **multi-encoder SoC that mixes protocols**: because the choice is per instance, every instance needs an **explicit** `EN_NTRACE/EN_ETRACE` — relying on the package default is only safe when every instance in the netlist wants the same protocol. `scripts/cli_etrace_test.sh mixed` (`tests/instruction/24_protocol_param`) is the gate for exactly this case: two differently parameterised encoders in one netlist, each checked against its own `atb_te_raw` framing advertisement.

26.2.6. Waveform debug of encoder internals crashes Vivado™ XSIM

Not a register-configuration mistake, but a tooling trap worth listing next to the others because it costs the same kind of debugging session: Vivado XSIM 2026.x crashes with waveform instrumentation enabled on this code base; `-debug off` is required (`doc/release-notes.adoc`, "Known limitations"). Use Verilator (with coverage, for `make coverage`) or `$display/counter` instrumentation instead of a live waveform when debugging encoder internals in simulation.

26.3. Recipe: bounding the re-anchor distance for a live/ring consumer

A live consumer that cuts a window of `W` bytes out of a ring buffer needs at least one sync message inside that window to anchor the decode. With the trace-quota cadence (`trTeControl.InstSyncMode = 4`, counting raw ATB bytes — whole output beats including alignment padding and flush idle beats), the maximum sync-to-sync distance becomes a **programmed** number instead of a workload property (`doc/integration.adoc`, "Live / ring operation — bounded re-anchor distance"):

1. Program `InstSyncMode = 4`.
2. Choose `InstSyncMax` so that $2^{(InstSyncMax+4)} \leftarrow W / 2$ — the factor 2 absorbs in-flight/CDC overshoot plus the sync message itself.
3. The RDL reset value 6 already gives an 8-KiB quota (a 16-KiB window), which is why only the mode has to be written for that window size; a 16-KiB dashboard window uses `InstSyncMax = 9` (8-KiB quota) instead.

Measured (simulation, gate `29_sync_quota_bytes`, 64-B quota `InstSyncMax = 2`, mixed compressible/incompressible workload): 12 syncs in 1100 raw ATB bytes, maximum re-anchor distance 96 B (the 64-B quota plus 32 B of in-flight/CDC overshoot); the analytic never-exceed ceiling for the overshoot is +324 B (`scripts/check_sync_window.py`).

Measured (board, KV260 ring-capture path, `InstSyncMode = 4`, `InstSyncMax = 9` → 8192-B

quota, 1-MiB URAM ring filled 3.08 times over, 16 631 920 PCs decoded from 131 071 messages): sync-to-sync distance was a constant **8192 B** over 127 gaps — the programmed quota exactly. 40 randomly cut 16-KiB windows were **40 / 40 anchorable**, with a maximum distance from the window start to its first sync of **8111 B** — 81 B under the programmed quota and 405 B under the never-exceed bound. That headroom is not a proven reserve (the workload behind this capture is highly repetitive, which is why the gaps come out constant); the guarantee is the bound, not the observed constancy. The cadence cost **0.061 %** of the stream (640 sync bytes in 1 048 571 raw bytes) on this capture. Discriminance check on the same capture path at `InstSyncMode = 0`: **0 / 40** windows anchorable (`doc/integration.adoc`, same section; raw data in the internal measurement archive, external to this delivery and not independently verifiable from it).

The message-count cadence (`InstSyncMode = 1`) works the same way with on-wire messages as the counted unit instead of bytes. While neither cadence mode is selected, the underlying counters hold at zero and add no trace bytes.

Chapter 27. Decoding the Trace Stream

CTTE ships a **second, independent** implementation of the wire format it emits — a decoder — because a decode-and-compare loop is what turns a trace byte stream into evidence (see the verification-strategy chapter for the full argument). This chapter is the integrator-facing counterpart: the three places a trace stream gets decoded, what a decoder needs from the stream to configure itself, and the two export formats a decode produces.

27.1. Three decoding paths

27.1.1. HDL-level decoding (in simulation)

`tests/lib/ct_nexus_decoder.sv` (`CEDARtools.TraceEncoder Nexus Decoder` (ATB → `nexus_message_t`)) is an RTL-side Nexus message parser instantiated directly inside testbenches. It is not the primary verification oracle — that role belongs to the PC-stream and CTPX comparison against `cpu_model` described below and detailed in the verification-strategy chapter — but a second, cross-checking implementation at the RTL/testbench boundary:

`tests/lib/test/ct_nexus_decoder_tb.sv` replays a captured ATB binary stream directly into `ct_nexus_decoder` and compares the decoded message sequence and key fields against a checked-in CTTD `-dump` text transcript of the same capture. Because it runs inside the simulator, this path catches message-framing defects without invoking an external binary.

27.1.2. Offline decoding — CTTD, the reference decoder

The reference decoder for every decode verdict in this repository and in its examples is **CTTD** (`CEDARtools.TraceDecoder`) — a **second, independent** implementation of the RISC-V N-Trace / Nexus wire format, not a CTTE component. It is derived from the RISC-V Nexus Trace TG reference decoder **NexRv** (ISC-licensed) and extended by Accemic with E-Trace decoding (`-decoe`), Data Acquisition decoding, multi-target decode (`-target`, for a funnelled multi-core stream) and CTPX export (`doc/verification.adoc`, "Components"; earlier informal names for this tool in source identifiers and older documentation include **NexRv** and **NexRv for C-Trace** — where those appear in scripts or environment variable names (`NEXRV`) they mean this same binary).

Since 2026-08-17 the decoder is **no longer committed** to this repository. It is fetched by version and platform:

```
1 py scripts/fetch_cttd.py          # this host
2 py scripts/fetch_cttd.py --all    # all three platforms (a board deploy needs linux-arm64)
```

The pin — version plus a sha256 per platform — lives in `scripts/cttd.pin` (at the time of writing: `v2.3.0-cttd3`); a checksum mismatch is a hard error, because every decode verdict in this repository is only worth what the decoder binary is (`bin/README.md`). The pin's `base_url` names the CTTD GitHub release the assets are downloaded from. Bumping the pin is deliberate, in its own commit, with the behavioural reason recorded — never a side effect of an unrelated change.

CLI surface used by this repository's own gates (`CT_NEXRV_OPTS` in `scripts/ct_env.sh`):

- `-deco` (decode a Nexus ATB capture), `-decoe` (decode an E-Trace `te_inst` capture), `-dump` (print one line per decoded message/packet, the format the HDL-level replay test above compares against), `-enco` (re-encode, used by cross-checking tools), `-pcinfo <file>` (program-structure input, see below), `-pcout <file>` (write the decoded PC stream), `-full`, `-bp` (branch prediction / jump-target-cache aware decode, needed iff those features are compiled in), `-none`, `-csstrict` (strict reading of the return-address mirror — the suite's default policy; a test that legitimately switches context sets `NEXRV_CS_LENIENT=1` for the lenient reading instead), and `-target <id>` for pulling one source's stream back out of a funnelled multi-core capture by its Nexus `SRC` field.

The decoder's own flags must match the programmed encoder features — e.g. `-bp` iff `InstEnBranchPrediction` was set (see the configuration chapter); getting this wrong is a decode-time error, not the auto-hang class of defect the "SendConfig suppressed" pitfall describes.

`scripts/decode_and_check.sh` wraps CTTD for this repository's test suite: it runs the decoder on a captured `.atb.bin` and compares against the `cpu_model`-generated expected files, selected by flag — `--pc` (PC stream), `--ctxp` (CTXP records), `--data` (loads/stores, address and size only, no CTXP), `--sync N` (at least N sync messages present), `--disabled` (a trace-off correlation message must be present), `--overflow` (an Error/QueueOverrun message must be present), `--soft` (report a divergence without failing the gate).

27.1.3. Online / live decoding

No live/streaming decoder ships in this repository. What the encoder's wire format is built to support is a **bounded re-anchor distance** for a consumer that decodes from a moving window instead of a complete capture — the trace-quota cadence recipe in the configuration chapter is exactly that mechanism, with measured numbers from a board ring-buffer capture. Accemic's live-processing product line, **CEDARtools**, decodes a trace stream in FPGA hardware as it arrives; the top-level `README.md` names it explicitly as one of the targets this design aims for compatibility with ("Targeted for live processing... for improved compatibility with technologies like CEDARtools"). CEDARtools itself is a separate product (see the CTTE-in-context chapter) and outside this repository's scope — this manual documents the wire contract CEDARtools decodes, not the CEDARtools decoder.

27.2. Auto-configuration from the stream

A decoder does not need an out-of-band configuration file to know which optional features a given capture uses: with `trTeControl.SendConfig` enabled (the RDL reset is `CFG_ONCE` — one Config message per trace-on edge), the encoder emits a Vendor Config message (TCODE 58) carrying its compiled-in capability map (CAPS), the currently active runtime configuration (ENAB), and structural parameters (source-ID width, sync mode/period, timestamp configuration, return-stack/branch-predictor/ jump-target-cache sizes). CTTD reads this and configures itself — a flag-less decode of a stream that carries a Config message equals the explicitly-flagged decode. The full field layout is in the self-identification chapter; the pitfall of a control word that accidentally suppresses this message is documented in the configuration chapter ("Vendor Config message silently suppressed").

One field in the capability map is not a feature switch: CAPS bit 23 (ADDR64) tells the decoder whether this netlist carries 64-bit addresses (`CT_XLEN = 64`) — information a decoder cannot otherwise infer, because Nexus address fields are variable-length with leading zeros stripped, so a 40-bit and a 64-bit address whose top bits happen to be zero are byte-for-byte identical on the wire.

27.3. The CTXP export format

Besides the plain PC stream, CTTD reconstructs a second, richer view of a decoded capture: **CTXP** (CTTE eXPort records) — <https://github.com/accemic/CTXP-format>. CTXP records cover control flow (SYNC, BRANCH_*, CALL, RETURN), memory accesses (MEMREAD_n / MEMWRITE_n, carrying the data value, unlike the address/ size-only `--data` check) and instrumentation (DAQ_*, the ACT-CAP/ACT-ST output described in the hardware-instrumentation chapters). `cpu_model` emits the matching expected-CTXP files during simulation, and `decode_and_check.sh --ctxp` compares record-for-record (`doc/verification.adoc`, "The loop" / "Components").

CTTD writes the CTXP text export when the environment variable `CTXP_TEXT_TRACEFILE` names an output file, alongside `-deco/-decoe`. CTXP records additionally carry the absolute time CTTD reconstructed for them, which the `--tsmono` check in `decode_and_check.sh` verifies is monotone.

On the E-Trace side, `tools/etrace/etrace_ctxp_ref.py` is a second, **independent** CTXP producer — a Python reference model that reconstructs control-flow records directly from a `te_inst` raw stream plus an objdump listing, without going through CTTD at all. The E-Trace CTXP gate (`scripts/cli_etrace_ctxp_test.sh`, regression class EC) checks CTTD's CTXP output against this independent model (EC3), against a `dump_te.py` interrupt count from the raw `te_inst` stream (EC4, `tools/etrace/dump_te.py` — the E-Trace sibling of a Nexus `-dump`), and against the simulation's own expected files — three independent cross-checks of the same decode, not one tool trusted three times.

27.4. Program-structure input: PCINFO

A decode that wants to classify control-flow events (direct vs. indirect jump, call vs. return) rather than just list PC values needs a static description of the program under trace. `-pcinfo <file>` takes a CSV-like per-address classification, one line per instruction address: address, a two-letter class plus instruction size in bytes, and — for direct/relative control flow — the resolved target address. Example, from a generated run file (`prog.pcinfo` of an `examples/kv260/mbv/board/run`; run directories stay local to the measurement host and are not part of the published tree):

```
1 0x10, BD4, 0x20
2 0x1C, JD4, 0x10
```

— a 4-byte direct branch at `0x10` targeting `0x20`, and a 4-byte direct jump at `0x1C` targeting `0x10`.

#	Code	Description	RISC-V <code>itype</code> this maps from
1	JD	Jump, direct	INFERRABLE_TAIL_CALL and other direct jumps
2	JI	Jump, indirect	UNINFERRABLE_JUMP
3	BD	Branch, direct (always direct on RISC-V)	TAKEN_BRANCH, NON_TAKEN_BRANCH
4	CD	Call, direct	direct <code>jal</code>
5	CI	Call, indirect	indirect <code>jalr</code>
6	R	Return (always indirect)	<code>jalr</code> to the link register
7	L	Linear	everything else (OTHER)

This repository's own PCINFO producer is `scripts/objdump_to_oracle.py`, which reads `objdump -d -M no-aliases,numeric` output — the `no-aliases,numeric` form is required, because the aliased form (`jalr a5,ret,j`) hides the destination register and misclassifies an indirect call as an indirect jump. Companion tools in the same family: `scripts/nm_to_json.py` (symbol table) and `scripts/bin_to_memh.py` (binary-to-memory-image conversion for simulation); on the E-Trace side, `tools/etrace/objdump2listing.py` and `tools/etrace/pcinfo2listing.py` convert between the `objdump`-derived listing format and a `.pcinfo/` `.nexrv.info` file.

Chapter 28. Demonstrators and Reference Designs

Everything under `examples/` in the CTTE repository shows the encoder doing real work: driven by a real RISC-V core, captured by a real sink, decoded by the real reference decoder (CTTD, see the decoding chapter). This chapter catalogs the demonstrators, what each one gates, and – for the ones that have run on hardware – the verdict with its exact numbers and the commit that produced them. Sources: `examples/README.md` (commit `586cad97fa`, 2026-08-17) cross-checked against the board-gate scripts and logs it summarizes, and the board runs of 2026-08-21, recorded in the internal measurement archive (external to this delivery and not independently verifiable from it).

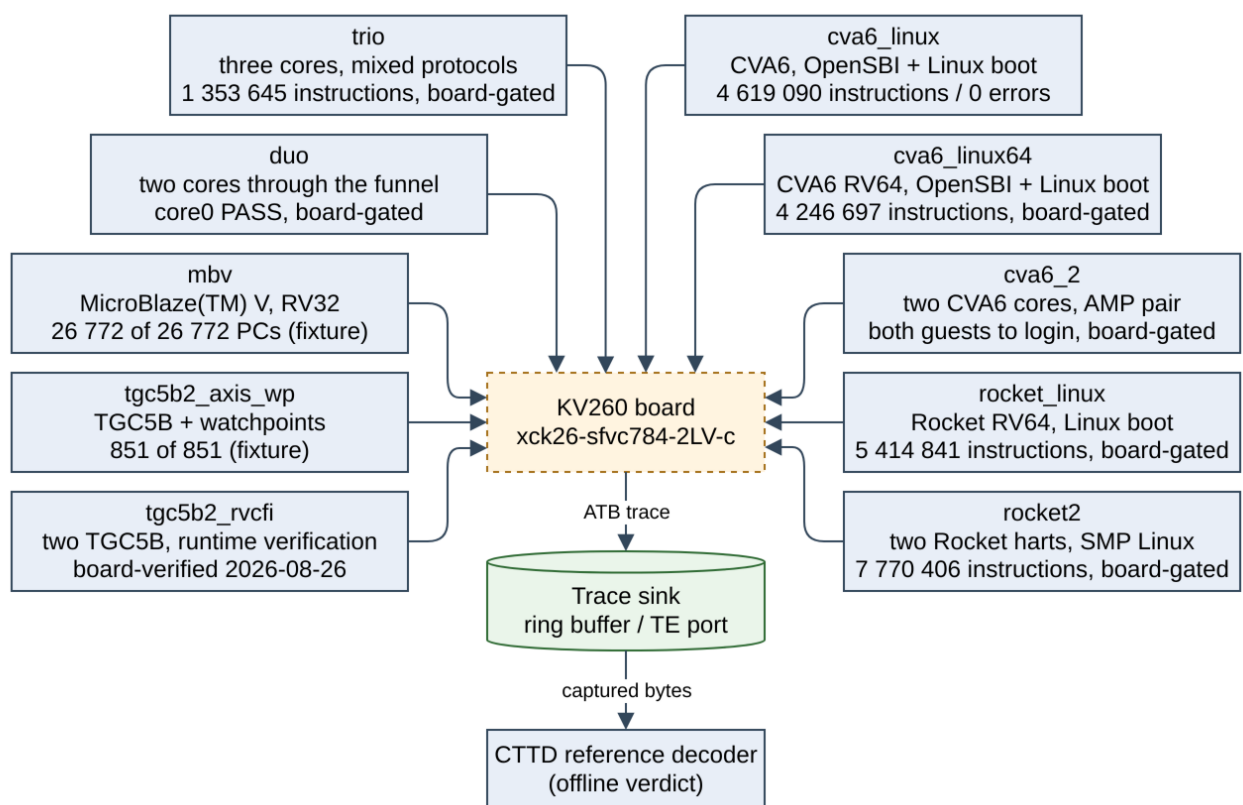


Figure 14. The ten KV260 demonstrators

Two of the ten, `mbv` and `tgc5b2_axis_wp`, carry `(fixture)` in their verdict line: their figures are byte-identical fixture reproductions from the predecessor project rather than a fresh capture from this tree – see the provenance note above. The tenth, `tgc5b2_rvcfi`, gained its first board verdicts on 2026-08-26 (see its catalog row below); it is gated by its own twin-table board gate rather than by the shared decode gate of the other nine.

The other nine share one board, one sink and one offline decoder.

The registers these flows drive – the control window every KV260 example exposes to the processing system, and the address spaces of the TGC5B example SoC – are in [Chapter 35](#),

generated from the same SystemRDL the example RTL is built from.

IMPORTANT

Provenance of the board verdicts. Nine KV260 examples are described below as board-gated. Seven of them (`duo`, `cva6_linux`, `cva6_linux64`, `rocket_linux`, `rocket2`, `cva6_2`, `trio`) were run and verified from **this repository** — `duo` and `cva6_linux` on 2026-08-18, the other five plus a re-run of `duo` on 2026-08-21 — with the run logs committed under `examples/kv260/<name>/board/` or the run driven by the board recipes under `examples/dashboard/boot/`. The other two (`mbv`, `tgc5b2_axis_wp`) carry verdicts that originate from the predecessor project (an internal archive, not part of this repository) and were reproduced byte-identically against the historical fixtures when their Bash board-gate scripts were ported into this repository — the porting commits said so explicitly (translated: "NOT YET run on the board"; "ALL still NOT run on the board (H3)").

The numbers are real board results, but their most recent confirmation **from this tree** is a fixture reproduction, not a fresh capture; the local run artifacts that would show a fresh capture (`examples/kv260/<name>/board/run/`) are gitignored and are not part of the versioned repository. This distinction is carried into each subsection below rather than glossed over.

A second distinction runs alongside it and carries the same weight: a verdict measured against a **simulation oracle** — what the program is defined to do — is not the same as a verdict measured against a **recorded reference**, which holds what the hardware did on an earlier run. Both kinds appear below and are named as such, per core.

28.1. Entry levels

Three ways to see CTTE work, by rising effort (`examples/README.md`):

Level	What it needs
Dashboard demo mode	No hardware, no FPGA tools, stdlib-only Python. <code>py examples/dashboard/server.py --demo</code> , then open http://localhost:8099 — replays committed captures from real runs.
Simulation	<code>make sim-examples</code> runs every simulable integration (needs <code>abc</code> plus a simulator; program artifacts are committed, so no RISC-V toolchain is required).
KV260 board flows	Build a bitstream, deploy, watch live trace, under <code>examples/kv260/</code> . Without Vivado: every demo ships its packaged, ready-to-load app in the repository under <code>examples/kv260/<demo>/fpga/prebuilt/</code> (bundle provenance and sha256 recorded in <code>scripts/demo.pin</code> ; verify a set with <code>sha256sum -c MANIFEST.sha256</code>).

28.2. Catalog overview

Example	Core(s)	Status (examples/README.md)
kv260/common/tgc5b/	MINRES TGC5B (RV32)	In tree — not an example but the shared library eight examples build on: the vendored core, the AXI-Lite-to-Wishbone bridge, SoC RAM/CLINT/INTC with their SystemRDL, and the <code>hello_trace</code> program. Its own SoC bench is <code>make sim-tgc5b-soc</code> .
dashboard/	any (consumes captures)	In tree — browser dashboard; demo mode replays committed captures without hardware.
kv260/mbv/	AMD MicroBlaze™ V (RV32)	In tree, board-gated 2026-08-17 (provenance note above) — 11 bare-metal programs, TRACE-bus ingress adapter.
kv260/cva6_linux/	CVA6 (RV32, config <code>cv32a6_ima_sv32_fpga</code>)	In tree; boots on the board 2026-08-17/18 — OpenSBI + Linux kernel.
kv260/cva6_linux64/	CVA6 (RV64, core <code>cv64a6_imac_sv39_ctrace</code>)	In tree, board-gated 2026-08-21 — boots to <code>buildroot login:</code> (11 004 bytes of guest console, 0 drops); a one-shot capture from core start decodes 4 246 697 instructions.
kv260/cva6_2/	CVA6 (AMP pair)	In tree, board-gated 2026-08-21 — AMP pair, both guests boot to <code>buildroot login::</code> 544 507 540 and 542 494 739 retired instructions, both PCs in the kernel idle loop, 22 389 bytes of guest console, 0 drops, both window guards at 0.
kv260/rocket_linux/, kv260/rocket2/	Rocket (RV64, 2-hart SMP)	In tree, board-gated 2026-08-21 — <code>rocket_linux</code> boots to <code>buildroot login:</code> and 5 414 841 instructions decode against the pinned <code>pcinfo</code> ; <code>rocket2</code> reports <code>M5_TWOHART_HW PASS</code> with both harts retiring (+25 087 923 / +34 975 513 in one second), reaches the login prompt, and 7 770 406 instructions decode from its DDR sink. Board recipes: <code>examples/dashboard/boot/rocket_linux64_run.sh</code> and <code>rocket2_linux_run.sh</code> .
kv260/duo/	AMD MicroBlaze™ V + MINRES TGC5B	In tree; board-gated 2026-08-18 (fresh capture, this tree) — multi-core funnel, first-ever duo board run.
kv260/trio/	AMD MicroBlaze™ V + MINRES TGC5B + CVA6	In tree — per-instance protocol mix (N-Trace next to E-Trace raw in one CTMX container). Run the E-Trace branch on its own: a mixed stream does not decode (see below). Board-gated 2026-08-21 — <code>DUO_BOARD PASS</code> over the same gate as duo (<code>--app trio_ctrace_kv260 --pl-mhz 68</code>), 1 353 645 instructions, core 0 against the simulation oracle; the CVA6 branch stays stopped in that run.
kv260/tgc5b2_axis_wp/	2x MINRES TGC5B	In tree, board-gated 2026-08-17 (provenance note above) — watchpoint/DAQ testbed.

Example	Core(s)	Status (<code>examples/README.md</code>)
<code>kv260/tgc5b2_rvcfi/</code>	2x MINRES TGC5B	Board-verified 2026-08-26 – runtime-verification testbed: shared UltraRAM memory, per-core consoles, a doorbell ACT-CAP TIP adapter that reinterprets a store to a fixed address as the CSR beat the encoder expects (see Chapter 20), and one DDR record ring per core. Its own board gate reports <code>BOARD_VERDICTS_OK</code> over a twelve-leg twin table (both record transports: MM-FIFO and DDR ring); a full-tilt run of ~1.14 million records per core finished with ring drops 0/0, where the FIFO path under the same load loses records by design and says so. The verdict tables are the closing status chapter of its <code>TUTORIAL_runtime_verification.md</code> ; its register maps are printed in Section 35.6 .

28.3. KV260 board-gated examples

28.3.1. MicroBlaze™ V (`mbv`)

A KV260 (Kria™ K26 SOM, `xck26-sfvc784-2LV-c`) demonstrator that traces a bare AMD MicroBlaze™ V RISC-V core. The Zynq UltraScale+ PS runs Linux and drives the whole SoC through a single AXI4-Lite aperture (`0xA000_0000`); the PL holds the MicroBlaze™ V core, the `rtl/adapters/amd_microblaze_v/TRACE-bus-to-tip_if` adapter, one CTTE encoder instance and a 1 MiB URAM capture ring readable from Linux (`examples/kv260/mbv/README.md`). Migrated from the predecessor project at commit `9cddf14c` (2026-08-17).

Verdict (`examples/README.md`, "board-gated 2026-08-17"; program oracle from the predecessor project, reproduced by `examples/kv260/mbv/board/mbv_board_gate.sh`, commit `c0c2476d33`):

26 772 / 26 772 PCs prefix-identical against the simulation oracle (`check_pcout_vs_retired.py --ref-prefix-ok`), Vivado 2026.1 build, WNS +2.094 ns. As stated in the provenance note above, this figure was established on the predecessor project's board run and reproduced byte-identically as a fixture when the gate script was ported here; the Bash port itself had not been executed against live hardware from this repository as of the commit that introduced it.

28.3.2. Dual TGC5B watchpoint/DAQ testbed (`tgc5b2_axis_wp`)

Two independent MINRES TGC5B RISC-V cores on one KV260, each with its own CTTE encoder instance and its own `ct_axis_wp_shim` that turns the encoder's 96-bit AXIS watchpoint-hit stream into 32-bit records for a Linux host to drain over `/dev/mem`. In parallel, both encoders' Nexus (N-Trace) ATB output is merged by `ct_L1_funnel` into the same three-sink subsystem (URAM ring + DDR4 + PIB) the `duo/trio` examples use, so a program-flow capture is available alongside the watchpoint records for cross-checking

(`examples/kv260/tgc5b2_axis_wp/README.md`). Migrated from the predecessor project at commit `9cddf14c` (2026-08-17).

Verdict (`examples/README.md`, "board-gated 2026-08-17"; `examples/kv260/tgc5b2_axis_wp/board/wp_board_gate.sh`, commit `d0cdd64fc7`): bitstream built here, Vivado 2026.1, WNS +1.575 ns;

851 / 851 watchpoint records per core on the KV260, cross-core merge monotone (`checksa` phase reproduced against fixtures, plus a 1702/0 cross-core merge check). The software pipeline gate is independent of the board and runs on this workstation:

`examples/kv260/tgc5b2_axis_wp/sw/check_consistency.py` prints `E0_ALL_PASS` over the full 1023-entry watchpoint set (`examples/kv260/tgc5b2_axis_wp/README.md`). The same provenance caveat as `mbv` applies to the 851/851 figure: reproduced from the predecessor project's fixtures, not yet a fresh capture from this repository's own gate script.

28.3.3. MicroBlaze™ V + TGC5B, funnelled (duo)

Two independent RISC-V cores traced at once: an AMD MicroBlaze™ V (same branch as `mbv`) and a MINRES TGC5B (from the shared TGC5B base), each with its own CTTE encoder instance. A `ct_L1_funnel` (`N_STREAMS=2`, `MDO_WIDTH=6`) merges both encoders' ATB (Nexus) streams into one packet-atomic stream; the decoder tells the two sources apart by the Nexus SRC field (`examples/kv260/duo/README.md`). Migrated from the predecessor project at commit `9cddf14c` (2026-08-17).

Verdict — fresh capture from this repository

(`examples/kv260/duo/board/duo_board_gate.sh`, commit `f77005f22c`, 2026-08-18; the run directory `examples/kv260/duo/board/board_run_duo_ctrace_kv260/` — including a `board.err` transcript of the live SSH/board session — stays local to the measurement host: `examples/kv260/duo/board/.gitignore` excludes run directories, so these artefacts are not part of the published tree):

- **core0 (MicroBlaze™ V): PASS, 31 240 / 31 240 PCs** — the simulation oracle is completely contained as a prefix of the decoded sequence (`check0.log`). A second core-0 verdict exists from the re-run of 2026-08-21 and reads **26 772 / 629 371**; the two are not in conflict and the denominators say why. Both gates compare an oracle prefix against a capture, and the pair is (oracle length / captured length): the 2026-08-18 run compared a 31 240-PC oracle against a capture of the same length, while the re-run compared the 26 772-PC MicroBlaze™ V oracle — the same oracle length quoted for the `mbv` example — against a 629 371-PC capture. Neither figure is a coverage ratio, and 31 240 / 31 240 in particular does not mean "the whole capture was checked".

- **core1 (TGC5B): PASS against the recorded reference.** That reference is a **reproduction** result, not an oracle result: this repository holds no TGC5B simulation reference, so the file records what the hardware did on an earlier run. Since 2026-08-21 the gate defaults to `refs/core1_hello_trace.recorded_20260821.pcs`; the 2026-08-19 recording is kept beside it and no longer reproduces. Three board runs of 2026-08-21 — `duo` at 75 MHz, `duo` at 68 MHz and `trio` at 68 MHz — diverge from the older file at the same index 855 with identical PCs while being identical to **each other** over all 720 238 decoded core-1 PCs, which rules out interrupt timing as the cause; the older file's own context shows `0x20` three times after `0x1ec`, a stopped core rather than a running program. Which of the two recordings is **correct** is open — neither has been held against what `hello_trace` is defined to do (`examples/kv260/duo/board/refs/README.md`).
- **Re-run of 2026-08-21, at duo's own 75 MHz — a different clock from the one the reference was recorded at: `DUO_BOARD PASS**, core 0 against the real oracle (26 772 of 629 371) and core 1 against the new reference (100 000 of 748 081).
- Combined decode (both cores through the same TE port): **Decoded OK, 899 799 instructions** (commit message `f77005f22c`); the run directory's `deco.log` (local-only, see above) reports 0 error messages and a very close 898 924-instruction decode from the same capture, consistent within one re-run.
- This is the first duo board run over the TE port from this repository. It was re-run on 2026-08-21 at 75 MHz and at 68 MHz; both runs decode identical PC sequences on both cores, so the PL clock does not change the instruction stream.

28.3.4. CVA6 Linux (`cva6_linux`)

A single RV32IMA CVA6 core (config `cv32a6_ima_sv32_fpga`, no C extension) booting Linux, traced with CTTE. The Zynq UltraScale+ PS drives the SoC through a single AXI4-Lite aperture (`0xA000_0000`); the PL holds the CVA6 core, an atomics/address-demux crossbar, a minimal CLINT + 8250 peripheral, the `rtl/adapters/cva6/cva6_trace_wrap/cva6 iti_to_ctte_tip` chain, one CTTE encoder instance and three trace sinks (on-chip capture ring, linear DDR sink, optional parallel PIB port, [Section 12.2](#)) (`examples/kv260/cva6_linux/README.md`). Migrated from the predecessor project at commit `9cddf14c` (2026-08-17).

Verdict — fresh capture from this repository

(commit `2664bb5c99afdcb2b00df5977a7e6db336896433`, `scripts/demo.pin` entry `cva6_linux_ctrace_kv260`, 2026-08-18): bitstream `4dec7b54`
board-gated, **OpenSBI and Linux boot captured and decoded: 4 619 090 instructions, 0 errors, over the TE port.**

This run only succeeded after a configuration defect was found and fixed the same night: the board driver's `trTeControl` constant suppressed the Vendor Config message (TCODE 58), which left the decoder without the CAPS map it needed once the periodic-sync history buffer overflowed for the first time. See the "SendConfig suppressed" pitfall in the configuration recipes chapter (commit `7d91c2f8e0`) — the fix (`--sendconfig 1`, the RDL reset) is what this verdict is measured with.

28.4. The shared TGC5B base (kv260/common/tgc5b)

This is the lowest entry step in the catalog, and the only one that needs no board at all. It is also no longer an example: since 2026-08-21 the directory holds the building blocks eight KV260 examples share, and the example it grew out of is gone (`examples/README.md`). What it still carries, and what makes it the entry point, is a complete SoC bench of its own.

The contents are a MINRES TGC5B (RV32I) core wired to `ct_encoder` with just enough SoC around it — program/data RAM, a timer, an interrupt controller — to run a small program and check the encoded trace against the reference decoder. Three nested modules, all under `examples/kv260/common/tgc5b/` (`README.md` there):

- `ct_soc_synth_wrap` — the bare SoC: TGC5B core, RAM, CLINT + INTC, the encoder fed from the core's H2E trace port through `ct_tip_adapter`.
- `ct_soc_top` — the control port: one AXI4-Lite slave for control/status registers, program load, the encoder CSRs and two capture BRAMs (16 KiB ATB, 4 KiB AXIS/DAQ).
- `ct_soc_kv260_top` — the bitstream top, Zynq PS reaching the SoC over `M_AXI_HPM0_FPD`, based at `0xA000_0000`.

The bench runs fully in simulation with `make sim-tgc5b-soc` (Verilator via `abc`), which is the cheapest way to see the encoder produce a decodable stream — no board, no Vivado. The same SoC also runs on KV260 hardware, driven entirely from Linux with `devmem` and no custom driver.

Beyond the bench, the directory supplies the parts the other examples reuse: the vendored core and its licence sidecar under `cpu/`, the AXI-Lite-to-Wishbone bridge, the SoC RAM, CLINT and interrupt controller together with the SystemRDL that describes them ([Section 35.4](#)), the AXI-Stream observation buffer `examples/kv260/common/tgc5b/rtl/ct_soc_axis_buf.sv`, and the `hello_trace` program that several board gates decode against. A change here is therefore a change to eight examples at once, which is the reason it is documented as a library and not as a demonstrator.

28.5. More KV260 examples, board-gated 2026-08-21

Four further examples were migrated with their bitstream flow in place and were run on the board from this repository on 2026-08-21. They are grouped here rather than above because they were gated in one night, and partly through the board recipes under `examples/dashboard/boot/` instead of a per-example `examples/kv260/<name>/board/` gate script. Measurements and raw data live in the internal measurement archive, external to this delivery and not independently verifiable from it.

28.5.1. CVA6 RV64 Linux (cva6_linux64)

An RV64 CVA6 (core `cv64a6_imac_sv39_ctrace`, `sv39` paging) booting Linux — not a second RV32 top.

Verdict — fresh capture from this repository (KV260 reference board, PL clock 68 MHz, app `cva6_linux64_ctrace_kv260`, recipe `examples/dashboard/boot/cva6_linux64_run.sh`): the 17 703 992-byte payload is written into the guest window and reads back byte-identically, the guest reaches **buildroot login**: with 11 004 bytes of guest console and 0 drops, the live trace stream adds 1 124 208 bytes in 2 s (`LIVE_OK`), and a one-shot capture from core start decodes **4 246 697 instructions** against a freshly built `pcinfo`. The decode stops at `0x64ba007a` — an address the listing deliberately does not cover, since user space is excluded from it — not at a decode error.

28.5.2. Rocket RV64, single and dual hart (`rocket_linux`, `rocket2`)

Two Rocket tops from the same Buildroot `br2_external` sources: one hart booting Linux, and the two-hart AMP variant behind the multi-hart funnel (`ct_L1_funnel`, `CTTD -target`).

Verdict — fresh captures from this repository (KV260 reference board, PL clock 75 MHz, recipes `examples/dashboard/boot/rocket_linux64_run.sh` and `rocket2_linux_run.sh`):

- `rocket_linux` (app `rocket_x64_ctrace_kv260`): 34 678 856-byte payload written and read back byte-identically, **buildroot login**: reached with 15 331 bytes of guest console and 0 drops, `RETIRES 973 127 483` with `WIN_ERR_CNT 0`, and **5 414 841 instructions** decoded against `dashboard/demo/rocket64.pcinfo` — the decode stops at the listing boundary `0x8020_0000`, not at an error.
- `rocket2` (app `rocket2_ctrace_kv260`): same payload, byte-identical read-back, boot arbitration clean (`_boot_lottery/coldboot_done 0/0` after loading), **M5_TWOHART_HW PASS** with hart 0 retiring +25 087 923 and hart 1 +34 975 513 instructions within one second, both PCs virtual and therefore with paging active, DDR sink 27 452 136 bytes at 0 drops and `WIN_ERR_CNT 0` with the funnel in round-robin (`0x11`), **buildroot login**: reached with 15 556 bytes of console and 0 drops, and **7 770 406 instructions** decoded from the first 4 MiB of the DDR sink against `dashboard/demo/rocket2.pcinfo`.

Both payloads are 34.7 MB, more than twice the 16 MiB limit described under the loader rule in [Section 28.7](#); without that fix neither board leg was available.

NOTE

Both Rocket examples run at **75 MHz**, unlike `cva6_linux64`, `trio` and both `cva6_2` configurations, which run at 68 MHz. The label is not a preference in either case but the timing reserve: 75 MHz costs 0.729 ns against the 71.114 MHz constraint, and `rocket_linux` has +2.114 ns of slack, `rocket2` +3.578 ns — whereas `cva6_linux64` has +0.603 ns, `trio` +0.215 ns, which is less than the step costs. Both Rocket device trees declare `clock-frequency = 75000000`, so while the runner labelled them 68 the guest computed roughly 10 % fast: nothing

crashed, but every delay, timeout and in-guest timestamp was wrong by that factor. The PL clock does not touch the **decoded** trace at all — measured, not assumed: both Rocket captures decode to exactly the same instruction counts against the pinned pcinfo at either clock, and a `duo` run at 75 MHz and one at 68 MHz produce identical PC sequences over all 720 238 decoded core-1 PCs. What the clock changes is wall-clock time inside the guest.

28.5.3. CVA6 AMP pair (`cva6_2`)

The only AMP demonstrator in the tree: two CVA6 cores on one KV260, each booting its own RV64 Linux into a private 32 MiB window, both traced through one funnel.

Verdict — fresh capture from this repository (KV260 reference board, 2026-08-21

07:22, `examples/kv260/cva6_2/board/cva6_2_run.sh` on a power-cycled board):

both guests boot to buildroot login: — 544 507 540 and 542 494 739

retired instructions, both PCs in the kernel idle loop, both boot lotteries

won, 22 389 bytes of guest console, 0 drops, both window guards at 0.

Measurements and raw data live in the internal measurement archive, external to this delivery and not independently verifiable from it.

CAUTION

That verdict is a **single** successful run, and the requirement list below is necessary but demonstrably not sufficient. Two later runs with the same register configuration did not reach the prompt — core 1 stopped at 15 and at 5 338 retires. A stopping point that moves looks like a race, not like a width; the honest reading of this example today is that the conditions below are all needed and something beyond them is still unresolved. A single won race is not a solution.

Five requirements had to be met before that verdict, none of them visible from the build output and each one sufficient on its own to prevent the boot. Four concern the guest image; the fifth is not in the image at all, but in the PS (see [Section 28.7](#)).

#	Requirement	Without it
1	FW_PAYLOAD_FDT_OFFSET=0x01F00000 when OpenSBI is built	OpenSBI with PLATFORM=generic defaults the offset to FW_JUMP_FDT_OFFSET = 0x2200000 (34 MiB), which lies 2 MiB past the end of a 32 MiB window. The guest never finds its devicetree, and the hart is parked in <code>sbi_hart_hang</code> without emitting a single console character — which makes the failure look like a dead core rather than a misplaced pointer. The working value sits above the payload and 1 MiB below the end of the window.

#	Requirement	Without it
2	A kernel without loadable modules, and the driver classes that switch pulls in turned off as well	23 of 29 MiB of the rootfs were kernel modules — 340 symbols that <code>arch/riscv/configs/defconfig</code> builds as <code>=m</code> and that nothing on this SoC ever loads, carried into the image because the rootfs is an <code>initramfs</code> inside it. <code>CONFIG_MODULES=n</code> alone makes the image larger , because <code>Kconfig</code> promotes <code>=m</code> drivers to <code>=y</code> wherever their default says so; the driver classes (PCI, DRM, FB, sound, USB, MMC, SCSI, ATA, NVMe, network and WLAN cards, media, virtio) have to go with it.
3	<code>CONFIG_FLATMEM</code> instead of <code>SPARSEMEM</code>	<code>SPARSEMEM_EXTREME</code> sizes its section table from <code>MAX_PHYSMEM_BITS</code> , not from the RAM actually present: a 16 MiB table for a 32 MiB machine. The result is the panic <code>memblocks_present: Failed to allocate 16777216 bytes in sparse_init</code> .
4	A second kernel cut: network stack, <code>io_uring</code> , journaling file systems, <code>ftrace</code> and <code>kprobes</code> off, <code>CC_OPTIMIZE_FOR_SIZE</code> on	A 17.7 MiB kernel next to a 6.4 MiB <code>initramfs</code> leaves nothing to run in 32 MiB: the guest reaches the init phase and dies after about 10 s with <code>System is deadlocked on memory</code> . After the cut the kernel is 15.61 MB instead of 24.19 MB.
5	The PS port of each core configured — <code>saxigp4</code> (AFIFM4) as well as <code>saxigp3</code> (AFIFM3)	Core 1 cannot write at all and stands at <code>RETIRES = 14</code> on <code>OpenSBI's first amoswap.w</code> , while looking perfectly alive because its read path is unaffected. The rule and the one-variable proof are in Section 28.7 .

Sizes worth quoting: Image 15 606 820 B, `rootfs.cpio` 6 438 912 B, `fw_payload.bin` 17 703 992 B = 16.88 MiB per core against a 32 MiB window.

28.5.4. Three cores with mixed protocols (**trio**)

Three cores — MicroBlaze™ V, TGC5B and CVA6 — behind one funnel, adding the per-instance protocol mix: N-Trace next to E-Trace raw framing in one CTMX container, with a demultiplex tool vendored in the example.

Verdict — fresh capture from this repository (KV260 reference board, PL clock 68 MHz): the control map for MicroBlaze™ V and TGC5B matches `duo`, so the proven `duo_board_gate.sh` was pointed at the `trio` app unchanged (`--app-dir ../trio_ctrace_kv260 --app trio_ctrace_kv260 --pl-mhz 68`). Result: **DUO_BOARD PASS** — core 0 against the real simulation oracle (26 772 PCs complete as a prefix of 624 064), core 1 against the recorded reference discussed under `duo` above, **1 353 645 instructions** decoded. The CVA6 branch stays stopped in that run, for the reason in the box below.

CAUTION

Run the E-Trace branch of `trio` on its own. A mixed stream is not decodable, and this is a property of the sinks, not a configuration mistake. The frame length of each beat lives in the ATB signal `ATBYTES` (4 bytes for N-Trace, 1 byte for E-Trace), and no sink keeps it per beat: the URAM ring does evaluate the signal, but only as a running sum in its byte counter (`examples/kv260/common/ct_soc_trace_ring.sv:97`), while

the DDR sink and the PIB never receive it at all (`ct_soc_ddr_sink.sv:64-65`, `ct_soc_pib.sv:64-65`). Every E-Trace byte therefore lands in the ring as `nn FF FF FF`, and a decode run aborts at the framing with `MSEO='10'` is not allowed, for **all** targets in the container, not only the third. Measured on the board. The `0xFF` padding itself is not what aborts the decode — `0xFF` carries `MSEO='11'` and is discarded as IDLE; the abort comes from the payload byte in the lowest lane.

How much of the container is padding depends on the mix. A pure E-Trace stream is 75.00 % padding (measured on `evidence/board_20260819/trio_cva6.bin`), the mixed stream of all three cores 44.73 % (`trio3.bin`), a pure N-Trace stream 18.75 % (`ddr_ok.bin`). Reading the 75 % figure as the mixed case underestimates the payload.

The sum that is kept is still worth reading. Because the ring's byte counter sums `ATBYTES` while its beat counter does not, `TRACE_BYTES` against `TRACE_BEATS` already shows **whether** E-Trace beats are in the container (`bytes == beats` instead of `bytes == 4 * beats`) — but not **which** ones.

It cannot be repaired after the fact: an N-Trace beat with idle padding is byte-for-byte indistinguishable from an E-Trace byte with padding. Running the E-Trace branch alone puts a pure E-Trace stream in the ring, which decodes cleanly. What a later rework would cost is recorded as item B-01 of the internal backlog, outside this delivery.

28.6. Dashboard demo mode

`examples/dashboard/` is a browser dashboard that consumes captures from any of the examples above; its demo mode (`py examples/dashboard/server.py --demo`) replays committed captures with no hardware and no FPGA tools attached, which is why it is listed as the lowest-effort entry level. It also participates in the feature matrix of `examples/README.md` — program trace, sync/quota, watchpoints, DAQ/AXIS, filters and overflow/resync recovery are all exercised through captures it ships with.

28.6.1. Protocol interlock

Since commit `be3f1c675b2` the dashboard refuses to run N-Trace and E-Trace cores at the same time, and it does so **before** the attempt rather than failing afterwards. Expect the lock; it is not a defect:

- The server rejects starting a core while a core of the other protocol is running, and likewise rejects the collective start (CONTROL bit 0 releases all cores at once). The rejection names `ATBYTES` as the reason and states the way around it.
- In the interface, the run button of a blocked core is disabled before the click and carries the reason in its tooltip.
- The mapping is data-driven: every core carries a `protocol` field (`ntrace / etrace`) in `scenarios.json`. The code does not compare names — which back end an encoder was built with is a synthesis parameter, and the register reporting it (`trTeProtocolSel`) is read-only.

The reason behind the lock is the one described for `trio` above: the sinks do not store the per-beat frame length, so a mixed stream cannot be demultiplexed afterwards.

28.7. Operating the KV260 flows

Five rules that no script enforces and that cost time when missed.

Stop the dashboard service before reprogramming the PL. It holds

`/dev/mem` mappings into the PL aperture; changing the PL underneath them hangs the AXI interconnect on the next access — every IP dead, the board no longer answers ssh, and only the power switch recovers it. Reproduced three times in a row on 2026-08-21, one attempt each, until the service was stopped first; the immediately following attempt loaded cleanly. The order is: stop the service, `xmutil unloadapp`, set the PL clock, `xmutil loadapp`. The clock belongs **between** unload and load: a frequency change under a running design is a clock glitch. Since 2026-08-21 `deploy_kv260_app.sh` stops the service itself before touching the PL, and deliberately does **not** start it again — a different design sits in the PL after a deploy than the one the dashboard was serving.

After a guest run, `prep` and `start` do not return the core to its initial state — only a fresh `loadapp`, which is a real PL reset, does. Whoever skips it measures states instead of changes.

Configure the PS port width of every port the design uses — including the second one. `psu_init` does not run for a DFX app, so the AFIFM ports keep their 128-bit reset value until a board recipe sets them. Every recipe in this repository configured **two** ports, because every other design has one core and therefore one guest window. `cva6_2` gives each core a **private** PS port: core 0 on `saxigp3` (AFIFM3), core 1 on `saxigp4` (AFIFM4, `0xFD3A_0000`). With AFIFM4 at its reset value, core 1 cannot write at all — no store reaches DDR, and the window guard reports nothing because the accesses never get that far — while its read path is unaffected, so the core

looks perfectly alive. The first operation that **waits** for its answer, OpenSBI's boot lottery `amoswap.w`, then hangs forever and the core stands at `RETIRE_S = 14`. One-variable discrimination, reproduced in both directions on a power-cycled board: AFIFM4 at its reset value gives one guest and `RETIRE_S1 = 14`; AFIFM4 at 64 bit lets both guests boot to `buildroot login:.`

And write those registers read-modify-write. `FABRIC_WIDTH` occupies bits [1:0] of `RDCTRL/WRCTRL`, whose reset value is `0x3B0`: several bits above [1:0] are marked reserved, yet are RW and set out of reset (UG1087). Writing the whole register with the width code clears them. Correct is `new = (old & ~0x3) | width`, that is `0x3B1` for 64 bit and `0x3B2` for 32 bit. All five board recipes use a read-modify-write helper for this.

A board restart empties `/tmp` — tools and payloads placed there are gone afterwards, and the next step fails silently on the missing script.

Reach the guest window with `mmap` only, never with `read()`/`write()`, and keep the accesses 32 bit wide. For `no-map` reserved regions the `/dev/mem` write path answers "Bad address"; and a glibc `memcpy` — which is what every `mmap` slice assignment compiles to — issues NEON stores that abort with SIGBUS on a Device-nGnRnE mapping, measured at exactly 16 MiB. A loader that did this halved payloads above 16 MiB **silently**, and the guest still printed a fully plausible kernel log, because OpenSBI and the head of the kernel lie within the first 16 MiB and only the tail with the `initramfs` was missing. A word loop over `ctypes` writes the same payload completely, verified by byte-identical read-back.

28.8. Tool-version note for the KV260 flows

Two Vivado generations are in play on purpose, not by drift (`examples/README.md`, "Vivado versions, honestly"):

- Core simulation and the gate battery (`make sim, make sim-examples`) pin **Vivado 2022.1** (`xsim`, via `.abc.config` at the repository root) — every core verdict in `ci/` is calibrated against this version.
- The `kv260/` bitstream flows are verified with **Vivado 2026.1** (`C:\Xilinx\2026.1\Vivado` on the reference workstation); every KV260 board gate in this chapter was built with it.

The two generations are not interchangeable, and the split is deliberate: a version bump on either side is always a measured change, never a side effect. Anyone reproducing a resource or timing figure from this manual must use the version the figure was taken with — the profile table in [Chapter 24](#) names it per row.

Chapter 29. Verification Strategy

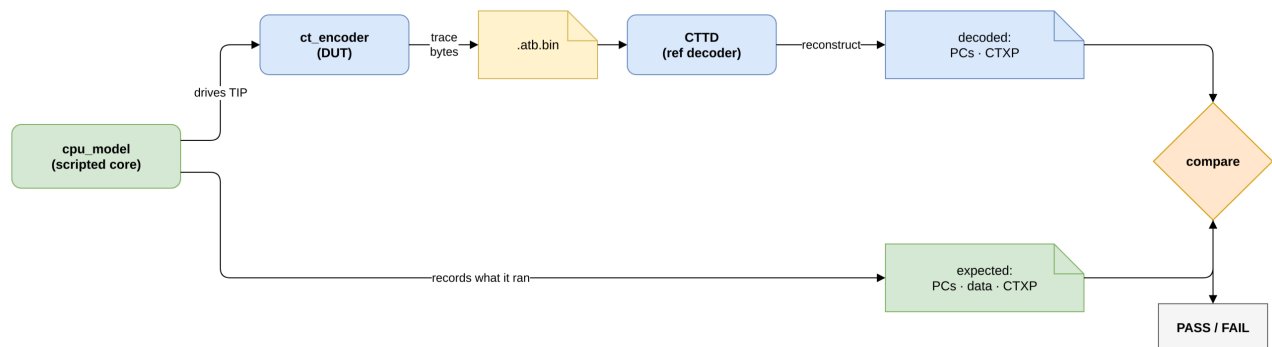


Figure 15. The verification round trip – the scripted core drives the encoder, the reference decoder reconstructs, and the two are compared

29.1. The oracle principle

CTTE’s test harness is built around a single idea: a scripted CPU model is **both the stimulus and the answer key**. `cpu_model` (`tests/lib/cpu_model.sv`) is a task-based scripted core – `run`, `branch_taken`, `call`, `ret`, `load`, `store`, `csr_write`, `interrupt`, ... – that drives the encoder’s TIP interface exactly as a real hart would, and simultaneously logs a sideband event stream of what it just did. No binary image and no instruction-set simulator are needed; the tasks themselves **are** the program. The shared `ct_env` harness (`tests/lib/ct_env.sv`) wires the model to the DUT and to the trace sinks.

The encoder turns the TIP stream into an N-Trace or E-Trace byte stream on the ATB. An independent reference decoder, **CTTD (CEDARtools.TraceDecoder)** – a second, independent implementation of the wire format, not a CTTE component – turns the bytes back into a trace. `scripts/decode_and_check.sh` runs CTTD against the captured `.atb.bin` and diffs the result against the `cpu_model`-logged expected files. If the encoder mangles the trace anywhere in the pipeline, the reconstruction will not match the log, and the check fails. CTTD reconstructs two views from the same bytes: the PC stream (instruction trace) and the **CTXP** export – control-flow, memory-access and instrumentation records with data values – and `cpu_model` emits matching expected files for both.

CTTD is derived from the RISC-V Nexus Trace TG reference decoder NexRv (ISC-licensed) and extended by Accemic (E-Trace `-decode`, DAQ, multi-target `-target`, CTXP export). Since 2026-08-17 it is no longer committed to the encoder tree: `scripts/cttd.pin` names the release and the per-platform sha256, and `scripts/fetch_cttd.py` downloads it from the CTTD repository that the pin names in its `base_url` (assets on the `dist` branch) into the gitignored `bin/`, with a checksum mismatch treated as a hard error.

`decode_and_check.sh` supports the check flags `--pc` (PC stream), `--ctxp` (memory accesses with data values plus DAQ instrumentation), `--data` (loads/stores, address and size only), `--sync N` (at least N sync messages), `--disabled` (trace-off correlation present), `--overflow` (an Error/QueueOverrun message must be present) and `--soft` (report divergence without failing the

gate). It also fixes one decoder policy for the whole regression suite: decoding runs with `-csstrict`, the strict reading of the implicit-return address mirror. A scripted `cpu_model` never switches task context, so a mirror mismatch there is a genuine defect and must be fatal; a test that legitimately switches context (real-core simulation legs, see [Section 29.9](#)) sets `NEXRV_CS_LENIENT=1` to get the lenient reading instead.

29.2. What makes a check a gate

A passing check proves nothing by itself; what makes it evidence is that it **would have failed** for the defect it claims to catch. A gate that cannot turn red for its own defect class is a green light with no lamp behind it. Every verification mechanism in CTTE's suite therefore carries one of four falsifiability shapes:

Historical counter-proof

the check is re-run against a git worktree of the pre-fix commit and must fail there. Used for defect classes that really occurred — the class-8 and class-9 formal targets are red at a named BMC step on their pre-fix trees (see [Section 29.8](#)).

Mutation counter-proof

where no historical defect commit exists, the check is run against a deliberately broken copy of the design — a wrong end-of-message code, a suppressed EOM, a context cast narrowed back to its old width, a filter made ineffective, one instruction shifted by two bytes in a reference listing. `run_red.sh` for formal targets; `MUTATE=1` or an explicit `-ExpectFail` leg for the script gates.

Vacuity leg

an implication whose antecedent never occurs is green for free and proves nothing. Assertions of that shape need a scenario that demonstrably exercises the antecedent — invariant I10 ([Section 29.7](#)) has one.

Reachability

a check nobody runs is not a check. This class has been found three times in this project's own history and is recorded rather than quietly fixed: a bound checker whose counter was too narrow to **represent** the violation it guards ([Section 29.7](#)); assertion failures that XSIM wrote only into a log the CLI scripts silenced (now scanned by `scripts/cli_status_test.sh`); and CTTD's own five test runners, which were wired into no CI pipeline at all until 2026-08 ([Section 29.12](#)).

The question to ask of any gate in this chapter is therefore not "does it pass" but "what was done to make it fail".

29.3. Components

Component	Role
<code>cpu_model</code> (<code>tests/lib/cpu_model.sv</code>)	Task-based scripted core; both the stimulus and the reference for every test in the suite. See the oracle principle above.
CTTD (<code>scripts/fetch_cttd.py</code> → <code>bin/cttd-<platform></code>)	Reference decoder for the N-Trace wire format; a second, independent implementation, pinned by hash (see Section 29.4).
CTTD's own regression runners	Five runners shipped by the decoder repository (<code>run_etrace_ctxp</code> , <code>run_callstack</code> , <code>run_ctxswitch</code> , <code>run_reserved_tcode</code> , <code>run_addr64</code>), collected behind that repository's own runner script – not a file of the encoder tree, whose test entry points are the <code>make sim*</code> targets and <code>tests/README.md</code> . CI stage 1 runs them when a decoder source tree is available (<code>NEXRV_SRC</code>); a plain clone of the encoder repository reports <code>SKIP</code> for this step – the decoder source is <code>Accemic-internal</code> .
<code>decode_and_check.sh</code>	Runs CTTD on the captured ATB dump and compares against the <code>cpu_model</code> -logged expected files; see the flag list above.

29.4. The reference decoder is a gate input, not a given

Every PASS in this suite is a statement about **two** independent implementations: the encoder wrote the bytes and the decoder read them back. That makes the decoder binary a measuring instrument, which has to be verified before its readings are trusted. A re-pin of CTTD is therefore treated like a wire-format change: it is accepted only with evidence recorded in the `scripts/cttd.pin` commit and the CTTD repository's own changelog, never in an unrelated commit. The evidence a re-pin must carry is the defect it removes (reproduced on the **old** binary with the same input, not argued from source reading), byte neutrality on everything that already passed (old and new binary run over the frozen corpora, comparing the full log and exit code, not just the PC list), and its measured price (a decoder change that removes an abort also removes whatever that abort used to catch).

Two re-pins in 2026-08 illustrate the discipline:

- **64-bit addresses.** The prior decoder could not decode a 64-bit address at all – on an Sv39 kernel window it truncated to the low 32 bits and exited 9. The replacement derives its output width from CAPS bit 23 (`ADDR64`) in the configuration message rather than a CLI switch, and was shown byte-identical to the binary it replaced across 525 of 525 32-bit artefacts. Its immediate successor then fixed a follow-on defect (a process-global width flag bleeding across funnel sources): guard `run_addr64` went from `PASS=16/FAIL=6` to `PASS=22/FAIL=0`, with 440 of 440 artefacts unchanged at 32-bit.
- **Context switches are not decoder errors.** The implicit-return address mirror predicts a `ret` target; under a preemptive kernel, a task switch makes the prediction wrong while the stream is right (the target is carried explicitly). The old pin treated that as fatal, so no multitasking kernel was decodable past its first context switch. The new pin reports a visible re-anchor and keeps decoding – on the same recorded board stream, decode moved from 34,537 instructions plus an abort to 818,663 instructions with zero error messages. The price was measured, not assumed: a one-byte mutation sweep over 40 cases is caught 25 times by the new binary versus 27 by the old one (the two losses are `UADDR` flips at returns the strict mirror caught by accident); the `-csstrict` default (above) restores the old detection strength for the scripted suite while

leaving real-kernel streams decodable.

CAUTION

the general rule behind this section applies to every tool in a verification chain: **a gate that can only fail together with its own tool judges nothing**. This is why the 64-bit emission gate (`instruction/35_addr64`, [Chapter 30](#)) reads its verdict off the raw ATB bytes rather than asking the decoder, and why the decoder itself is cross-checked against independent artefacts (oracles, disassembly listings, its own fixtures) rather than solely against the encoder it is meant to be checking.

29.5. Test scenarios and round-trip gates

Scenarios live under `tests/`; `make sim` runs the full suite and prints a per-category PASS/FAIL summary, `make sim-<name>` runs one. The full catalog, by directory and gate, is [the test catalog chapter](#); this section states the checking principle each category shares.

Category	Location	Round-trip principle
Instruction trace	<code>tests/instruction/</code> (39 numbered scenarios)	CTTD <code>--pc</code> reconstruction of the PC stream against the <code>cpu_model</code> log; strict return-address mirror (<code>-csstrict</code>).
Data trace	<code>tests/data/</code> (6 scenarios)	<code>--ctxp/--data</code> reconstruction of memory accesses (address, size, and where applicable value) against the logged access stream.
Hardware-supported instrumentation (HSI)	<code>tests/hsi/</code> (1 scenario, all ACT-CAP commands)	AXIS beat self-check plus CTEX DAQ record reconstruction.
Overflow / robustness	<code>tests/overflow/</code> (15 scenarios)	Full overrun lifecycle (inject or natural overflow, resync, soft-reset recovery); <code>--overflow</code> is a hard gate. See Section 29.6 .
Combined	<code>tests/combined/</code> (1 scenario)	Instruction + data + ACT-CAP sync exercised together, to catch cross-feature interaction the per-feature legs miss in isolation.

Known-failing regression gates can be listed in `SIM_XFAIL` in the `Makefile`: they run and report `XFAIL` without failing `make sim`, and a surprise pass shows as `XPASS` – a signal to promote the test once the underlying fix lands.

The compression features additionally carry byte-identity legs (`cli_{bp,ir,jtc,rh,rb}_test.sh`): the same stimulus run with the feature OFF and ON from one build. The OFF prefix must equal the reference byte-for-byte; the ON stream must decode to the **same** PC stream at a strictly smaller byte count. These legs pin the wire format itself – any incidental byte change from an unrelated fix turns them red.

29.6. Robustness gates (state across discontinuities)

Every cross-message compression state (JTC mirror, branch predictor, implicit-return stack, ICNT/HIST accounting) has to re-converge at every stream discontinuity: a FIFO-overflow recovery, a trace pause, a session stop. Each defect class first found on real hardware (KV260 robustness campaigns) carries a standing regression gate with a documented red counter-proof; the fifteen `tests/overflow/` scenarios plus three `tests/instruction/` gates that guard this state family are cataloged individually in [Chapter 30](#). Two properties of the family are worth stating here because they explain **why** this many scenarios exist for what looks like one contract: several defects were found only by combining two features that were each clean in isolation (e.g. branch prediction alone was green, periodic sync alone was green, the combination was `not-overflow/06_suite_isync`), and several were found only by forcing an anchor onto a **specific** retire kind – a control-flow retire, a not-taken branch, a trap marker – because the injector's `inject_hold` logic covered some retire kinds and not others.

Honesty note (2026-08-07): the 2026-07 E-Trace verification claim "mid-trace periodic resync solved" was too broad. The P10 seed-driven soak campaign found three pre-existing defects the original directed leg (`instruction/23_etrace_resync`) never crossed: a resync anchoring on a predicted-return retire, a target report whose address is also linearly reachable, and an overflow anchor landing on a branch. These are closed by the `resyncir` gate (`instruction/34_etrace_resync_ir`) and an overflow-anchor deferral in `ct_L2_te_inst_gen.sv`. One class remains open by wire-format necessity – see [Chapter 33](#).

29.7. Standing in-RTL invariants (SVA)

The most expensive lessons of the hardware campaigns are encoded as concurrent assertions inside the RTL itself (simulation-only, `pragma translate_off`), so they run in **every** simulation of every consumer and fire at the first sim rather than on a board:

ID	Invariant
I1 / I2	No ordinary control-flow eTIP while a pause re-anchor is outstanding, or while instruction tracing is paused.
I3	The implicit-return stack is empty when the trace-off correlation is emitted.
I4	A <code>next_iaddr</code> capture fires only when a control-flow event owes one (pairing conservation).
I5	A gap re-anchor sync on a call/return retire leaves the return stack untouched (side-effect-free anchors).
I6	A correlation message implies a cleared JTC mirror and predictor count.
I7	A FIFO-overflow recovery sync carries <code>ICNT=0</code> (pure re-anchor).
I8	<code>trTeControl.Empty</code> implies an empty ATB output queue per backend tail.
I9	Device ID precedes the config message precedes the first synchronizing message, on the wire, when both <code>CT_EN_DEVICE_ID</code> and <code>CT_EN_CONFIG_MSG</code> are enabled.
I10	A watchpoint slot never coexists with a DAQ slot in one beat – the exclusion that lets <code>CT_EN_WATCHPOINT_MSG</code> share the DAQ slot instead of widening the eTIP slot count.

ID	Invariant
I11	No ordinary control-flow eTIP inside a filter-suppressed gap (<code>CT_EN_FILTER_SYNC</code> ; vacuous while no filter is programmed, i.e. in every unfiltered run).
I12	Every emitted instruction count fits the N-Trace I-CNT field width (8 bit, or 16 with <code>InstEnWideIcnt</code>)—the only invariant in this set that guards conformance rather than internal consistency: an over-cap value costs one extra wire byte, decodes correctly, and keeps every other gate green while the stream has left the standard.

Because XSIM writes assertion failures only into the per-leg log and the CLI scripts silence the simulator's own output, `scripts/cli_status_test.sh` scans those logs and fails the gate on any `Error: /Fatal: line`—an assertion nobody reads is not a gate (the reachability shape above). A bound check is only as good as its counter's ability to **represent** a violation: `msg_id_next` (guarded by `a_p4_slot_bound`, feeding I10's slot-sharing argument) used to be sized for eight allocation sites when nine could be simultaneously enabled, which would have wrapped silently past the bound it exists to catch. The width now follows `ct_pkg: :ETIP_SLOT_SITES`, computed from the source rather than hand-set, and `formal/composer_slots/run.sh` counts the real allocation sites before every proof run so a newly added arm fails the gate rather than the field.

29.8. Formal property gates (SymbiYosys)

The three encoder defect classes found in the 2026-08 board campaigns (trap anchor, resume-anchor race, flush clobber) shared one shape: local cycle collisions in a tiny state space, phase-dependent and found dynamically only by lucky draws—exactly what formal methods find in seconds. The emission core therefore carries property gates on the license-free open-source route (sv2v + Yosys/SymbiYosys, `smtbmc`):

Target	What is proved	Result
<code>ovf_injector</code> (parallel and <code>SEQ_INJECT</code> variants)	Element conservation, episode debt, <code>inject_done/inject_hold</code> semantics—full functional correspondence against an independent mirror model.	Unbounded (k-induction) + cover
<code>ct_L23_preproc_sync</code>	No periodic sync while a trace-enable re-anchor is outstanding (class-9 fix 335393b6); one-shot arming/drop semantics; sync anchors only on qualified retires; the trace-output quota protocol including the real CDC pairs; the explicit sync-request pacing protocol (<code>ct_sync_req_pacer</code>) with no lost and no double request.	Mixed unbounded / bounded + cover (see caveat below)
<code>ct_L2_msg_gen</code> (abstracted eTIP payload)	A consume beat never coincides with a flush-marker emission (class-8 fix 8f8e85aa); backpressure holds <code>TraceMsg</code> bit-stable; the latched flush debt is never lost.	BMC depth 50 + cover
MDO/MSEO output stage	Reserved dual-pin code never emitted; the IEEE-ISTO-5001-2012 Table 5-1 encoding table; message conservation (one end-of-message chunk per accepted message); CDC-seam non-overflow; beat alignment.	BMC depth 40 + cover + bounded liveness (depth 120)
<code>ct_L23_preproc_composer_etip</code> eTIP slot bound	<code>msg_id_next</code> = <code>ETIP_PAR_MSG</code> at the allocation site, over a fully free environment, for both <code>SPLIT_DATA_ACCESS</code> settings.	Unbounded (k-induction)

Every regression property carries a two-sided red counter-proof: it passes on the fixed RTL and fails on a git worktree of the pre-fix commit. `ct_L23_preproc_sync`'s CDC-closing properties

are proved at **equal** clocks (this environment's single-clock assumption) – what is proved is the pacing protocol and the toggle synchronizers, not the general behaviour of a toggle synchronizer between two independent clocks, which remains a CDC-methodology argument rather than a formal one. All targets are proved at ``ct_pkg::CT_XLEN = 32` with **hand-retyped address ports**; only `nexus_formatter` additionally runs as `nexus_formatter_xlen64` against a width-flipped source copy. Until 2026-08 every formal gate had only ever run at 32 bit – an audit finding, not a plan.

One documented pitfall for one-directional properties (`formal/README.md`, condition A-2): a property whose model appears only on the **conclusion** side of an implication gets weaker, never stronger, as the model's own arming widens, and a DUT that stops arming altogether satisfies it vacuously. Measured on `ct_L23_preproc_sync`: removing the TCI trigger action from the RTL entirely still reported "successful proof by k-induction" for that property. The completeness obligation on the **other** side (a bounded next-cycle requirement) is what actually catches a DUT that stopped arming, and it does go red on the same mutation.

29.9. Hardware-in-the-loop and SoC integration

Everything above runs against a scripted `cpu_model`, which cannot produce a real hart's retirement timing, a real memory system's backpressure, or a real operating system's behaviour. Three ingress adapters exist outside the encoder core – AMD MicroBlaze™ V's TRACE bus, the CVA6/cv64a6 instruction-trace interface fed from its RVFI probes, and the Rocket TraceCoreInterface – and each is characterized before it is trusted: a truth table per event class, measured against the core's own retire stream, is the entry gate for a new adapter.

Robustness campaign (MicroBlaze V, KV260). A directed 95-run matrix over overflow regimes (including ATB backpressure profiles), feature sets, workload mixes, sync modes, trace-window/stop/flush edges, IRQ, BTM, one-factor bisection legs and a determinism pair, judged against an RV32 reference model on eight verdict axes (decode integrity, loss accounting, post-recovery losslessness, counter consistency, liveness, determinism, window semantics, flush completeness). Result as of 2026-08-01 (campaign `rob_allgreen`): **95 PASS / 0 FAIL / 0 invalid**. A full-SoC XSIM twin of the same workload (real MicroBlaze V plus adapter at the board's 1:1 clock profile, 343,629 retires, 24 natural overflow recoveries) decodes transitions-exact as the simulation-side positive control. A seed-driven soak campaign on top of this directed matrix is what found the three 2026-08 encoder defect classes now held by the formal gates above, plus the four pre-existing E-Trace classes named in the honesty note ([Section 29.6](#)).

64-bit harts, cv64a6 and Rocket (2026-08-08). `ct_pkg::CT_XLEN = 64` is established beyond a testbench: both cores were brought up in the KV260 PL beside the encoder and boot Linux (Buildroot + OpenSBI) to a login prompt. Simulation leg (cv64a6): the full chain core → RVFI → ITI → shim → encoder → ATB → CTTD → scoreboard reconstructs 10,167 of 10,167 instructions, delta 0, in the low address window and again with Sv39 kernel addresses. Board leg: a recorded window of the running kernel decodes to 34,453 instructions (32,413 with addresses $\geq 2^{32}$), zero error messages, verified by an independent flow check that walks the decoded PC sequence against the disassembly rather than trusting the decoder – PASS, with five deviations all landing on the same

address, OpenSBI's M-mode trap vector (the physical target of a virtually-addressed kernel). With the current decoder pin the same stream decodes 818,663 instructions uncompressed and 883,577 with the compression suite armed, both zero-error and PASS on the independent flow check. Two open items at time of writing: parallel retirement is not exercised (both harts run a single commit port), and an E-Trace instance with `CT_XLEN = 64` is rejected at elaboration rather than truncating silently — see [Chapter 33](#).

Tracing one process, end to end (`trTeComp[] . PInput = 1`, context filter). Verified as a chain rather than a register: two programs with different ASIDs run on the cv64a6 model. Unfiltered, both appear (204 and 168 instructions in their ranges) and Ownership messages carry both ASIDs. Filtered to process A: 204 of 204 for A, 0 of 168 for B, only A's ASID on the wire — and the substance of the feature is that the filter suppresses **at the source**, not that it marks: two counter-proofs confirm this is not vacuous (an ineffective filter returns the full 437-PC stream; compiling out the filter anchoring produces zero decodable instructions, the pre-fix behaviour the anchoring exists to prevent). This leg is simulation with real core RTL; filtering real Linux processes on a board, where the ASID is written by the kernel rather than the test, is a follow-on package and not claimed here.

29.10. Timing and resource verification

Out-of-context synthesis timing is gated at two budgets per build profile: 13.333 ns / 75 MHz (the KV260 reference design's constrained clock — the pass criterion, "all user specified timing constraints are met" with zero failing endpoints) and 5.0 ns / 200 MHz (reported headroom, not required).

`scripts/p10_timing_matrix.sh` drives all five profiles and files every report under `verification/evidence/P10/<profile>/. CAUTION: the constrained clock is not automatically the clock the board delivers — loading an application overlay that names only a firmware file can leave the PS clock register at its boot default (measured 2026-08-08: 100 MHz instead of the constrained 75 MHz), which is a property of the loading mechanism, not of the encoder, but has to be set explicitly by an integration. The current per-profile figures, tool version and artefact paths belong to the footprint-profiles chapter (D6), not here, to avoid a second, drifting copy of the same numbers.`

29.11. Reference-stream families and supersession

Byte-identity gates compare emitted streams against a pinned family of reference md5 hashes (the `REF_FINAL` family), minted by `scripts/r2_final_mint.sh`. A family is re-minted only when the byte delta against the previous family is **provably** explained: the delta must be localized to a specific message-dump difference, a build with the causing feature compiled back out must reproduce the previous family md5-exactly, and the superseded hashes stay in the mint log as documentation rather than being discarded. Wire-neutral RTL work therefore never touches the family; a deliberate wire evolution (for example, a CAPS-map extension for a new config-message bit) produces exactly one documented supersession, with the byte-neutrality cross-check as the proof. A family is **selected** by matching the build's own CAPS word, not inherited positionally — `scripts/ref_family.py select` computes the word a build's `CT_EN_*` switches produce and picks the archived family with that word (newest mint stamp wins among same-word candidates); a build whose CAPS word matches no archived family is a hard error rather than a silent

mismatch.

29.12. CI stages

Since 2026-08-21 the public continuous integration runs as GitHub Actions on hosted runners, in two workflows. `.github/workflows/ci.yml` carries five jobs — **Lint + static guards** (`make lint`: the drift guards plus Verible over the RTL and tests), **Publication guards** (`make check-publication`: the six checks that decide whether the tree is fit to be published), **Dashboard self-tests**, **REUSE / SPDX compliance**, and **Simulation (Verilator)** (`make sim` and `make sim-examples`, with the decode verdict when the pinned decoder is reachable).

`.github/workflows/formal.yml` runs the SymbiYosys property gates through `scripts/run_formal.sh` on a licence-free toolchain: a fast subset per pull request, the full set nightly (`RED=1` additionally runs the red counter-proofs against the pre-fix worktrees). Stage nomenclature and the lab-runner stage of earlier revisions are gone with the `ci/` directory they lived in.

What a hosted runner **cannot** do is stated in the repository rather than left to the reader (`CONTRIBUTING.md`): the gate battery `scripts/run_gates.sh` — **51 gates** named in `scripts/gates.list` — needs Vivado™ XSIM, hardware verdicts need a real KV260 board, and the decode verdict needs the pinned CTTD reference decoder. `make sim` is the simulator-agnostic subset that CI does run.

CAUTION

The 51 gates of that battery and the 62 system-level testbenches of [Chapter 30](#) count different things and must not be added or compared. The testbench count is the number of scenario directories under `tests/`; the gate count is the number of **runs** the battery drives, several of which exercise one scenario in more than one configuration, and some of which are not scenario runs at all.

Stage 1 carries eight host guards, six of them drift guards over the register map, the profiles and the harness. The other two guard the evidence itself, because the same two findings recurred in every package audit of this campaign: `scripts/check_doc_evidence.py` requires every documentation table cell quoting LUTs, flip-flops or block RAM to name an existing `.rpt` file under `verification/evidence/` and matches the cell value against that report (twice, a cost table pointed into `gitignored bld/`, which the next run silently overwrote with a different configuration); `scripts/check_evidence_stamp.py` requires every verdict under `verification/evidence/` to carry a `# HEAD` stamp naming a commit in the branch's history, or an explicit `# superseded-by` pointer — five logs were once stamped with a rebased-away commit and nothing had shown it.

A gate reports `SKIP` when an input it needs is not present in the repository and `TOOL` when the toolchain or build tree could not be established; neither reads as a property failure.

Chapter 30. Test Catalog

This chapter catalogs every high-level testbench under `tests/` as of this writing: **39** instruction-trace scenarios, **6** data-trace scenarios, **1** hardware-supported-instrumentation scenario, **15** overflow/robustness scenarios and **1** combined end-to-end scenario – **62** system-level testbenches in total, plus the shared verification infrastructure under `tests/lib/` (`cpu_model`, `ct_env`, decoder wrappers) that every one of them uses. This supersedes any "Tests 06-20" framing: the suite has grown well past that range, and several of the newest scenarios (Sections [Section 30.4](#), [Section 30.6](#)) exist specifically to close gaps the 06-20 generation did not cover.

Every scenario follows the pattern in [Chapter 29](#): a `cpu_model`-scripted stimulus is both driven into the encoder and logged as the oracle; a CLI wrapper script (`scripts/cli_<name>_test.sh`, or `scripts/decode_and_check.sh` directly for the plainer scenarios) builds the testbench, runs it, decodes the captured ATB stream with CTTD and diffs the result against the oracle. Per-module unit testbenches (`rtl/<module>/test/`) are out of scope for this chapter; per `tests/README.md` they are not part of the system/integration suite cataloged here.

30.1. Basic control-flow and exception semantics

Test	Stimulus	Verification principle and gate
<code>instruction/01_basic</code>	Linear execution, taken/not-taken conditional branches, a call/return pair.	Smoke gate for the instruction-trace path. <code>make sim-basic</code> ; CTTD <code>--pc</code> round-trip plus the trace-off correlation message (TCODE 33).
<code>instruction/02_interrupts</code>	Single interrupt with <code>mret</code> , exception with <code>mret</code> , nested interrupts, back-to-back interrupts, exception followed by an interrupt.	<code>make sim-interrupts</code> ; PC round-trip across every interrupt/exception shape named in <code>tests/README.md</code> .
<code>instruction/03_stress</code>	A scheduler-style dispatch loop that builds and overflows branch HIST, mirroring a pattern observed on real customer workloads that made the stream undecodable.	Regression gate for a fixed encoder bug: on a HIST overflow the encoder must not zero its instruction-count accumulator, because inferable/linear instructions do not all carry a HIST bit. <code>make sim-stress</code> ; PC round-trip.
<code>instruction/04_periodic_sync</code>	Periodic sync firing while the encoder's HIST register still holds pending branch-direction bits.	Regression gate for <code>cf_sync_hist_flush_hold</code> (<code>rtl/ct_l2_msg_gen.sv</code>): a periodic <code>ProgTraceSync</code> (TCODE 9, no HIST field) must be preceded by a HIST pre-flush <code>ResourceFull</code> , or the decoder desyncs from <code>FADDR</code> . <code>make sim-periodic-sync</code> ; PC round-trip.
<code>instruction/05_exceptions</code>	Illegal-instruction trap where the faulting instruction never retires (<code>iretire=0</code>), a co-reported exception for contrast, and the same never-retired trap right after a taken branch.	<code>make sim-exceptions</code> ; PC round-trip, exercising the <code>EXCEPTION_TRAP</code> branch the <code>02_interrupts</code> group (<code>iretire=1</code>) does not reach.

30.2. Compression suite

Test	Stimulus	Verification principle and gate
instruction/06_implicit_return	A long loop of call/return pairs (plus branches and nested calls) with periodic instruction sync, so a large number of predicted returns are folded.	With <code>InstEnImplicitReturn</code> set, a predicted return is folded like an inferable branch; the decoder recovers the target from its own call stack. Byte-identity leg <code>scripts/cli_ir_test.sh</code> : OFF prefix equals reference, ON decodes to the same PCs at a strictly smaller byte count.
instruction/07_repeated_history	A tight all-taken counting loop (uniform HIST patterns) plus segments designed to miss the exact-match detector.	With <code>InstEnRepeatedHistory</code> set, an exact repeat of a full HIST pattern is counted instead of re-sent (<code>ResourceFull</code> <code>RCODE=2</code>). Byte-identity leg <code>scripts/cli_rh_test.sh</code> .
instruction/08_repeat_branch	A tight uninferable-jump dispatch loop producing byte-identical <code>IndirectBranchHist</code> messages lap after lap, ending on a coda that forces the drain path.	With <code>InstEnRepeatBranch</code> set, an IBH identical to the last emitted one is suppressed and counted, closed by <code>RepeatBranch</code> (TCODE 30). Byte-identity leg <code>scripts/cli_rb_test.sh</code> .
instruction/09_jtc	An indirect dispatch cycle over three far-apart code blocks – the case where a differential UADDR is expensive and a cache-index hit wins.	With <code>InstEnJumpTargetCache</code> set, a cache-hit plain IBH goes out as <code>VendorJTC</code> (TCODE 57, 6-bit index) instead of a differential address; first lap misses (installs), later laps hit. Byte-identity leg <code>scripts/cli_jtc_test.sh</code> .
instruction/10_branch_predict	A trained counting loop (best case for the predictor) plus a cold-start/mispredict segment.	With <code>InstEnBranchPrediction</code> set, encoder and decoder run bit-identical $2^9 \times 2$ -bit predictors; only a mispredict emits <code>VendorBP</code> (TCODE 56). Byte-identity leg <code>scripts/cli_bp_test.sh</code> .
instruction/13_ibhs	A short periodic-sync window so syncs repeatedly fire with pending HIST bits.	With <code>CT_EN_IBHS</code> set, <code>IndirectBranchHistorySync</code> (TCODE 29) carries the pending history directly instead of a pre-flush pair. <code>scripts/cli_ibhs_test.sh</code> ; both OFF and ON legs PC-lossless.
instruction/14_repeat_instr	A single-instruction spin loop (60 iterations) inside a short periodic-sync window, so syncs repeatedly land on the loop branch.	With <code>InstEnRepeatInstr</code> set (TCODE 31/32, reserved in ratified N-Trace 1.0, hence reset 0), spin-loop iterations after the first are counted rather than resent. <code>scripts/cli_rpti_test.sh</code> .

30.3. Robustness, status and register features

Test	Stimulus	Verification principle and gate
instruction/11_robustness	One rich control-flow workload run under four CSR configurations from the same binary: compression off, HIST-family stack, BP-family stack, and a deliberate misprogramming (+ROBUST_ALL, all six compression bits) that must be byte-identical to the BP-family leg because BP hardware-excludes RH/RB.	scripts/cli_robust_test.sh; every leg PC-lossless, plus a negative gate (a BP stream decoded without -bp must fail hard).
instruction/12_debug_power_events	Debug entry with pending branch history, debug exit, an EVTI pulse while tracing.	Regression gate for CT_EN_DEBUG_EVENTS/CT_EN_POWER_EVENTS/CT_EN_EVTI: Correlation EVCODE 0 on debug entry (residual ICNT + pending HIST), SYNC=3 (not 5) on the very first post-debug retire, SYNC=0 upgrade on an EVTI pulse. scripts/cli_dbg_test.sh.
instruction/15_trig_seq_own	One workload, four CSR legs: reset defaults, +InstTrigEnable, +InstSeqSyncEnable, +Context.	Trigger-Sync (SYNC=6), Seq-Sync (SYNC=4, a documented Accemic extension – N-Trace binds SYNC=4 to BTM) and Ownership (TCODE 2) against one identical PC reference per leg. scripts/cli_tso_test.sh. Also the gate for the TIP context-width build knob (ct_pkg: :CT_CONTEXT_WIDTH), rebuilt in a detached worktree per width and checked for the predicted PROCESS value (scripts/cli_ctxwidth_test.sh); red counter-proof narrows the composer's context cast back to [1:0].
instruction/16_config_msg	One branch-rich workload, five CSR legs: SendConfig=CFG_NONE/CFG_ONCE/CFG_ON_SYNC, plus a BP leg decoded via CTTD autoconfig and its negative control.	Vendor config message (TCODE 58): none/exactly-one-as-message-0/one-per-sync placement; and "flagless decode == flagged decode" via ENAB.5 autoconfig. scripts/cli_cfg_test.sh, 12/12 gates.
instruction/17_btm	One branch-rich workload, two CSR legs: HTM (InstMode=6, reset default) and BTM (InstMode=3).	Proves both N-Trace 1.0 instruction-trace modes (Table 8, "3 or 6 settable") against the same cpu_model reference. scripts/cli_btm_test.sh.
instruction/18_fifo_histogram	Dense periodic syncs, Ownership and config messages under a serialized 1-slot/cycle drain to pressure the eTIP CVS FIFO, then a quiescent read of the 16 histogram bins.	CT_FIFO_HIST_BINS diagnostic CSRs (trTeTipFifoHist0..7): at least one nonzero bin, HistClear zeroes all bins, a second run re-arms. scripts/cli_fifohist_test.sh; stream PC-losslessness checked as usual.
instruction/19_feature_matrix	A sequence of short configuration phases over a linear workload: comparator P-function sweep, input mux (CONTEXT/TVAL/DADDR), S-function sweep plus MODE1 latch, filter chain, performance-counter thresholds, DF range memory, timestamp prescale and core source, SRC field, external sync sources, and (phase 10) ACT-ST watchpoint hits.	The comparator/filter/perfcnt/timestamp/sync configuration CSRs are Enable-locked, so every phase configures inside an off-tracing window with drain idles at the transitions. scripts/cli_featurematrix_test.sh. This is the test that closed the largest coverage-gap classes in ct_L23_preproc_comp_filters and act_proc/act_st (see Chapter 31).

Test	Stimulus	Verification principle and gate
instruction/31_status_msgs	One workload, seven CSR legs covering Device ID (TCODE 1) and Watchpoint (TCODE 15) status messages, including a WARL negative control (illegal <code>SendDeviceId</code> encoding).	Both message types are PC-neutral (no ICNT, no address reference). Contract DO-1: Device ID, when sent, is message #0 – before the config message and before the first synchronizing message. <code>scripts/cli_status_test.sh.</code>
instruction/32_trig_regs	One workload, six CSR legs covering the TCI trigger configuration registers (<code>trTeTrigExtInControl</code> at <code>0x050/0x054/0x058</code>), including the historical <code>InstTrigEnable</code> path for contrast.	CTTE owns exactly one external trigger input; every other TCI trigger field reads back the read-only 0 the spec prescribes for a trigger that does not exist. <code>scripts/cli_trigregs_test.sh.</code>
instruction/33_tesync_req	One workload per scenario: no request, one write, two back-to-back writes, three back-to-back writes, a request during an active byte quota, a request during an overflow storm, a request colliding with an ACT-CAP <code>CF_SYNC</code> on the same retire.	<code>trTeControl.InstSyncReq</code> (bit 27): one write → one <code>SYNC=14</code> message, queue depth of one in flight plus one remembered (two writes → two messages, three → still two), <code>SyncReqSource=4</code> read back, no message when nothing was requested. <code>scripts/cli_tesyncreq_test.sh.</code>

30.4. Sync cadence, ICNT and wide-field edge cases

Test	Stimulus	Verification principle and gate
instruction/20_icnt_overflow	One long straight-line block (longer than the 255-halfword narrow ICNT cap), three CSR legs: default HTM, +BTMLEG (BTM), and the wide-ICNT variant.	Closes coverage-worklist item T1 (no other test exercised the <code>ResourceFull</code> (ICNT_OVERFLOW) drain family before this test existed). <code>scripts/cli_i20_test.sh.</code>
instruction/29_sync_quota_bytes	Alternating compressibility: a long compressible linear run, then a jump-dense incompressible run, against <code>InstSyncMode=TRACE_BYTES</code> .	The trace-output byte quota bounds re-anchor distance by emitted bytes ($2^{(\text{Max}+4)}$), independent of workload compressibility – unlike an instruction-count sync, whose byte distance varies with how compressible the code is. <code>scripts/cli_syncquota_test.sh.</code>
instruction/30_sync_quota_msgs	The message quota (<code>InstSyncMode=TRACE_MSG</code> , every 16 messages) exercised together with an ACT-CAP <code>CF_SYNC</code> command and an ATB sync-request pulse, per collision design D8.	Message quota counts on-wire EOM boundaries; the collision legs prove the quota does not double up with the other two explicit sync sources. <code>scripts/cli_syncquota_test.sh.</code>
instruction/35_addr64	Deliberately plain control flow; what matters is the base PC (<code>0x0000_1000</code> at <code>CT_XLEN=32</code> vs. <code>0xFFFF_FFC0_0000_1000</code> – above 2^{32} , with a plausible-looking low half – at <code>CT_XLEN=64</code>).	The verdict is read off the raw ATB bytes (<code>scripts/check_addr64_emission.py</code> reconstructs <code>PC_FADDR</code> from the MDO nibbles), never from a decoder – per Section 29.2 , a gate that can only fail together with its own tool judges nothing, and the reference decoder had no 64-bit support when this gate was written. <code>scripts/cli_addr64_test.sh.</code>

Test	Stimulus	Verification principle and gate
instruction/36_block_tip	One instruction sequence written once, driven two ways in the same block build: one instruction per TIP beat (+SERIAL), or blocks of 1-4 instructions per beat terminated by every retire kind.	CT_EN_BLOCK_TIP (R1.3): packaging changes, reconstructed control flow does not. Both legs log a per-instruction oracle and must decode to it exactly; anti-vacuity witness measured at tip_if (76 halfwords in 14 beats vs. 38 beats, largest beat 8 halfwords). Four red counter-proofs (MUT=M1..M4). scripts/cli_blocktip_test.sh; byte-neutrality of the default (off) build via scripts/r13_off_neutrality.sh (31 of 31 reference artefacts identical).
instruction/38_core_xlen	Three legs: a DECLARED_XLEN matching the netlist's ct_pkg::CT_XLEN (accept), a mismatch, and an undeclared (0) value.	Elaboration guards genCoreXlenMismatch/genCoreXlenUndeclared (see Chapter 5) must reject the latter two. scripts/cli_corexlen_test.sh.
instruction/37_sync_cadence	A 100,010-retire, 15 %-control-flow-density scenario against a finite -bandwidth sink (one accepted ATB beat per 10 cycles = 40 MB/s — an always-ready sink cannot pose this question at all). The positive leg deliberately does not program InstSyncMax (flies the reset value); the counter-proof leg forces +SYNCMAX=0 (minimum period).	Reset-value regression gate (P0-02): zero Error messages and a strictly lossless decode (100,008 of 100,008 PCs) at the shipped reset, versus 790 Error messages and 5,064 dropped events at the old reset (InstSyncMax=0) under the same finite sink. The cadence is read back off the wire (config-message P1[12] bits [10:7]) and cross-checked against both the RDL declaration and the generated RTL reset — a three-way agreement no source read alone could establish. scripts/cli_synccadence_test.sh.

30.5. Multi-source separation

One scenario stands apart from the groups above because it is the only unit-level testbench that runs **two** encoders at once.

Test	Stimulus	Verification principle and gate
instruction/39_dual_src	Two encoder instances with different trTeInstFeatures.SrcID at SrcBits = 2 — the width every multi-core demonstrator uses — pushing into one ct_L1_funnel from a single fork. The two sources deliberately run different programs at disjoint address ranges (base 0x0000_7000 and 0x2000_0000).	Separation is shown, not asserted. Seven legs: the merged stream is well formed (0 error messages); both SRC values appear and each source's config message carries its own {SrcID, SrcBits}; the funnel actually interleaved (a floor on source changes between consecutive messages); each half of the merged stream decodes to the reference PC sequence of its own source; decoding one source against the other source's PCInfo must fail (falsifiability); merge-then-split is byte-lossless against what each encoder produced alone; and the multi-target command line the board gates use (-target 0 ... -target 1 ...) yields both sequences. scripts/cli_dualsrc_test.sh.

Why the disjoint programs matter is worth stating, because it is the reason the scenario exists at all: in a capture where both sources run the same image at the same addresses, a decoder that evaluates SRC perfectly and one that ignores it produce identical output, so such a capture can neither confirm nor refute separation. Before this scenario the only evidence for source separation was a whole-SoC demonstrator, where a failure has a dozen possible causes.

30.6. E-Trace and protocol-parameter legs

All of the following are driven by one shared runner, `scripts/cli_etrace_test.sh <leg>`, unless noted otherwise.

Test	Stimulus	Verification principle and gate
<code>instruction/21_etrace_ir (leg ir)</code>	Nested calls with stack-predicted returns, a branch pair and an uninferable jump, ending with a clean trace-off (<code>Enable=0</code> → correlation beat), unlike the <code>06_implicit_return</code> donor scenario.	<code>CT_EN_ETRACE</code> implicit-return mode: OFF and ON from one build, both PC-lossless, ON strictly smaller.
<code>instruction/22_etrace_fln_trap</code>	An async interrupt with <code>icnt==0</code> directly after branch #31 of a full Format-1 branch map, and the same a few linear instructions later; all branches not-taken for a trivially checkable PC path.	Regression guard for the "trap right after a full 31-bit branch map" corner, where the decoder's position after a Format-1-without-address packet is the 31st branch, not the last address-packet target.
<code>instruction/23_etrace_resync</code>	Linear stretches, branches, a call/return pair and an uninferable jump, with a small periodic-sync window firing several times mid-trace.	Composer periodic syncs must re-anchor via deferred Format 3.0 packets rather than being dropped; the cli leg requires ≥ 2 mid-trace F3.0 anchors. See the honesty note in Section 29.6 – this leg's original 2026-07 claim was narrower than three defects the P10 soak campaign later found; those are closed by <code>34_etrace_resync_ir</code> below.
<code>instruction/24_protocol_param (leg mixed)</code>	Two <code>ct_encoder</code> instances elaborated side by side from the same sources, one N-Trace, one E-Trace, each with its own Wishbone CSR port.	Proves the P9 per-instance protocol parameter: each instance reports its own <code>atb_te_raw</code> framing and <code>trTeProtocolSel.Protocol/trTeImpl.ProtocolMajor</code> discovery, and a write to the discovery register has no effect (read-only).
<code>instruction/25_etrace_jtc (leg jtc)</code>	A fixed dispatcher round executed three times; round 1 installs five jump targets, rounds 2-3 hit the 64-entry cache.	<code>InstEnJumpTargetCache</code> on the E-Trace back end: OFF/ON from one build, both lossless, ON smaller, ≥ 2 Format-0.1 packets.
<code>instruction/26_etrace_bp (leg bp)</code>	An 80-iteration self-loop with a highly predictable taken branch.	<code>InstEnBranchPrediction</code> on the E-Trace back end: first map fill goes out as Format 1 (initial mispredicts), the second fill switches to COUNT mode (no packet), the loop exit closes as Format 0.0. OFF/ON both lossless, ON smaller, ≥ 1 F0.0 packet.
<code>instruction/27_etrace_df (leg df)</code>	Branches interleaved with loads/stores of all four sizes, aligned and unaligned, with values exercising the <code>data_len</code> sign strip.	<code>te_data</code> packets (<code>msg_type 3</code>) checked byte-exact via <code>etrace_data_check.py</code> , with the interleaved PC trace staying lossless.

Test	Stimulus	Verification principle and gate
instruction/28_etrace_daq (leg daq)	The deterministic DAQ commands (DIRECT_DATA, DATA, DADDR, DATA_DADDR, PC_CURR) via ACT-CAP, routed to the Nexus sink.	Byte-exact vendor packet content (idtag + three 64-bit elements) via etrace_data_check.py --daq; interleaved PC trace stays lossless.
instruction/34_etrace_resync_ir (leg resyncir)	A braid of far-call/return and call-to-next-address/double-return shapes under a tight periodic sync window, implicit return off and on, both with ≥ 3 mid-trace F3.0 anchors.	Regression guard for the P10 soak findings S-1/S-2 (pre-existing E-Trace defects, since closed): a periodic sync anchoring on a predicted-return retire must stay folded, and a target report whose address is also linearly reachable must carry the updiscon flag. Red counter-proof: on the pre-fix tree the implicit-return-OFF half already fails (prefix mismatch, 332/336).

One class remains open by wire-format necessity rather than by test coverage: with `irdepth` field width 0, an implicit-return-folded loop crossing a positional binding point cannot express its visit count (measured 6/33 red among implicit-return-armed runs of a free-axis calibration, seed 20260806). The implicit-return bit is masked from the E-Trace soak campaign until an `irdepth` field exists; see [Chapter 33](#).

30.7. Data trace

Test	Stimulus	Verification principle and gate
data/01_basic	Loads and stores at every supported size (byte/halfword/word/doubleword) interleaved with short linear runs, data trace only (instruction trace off).	make sim-data-basic; the encoder must emit DataRead/DataWrite messages independent of the instruction-trace path.
data/02_split_load	The SPLIT_DATA_ACCESS=1 elaboration leg: stores emit their DF at dretire, loads capture the filter hit at dretire and emit at lresp; an overwritten pending load is deliberately never traced.	scripts/decode_and_check.sh --pc --data split_load_tb; the --data oracle encodes the split-load pending-slot contract explicitly (the TB marks the overwritten access untraced).
data/03_addr_compress	WARL probes of the unimplemented DIFF/XOR_DIFF modes (must legalize to FULL), block accesses (small XOR deltas), scatter accesses (large deltas, byte granularity), and a DataTracing=0 window with accesses inside (negative gate).	CT_EN_DF_ADDR_COMPRESS with trTeDataAddrCompress=XOR — the only implemented mode (E-P3-2). scripts/cli_dfcompress_test.sh.
data/04_data_sync	A combined instruction+data stream: a clean baseline, an ATB stall plus a jump-indirect storm with interleaved data accesses that trips a QueueOverrun, then release and drain.	Re-anchor contract T2: every synchronizing CF message and every emitted ERROR re-arms the DF re-anchor (first post-event data message must carry a full address, TCODE 13/14). scripts/cli_dfcompress_test.sh.
data/05_df_workload	The same access sequence driven twice: FULL-mode (uncompressed) and XOR-compressed addressing.	P3 DF-bandwidth workload; scripts/cli_dfworkload_test.sh.

Test	Stimulus	Verification principle and gate
data/06_df_drop	One stimulus, two legs differing only in <code>trTeDataControl.DataDropEna</code> : without it, a stalled-ATB burst overflows the eTIP queue and drops instruction trace with it; with it, the drop watermark sheds data-trace messages first and instruction trace stays lossless.	Status-bit contract (G12, RW1C, checked in-sim since it is not wire-observable): <code>InstStallOrOverflow / DataDrop</code> set correctly per leg. <code>scripts/cli_dfdrop_test.sh</code> .

30.8. Hardware-supported instrumentation

Test	Stimulus	Verification principle and gate
hsi/01_csr_cap	Every ACT-CAP command issued as a functional NOP CSR write (RISC-V CSR 0x0B10): all <code>DAQ_*</code> commands routed to both the AXIS and Nexus sinks, plus <code>ACT_CAP_ST_CF_SYNC</code> routed to the Nexus sink only.	<code>make sim-hsi-csr-cap</code> ; AXIS beat self-check (TID = command) plus CXP DAQ record reconstruction on the Nexus side. The AXIS-only variant (element-2 compiled out) is the <code>scripts/cli_axists_test.sh</code> leg of the same testbench.

30.9. Overflow and robustness (cross-cutting)

Every scenario in this group was written **from** a defect first observed on real hardware (KV260 soak/robustness campaigns), and each carries a documented red counter-proof – see [Section 29.6](#) for why this many scenarios exist for what is structurally one contract (state re-convergence across a stream discontinuity).

Test	Stimulus	Verification principle and gate
overflow/01_overflow_recovery	Instruction and data tracing both on; clean baseline, then an ATB stall plus dense CF storm that saturates the composer's CDC FIFOs, then release and recovery.	Full overrun lifecycle – inject, resync, soft-reset recovery. <code>make sim-overrun-recovery</code> ; official gates: <code>--overflow (3 ERROR`s)</code> , <code>--disabled correlations</code> , decode to stream end, data 7/7, byte-neutral REF.
overflow/02_natural_overflow	Two 700-jump storm bursts of back-to-back uninferable jumps (disjoint fresh targets) separated by a calm phase, at the MicroBlaze™ V KV260 equal-clock regime (no forced injector).	Reproducer for a natural (non-injected) overflow regime the injector-forced tests never enter. <code>scripts/cli_natovf_test.sh</code> .
overflow/03_jtc_overflow	A ring of repeating jump targets (not all-fresh) with JTC enabled, so cache hits occur whose install may lie inside a drop window.	Regression guard: after a <code>FIFO_OVERRUN</code> recovery, the encoder must not reference a JTC entry the decoder never saw installed. <code>scripts/cli_jtcovf_test.sh</code> .
overflow/04_ir_overflow	Calls and returns inside a natural-overflow discard window, structured like <code>03_jtc_overflow</code> .	Regression guard: the encoder's implicit-return stack must be cleared at the <code>FIFO_OVERRUN</code> recovery anchor (<code>ret_sp</code> clear in <code>ct_L23_preproc_composer_etip</code>), or a stale frame desyncs the decoder's own call stack. <code>scripts/cli_irovf_test.sh</code> .

Test	Stimulus	Verification principle and gate
overflow/05_bp_hwsync	A dense branch-heavy jalr ring with branch prediction and half-word periodic sync, no storm and no stall; iteration length coprime to the sync period so every phase offset is hit.	Isolates a sync-boundary attribution defect (a branch retiring in the same beat as a periodic-sync emission) without any overflow in the picture. <code>scripts/cli_bphws_test.sh</code> .
overflow/06_suite_isync	A trained counting loop plus an indirect jalr ring, full compression suite plus periodic sync, no overflow.	Reproducer for a defect the single-factor legs (BP alone, BP + IRQ, 05_bp_hwsync) do not reach – it takes the full suite combination. <code>scripts/cli_suiteisync_test.sh</code> .
overflow/07_board_replay	The exact instruction sequence of a hardware run (auto-generated oracle via <code>scripts/gen_replay.py</code>) replayed with the exact hardware configuration.	Bisects the encoder core against the adapter path: if the replay reproduces a hardware-observed defect, it is in the core (fixable in sim); if it stays clean, the defect is adapter-specific. <code>scripts/cli_boardreplay_test.sh</code> .
overflow/08_bp_syncbranch	A four-instruction loop under a 16-instruction sync period (a multiple of the loop length), all four phase alignments run, branch prediction on.	Forces the collision fingerprint identified from a hardware-vs-replay message diff (a sync-carried branch, IBHS with HIST=2) that the 07_board_replay phase alignment never happened to hit. <code>scripts/cli_bpsyncbr_test.sh</code> .
overflow/09_bp_ovfanchor	Periodic sync off, overflow active, branch prediction on – the recovery-anchor variant of the same state family.	Regression guard: a FIFO_OVERRUN anchor must not land on a not-taken-branch retire without being held, since that retire is not itself a control-flow change but still owns an outcome bit. <code>scripts/cli_bpovfanchor_test.sh</code> .
overflow/10_replay_ovf	A 30,000-instruction oracle prefix (from <code>scripts/gen_replay.py</code>) with real predictor training (one branch taken 14,376 times), natural overflows forced by ATB throttling.	Adds real branch-outcome history to the 09_bp_ovfanchor class – exactly the ingredient behind the hardware-observed VendorBP BCNT misattribution after FIFO_OVERRUN anchors. <code>scripts/cli_replayovf_test.sh</code> .
overflow/11_pause_edge	InstTracing toggled off and on asynchronously, phase-swept, in the middle of a CF-dense retire stream.	Regression guard for a defect with no overflow involved: a live-sampled trace-active qualifier could lose or misplace up to <code>max_delay</code> instructions at the pause edges. <code>scripts/cli_pausedge_test.sh</code> .
overflow/12_trap_anchor	An indirect call ring (ATB-throttled, hundreds of natural recoveries) where every fourth iteration traps synchronously (no-retire <code>ecall</code> , handler, <code>mret</code>).	Third case of the recovery-anchor assumption, after CF and not-taken-branch: an anchor must not land on a trapping instruction that never retires. <code>scripts/cli_trapanchor_test.sh</code> .
overflow/13_jtc_stopdrain	JTC-only configuration, a call ring with a constant return target, dense periodic sync, ATB throttle (backpressure plus natural overflows), InstTracing lowered on a running core.	Regression guard for message-ordering-under-backpressure at the stop drain: a VendorJTC message must not overtake a later sync in the stream. <code>scripts/cli_jtcstopdrain_test.sh</code> .
overflow/14_ovf_window	Cross-term stress: trace-window edges combined with FIFO overflow, branch prediction / JTC and dense periodic sync.	Guards two hardware-found ICNT-accounting defects at emission-cycle collisions between the overflow injector and the drain chains. <code>scripts/cli_ovfwindow_test.sh</code> .

Test	Stimulus	Verification principle and gate
overflow/15_flush_clobber	ATB flush pulses in the middle of a dense JTC stream under backpressure.	Regression guard for a message-granular silent loss: a trailing <code>if (FlushRequest) send_flush_msg()</code> could overwrite a just-composed message when a flush service cycle collided with a <code>consume_etip</code> beat. <code>scripts/cli_flushclobber_test.sh</code> .

30.10. Combined end-to-end

Test	Stimulus	Verification principle and gate
combined/01_all	Linear code, a taken branch, a call/return pair, varied-size loads and stores interleaved with the code, and one <code>ACT_CAP_ST_CF_SYNC</code> issued mid-stream via the ACT-CAP CSR.	Instruction, data and ACT-CAP paths run together to catch cross-feature interactions the per-feature tests miss in isolation. Three checks on one capture, all from the single decode driver: <code>scripts/decode_and_check.sh</code> (PC stream), the same script with <code>--data</code> (DataRead/DataWrite) and with <code>--sync 3</code> (at least three synchronization messages). NOTE: with both instruction and data tracing on, data-trace messages flush ahead of buffered instruction messages, so record order is not program order – this scenario checks <code>--data</code> , not <code>--ctxp</code> (see Chapter 29).

Chapter 31. Coverage Evidence

The line-coverage evidence for CTTE is a single measured figure: **93.42 % measured (4,972 of 5,322 product-RTL lines) and 100.00 % effective (4,972 of 4,972) line coverage**, Verilator 5.040, full-profile build with both the N-Trace and the E-Trace back end compiled in, measured 2026-07-25 (Round 5, the first round to measure both protocols together) and recorded in an internal measurement report outside the published tree ([Section 31.7](#)). Three qualifications belong to that sentence as much as the number does, and all three are stated in full further down this chapter rather than left implicit: the recording location is outside this repository's version control ([Section 31.7](#)); the figure predates a later RTL change that removed measured lines ([Section 31.8](#)); and the **effective** half can no longer be recomputed from the published tree at all, because the tools that computed it were removed on 2026-08-24 ([Section 31.5](#)).

31.1. Measured vs. effective, and what closes the gap between them

"Measured" is the raw Verilator line-coverage rate over `rtl/` (excluding test infrastructure under `rtl/external/`'s simulation-only tasks, which are covered separately). It is a **union** metric: ``scripts/coverage_report.sh` merges every testbench's `coverage_*.dat` with `verilator_coverage --write-info`, and a line counts as covered if **any** test in the suite hit it—it is not an average of per-test rates (individual testbench rates run 44-50 %, which is expected and not a suite-quality figure in itself).

"Effective" is measured coverage with every line that carries an **ACCEPTED** waiver excluded from the denominator. The waiver mechanism (a triage tool and its waiver ledger; neither has ever been part of the published tree, and the tool was removed on 2026-08-24—see [Section 31.5](#) and [Section 31.7](#)) auto-classifies two trivial shapes on its own—`$stop/$fatal/$error` guard paths whose correctness criterion is non-firing, and empty `default`: arms of an otherwise-exhaustive case—and seeds every other uncovered line as a candidate that a human must disposition with a file/line anchor, a rationale and a source citation. A line reaches "effective 100 %" only when every one of its uncovered siblings is either one of the two auto-trivial shapes or has an **ACCEPTED** waiver; there is no third way to make a line stop counting.

31.2. The waiver ledger

The waiver ledger held **68 entries** at Round 5 (2026-07-25):

Status	Count
ACCEPTED	57
REJECTED	8
PROPOSED	3

A **REJECTED** entry is not a residual gap: all eight are auto-rule duplicate findings on lines a **different**,

ACCEPTED waiver already covers (e.g. AUTO-0017/AUTO-0018 on `rtl/external/stream/sink_if.sv/source_if.sv`, superseded by the ACCEPTED W-0004/W-0005) or a structural-arm finding retired because a later edit moved the line (AUTO-0031). They are kept in the store as an audit trail of what the auto-rule found, not because they exclude anything themselves.

A PROPOSED entry is the opposite: a completely uncovered function or task, auto-flagged as "unused convenience OR missing test – review required" and **not yet** dispositioned. Three remain PROPOSED in the Round-5 store – AUTO-0014 (`send_icnt_overflow_msg`, `rtl/ct_L2_msg_gen.sv`), AUTO-0027 (`unpack_act_st_cmd`, `rtl/preproc/ct_L23_preproc_act_st.sv`) and AUTO-0029 (`set_match_next`, `rtl/preproc/ct_L23_preproc_comp_filters.sv`) – and are not folded into either the "effective 100 %" figure or the internal gap worklist's "0 real residual gaps" statement; see the BEFUND at the end of this chapter for why that statement and the open PROPOSED entries do not currently agree.

By waiver type (all 68 entries of the Round-5 ledger):

Type	Count
STRUCTURAL_ARM	14
UNUSED_CODE	12
DEFENSIVE_UNREACHABLE	11
CARRYOVER_P3	7
ELAB_DEAD	4
CARRYOVER_P2	4
UNUSED_CONVENIENCE_OR_GUARD	3
CONFIG_DEAD	3
DIAGNOSTIC	3
NEVER_FIRE_GUARD	2
TEST_INFRA, GENERATED_CODE, TOOL_ATTRIBUTION, UNREACHABLE_INTEGRATION, UNREACHABLE_SIM	1 each

Representative dispositions: `rtl/external/wishbone/wb_package.sv` (W-0001, TEST_INFRA) is Wishbone simulation tooling, not product RTL; `rtl/external/common/counter.sv` (W-0002, ELAB_DEAD) carries generic-IP MODE arms this design never instantiates; the `sink_if/source_if/cvsink_if` have_available functions (DEFENSIVE_UNREACHABLE) are error detectors whose non-firing is the correctness criterion – a hit would itself be a test failure; `ct_cs_cpuif.sv`, the PeakRDL register-block generate, carries uncovered read/write-decode arms for rarely-touched CSRs, dispositioned as "generator correctness, not hand-written logic" with the CSR-contract-relevant fields covered separately by the `hsi/csr_cap` WARL tests.

31.3. Gap-closure history

Round	Date	Scope	measured / effective
1-2	2026-07-21 (morning)	OFF-legs only, then + ON-leg pass (scripts/coverage_on_legs.sh)	79.65 % / – then 84.67 % / –
3-4	2026-07-21	+ Tests 19/20, split-load leg, 6 new ON-legs, one RTL bugfix (below)	93.17 % / 99.34 % (26 residual lines, 5 clusters)
5	2026-07-25	+ E-Trace and DUAL profiles measured together, four new coverage phases, one RTL bugfix (below)	93.42 % / 100.00 % (0 residual lines against the store as it stood that day)

Round 3-4's largest single cause was methodological, not a test deficiency: the OFF-only union under-measured `ct_L2_msg_gen` (52.25 % → 77.50 % once ON-legs ran), `ct_L2_nexus_formatter` (70.10 % → 98.97 %) and `ct_L23_preproc_composer_etip` (93.50 % → 95.07 %), because the feature testbenches existed and ran, but only their OFF leg was part of the merged `.dat` set. `scripts/coverage_on_legs.sh` is now a fixed part of every coverage run for exactly this reason.

Round 5 closed 103 fresh residual-gap lines from that round's first pass: 57 through new test stimulus (four plusarg-gated coverage phases on existing E-Trace testbenches – +BPX, +JTCX, +ANCHORJUMP, +TH0, plus `natural_ovf_tb` run under the E-profile) and 46 through waivers (21 new: 11 diagnostic-instrumentation lines gated behind +TE_BEAT_DUMP/+TE_PKT_DUMP plusargs, W-0122-W-0124, and a BpCount saturation arm at 2³²-1 correct predictions, W-0125, UNREACHABLE_SIM; 25 carryover dispositions of the Round-4 clusters below).

31.4. Residual carryover clusters from Round 3-4, dispositioned in Round 5

Cluster (Round-4 residual)	Lines	Round-5 disposition
<code>bit_slicer_impl</code> field-combination edge arms	15	W-0126-W-0129, CARRYOVER_P2: reachable only by dedicated field-list unit stimulus; the N-Trace system legs do not reach the edge lists.
<code>composer_axis</code> sink-routing else arms	5	W-0130-W-0134, CARRYOVER_P3: full sink-routing matrix; the Nexus-sink side is covered via the hsi/DAQ legs.
<code>comp_filters</code> MatchMode-INPUT edge arms	2	W-0135, CARRYOVER_P3, with a findings trail: comparator mechanics proven identical via the P-function formal target; the hierarchical interface probe needed to reach this arm directly crashes Verilator 5.040 (XSIM-wave clarification pending).
<code>act_proc</code> ACT_CAP_TE_DATA_TRACING arm	3	W-0136, CARRYOVER_P3: watchpoint stimulus exists (Test 19, phase 10) and reaches a hit, but not this specific arm; the structurally identical INSTR_TRACING twin is covered.

Cluster (Round-4 residual)	Lines	Round-5 disposition
ct_cs_cpuif_wb single line	1	Covered by the Round-5 runs without a waiver.

31.5. Measurement mechanics

Reproduction (c-trace repo root):

```
1 bash scripts/coverage_suite.sh          # OFF-legs, resumable, 900 s per-task timeout
2 bash scripts/coverage_on_legs.sh        # feature ON-legs over the built Verilator executables
3 bash scripts/coverage_etrace.sh          # E and (pre-P9) DUAL profiles, see the currency note
  below
4 bash scripts/coverage_report.sh          # union merge -> bld/coverage/merged.info
```

IMPORTANT

That sequence reproduces the **measured** rate only. The three tools that produced the per-module gap report, the HTML report and above all the waiver triage — `scripts/coverage_gap_report.py`, `scripts/coverage_html_report.py` and `scripts/sv_cov_triage.py` — were removed from the repository on 2026-08-24 (5ed46b6abb6, "Remove unused tools"). The removal was deliberate, and the repository says so in the same commit: the waiver ledger, the per-gap dispositions and the triage tooling that maintains them are not part of the published tree, so **the effective rate cannot be recomputed from this repository**. The 93.42 % measured half stays reproducible with the four scripts above; the 100.00 % effective half is a recorded measurement of 2026-07-25 and, with the tools in the tree today, is not.

Measurement caveats recorded alongside the Round-5 figures and carried forward here because they qualify the headline number:

- **Two tools, two exclusion definitions.** the triage tool computes effective coverage over auto-rules plus ACCEPTED waivers (350 excluded lines) and reports 100.00 %; the separately generated gcovr-style HTML report of that round excludes only the ACCEPTED waivers themselves (348 lines) and reports 99.96 % — a 2-line difference made up of auto-classified trivial lines that carry no store entry. Both numbers are correct for their own definition.
- **Profile-dependent generated file.** `rtl/pkg/ct_cs_cpuif.sv` (the RDL register-block generate) has a different line count per build profile, which makes the gcovr tool print a line-count mismatch warning. This is annotation noise, not a biased measurement: the file's rate is identical ($1,884/2,038 = 92.44\%$) in the default-only merge and in the combined merge — the profile variant adds no lines to the union.
- **Four suite tasks contribute no .dat.** Three Verilator-discrepancy testbenches (`funnel`, `formatter`, `formatter_err` — 100 % green under XSIM, the reference gate; the Verilator failures are simulator-scheduling artefacts — TSTAMP/UADDR values arriving as 0, or a handshake timeout — not product defects) and `ct_nexus_decoder_tb`, which cannot build at all: `tests/lib/ct_nexus_decoder.abc` references `../../nexus/mseo2_decoder.sv`, a path that exists in neither the vendored tree nor the upstream pin (verified 2026-07-21 by

sparse-cloning the encoder repository — then still published under its retired name `accemic/C-Trace`, today `accemic/TraceEncoder` — at `3a74ea59: no nexus/ directory, no mse02* file`). None of the four are product RTL under test — the DUT modules of the three discrepancy testbenches are covered through the system testbenches instead, and the decoder testbench is verification infrastructure (`tests/lib`).

31.6. RTL defects the coverage campaign found

Coverage-driven stimulus surfaced two RTL defects that no directed test had reached, both fixed with byte-neutrality evidence before the round closed:

- **Round 3-4:** the wrapper mapping block `ct_cs_cpuif_wb.sv` did not map `PInput/SInput` — the `CONTEXT/TVAL/DADDR` comparator inputs were silently inert regardless of configuration. Two-line fix; byte-neutral (`cli_ir_test.sh` OVERALL PASS, md5-prefix gates green).
- **Round 5:** the `+ANCHORJUMP` coverage phase tripped the pre-existing `icnt-convention` assertion. Root cause: when a FIFO-overflow recovery or trace-enable anchor lands on a control-flow instruction, that instruction's half-words count into the **following** segment by composer convention (exclusive anchor), but `NextAddr` was set to the jump target instead of **target minus anchor width** — desynchronizing retirement tracking by one instruction width. Fixed in `ct_L2_te_inst_gen.sv`, commit `3e066b22`; the full function matrix plus the N-Trace regression suite were re-run afterward.

31.7. Provenance

NOTE

The coverage figures in this chapter are quoted with date, tool version and measurement scope, but the underlying report, gap worklist and waiver ledger are not currently version-controlled: `docs/` is excluded by that repository's `.gitignore`. There is no commit hash to cite for the 93.42 % / 100.00 % figures, and this chapter cannot be independently reproduced against a tagged state of the source repository until that is corrected. Once `docs/COVERAGE_verilator.md`, `docs/COVERAGE_gaps_worklist.md` and `docs/coverage_waivers.json` are brought under version control, this note is to be replaced by the commit hash of the versioned report; no other wording in this chapter needs to change.

31.8. Currency

The 2026-07-25 figures were measured on the RTL as it stood before a later synthesis-parameter change (P9, 2026-08-04) removed the dual-protocol build of `ct_encoder` — roughly 135 lines, plus the associated `etrace_dual_tb` testbench and the DUAL coverage profile in `scripts/coverage_etrace.sh`. Both the numerator and the denominator of the headline figure move with that change, so the 93.42 % / 100.00 % pair is **not** line-for-line comparable to a measurement on the current tree, and `doc/verification.adoc` and `doc/release-`

`notes.adoc` both say so explicitly: the coverage rate "has not been re-measured since P9 changed numerator and denominator." No later measurement exists in the repository at the time of writing. A fresh run of the reproduction sequence above is required before this figure can be cited as **current** rather than as **last measured**.

Chapter 32. Implementation Status Matrix

This chapter gives a review-friendly, feature-by-feature view of what is implemented in the CTTE RTL, checked against the tree as of this writing. Unless stated otherwise: the RTL in `rtl/` is authoritative; register semantics are authoritative per the SystemRDL sources and the generated PeakRDL outputs (`rtl/pkg/ct_cs_cpuif*.sv`); verification evidence refers to the test catalog ([Chapter 30](#)) and the coverage measurement ([Chapter 31](#)).

32.1. Legend

Implemented

Feature exists in RTL, is verified (a directed test with a CTDD round trip and/or a byte-identity gate) and covered by the line-coverage union.

Partial

Feature exists but is known incomplete, constrained, or not fully verified.

Not implemented

Feature is explicitly not present in this repository snapshot (an open item).

n/a (by design)

Capability deliberately not offered – no implementation gap. Example: Program Trace Correction (TCODE 10) is not adopted by ratified N-Trace 1.0 (Table 9 keeps it Reserved), so it is not tracked as a CTTE gap.

32.2. Status Matrix

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
Trace Encoder Control/Status register block (TCI mapping)	Implemented	PeakRDL outputs <code>rtl/pkg/ct_cs_cpuif*.sv</code> ; profile-dependent register omission via the RDL preprocessor (<code>scripts/gen_rdl_profile.py</code>) – registers of compiled-out units are omitted entirely (no address decode / readback), never merely masked.	<code>hsi/01_csr_cap</code> WARL tests; full-profile regenerate byte-identical.
Instruction trace – Branch History Trace (ITR_BRANCH_HIST, InstMode 6)	Implemented	<code>ct_L2_msg_gen.sv</code> (history accumulation, IBH/IBHS, sync forms).	Instruction-trace system suite; CTDD PC-lossless round trips; REF_FINAL md5 family (Chapter 29).

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
Instruction trace – Branch Trace Messaging (ITR_BRANCH, InstMode 3, TCODE 3/4)	Implemented	CT_EN_BT_M (full=1, slim=0, synthesizable-away); ct_L2_msg_gen.sv ITR_BRANCH arm; WARL accepts InstMode 3 and 6.	instruction/17_btm (scripts/cli_btm_test.sh): HTM and BTM legs both PC-lossless against the same cpu_model reference.
ICNT overflow drains (RCODE=0) – all silent-CF accumulation paths	Implemented	ct_L2_msg_gen.sv: indirect-/BTM-/BP-drain holds and the inline guards of the silent arms; send_icnt_overflow_msg.	instruction/20_icnt_overflow (scripts/cli_i20_test.sh): whole drain family covered; decode with ResourceFull ICNT adjustments PC-lossless.
Timestamps (system + core source, prescaler)	Implemented	ct_L23_preproc_ts.sv; trTsControl – enum values TR_TS_SYSTEM='h2, TR_TS_CORE='h3 (0/1 reserved).	Timestamps are on implicitly in every scenario; prescale stages 1-3 and the core-source arm covered (instruction/19_feature_matrix phase 7); TSTAMP decoded by CTTD.
Periodic synchronization (cycles / instructions / halfwords / ATB / trace-quota bytes and messages) + explicit sync requests	Implemented	ct_L23_preproc_sync.sv + egress quota counters (CT_EN_QUOTA_SYNC, CAPS bit 18); trTeSyncStatus.SyncReqSource (none/CSR/ATB/quota/TE-register); vendor SYNC 14. Reset value of trTeControl.InstSyncMax changed 0 → 6 (period 16 → 1024 units) on 2026-08-12.	instruction/04_periodic_sync, 12, 15, 19; trace-quota bytes/messages (instruction/29_sync_quota_bytes, 30_sync_quota_msgs, scripts/cli_syncquota_test.sh); reset-value regression instruction/37_sync_cadence (scripts/cli_synccadence_test.sh) – zero Error messages / lossless at the shipped reset vs. 790 `Error`s / 5,064 dropped events at the pre-2026-08-12 reset, under an identical finite-bandwidth sink.
Self-identification – NEXUS_MSG_VENDOR_CONFIG (TCODE 58, SendConfig)	Implemented	Config message carries CAPS/ENAB/P0-P3 sampled at emission from cs_proc; structure constants in ct_pkg (SSOT); CT_EN_CONFIG_MSG (slim default 0).	instruction/16_config_msg (scripts/cli_cfg_test.sh), 12/12 gates; flag-less CTTD decode equals flagged decode, negative control diverges.
Ownership messages (TCODE 2)	Implemented	ctype/priv-change/after-sync emission; PROCESS LSB-first; on-wire CONTEXT width = TIP_CONTEXT_WIDTH (integration parameter, now ct_pkg::CT_CONTEXT_WIDTH, default 2).	instruction/15_trig_seq_own (scripts/cli_tso_test.sh), exact PROCESS values for FORMAT 0/2.
Debug / Power / EVTI events (SYNC 3/9/0, correlation EVCODE 0/1)	Implemented	Generic event ports (tip.debug_mode /tip.evti/tip.power_down).	instruction/12_debug_power_events (scripts/cli_dbg_test.sh), 6/6 gates.
IBHS (TCODE 29) – syncs carry pending HIST	Implemented	CT_EN_IBHS; 1:1 replacement of the pre-flush.	instruction/13_ibhs (scripts/cli_ibhs_test.sh): payload -17 %/messages -45 %, PC-lossless both legs.
RepeatInstruction (TCODE 31/32)	Implemented	CT_EN_REPEAT_INSTR; spin-loop compression.	instruction/14_repeat_instr (scripts/cli_rpti_test.sh): payload -67 %, PC-lossless.

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
Implicit Return compression	Implemented	CT_EN_IMPLICIT_RETURN + trTeInstFeatures.InstEnImplicitReturn; composer return-stack prediction, the decoder recovers targets from its own stack.	instruction/06_implicit_return (scripts/cli_ir_test.sh); PC-lossless OFF==ON at a strictly smaller byte count on the dedicated stimulus.
Repeated-History compression (RCODE 2) incl. 28/27-bit partial-match conversion	Implemented	CT_EN_REPEATED_HISTORY; hist_partial_match_shift (s=1/s=2 trimmed-match conversion).	instruction/07_repeated_history (scripts/cli_rh_test.sh, with WideICNT); coverage phase exercises the partial-match conversion arms.
RepeatBranch (TCODE 30) – identical-IBH suppression (jump and return repeats)	Implemented	CT_EN_REPEAT_BRANCH; separate arms in the plain-IBH group and in the RETURN arm.	instruction/08_repeat_branch (scripts/cli_rb_test.sh); coverage phase exercises the return-repeat arm.
Jump Target Cache (vendor TCODE 57)	Implemented	CT_EN_JTC; XOR-fold index (widened to the full address at CT_XLEN=64, see below); learn-on-miss on plain IBH and on unpredicted returns.	instruction/09_jtc (scripts/cli_jtc_test.sh, 595 x TCODE 57); coverage phase exercises return-target hits.
Branch Prediction (vendor TCODE 56)	Implemented	CT_EN_BP; bit-identical 2^9x2-bit predictor on both sides; TCODE 56 only on mispredict.	instruction/10_branch_predict (scripts/cli_bp_test.sh); BP legs also exercised in instruction/11, 16, 20.
Wide ICNT (16-bit cap)	Implemented	CT_EN_WIDE_ICNT + trTeInstFeatures.InstEnWideIcnt (narrow default: 8-bit cap = 255 halfwords).	instruction/07_repeated_history leg; per-cap drain behavior in instruction/20_icnt_overflow.
Comparator-based trace filters (instruction/data qualification)	Implemented	ct_L23_preproc_comp_filters*.sv; P/S functions EQ.ALWAYS; inputs IADDR/CONTEXT/TVAL/DADDR; latch mode is MODE3 ('h3).	instruction/19_feature_matrix (function/input/mode sweeps, filter chain, ecause/interrupt predicates, scripts/cli_featurematrix_test.sh); instruction/11_robustness CSR matrix.
ACT-CAP (CSR-based control access protocol) + ACT-ST (Smart Trigger)	Implemented	ct_L23_preproc_act_cap/act_st/act_proc.sv. Watchpoint memory is the backing store of a perfect binary search tree (vector_binary_search_2clk); capacity grew from 4 levels/15 slots to 10 levels/1,023 slots (MO_DIM=10, MO_N=1,023) on 2026-08-13 (C0b). The direct flat-address write window at 0x4100 was removed in the same change (1,023 entries would collide with the df component at 0x5000); loading and readback now go exclusively through trWpIndex (0x400C, auto-increment), trWpDataLow /trWpDataHigh (0x4010/0x4014, commit-on-High), trWpReadLow /trWpReadHigh (0x4018/0x401C) and trWpCap (0x4020, capacity discovery). Load registers are Enable-locked. Software must still fill all trWpCap.Entries slots in sorted order (partial fill leaves the tree root empty).	instruction/19_feature_matrix phase 10 (indirect-table load/readback, full-table load, wrap behavior, Enable-lock, soft-reset-to-0 – see the testbench's own "the INDIRECT path (C0b – the direct 0x4100 window is gone)" comment); watchpoint-hit → SINK_TE INSTR_TRACING /DATA_TRACING verified via the same phase; ACT-CAP direct CSR commands incl. DAQ range-error and unknown-command arms.

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
Performance counters (thresholds + ranges) and DAQ	Implemented	ct_L23_preproc_perfcnt.sv (threshold FSM; the IDLE state itself saturates into TH_ITH_OVERFLOW during long instruction-less stretches); DAQ via ACT commands.	instruction/19_feature_matrix phase 5 (threshold FSM incl. saturation and iretire exit); hsi/01_csr_cap.
Split Load tracing (SPLIT_DATA_ACCE SS elaboration parameter, default 0)	Implemented	ct_L23_preproc_df.sv (PendingLoadHit), ct_L23_preproc_composer_etip.sv (STORE at dretire, LOAD at lresp), trTeDataControl.DataSplitLoad.	data/02_split_load (SPLIT=1 elaboration leg; store/load/lresp and pending-overwrite paths, scripts/decode_and_check.sh --pc --data).
Data address compression (XOR)	Implemented	CT_EN_DF_ADDR_COMPRESS (rtl/pkg/ct_pkg.sv:164, =1); trTeDataAddrCompress selects XOR mode (E-P3-2: the only implemented mode – DIFF/XOR_DIFF are WARL-legalized back to FULL). TCODE 13/14 re-anchor on every synchronizing CF message and every emitted ERROR.	data/03_addr_compress, data/04_data_sync, data/05_df_workload (scripts/cli_dfcompress_test.sh, scripts/cli_dfworkload_test.sh).
Overflow / overrun handling (ERR / resync)	Implemented	ovf_injector (serialize and parallel paths). A substantial defect family was found and fixed across the 2026-07/2026-08 hardware campaigns – not a fixed count, see Chapter 30 for the full, currently 15-scenario tests/overflow/ catalog (natural overflow, JTC/implicit-return state across recovery, sync-boundary attribution, board-sequence replay, pause edges, trap-window anchors, stop-drain ordering, flush clobber).	tests/overflow/01_overrun_recovery through 15_flush_clobber (Chapter 30); official gates on 01: --overflow (3 ERROR`s), `--disabled correlations, decode to stream end, data 7/7; byte-neutral REF.
eTIP serialization (CT_ETIP_SERIALIZE) and Compact Packer (CT_COMPACT_PACKER)	Implemented	Serialize is the full-profile default and is byte-identical; the compact packer replaces formatter, slicer and MSEO controller in the control-flow-only path. Resource figures for both are in Chapter 24 , which carries the tool version, commit and report path per row.	Serialize proof 15/15 byte-identical; compact-pair tests 12/12 verdicts / 14/14 message-identical.
FIFO fill-level histogram (diagnostics)	Implemented	CT_FIFO_HIST_BINS (default 16); DSP48 counters; serial readout CSR pair (map independent of BINS); no CDC (trace-quietescent read contract).	instruction/18_fifo_histogram (scripts/cli_fifohist_test.sh) incl. BINS=8 smoke; slim profile exactly neutral via register omission.

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
SRC field (multi-source)	Implemented	<code>trTeControl.InhibitSrc</code> (bootstrap rule) + <code>trTeInstFeatures.SrcBits/SrcID</code> ; the formatter keeps the field array dense.	<code>instruction/19_feature_matrix</code> phase 8 (field emitted with <code>SrcBits=4</code>); since 2026-08-19 also <code>instruction/39_dual_src</code> (<code>scripts/cli_dualsrc_test.sh</code>), which runs two encoders with different <code>SrcID</code> at <code>SrcBits=2</code> into one funnel and takes the merged stream apart again with the decoder's <code>-src/-srcfilter</code> and <code>-target</code> paths—including the falsifiability leg, in which decoding one source against the other's <code>PCInfo</code> must fail (Section 30.5). The decode-verification follow-up carried here from the predecessor status is thereby closed.
Multiple retirements per TIP block	Implemented, off by default	<code>CT_EN_BLOCK_TIP</code> (default 0) turns <code>iretire</code> from a strobe into the halfword count of the retiring block, <code>iaddr</code> naming the first instruction and <code>ilastsize</code> the size of the last. At the default the encoder behaves exactly as before.	<code>instruction/36_block_tip</code> (<code>scripts/cli_blocktip_test.sh</code>): one instruction sequence driven both as blocks and serially, both must decode to the identical per-instruction oracle; four red counter-proofs (<code>MUT=M1..M4</code>); byte-neutrality of the default build, 31 of 31 reference artefacts (<code>scripts/r13_off_neutrality.sh</code>). Exercising it on real hardware still needs a core configured for two commit ports—both RV64 hardware harts brought up to date (Section 29.9) run a single commit port, so this remains a simulation-only claim.

32.3. Features added in the 1.0.0 cycle

The features below were added on the way to **release 1.0.0** (2026-08-28) and are documented in that release's notes (`doc/release-notes.adoc`). They are cataloged here for completeness rather than folded into the table above, because none of them existed at the point the inherited rows were written—those rows come from a status document that was retired on 2026-08-18 and is named in [Chapter 36](#) for anyone tracing their provenance.

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
Protocol as a synthesis parameter (N-Trace / E-Trace back-end selection)	Implemented	<code>EN_NTRACE/EN_ETRACE</code> instance parameters of <code>ct_encoder</code> (2026-08-04, P9); the historical runtime-selected dual build (both back ends plus an eTIP demux and ATB mux) is removed. <code>trTeProtocolSel.Protocol/trTeImpl.ProtocolMajor</code> are read-only, hardware-driven discovery.	<code>instruction/24_protocol_param</code> (<code>scripts/cli_etrace_test.sh</code> mixed): two <code>ct_encoder</code> instances elaborated side by side, one N-Trace, one E-Trace, each correctly self-reporting.

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
64-bit addresses as a build knob (<code>ct_pkg : CT_XLEN</code> , default 32)	Implemented (32-bit builds unaffected; E-Trace at 64-bit is n/a by design)	One width definition replaces five package-local constants; sizes TIP address, Nexus F-ADDR/U-ADDR, comparators, JTC index fold, ACT-ST/DF-range keys and the eTIP payload. Announced on the wire as CAPS bit 23 (ADDR64). <code>genEtrace64</code> elaboration guard rejects a 64-bit E-Trace instance (<code>te_inst</code> address fields are sized <code>CT_ETRACE_ADDR_BITS=31</code> and would truncate silently).	<code>instruction/35_addr64</code> (<code>scripts/cli_addr64_test.sh</code> , raw-byte verdict, not decoder-dependent, Section 29.2); RV64 hardware bring-up of cv64a6 and Rocket, both booting Linux with zero decoder error messages (Section 29.9).
Trace one process, not one address range (per-process filter, <code>CT_EN_FILTER_SYNC</code>)	Implemented	<code>trTeComp[] . PInput=1</code> (context) turns the existing comparator bank into a per-process filter; region-exit correlation (TCODE 33, EVCODE "Program Trace Disabled") and region-entry <code>SYNC=5</code> close the hole where a filtered-out anchor retire previously produced no re-anchor and left the stream undecodable. No new message type, no CAPS bit.	<code>verification.adoc</code> "Tracing one process, end to end" (cv64a6 simulation, two ASIDs): unfiltered both processes decode (204 + 168 instructions, both ASIDs on the wire); filtered to A: 204/204 for A, 0/168 for B; two counter-proofs (ineffective filter returns the full 437-PC stream; compiled-out filter anchoring decodes zero instructions). Real-Linux-process filtering on a board is a follow-on package, not claimed here.
Explicit sync request over the TE register (<code>trTeControl.InstSyncReq</code> , bit 27)	Implemented	<code>CT_EN_INST_SYNC_REQ</code> ; the TCI-standard request path (existed and auto-cleared before, but had no consumer). On the wire: <code>SYNC=14</code> , same code as the ACT-CAP <code>CF_SYNC</code> command and the ATB sync-request input. <code>SyncReqSource</code> grew to 3 bits (new source value 4).	<code>instruction/33_te_sync_req</code> (<code>scripts/cli_tesyncreq_test.sh</code>): one write → one message; queue depth one-in-flight-plus-one-remembered (two writes → two messages, three → still two); byte-neutral in both builds, 31 of 31 streams md5-identical ON and OFF.
TCI trigger configuration registers	Implemented	<code>trTeTrigExtInControl</code> (0x054, <code>CT_EN_TRIG_REGS</code>): 0 none / 2 trace-on / 3 trace-off / 4 trace-notify (TCI Table 20), additive to the historical <code>InstTrigEnable</code> → <code>SYNC=6</code> path. <code>trTeTrigDbgControl</code> (0x050) and <code>trTeTrigExtOutControl</code> (0x058) read 0 – the TCI-prescribed answer for a trigger sideband and a trigger output port CTTE does not have.	<code>instruction/32_trig_regs</code> (<code>scripts/cli_trigregs_test.sh</code>).
Data-trace drop policy (<code>trTeDataControl.DataDropEna</code>)	Implemented	<code>CT_EN_DF_DROP</code> (CAPS bit 22): sheds data-trace messages at 3/4 eTIP-queue fill to keep instruction trace intact; one <code>Error/ECODE=0x02</code> marker per drop episode, deliberately without the generic overflow's <code>SYNC=7</code> re-anchor. <code>InstStallOrOverflow/DataStallOrOverflow/DataDrop</code> implemented as sticky RW1C (overflow half only – the stall half has no hart-side back channel and stays unimplemented).	<code>data/06_df_drop</code> (<code>scripts/cli_dfdrop_test.sh</code>): one stimulus, drop-disabled vs. drop-enabled legs contrasted directly.

Feature / Topic	Status	Evidence (RTL / RDL)	Evidence (Verification)
Device ID (TCODE 1) and Watchpoint (TCODE 15) status messages	Implemented	<code>SendDeviceId</code> (WARL, <code>DID_NONE</code> / <code>DID_ONCE</code>), <code>trWpMask.WEM</code> ; both message types are PC-neutral (no ICNT, no address reference). Device ID, when sent, precedes the config message and the first synchronizing message on the wire (contract DO-1, also SVA invariant I9, Section 29.7).	<code>instruction/31_status_msgs</code> (<code>scripts/cli_status_test.sh</code>): seven CSR legs including a WARL negative control.
Standing in-RTL invariants (SVA, I1-I12)	Implemented	Twelve concurrent assertions (simulation-only, <code>pragma translate_off</code>) encoding the board campaigns' contract lessons directly in the RTL; run in every simulation of every consumer.	See Section 29.7 for the full list and the falsifiability argument for each.
<code>trTeControl.Empty</code> driven end to end	Implemented	Was structurally tied inactive before 2026-08; now a conservative empty chain per backend tail (2-FF synchronizer plus a quiet filter into the ATB domain), covering the preprocessor's CDC FIFOs and whatever the generator still holds.	SVA invariant I8; exercised by the pause-edge and stop-drain overflow scenarios (Chapter 30).

Chapter 33. Known Limitations and Deviations

This chapter merges the predecessor status document's limitations list with the current, more detailed "Known limitations" section of `doc/release-notes.adoc`, and removes what has since been resolved. Where a limitation is a deliberate design contract rather than a gap, it is stated as a contract, not apologized for.

33.1. Limitations that constrain integration (by design)

These are read-before-filing-a-bug items: each is deliberate behavior with a stated reason, not an open defect.

Topic	Contract
Empty with a stopped core	With <code>InstTracing=1</code> and the core halted, exactly one control-flow event is permanently in flight (its successor address can never be observed) — <code>Empty=0</code> is the correct report. Close the stream (<code>InstTracing=0</code>) while the core still retires, then stop.
Byte-level run-to-run determinism under overflow	FIFO-overflow drop positions depend on real CDC/ATB phase relations; two identical runs produce equivalent but not byte-identical streams. The verified property is decode equivalence. Byte identity holds in drop-free regimes with timestamps off.
Trace-off correlation under overflow	The correlation is an ordinary message: if a pause edge falls into a drop episode it is lost and announced (<code>ERROR</code> plus recovery sync); the resume re-anchor still recovers the decoder.
Post-pause position ambiguity	After a pause, the first decoded segment may be positionally ambiguous against a full-execution reference when the code is homogeneous (the pause length is unknowable from the stream alone). Content stays fully checkable; tooling should pair by anchor and verify transitions rather than forcing positional alignment.
Trace-quota window semantics	The quota counter rearms when the sync is emitted — the sync message's own bytes count into the new window, and no second request is raised while one is pending (no sync avalanche at small quotas). After a FIFO-overflow recovery the counter is not re-based: the first post-recovery quota sync can arrive earlier than one full quota (a harmless extra re-anchor, never a late one). With <code>CT_EN_QUOTA_SYNC=0</code> , <code>InstSyncMode 1/4</code> stay writable but inert (no WARL clamp — exactly the pre-feature behaviour).
Non-standard core trace buses	Core-side adapters (e.g. the AMD MicroBlaze™ V <code>TRACE</code> bus) are pinned to the characterized core/tool version; the bus is documented as not backward-compatible across core or tool-version changes.
XSIM waveform debug of encoder internals	Vivado™ XSIM 2026.x crashes with wave instrumentation on this code base (<code>-debug off</code> required); use Verilator (with coverage, Chapter 31) or <code>\$display/counter</code> instrumentation instead.

33.2. Spec-compatibility corners

- **CDF** is emitted as 1 in both instruction-trace modes; **N-Trace** requires 0 in branch mode. For the Program Trace Correlation message (TCODE 33), N-Trace 1.0 Table 24 binds the `CDF` field to the mode: 1 in history mode (HTM), 0 in branch mode (BTM). The encoder emits the constant `2'b01` in both. In HTM this is correct; in BTM it is a deviation from the ratified specification. The practical consequence is confined to decoders that dispatch on `CDF` to decide whether a `HIST` field follows: the encoder always emits the field, encoded as `0x1` when the history is empty, so a

decoder that reads it unconditionally — including the reference decoder — reconstructs correctly either way. A decoder written strictly to Table 24 and run against a BTM stream will not. Field detail in [Chapter 14](#). Disclosed rather than corrected: the fix is a one-line mode dependency in the formatter, but it changes the on-wire bytes of every stream containing this message and therefore invalidates the byte-identity reference set, which is a versioned change rather than an editorial one.

- **Two vendor-extension enables are writable where N-Trace requires them hard-wired to zero.** N-Trace 1.0 Table 8 states that `trTeInstEnBranchPrediction` and `trTeInstEnJumpTargetCache` "must be hard coded as 0" — the specification reserves the bits but forbids an implementation from making them settable, because the features behind them are not part of N-Trace. CTTE implements both features as declared vendor extensions and therefore makes both bits writable. A configuration tool that assumes the bits read back as zero will be surprised; a decoder is not affected, because it learns the active feature set from the configuration message ([Chapter 18](#)) rather than from the register. The full deviation list with the specification clauses is in [Chapter 4](#).
- **ICNT 8-bit cap not enforced for control-flow-free runs longer than 255 halfwords.** The composer-side `ResourceFull (RCODE=0)` drain item is computed combinatorially but the beat never commits without an accompanying event, so for a purely linear run longer than the cap the item never reaches `ct_L2_msg_gen`. Correctness is unaffected — the variable-length ICNT field carries the true count regardless, and decode stays PC-lossless (measured 950/950 in `instruction/20_icnt_overflow` phase 0, a 300-halfword linear run) — but the 8-bit spec-compatibility cap itself is not enforced for such runs. Tracked as waiver W-0118 (ACCEPTED, type UNREACHABLE_INTEGRATION) rather than a functional defect.
- **E-Trace implicit-return visit count at a zero-width `irdepth` field.** With `irdepth` field width 0, an implicit-return-folded loop that crosses a positional binding point (a trap/off flush, a window or resync anchor) cannot express its visit count on the wire. Measured 6 of 33 red among implicit-return-armed runs of a free-axis calibration (seed 20260806). The implicit-return bit is masked out of the E-Trace soak campaign until an `irdepth` field exists ([Chapter 30](#), E-Trace section); every other compression bit is drawn normally.
- **Comparator, ACT-ST and DF-range bounds are CSR-programmable in the low 4 GiB only, at `CT_XLEN=64`.** The compared address itself is full width; the CSR side of these three subsystems stays a 32-bit word pair.

33.3. Deliberately not implemented / follow-on packages

- **64-bit E-Trace.** `ct_pkg : CT_XLEN=64` together with `EN_ETRACE=1` is rejected at elaboration (`genETrace64` guard, [Chapter 5](#)): the `te_inst` address fields are sized `CT_ETRACE_ADDR_BITS=31` and would truncate silently rather than fail loudly. n/a by design until a wider E-Trace address field exists.
- **Parallel retirement on RV64 cores is not exercised.** Both RV64 harts brought up to date (cv64a6, Rocket) run a single commit port; the multi-retire path (`CT_EN_BLOCK_TIP`) remains verified only in simulation with a scripted multi-instruction-per-beat stimulus ([Chapter 32](#)), not on real multi-retire hardware.

- **Filtering real Linux processes on a board.** The per-process filter chain (CT_EN_FILTER_SYNC) is verified end to end in simulation with real core RTL and a scripted ASID; filtering where the ASID is written by a running kernel on a board is a follow-on package and not claimed here.
- `count_fields` (mse0 formatter), referenced by the predecessor RM, no longer exists in the RTL tree – see the BEFUND note at the end of this chapter.

33.4. Caller-less helper code (retirement candidates)

Two functions in `ct_L23_preproc_comp_filters.sv` remain without a caller in the current tree: `set_match_now` (line ~414, W-0119, ACCEPTED/UNUSED_CODE – only `set_match_next` is invoked, at line ~502) and `set_match_next` itself (AUTO-0029, still PROPOSED – not yet dispositioned as either a genuine test gap or a further retirement candidate; see the coverage-ledger caveat in [Chapter 31](#)). Both are candidates for removal rather than functional defects.

33.5. Vendor extensions – no standard-conformance claim

- Vendor TCODEs 56 (BranchPredict), 57 (JumpTargetCache) and 58 (Config) occupy the N-Trace vendor TCODE range; no standard-conformance claim is made for them.
- SYNC=4 (sequential instruction counter) is bound to BTM by N-Trace; its use with HTM is a documented Accemic extension (runtime reset 0).
- Sync reasons 12/13 were replaced by the single vendor SYNC=14, with `SyncReqSource` diagnosing the actual origin (CSR, ATB, quota, or the TE register, [Chapter 32](#)); the standard-reserved codes 12/13 are not emitted.

Chapter 34. Development Support

This chapter describes the internal SystemVerilog testbench patterns, the simulation verification infrastructure, and the recommended development workflow for CEDARtools.TraceEncoder (CTTE). It complements the Verification chapters (Part F), which describe test **strategy** and **coverage**; this chapter describes the **tooling** those tests are built from.

34.1. Repository layout and build flow

The build driver is **abc** ([abc-flow](#)): Vivado™-based, text-defined projects. Tool versions are pinned in `.abc.config`. Every umbrella command routes through the `Makefile`:

Target	What it does
<code>make help</code>	List available targets.
<code>make sim</code>	Run all testbenches; print a PASS/FAIL summary.
<code>make sim-<name></code>	Run one testbench (e.g. <code>sim-basic</code>).
<code>make sim-examples</code>	Run every simulable example under <code>examples/</code> .
<code>make lint</code>	The host guards, then <code>verible-verilog-lint</code> over <code>rtl/</code> and <code>tests/</code> .
<code>make clean</code>	Remove the (gitignored) <code>bld/</code> directory.
<code>make rdl</code>	Regenerate the PeakRDL-derived SystemVerilog from <code>rdl/*.rdl</code> .
<code>make rdl-html</code>	Render the browsable HTML register reference into <code>bld/rdl-html/</code> .
<code>make bld-bootstrap</code>	Generate the xsim projects the <code>cli_*</code> gates clone.
<code>make bld-refresh</code>	The same, but regenerate them even if they exist.

RTL is organized under `rtl/`: top-level encoder modules directly in `rtl/` (`ct_encoder.sv`, `ct_L1_funnel.sv`, `ct_L2_nexus_formatter.sv`, `ct_L2_mseo_mdo_formatter.sv`, ...), `rtl/pkg/` for type and parameter packages, `rtl/preproc/` and `rtl/mseo_mdo/` for the respective sub-pipelines, and `rtl/external/` for vendored infrastructure shared across sibling Accemic repositories (AMBA interfaces, Wishbone, stream primitives, the `tt` test package). Each subsystem carries its own `test/` subdirectory for the unit testbenches that exercise it (`rtl/preproc/test/`, `rtl/mseo_mdo/test/`, `rtl/test/`); the top-level `tests/` directory is reserved for high-level / system / integration tests that span multiple modules (`tests/instruction/`, `tests/data/`, `tests/hsi/`, `tests/overflow/`, `tests/combined/`, and shared infrastructure in `tests/lib/`).

34.1.1. Starting from a fresh clone or worktree

`bld/` is gitignored, and every `cli_*_test.sh` gate drives `xvlog/xelab/xsim` by hand over an xsim project that `abc` generates below it. The gates bootstrap the project they need (`ct_need_prj` in `scripts/ct_env.sh`), so a fresh worktree runs them without a separate warm-up step; the first gate to need a given project pays roughly a minute for it.

Three outcomes are reported distinctly:

Exit / line	Meaning
77 → SKIP	The gate needs an input this repository does not ship (a captured hardware run, a decoder build with <code>-decoe</code>). Not a pass, not a failure.
78 → TOOL	The toolchain or the build tree could not be established (no Vivado, no <code>abc</code> , no working Python, a bootstrap that did not produce a project). Never a statement about the encoder.
anything else → FAIL	The gate ran and its check failed.

A byte-neutrality run whose legs did not all complete reports `BYTE_NEUTRALITY: INCONCLUSIVE` rather than `DRIFT` — an incomplete manifest cannot claim byte identity either way.

34.1.2. Simulation backend

`.abc.config pins sim_backend=verilator` — the Vivado-free default that `make sim` and the coverage runs use. The `cli_*` gate battery needs an `xsim` project, so those scripts request the Vivado backend explicitly (`abc --sim-backend vivado -sim ...`) rather than depending on the repository-wide setting.

34.1.3. Toolchain discovery

Scripts under `scripts/` and `ci/` source `scripts/ct_env.sh`, which resolves the toolchain without machine-specific paths: `CT_VIVADO_BIN` / `CT_VIVADO_ANY`, `CT_ABC_BIN`, `CT_ABC_TIMEOUT` (seconds a bootstrap `abc -sim` may take; default 900), `CT_PRJ_REFRESH`, `CT_PYTHON_BIN` (resolved via `ct_need_python`, because a fresh Windows shell's bare `python` / `python3` are often Microsoft Store stubs), `NEXRV`, `VERILATOR_ROOT`, and the replay-only `CT_REPLAY_PCINFO` / `NEXRV_DECOE` (without these the two hardware-replay gates skip with exit 77 rather than failing).

34.2. Register (RDL) flow

SystemRDL is the source of truth for the register map (`rdl/`). The committed PeakRDL-derived SystemVerilog under `rtl/pkg/` (and the Wishbone helper under `tests/lib/`) is generated from it and must never be hand-edited:

1. Edit `rdl/ct_cs_cpuif.rdl`.
2. Run `make rdl` to regenerate `rtl/pkg/ct_cs_cpuif.sv`, `rtl/pkg/ct_cs_cpuif_pkg.sv`, `rtl/pkg/ct_cs_cpuif_wb_pkg.sv`, `rtl/pkg/ct_cs_cpuif_types_pkg.sv`, and `tests/lib/ct_cs_cpuif_wb_helper.sv`. The Wishbone↔CPUIF adapter `rtl/pkg/ct_cs_cpuif_wb.sv` is hand-written and not regenerated.
3. Commit the RDL change and the regenerated SystemVerilog in the same commit — CI fails a PR whose generated SV is stale.
4. Run `make rdl-html` to render the browsable register reference into `bld/rdl-html/` (the bare encoder map, and the example SoC's KV260 app map with absolute `devmem` addresses).

`make rdl` provisions a pinned, local toolchain (`peakrdl==1.5.0`, `peakrdl-regblock==1.3.1`, `peakrdl-html==2.12.2`, `systemrdl-compiler==1.32.2`, `per rdl/requirements.txt`) into a gitignored `.venv-rdl*/`; a linked git worktree resolves the main checkout's venv via `git rev-parse --git-common-dir (scripts/gen_rdl_profile.py)`. If no pinned venv is found, the flow refuses to fall back to a `peakrdl` found on `PATH`—an unpinned install has silently produced a `regblock` without the `addrmap localparams` in the past, which fails simulation hours later with an error that does not point at the real cause.

34.3. Testbench architecture

Two related, queue-based patterns are in use, depending on scope.

34.3.1. Per-module: directed queue-based scoreboarding

Per-module unit testbenches under `rtl/<subsystem>/test/` use a lightweight, directed, queue-based scoreboard pattern. The pattern recurs across many testbenches, e.g.

`rtl/preproc/test/ct_L23_preproc_tb.sv` and
`rtl/preproc/test/ct_L23_preproc_comp_filters_tb.sv`.

Queue-based scoreboard provides:

- deterministic, directed tests (no hidden randomness);
- decoupling of stimulus generation from checking;
- clear "what was expected" bookkeeping even with variable pipeline latency.

Core components, as implemented in

`rtl/preproc/test/ct_L23_preproc_comp_filters_tb.sv`:

1. **Test item struct**—test data as a packed struct plus metadata (description, test ID, expected flags).
2. **Input queue**—stimulus is generated once and pushed into `input_queue[$]`.
3. **Expected queue(s)**—expected results are pushed into one or more `expected_queue[$]`; a multi-sink DUT (e.g. one that drives both a control-flow and a data-flow filter) uses separate queues per sink.
4. **Feeding task**—pops from the input queue and drives the DUT.
5. **Checking task(s)**—wait for valid DUT outputs, pop the matching expected item, and compare, e.g.:

```

1 if (cf_filter.hit_valid) begin
2     exp = expected_queue.pop_front();
3     void'(tt_assert(cf_filter.hit == exp.cf_hit,
4         $sformatf("%0.2f: Line %0d / Test %0d (%s) mismatch",
5             $realtime, `__LINE__, exp.test_id, exp.desc)));
6 end

```

6. **Parallel execution** – feeding and checking commonly run concurrently via `fork ... join` plus a `FeedingDone && CheckingDone wait`.

Pitfalls: guard every queue pop with a `size() > 0` check; if the DUT can emit multiple outputs per cycle, the checker must pop the matching number of expected items; a testbench that never calls `tt_assert(...)` anywhere gets flagged by `tt_evaluate()` as having checked nothing (a silent pass is not allowed to look like a real one).

34.3.2. System level: scripted CPU model with an oracle event stream

High-level / system tests under `tests/` (`tests/instruction/`, `tests/data/`, `tests/hsi/`, `tests/overflow/`, `tests/combined/`) are driven by a different, complementary mechanism: a scripted RISC-V CPU model, `tests/lib/cpu_model.sv`, that mimics a core via a task-based API (`enter / run / branch_taken / branch_not_taken / jump_to / uninferable_jump / call / ret / interrupt / load / store / csr_write / ...`). Each task both drives the encoder-facing TIP signals with the correct `itype/iaddr/daddr/...` and logs a high-level event into a sideband `event_q` that the scoreboard cross-checks against the encoder's decoded output. No binary program or separate stimulus file is needed – the test's task calls are the program; there is no binary-trace player in this suite.

A test instantiates the shared `tests/lib/ct_env.sv`, which wires `cpu_model`, the DUT, and the trace sinks together, drives the scenario through the `cpu_model` task API, then writes the expected.* reference files that `scripts/decode_and_check.sh` diffs against the CTTD decode (see [Section 34.6](#)). Test directories are named `NN_<scenario>/` with a numeric prefix so tests sort simple to sophisticated; each has a `<scenario>_tb.sv` + `<scenario>_tb.abv` pair.

34.3.3. tt assertions (`rtl/external/testtools/tt.sv`)

The `tt` package (`rtl/external/testtools/tt.sv`) is the shared simulation-only assertion helper used by both testbench styles above (it is not synthesizable – string operations are used throughout). Import with `import tt::*;`

- `tt_assert(cond, msg)` increments the assertion counters; on failure it stores the message in an internal error queue and calls `$error(msg)` (optionally `$stop` if stop-on-error is enabled). Typical use:

```
1 void'(tt_assert(dut_out == expected,
2   $sformatf("%0.2f: Line %0d / Test %0d: ...", $realtime, `__LINE__, test_id)));
```

- `tt_evaluate()`, called at the end of simulation, prints a concise summary: the list of testcases with pass/fail status, the number of checked assertions, and any captured error messages. A testbench always terminates with `tt_evaluate(); $finish();`.

34.4. Simulation helper modules

Reusable, simulation-only infrastructure modules, as they exist in the repository today:

Module	Location	Purpose
TIP dumper (tip_dump)	tests/lib/tip_dump.sv	Dumps the TIP signal stream to a file after flush, in RAW, PCINFO (address, type, <dest>), or PLAIN (per-address branch-kind counts) mode.
AXIS dump, DAQ-specific (ct_axis_dump)	tests/lib/ct_axis_dump.sv	Simulation-only dump of CTTE's DAQ AXI-Stream transactions. Writes both a text dump and a fixed-record-size binary raw dump (<CT_DATE_TIME>_<realtime>_ct_axis.{txt, raw}); unconditional – the instance presence is the gate.
AXIS dump, generic (axis_dump)	rtl/external/amba/test/axis_dump.sv	Lower-level, RAW-only AXI-Stream dumper shared across Accemic AMBA testbenches (not CTTE-DAQ-specific).
AXIS decoder (ct_axis_decoder)	tests/lib/ct_axis_decoder.sv	Converts one AXIS beat into a structured record (ct_axis_decoder_pkg::ct_axis_msg_t) so a testbench can assert on fields without manually slicing tdata/tid/tstrb.
ATB dump (atb_dump)	rtl/external/amba/test/atb_dump.sv	Writes accepted ATB beats (atvalid && atready) to a binary file; prints the final path and byte count at end of simulation even if no flush occurred.
ATB sink model (atb_sink_model)	rtl/external/amba/test/atb_sink_model.sv	Minimal ATB consumer with deterministic stalls: releases atready after STARTUP_STALL_CYCLES, then injects stall windows of 0..MAX_STALL_CYCLES drawn from an LFSR (reproducible under Verilator and across a simulation resume, unlike \$random).
ATB stall injector (atb_stall_injector)	rtl/external/amba/test/atb_stall_injector.sv	ATB pass-through that injects Aurora-like bursty backpressure between an ATB master and slave, to emulate SerDes-link flow control. Sizing rule: average sink rate >= average master rate, and STALL_PERIOD several times larger than STALL_LENGTH_MAX (enforced by an elaboration check).
Overflow injector (ovf_injector)	rtl/external/stream/ovf_injector.sv	Counted-vector stream overflow marker injector: forwards input to output while there is room; on downstream full (or a driven force_inject) it discards concurrent input, counts the discards, and once room exists emits one or two configured marker elements. A SEQ_INJECT parameter forces the two markers into separate cycles when the downstream sink is single-element-per-beat wide – combining them into one beat silently truncates and loses a marker (found 2026-07-20 via the serialized eTIP path).

34.5. Long-running simulations: watchdog policy

Two independent timeout mechanisms exist today, at different levels:

- **Per-testbench, simulation-time watchdogs.** Several testbenches arm their own \$fatal-based watchdog that fires after a fixed amount of **simulated** time or a fixed cycle count if the DUT

never reaches the expected state — e.g.

`rtl/preproc/test/ct_L23_preproc_sync_tb.sv:181-186` (WATCHDOG_TIMEOUT_NS, sized to roughly two orders of magnitude above the longest legitimate wait in that test) and `rtl/mseo_mdo/test/ct_L2_mseo_mdo_formatter_tb.sv:288-291` (MAX_SIM_CYCLES). This is a simulation-content watchdog: it protects against the DUT hanging or a testbench waiting on an event that never comes, independent of wall-clock runtime.

- **Bootstrap / compile timeout.** `CT_ABC_TIMEOUT` (default 900 s, see [Section 34.1.1](#)) bounds how long a `bld/-project bootstrap abc -sim` may take at the tool level.

34.6. Recommended workflow

1. Implement or change RTL, and regenerate every register model the change touches in the same commit: `make rdl` for `rdl/ct_cs_cpuif.rdl`, `make rdl-soc` for `examples/kv260/common/tgc5b/rdl/ct_soc.rdl`, and `make rdl-kv260` for the demonstrator maps under `examples/kv260/`. Skipping the last one leaves the KV260 address map behind, which the drift gate of the documentation build then reports rather than the build that caused it.
2. Update or extend the corresponding testbench(es) if behavior changed.
3. Run the relevant testbench(es): `make sim-<name>` for a single **system** scenario under `tests/`, `make sim` for the full suite, `make sim-examples` for the worked integrations. The unit testbenches of a subsystem are not `sim-*` targets — they live beside their RTL in `rtl/<subsystem>/test/` and are driven from there.
4. For an end-to-end / system test, use `scripts/decode_and_check.sh` to check the CTTD decode of the captured ATB stream against the test's `expected.*` reference. The script decodes once and can then run any combination of: `--pc` (exact PC-stream match, or `--soft` for a warn-only prefix check), `--data` (data-access sequence), `--ctxp` (value-aware CTXP export match), `--tsmono` (timestamp monotonicity), `--sync N / --hist N` (minimum synchronization / branch-history message counts), `--disabled` (trace-off correlation message present), and `--overflow` (a Nexus Error / QueueOverrun message was actually emitted). With no flag, `--pc` is assumed.
5. Ensure the testbench ends with `tt_evaluate(); $finish();`.
6. Run `make lint` (Verible; opt-in ruleset, `.rules.verible_lint`) before submitting.
7. Every new file carries an SPDX header; contribution and CLA rules are covered in [Chapter 37](#).

Chapter 35. Demonstrator Register Maps

The demonstrator chapter [Chapter 28](#) describes what each example design does; this appendix states which registers drive them. It exists because the reference manual is meant to be the single source for every SystemRDL register description that CEDARtools.TraceEncoder ships, and the example designs bring their own – the control window a host uses to start a core and drain a trace buffer is not part of the encoder map in [Chapter 16](#), and reading a bare offset out of a prose sentence is not the same as reading a register with its access type and its reset value.

The tables here are generated from the same SystemRDL sources the example RTL is built from, by the same generator and under the same drift gate as the encoder tables. What is written by hand is the prose: which map applies to which design, how far each one matches the hardware as built, and where a value comes from that the table cannot state.

35.1. Scope: which SystemRDL source appears where

Eleven SystemRDL files live in the encoder repository, outside its build tree (the per-profile copies under `build/` are build output, not sources). They serve three different purposes, and this manual treats them accordingly:

SystemRDL source	Printed	Why
<code>rdl/ct_cs_cpuif.rdl</code> (with <code>rdl/ct_profile.inc.rdl</code>)	Section 16.7	The encoder control and status registers: the product interface, and the map every integration programs.
<code>examples/kv260/common/rdl/ct_kv260_ctrl.rdl</code> (with <code>ct_kv260_ctrl/ct_kv260_common.rdl</code> and <code>ct_kv260_ctrl/ct_kv260_sinks.rdl</code>)	Section 35.3	The control window every KV260 example exposes to the Zynq UltraScale+ processing system: core run control, capture counters, and the three trace sinks.
<code>examples/kv260/common/tgc5b/rdl/ct_soc.rdl</code> (with <code>ct_soc/ct_ps_ctrl.rdl</code>)	Section 35.4	The address spaces of the TGC5B example SoC, and the registers of its processing-system control port.
<code>examples/kv260/common/tgc5b/rdl/ct_soc/ct_clint.rdl</code> , ... <code>/ct_intc.rdl</code>	referenced, not reprinted – Section 35.7	Two minimal interrupt peripherals of the example SoC. Seven registers whose names follow an established convention and whose behaviour the example needs but the encoder does not; a transcription here would suggest they are part of the trace subsystem.
<code>examples/kv260/tgc5b2_axis_wp/rdl/ct_tgc5b2_kv260.rdl</code>	Section 35.5	The address map of the delivered two-core device as the processing system sees it: both cores, both encoder instances, the core control block, the merged trace ring and the two stream FIFOs the watchpoint records are drained from. This is the map a reader holding the KV260 bundle is looking for, which is why it is printed rather than referenced.

SystemRDL source	Printed	Why
examples/kv260/tgc5b2_rvcfi/rdl/ct_tgc5b2_rvcfi_kv260.rdl	Section 35.6	The address maps of the runtime-verification testbed, including the banks this manual could not print before that SystemRDL existed: the read-only observation bank, the per-core console, the two DDR record rings and the core-side map with the ACT-CAP doorbell.

Two limits of this appendix are worth stating before the tables rather than after them.

These maps are examples, not product interfaces. The interface-stability rule that governs the encoder map — additive change preferred, a breaking change justified in writing — does not extend to the example designs. An example is free to move a register when its design changes, and several have; [Section 35.3](#) says how far.

A register map is not a driver. The sequence in which these registers are written, and the traps that sequence holds, are in [Section 28.7](#). This appendix answers what a register is, not when to write it.

35.2. Reading the generated tables

Each table is produced by `scripts/gen_ctte_regs.py` in this documentation repository, one fragment per SystemRDL top:

```
1 python3 scripts/gen_ctte_regs.py --list # the sources and their fragments
2 python3 scripts/gen_ctte_regs.py      # regenerate all of them
3 make check-generated                  # the gate the documentation build runs
```

Four conventions apply to every table below. They are the encoder map's conventions as well, and not by agreement: the same generator writes both, so a column means the same thing here as it does in [Chapter 16](#).

- **Offsets.** Each table states what its offset column is relative to. The distinction matters in this appendix more than in the encoder chapter, because the TGC5B map is printed in four views of one design — the same register appears at a physical address in one and at an aperture-relative offset in the next.
- **Access.** `RW` and `RO` are the software access the SystemRDL declares. A parenthesised qualifier is the property that changes what a write or a read does: `hw-written` marks a status field the hardware drives, `self-clearing` a write-one pulse that never reads back as set.
- **Reset.** The value the SystemRDL declares. Where a top instantiates the block with a different elaboration parameter, the table shows the declared default and the description says `so-sinks.ddr_base` is the case that matters.
- **Provenance.** Each generated fragment carries, in its header comment, the SHA-256 of every SystemRDL file that fed it. A reader holding a printed manual and a checkout can therefore establish whether the two describe the same hardware, which is the whole purpose of generating these tables instead of typing them.

35.3. The unified KV260 control window

Every KV260 example places a control window at the base of its own AXI4-Lite aperture, and the processing system reaches that aperture at `0xA000_0000`. The window is how a Linux host on the Zynq UltraScale+ processing system starts and stops the traced core, re-arms the capture path, reads how much trace has accumulated, and configures the three trace sinks — all with `devmem`, without a custom driver.

The SystemRDL below is the **target** layout for that window, and the distinction is not academic. The first `0x38` bytes are intended to be identical across all nine example tops. Measured against the tops as built, five of the nine diverge somewhere in that range, and the divergent tops were deliberately not retrofitted when the layout was fixed (`examples/kv260/common/rdl/README.md`, section "As-built drift", carries the per-top measurement). A reader working on one specific design therefore verifies the offsets against that design's own module header; a reader writing new host code against the shared layout uses the table below.

Three properties of this window come from its history and are invisible in the table:

The two halves have different standing. Offsets `0x00--0x14` (**common**) **document** registers that each top still implements in its own AXI4-Lite front-end — there is no shared module behind `CONTROL` and `STATUS`. Offsets `0x18--0x38` (**sinks**) are transcribed one to one from `examples/kv260/common/ct_trace_sinks.sv`, a single shared module that three tops instantiate verbatim. The sink half describes hardware that exists once; the common half describes an agreement between several implementations of the same thing.

Only three bits are universal. An inventory across the nine as-built tops found `CONTROL.core_run`, `CONTROL.trace_clear` and `STATUS` bit 0 identical in position and meaning everywhere. Every other bit differs by design — per-core run bits, console clear, window-guard clear, interrupt-generator enable — and is therefore modelled as one undifferentiated passthrough field (`top_ctrl`, `top_status`) rather than as named bits. Naming a bit that means something else on the next top would misdocument the hardware, which is worse than leaving the field open.

One offset is deliberately absent. Local `0x1C` of the sink block, absolute `0x34`, carries no register in the shared map: the sink module reserves it for the instantiating top, where it is a funnel control register today. A gap in the offset column is that reservation, not an omission.

The unified CTRL window of the KV260 example designs, as the SystemRDL describes it. Offsets are relative to the base of the window; every example top places that base at the start of its own AXI4-Lite aperture. Register and field descriptions in the tables below are reproduced verbatim from that SystemRDL, which is Accemic's own text; the column heading names it as the source.

35.3.1. Component overview

Base	Component	Registers
0	common	6

Base	Component	Registers
0x18	sinks	8

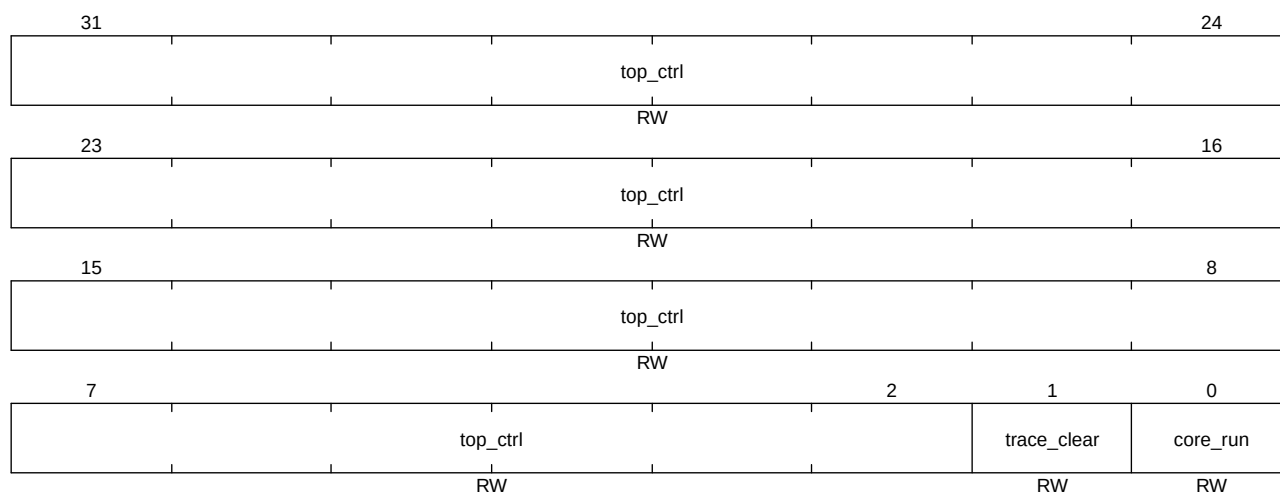
35.3.2. Component common (base 0)

KV260 example SoC - common CTRL registers (CONTROL..TRACE_BUFSZ)

Offset	Register	Fields
0x000	<code>common.control</code>	3
0x004	<code>common.status</code>	2
0x008	<code>common.trace_beats</code>	1
0x00C	<code>common.trace_bytes</code>	1
0x010	<code>common.axis_beats</code>	1
0x014	<code>common.trace_bufsz</code>	1

`common.control` – offset 0x000

Board-level control, written over devmem to drive the SoC. Bits 2..31 are top-specific – see `top_ctrl` and the owning top's own module header (e.g. b2 is `trace_flush` on `mbv/duo/trio/tgc5b2_axis_wp` but `con_clear` on the `cva6_linux/cva6_linux64/rocket_linux/rocket2/cva6_2` family; README.md 'CONTROL b2' finding).

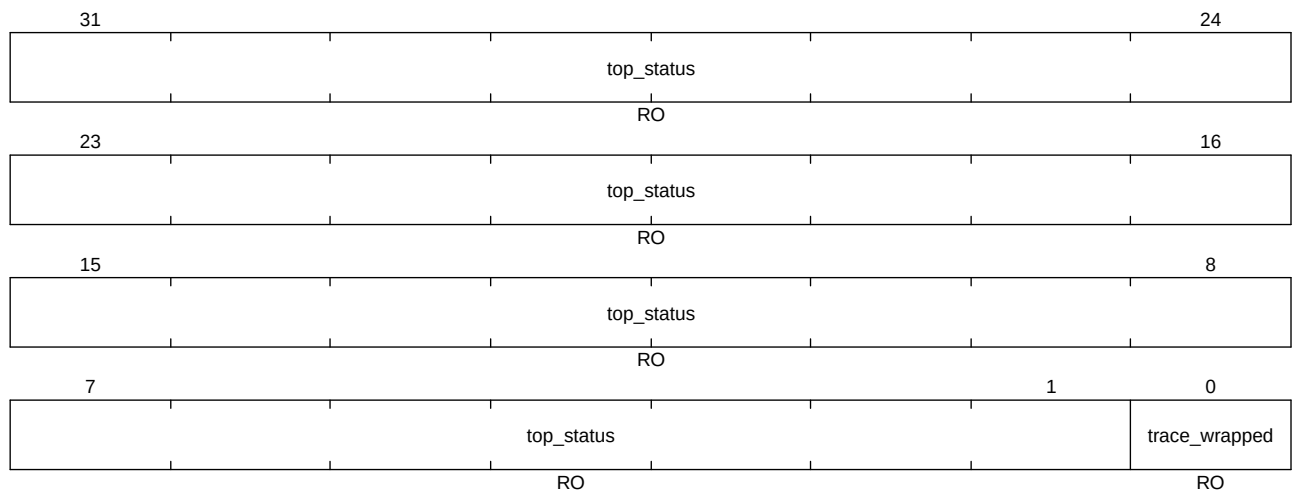


Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:2	<code>top_ctrl</code>	RW	0	Top-specific control bits (b2..b31): <code>trace_flush</code> , <code>con_clear</code> , <code>win_err_clear</code> , <code>obs_clear</code> , <code>irq_gen_en</code> , per-core run bits (<code>core0_run/core1_run/cva6_run/cva6_run2</code>), etc. – see the owning top's header. NOT wired to any hardware behaviour by this shared regblock; a future per-top instantiation drives its own subset of these bits itself.

Bits	Field	Access	Reset	Description (SystemRDL of the example)
1	trace_clear	RW	0	Level: while set, re-arm the trace capture path (ring, and where present AXIS/sink counters). Universal across every observed top.
0	core_run	RW	0	0 = hold the core(s) in reset, 1 = run. On multi-core tops this is the COLLECTIVE bit (b0 OR each top's own per-core run bit in top_ctrl); on single-core tops it is the only run bit there is.

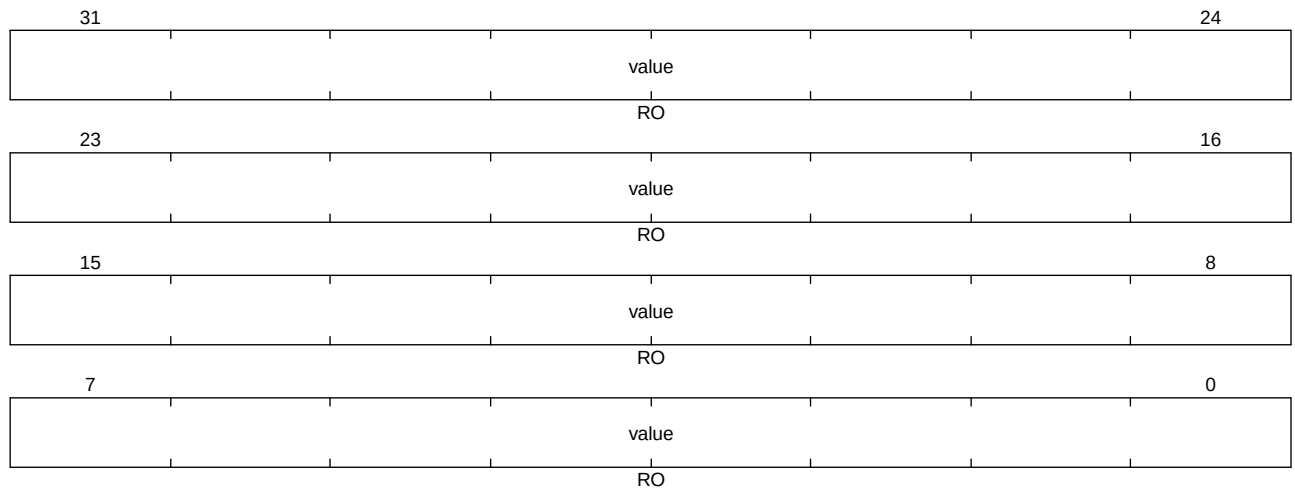
common.status – offset 0x004

Board-level status (read-only). Bits 1..31 are top-specific – see top_status and the owning top's own module header (e.g. b1 is axis_overflow on mbv/duo/trio but uram_stopped on the cva6_linux/cva6_linux64/rocket_linux/rocket2/cva6_2 family; README.md 'STATUS b1' finding).



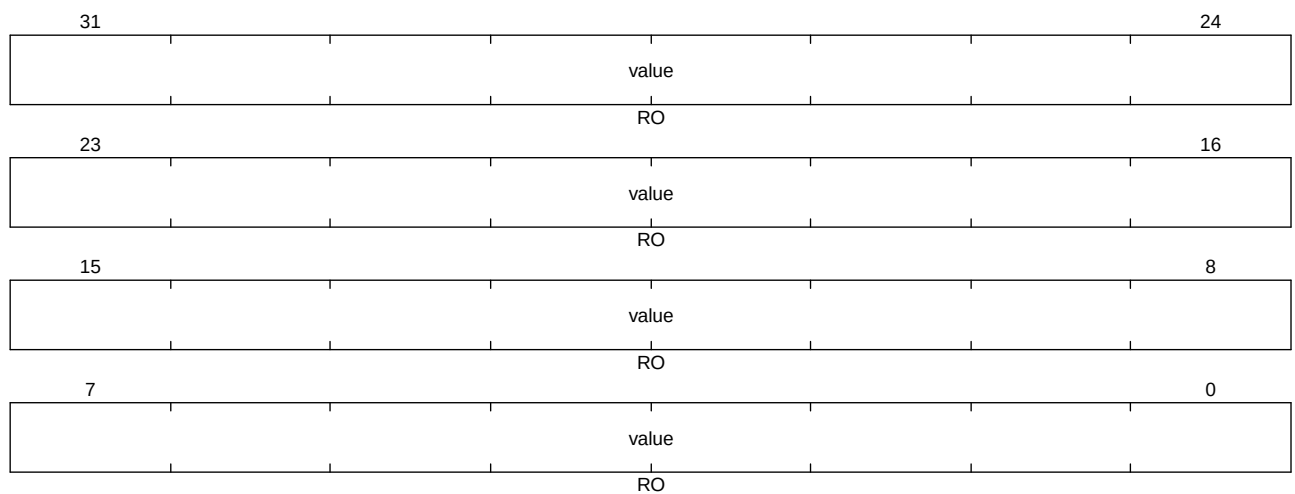
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:1	top_status	RO (hw-written)	0	Top-specific status bits (b1..b31): axis_overflow, uram_stopped, win_err_sticky(+_was_write), core_ndreset/core_rst_hold, per-core running mirrors, observation-channel stickies, last-seen privilege level, timer_irq/sw_irq, etc. – see the owning top's header.
0	trace_wrapped	RO (hw-written)	0	1: the trace ring/capture buffer has wrapped or overflowed at least once since the last trace_clear (naming differs – trace_wrapped on most tops, trace_overflow on duo/trio/tgc5b2_axis_wp – the condition is the same: capture continues, nothing stalls, the oldest beats are gone). Universal across every observed top.

common.trace_beats – offset 0x008



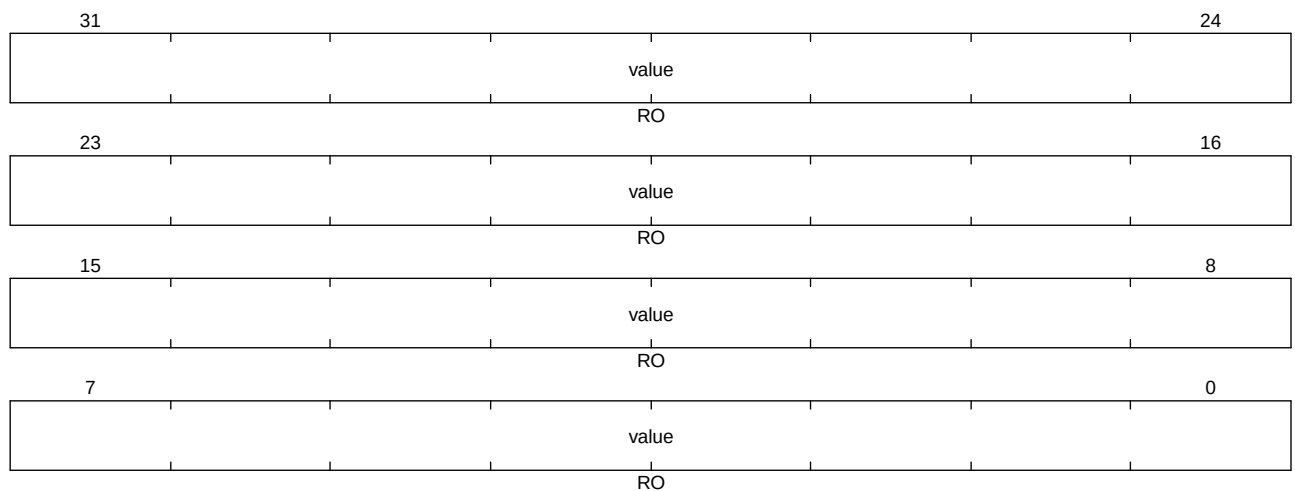
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	ATB beats captured since the last trace_clear (monotonic, does NOT stop at the ring capacity – read TRACE_BUFSZ for the capacity). Ring write position = TRACE_BEATS % (TRACE_BUFSZ/4). Identical offset and meaning on every observed top.

`common.trace_bytes` – offset 0x00C



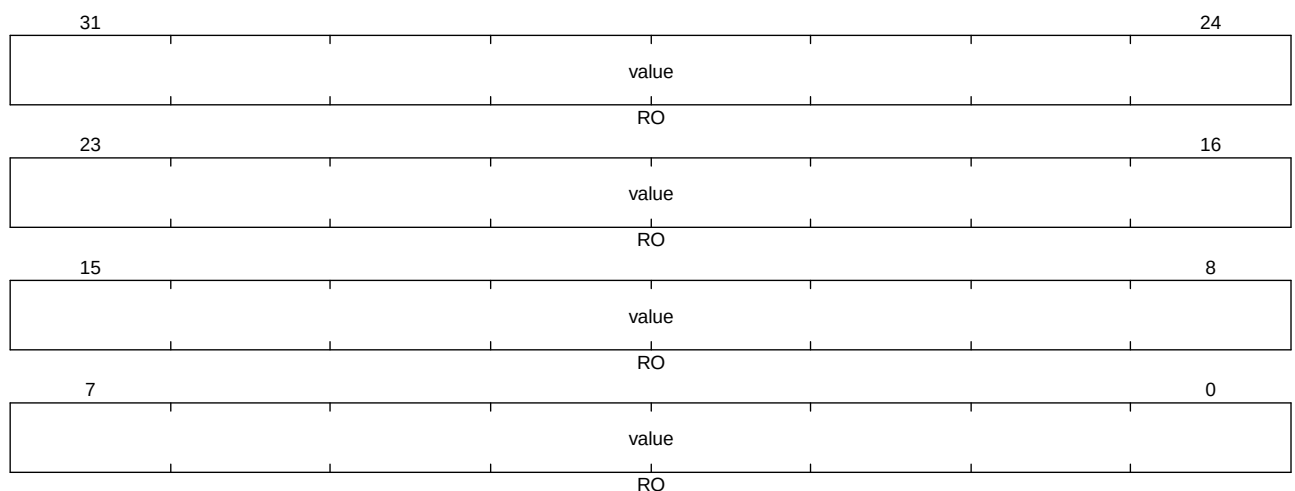
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	ATB bytes captured since the last trace_clear (monotonic). Poll delta = live trace bandwidth. Identical offset and meaning on every observed top.

`common.axis_beats` – offset 0x010



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	AXIS instrumentation beats captured (read-only). Populated only on tops with an AXIS capture path (mbv, duo, trio, per README.md); reads 0 on every other top, consistent with the 'outside the window reads 0' convention <code>ct_trace_sinks.sv</code> already establishes for its own CTRL indices. This is the canonical slot for <code>AXIS_BEATS</code> in the unified layout – see README.md for the per-top offset this displaces (<code>AXIS_BEATS</code> was already at this offset on <code>mbv/duo/trio</code> ; <code>TRACE_BUFSZ</code> moves off of it on the tops that used to park it here, see @0x14 below).

`common.trace_bufsz` – offset 0x014



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Trace ring capacity in bytes (read-only; <code>TRACE_DEPTH * 4</code>). Identical MEANING on every observed top; this is the canonical offset (0x10 <code>TRACE_BUFSZ</code> on six of the nine tops already; <code>AXIS_BEATS</code> moves off of 0x10 on the other three – see README.md).

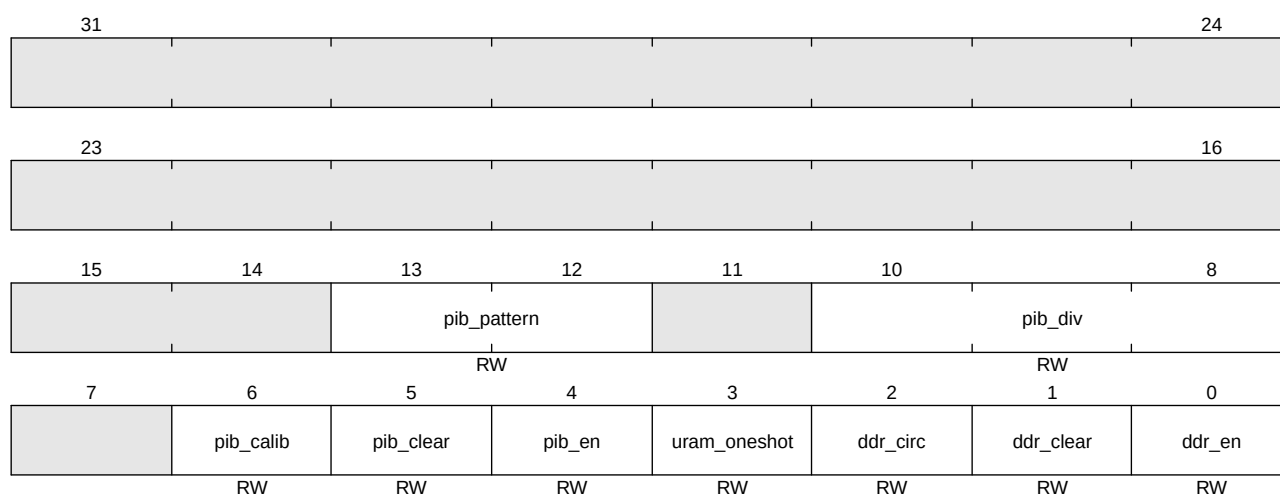
35.3.3. Component sinks (base 0x18)

KV260 example SoC - shared three-sink subsystem CTRL window

Offset	Register	Fields
0x018	<code>sinks.sink_ctrl</code>	9
0x01C	<code>sinks.ddd_base</code>	1
0x020	<code>sinks.ddd_size</code>	1
0x024	<code>sinks.ddd_wptr</code>	1
0x028	<code>sinks.sink_stat</code>	5
0x02C	<code>sinks.ddd_drops</code>	1
0x030	<code>sinks.pib_drops</code>	1
0x038	<code>sinks.ddd_beats</code>	1

`sinks.sink_ctrl` – offset 0x018

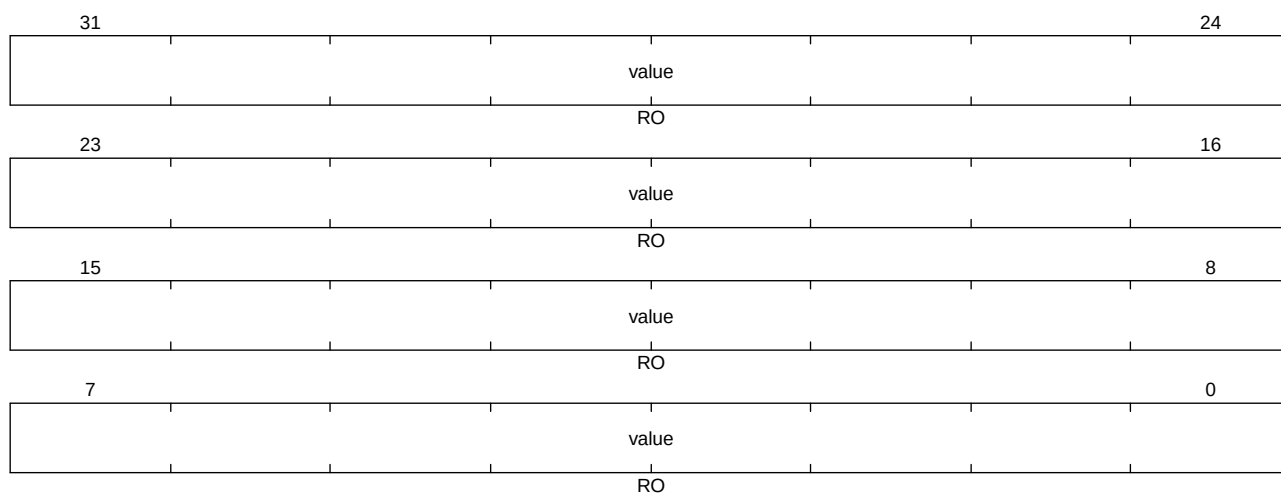
Trace sink enable/config. Stored bits use write mask 0xFFFF_FFDD (everything except the two write-1 pulses); EN_DDR=0 additionally masks b0/b2, EN_PIB=0 masks b4/b6/b[10:8]/b[13:12] (ct_trace_sinks.sv SINK_WR_MASK).



Bits	Field	Access	Reset	Description (SystemRDL of the example)
13:12	<code>pib_pattern</code>	RW	0	PIB calibration pattern (reference <code>trPibCalibPattern</code>). Values: 0 = STANDARD; 1 = MOVING_ONE; 2 = MOVING_ZERO.
10:8	<code>pib_div</code>	RW	0	PIB port clock divider: $\text{pib_clk} = \text{clk} / 2^{(\text{div}+1)}$; 0 is clamped to 1 in the PIB module.
6	<code>pib_calib</code>	RW	0	1: PIB emits the calibration pattern instead of trace (reference <code>trPibCalibrate</code>); the trace input is not consumed while set.

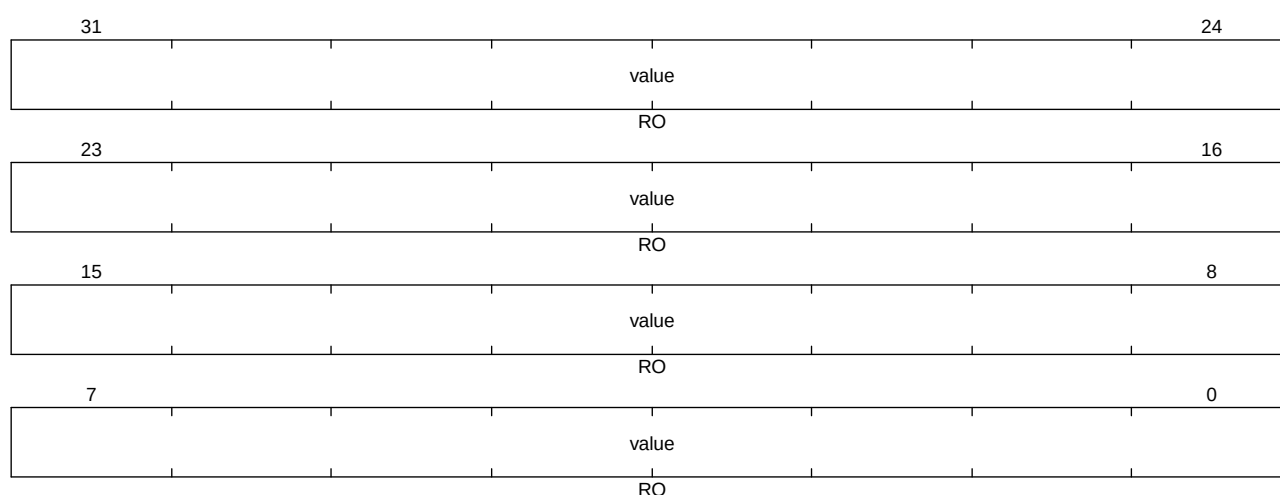
Bits	Field	Access	Reset	Description (SystemRDL of the example)
5	pib_clear	RW	0	Write-1 pulse (captured one cycle after the write, NOT stored – always reads back 0): reset the PIB drop counter. The hardware does not enforce pib_en=0 first (ct_trace_sinks.sv:125,142-146).
4	pib_en	RW	0	1: PIB parallel trace port enabled (pib_clk runs, beats serialized).
3	uram_oneshot	RW	0	URAM ring mode: 0 = circular (default, keeps the most recent capacity), 1 = one shot (stop when full, SINK_STAT.uram_stopped sets).
2	ddr_circ	RW	0	DDR capture mode: 0 = one shot (stop when full), 1 = circular (wrap modulo DDR_SIZE, write offset = DDR_WPTR % DDR_SIZE).
1	ddr_clear	RW	0	Write-1 pulse (captured one cycle after the write, NOT stored in this register – always reads back 0): reset DDR_WPTR/full/wrapped/err/drops and SINK_STAT. ddr_cfg_rej. The hardware does not enforce ddr_en=0 first.
0	ddr_en	RW	0	1: DDR4 sink enabled (writes beats to PS DDR at DDR_BASE).

sinks. ddr_base – offset 0x01C



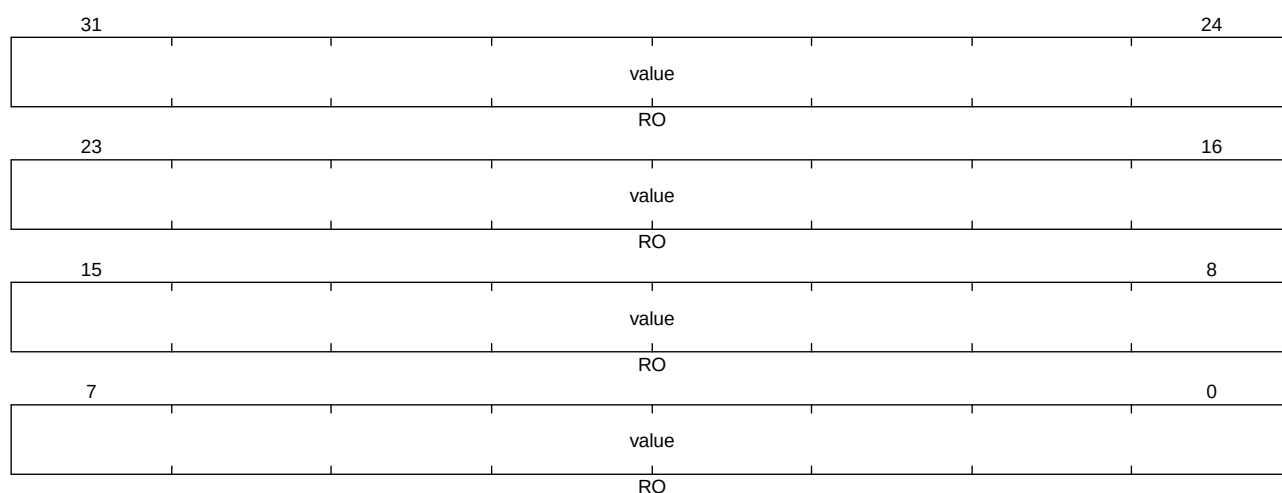
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0x50000000	DDR4 sink base address in PS DDR (byte address, 32-byte aligned – low 5 bits are masked). U9-1 (2026-08-16): hardware-enforced WARL with an EMPTY set of legal software values – the register is hard-wired to the DDR_BASE_RST elaboration parameter for the life of the bitstream, EVERY write is discarded (not just while armed, as the earlier U6 interlock did), and every discarded write latches SINK_STAT.ddy_cfg_rej. To relocate the window: change DDR_BASE_RST, rebuild the bitstream, move the reserved-memory node with it (correspondence rule, ct_trace_sinks.sv header). Reset value shown is the DDR_BASE_RST default (0x5000_0000, address plan v4); tops may instantiate ct_trace_sinks with a different DDR_BASE_RST/DDR_SIZE_RST – read the register, the elaboration parameter is not visible over devmem.

sinks.ddy_size – offset 0x020



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0x10000000	DDR4 sink buffer size in bytes (4-byte aligned – low 2 bits are masked). Same U9-1 hardware-enforced WARL as DDR_BASE: fixed to DDR_SIZE_RST for the life of the bitstream, every write discarded and latched in SINK_STAT.ddy_cfg_rej.

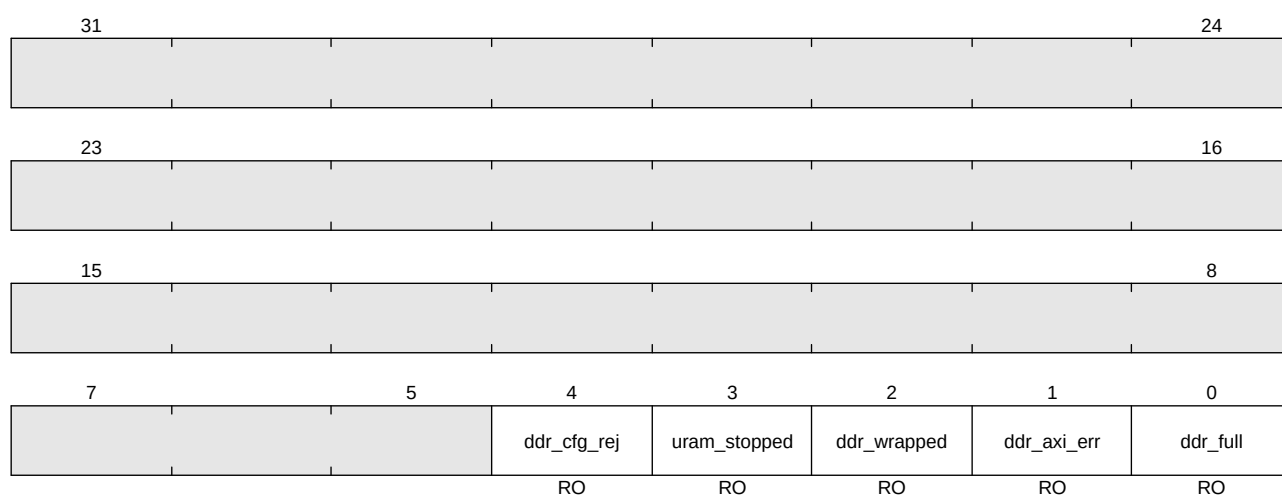
sinks.ddy_wptr – offset 0x024



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	DDR4 sink write pointer: TOTAL bytes written since the last ddr_clear (monotonic, read-only). One shot: host reads [DDR_BASE, DDR_BASE+min(WPTR,SIZE)). Circular: write offset = WPTR % SIZE (same contract as the URAM ring).

sinks.sink_stat – offset 0x028

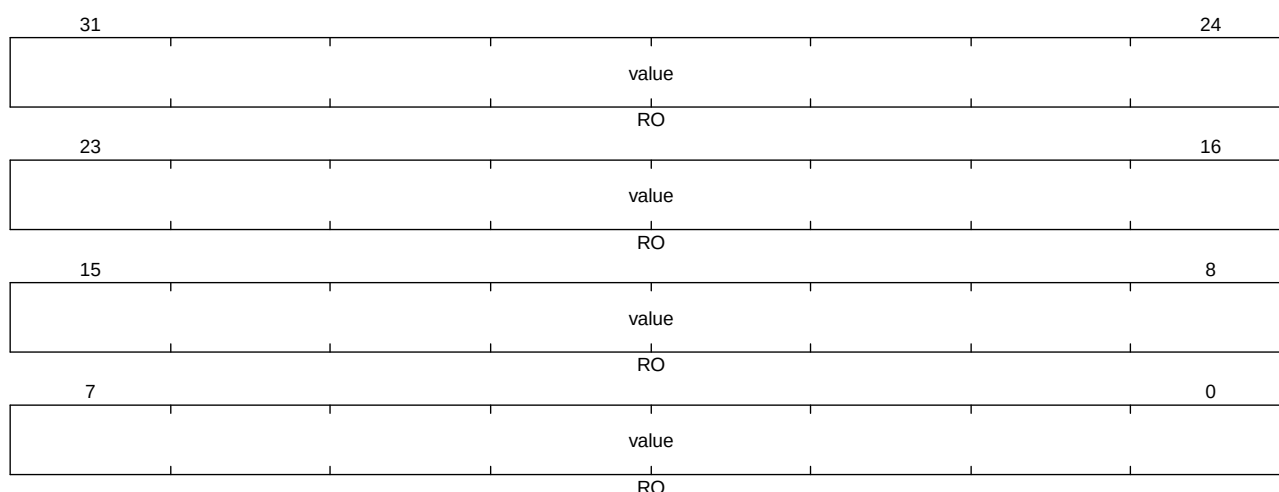
Trace sink status flags (read-only; the DDR bits clear on ddr_clear).



Bits	Field	Access	Reset	Description (SystemRDL of the example)
4	ddr_cfg_rej	RO (hw-written)	0	1: a DDR_BASE/DDR_SIZE write was REFUSED (U9-1: EVERY write is refused, armed or not; sticky until ddr_clear).
3	uram_stopped	RO (hw-written)	0	1: URAM one-shot capture stopped (buffer full, first capacity kept).
2	ddr_wrapped	RO (hw-written)	0	1: circular DDR buffer wrapped at least once (oldest bytes overwritten).

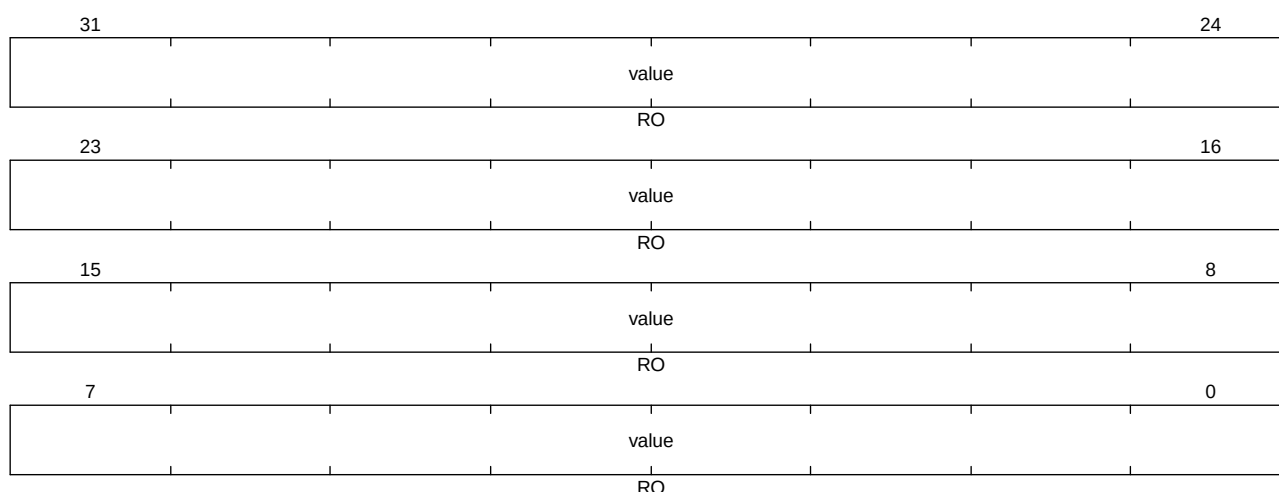
Bits	Field	Access	Reset	Description (SystemRDL of the example)
1	ddr_axi_err	RO (hw-written)	0	1: an AXI write returned BRESP != OKAY (sticky, clears on ddr_clear).
0	ddr_full	RO (hw-written)	0	1: DDR buffer full in one-shot mode; further beats drop.

sinks.ddd_drops – offset 0x02C

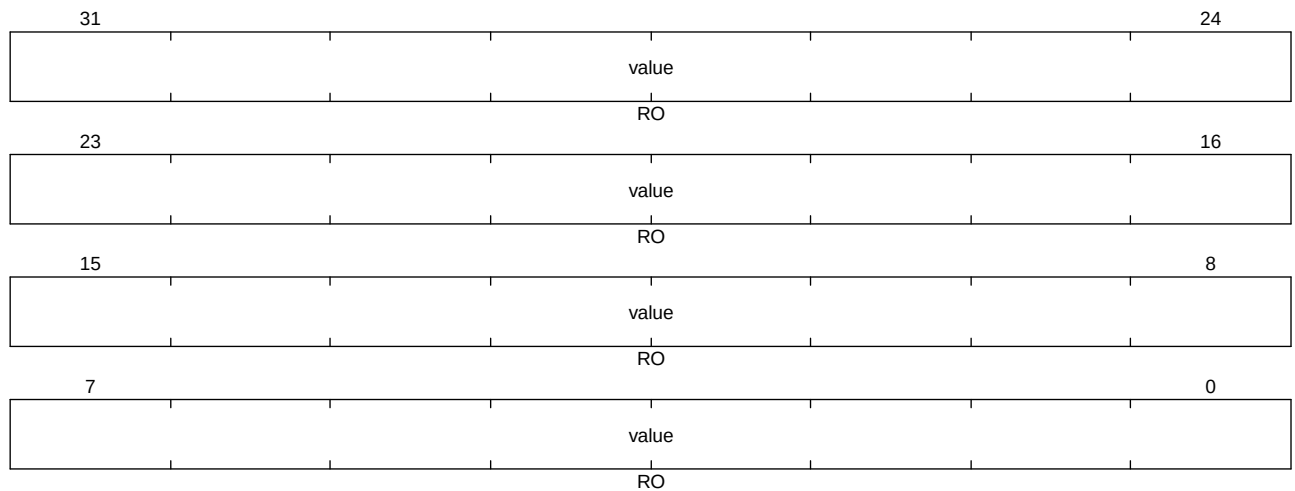


Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	DDR4 sink dropped beats (FIFO overflow or buffer full; saturating; read-only; clears on ddr_clear).

sinks.pib_drops – offset 0x030



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	PIB sink dropped beats (FIFO overflow when the port rate is below the trace rate; saturating; read-only; clears on pib_clear).

sinks.ddd_beats – offset 0x038

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Beats offered to the DDR sink while ddr_en (accepted = this value - DDR_DROPS; cleared by ddr_clear; read-only) – the cheap proof that beats reach the sink at all. T2 addition: this counter sat at CTRL 0x30 on the pre-T2 tgc5b2 COB_DDR build; since the shared three-sink subsystem it is here, 0x38 absolute, and 0x30 is PIB_DROPS.

The two DDR registers deserve one sentence the table cannot carry. `sinks.ddd_base` and `sinks.ddd_size` read as writable in the register model and are not: the hardware discards **every** write and latches `sinks.sink_stat.ddd_cfg_rej` when it does. Both are fixed for the life of the bitstream to the elaboration parameters the top instantiated the sink module with, so the reset values in the table are the defaults of that module and not necessarily what a given board reports. Relocating the DDR window means changing the parameter, rebuilding the bitstream and moving the reserved-memory node with it. Software reads the registers; it cannot negotiate with them.

35.4. The TGC5B example SoC

The TGC5B example SoC is the design that runs both in simulation and on the board, and its register map shows something the encoder chapter cannot show: the encoder is one addressable component inside a larger address space, and where that space puts it is the integrator's choice. The same encoder map appears twice in the tables below – at `0x2000_0000` on the CPU data bus and at `0x0001_0000` in the processing-system control port – and both are the map of [Section 16.7](#), which is why it is not reprinted here.

Three facts about these views are structural, and each one has cost debugging time somewhere:

- **The memory windows are wider than the memories behind them.** The control port selects a region from the top address bits of the aperture, so each window spans 1 MiB while holding 64 KiB, 16 KiB or 4 KiB. A read past the last entry aliases back to entry 0 instead of failing. A host that walks a window to its end therefore sees the buffer repeat, not the end of the data.

- **The program RAM window is gated by the run bit.** The load window is accessible only while `ctrl.control.core_run` is 0. Loading a program into a running core does not fail loudly; it does not arrive.
- **Only one of the four maps becomes RTL.** Only the peripheral block is generated from this SystemRDL. The control port, the data-bus decode and the physical placement are hand-written modules, described here so that the map is complete rather than because a generator consumes them. Keeping the two in step is a manual obligation stated in the SystemRDL itself.

The address spaces of the TGC5B example SoC, and the registers of its PS control port. The encoder registers reachable in these spaces are the map of the register-model chapter and are not reprinted here. Register and field descriptions in the tables below are reproduced verbatim from that SystemRDL, which is Accemic's own text; the column heading names it as the source.

35.4.1. KV260 application view (physical addresses)

Physical-address view on the KV260: the Zynq PS FPD window places the `ct_soc_top` control port at `0xA000_0000` (`M_AXI_HPM0_FPD`).

The offsets in this table are physical addresses: what `devmem` takes on the board.

Offset	Component	Kind	Size	Content
0xA0000000	soc	address map	3076 KiB	TGC5B example SoC - PS control port

35.4.2. PS control port (aperture-relative)

The AXI4-Lite control port `ct_soc_top` presents to the Zynq PS. Offsets are relative to the port aperture (the RTL decodes the low 22 address bits); see `ct_soc_kv260_top` for the absolute `devmem` addresses on the KV260.

Offsets are relative to the base of the control-port aperture.

Offset	Component	Kind	Size	Content
0	ctrl	register file	20 B	<code>ct_soc_top</code> control / status
0x10000	encoder	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers
0x100000	ram	memory	64 KiB	Program RAM load window (<code>ct_soc_ram</code>): 16384 words = 64 KiB (<code>MEM_WORDS</code>). Accessible only while <code>core_run</code> = 0. 16384 entries of 32 bit.
0x200000	trace	memory	16 KiB	Captured ATB read-back (<code>ct_soc_trace_buf</code>): 4096 words = 16 KiB (<code>TRACE_DEPTH</code>), one word per ATB beat. Read <code>TRACE_BEATS</code> words and keep the first <code>TRACE_BYTES</code> bytes. 4096 entries of 32 bit.

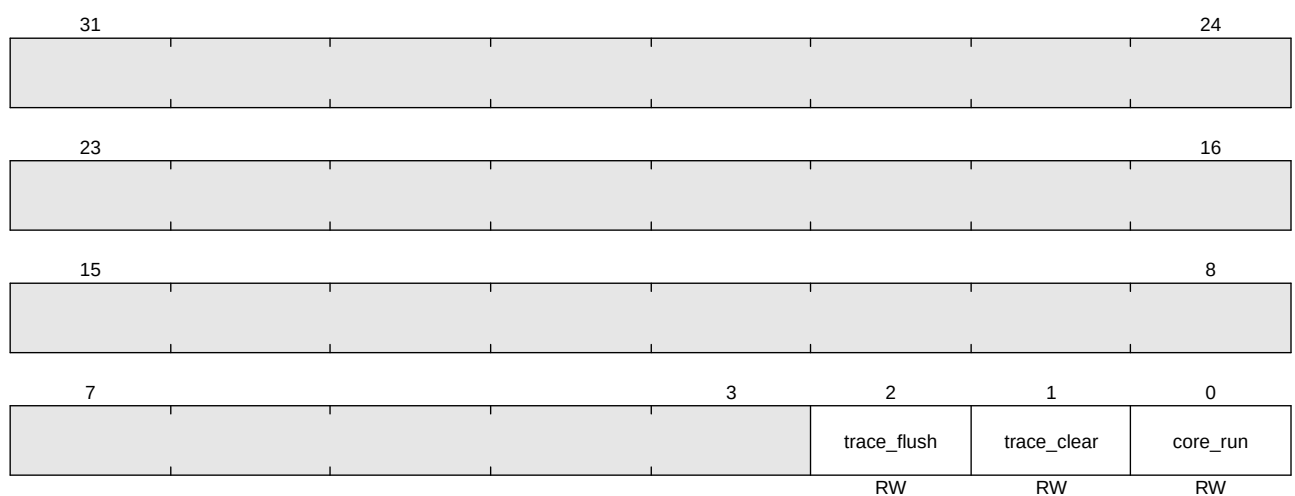
Offset	Component	Kind	Size	Content
0x300000	axis	memory	4 KiB	Captured AXIS instrumentation read-back (ct_soc_axis_buf): 256 DAQ beats of four words = 1024 words = 4 KiB (AXIS_DEPTH). Read AXIS_BEATS * 4 words. 1024 entries of 32 bit.

35.4.3. Registers of `ctrl` (base 0)

`ct_soc_top` control / status

`ctrl.control` – offset 0x000

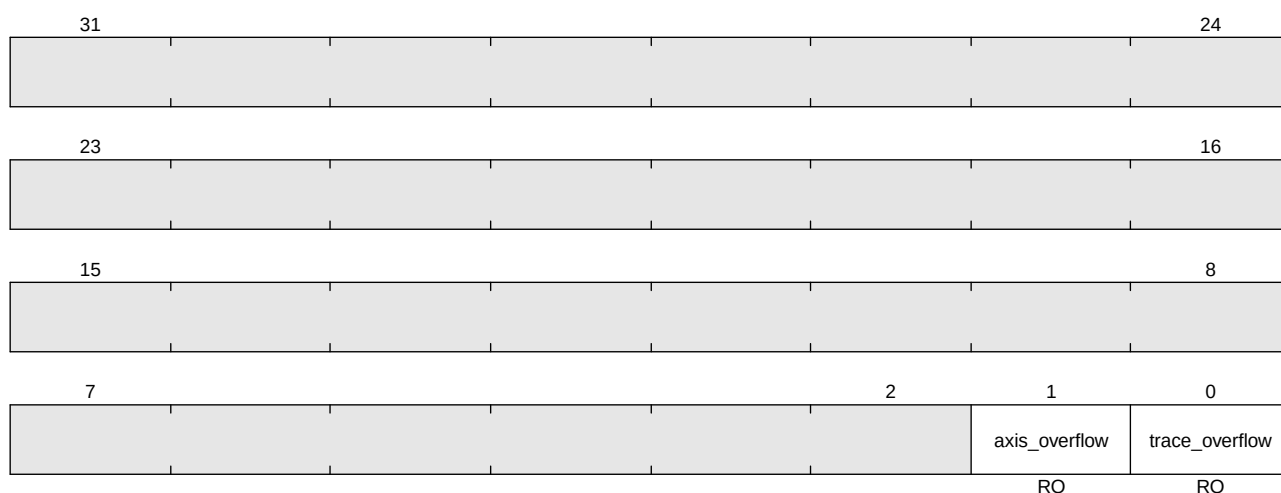
Control register, written over devmem to drive the SoC.



Bits	Field	Access	Reset	Description (SystemRDL of the example)
2	<code>trace_flush</code>	RW	0	Request an encoder trace flush.
1	<code>trace_clear</code>	RW	0	Level: while set, re-arm BOTH capture buffers (ATB and AXIS).
0	<code>core_run</code>	RW	0	0 = hold core in reset, 1 = run.

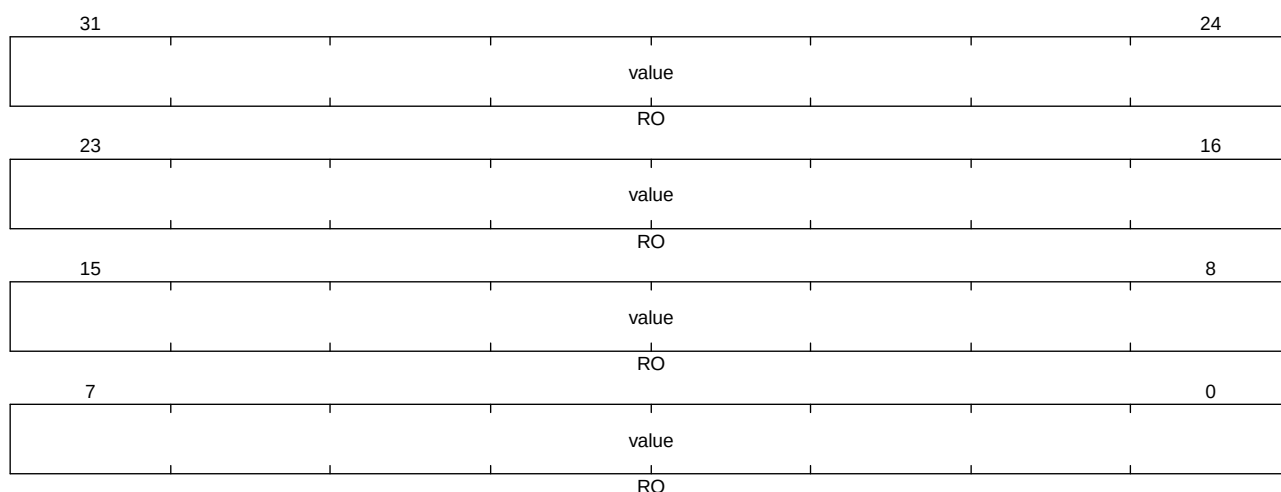
`ctrl.status` – offset 0x004

Status register. An overflow bit latches when its buffer is full and a further beat arrives – the capture is then truncated, not complete. Cleared by `trace_clear`.



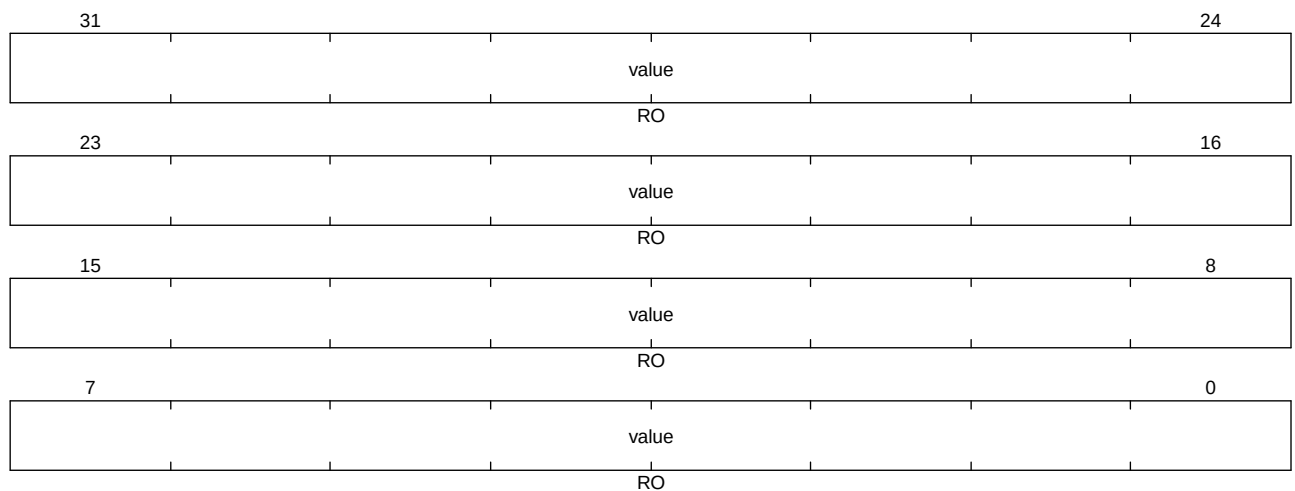
Bits	Field	Access	Reset	Description (SystemRDL of the example)
1	axis_overflow	RO (hw-written)	0	AXIS capture buffer full (>256 beats).
0	trace_overflow	RO (hw-written)	0	ATB capture buffer full (>4096 beats).

`ctrl.trace_beats` – offset 0x008



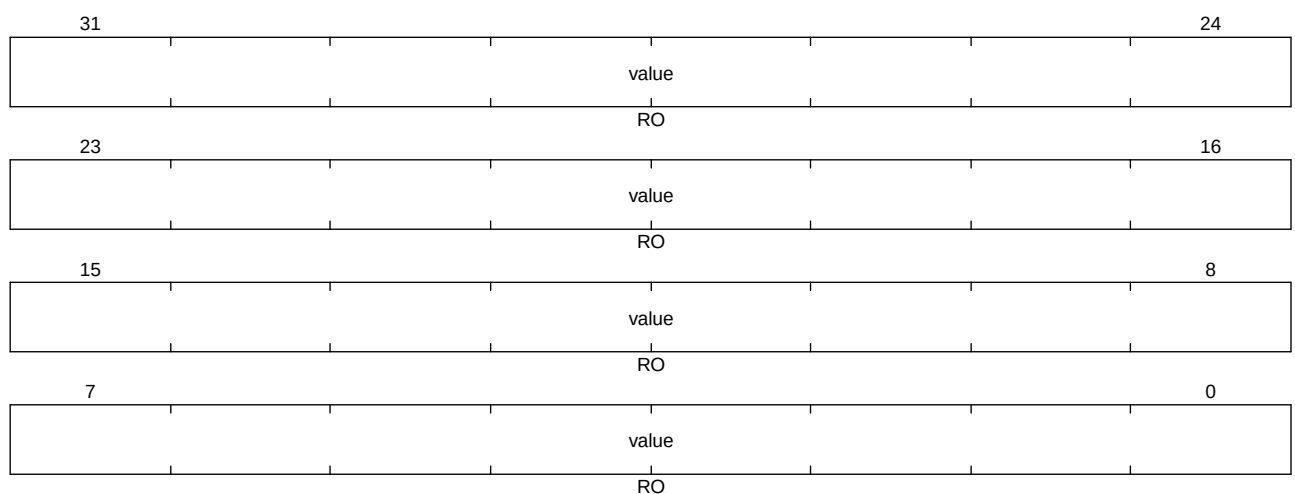
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	ATB beats captured (one 32-bit word per beat), saturating at the buffer depth of 4096 words = 16 KiB.

`ctrl.trace_bytes` – offset 0x00C



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Valid ATB bytes captured, saturating at 16384 (4 * 4096). Read TRACE_BEATS words from the TRACE window and keep the first TRACE_BYTES bytes – only the last beat can be partial.

ctrl.axis_beats – offset 0x010



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	AXIS instrumentation beats captured, saturating at the buffer depth of 256 beats = 4 KiB. One 128-bit record – four 32-bit words at AXIS+0x0..0xC – per beat: tdata elem0..2, then {tlast, tid[7:0], tkeep[11:0]}; tid[5:0] is the ACT-CAP/ACT-ST command.

35.4.4. CPU data bus (dBus)

Address map the CPU's AXI4-Lite dBus sees, decoded by ct_soc_synth_wrap.

Offsets are addresses on the CPU data bus.

Offset	Component	Kind	Size	Content
0	ram	memory	64 KiB	Program + data RAM (ct_soc_ram): 16384 words = 64 KiB (MEM_WORDS). 16384 entries of 32 bit.
0x10000000	clint	register file	20 B	Core-local interruptor (timer + software interrupt)
0x10001000	intc	register file	8 B	Minimal external interrupt controller
0x20000000	encoder	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers

35.4.5. Generated peripheral block

CLINT + INTC register block (AXI4-Lite), based at PERIPH (0x1000_0000).

Offsets are relative to the base of the generated block.

Offset	Component	Kind	Size	Content
0	clint	register file	20 B	Core-local interruptor (timer + software interrupt)
0x1000	intc	register file	8 B	Minimal external interrupt controller

35.5. The delivered two-core device (tgc5b2_axis_wp)

This is the only map in this appendix that describes a **whole device** rather than a building block: two MINRES TGC5B cores with one encoder instance each, the control block that starts and holds them, the merged trace ring the funnel writes, and the two AXI-Stream FIFOs the watchpoint and instrumentation records are drained from. A reader holding the loadable KV260 application has this map in front of them at the physical addresses `devmem` takes.

Three things are worth knowing before the tables:

- **The encoder map appears twice.** Both instances are the map of [Section 16.7](#), at 0x0001_0000 and 0x0002_0000 inside the control-port aperture. It is not reprinted here.
- **Only the encoder registers are generated into RTL.** The control block, the watchpoint-shim status block and the address router are hand-written modules. The SystemRDL describes them so that the map is complete in one place, and says so itself; keeping the two in step is a manual obligation, the same one [Section 35.4](#) states.
- **The memories are memories, not registers.** The two core RAMs and the trace ring are declared external, and their entry counts mirror the RTL parameters rather than being read back from hardware.

The address map of the delivered tgc5b2_axis_wp KV260 design: two TGC5B cores, their two encoder instances, the core control block, the merged trace-sink window and the two AXI-Stream FIFOs the watchpoint and instrumentation records are drained from. The encoder registers

instantiated twice in these spaces are the map of the register-model chapter and are not reprinted here. Register and field descriptions in the tables below are reproduced verbatim from that SystemRDL, which is Accemic's own text; the column heading names it as the source.

35.5.1. KV260 device view (physical addresses)

Absolute addresses on the board. The Zynq PS reaches all of this through M_AXI_HPM0_FPD → width converter → AXI4-Lite → a 1:4 router. pl_clk0 must be driven to 75 MHz before use.

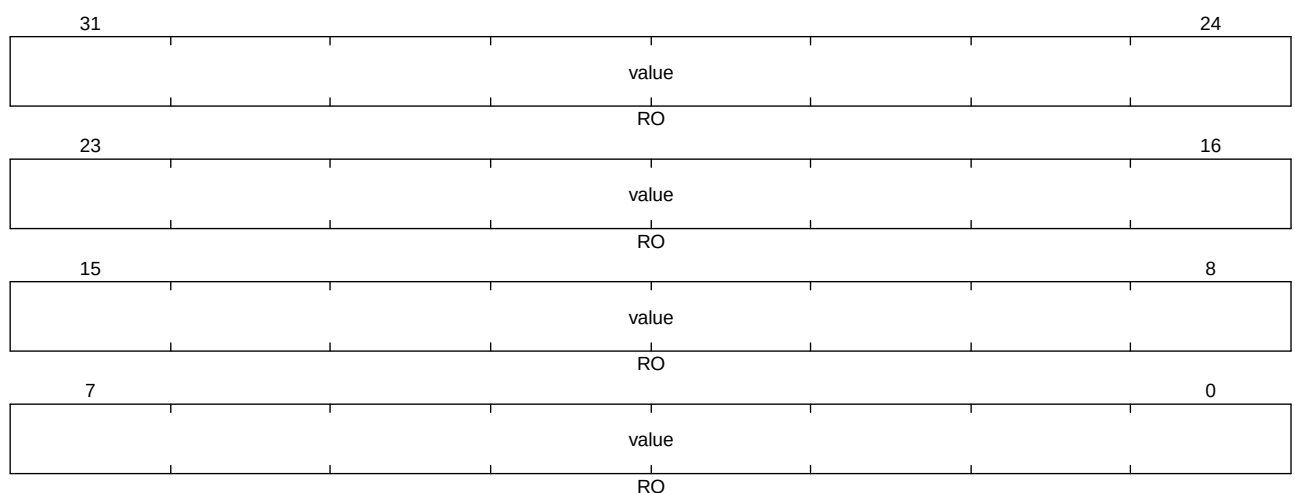
The offsets in this table are physical addresses: what `devmem` takes on the board.

Offset	Component	Kind	Size	Content
0xA0000000	soc	address map	3 MiB	tgc5b2_axis_wp SoC — PS control-port view (aperture-relative)
0xA0400000	wpctrl	register file	44 B	AXIS watchpoint shim status
0xA0410000	fifo0	register file	52 B	AXI4-Stream FIFO (PG080), RX path
0xA0420000	fifo1	register file	52 B	AXI4-Stream FIFO (PG080), RX path

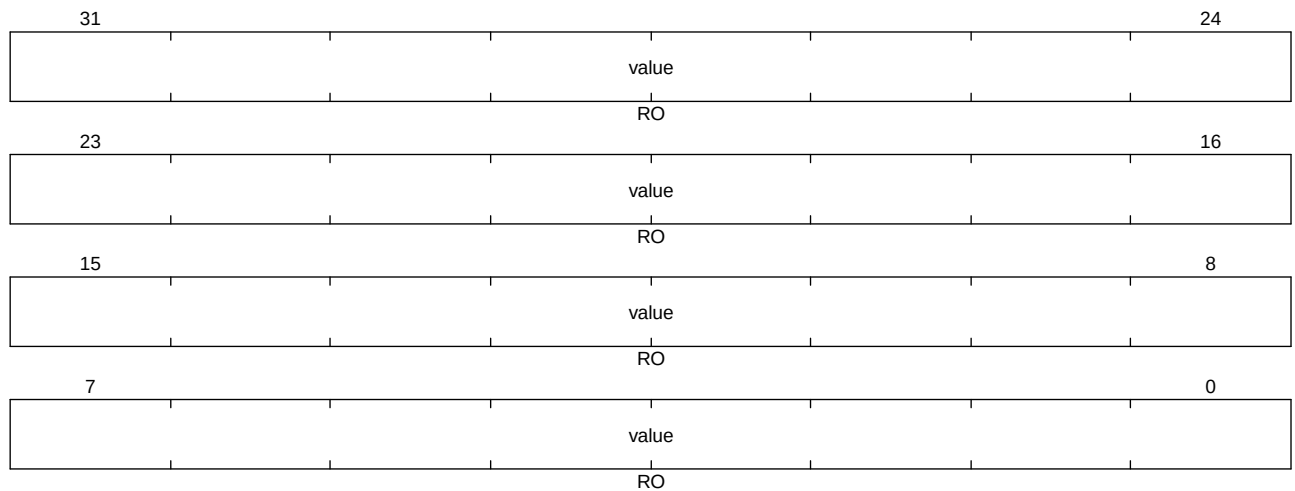
35.5.2. Registers of wpctrl (base 0xA0400000)

AXIS watchpoint shim status

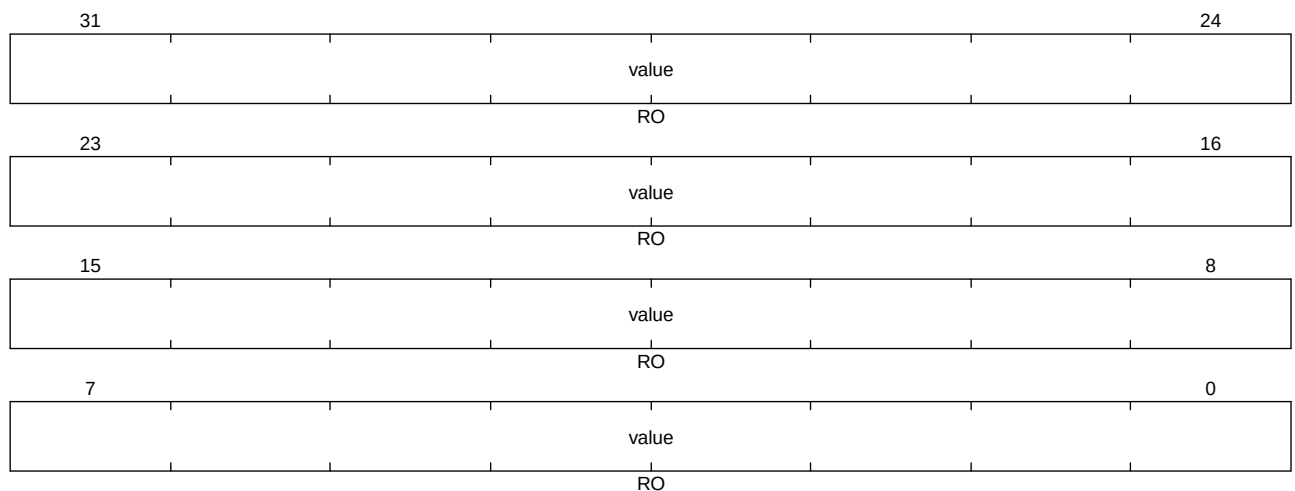
wpctrl.magic — offset 0xA0400000



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0x41575031	Magic identifier, reads 0x41575031 ("AWP1"). This is the proof that the PL is loaded and answering on the AXI bus — read it before any other access to the design.

wpctrl1.shim0_drop – offset 0xA0400004

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Core 0 shim: records dropped because its FIFO was full (saturating).

wpctrl1.shim0_fill – offset 0xA0400008

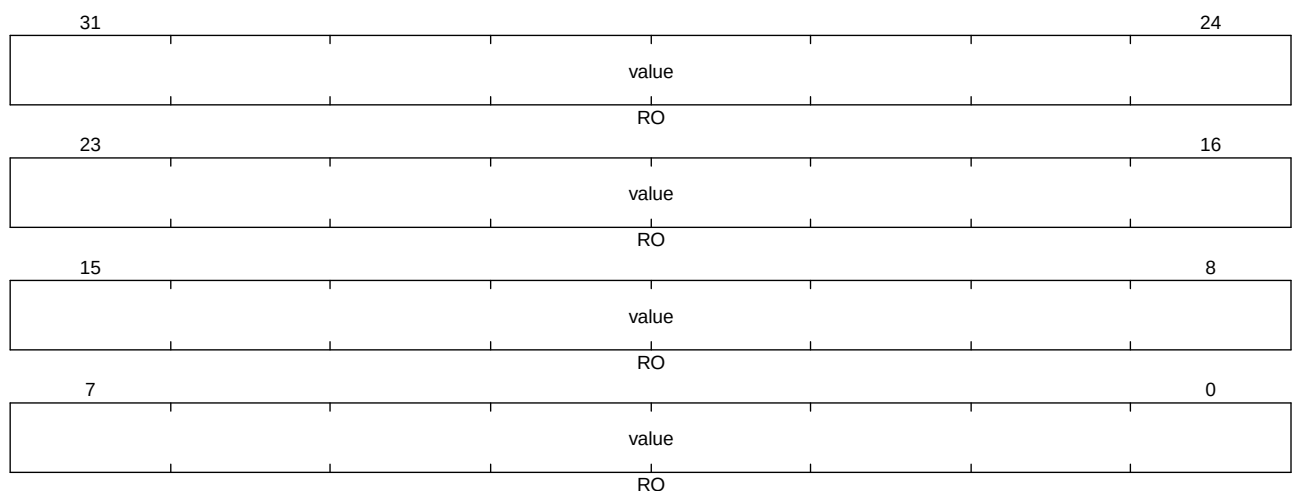
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Core 0 shim: current FIFO fill level in records.

wpctrl1.shim0_ovf – offset 0xA040000C



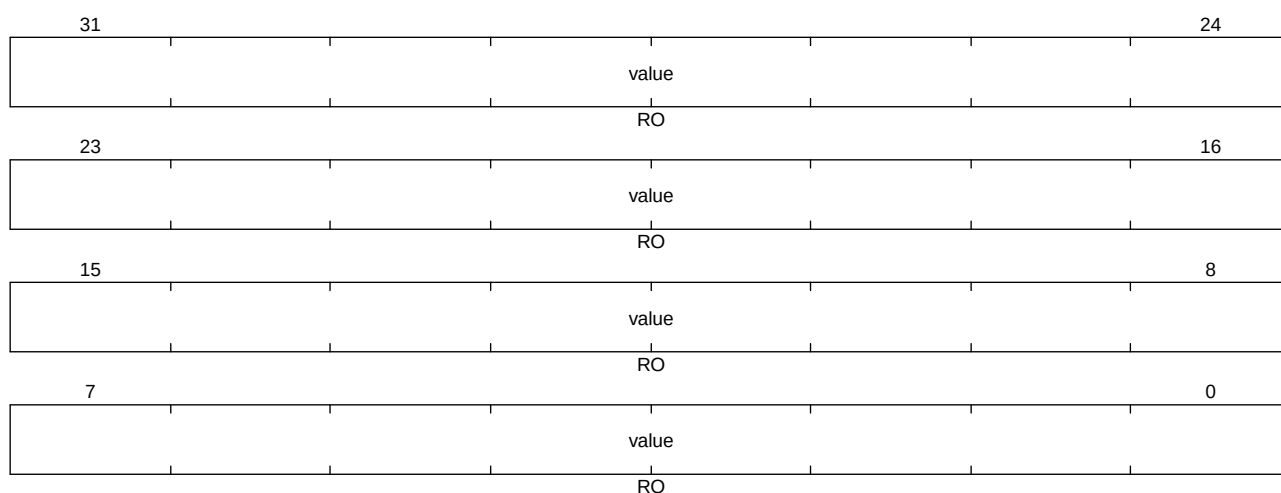
Bits	Field	Access	Reset	Description (SystemRDL of the example)
0	overflow	RO (hw-written)	0	Core 0 shim: sticky overflow flag (latches on the first drop).

`wpctrl.shim1_drop` – offset 0xA0400010



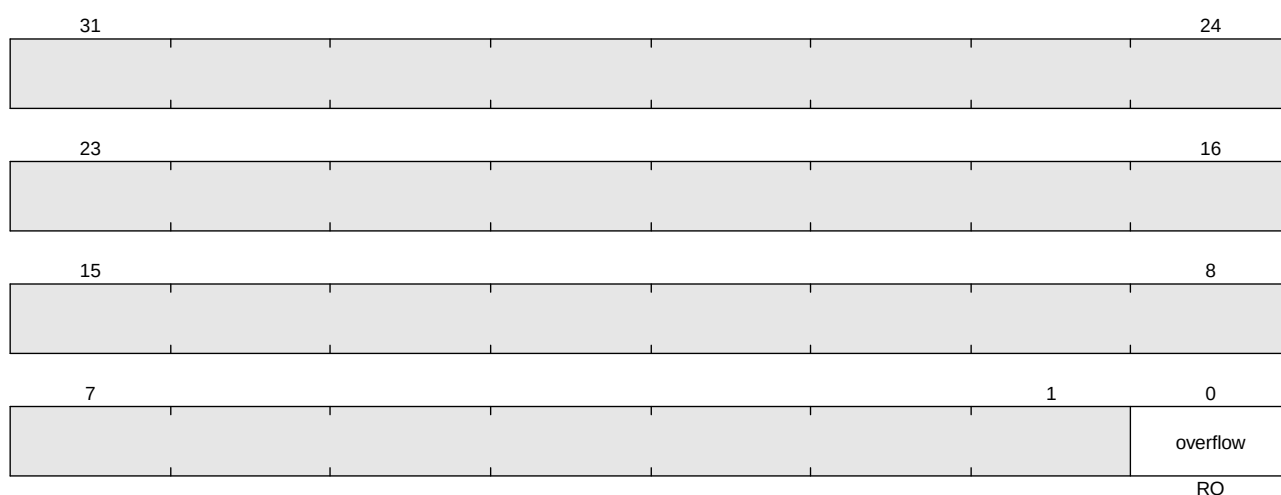
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Core 1 shim: records dropped because its FIFO was full (saturating).

`wpctrl.shim1_fill` – offset 0xA0400014



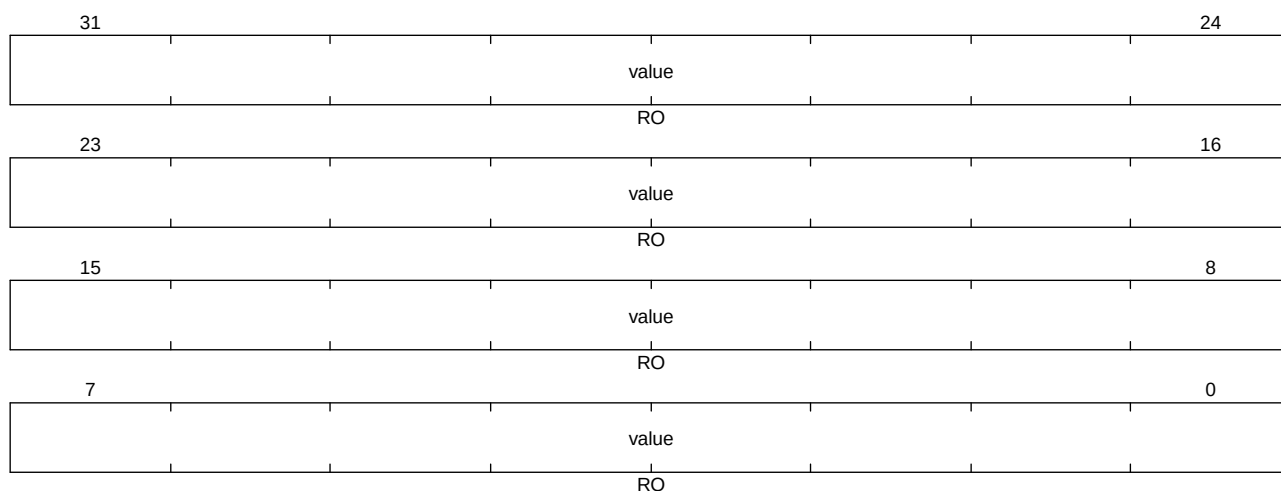
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Core 1 shim: current FIFO fill level in records.

wpctrl.shim1_ovf – offset 0xA0400018



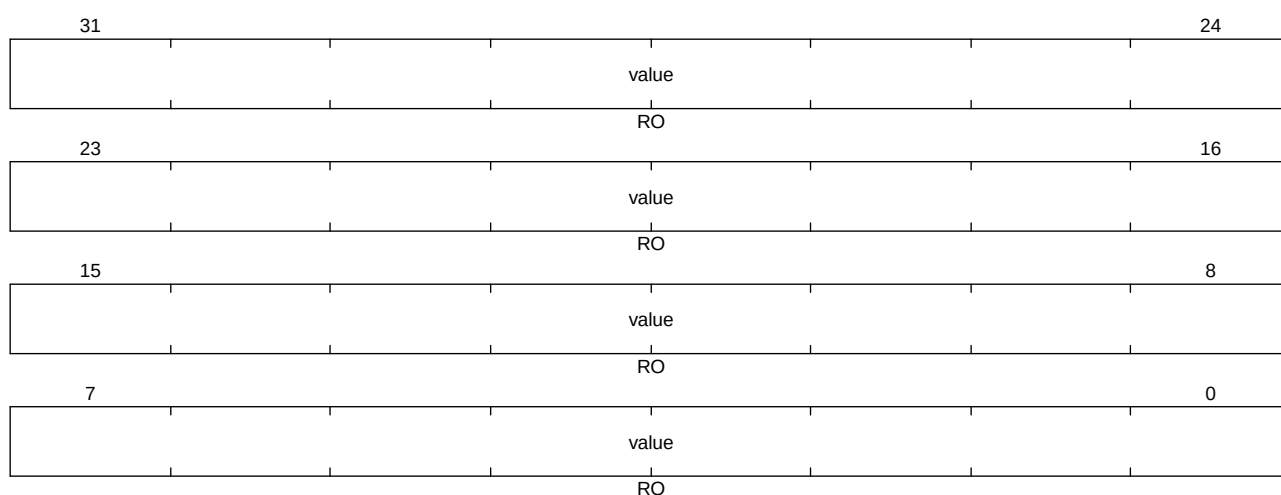
Bits	Field	Access	Reset	Description (SystemRDL of the example)
0	overflow	RO (hw-written)	0	Core 1 shim: sticky overflow flag (latches on the first drop).

wpctrl.ftime_lo – offset 0xA040001C



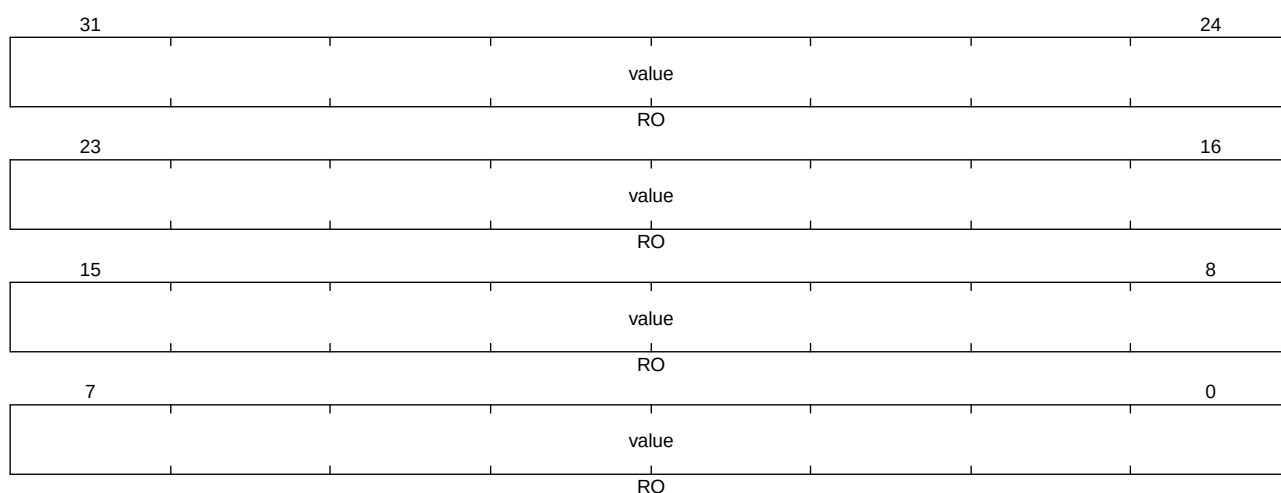
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Low half of the free-running 64-bit fabric counter – the SHARED time base both encoders stamp their records with. READ ORDER MATTERS: reading FTIME_LO atomically snapshots the high half into FTIME_HI. Read LO first, then HI, or the two halves can straddle a carry.

wpctrl.ftime_hi – offset 0xA0400020



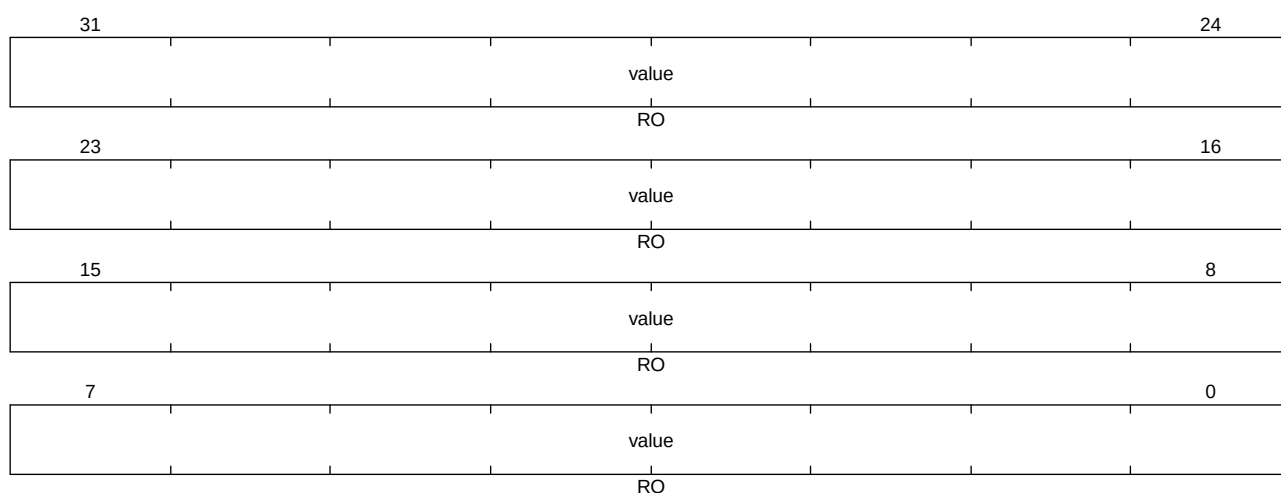
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	High half of the fabric counter, as snapshotted by the last FTIME_LO read.

wpctrl.rx_fifo_words – offset 0xA0400024



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0x1000	Build parameter mirror: RX FIFO depth in 32-bit words per axi_fifo_mm_s (4096 = 1024 records).

wpctrl.shim_records – offset 0xA0400028



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0x100	Build parameter mirror: internal record depth per ct_axis_wp_shim.

35.5.3. PS control port (aperture-relative)

The 22-bit window the AXI4-Lite router forwards to the SoC.

Offsets are relative to the base of the 22-bit control-port aperture the router forwards to the SoC.

Offset	Component	Kind	Size	Content
0	ctrl	register file	60 B	tgc5b2_axis_soc_top control / status

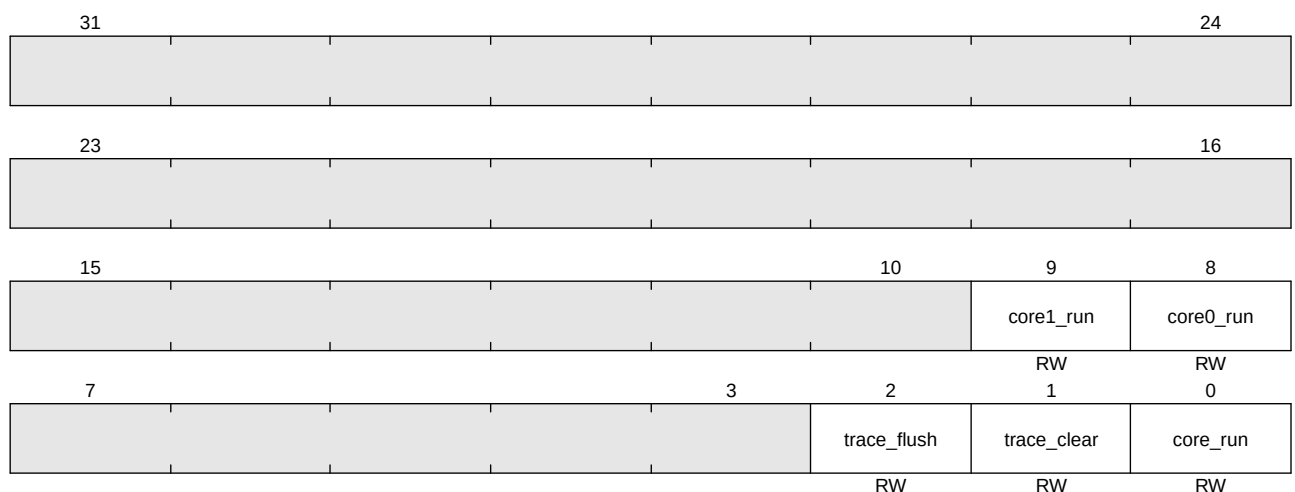
Offset	Component	Kind	Size	Content
0x10000	enc0	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers
0x20000	enc1	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers
0x80000	ram1	memory	64 KiB	TGC5B core 1 program/data RAM, 64 KiB (MEM_WORDS). Write only while core 1 is held – see CONTROL. 16384 entries of 32 bit.
0x100000	ram0	memory	64 KiB	TGC5B core 0 program/data RAM, 64 KiB (MEM_WORDS). Write only while core 0 is held – see CONTROL. 16384 entries of 32 bit.
0x200000	trace	memory	1 MiB	Merged ATB ring (TRACE_DEPTH = 1 MiB). Both encoders' N-Trace output after ct_L1_funnel; the SRC field attributes each message back to its source encoder. Read TRACE_BEATS words and keep the first TRACE_BYTES bytes. 262144 entries of 32 bit.

35.5.4. Registers of `ctrl` (base 0)

tgc5b2_axis_soc_top control / status

`ctrl.control` – offset 0x000

Control register, written over devmem to drive the SoC. The two cores have nothing to do with each other – there is no SMP here. Effective run state of core *i* is `core_run` OR `core_i_run`. LOADER CONTRACT, PER CORE: only write `RAM_i` while `core_i_run` (and `core_run`) are 0. An access to a RUNNING core's RAM never gets ready and hangs the transaction.

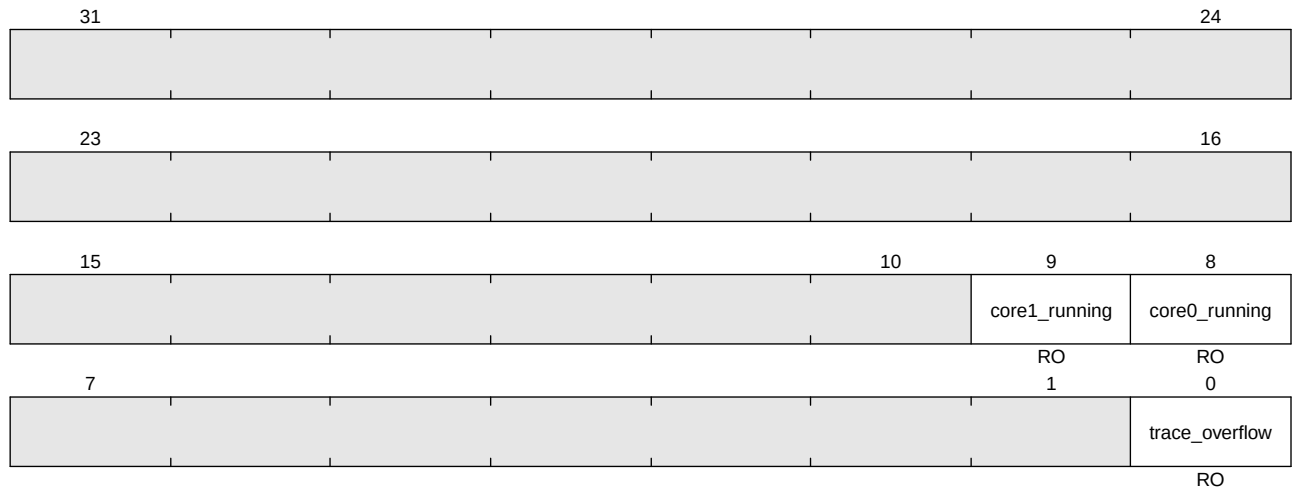


Bits	Field	Access	Reset	Description (SystemRDL of the example)
9	<code>core1_run</code>	RW	0	Run core 1 individually.
8	<code>core0_run</code>	RW	0	Run core 0 individually.
2	<code>trace_flush</code>	RW	0	Request a global funnel flush.
1	<code>trace_clear</code>	RW	0	Level: while set, re-arm the ATB ring.

Bits	Field	Access	Reset	Description (SystemRDL of the example)
0	core_run	RW	0	Collective bit: run BOTH cores.

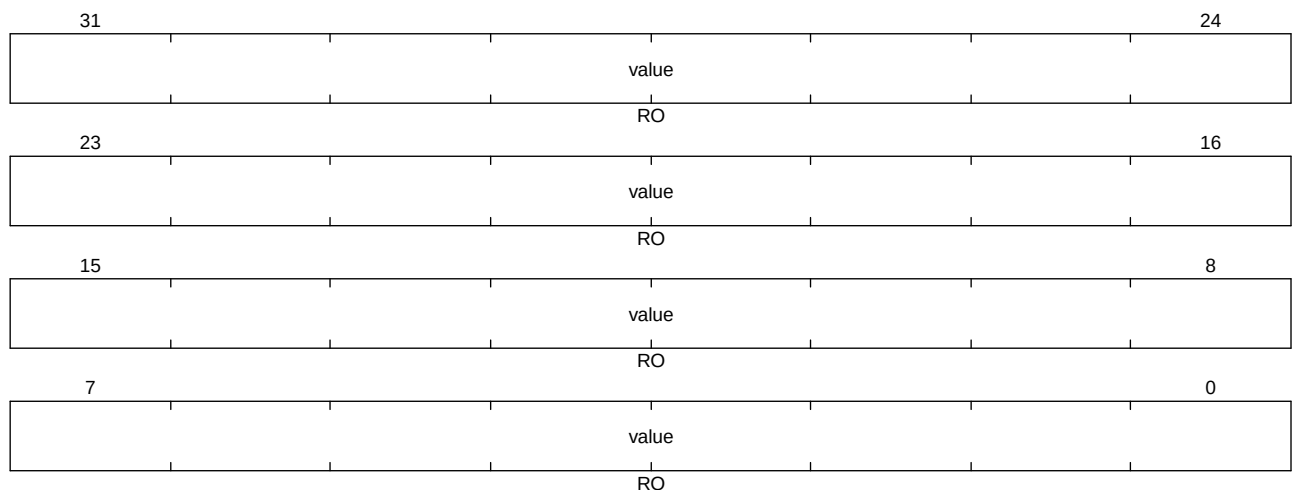
ctrl.status – offset 0x004

Status register. Cleared by trace_clear.

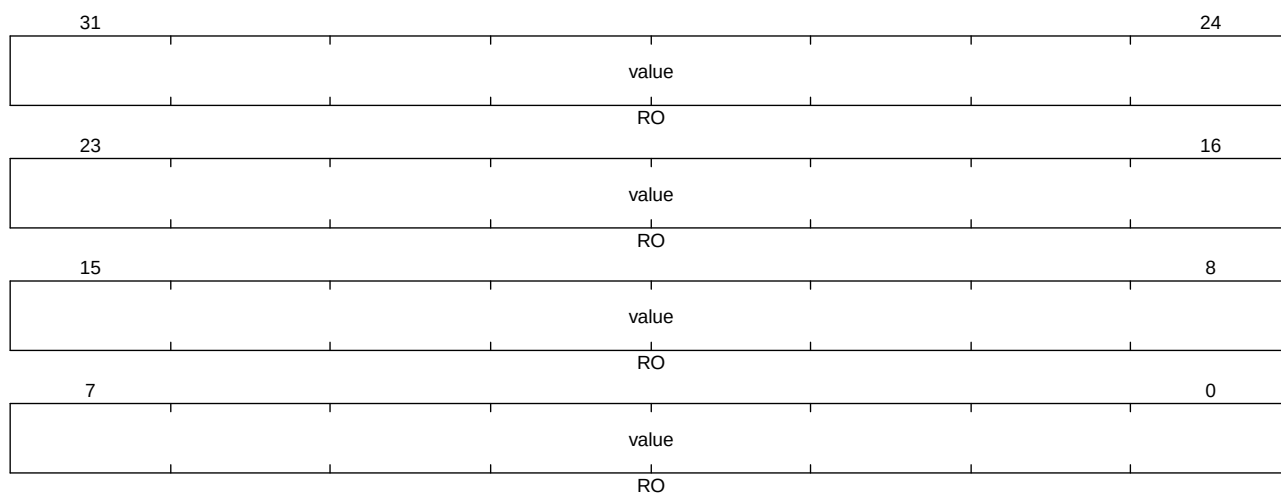


Bits	Field	Access	Reset	Description (SystemRDL of the example)
9	core1_running	RO (hw-written)	0	Core 1 running (effective state).
8	core0_running	RO (hw-written)	0	Core 0 running (effective state).
0	trace_overflow	RO (hw-written)	0	ATB ring wrapped – capture truncated.

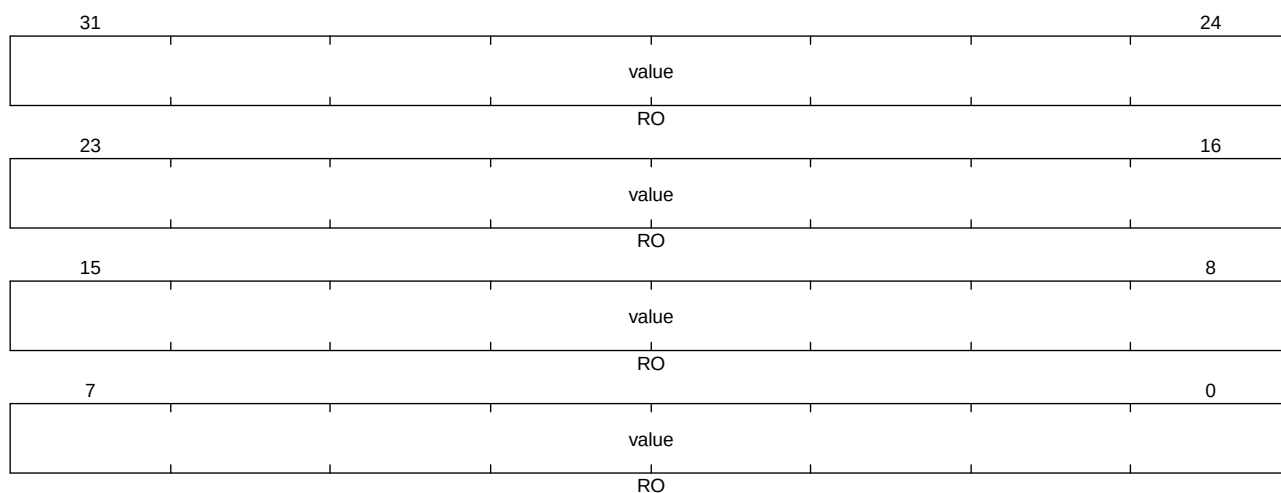
ctrl.trace_beats – offset 0x008



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Merged ATB beats captured into the ring (one 32-bit word per beat).

ctrl.trace_bytes – offset 0x00C

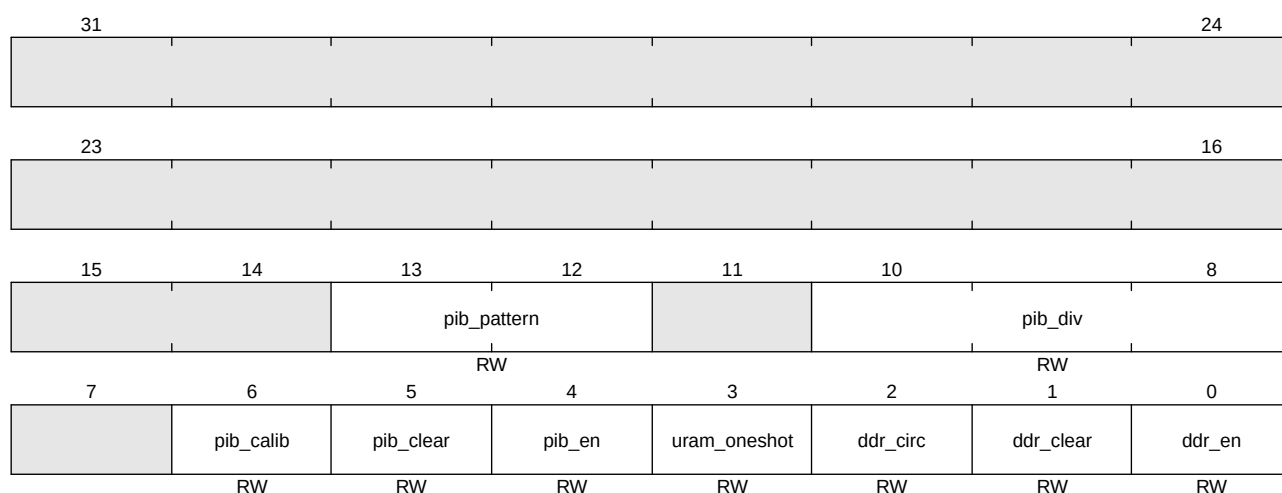
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Valid ATB bytes captured. Only the last beat can be partial.

ctrl.trace_bufsz – offset 0x010

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Ring capacity in bytes (TRACE_DEPTH * 4 = 1 MiB).

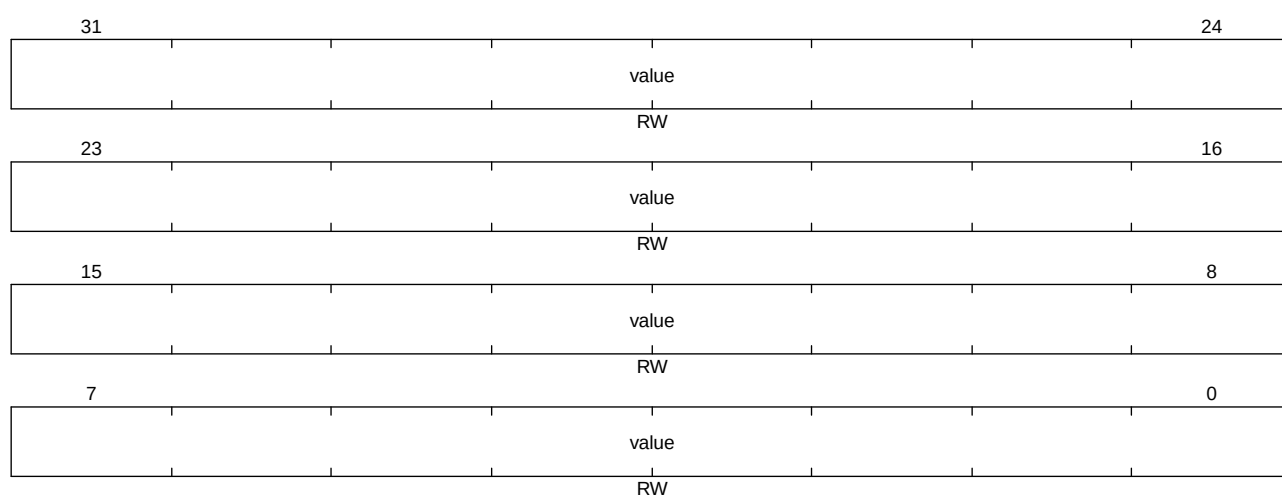
ctrl.sink_ctrl – offset 0x018

Sink control. The DDR4 sink and the PIB parallel port are additive observers on the funnel output; the URAM ring always runs.



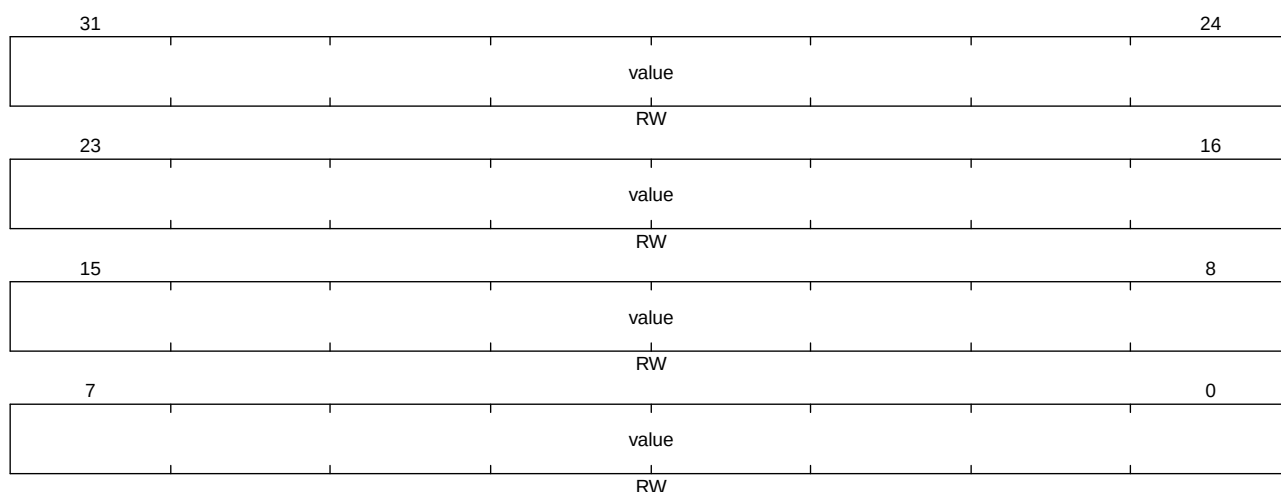
Bits	Field	Access	Reset	Description (SystemRDL of the example)
13:12	pib_pattern	RW	0	PIB calibration pattern select.
10:8	pib_div	RW	0	PIB clock divider.
6	pib_calib	RW	0	Emit the PIB calibration pattern.
5	pib_clear	RW	0	Pulse: clear PIB counters.
4	pib_en	RW	0	Enable the PIB parallel trace port.
3	uram_oneshot	RW	0	URAM ring one-shot (stop on full).
2	ddr_circ	RW	0	Circular mode (wrap instead of stop).
1	ddr_clear	RW	0	Pulse: clear DDR counters/flags.
0	ddr_en	RW	0	Arm the DDR4 sink.

ctrl. ddr_base – offset 0x01C



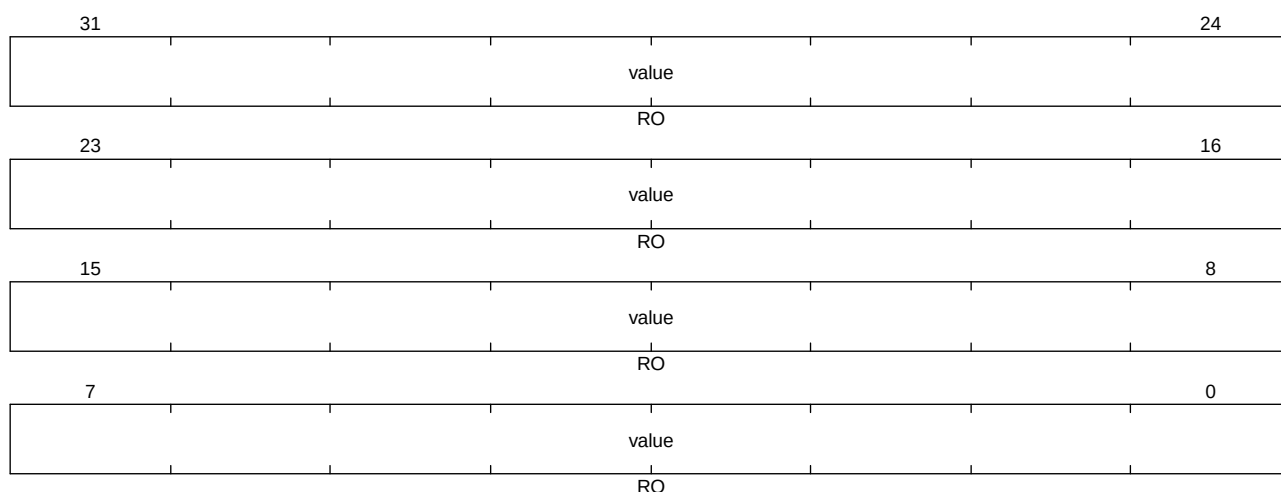
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RW	0x50000000	DDR4 sink window base. Writable only while ddr_en = 0; a write while the sink is armed is rejected and reported in SINK_STAT. ddr_cfg_rej. Reset is the reserved-memory window that the boot device tree must carve out (no-map).

ctrl. ddr_size – offset 0x020



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RW	0x10000000	DDR4 sink window size in bytes (reset 256 MiB). Same write-while-disarmed rule as DDR_BASE.

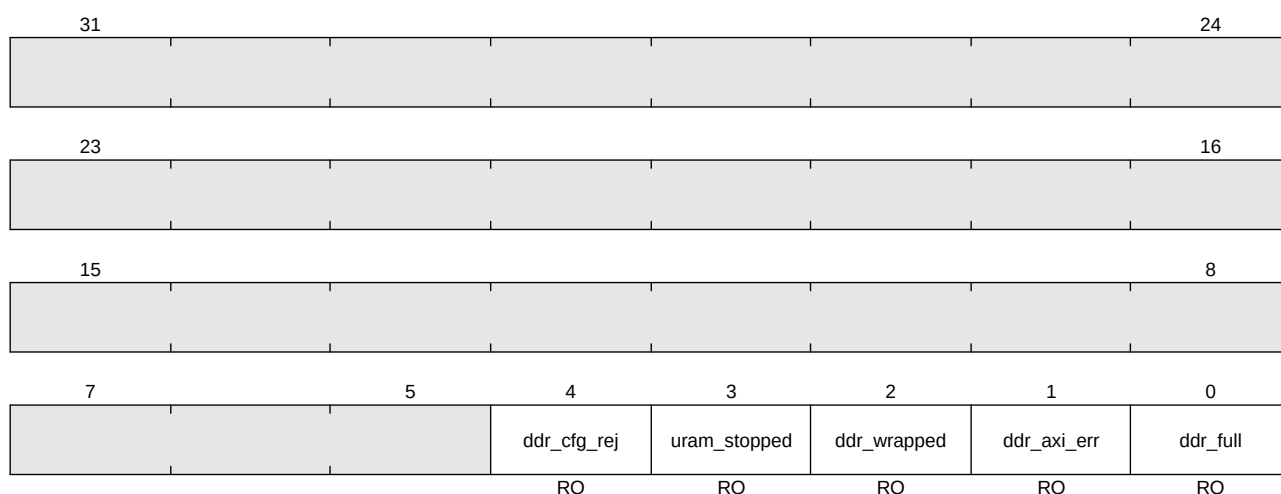
ctrl. ddr_wptr – offset 0x024



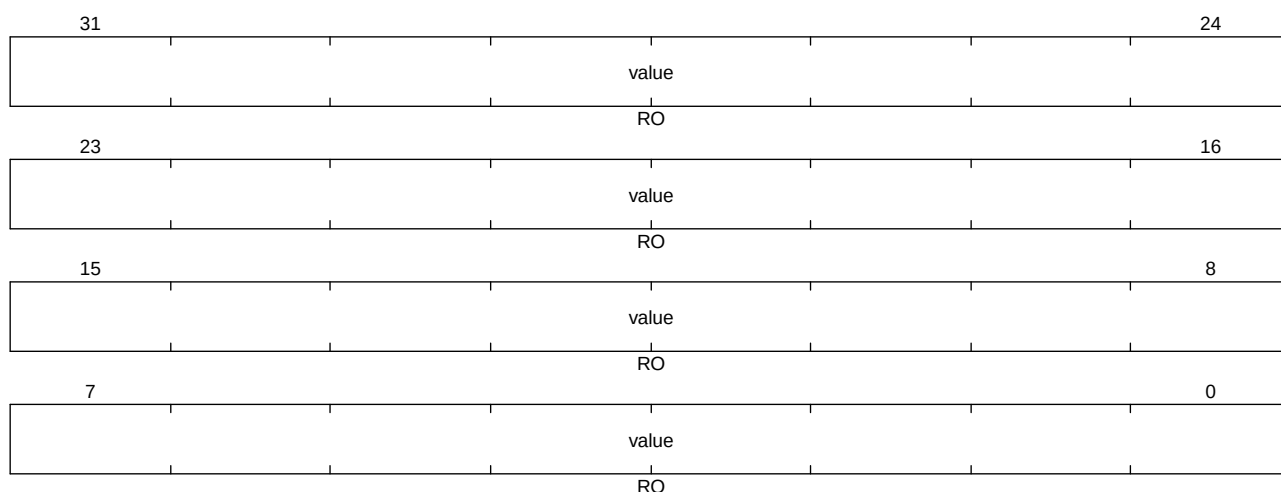
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Bytes written to DDR in TOTAL (monotonic, not modulo the window).

ctrl.sink_stat – offset 0x028

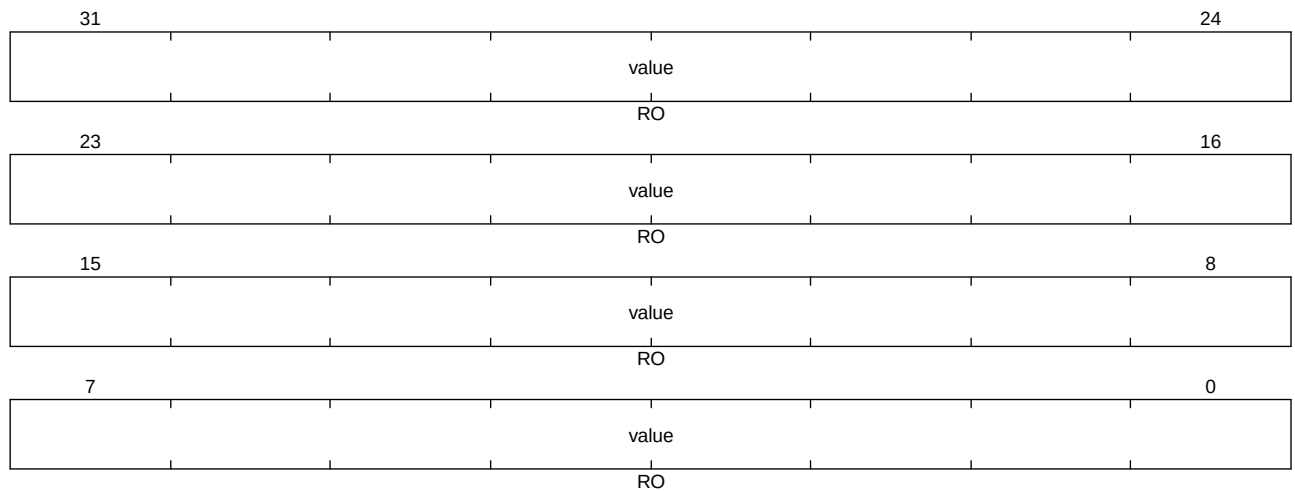
Sink status.



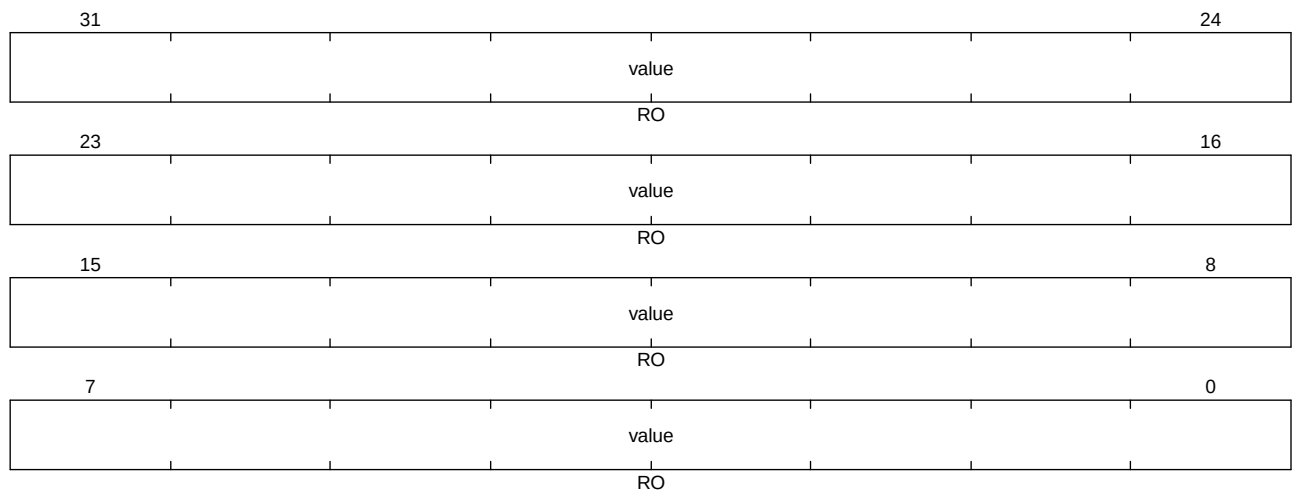
Bits	Field	Access	Reset	Description (SystemRDL of the example)
4	ddr_cfg_rej	RO (hw-written)	0	A window write was rejected because the sink was armed.
3	uram_stopped	RO (hw-written)	0	URAM ring stopped (one-shot).
2	ddr_wrapped	RO (hw-written)	0	DDR window wrapped (circular mode).
1	ddr_axi_err	RO (hw-written)	0	DDR AXI error response seen.
0	ddr_full	RO (hw-written)	0	DDR window full.

ctrl. ddr_drops – offset 0x02C

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Beats dropped by the DDR sink (saturating).

ctrl.pib_drops – offset 0x030

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Beats dropped by the PIB port (saturating).

ctrl. ddr_beats – offset 0x038

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	ATB beats OFFERED to the sink since <code>ddr_en</code> – the proof that beats reach the sink at all. Cleared via <code>ddr_clear</code> .

35.6. The runtime-verification testbed (tgc5b2_rvcfi)

The runtime-verification testbed is the second whole-device map in this appendix: two TGC5B cores with one encoder instance each, the shared memory both programs race over, and the banks that exist for runtime verification specifically – a read-only observation bank, a per-core console, and one DDR record ring per core. Until 2026-08-27 this appendix could not print any of it: the banks existed only in the RTL, and an earlier revision of this manual said so rather than transcribing them

by hand. The example now carries its own SystemRDL source, written as-built against the RTL and under the same drift gate as every other table here, so the previous deliberate absence is closed the only way this appendix closes gaps – by generating.

Four things are worth knowing before the tables:

- **Nothing in this SystemRDL generates RTL.** The control window, the banks and the router are hand-written modules; only the two encoder CSR windows are generated, and they are the map of [Section 16.7](#), at 0x01_0000 and 0x02_0000 in the SoC aperture. The SystemRDL exists so the map is readable in one place, and it names the RTL, host-tool and tutorial locations it must be kept in step with.
- **The observation bank is read-only by design, and the design says why.** Offset 0x40 differs from `CONTROL` in a single address bit; a write alias there would start both cores by accident, so writes into the bank are discarded in hardware.
- **The ring base and size are WARL.** A write that is misaligned, out of the reserved DDR window, or arrives while the ring is enabled is rejected and latches `ring_stat.cfg_rej` – on this board family an unchecked AXI write outside the reservation does not error, it wedges the interconnect until the power switch, which is why the check is in hardware rather than in the driver.
- **The core-side map is the one map the processing system cannot reach.** The ACT-CAP doorbell in particular is a core-side address; [Chapter 20](#) describes the command word it carries.

The address maps of the `tgc5b2_rvcfi` runtime-verification demonstrator: two TGC5B cores with one encoder each, the shared memory they race over, the RV/CFI observation, console and DDR-record-ring banks, and the core-side map with the ACT-CAP doorbell. The two ring banks (`ring0 @ 0x80`, `ring1 @ 0xA0`) are instances of the same register file; the bit-level table is printed once for `ring0`. Register and field descriptions in the tables below are reproduced verbatim from that SystemRDL, which is Accemic's own text; the column heading names it as the source.

35.6.1. KV260 device view (physical addresses)

The SoC behind the 1:4 AXI4-Lite router, plus the shim status block and the two PG080 FIFOs the records are drained from in FIFO mode. Router select bits: address [17:16] within the 4 MiB region above the SoC.

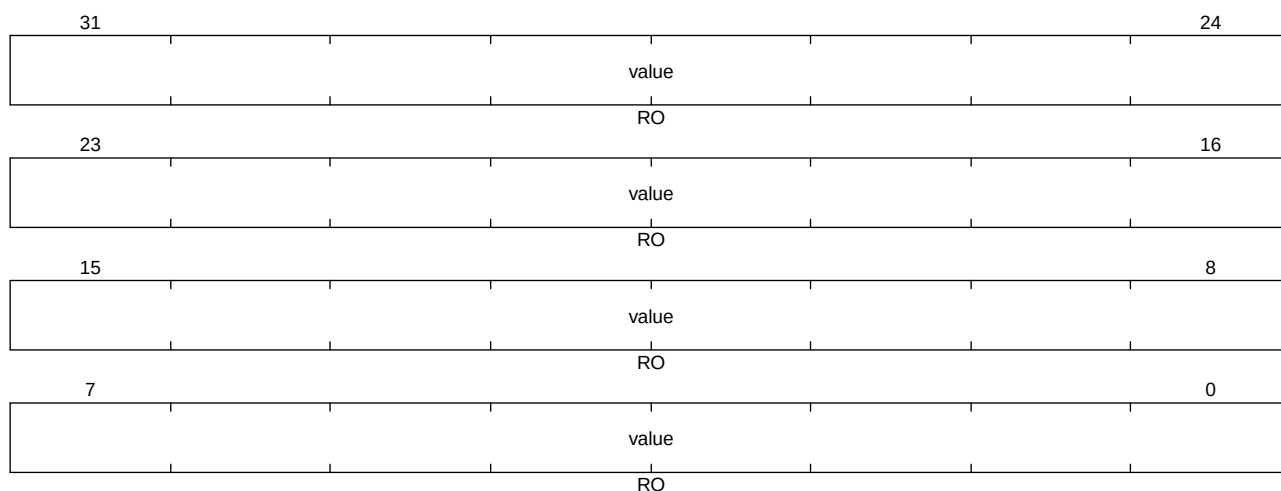
The offsets in this table are physical addresses: what `devmem` takes on the board.

Offset	Component	Kind	Size	Content
0xA0000000	<code>soc</code>	address map	3 MiB	RV/CFI demo SoC (22-bit AXI4-Lite aperture)
0xA0400000	<code>wpctrl</code>	register file	28 B	WPCTRL – watchpoint shim status
0xA0410000	<code>fifo0</code>	memory	64 KiB	Core 0's record FIFO in FIFO mode. Idle when that core's <code>ring_ctrl.route_ddr</code> is set. 16384 entries of 32 bit.
0xA0420000	<code>fifo1</code>	memory	64 KiB	<code>axi_fifo_mm_s 1</code> (PG080 lite register set) 16384 entries of 32 bit.

35.6.2. Registers of `wpctr1` (base 0xA0400000)

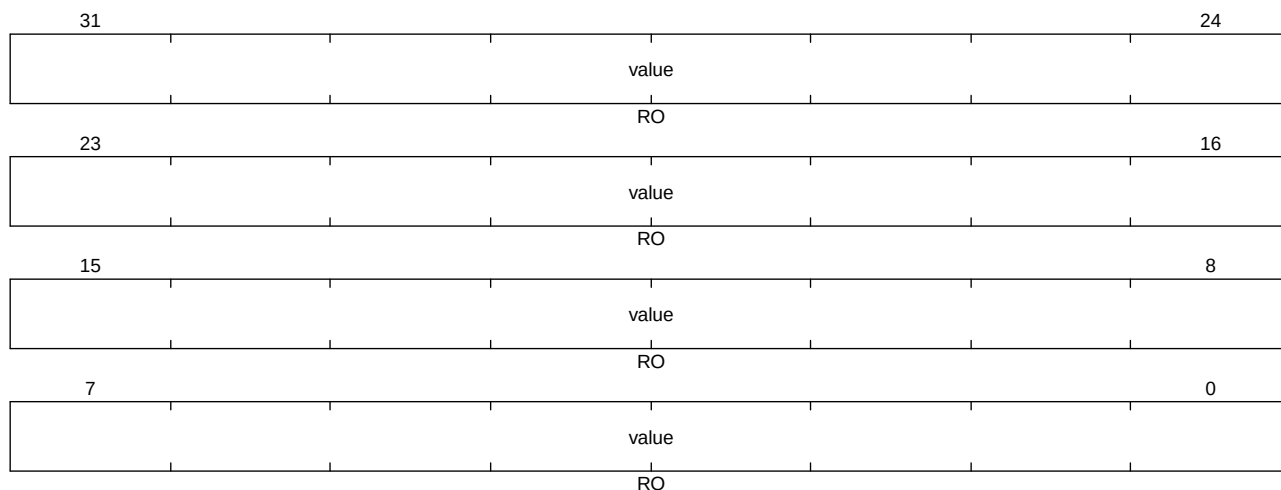
Per-shim record accounting. The drop counters are CUMULATIVE since bitstream load and have no clear register on purpose (a counter that can be cleared can be cleared by accident): a run is judged by the DELTA over its own window.

`wpctr1.shim0_drops` – offset 0xA0400004

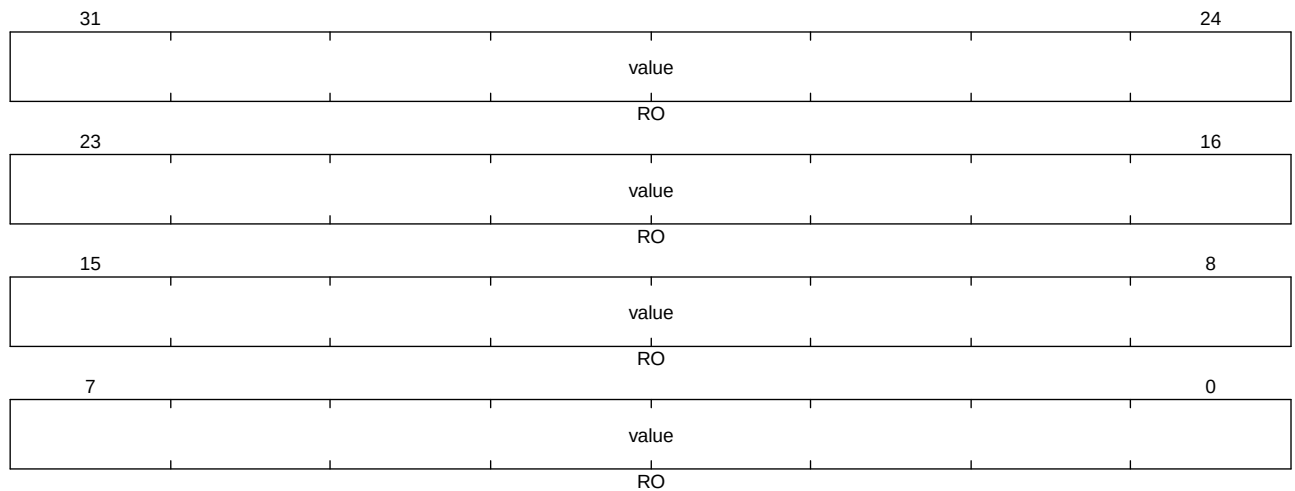


Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shim 0: records dropped because its FIFO was full.

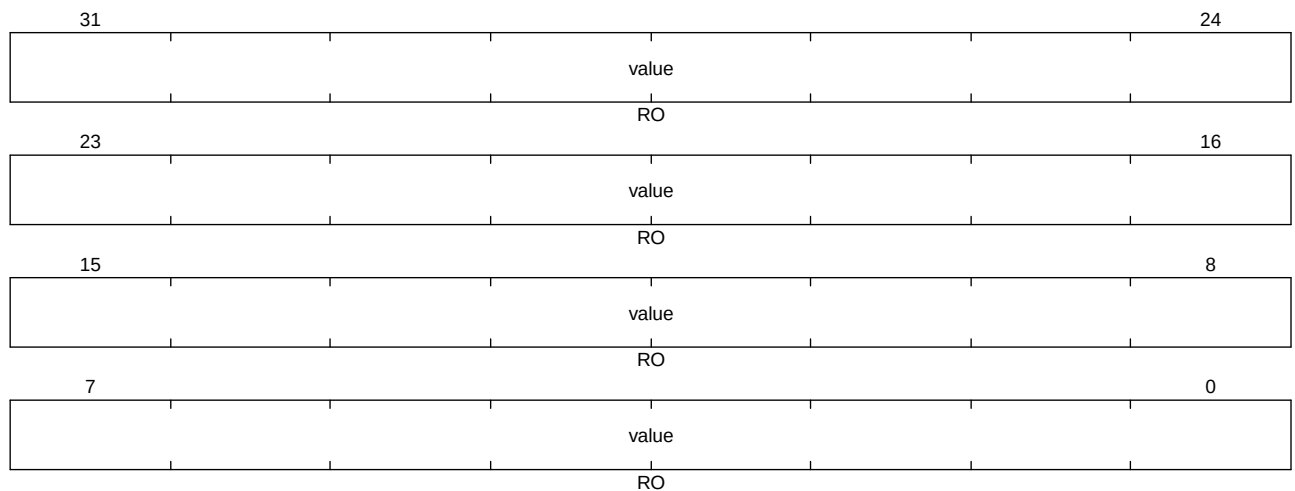
`wpctr1.shim0_fill` – offset 0xA0400008



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shim 0: current FIFO occupancy (watch it approach the depth).

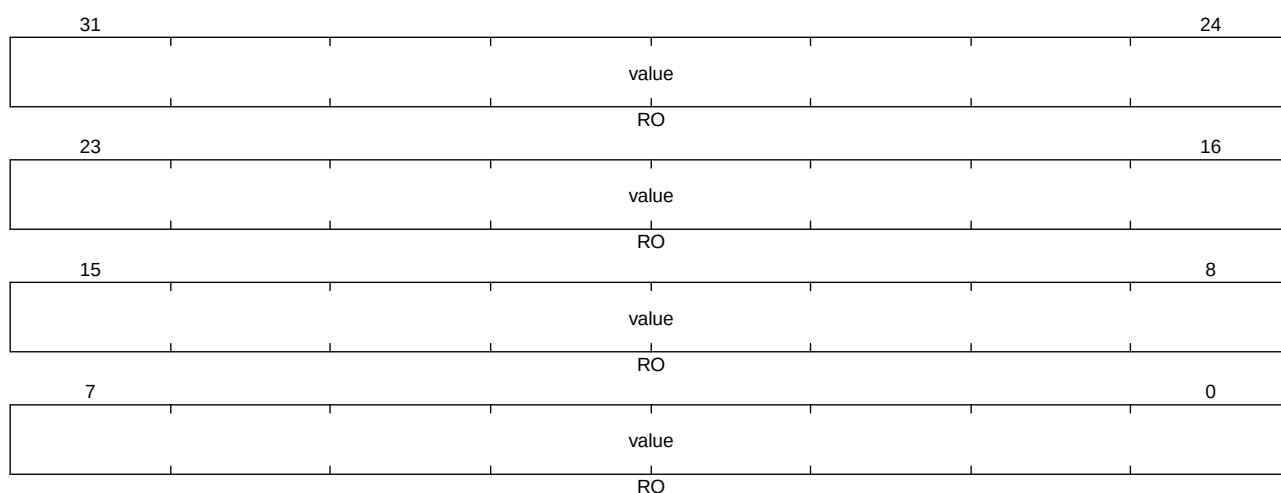
wpctrl.shim0_overflow – offset 0xA040000C

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shim 0: sticky 'it overflowed at least once'.

wpctrl.shim1_drops – offset 0xA0400010

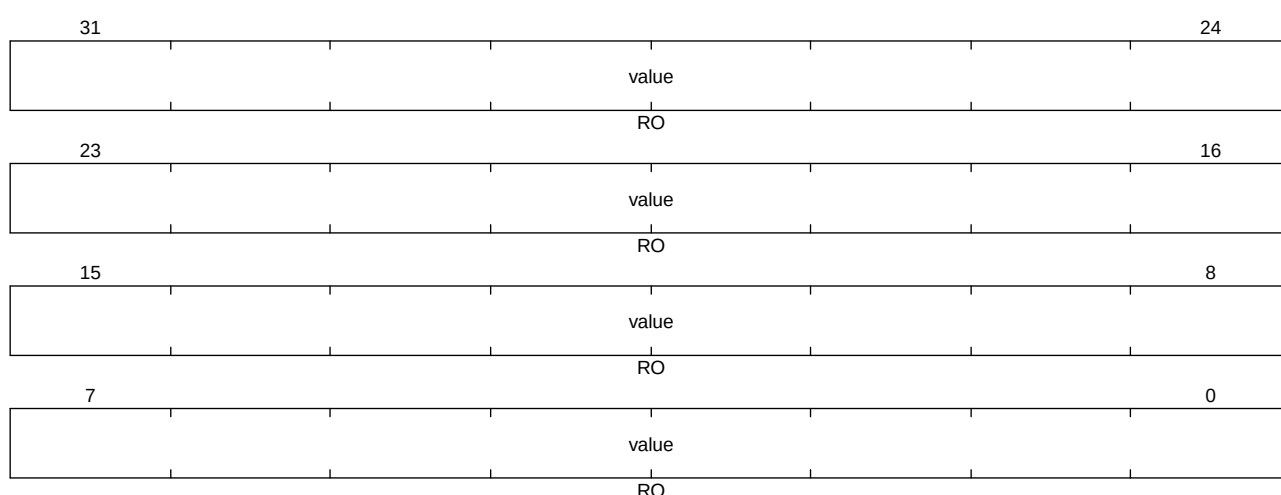
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shim 1: records dropped because its FIFO was full.

wpctrl.shim1_fill – offset 0xA0400014



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shim 1: current FIFO occupancy.

wpctrl.shim1_overflow – offset 0xA0400018



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shim 1: sticky overflow flag.

35.6.3. PS control port (aperture-relative)

Two TGC5B cores with one CTTE encoder each, a shared memory, the merged ATB ring, and the RV/CFI banks. Hand-written decode in rtl/tgc5b2_rvcfi_soc_top.sv; only the ENC windows are generated.

Offsets are relative to the base of the 22-bit control-port aperture the router forwards to the SoC.

Offset	Component	Kind	Size	Content
0	ctrl	register file	20 B	tgc5b2_rvcfi_soc_top control / status

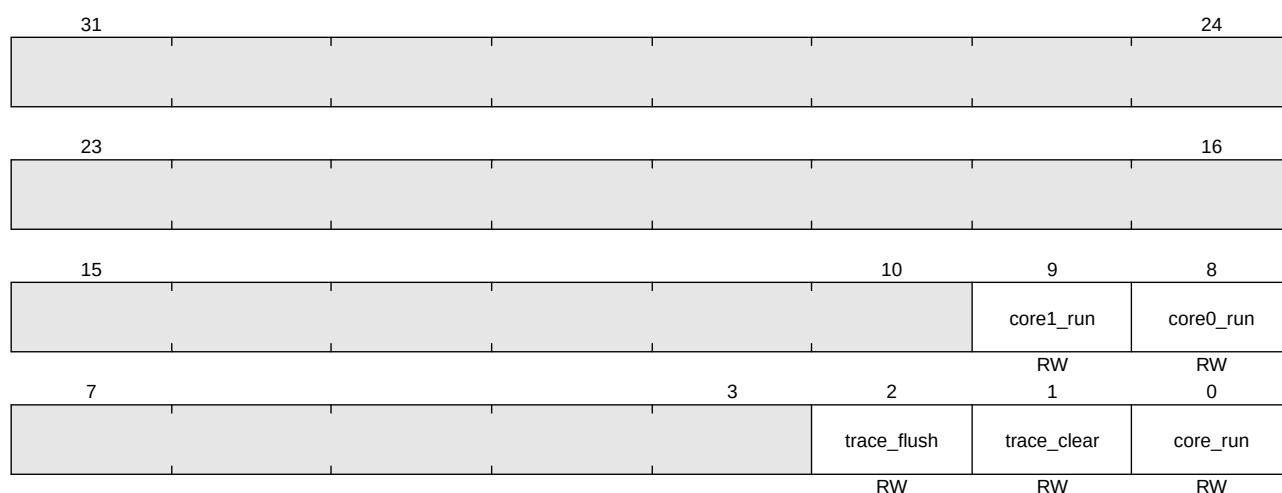
Offset	Component	Kind	Size	Content
0x40	obs	register file	32 B	RV/CFI observation bank
0x60	console	register file	28 B	Per-core console
0x80	ring0	register file	28 B	DDR record ring (one instance per core)
0xA0	ring1	register file	28 B	DDR record ring (one instance per core)
0x10000	enc0	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers
0x20000	enc1	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers
0x40000	shared	memory	256 KiB	The memory both programs race over. UltraRAM: no bitstream initialization, and NO byte-write enables — a partial strobe is refused (SLVERR at the block), so every write is a whole word. One 32-bit word per address, deliberately: two words per URAM row would collide exactly on Peterson's flag pair. Reachable from the PS ONLY while both cores are held. 65536 entries of 32 bit.
0x80000	ram1	memory	64 KiB	Load only while core1_run = 0 (loader contract). 16384 entries of 32 bit.
0x100000	ram0	memory	64 KiB	Load only while core0_run = 0 (loader contract). 16384 entries of 32 bit.
0x200000	trace	memory	1 MiB	Both encoders' ATB streams after the funnel. Read-only; a write is ignored. mementries mirrors TRACE_DEPTH. 262144 entries of 32 bit.

35.6.4. Registers of ctr1 (base 0)

tgc5b2_rvcfi_soc_top control / status

ctr1.control — offset 0x000

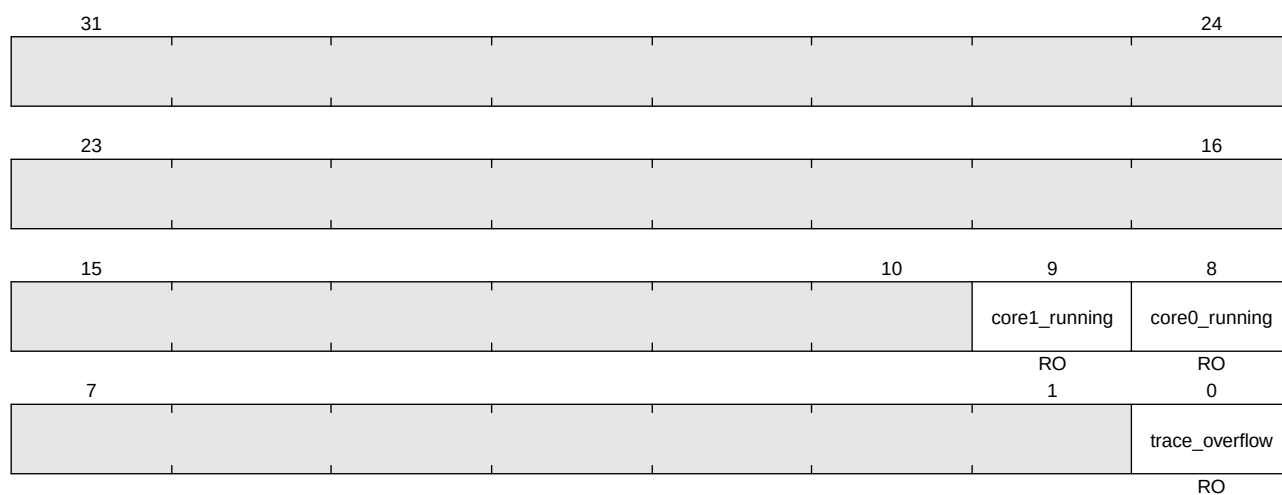
Control register, written over devmem. The two cores have nothing to do with each other — own RAM, own CLINT, own encoder, no SMP. Effective run state of core *i* is core_run OR core_i_run. LOADER CONTRACT, PER CORE: only write RAM_i while core_i_run (and core_run) are 0. An access to a RUNNING core's RAM never gets ready and hangs the transaction. SHARED-MEMORY CONTRACT: the PS window onto SHARED is muxed away while EITHER core runs. Publish the control area and open the barrier BEFORE the release; a write afterwards never gets ready.



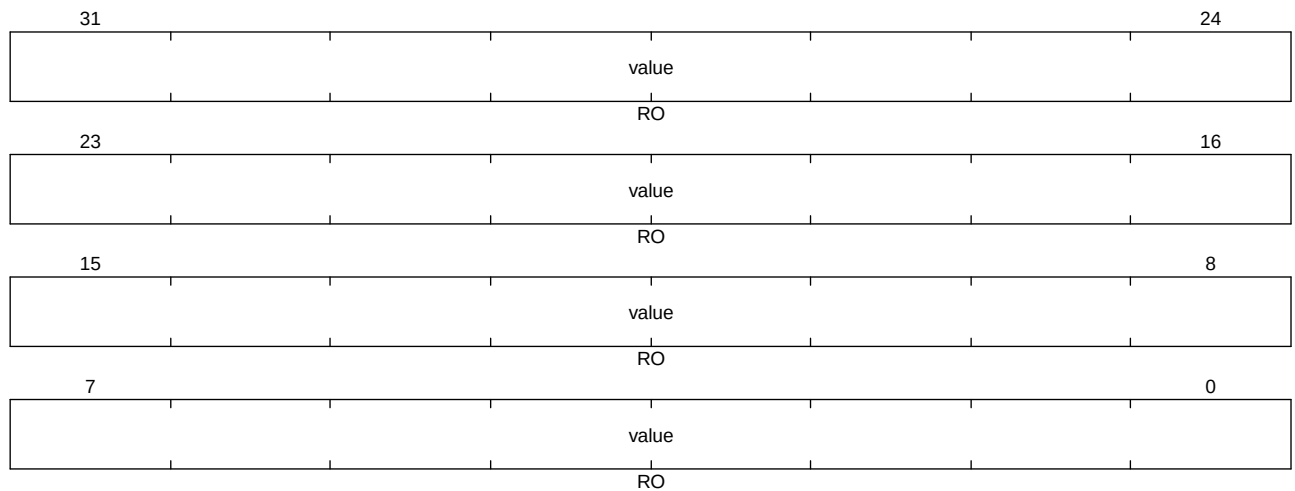
Bits	Field	Access	Reset	Description (SystemRDL of the example)
9	<code>core1_run</code>	RW	0	Run core 1 individually.
8	<code>core0_run</code>	RW	0	Run core 0 individually.
2	<code>trace_flush</code>	RW	0	Request a global funnel flush.
1	<code>trace_clear</code>	RW	0	Level: while set, re-arm the ATB ring.
0	<code>core_run</code>	RW	0	Collective bit: run BOTH cores.

`ctrl.status` – offset 0x004

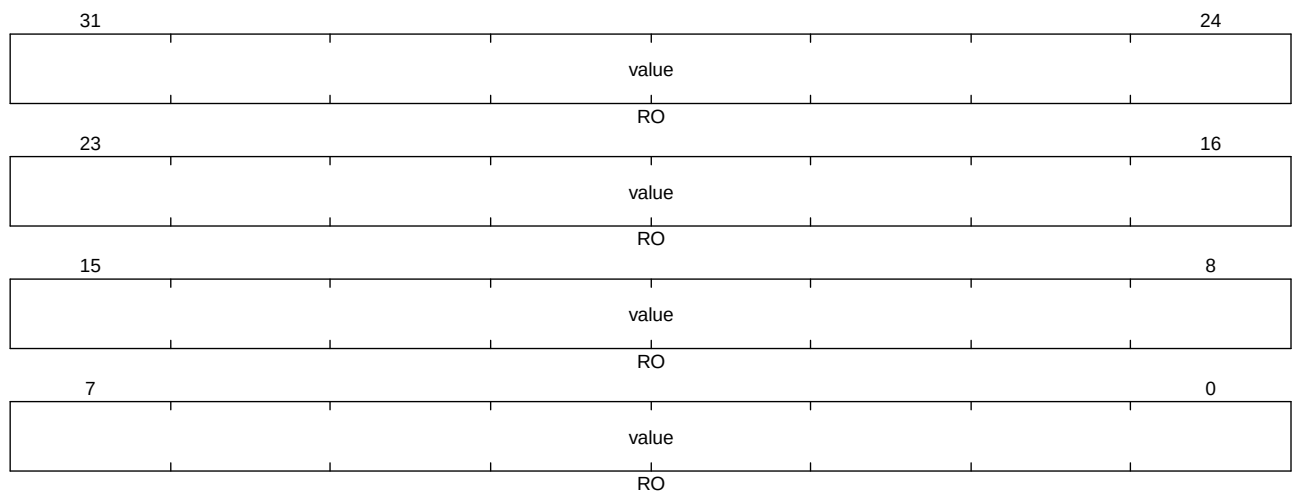
Status register. Cleared by `trace_clear`.



Bits	Field	Access	Reset	Description (SystemRDL of the example)
9	<code>core1_running</code>	RO (hw-written)	0	Core 1 running (effective state).
8	<code>core0_running</code>	RO (hw-written)	0	Core 0 running (effective state).
0	<code>trace_overflow</code>	RO (hw-written)	0	ATB ring wrapped – capture truncated.

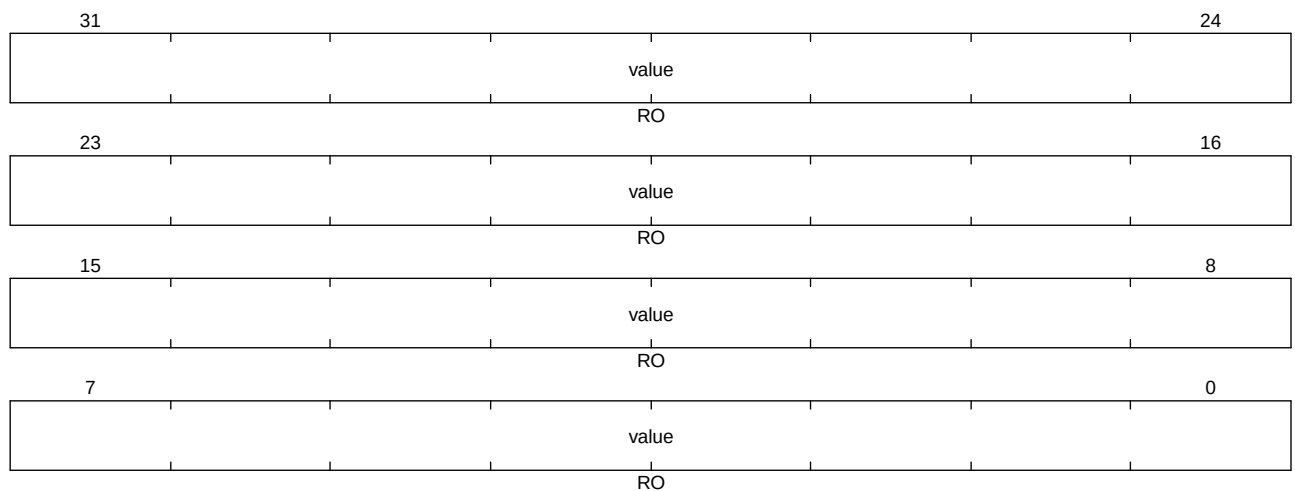
ctrl.trace_beats – offset 0x008

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Merged ATB beats captured into the ring (one 32-bit word per beat).

ctrl.trace_bytes – offset 0x00C

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Valid ATB bytes captured. Only the last beat can be partial.

ctrl.trace_bufsz – offset 0x010

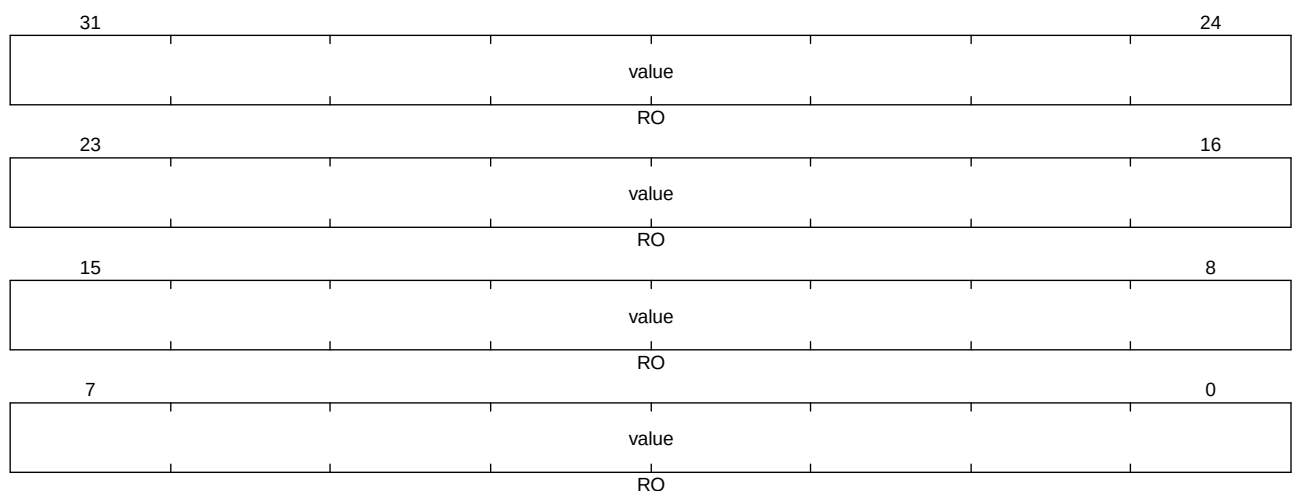


Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	ATB ring capacity in bytes (TRACE_DEPTH * 4).

35.6.5. Registers of obs (base 0x40)

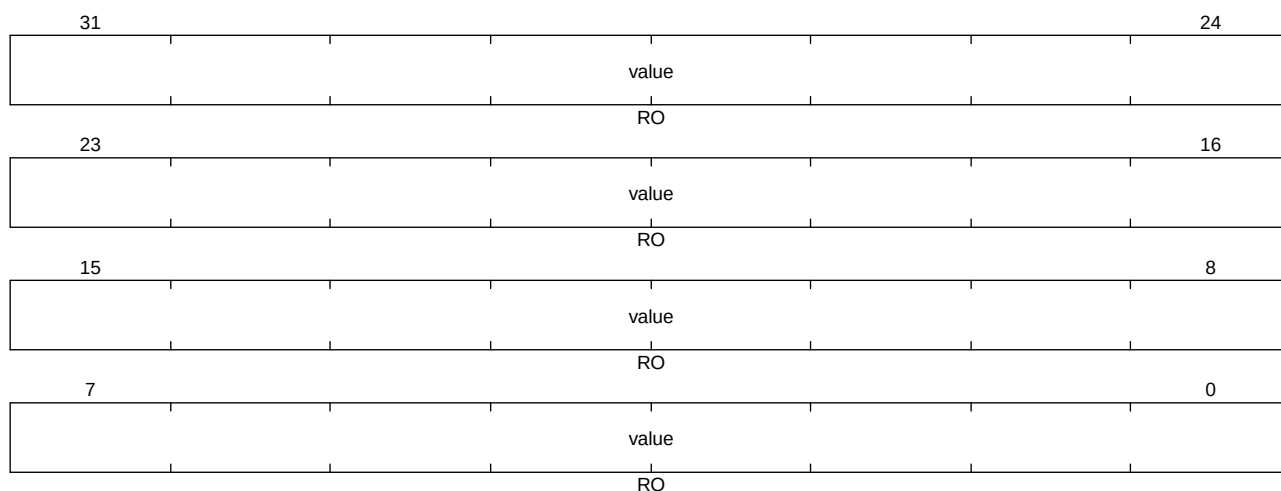
RV/CFI observation bank

obs.db0_hits – offset 0x040



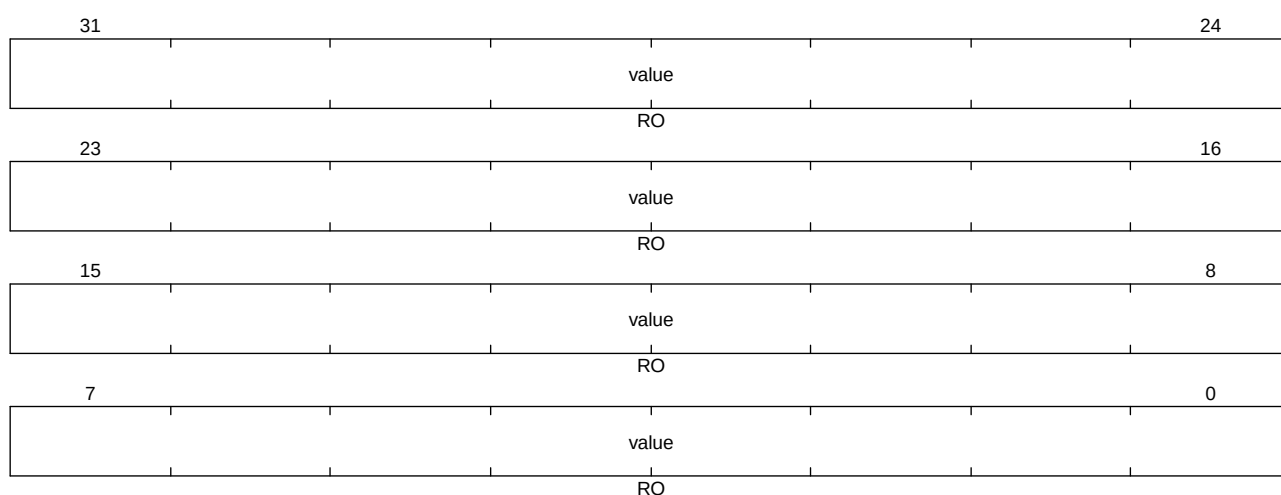
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Doorbell stores accepted, core 0 (saturating).

obs.db0_last – offset 0x044



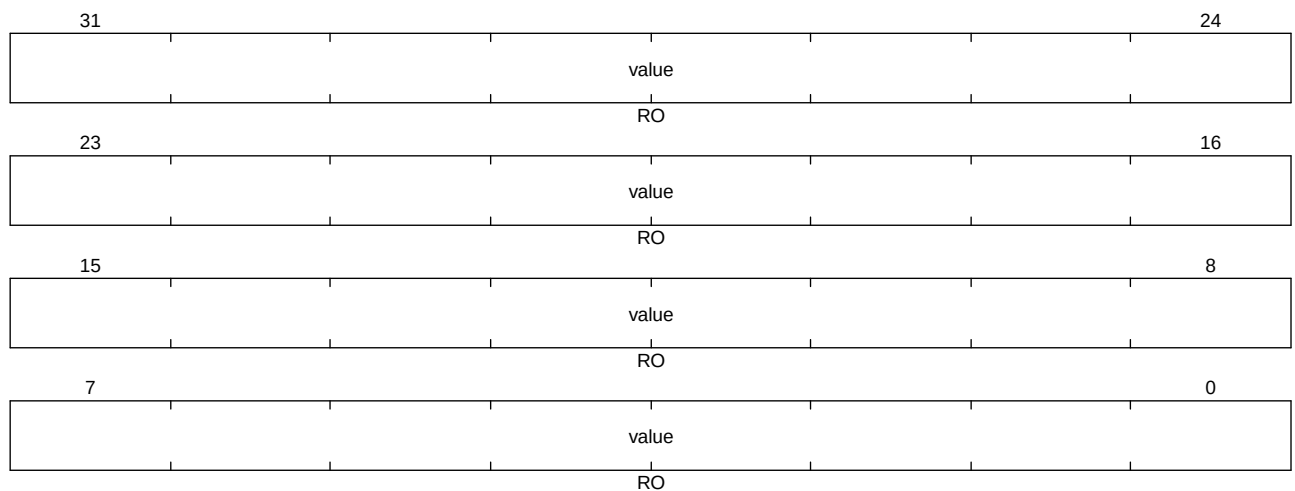
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Last ACT-CAP command word written by core 0.

`obs.db1_hits` – offset 0x048



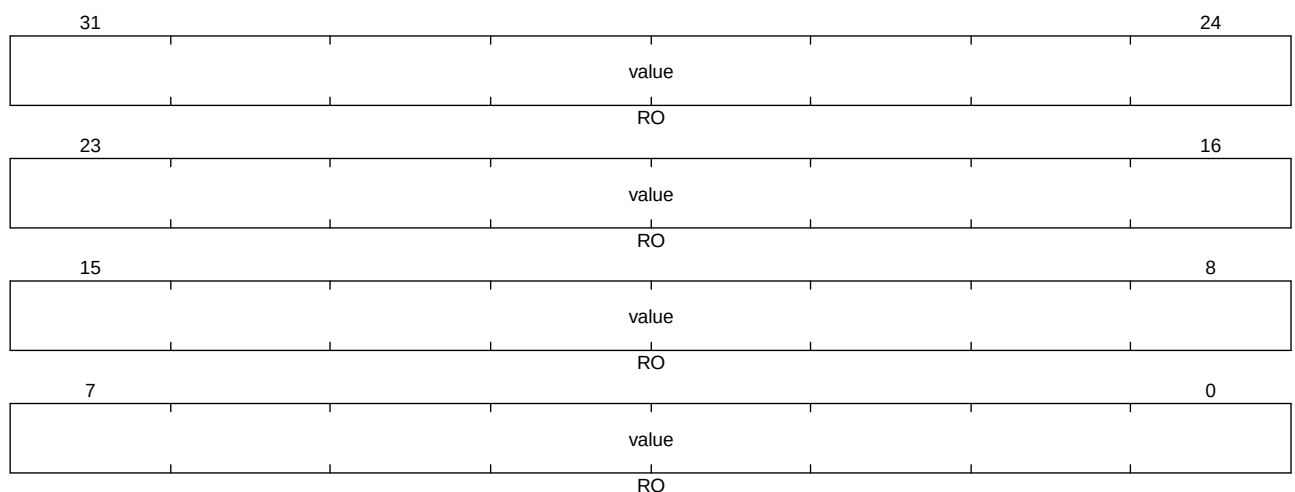
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Doorbell stores accepted, core 1 (saturating).

`obs.db1_last` – offset 0x04C



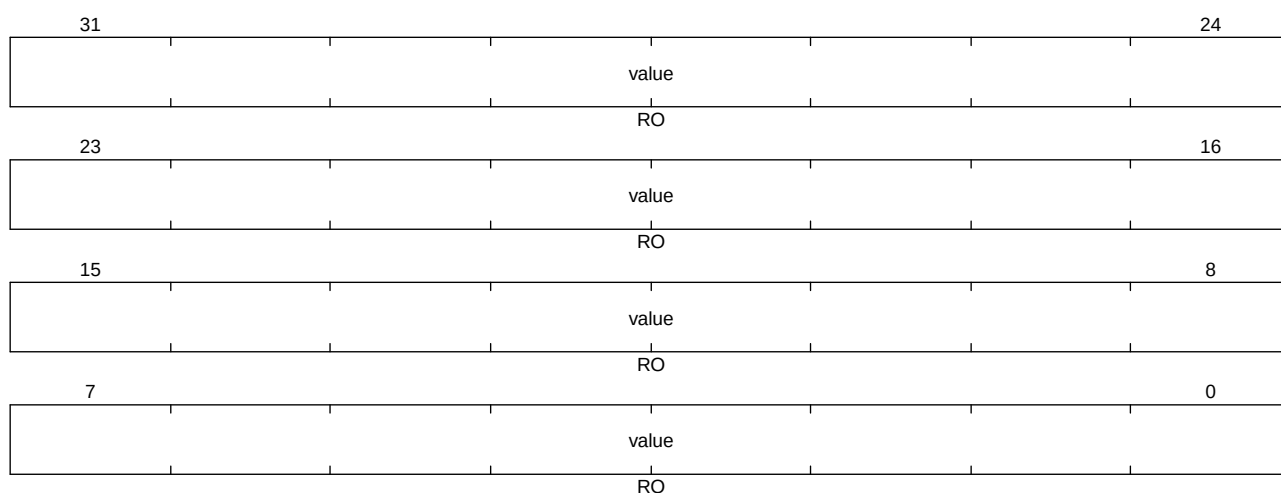
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Last ACT-CAP command word written by core 1.

`obs.actcap0` – offset 0x050



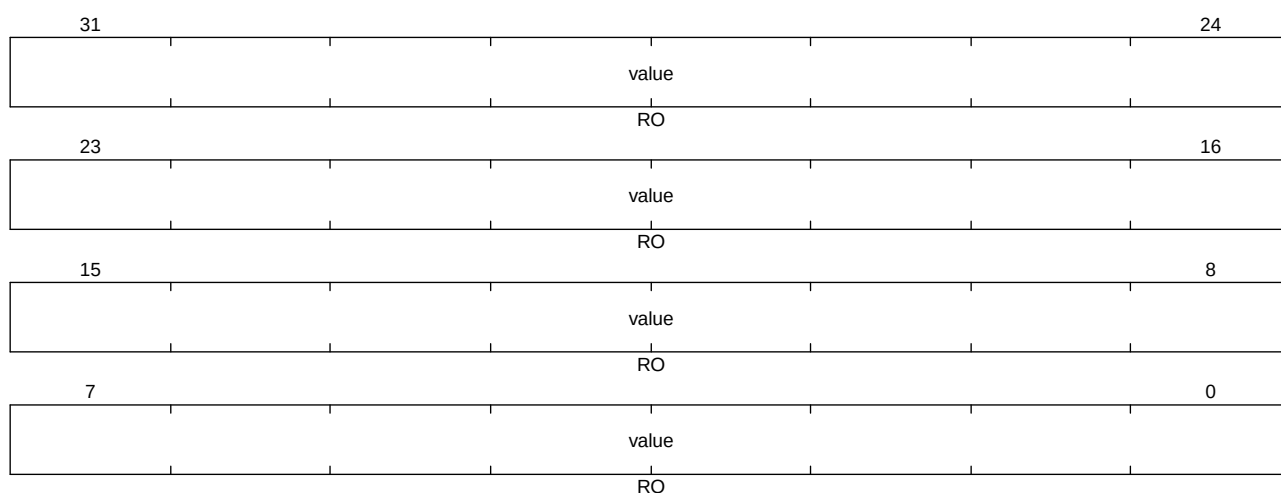
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	TIP beats CONVERTED into ACT-CAP by the adapter, core 0.

`obs.actcap1` – offset 0x054



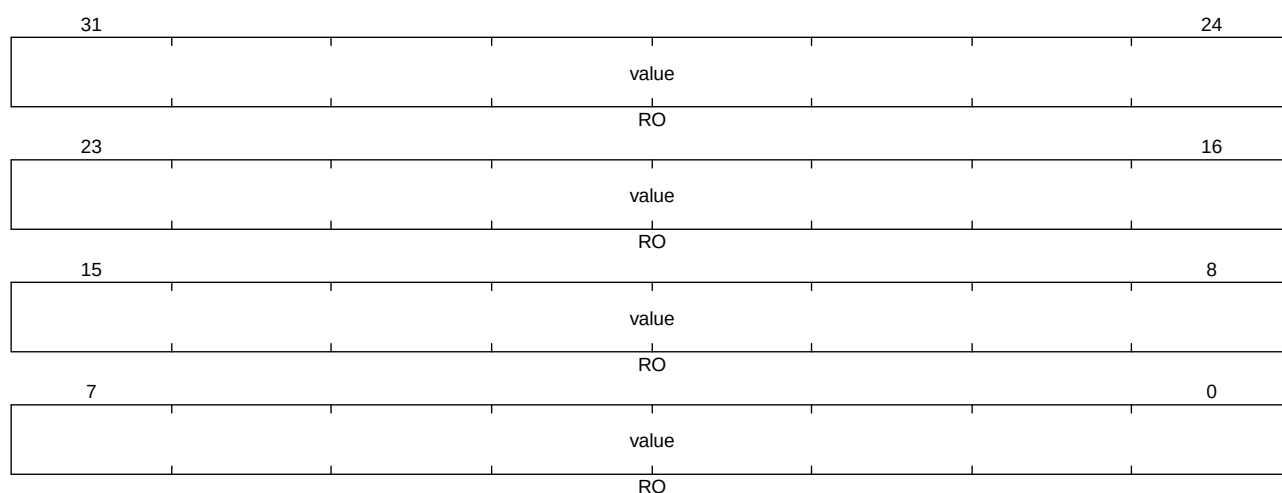
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	TIP beats CONVERTED into ACT-CAP by the adapter, core 1.

obs.shared_sz – offset 0x058



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Shared memory size in bytes (SHARED_KIB * 1024).

obs.magic – offset 0x05C



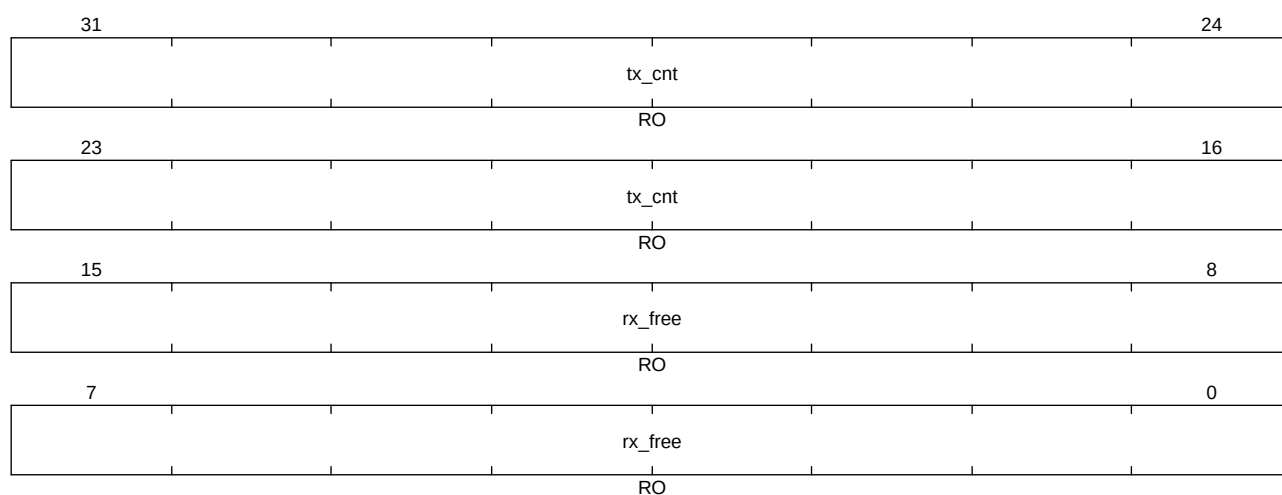
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0x52564349	0x5256_4349 ("RVCI"). The probe that says THIS design is in the PL – an app listed by xmutil is not evidence.

35.6.6. Registers of console (base 0x60)

Per-core console

console.con0_stat – offset 0x060

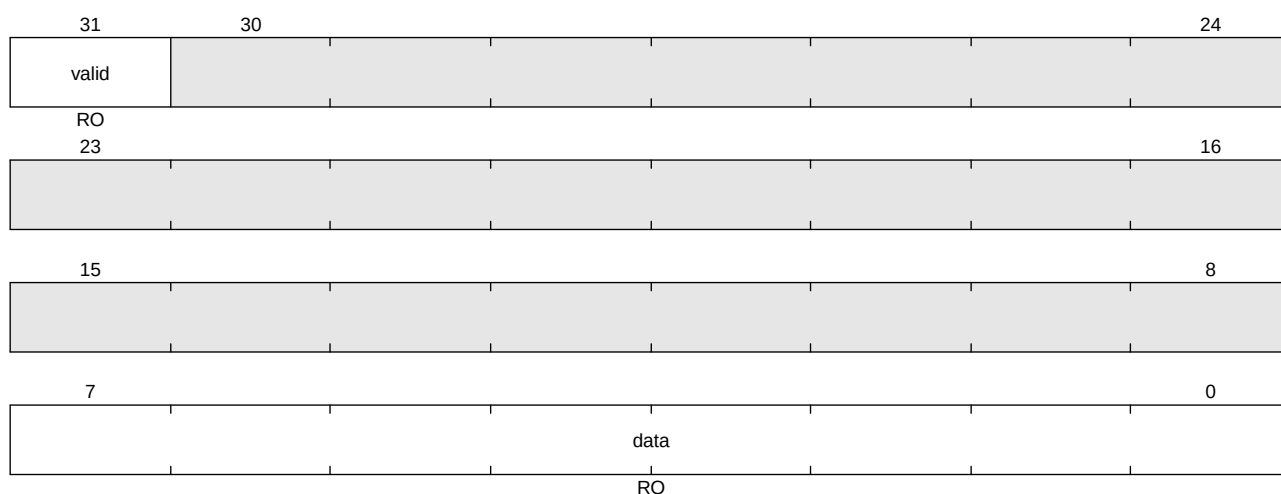
Core 0 console status.



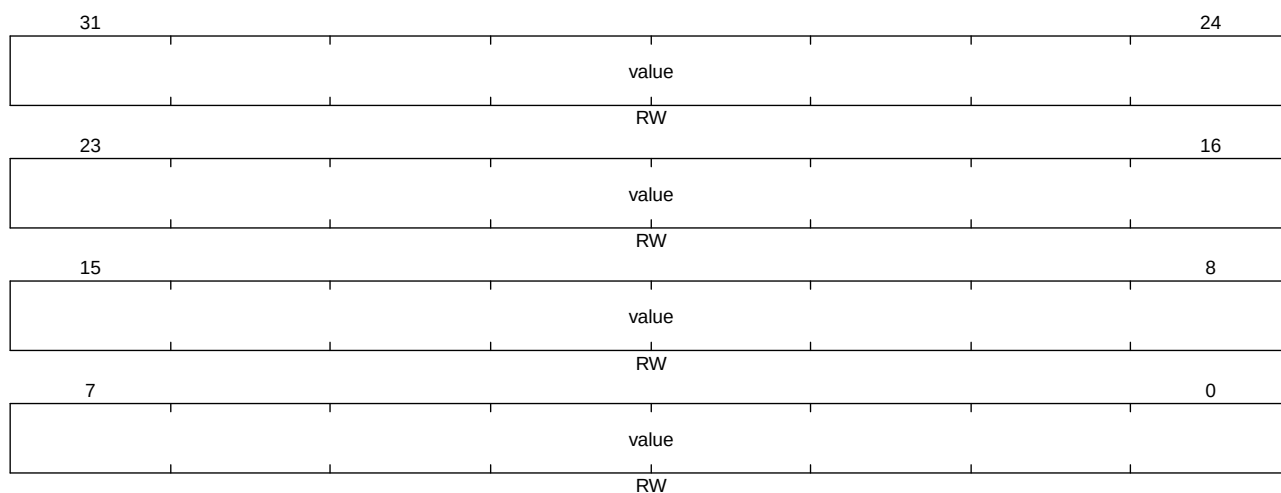
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:16	tx_cnt	RO (hw-written)	0	Bytes waiting in the core's TX FIFO.
15:0	rx_free	RO (hw-written)	0	Free space in the core's RX FIFO.

console.con0_pop – offset 0x064

Core 0 TX pop. READING THIS REGISTER POPS one character.



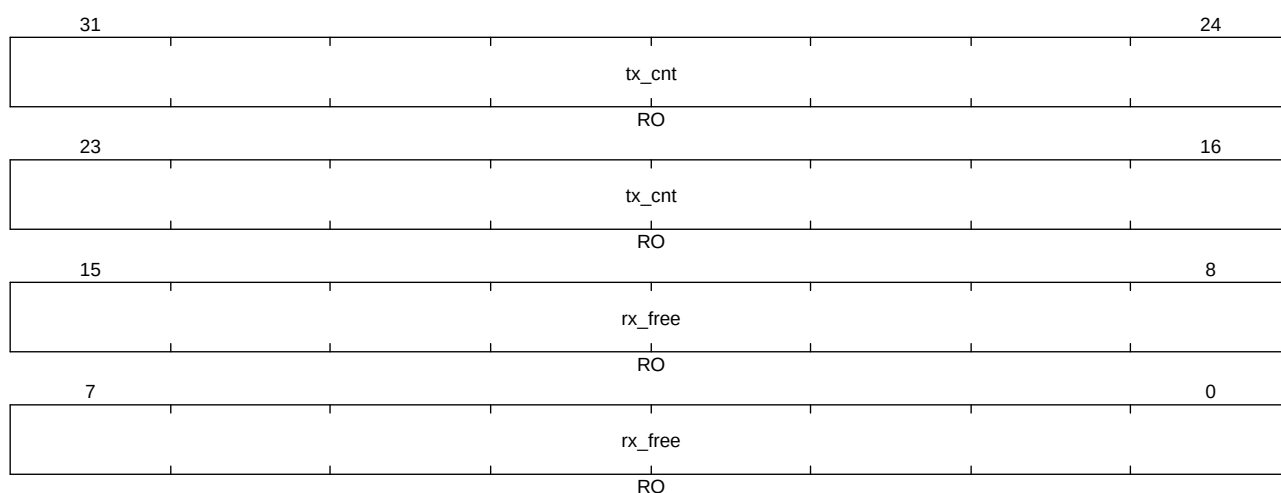
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31	valid	RO (hw-written)	0	1: data valid – 0: FIFO was empty.
7:0	data	RO (hw-written)	0	The popped character.

console.con0_push_drops – offset 0x068

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RW	0	Core 0: WRITE pushes one character into the core's RX FIFO; READ returns the RX overflow counter (pushes dropped because the FIFO was full – counted, never silent).

console.con1_stat – offset 0x070

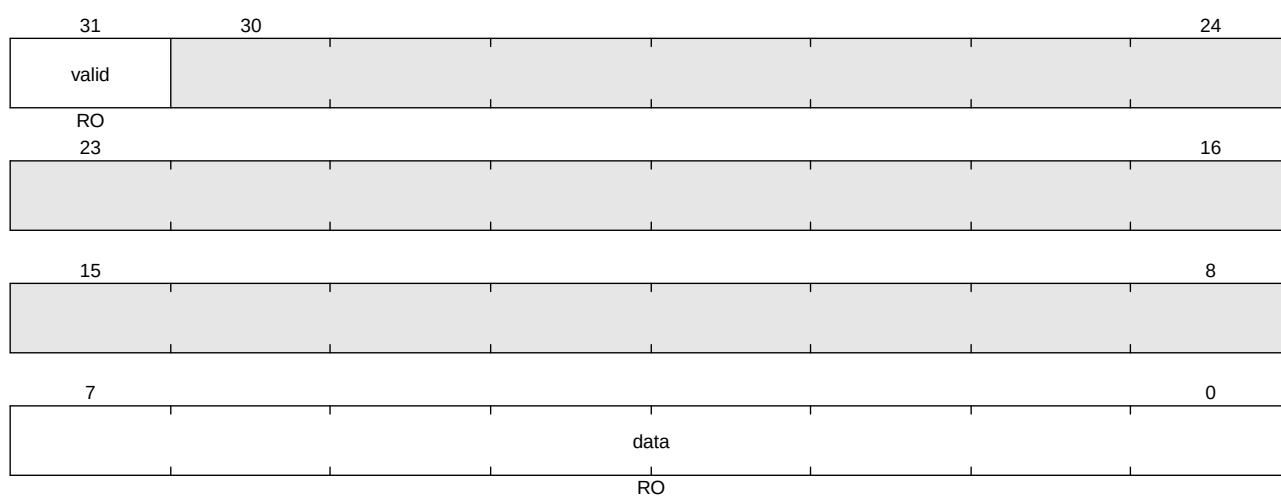
Core 1 console status.



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:16	<code>tx_cnt</code>	RO (hw-written)	0	Bytes waiting in the core's TX FIFO.
15:0	<code>rx_free</code>	RO (hw-written)	0	Free space in the core's RX FIFO.

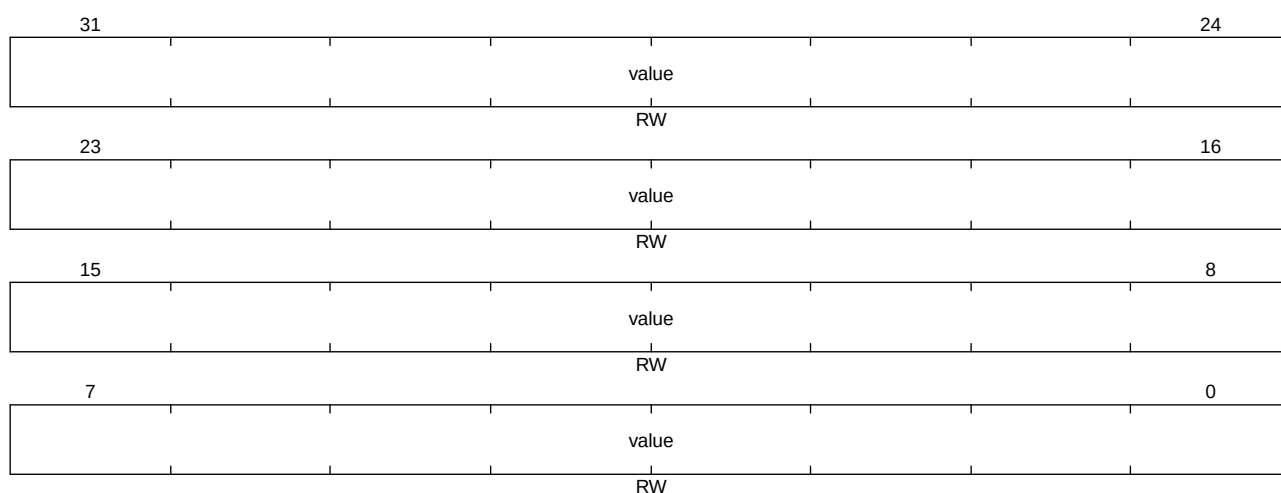
`console.con1_pop` – offset 0x074

Core 1 TX pop. READING THIS REGISTER POPS one character.



Bits	Field	Access	Reset	Description (SystemRDL of the example)
31	<code>valid</code>	RO (hw-written)	0	1: data valid – 0: FIFO was empty.
7:0	<code>data</code>	RO (hw-written)	0	The popped character.

`console.con1_push_drops` – offset 0x078



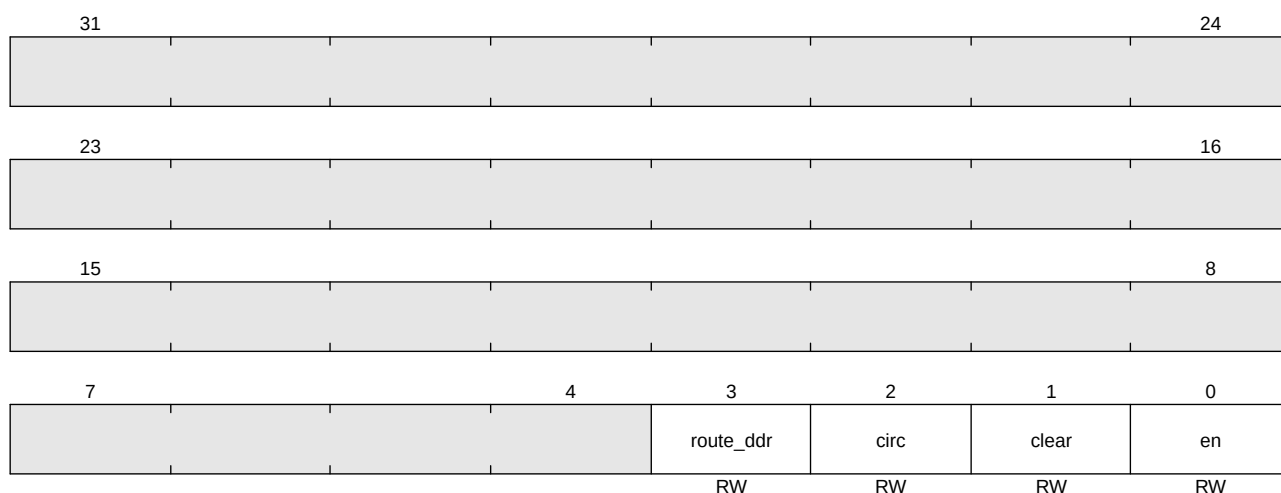
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RW	0	Core 1: WRITE pushes into RX; READ returns the RX overflow count.

35.6.7. Registers of ring0 (base 0x80)

DDR record ring (one instance per core)

ring0.ring_ctrl – offset 0x080

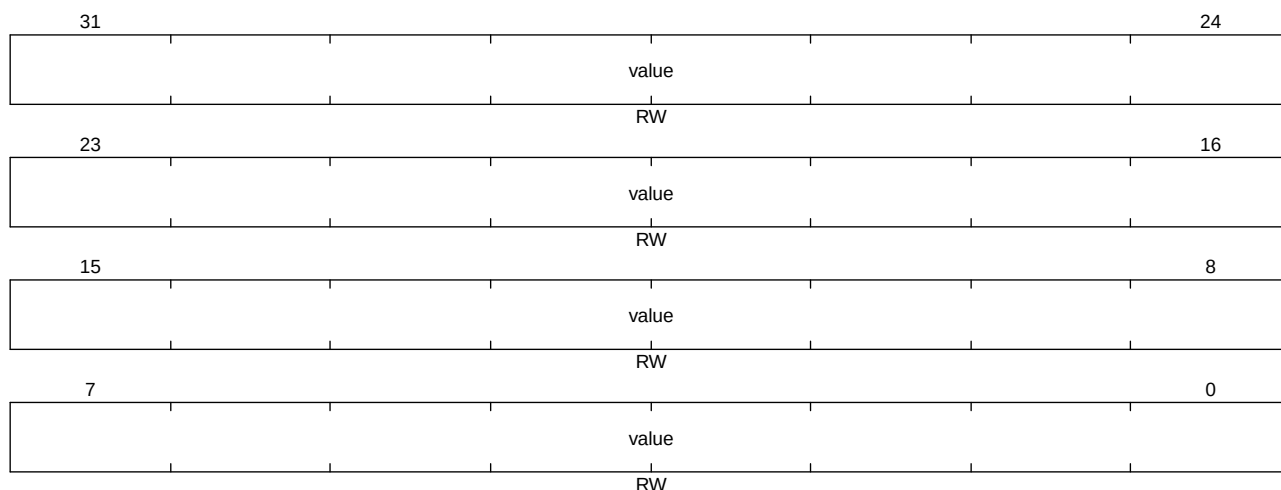
Ring control. Switch the route only with the cores stopped and a clear pulse issued – a mid-record switch tears one record across the two consumers.



Bits	Field	Access	Reset	Description (SystemRDL of the example)
3	route_dds	RW	0	1: records go to the ring – 0: to the MM-FIFO (default, pre-N3 behaviour).
2	circ	RW	1	1: circular (wrap) – 0: one shot.
1	clear	RW	0	Write-1 pulse: clear wptr/stat/beats. Use while en = 0.

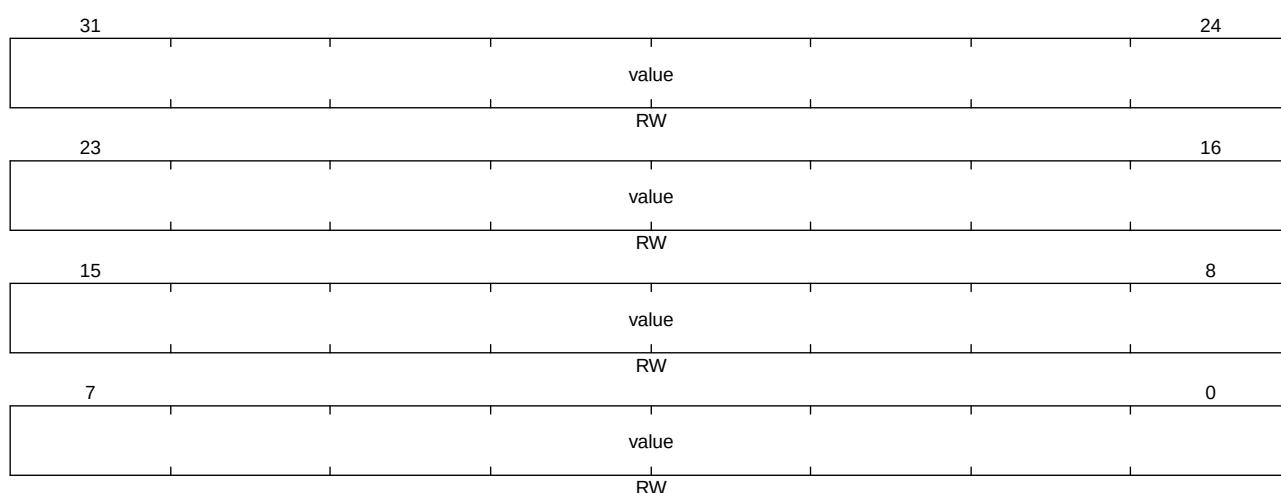
Bits	Field	Access	Reset	Description (SystemRDL of the example)
0	en	RW	0	Enable the sink.

ring0.ring_base – offset 0x084

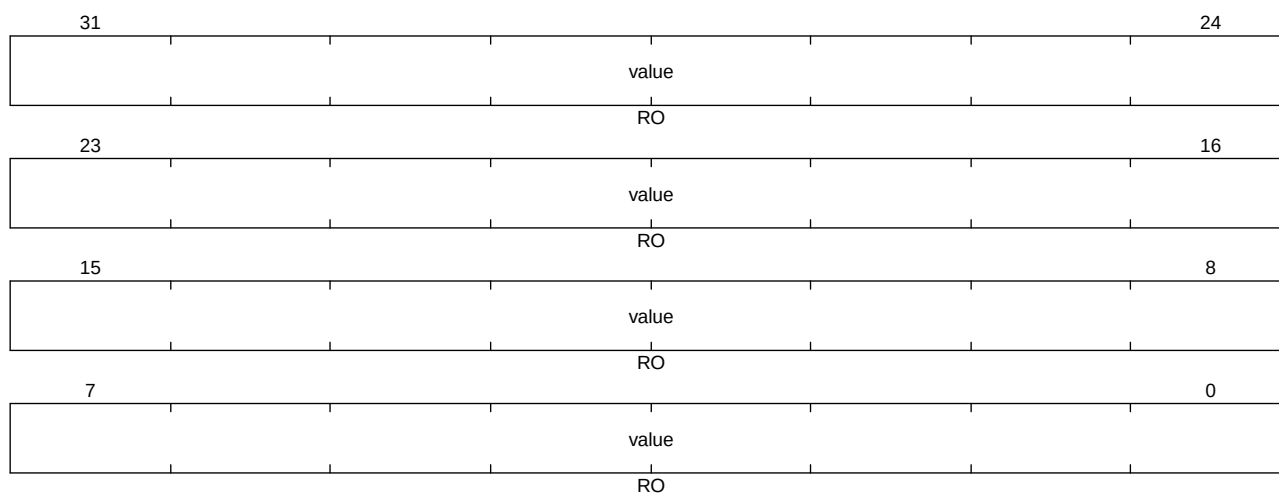


Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RW	0x50000000	Ring base address. WARL: 32-byte aligned and inside [0x5000_0000, 0x6000_0000); writes while en = 1 are rejected and set <code>cfg_rej</code> . Reset: 0x5000_0000 (core 0) / 0x5800_0000 (core 1) – the two halves of the 256-MiB reserved window.

ring0.ring_size – offset 0x088



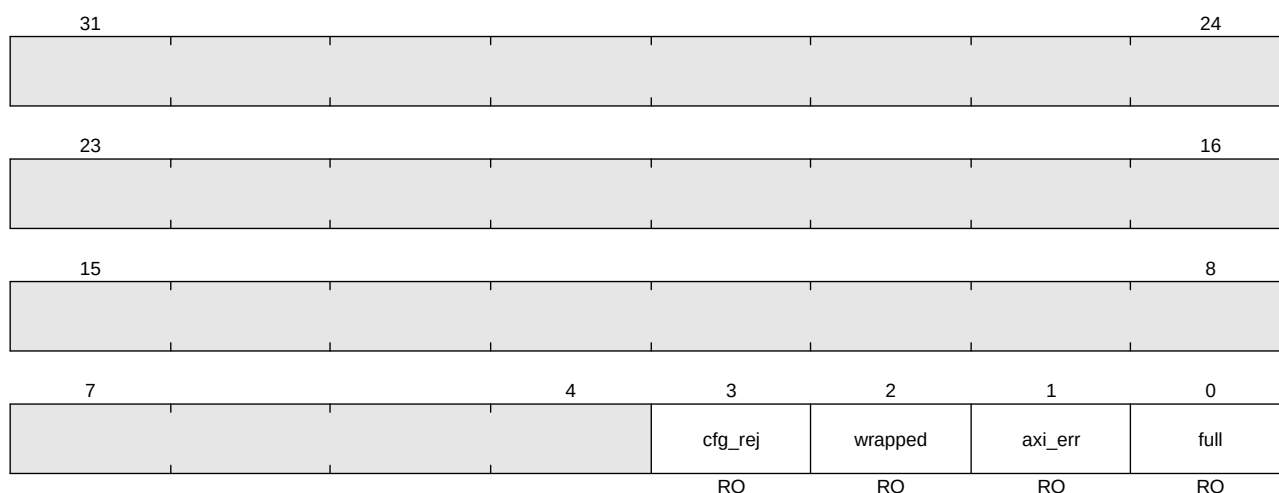
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RW	0x8000000	Ring size in bytes. WARL: multiple of 32, non-zero, and base + size within the window (an exact fit at the window end is legal); writes while en = 1 are rejected. Reset: 128 MiB.

ring0.ring_wptr – offset 0x08C

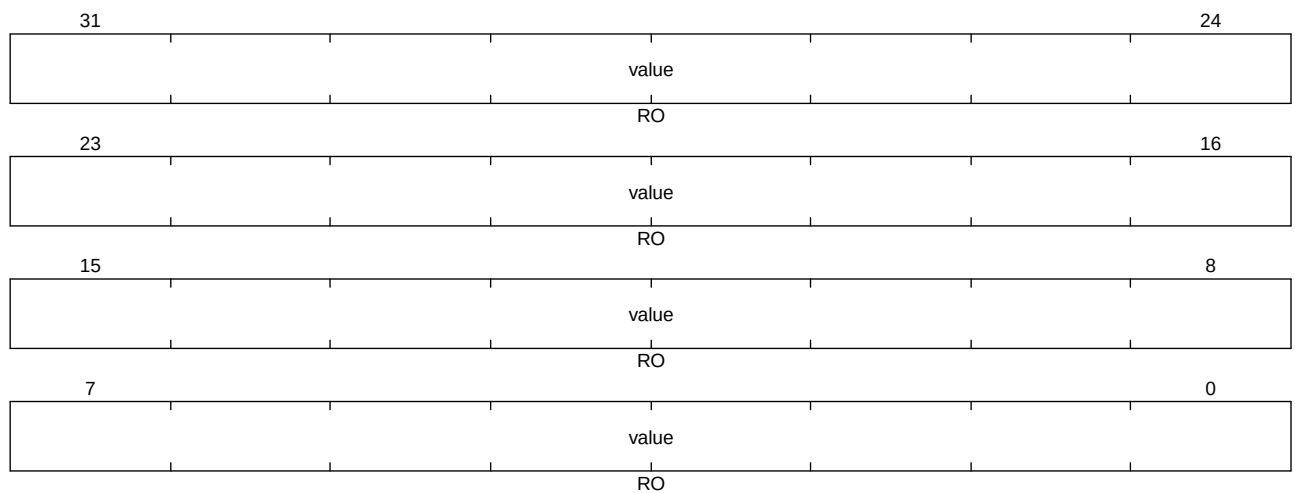
Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	TOTAL bytes written, monotonic – NOT an offset. The write offset is wptr % size; when wrapped, the oldest record starts there and the host re-orders [off..size) + \+ [0..off).

ring0.ring_stat – offset 0x090

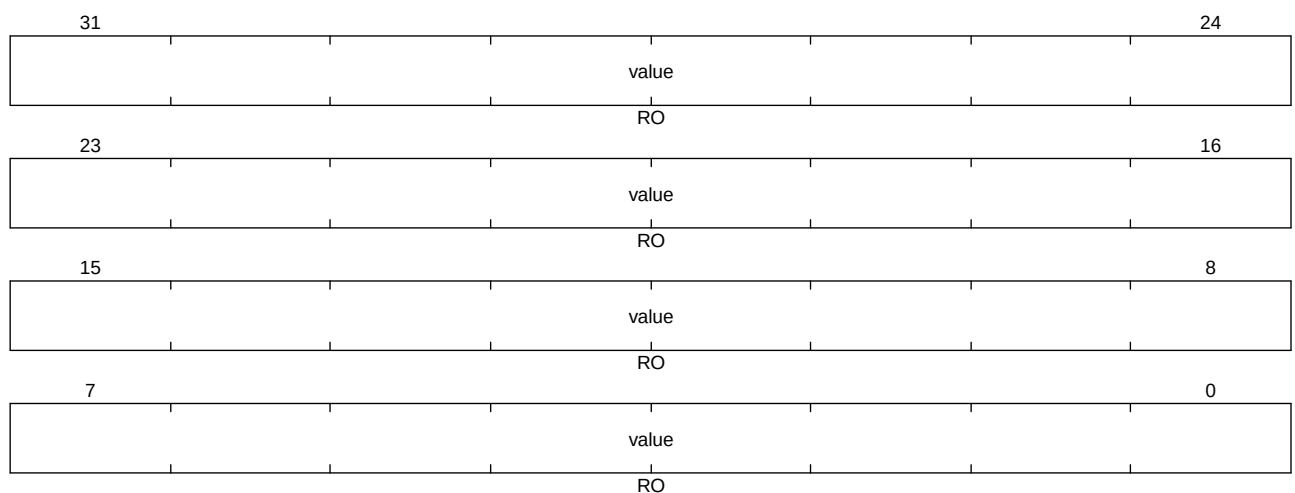
Ring status.



Bits	Field	Access	Reset	Description (SystemRDL of the example)
3	cfg_rej	RO (hw-written)	0	Sticky: a base/size write was rejected (illegal value, or arrived while enabled). Cleared by the clear pulse.
2	wrapped	RO (hw-written)	0	Circular mode: wrapped at least once.
1	axi_err	RO (hw-written)	0	Sticky: a write response was not OKAY.
0	full	RO (hw-written)	0	One-shot mode only: buffer full.

ring0.ring_drops – offset 0x094

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Records the sink itself had to drop (its FIFO was full), saturating. At record rates this must stay 0 – the bench and rvmon both treat any nonzero value as a failed run.

ring0.ring_beats – offset 0x098

Bits	Field	Access	Reset	Description (SystemRDL of the example)
31:0	value	RO (hw-written)	0	Words offered to the sink while enabled: the proof that the feed reaches it at all. wptr = 4 * beats in a healthy run. Cleared by the clear pulse.

35.6.8. Core-side map (not reachable from the PS)

Core-side map (identical for both cores)

Offsets are addresses on the RISC-V cores' own bus; the map is identical for both cores.

Offset	Component	Kind	Size	Content
0	ram	memory	64 KiB	Own program/data RAM 16384 entries of 32 bit.
0x10000000	periph	memory	64 KiB	Own CLINT / INTC 16384 entries of 32 bit.
0x20000000	encoder	address map	24696 B	CEDARtools.TraceEncoder Control / Status Registers
0x30000000	shared	memory	256 KiB	Shared memory – the SAME view from both cores 65536 entries of 32 bit.
0x40000000	actcap_doorbell	register	4 B	ACT-CAP doorbell

35.7. The interrupt peripherals of the example SoC

The example SoC instantiates two small peripherals that the trace subsystem does not use and a booting core does: a core-local interruptor providing a machine timer and a software interrupt, and a minimal external interrupt controller with a single software-triggerable source. Their register names – `msip`, `mtimecmp`, `mtime` – are the names the RISC-V privileged architecture uses for those facilities, and reusing them is what makes the example’s startup code look like ordinary RISC-V startup code.

Neither block is a conformant implementation of the corresponding standard peripheral, and neither claims to be. The SystemRDL calls each one a deliberately minimal teaching peripheral: five registers where a full core-local interruptor has per-hart arrays, two registers where a platform-level interrupt controller has priority, threshold and claim/complete machinery. Their register tables are therefore not printed in this manual. Two reasons, in this order: a reader who needs them is working in the example’s own source tree, where

`examples/kv260/common/tgc5b/rdl/ct_soc/ct_clint.rdl` and `ct_intc.rdl` are twenty-odd lines each and authoritative; and a table in a CEDARtools manual, printed in the same form as the encoder registers, would suggest a conformance claim that the blocks do not make.

The address at which each block sits is printed – it is part of the address space, and [Section 35.4](#) shows it in both the data-bus view and the generated peripheral block.

NOTE

For the machine-mode timer and software-interrupt facilities these two blocks stand in for, the normative document is the RISC-V privileged architecture specification ([riscv/riscv-isa-manual](#)). It is named here for orientation only. It is not one of the specifications CTTE is verified against – those four, and the conformance statement for each, are in [Chapter 36](#) and [Chapter 4](#).

Chapter 36. References

This chapter lists the normative specifications, source repositories, tools, and related Accemic documents behind CEDARtools.TraceEncoder (CTTE). Where a predecessor reference could not be re-verified against a live source, it is marked accordingly instead of being carried over silently.

36.1. Standards and specifications

CTTE targets four public specifications. The authoritative in-repo statement of which parts of each are implemented, and which deviations exist, is

[doc/trace-format.adoc](#)

(conformance section) in the public repository; this reference list only points to the source documents themselves.

Specification	Public source
The Nexus 5001 Forum™ Standard for a Global Embedded Processor Debug Interface, IEEE-ISTO 5001-2012, v3.0.1	riscv-nexus-trace — cited under the <code>:nexus-spec:</code> attribute in <code>doc/architecture.adoc</code> , <code>doc/trace-format.adoc</code> and <code>doc/enhanced-features.adoc</code> .
RISC-V N-Trace (Nexus-based Trace) Specification, Version 1.0 (ratified 2024-11-21), RISC-V International	riscv-non-isa/riscv-nexus-trace (working repository of the RISC-V Nexus Trace Task Group; also hosts the Trace Control Interface and Trace Connectors specifications). CTTE's own citation for the N-Trace text points to riscv-non-isa/riscv-trace-spec (<code>doc/trace-format.adoc</code> , its <code>specification-link</code> attributes) — both repositories currently serve N-Trace content; consult the upstream Task Group for the current split.
RISC-V Trace Control Interface Specification, Version 1.0 (ratified 2024-11-21), RISC-V International	docs.riscv.org (cited in <code>doc/architecture.adoc</code>)
Efficient Trace for RISC-V, Version 2.0 (ratified 2022-05-05), RISC-V International — the E-Trace <code>te_inst</code> packet format CTTE's E-Trace profile emits (<code>ct_L2_te_inst_gen</code> , see <code>doc/integration.adoc</code> , "Trace egress")	riscv-non-isa/riscv-trace-spec
NexRv Tool (Nexus trace for RISC-V) description	see Section 36.5 (<code>accemic/CTTD</code> , which absorbed the earlier NexRv fork)

36.2. Core and trace-source adapters

Sources for the cores CTTE's ingress side is verified against, and for the RTL that adapts each core's native trace/retire interface to CTTE's TIP (`tip_if`, `rtl/pkg/tip_pkg.sv`):

Core / interface	Where the adapter lives
MicroBlaze™ V TRACE bus	rtl/adapters/amd_microblaze_v/ (RTL); semantics and signal mapping in doc/adapters/microblaze_v_adapter_mapping.adoc, microblaze_v_trace_ports.adoc, microblaze_v_trace_semantics.adoc, microblaze_v_supported_configurations.adoc, and the itype decode vectors in itype_decoder_vectors.csv.
CVA6 Instruction Trace Interface (ITI)	rtl/adapters/cva6/ (RTL: cva6_iti_to_ctte_tip.sv, cva6_riscv_itype_refine.sv); semantics in doc/adapters/cva6_iti_semantics.adoc. The upstream CVA6 ITI itself was added in OpenHW Group's cva6 repository, commit f314dcb1 ("Instruction Trace Interface", PR #2927; verified reachable 2026-08-18).
Rocket Trace Core Interface (TCI)	rtl/adapters/rocket/ (RTL: rocket_tci_to_ctte_tip.sv); semantics in doc/adapters/rocket_tci_semantics.adoc.
MINRES TGC5B (TGC5B_AXI4L_H2E), example-SoC core only	Vendored verbatim at examples/kv260/common/tgc5b/cpu/TGC5B_AXI4L_H2E.sv; provenance and dual license (CERN-OHL-S-2.0 OR LicenseRef-MINRES-Commercial) in examples/kv260/common/tgc5b/cpu/README.md. Not part of the CTTE IP itself — see Chapter 37 .

NOTE

The predecessor document listed a partner-channel TRISTAN Trace IP repository (git.minres.com/TRISTAN/TraceIP) for MINRES integration collateral. That host requires partner/internal credentials and was not re-verified here; it is not a public reference.

36.3. Tools

Tool	Version (as pinned)	Purpose / source
abc-flow	pinned per .abc.config	Text-defined, Vivado-based build driver for every make target. Public repository: accemic/abc-flow (verified 2026-08-18).
Verilator	resolved via VERILATOR_ROOT (see doc/development.adoc)	Default simulation backend (make sim, coverage runs). Project: veripool.org/verilator . Guided by the CHIPS Alliance and Linux Foundation.
Xilinx® Vivado™	pinned in .abc.config	Alternate simulation backend for the cli_* gate battery (abc --sim-backend vivado -sim ...); required for the xsim-based byte-neutrality and gate scripts (doc/development.adoc).
Verible (verible-verilog-lint)	n/a (opt-in ruleset, .rules.verible_lint)	Static lint for rtl/ and tests/, invoked by make lint. Project: chipsalliance/verible (CHIPS Alliance).
PeakRDL toolchain (peakrdl, peakrdl-regblock, peakrdl-html, systemrdl-compiler)	1.5.0 / 1.3.1 / 2.12.2 / 1.32.2	Register (RDL) flow: make rdl / make rdl-html; versions pinned in rdl/requirements.txt. Project: peakrdl.readthedocs.io .
CTTD reference decoder (bin/cttd-<platform>)	pinned by version and per-platform sha256 in scripts/cttd.pin	Offline oracle decoder used by scripts/decode_and_check.sh. Fetched by scripts/fetch_cttd.py, never committed; descended from the IAR Systems NexRv tool. See Section 29.4 .

36.4. Related Accemic documents

- The public documentation set in the `TraceEncoder` repository (`doc/architecture.adoc`, `doc/development.adoc`, `doc/enhanced-features.adoc`, `doc/integration.adoc`, `doc/release-notes.adoc`, `doc/trace-format.adoc`, `doc/verification.adoc`, `doc/adapters/*.adoc`) is this Reference Manual's primary technical source for RTL-facing content; where this RM condenses or reorganizes that material, the public chapter is the fuller treatment.
- `doc/release-notes.adoc` – version history and the profile-level resource/timing figures cited in the Footprint profiles chapter.
- This Reference Manual's own predecessor, `rm-ctrace.adoc` (revisions 1.0 to 1.3), which this edition supersedes. It was removed from the `doc` repository when this edition replaced it and remains available in the repository history; an internal migration plan records what changed and why.
- CEDARtools.TraceDecoder (CTTD) – the reference decoder, public under [accemic/CTTD](#) since 2026-08-27 with sealed binary releases (current `v2.3.0-cttd3`: three platform binaries plus `SHA256SUMS`); CTTE pins it by tag and checksum in `scripts/cttd.pin`.

36.5. Public repositories

Repository	Public URL	Notes
CEDARtools.TraceEncoder	accemic/TraceEncoder	The GitHub address is the repository's public home and the one its own README links. The earlier short name C-Trace is retired from prose and survives only in identifiers and in names fixed outside the repository (Chapter 1); addresses built from it are not guaranteed to resolve and should not be relied on.
CEDARtools.TraceDecoder (CTTD)	accemic/CTTD	The reference decoder (previous section), derived from the IAR Systems NexRv tool; its earlier repository names (most recently <code>NexRv-for-TraceEncoder</code>) redirect to the current one (verified 2026-08-28). Every decode verdict in this manual is CTTD's; see Section 29.4 .
CTXP format specification	accemic/CTXP-format	"Format Specification for Unified Processor Trace." Renamed from its C-Trace-era name; the encoder repository's verification chapter cites the current name directly. Not the same repository as CTTD (previous row).
Trace-FS (TRISTAN demonstrator)	Fraunhofer-IPMS-DCC/Trace-FS	Functional-safety (ISO 26262 / IEC 61508) tracing demonstrator on a KR260 lockstep RISC-V subsystem (Fraunhofer IPMS EMSA5), with a TSN trace link. The encoder integrated in the repository as published is an earlier C-Trace release, predating CTTE release 1.0.0 – its README already uses the CEDARtools.TraceEncoder name, but nothing in this manual's verification evidence covers that integration (checked 2026-08-27).

Repository	Public URL	Notes
Trace-Link	Fraunhofer-IPMS-DCC/Trace-Link	IP core packaging ATB trace data into IPv4/UDP Ethernet frames over AXI4-Stream, used by the Trace-FS demonstrator.
abc-flow	accemic/abc-flow	Build driver, see Section 36.3 above.

All URLs in this section were confirmed reachable on 2026-08-28. Several carry a rename history (old names still resolve via GitHub's redirect); where this document's own predecessor cited the pre-rename name, that is called out above rather than silently updated.

36.6. TRISTAN acknowledgment

Parts of CTTE were developed as part of the TRISTAN project, a European Union research initiative. The full acknowledgment text and funding reference are given in `README.md` of the public repository: TRISTAN, project no. 101095947, is supported by the Chips Joint Undertaking (CHIPS-JU) and its member states; see <https://tristan-project.eu/> for details.

36.7. Related ecosystem references

Prior art and adjacent open trace implementations referenced during CTTE's development (all links verified reachable 2026-08-18):

- [Minres GitHub](#) – vendor of the TGC5B core used in the example SoC.
- [pulp-platform/rv_tracer](#) – "RISC-V E-trace specification RTL implementation" (ETH Zurich / University of Bologna PULP project).
- [CVA6 instruction tracing tree](#) (OpenHW Group `cva6`, default branch `master`).
- [RVFI-to-TIP reference](#) – `cva6_iti` module at a pinned historical commit.
- [RISC-V Formal Verification Framework](#), in particular its [PC checks](#) procedure.
- [CORE-V-VERIF](#) quick start.
- Siemens, "RISC-V Trace Encoder – Complete Trace Solution" product flyer.
- Gaisler GRLIB IP Core User's Manual (GRIP): LEON3 trace (p. 226), LEON4 trace (p. 242).

Chapter 37. Copyright, License Model and Trademarks

This chapter states two **separate** things that must not be conflated: the license of **this documentation**, and the license of the **CEDARtools.TraceEncoder (CTTE) IP** it describes. It also states the trademark rules that apply when writing about CTTE, RISC-V, and the cores and tools it interoperates with.

37.1. This document's license

Copyright © 2026 Accemic Technologies GmbH. This document (and CTTE's documentation generally) is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). Under this license the work may be shared and adapted for any purpose, including commercial, provided appropriate credit is given, a link to the license is provided, and any changes are indicated. Full legal text: <https://creativecommons.org/licenses/by/4.0/legalcode.en> (plaintext: <https://creativecommons.org/licenses/by/4.0/legalcode.txt>), SPDX identifier CC-BY-4.0.

NOTE

The IP-core license of the CTTE encoder itself is **separate** from this documentation license and is described in [IP license model](#) below.

37.1.1. References used, and attribution for excerpts

This document, and CTTE's public documentation generally, cites and/or draws on the following technical specifications; excerpts, paraphrases, or derived explanations from these sources are credited accordingly (title, author, source, license):

- RISC-V N-Trace (Nexus-based Trace) Specification, Version 1.0 (ratified 2024-11-21), RISC-V International; CC BY 4.0 per the document's own copyright and license section. Source: see the References chapter.
- RISC-V Trace Control Interface Specification, Version 1.0 (ratified 2024-11-21), RISC-V International; CC BY 4.0, same terms.
- Efficient Trace for RISC-V, Version 2.0 (ratified 2022-05-05), RISC-V International; CC BY 4.0 per the source repository's `LICENSE` file.
- The Nexus 5001 Forum™ Standard for a Global Embedded Processor Debug Interface, IEEE-ISTO 5001-2012 v3.0.1, IEEE Industry Standards and Technology Organization (IEEE-ISTO) / Nexus 5001 Forum. **Not CC BY 4.0** — this document carries its own copyright terms, which are reproduced in [Section 37.1.2](#) below as those terms require.

Where this document quotes, paraphrases, or adapts figures or tables from the **three RISC-V International** specifications above, that content is marked and attributed, and modifications are identified, consistent with CC BY 4.0's attribution and "indicate changes" requirements. Material

from the Nexus standard is covered by [Section 37.1.2](#) instead. Citing or excerpting these specifications does not imply endorsement by RISC-V International, the IEEE-ISTO, or any contributor to the cited specifications, and no affiliation is claimed beyond citation and attribution.

A copy of the CC BY 4.0 license (or its URL) accompanies any redistributed or adapted version of this work, with attribution retained and changes identified.

37.1.2. Notice required by the Nexus 5001 Forum™ standard

This manual explains and assists in the implementation of the Nexus standard and quotes field descriptions from it. The standard permits exactly that, on the condition that its copyright notice, the paragraph below and the document title are carried on every such work. They are therefore reproduced here verbatim:

Copyright © 2012 IEEE-ISTO. All rights reserved.

This document may be copied and furnished to others, and derivative works that comment on, or otherwise explain it or assist in its implementation may be prepared, copied, published and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice, this paragraph and the title of the Document as referenced below are included on all such copies and derivative works. However, this document itself may not be modified in any way, such as by removing the copyright notice or references to the IEEE-ISTO and the Nexus 5001 Forum™.

Title: The Nexus 5001 Forum™ Standard for a Global Embedded Processor Debug Interface, Version 3.0

– IEEE-ISTO 5001-2012 v3.0.1, Copyright notice

The IEEE-ISTO and the Nexus 5001 Forum™ disclaim all warranties, express or implied, for the standard itself; that disclaimer is theirs and is not repeated on Accemic's behalf. The standard may be updated, replaced or made obsolete at any time; this manual describes CTTE against the version named above.

This document is provided "as-is" without warranties of any kind; CC BY 4.0 does not imply endorsement or constitute legal advice. All product names, logos, and brands mentioned remain the property of their respective owners; see [Trademarks and naming](#) below for the specific rules that apply to this repository.

For attribution corrections, permissions beyond CC BY 4.0, or questions about included third-party material, contact info@accemic.com.

37.2. IP license model

CTTE is licensed **per artifact type**: which license applies to a given file is declared by that file's own `SPDX-License-Identifier` header, or – for files that cannot carry a header – by `REUSE.toml`.

The repository states that it is **REUSE**-compliant, so the machine-readable headers are the authoritative statement; this chapter is the human-readable summary (`LICENSE.md`). The full text of every license used ships in `LICENSES/`.

Artifact	What it covers	License
Hardware IP	RTL (<code>.sv</code>), register description (<code>.rdl</code>), timing/placement constraints (<code>.xdc</code>), and the SystemVerilog/ <code>.abc</code> verification testbenches	CERN-OHL-S-2.0 OR LicenseRef-Accemic-Commercial (dual)
Software	Build, tooling and CI – shell/Python scripts, <code>Makefile</code> , GitHub workflows, config and metadata	ISC
Documentation	Markdown, AsciiDoc, and documentation images	CC-BY-4.0
Third-party IP	MINRES TGC5B core, vendored for the example SoC (<code>examples/kv260/common/tgc5b/cpu/</code>)	CERN-OHL-S-2.0 OR LicenseRef-MINRES-Commercial (dual – licensed by MINRES, not Accemic)
Fetches, not vendored	the CTTD reference decoder under <code>bin/</code> (gitignored; descended from the IAR Systems NexRv tool)	ISC upstream; CTTD itself is a separate Accemic product under its own terms and is not conveyed with this repository

37.2.1. Hardware IP – dual license

The hardware design and its testbenches are dual-licensed. Either:

1. **CERN Open Hardware License Version 2 – Strongly Reciprocal (CERN-OHL-S-2.0)** – the default open-source license. It ensures modifications and derivative works remain open when distributed. Full text: `LICENSES/CERN-OHL-S-2.0.txt`; SPDX record: <https://spdx.org/licenses/CERN-OHL-S-2.0.html>.
2. **Accemic Commercial License** – for organizations that require integration without the copyleft obligations of CERN-OHL-S-2.0. Text: `LICENSES/LicenseRef-Accemic-Commercial.txt`. Inquiries: sales@accemic.com.

The SPDX expression on these files is

SPDX-License-Identifier: CERN-OHL-S-2.0 OR LicenseRef-Accemic-Commercial.

37.2.2. Third-party IP – the MINRES TGC5B core

`examples/kv260/common/tgc5b/cpu/TGC5B_AXI4L_H2E.sv` is not Accemic's work: it is the MINRES TGC5B RISC-V core (Copyright 2020-2022 MINRES Technologies GmbH), vendored so the integration example builds and simulates against a real core. MINRES has permitted its publication under the same dual-license structure this repository uses for its own hardware IP: `SPDX-License-Identifier: CERN-OHL-S-2.0 OR LicenseRef-MINRES-Commercial` – but **MINRES**, not Accemic, is the licensor of both arms (`LICENSES/LicenseRef-MINRES-Commercial.txt`). Accemic conveys the file verbatim under CERN-OHL-S-2.0 and holds no right to sublicense it: the file is byte-identical to MINRES' delivery, and its SPDX metadata lives in a `REUSE .license` sidecar so the upstream notices stay intact. A commercial license for the core comes from MINRES only; an Accemic commercial license for the encoder conveys no rights to the core, and vice versa. The permission covers this specific delivered netlist (core configuration

TGC5B_AXI4L_H2E) only, not other TGC5B configurations or versions. The encoder IP itself does not depend on the core – nothing under `rtl/`, `rdl/`, or `tests/` references it.

What this means when shipping a product:

You want to ship	Encoder	TGC5B core	License(s) needed
An open-source design	CERN-OHL-S-2.0	CERN-OHL-S-2.0	none – both open arms apply
A closed-source design with your own core	Accemic commercial	not used	Accemic only
A closed-source design containing the TGC5B	Accemic commercial	MINRES commercial	both, negotiated separately
Evaluation, simulation, internal development	either	either	none

The third row follows from CERN-OHL-S-2.0 itself: including Covered Source in a larger work makes the larger work modified Covered Source (§3.2), and conveying a Product built from it requires providing the Complete Source (§4). Keeping the TGC5B in a product therefore keeps the product under CERN-OHL-S-2.0 regardless of the encoder's license. The second row is the usual case – the example SoC is a worked integration, not part of the IP deliverable, and integrators bring their own core.

37.2.3. Software – ISC

Build scripts, tooling, CI, and configuration are licensed under the permissive ISC license. Full text: `LICENSES/ISC.txt`.

37.2.4. Documentation – CC BY 4.0

Public documentation (Markdown, AsciiDoc) and documentation images are licensed under CC-BY-4.0 – shareable and adaptable with attribution. Full text: `LICENSES/CC-BY-4.0.txt`. This is the **IP repository's** documentation license; this internal Reference Manual carries its own, separately stated, [CC BY 4.0 header](#) above.

37.2.5. The reference decoder is fetched, not vendored

No decoder binary is part of this repository. Earlier revisions of this manual described a committed `bin/NexRv` built from an Accemic fork of the IAR Systems NexRv tool (ISC); that fork is superseded by **CTTD (CEDARtools.TraceDecoder)** and the binaries left the tree on 2026-08-17. `bin/` is gitignored and holds whatever `scripts/fetch_cttd.py` places there against the pin in `scripts/cttd.pin` ([Section 29.4](#)). CTTD is a separate Accemic product under its own licence terms, not a component of the CTTE delivery, and `REUSE.toml` therefore carries no entry for it. The NexRv lineage remains relevant as **provenance** – CTTD descends from the RISC-V Nexus Trace Task Group reference code, copyright IAR Systems AB and Accemic Technologies GmbH, ISC-licensed – but nothing in this repository conveys it.

37.2.6. Commercial licensing

Organizations that cannot comply with the copyleft obligations of CERN-OHL-S-2.0 for the hardware IP may obtain a commercial license from Accemic Technologies GmbH. Inquiries: `sales@accemic.com`. That license covers the CTTE IP—everything in `rtl/`, `rdl/`, `tests/`, and the Accemic-authored parts of `examples/`. It does **not** cover the vendored MINRES TGC5B core, which carries its own dual license from MINRES (see above).

37.2.7. Evaluation

Per `LICENSE.md`, evaluation, simulation, and internal development use of the hardware IP requires no license beyond choosing either the open (CERN-OHL-S-2.0) or the commercial arm—there is no additional gate for internal-only assessment stated in the current license documents.

37.3. Contributions and the Contributor License Agreement (CLA)

To preserve the dual-license option for the hardware IP, contributions require acceptance of a Contributor License Agreement (CLA) assigning the necessary rights to Accemic Technologies GmbH, while the contributor retains copyright over their own work. Contact `info@accemic.com` to arrange the CLA before a contribution can be merged (`CONTRIBUTING.md`).

The CLA is not a standalone document in this repository; it is arranged by contacting Accemic directly. `CONTRIBUTING.md`'s quick checklist for a pull request also requires: a branch named `<handle>/<short-description>` off main; every new file carrying its SPDX header; make `lint` and make `sim` passing; regenerated RDL output committed alongside any `rdl/ct_cs_cpuif.rdl` change; commit messages that explain **why**; and CI green (lint, make `sim` and make `sim-examples` under Verilator, REUSE) plus at least one maintainer review before merge.

Contributors who do not sign the CLA are not thereby excluded from using the project: the hardware IP's open arm, CERN-OHL-S-2.0, applies regardless of CLA status to anyone using, modifying, or redistributing it on that license's terms (strong copyleft applies to redistributed modifications either way). What the CLA specifically enables is **upstream** inclusion under the maintainers' dual-license offering.

37.4. Mechanisms described by earlier revisions

Revisions up to 1.3 of this manual described five licensing mechanisms that no longer exist: a CLA-gated feature switch (`LICENSED_FEATURES`) that would have enabled part of the functionality only for signatories, a "community branch" for contributions without a signed CLA, a separate Evaluation License Agreement, a name- and logo-use restriction tied to CLA status, and a dedicated attribution obligation for third-party implementations of ACT-CAP and ACT-ST.

All five belong to an earlier plan under which the encoder would have been published as **two** repository states — an open one and a commercial one with additional functionality. That plan was dropped in favour of releasing the full encoder and differentiating through the dual license instead. With one repository and one feature set, a mechanism that gates functionality by license status has nothing left to gate, and the constructs built around it became unnecessary along with it.

They are named here rather than silently removed, because a reader who knows the earlier manual will look for them. The terms that apply are the ones in [Section 37.2](#): the same code for everyone, under whichever arm of the dual license the user chooses.

37.5. ACT-CAP / ACT-ST attribution

The Accemic CEDARtools CSR Access Protocol (ACT-CAP) and CEDARtools Smart Trigger (ACT-ST) mechanisms are implemented in the hardware IP (`rtl`) and are covered by the same dual license as the rest of the hardware IP — see [Section 37.2](#). No additional attribution obligation beyond that license is currently sourced for third-party implementations of these mechanisms.

37.6. Trademarks and naming

CEDARtools is a registered trademark of Accemic Technologies GmbH. Its use in this project's name (CEDARtools.TraceEncoder) and documentation identifies the project's origin; it does not grant any trademark rights beyond what the licenses above state.

CTTE implements and references several open specifications and interoperates with third-party tools. Third-party marks appear **only descriptively**, to identify those specifications, interfaces, and tools (nominative use). Their use does not imply any affiliation with, sponsorship by, or endorsement by the trademark owners.

37.6.1. Attribution

- CEDARtools and Accemic are trademarks of Accemic Technologies GmbH.
- RISC-V, RISC-V International, and the RISC-V logos are trademarks of RISC-V International.
- MicroBlaze™, Vivado™, Xilinx™, and Kria™ are trademarks of Advanced Micro Devices, Inc.
- Arm®, AMBA®, AXI®, ATB®, and CoreSight™ are trademarks or registered trademarks of Arm Limited (or its affiliates).
- Nexus and IEEE-ISTO 5001 are marks of the IEEE Industry Standards and Technology Organization (IEEE-ISTO) / Nexus 5001 Forum.
- IAR and IAR Systems are trademarks of IAR Systems AB.
- All other product names, logos, and brands are the property of their respective owners and are used for identification only; such use does not imply affiliation, sponsorship, or endorsement.

37.6.2. Use of the RISC-V mark

Accemic Technologies GmbH is **not** a member of RISC-V International. The mark therefore appears only in its descriptive sense – naming the architecture and the specifications this encoder targets, as in "trace encoder for RISC-V cores" or "implements the RISC-V N-Trace specification."

This repository, and this Reference Manual, must not:

- use the RISC-V logo;
- use "RISC-V" as part of the product, module, or domain name – the product is CEDARtools.TraceEncoder, and RISC-V only describes what it targets;
- claim membership, partnership, certification, approval, or endorsement;
- claim conformance or compatibility certified by any official RISC-V programme. Deviations from the specifications are listed openly (see the Standards baseline and Conformance chapters, Part A), and the vendor TCODEs 56/57/58 are Accemic extensions that imply no claim of standard conformance.

"RISC-V" is written in block letters with a hyphen; the only admissible spellings are **RISC-V** and **RISC-V International**.

37.7. Summary of obligations

Scenario	License needed	Key obligations
Use, modify, or redistribute the hardware IP under the open arm	CERN-OHL-S-2.0	Strong copyleft on redistributed modifications; provide Complete Source when conveying a Product built from it (§4).
Contribute changes intended for upstream inclusion	CLA (in addition to using either license arm)	Grants Accemic the rights needed to keep offering the dual-license model; contact info@accemic.com . Contributions without a CLA remain usable under CERN-OHL-S-2.0 on that license's own terms, but are not merged upstream.
Integrate the encoder in a closed-source product with your own core	Accemic Commercial License	No copyleft disclosure obligation; contact sales@accemic.com .
Integrate the encoder together with the vendored MINRES TGC5B core in a closed-source product	Accemic Commercial License and a MINRES Commercial License, negotiated separately	Neither commercial license alone covers the other party's IP.
Evaluate, simulate, or develop internally without redistribution	either license arm	No additional license needed beyond choosing CERN-OHL-S-2.0 or the commercial arm.
Use, adapt, or redistribute this documentation	CC-BY-4.0	Attribution, link to the license, indicate changes.